

Gemini USage

Gemini

<https://claude.ai/chat/a95a9f31-dff6-4c1d-9769-f237d5af971d>

[https://claude.ai/chat/199707df-6120-4ac3-9bcd-37859c72e29b\](https://claude.ai/chat/199707df-6120-4ac3-9bcd-37859c72e29b)

All the best faseeh!!

Prompt Final

Project Information Header

- **Project:** [Project Name]
 - **Company:** [Company Name]
 - **Role:** [Your Title]
 - **Duration:** [Start Date - End Date]
-

Summary STAR (1.5-2.5 minutes speaking time)

Write a conversational, narrative overview (2-3 sentences) covering the main situation, task, action, and result in paragraph form. Write as if you're speaking/telling the story briefly to someone.

Example: "I was tasked with automating our manual budget process that was taking 60+ hours per month. I built a Streamlit web portal that let staff upload Excel files and automatically generated Word documents, which reduced processing time by 60% and eliminated calculation errors entirely."

Detailed STAR

S - SITUATION

Situation Summary: *Brief narrative of the context and problem (2-3 sentences)*

Detailed Context:

- Company context & business problem
- Team structure & your role
- Timeline & constraints
- Scale/scope of the challenge

T - TASK/CHALLENGE

Task Summary: *Brief narrative of what you were responsible for (2-3 sentences)*

Detailed Responsibilities:

- Your specific responsibilities

- Technical challenges faced
- Business/stakeholder challenges
- Success criteria and constraints

A - ACTION (*Detailed Breakdown*)

Action Summary: *Brief narrative of how you approached the work (2-3 sentences)*

Technical Stack Used:

- Languages, frameworks, tools, platforms
- Why you chose each technology
- Architecture decisions and trade-offs

Collaboration Details:

- Who you worked with (roles, seniority)
- How decisions were made
- Communication challenges/successes
- Cross-functional coordination

Leadership Moments:

- Decisions you drove
- People you influenced/mentored
- Processes you improved
- Initiative you took beyond assigned tasks

Problem-Solving Process:

- Approach to technical challenges
- Alternative solutions considered
- Trade-offs made and reasoning
- How you validated your approach

R - RESULTS

Result Summary: *Brief narrative of the outcomes and impact (2-3 sentences)*

Detailed Impact:

- **Quantified business impact:** Metrics, cost savings, efficiency gains
- **Technical achievements:** Performance improvements, system capabilities
- **Team/process improvements:** Workflow enhancements, knowledge transfer
- **Personal growth/skills gained:** New competencies, lessons learned

Writing Guidelines:

- **Summaries:** Conversational, first person ("I built...", "I was tasked with...")
 - **Detailed sections:** Can use bullet points and technical language
 - **Tone:** Natural, like explaining to someone in person
 - **Technical details:** Keep in detailed sections, not summaries
 - **Quantify everything:** Include specific numbers, percentages, timelines
 - **Show progression:** Demonstrate growth and increasing responsibility
-

Usage Instructions:

1. Fill out the Project Information Header
2. Write the Summary STAR as a brief narrative
3. Complete each detailed section with both summary and bullets
4. Ensure summaries sound conversational and natural
5. Keep all technical details in the detailed breakdowns
6. Review for consistency between summary and detailed versions

This template gives you both a quick elevator pitch version (Summary STAR) and a comprehensive deep-dive version (Detailed STAR) for different interview situations.

Prompt 2

STAR Format Enhancement Prompt Template

Please take my STAR format internship/project description and enhance it by adding summary sections while keeping all original content intact. Here's exactly what I need:

Structure Required:

1. Project Title

1. Summary STAR (New Addition)

- * Write a conversational, narrative overview (2-3 sentences)
- * Cover the main situation, task, action, and result in paragraph form
- * Write as if I'm speaking/telling the story briefly

2. Detailed STAR (Enhanced Version)

- * Keep ALL original content, structure, and sub-headings exactly the same
- * Add a "Summary" subsection under each main section (S-T-A-R):
 - * Situation Summary: Brief narrative of the context and problem
 - * Task Summary: Brief narrative of what I was responsible for
 - * Action Summary: Brief narrative of how I approached the work
 - * Result Summary: Brief narrative of the outcomes and impact

Writing Style:

- * Keep summaries conversational and easy to understand
- * Write in first person ("I built...", "I was tasked with...")
- * Make it sound natural, like I'm explaining it to someone
- * Keep all technical details in the original bullet points
- * Don't change any existing content, just add the summary sections

Final Format:

Project Title: [Project Title]

Summary STAR

[conversational overview paragraph]

Detailed STAR

[Original content with added summary subsections under S-T-A-R]

Please enhance my STAR description following this exact format.

Tab 4

Interview Prep Assistant Prompt

Interview Prep Assistant Prompt

You are an expert interview coach specializing in helping candidates effectively communicate their technical projects. Your goal is to improve my communication skills through structured practice and detailed feedback.

Your Role & Approach

Primary Function: Help me practice explaining my professional projects clearly, confidently, and professionally through mock interview questions.

Process Flow:

1. **Ask Question** - Pose relevant technical or behavioral questions based on my project details
2. **Wait for My Response** - Let me answer completely
3. **Provide Model Answer** - Show an ideal response with proper structure and language
4. **Evaluate & Compare** - Analyze my response against your model answer
5. **Give Specific Feedback** - Provide actionable improvement suggestions

Question Types to Ask

Technical Questions (40%):

- Architecture decisions and trade-offs
- Technology choices and alternatives
- Performance optimization strategies
- Integration challenges and solutions
- Scalability and security considerations

Behavioral Questions (40%):

- Leadership and decision-making moments
- Problem-solving under pressure
- Team collaboration and communication
- Handling obstacles and setbacks
- Learning and adaptation experiences

Hybrid Questions (20%):

- "Walk me through how you solved [specific technical challenge]"
- "Tell me about a time you had to explain technical concepts to non-technical stakeholders"

Model Answer Requirements

Your model answers must include:

Structure:

- Clear STAR format for behavioral questions
- Logical flow: Context → Challenge → Solution → Results
- 2-3 minute speaking time (150-250 words)

Language Elements:

- Confident, ownership-oriented phrasing ("I architected", "I implemented", "I drove")
- Quantified impact with specific metrics
- Professional technical terminology
- Action-oriented verbs

Content Depth:

- Appropriate technical detail for the audience
- Business impact and value demonstration
- Problem-solving methodology
- Learning outcomes or growth

Evaluation Criteria

Rate my responses (1-10) on:

Clarity (25%): Can a non-technical interviewer understand?

- Simple explanations of complex concepts
- Logical flow and structure
- Avoiding excessive jargon

Technical Accuracy (25%): Correct details and depth

- Accurate technology usage
- Appropriate level of detail
- Sound technical reasoning

Impact Communication (25%): Business value demonstration

- Quantified results and metrics
- Clear problem-to-solution narrative
- Stakeholder benefit explanation

Professionalism (25%): Delivery and confidence

- Confident language and ownership
- Concise, focused responses
- Professional tone and pacing

Feedback Format

After each question, provide:

Model Answer: [Your ideal 150-250 word response]

Your Response Analysis:

- **Strengths:** What you did well
- **Gaps:** What was missing vs. model answer
- **Language Improvements:** Better phrasing suggestions
- **Structure Issues:** Flow and organization feedback

Specific Action Items:

- 2-3 concrete improvements for next time
- Key phrases to incorporate
- Common patterns to avoid

Context for Questions

Base your questions on this project information I'll provide. When I share a new project or role details, adapt your questions accordingly. Focus on areas where I can demonstrate:

- Technical leadership and decision-making
- Problem-solving under constraints
- Communication with diverse stakeholders
- Innovation and continuous improvement
- Results-driven engineering

Instructions

Start by asking me to share my project details, then begin with one behavioral question about leadership or problem-solving, followed by your model answer and evaluation process. Keep questions varied and progressively challenging.

Ready to begin my interview prep session?

Understand this once you understood say yes then when I says tart youc an start asking the uquestions

TMAY

I'm Faseeh who's passionate about building data systems that solve real business problems. Throughout my journey, I've focused on projects that create measurable impact - like automating budget processes at RIT that saved \$50,000 annually, and developing a federated system as part of my capstone.

What drives me is seeing how good engineering directly improves people's work lives. At Daiichi Sankyo, I built data pipelines processing terabytes of clinical data while maintaining zero data loss - that experience showed me how critical reliable data systems are in regulated environments.

I'm drawn to DataBahn because you're solving the exact challenge I'm passionate about - helping enterprises get value from their data without drowning in complexity.

I bring a combination of hands-on experience with data processing systems, a track record of delivering cost-saving automation, and genuine enthusiasm for engineering scalable systems. I'm ready to contribute to your engineering team and help build the future of AI-powered data platforms.

Tab 43

Summ Projects Professional

FedDisco Mobile Federated Learning Application

Project: FedDisco Mobile Federated Learning Application

Company: Academic Capstone Project

Role: Lead Developer/Researcher

Duration: 5 Weeks

- **Situation:** Traditional machine learning for mobile apps required centralizing sensitive user data (images, text), creating major privacy risks and communication bottlenecks.
- **Task:** Implement the FedDisco federated learning algorithm from scratch to enable distributed, privacy-preserving ML on mobile devices, delivering a working prototype in 5 weeks for academic review.
- **Action:**
 - **Tech Stack:** Built a complete system using **React Native** (mobile client), **Python FastAPI** (server), and **TensorFlow Lite** (on-device training).
 - **Implementation:** Implemented a custom FedDisco algorithm with optimized parameters (K=5 clients, E=3 epochs, B=16 batch size), starting with a modular architecture for image processing.
- **Result:**
 - Delivered a working prototype in 5 weeks, passing academic validation.
 - Achieved a **70% reduction in communication overhead** and **60% memory optimization**.
 - Demonstrated 40% faster convergence, achieving stability in **15 rounds** versus the theoretical 25.

Budget Justification Data Transfer Automation

Project: Budget Justification Data Transfer Automation

Company: Rochester Institute of Technology (Office of VP of Research)

Role: Machine Learning Engineer Co-op

Duration: June 2025 - Present

- **Situation:** RIT's Pre-Award team manually transferred budget data from 800+ annual Excel proposals into Word templates, causing 5-8 calculation errors monthly and severe bottlenecks.
- **Task:** Design and build a comprehensive automation solution to eliminate manual data transfer, ensure 100% calculation accuracy, and provide a user-friendly interface for non-technical staff.
- **Action:**
 - **Tech Stack:** Built a **Streamlit** web portal using **Python**.

- **Core Logic:** Implemented robust Excel parsing with **pandas/openpyxl** and automated Word document generation using **python-docx**.
- **Collaboration:** Worked closely with the Pre-Award Director and stakeholders across 5 colleges to handle diverse Excel formats.
- **Result:**
 - Achieved a **60% reduction in manual processing time**.
 - **Eliminated calculation errors** (down from 5-8 monthly to zero).
 - Saved an estimated **\$50,000 annually** in administrative costs and successfully automated all 800+ annual proposals.

Conversational AI System for Clinical Trial Data Quality Analysis

Project: Conversational AI System for Clinical Trial Data Quality Analysis

Company: Daiichi Sankyo

Role: R&D Data Governance Intern

Duration: Ongoing

- **Situation:** Daiichi Sankyo wanted to replace manual Tableau-based data quality analysis with a conversational AI, but in a highly regulated environment with severe technical constraints: no cloud access, no GPU, and only 32GB RAM systems.
 - **Task:** Build a proof-of-concept (POC) conversational AI that could query complex, relational clinical trial data locally while maintaining regulatory compliance.
 - **Action:**
 - Developed three iterative versions, progressing from basic tabular models (TAPAS) to a sophisticated **RAG architecture** using **Mistral 7B**.
 - Deployed the model locally using **Ollama** to function on CPU-only hardware.
 - Created custom clinical datasets and a modular system with "Fast," "Balanced," and "Quality" configurations to manage performance trade-offs.
 - **Result:**
 - Successfully delivered a POC demonstrating conversational AI feasibility on CPU-only hardware, with 4-9 minute response times.
 - Established a **regulatory-compliant AI development framework** for local deployment.
 - Identified a clear production pathway using a **SQL Translation Layer** (with Phi-3-mini) that would reduce response times to seconds once cloud approval is granted.
-

Automated ICF Classification System for Clinical Trial Compliance

Project: Automated ICF Classification System for Clinical Trial Compliance

Company: Daiichi Sankyo

Role: R&D Data Governance Intern

Duration: 10 Months (June 2024 - April 2025)

- **Situation:** Legal and regulatory teams spent 4-5 minutes manually reviewing each Informed Consent Form (ICF) for data sharing clauses, creating critical bottlenecks across hundreds of documents.
- **Task:** Build a complete automated classification and document management system to dramatically reduce review time, with zero tolerance for misclassification errors.
- **Action:**
 - Developed the system over 10 months through four iterative versions, evolving from rule-based logic to a sophisticated **two-stage transformer architecture**.
 - The final system used **FAISS embeddings** (all-MiniLM-L6-v2) for fast retrieval and a fine-tuned **DistilBERT** model for accurate classification.
 - Built a comprehensive document management platform (Flask + PDF.js) with filtering (study, country, version), version control, and batch processing.
- **Result:**
 - Achieved a **0.93 F1 score** and **0.96 accuracy**, meeting regulatory-grade standards.
 - **Reduced processing time by 87%**, from 4-5 minutes manually to just 33 seconds.
 - Delivered a production-ready system that eliminated critical bottlenecks for multi-study, multi-country trial operations.

CTMS Database Migration System (Alteryx to Python)

Project: CTMS Database Migration System - Alteryx to Python Migration

Company: Daiichi Sankyo

Role: R&D Data Governance Intern

Duration: 2 Weeks

- **Situation:** The R&D department's Alteryx-based system for migrating clinical trial (CTMS) data created licensing bottlenecks, had limited customization, and dependency issues.
- **Task:** In just 2 weeks, completely migrate the entire data migration system from Alteryx to a custom Python solution, preserving all functionality while adding enhancements.

- **Action:**
 - Reverse-engineered all existing Alteryx workflows.
 - Rewrote the entire system in Python, using **SQLAlchemy** for database connections and **pandas** for data processing.
 - Implemented parallel processing using `concurrent.futures.ThreadPoolExecutor`, a capability the original Alteryx system lacked.
- **Result:**
 - Delivered a production-ready Python system in 2 weeks, **eliminating Alteryx licensing dependencies**.
 - Achieved **zero data loss** during migration and improved performance via parallel processing.
 - Provided the team with enhanced error handling and monitoring capabilities.

AI-Driven Content Automation & SEO Optimization System

Project: AI-Driven Content Automation & SEO Optimization System

Company: White Label Resell

Role: Machine Learning Engineer (Sole Backend/AI/Cloud Developer)

Duration: Ongoing

- **Situation:** White Label Resell's clients needed large-scale, multi-language, SEO-optimized content, but manual creation was too slow and expensive to scale for e-commerce needs.
 - **Task:** As the sole backend/AI/cloud developer, single-handedly build a comprehensive AI-powered content automation system from scratch to generate over 50,000 articles monthly.
 - **Action:**
 - **AI Models:** Integrated **GPT-3.5**, **BERT**, and **FLAN-T5** for text; **Stable Diffusion** for images; and **Google Translate API** for 50+ languages.
 - **Infrastructure:** Architected a complete **AWS** cloud system (API Gateway, ECS, Lambda, S3, SageMaker).
 - **Core Feature:** Developed a custom graph-based AI algorithm for automated internal SEO linking.
 - **Result:**
 - As a solo developer, built an enterprise-scale system generating 10,000+ articles monthly, securing **\$50-100K in quarterly contracts**.
 - Achieved a **60% increase in organic traffic** for clients via the AI-powered internal linking.
 - Boosted international engagement by **40%** with multi-language support.
-

Sky Scraper - AI-Powered Chrome Extension

Project: Sky Scraper - AI-Powered Chrome Extension for Scalable Content Automation

Company: SEOContent.ai

Role: AI Infrastructure Engineer

Duration: 6 Months

- **Situation:** SEOContent.ai clients spent hours on manual keyword research and article structuring, preventing them from building large-scale SEO content campaigns.
- **Task:** Build an AI-powered Chrome extension to automate the entire content workflow—from keyword research to generating clusters of up to 15 interconnected articles.
- **Action:**
 - Designed a **microservices architecture on AWS** (ECS, Lambda).
 - Used fine-tuned **BERT** models for keyword extraction and topic clustering.
 - Integrated **GPT models (GPT-3.5, GPT-NeoX)** for content generation.
 - The system combined web scraping (via the extension) with parallel processing NLP workflows.
- **Result:**
 - **Slashed content creation time by 60-70%** (from 4 hours down to 10 minutes + editing).
 - Successfully launched the beta, leading to contract renewals and measurable organic traffic improvements for clients using the automated content clusters.

Zero-Maintenance AI Tools Platform

Project: Zero-Maintenance AI Tools Platform with Dynamic Configuration System

Company: SEOContent.ai

Role: AI Infrastructure Engineer

Duration: 3 Weeks

- **Situation:** SEOContent.ai wanted a platform of 50+ AI tools, but knew that requiring developers to maintain each tool would create massive operational bottlenecks.
- **Task:** In 3 weeks, build a "zero-maintenance" platform where **non-technical content experts** could manage, update, and deploy tools themselves.
- **Action:**
 - Architected an innovative hybrid system: used **Google Sheets** as a simple CMS for non-technical staff.
 - An **Apps Script** trigger in Google Sheets called an **AWS Lambda** function, which processed the sheet and saved a new JSON configuration file to **Amazon S3**.

- The JavaScript frontend dynamically rendered the tools at runtime based on the JSON config file, which was served via **CloudFront CDN**.
- **Result:**
 - Launched in 3 weeks, gaining **5,000 users in the first month**.
 - **Cut platform operational costs by 98.4%** (\$50/month vs. \$2000+/month for no-code tools).
 - Tool updates now take **under 90 seconds** (done by content experts) instead of waiting days for a developer deployment.

AI-Powered Content Generation API

Project: AI-Powered Content Generation API for SEO Marketing

Company: SEOContent.ai

Role: AI Infrastructure Engineer

Duration: 3 Weeks

- **Situation:** SEOContent.ai needed to commercialize its existing internal AI content generation system by building a scalable, market-ready API for external clients.
- **Task:** In 3 weeks, architect a complete API system using AWS API Gateway to securely expose the existing AI models (GPT-3.5, BERT, Stable Diffusion) and handle thousands of concurrent requests.
- **Action:**
 - Built a comprehensive REST API using **AWS API Gateway** with **Lambda** functions as the integration layer to the existing AI pipeline.
 - Implemented **AWS Cognito** for secure API key management and integrated **Stripe** for usage-based billing.
 - Optimized performance using asynchronous processing patterns and API Gateway caching.
- **Result:**
 - Delivered a production-ready API in 3 weeks.
 - **Reduced content generation time by 85%** (from 2.5 minutes to 45 seconds) through optimization.
 - The system serves 10,000+ monthly requests with 99.9% uptime, enabling 90% successful client integrations.

Medical Image Data Engineering Pipeline

Project: Medical Image Data Engineering Pipeline for Pathology Workflows

Company: Digital Clinics Research and Services

Role: Data Scientist

Duration: 12 Months

- **Situation:** Pathologists at Digital Clinics faced major bottlenecks, unable to efficiently analyze massive 10-20GB medical imaging files (TIFF/SVS) for cancer diagnosis.
- **Task:** Engineer a complete, automated data processing pipeline to break down these massive files, create efficient annotation workflows, and integrate with the CADD4MBC clinical platform.
- **Action:**
 - Built a robust **Python** pipeline using **OpenCV** and **NumPy** for automated file processing (tiling, scaling, and format conversion).
 - Integrated **QuPath** (an annotation tool) with custom **Groovy scripts** to create a seamless annotation workflow for junior pathologists.
 - Used **MongoDB** for metadata tracking and **Flask/FastAPI** for integration APIs.
- **Result:**
 - The pipeline processed **1.5TB of data weekly** with **zero data loss**.
 - **Reduced manual processing time by 60%** and enabled the processing of 100,000+ images monthly.
 - This pipeline became the foundation for all downstream AI model development.

Tubule Detection using Distributed Deep Learning

Project: Tubule Detection for Breast Cancer Grading

Company: Digital Clinics Research and Services

Role: Data Scientist

Duration: 3 Months (within 12-month timeline)

- **Situation:** To automate the Nottingham Grading System for breast cancer, the clinic needed to automate tubule detection, a process that was slow and had high inter-observer variability among pathologists.
- **Task:** Develop a deep learning segmentation model with medical-grade accuracy (>90%) and optimize its training efficiency.
- **Action:**
 - Fine-tuned an **R2U-Net** (a Recurrent Residual U-Net) architecture, which is specialized for medical image segmentation.
 - Implemented **Horovod distributed training** to run the workload across **4 GPUs** (NVIDIA DGX), using the annotated dataset from my data pipeline (Project 10).
- **Result:**

- Achieved a **92% F1-score**, exceeding the clinical accuracy goal.
- **Reduced training time by 3.9x** (from 27 hours down to 7 hours).
- Presented these findings at the **SIIM-CMIMI22 International Conference** at Johns Hopkins Hospital.

Mitosis Detection using Multi-Architecture Deep Learning

Project: Mitosis Detection for Breast Cancer Grading

Company: Digital Clinics Research and Services

Role: Data Scientist

Duration: 4 Months (within 12-month timeline)

- **Situation:** The second critical component for the Nottingham Grading System was automating mitosis (cell division) detection, which was also extremely time-consuming and prone to disagreement between experts.
- **Task:** Implement and compare two different object detection architectures (two-stage vs. single-stage) to find the best model for this task, using the 56,258 annotated images from my pipeline.
- **Action:**
 - Implemented a two-stage detector, **Faster R-CNN**, using the **Detectron2** framework.
 - Implemented a single-stage detector, **YOLOv5**, and hypothesized it would offer a better speed-precision balance.
 - Conducted a systematic performance comparison using the massive annotated dataset.
- **Result:**
 - **YOLOv5 outperformed Faster R-CNN**, achieving an **84% F1 score** (a 7% improvement).
 - This comparative analysis was published in the **International Journal of Advanced Computer Science and Applications (IJACSA)**, establishing a new benchmark.

Software Portable Radio Simulator

Project: Software Portable Radio Simulator

Company: Edgeforce Solutions

Role: Data Scientist Intern

Duration: 3 Months

- **Situation:** The Indian Army needed a cost-effective software-based training simulator to replace expensive physical radio hardware.
- **Task:** As the sole developer, build a complete real-time voice communication system from scratch to connect 3-4 laptops with sub-500ms latency, while managing frequent client requirement changes.
- **Action:**
 - Built the entire system in **Python**. Used **WebRTC** for real-time communication protocols and **PyAudio** for audio processing.
 - Made a critical technical decision to switch the GUI from Streamlit to **Tkinter** to meet the client's mandatory **offline deployment** requirement.
 - Created a Software Requirement Specification (SRS) to freeze requirements and prevent scope creep.
- **Result:**
 - Delivered a successful system achieving **sub-200ms latency** (an 85% improvement from the initial 1.5s).
 - **Reduced hardware dependency by 80%** and helped Edgeforce Solutions win additional defense contracts.

AI-Powered Wine Inventory Management System (POC)

Project: AI-Powered Wine Inventory Management System (POC)

Company: Edgeforce Solutions

Role: Data Scientist Intern

Duration: 3-4 Weeks

- **Situation:** Edgeforce Solutions needed to prove to army clients that AI could solve their manual wine inventory problems, which were full of counting errors and inefficient tracking.
- **Task:** In just 3-4 weeks, build a complete proof-of-concept (POC) from scratch to validate that computer vision could accurately detect wine bottles and justify funding for a full-scale project.
- **Action:**
 - **Model:** Implemented **YOLOv5** for real-time object detection.
 - **Data:** Rapidly created an annotated dataset using **Roboflow**.
 - **Demo:** Developed a **Streamlit** web dashboard with live video streaming (via WebRTC) to present the solution live to stakeholders.
- **Result:**
 - Delivered the POC in 3 weeks, achieving **87% detection accuracy**.
 - Validated a potential **30% time reduction** in manual counting.
 - The demo successfully convinced army stakeholders, who **approved funding for the full-scale implementation**.

Here is a concise yet detailed version of your projects, synthesizing the key information from your summaries and detailed breakdowns.

FedDisco Mobile Federated Learning Application

Project: FedDisco Mobile Federated Learning Application

Company: Academic Capstone Project

Role: Lead Developer/Researcher

Duration: 5 Weeks

- **Situation:** Traditional machine learning for mobile apps required centralizing sensitive user data (images, text), creating major privacy risks and communication bottlenecks.
- **Task:** Implement the FedDisco federated learning algorithm from scratch to enable distributed, privacy-preserving ML on mobile devices, delivering a working prototype in 5 weeks for academic review.
- **Action:**
 - **Tech Stack:** Built a complete system using **React Native** (mobile client), **Python FastAPI** (server), and **TensorFlow Lite** (on-device training).
 - **Implementation:** Implemented a custom FedDisco algorithm with optimized parameters (K=5 clients, E=3 epochs, B=16 batch size), starting with a modular architecture for image processing.
- **Result:**
 - Delivered a working prototype in 5 weeks, passing academic validation.
 - Achieved a **70% reduction in communication overhead** and **60% memory optimization**.
 - Demonstrated 40% faster convergence, achieving stability in **15 rounds** versus the theoretical 25.

Budget Justification Data Transfer Automation

Project: Budget Justification Data Transfer Automation

Company: Rochester Institute of Technology (Office of VP of Research)

Role: Machine Learning Engineer Co-op

Duration: June 2025 - Present

- **Situation:** RIT's Pre-Award team manually transferred budget data from 800+ annual Excel proposals into Word templates, causing 5-8 calculation errors monthly and severe bottlenecks.

- **Task:** Design and build a comprehensive automation solution to eliminate manual data transfer, ensure 100% calculation accuracy, and provide a user-friendly interface for non-technical staff.
 - **Action:**
 - **Tech Stack:** Built a **Streamlit** web portal using **Python**.
 - **Core Logic:** Implemented robust Excel parsing with **pandas/openpyxl** and automated Word document generation using **python-docx**.
 - **Collaboration:** Worked closely with the Pre-Award Director and stakeholders across 5 colleges to handle diverse Excel formats.
 - **Result:**
 - Achieved a **60% reduction in manual processing time**.
 - **Eliminated calculation errors** (down from 5-8 monthly to zero).
 - Saved an estimated **\$50,000 annually** in administrative costs and successfully automated all 800+ annual proposals.
-

Conversational AI System for Clinical Trial Data Quality Analysis

Project: Conversational AI System for Clinical Trial Data Quality Analysis

Company: Daiichi Sankyo

Role: R&D Data Governance Intern

Duration: Ongoing

- **Situation:** Daiichi Sankyo wanted to replace manual Tableau-based data quality analysis with a conversational AI, but in a highly regulated environment with severe technical constraints: no cloud access, no GPU, and only 32GB RAM systems.
- **Task:** Build a proof-of-concept (POC) conversational AI that could query complex, relational clinical trial data locally while maintaining regulatory compliance.
- **Action:**
 - Developed three iterative versions, progressing from basic tabular models (TAPAS) to a sophisticated **RAG architecture** using **Mistral 7B**.
 - Deployed the model locally using **Ollama** to function on CPU-only hardware.
 - Created custom clinical datasets and a modular system with "Fast," "Balanced," and "Quality" configurations to manage performance trade-offs.
- **Result:**
 - Successfully delivered a POC demonstrating conversational AI feasibility on CPU-only hardware, with 4-9 minute response times.
 - Established a **regulatory-compliant AI development framework** for local deployment.
 - Identified a clear production pathway using a **SQL Translation Layer** (with Phi-3-mini) that would reduce response times to seconds once cloud approval is granted.

Automated ICF Classification System for Clinical Trial Compliance

Project: Automated ICF Classification System for Clinical Trial Compliance

Company: Daiichi Sankyo

Role: R&D Data Governance Intern

Duration: 10 Months (June 2024 - April 2025)

- **Situation:** Legal and regulatory teams spent 4-5 minutes manually reviewing each Informed Consent Form (ICF) for data sharing clauses, creating critical bottlenecks across hundreds of documents.
- **Task:** Build a complete automated classification and document management system to dramatically reduce review time, with zero tolerance for misclassification errors.
- **Action:**
 - Developed the system over 10 months through four iterative versions, evolving from rule-based logic to a sophisticated **two-stage transformer architecture**.
 - The final system used **FAISS embeddings** (all-MiniLM-L6-v2) for fast retrieval and a fine-tuned **DistilBERT** model for accurate classification.
 - Built a comprehensive document management platform (Flask + PDF.js) with filtering (study, country, version), version control, and batch processing.
- **Result:**
 - Achieved a **0.93 F1 score** and **0.96 accuracy**, meeting regulatory-grade standards.
 - **Reduced processing time by 87%**, from 4-5 minutes manually to just 33 seconds.
 - Delivered a production-ready system that eliminated critical bottlenecks for multi-study, multi-country trial operations.

CTMS Database Migration System (Alteryx to Python)

Project: CTMS Database Migration System - Alteryx to Python Migration

Company: Daiichi Sankyo

Role: R&D Data Governance Intern

Duration: 2 Weeks

- **Situation:** The R&D department's Alteryx-based system for migrating clinical trial (CTMS) data created licensing bottlenecks, had limited customization, and dependency issues.

- **Task:** In just 2 weeks, completely migrate the entire data migration system from Alteryx to a custom Python solution, preserving all functionality while adding enhancements.
- **Action:**
 - Reverse-engineered all existing Alteryx workflows.
 - Rewrote the entire system in Python, using **SQLAlchemy** for database connections and **pandas** for data processing.
 - Implemented parallel processing using `concurrent.futures.ThreadPoolExecutor`, a capability the original Alteryx system lacked.
- **Result:**
 - Delivered a production-ready Python system in 2 weeks, **eliminating Alteryx licensing dependencies**.
 - Achieved **zero data loss** during migration and improved performance via parallel processing.
 - Provided the team with enhanced error handling and monitoring capabilities.

AI-Driven Content Automation & SEO Optimization System

Project: AI-Driven Content Automation & SEO Optimization System

Company: White Label Resell

Role: Machine Learning Engineer (Sole Backend/AI/Cloud Developer)

Duration: Ongoing

- **Situation:** White Label Resell's clients needed large-scale, multi-language, SEO-optimized content, but manual creation was too slow and expensive to scale for e-commerce needs.
- **Task:** As the sole backend/AI/cloud developer, single-handedly build a comprehensive AI-powered content automation system from scratch to generate over 50,000 articles monthly.
- **Action:**
 - **AI Models:** Integrated **GPT-3.5**, **BERT**, and **FLAN-T5** for text; **Stable Diffusion** for images; and **Google Translate API** for 50+ languages.
 - **Infrastructure:** Architected a complete **AWS** cloud system (API Gateway, ECS, Lambda, S3, SageMaker).
 - **Core Feature:** Developed a custom graph-based AI algorithm for automated internal SEO linking.
- **Result:**
 - As a solo developer, built an enterprise-scale system generating 10,000+ articles monthly, securing **\$50-100K in quarterly contracts**.
 - Achieved a **60% increase in organic traffic** for clients via the AI-powered internal linking.
 - Boosted international engagement by **40%** with multi-language support.

Sky Scraper - AI-Powered Chrome Extension

Project: Sky Scraper - AI-Powered Chrome Extension for Scalable Content Automation

Company: SEOContent.ai

Role: AI Infrastructure Engineer

Duration: 6 Months

- **Situation:** SEOContent.ai clients spent hours on manual keyword research and article structuring, preventing them from building large-scale SEO content campaigns.
- **Task:** Build an AI-powered Chrome extension to automate the entire content workflow—from keyword research to generating clusters of up to 15 interconnected articles.
- **Action:**
 - Designed a **microservices architecture on AWS** (ECS, Lambda).
 - Used fine-tuned **BERT** models for keyword extraction and topic clustering.
 - Integrated **GPT models (GPT-3.5, GPT-NeoX)** for content generation.
 - The system combined web scraping (via the extension) with parallel processing NLP workflows.
- **Result:**
 - **Slashed content creation time by 60-70%** (from 4 hours down to 10 minutes + editing).
 - Successfully launched the beta, leading to contract renewals and measurable organic traffic improvements for clients using the automated content clusters.

Zero-Maintenance AI Tools Platform

Project: Zero-Maintenance AI Tools Platform with Dynamic Configuration System

Company: SEOContent.ai

Role: AI Infrastructure Engineer

Duration: 3 Weeks

- **Situation:** SEOContent.ai wanted a platform of 50+ AI tools, but knew that requiring developers to maintain each tool would create massive operational bottlenecks.
- **Task:** In 3 weeks, build a "zero-maintenance" platform where **non-technical content experts** could manage, update, and deploy tools themselves.
- **Action:**
 - Architected an innovative hybrid system: used **Google Sheets** as a simple CMS for non-technical staff.

- An **Apps Script** trigger in Google Sheets called an **AWS Lambda** function, which processed the sheet and saved a new JSON configuration file to **Amazon S3**.
- The JavaScript frontend dynamically rendered the tools at runtime based on the JSON config file, which was served via **CloudFront CDN**.
- **Result:**
 - Launched in 3 weeks, gaining **5,000 users in the first month**.
 - **Cut platform operational costs by 98.4%** (\$50/month vs. \$2000+/month for no-code tools).
 - Tool updates now take **under 90 seconds** (done by content experts) instead of waiting days for a developer deployment.

AI-Powered Content Generation API

Project: AI-Powered Content Generation API for SEO Marketing

Company: SEOContent.ai

Role: AI Infrastructure Engineer

Duration: 3 Weeks

- **Situation:** SEOContent.ai needed to commercialize its existing internal AI content generation system by building a scalable, market-ready API for external clients.
 - **Task:** In 3 weeks, architect a complete API system using AWS API Gateway to securely expose the existing AI models (GPT-3.5, BERT, Stable Diffusion) and handle thousands of concurrent requests.
 - **Action:**
 - Built a comprehensive REST API using **AWS API Gateway** with **Lambda** functions as the integration layer to the existing AI pipeline.
 - Implemented **AWS Cognito** for secure API key management and integrated **Stripe** for usage-based billing.
 - Optimized performance using asynchronous processing patterns and API Gateway caching.
 - **Result:**
 - Delivered a production-ready API in 3 weeks.
 - **Reduced content generation time by 85%** (from 2.5 minutes to 45 seconds) through optimization.
 - The system serves 10,000+ monthly requests with 99.9% uptime, enabling 90% successful client integrations.
-

Medical Image Data Engineering Pipeline

Project: Medical Image Data Engineering Pipeline for Pathology Workflows

Company: Digital Clinics Research and Services

Role: Data Scientist

Duration: 12 Months

- **Situation:** Pathologists at Digital Clinics faced major bottlenecks, unable to efficiently analyze massive 10-20GB medical imaging files (TIFF/SVS) for cancer diagnosis.
- **Task:** Engineer a complete, automated data processing pipeline to break down these massive files, create efficient annotation workflows, and integrate with the CADD4MBC clinical platform.
- **Action:**
 - Built a robust **Python** pipeline using **OpenCV** and **NumPy** for automated file processing (tiling, scaling, and format conversion).
 - Integrated **QuPath** (an annotation tool) with custom **Groovy scripts** to create a seamless annotation workflow for junior pathologists.
 - Used **MongoDB** for metadata tracking and **Flask/FastAPI** for integration APIs.
- **Result:**
 - The pipeline processed **1.5TB of data weekly** with **zero data loss**.
 - **Reduced manual processing time by 60%** and enabled the processing of 100,000+ images monthly.
 - This pipeline became the foundation for all downstream AI model development.

Tubule Detection using Distributed Deep Learning

Project: Tubule Detection for Breast Cancer Grading

Company: Digital Clinics Research and Services

Role: Data Scientist

Duration: 3 Months (within 12-month timeline)

- **Situation:** To automate the Nottingham Grading System for breast cancer, the clinic needed to automate tubule detection, a process that was slow and had high inter-observer variability among pathologists.
- **Task:** Develop a deep learning segmentation model with medical-grade accuracy (>90%) and optimize its training efficiency.
- **Action:**
 - Fine-tuned an **R2U-Net** (a Recurrent Residual U-Net) architecture, which is specialized for medical image segmentation.

- Implemented **Horovod distributed training** to run the workload across **4 GPUs** (NVIDIA DGX), using the annotated dataset from my data pipeline (Project 10).
- **Result:**
 - Achieved a **92% F1-score**, exceeding the clinical accuracy goal.
 - **Reduced training time by 3.9x** (from 27 hours down to 7 hours).
 - Presented these findings at the **SIIM-CMIMI22 International Conference** at Johns Hopkins Hospital.

Mitosis Detection using Multi-Architecture Deep Learning

Project: Mitosis Detection for Breast Cancer Grading

Company: Digital Clinics Research and Services

Role: Data Scientist

Duration: 4 Months (within 12-month timeline)

- **Situation:** The second critical component for the Nottingham Grading System was automating mitosis (cell division) detection, which was also extremely time-consuming and prone to disagreement between experts.
- **Task:** Implement and compare two different object detection architectures (two-stage vs. single-stage) to find the best model for this task, using the 56,258 annotated images from my pipeline.
- **Action:**
 - Implemented a two-stage detector, **Faster R-CNN**, using the **Detectron2** framework.
 - Implemented a single-stage detector, **YOLOv5**, and hypothesized it would offer a better speed-precision balance.
 - Conducted a systematic performance comparison using the massive annotated dataset.
- **Result:**
 - **YOLOv5 outperformed Faster R-CNN**, achieving an **84% F1 score** (a 7% improvement).
 - This comparative analysis was published in the **International Journal of Advanced Computer Science and Applications (IJACSA)**, establishing a new benchmark.

Software Portable Radio Simulator

Project: Software Portable Radio Simulator

Company: Edgeforce Solutions

Role: Data Scientist Intern

Duration: 3 Months

- **Situation:** The Indian Army needed a cost-effective software-based training simulator to replace expensive physical radio hardware.
- **Task:** As the sole developer, build a complete real-time voice communication system from scratch to connect 3-4 laptops with sub-500ms latency, while managing frequent client requirement changes.
- **Action:**
 - Built the entire system in **Python**. Used **WebRTC** for real-time communication protocols and **PyAudio** for audio processing.
 - Made a critical technical decision to switch the GUI from Streamlit to **Tkinter** to meet the client's mandatory **offline deployment** requirement.
 - Created a Software Requirement Specification (SRS) to freeze requirements and prevent scope creep.
- **Result:**
 - Delivered a successful system achieving **sub-200ms latency** (an 85% improvement from the initial 1.5s).
 - **Reduced hardware dependency by 80%** and helped Edgeforce Solutions win additional defense contracts.

AI-Powered Wine Inventory Management System (POC)

Project: AI-Powered Wine Inventory Management System (POC)

Company: Edgeforce Solutions

Role: Data Scientist Intern

Duration: 3-4 Weeks

- **Situation:** Edgeforce Solutions needed to prove to army clients that AI could solve their manual wine inventory problems, which were full of counting errors and inefficient tracking.
- **Task:** In just 3-4 weeks, build a complete proof-of-concept (POC) from scratch to validate that computer vision could accurately detect wine bottles and justify funding for a full-scale project.
- **Action:**
 - **Model:** Implemented **YOLOv5** for real-time object detection.
 - **Data:** Rapidly created an annotated dataset using **Roboflow**.
 - **Demo:** Developed a **Streamlit** web dashboard with live video streaming (via WebRTC) to present the solution live to stakeholders.
- **Result:**
 - Delivered the POC in 3 weeks, achieving **87% detection accuracy**.

- Validated a potential **30% time reduction** in manual counting.
- The demo successfully convinced army stakeholders, who **approved funding for the full-scale implementation**.

Proejcts

Azure Customer Insights Data Pipeline and KPI Dashboard

Client Project Description

A retail client hired me to modernize their customer analytics workflow by migrating data from their on-premises SQL database into Azure and building a complete analytics solution. The goal was to understand customer demographics, especially gender distribution by product category, and deliver a real-time KPI dashboard. I designed and implemented an end-to-end Azure data pipeline using Azure Data Factory, ADLS (Bronze–Silver–Gold), Azure Databricks, and Azure Synapse Analytics, with Power BI as the visualization layer. The solution runs on a daily schedule and provides stakeholders with continuously updated and query-ready data.

Resume-Ready Bullet Points

Designed and delivered an end-to-end Azure-based data pipeline for a retail analytics client, enabling daily ingestion of on-premises SQL data into Azure Data Lake Storage for centralized analysis.

Implemented a Bronze–Silver–Gold lakehouse architecture in ADLS and transformed raw demographic and sales data using Azure Databricks for enhanced data quality and query performance.

Built downstream relational models in Azure Synapse Analytics and integrated the curated Gold data into Power BI to produce customer gender distribution KPIs and product-category insights.

Developed automated workflows in Azure Data Factory to extract, load, and schedule daily transformations, ensuring up-to-date and accurate data availability for stakeholders.

Enforced enterprise-grade security through Azure Active Directory and Key Vault, ensuring controlled access, secret management, and compliance with data governance standards.

Delivered a user-friendly dashboard enabling drill-down analysis by gender, product category, time period, and total sales, improving business decision-making capabilities.

FedMob(TensorFlow, React Native, Federated Learning, PyTorch)

•

Built React Native app implementing FedAvg algorithm enabling AI training across mobile devices achieving privacy improvement

NLP

Prompt-Manager Library (Python pkg|Version Control|YAML Metadata|PyPI|Unit Tests)

[Link]

- Designed Python library for prompt management across LLM applications with version control and metadata tagging.
- Created hierarchical prompt storage system, reducing prompt iteration cycles and development time by 33%.

NutriPedia: Food Recognition & Calorie Estimation (TensorFlow|Inception-V3|GCP Storage|GPT-3.5 API|SQL)

[Link]

- Architected cloud-based nutrition platform integrating Inception-V3 CNN with GCP Storage for image processing|implementing GPT-3.5 API for personalized dietary insights from SQL nutritional database.
- Engineered full-stack data pipeline connecting computer vision classifications with NLP-powered contextual responses|creating flywheel architecture for continuous model improvement through user feedback.

COVID-19 Bot (Dialogflow|Flask|MongoDB|RapidAPI|Webhooks)

[\[Link\]](#)

- Architected full-stack Telegram conversational chatbot with DialogFlow NLP for intent recognition|Flask/Python backend|and MongoDB persistence|enabling real-time pandemic statistics retrieval via RapidAPI.
- Implemented context-aware entity extraction pipeline supporting multilingual medical assistance|resource localization|and vaccination scheduling with custom webhook fulfillment handlers.

ExpenseGenie Bot (Telegram|WhatsApp|Gemini LLM|Redis|Langfuse|Voice/Image Processing, GCP)

[\[Link\]](#)

- Architected modular microservices system for intelligent expense splitting across Telegram and WhatsApp platforms simultaneously.
- Implemented Redis-based state management and Langfuse monitoring achieving 99.9% reliability and comprehensive LLM observability.

Data Scientist/Data Analyst/Data Engineer

Chronic Kidney Disease Predictor (MERN Stack|Random Forest|Flask|MongoDB|Python|XGBoost)

[\[Link\]](#)

- Engineered full stack MERN application for early CKD detection utilizing Random Forest classifier achieving 99.75% diagnostic accuracy through intelligent data completion algorithm for handling missing values in medical datasets.
- Developed intuitive clinical decision support interface with secure MongoDB persistence, enabling healthcare professionals to input patient biomarkers for immediate risk assessment and early intervention.

KafkaSpark – User Data Processing Engine (Apache Kafka|Apache Spark|Airflow|Cassandra|Docker)

[\[Link\]](#)

- Engineered end-to-end data pipeline connecting API data sources to Apache Cassandra through Kafka queues|implementing distributed Spark processing and real-time data streaming.
- Developed containerized architecture with Docker Compose|integrating Apache Airflow for workflow orchestration and implementing comprehensive monitoring through Kafka Control Center.

Telco Customer Churn Pipeline (Python|MLflow|XGBoost|GitHub Actions|AWS ECR/EC2)

[\[Link\]](#)

- Designed an end-to-end MLOps pipeline with modular architecture|implementing automated CI/CD workflows that reduced development time by 30%.
- Engineered scalable ETL processes and predictive analytics system using MLflow experiment tracking|achieving 15% improvement in F1-score for churn prediction.
- Developed and deployed containerized prediction API on AWS EC2/ECR through GitHub Actions|enabling robust model serving with automated testing and versioning.

Airline Passenger Forecasting (SARIMA|Statsmodels|Pandas|Matplotlib|A/B Testing) [\[Link\]](#)

- Applied ARIMA and SARIMA models on airline passenger data (1949-1960)|performing A/B testing that demonstrated SARIMA's superior fit for seasonal passenger patterns.
- Implemented time series decomposition and stationarity testing (ADF|KPSS)|visualizing forecast accuracy with 95% confidence intervals to enable reliable capacity planning.

Flight Fare Predictor (Random Forest|ExtraTrees|Flask|Scikit-learn|Feature Engineering) [\[Link\]](#)

- Developed ML system predicting flight prices with 80% accuracy using Random Forest Regression.
- Engineered feature extraction pipeline identifying key price determinants (total stops|journey day|airline carrier) through ExtraTreesRegressor analysis across multiple parameters.

Twitter Metrics Dashboard (Tableau|SQL|Window Functions|Interactive Filters|KPI Design) [\[Link\]](#)

- Created interactive Tableau dashboard visualizing Twitter metrics|providing insights into impression rates and engagement patterns across different time periods.
- Designed a comprehensive performance tracking system with filter-based interactivity|enabling social media managers to identify optimal posting schedules and content strategies.

Google Play Store Apps Dashboard (Tableau|Tree Maps|Parameter Controls|Data Blending|SQL) [\[Link\]](#)

- Developed a comprehensive Tableau dashboard analyzing app performance metrics across categories|revealing correlations between ratings|installs|and content ratings.
- Created interactive visualization system with tree maps and filter controls|enabling product managers to identify competitive market gaps and optimize app development strategies.

FDI Sector Explorer (Tableau|Timeseries LODs|Dual-Axis Charts|CSV ETL|Calculated Fields) [\[Link\]](#)

- Created interactive Tableau dashboard visualizing Foreign Direct Investment trends across Indian sectors|revealing highest and lowest investment patterns over time.
- Designed an intuitive visual analytics system with comparative metrics and filter-based exploration|enabling stakeholders to identify strategic investment opportunities across economic sectors.

World Bank Project Success Classifier (XGBoost|Hyperopt|Imbalanced-learn|Python|SHAP) [\[Link\]](#)

- Developed predictive model achieving 71.7% F1 score to identify World Bank project success factors|implementing XGBoost classification with hyperparameter optimization.
- Engineered comprehensive data processing pipeline for 17,500+ hierarchical records|addressing 30%+ missing data and class imbalance to deliver actionable insights for resource allocation.

Analysis on Various Commodities (LSTM|TensorFlow|Web Scraping|BeautifulSoup|Plotly) [\[Link\]](#)

- Engineered comprehensive commodities analysis system with custom web scraper|extracting 10 years of daily price data to identify strong cross-market correlations.
- Implemented LSTM neural networks for time series forecasting across six major commodities|achieving low MSE scores (0.27-0.40) for accurate price movement predictions.

Car Price Predictor (Random Forest|EDA|Flask|Bootstrap|Pandas) [\[Link\]](#)

- Developed machine learning application predicting used car prices with Random Forest Regression|comparing multiple algorithms to identify optimal prediction accuracy.
- Implemented comprehensive data pipeline with feature engineering and EDA|creating an intuitive Flask-based web interface for real-time price estimation.

Telco Customer Churn Project [\[Link\]](#)

- Designed an end-to-end MLOps pipeline with modular architecture|implementing automated CI/CD workflows that reduced development time by 30%.
- Engineered scalable ETL processes and predictive analytics system using MLflow experiment tracking|achieving 15% improvement in F1-score for churn prediction.
- Developed and deployed containerized prediction API on AWS EC2/ECR through GitHub Actions|enabling robust model serving with automated testing and versioning.

Bollywood Movie Recommender System (Python|BeautifulSoup|Cosine Similarity|TF-IDF|Streamlit) [\[Link\]](#)

- Engineered comprehensive ETL pipeline extracting data from 3,100+ Bollywood movies using Python and BeautifulSoup|implementing statistical analysis workflows for recommendation algorithms.
- Developed automated data processing system with 11 normalized features|creating structured datasets that enabled content-based filtering and similarity analysis for personalized movie recommendations.

Bioacoustic Data Analytics (TensorFlow|STFT|CNN|Spectrogram Augment|Data Pipeline) [\[Link\]](#)

- Engineered end-to-end audio processing pipeline transforming raw forest recordings into spectrogram datasets using TensorFlow|achieving 100% precision and recall for Capuchin bird call detection.
- Implemented signal processing algorithms with Short-Time Fourier Transform for feature extraction|optimizing CNN architecture to distinguish target patterns from environmental noise.

Sports Predictive Analytics (Scikit-learn|GridSearchCV|Feature Engineering|Flask|RMSE Metric) [\[Link\]](#)

- Developed regression models predicting IPL cricket scores with RMSE of 20 runs|implementing feature engineering techniques and hyperparameter optimization through GridSearchCV to analyze complex sports metrics.
- Engineered end-to-end ML solution from data preprocessing to deployment|creating Flask web application with interactive interface for real-time cricket score predictions based on match conditions.

Automated License Recognition (TensorFlow OD API|MobileNet-SSD|EasyOCR|OpenCV|ROI Filtering) [\[Link\]](#)

- Implemented real-time computer vision system using TensorFlow Object Detection API and transfer learning with SSD MobileNet v2|creating an end-to-end pipeline for license plate detection.
- Engineered post-processing algorithms and OCR integration using EasyOCR|optimizing detection accuracy through ROI filtering and confidence thresholds for automated data extraction.

Anime Data Synthesis (PyTorch|DC-GAN|Custom Loss|Data Augment|Hyperparam Tuning) [\[Link\]](#)

- Implemented Deep Convolutional Generative Adversarial Network (DC-GAN) using PyTorch|engineering custom data collection pipelines and preprocessing workflows for large-scale anime image datasets.
- Developed specialized training architecture with modified loss functions|generating high-quality synthetic anime character images while maintaining style consistency and feature diversity.

Banking Investment Portfolio Dynamics (Tableau|Statistical Tests|SQL|Time-Series LODs|Regulatory Analysis) [\[Link\]](#)

- Analyzed regulatory impact on banking portfolios using statistical analysis and Tableau|revealing 4-fold HTM ratio increases in large banks following capital requirement changes.
- Created financial data visualizations tracking investment trend divergence between banking segments|providing actionable insights for regulatory compliance strategies.

Medical Appointment and Patient Record System (SQL|Data Modeling|Power BI|ETL|Validation Rules) [\[Link\]](#)

- Engineered SQL queries and database reporting system capturing patient-doctor relationships across 6 interconnected healthcare entities with complex data validation rules.
- Developed analytics dashboards reducing report generation time by 40%|enabling data-driven resource allocation decisions for healthcare administrators.

ACHIEVEMENTS

-
- Kaggle Notebook and Dataset Expert. || Teaching Assistant for Intro to Machine Learning (CSCI-739)
 - Open Source Contributor at OpenVino (Intel)|Ivy || 3x Hackathon Winner

RESEARCH PAPERS

-
- **Tubules Detection on Breast Carcinoma Whole-Slide Images** – C-MIMI (Johns Hopkins)|Sep 2022.
 - **Web-Based Mitosis Detection on Breast Cancer Images via Faster RCNN & YOLOv5** – IJACSA|Dec 2022.

RESEARCH PAPERS

- **Tubules Detection on Breast Carcinoma Whole Slide Images Using Artificial Intelligence** - Presented at C-MIMI Conference|Johns Hopkins University
- **Web-Based Mitosis Detection on Breast Cancer Whole Slide Images Using Faster R-CNN and YOLOv5** - International Journal of Advanced Computer Science Applications

Professional

Project Information Header

- **Project:** FedDisco Mobile Federated Learning Application
- **Company:** Academic Capstone Project
- **Role:** Lead Developer/Researcher
- **Duration:** 5 Weeks (First Prototype Completed)

Summary STAR (1.5-2.5 minutes speaking time)

S: For my academic capstone project, I saw that mobile apps handling sensitive user data like images and text faced a major problem - traditional machine learning required centralizing data, which created privacy risks and communication bottlenecks that users weren't comfortable with.

T: I needed to implement the FedDisco federated learning algorithm from scratch to enable distributed machine learning across mobile devices without compromising user privacy, delivering a working prototype in just 5 weeks for academic review.

A: I built a complete system using React Native for the mobile client, Python FastAPI for the server, and TensorFlow Lite for on-device training. I implemented custom FedDisco with optimized parameters and created a modular architecture starting with image processing while planning for text functionality.

R: Delivered a working prototype in 5 weeks that achieved 70% reduction in communication overhead, 60% memory optimization for resource-constrained devices, and demonstrated convergence in 15 rounds versus the theoretical 25 rounds - successfully passing academic validation while proving federated learning could work practically on mobile devices.

Detailed STAR

S - SITUATION Situation Summary: For my academic capstone project, I identified the critical need for privacy-preserving machine learning in mobile applications where traditional centralized approaches create privacy risks and communication bottlenecks, particularly problematic for personal content like images and text.

Detailed Context:

- **Company context & business problem:** As part of my capstone project, I identified the critical need for privacy-preserving machine learning in mobile applications where users' sensitive data (images, text) cannot be centralized due to privacy concerns. Traditional ML approaches require data centralization, creating privacy risks and communication bottlenecks, particularly problematic for mobile applications handling personal content.
- **Team structure & your role:** Independent capstone project with faculty advisor guidance. As Lead Developer/Researcher, I was responsible for the complete system architecture, implementation of both client and server components, and delivery of a working prototype demonstrating federated learning capabilities on mobile devices.
- **Timeline & constraints:** Aggressive 8-week timeline for first prototype delivery with significant system limitations. Need to balance proof-of-concept development with production-ready architecture decisions. Academic environment constraints including limited computational resources and requirement for comprehensive documentation.
- **Scale/scope of the challenge:** Complete federated learning system from scratch, mobile optimization, privacy preservation, academic-quality prototype, 8-week delivery

T - TASK/CHALLENGE Task Summary: I was tasked with building a complete mobile application implementing the FedDisco (Federated Learning) algorithm that enables privacy-preserving machine learning across distributed mobile devices, starting with image processing capabilities as the foundational component.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Design and implement FedDisco algorithm for mobile federated learning environment
 - Build client-side mobile application handling local training on user devices
 - Develop server-side aggregation system managing global model updates
 - Implement image processing pipeline as first prototype component
 - Create modular architecture supporting future text upload functionality
 - Ensure privacy-preserving design where raw data never leaves user devices
 - Deliver working prototype within 8-week academic deadline
- **Technical challenges faced:**
 - Implementing complex federated learning algorithm within mobile resource constraints
 - Balancing local computation efficiency with global model convergence quality
 - Managing communication optimization between multiple mobile clients and central server
 - Building modular system architecture supporting both image and text processing workflows
- **Business/stakeholder challenges:**
 - Delivering academic-quality prototype within tight 8-week timeline
 - Balancing comprehensive system design with practical implementation constraints
 - Managing scope effectively by prioritizing image processing before text functionality
 - Ensuring prototype demonstrates clear federated learning value proposition

- **Success criteria and constraints:** 8-week delivery, working federated learning, privacy preservation, mobile optimization, academic validation, modular architecture

A - ACTION (Detailed Breakdown) Action Summary: I successfully architected and implemented a comprehensive federated learning system using modular design principles, deployed the FedDisco algorithm with optimized communication protocols, and strategically executed phased development delivering a working image processing prototype while establishing infrastructure for text processing expansion.

Technical Stack Used:

- **Mobile Client:** React Native (cross-platform deployment enabling iOS/Android testing)
- **Server Architecture:** Python + FastAPI (lightweight server achieving efficient model aggregation)
- **Federated Learning:** Custom FedDisco implementation with optimized parameters (K=5 test clients, C=0.6 fraction, E=3 epochs, B=16 batch size)
- **Image Processing:** TensorFlow Lite (on-device training achieving 40% memory reduction)
- **Communication:** RESTful APIs with model compression (achieved 70% communication overhead reduction)
- **Security:** Encrypted model updates with SHA-256 hashing ensuring privacy-preserving aggregation

Collaboration Details:

- **Worked with:** Faculty advisor (algorithm validation), 3 peer testers (prototype validation), academic review committee
- **How decisions were made:** Made independent technical architecture decisions with weekly advisor consultations on federated learning convergence optimization
- **Communication challenges/successes:** Delivered comprehensive documentation and live demonstration showcasing federated learning capabilities
- **Cross-functional coordination:** Balanced academic research requirements with practical implementation constraints and peer validation

Leadership Moments:

- **Decisions you drove:** Successfully implemented phased development approach prioritizing image processing, enabling prototype delivery within 8-week constraint
- **People you influenced/mentored:** Technical innovation building modular architecture with plugin-based design enabling seamless future expansion
- **Processes you improved:** Established systematic testing protocol validating federated learning convergence across 5 simulated mobile clients
- **Initiative you took beyond assigned tasks:** Built comprehensive documentation framework enabling future researchers to build upon implementation

Problem-Solving Process:

- **Mobile Resource Constraints:** Solved memory limitations by implementing TensorFlow Lite with quantization reducing model size by 60%
- **Communication Efficiency:** Achieved 70% bandwidth reduction through gradient compression and selective model parameter updates
- **Convergence Optimization:** Tuned FedDisco parameters ($E=3$, $C=0.6$) achieving stable convergence in 15 rounds vs. theoretical 25 rounds
- **Alternative Solutions Considered:** Evaluated Flutter vs React Native (chose React Native for faster development), PyTorch vs TensorFlow (chose TensorFlow Lite for mobile optimization)

R - RESULTS Result Summary: Successfully delivered a working federated learning prototype demonstrating privacy-preserving machine learning on mobile devices, achieving significant performance improvements in communication efficiency and model convergence while establishing a robust foundation for comprehensive multi-modal federated learning applications.

Detailed Impact:

- **Quantified business impact:**
 - **8-week prototype delivery** successfully completed, demonstrating federated learning feasibility on mobile platforms
 - **70% reduction** in communication overhead through model compression and selective parameter updates
 - **60% memory optimization** enabling deployment on resource-constrained mobile devices
 - **Academic validation** with successful demonstration to review committee and peer testing completion
- **Technical achievements:**
 - **Working FedDisco implementation** achieving convergence in 15 rounds (40% improvement over theoretical baseline)
 - **Cross-platform mobile application** successfully tested on both iOS and Android with consistent performance
 - **Privacy-preserving architecture** validated through security audit showing zero raw data transmission
 - **Modular plugin architecture** built enabling text processing integration with 90% code reuse
- **Team/process improvements:**
 - **Rapid prototyping methodology** proved effective for delivering complex federated learning system within academic constraints
 - **Comprehensive test suite** established covering federated learning convergence, mobile performance, and privacy validation
 - **Documentation framework** created enabling future researchers to build upon federated learning mobile implementation
 - **Peer validation process** implemented with 3 independent testers confirming prototype functionality

- **Personal growth/skills gained:**

- **Federated learning expertise:** Practical implementation experience with FedDisco algorithm optimization for mobile environments
- **Mobile AI development:** Advanced skills in TensorFlow Lite optimization and on-device machine learning deployment
- **System architecture mastery:** Proven ability to design modular, extensible systems under tight timeline constraints
- **Academic research execution:** Successfully delivered research-quality prototype meeting academic standards while demonstrating practical applicability

Key Prototype Achievements:

- **5 concurrent clients** successfully participating in federated learning rounds
- **87% image classification accuracy** maintained while preserving complete data privacy
- **45 seconds average training time** per client per round on standard mobile devices
- **Successful model convergence** demonstrated across heterogeneous mobile device configurations

Project Information Header

- **Project:** Budget Justification Data Transfer Automation
- **Company:** Rochester Institute of Technology (Office of Vice President of Research)
- **Role:** Machine Learning Engineer Co-op
- **Duration:** June 2025 - Present

Summary STAR (1.5-2.5 minutes speaking time)

S: RIT's Pre-Award team was drowning in manual work - they had to process 800+ annual federal research proposals by manually transferring budget data from Excel files into Word templates, which was creating bottlenecks during peak periods and causing 5-8 calculation errors monthly that could jeopardize proposal compliance.

T: I needed to design and build a comprehensive automation solution that could eliminate all the manual data transfer, ensure 100% calculation accuracy, and provide a user-friendly interface for non-technical staff while maintaining strict federal compliance requirements.

A: I built a Streamlit web portal with robust Excel parsing using pandas/openpyxl and automated Word document generation with python-docx. I worked closely with the Pre-Award Director and stakeholders across 5 colleges to handle diverse Excel formats and ensure seamless integration with existing workflows.

R: Achieved a 60% reduction in manual processing time, eliminated calculation errors entirely (down from 5-8 monthly), saved \$50,000 annually in administrative costs, and successfully automated all 800+ annual proposals while maintaining federal compliance - allowing staff to focus on strategic review instead of data entry.

Detailed STAR

S - SITUATION Situation Summary: Rochester Institute of Technology's Pre-Award team was struggling with managing 800+ annual federal research proposals requiring manual data entry from Excel to Word templates, creating bottlenecks during peak submission periods and causing calculation errors that could jeopardize proposal compliance.

Detailed Context:

- **Company context & business problem:** RIT's Pre-Award team managed 800+ annual federal research proposals requiring detailed budget justifications and corresponding narratives. Each proposal demanded manual data entry from Excel into Word document templates, creating bottlenecks during peak submission periods. The team of 2 administrators and 4 faculty researchers struggled with year-end budget transfers, often working overtime to meet federal deadlines. Manual processes led to formatting inconsistencies and calculation errors that could jeopardize proposal compliance.
- **Team structure & your role:** Office of Vice President of Research team including Pre-Award Director, 2 senior administrators, 4 faculty researchers, and budget officers from 5 different colleges. As Machine Learning Engineer Co-op, I was responsible for designing and implementing the complete automation solution to eliminate manual data transfer processes.
- **Timeline & constraints:** Ongoing project from June 2025 requiring immediate impact during peak submission periods. Critical requirement for 100% accuracy in financial calculations and federal compliance. Need for user-friendly interface for non-technical staff members.
- **Scale/scope of the challenge:** 800+ annual proposals, diverse Excel formats, federal compliance requirements, non-technical user base, peak period bottlenecks

T - TASK/CHALLENGE Task Summary: I was tasked with designing and implementing a comprehensive automation solution that could eliminate manual budget data transfer processes, ensure 100% calculation accuracy, and provide a user-friendly interface for non-technical staff while maintaining federal proposal compliance.

Detailed Responsibilities:

- **Your specific responsibilities:**

- Design an automated solution to eliminate manual budget data transfer
- Ensure 100% accuracy in financial calculations and formatting
- Create user-friendly interface for non-technical staff
- Maintain compliance with federal proposal requirements
- Build robust system handling diverse Excel formats and data validation
- Implement automated Word document generation with consistent formatting

- **Technical challenges faced:**

- Building intuitive Streamlit interface for non-technical users
- Handling diverse Excel formats and data validation
- Automating Word document generation with consistent formatting
- Real-time file processing and error handling in web environment
- Ensuring data security during file upload/download processes

- **Business/stakeholder challenges:**

- Minimizing disruption to existing workflows during implementation
- Training staff on new automation tools
- Ensuring backup procedures for system failures during critical deadlines
- Managing change management across multiple colleges and departments

- **Success criteria and constraints:** 100% calculation accuracy, 60%+ time reduction, federal compliance, user-friendly design, 800+ proposal capacity

A - ACTION (Detailed Breakdown) Action Summary: I built a comprehensive Streamlit web portal with robust Excel parsing and automated Word generation capabilities, working closely with stakeholders across multiple colleges to ensure seamless integration with existing workflows while achieving complete automation of the budget transfer process.

Technical Stack Used:

- **Python:** Core automation logic and data processing (chosen for robust data manipulation and document generation capabilities)
- **Streamlit:** Interactive web portal interface (selected for rapid development and user-friendly design for non-technical staff)
- **pandas/openpyxl:** Excel file reading and data manipulation (comprehensive support for diverse Excel formats and calculations)
- **python-docx:** Automated Word document generation (precise formatting control and template integration)
- **Word Mail Merge templates:** Consistent formatting (ensuring federal compliance and professional presentation)
- **File upload/download handlers:** Seamless user experience (secure temporary storage and automatic cleanup)

Collaboration Details:

- **Worked with:** Pre-Award Director (requirements), 2 senior administrators (workflow validation), faculty researchers (budget requirements), budget officers from 5 colleges (diverse use cases)
- **How decisions were made:** Led requirements gathering sessions with 15+ end users to understand diverse budget requirements and workflow constraints
- **Communication challenges/successes:** Regular stakeholder meetings ensuring broad adoption across different colleges and departments
- **Cross-functional coordination:** Partnered with IT Security team for system integration approval while balancing automation needs with security requirements

Leadership Moments:

- **Decisions you drove:** Led requirements gathering sessions with 15+ end users and drove decision to adopt modular architecture for future scalability
- **People you influenced/mentored:** Mentored 1 undergraduate intern on automation scripting techniques and established testing protocols
- **Processes you improved:** Established comprehensive testing protocols involving real proposal data validation and standardized budget formatting
- **Initiative you took beyond assigned tasks:** Created documentation and training materials for knowledge transfer beyond basic system requirements

Problem-Solving Process:

- **Standardization Challenge:** Analyzed 200+ budget templates to design standardized Excel input format accommodating diverse requirements
- **User Experience:** Built Streamlit prototype with file upload/download functionality ensuring intuitive operation for non-technical users
- **Quality Assurance:** Implemented real-time data validation and error messaging in portal preventing calculation errors
- **Trade-off Analysis:** Chose web-based Streamlit portal for accessibility vs. desktop application for advanced features, prioritizing user adoption

R - RESULTS Result Summary: The automation system achieved remarkable efficiency improvements with 60% reduction in processing time, eliminated calculation errors entirely, saved \$50,000 annually in administrative costs, and successfully automated all 800+ annual proposals while maintaining federal compliance standards.

Detailed Impact:

- **Quantified business impact:**
 - **60% reduction** in manual processing time for budget preparation
 - **Zero calculation errors** since implementation (previously 5-8 errors monthly)
 - **\$50,000 annual cost savings** in administrative overtime elimination
 - **800+ proposals** now processed automatically per year with consistent quality

- **Technical achievements:**
 - **Built user-friendly Streamlit web portal** accessible to all staff members regardless of technical background
 - **Created robust Excel parsing system** handling multiple budget formats and diverse college requirements
 - **Developed automated Word document generation** with consistent styling meeting federal compliance standards
 - **Implemented comprehensive error handling** and user feedback systems preventing processing failures
- **Team/process improvements:**
 - **Strategic focus shift:** Staff can now focus on strategic proposal review vs. manual data entry tasks
 - **Standardized budget formatting** across all funding agencies ensuring consistency and compliance
 - **Created comprehensive documentation** and training materials enabling knowledge transfer and system maintenance
 - **Improved deadline management:** Eliminated overtime work during peak submission periods
- **Personal growth/skills gained:**
 - **Enterprise automation expertise:** Gained expertise in enterprise automation and change management for research environments
 - **Stakeholder communication mastery:** Developed skills in stakeholder communication and requirements gathering across diverse user groups
 - **Compliance understanding:** Enhanced understanding of federal research compliance requirements and proposal formatting standards
 - **User-centric design:** Experience building intuitive systems for non-technical users in high-stakes environments

Project Information Header

- **Project:** Conversational AI System for Clinical Trial Data Quality Analysis
- **Company:** Daiichi Sankyo
- **Role:** R&D Data Governance Intern
- **Duration:** [Duration not specified - ongoing project]

Summary STAR (1.5-2.5 minutes speaking time)

S: Daiichi Sankyo wanted to explore whether Large Language Models could replace their manual Tableau-based data quality analysis for clinical trial data, but I had to work in a highly regulated

environment where cloud services took months to approve, with only 32GB RAM systems, no GPU acceleration, and no specialized clinical datasets.

T: I needed to build a proof-of-concept conversational AI system that could handle complex relational clinical trial data queries while maintaining regulatory compliance, despite severe computational constraints and limited access to advanced AI infrastructure.

A: I developed three iterative versions, progressing from tabular models (TAPAS, multitab-qa-base) to sophisticated RAG architecture using Mistral 7B with local deployment via Ollama, creating custom clinical datasets and building a modular system with Fast, Balanced, and Quality configurations.

R: Successfully delivered a proof-of-concept with three distinct configurations achieving 4-9 minute response times,

I also established a regulatory-compliant AI development framework, and identified a clear production pathway using SQL Translation Layer that could reduce response times to seconds once cloud approvals come through.

Detailed STAR

S - SITUATION Situation Summary: Daiichi Sankyo's R&D team needed to transform their manual Tableau-based data quality analysis into a conversational AI interface for clinical trial data, while exploring LLM capabilities within a highly regulated environment that severely limited cloud access and computational resources.

Detailed Context:

- **Company context & business problem:** Daiichi Sankyo's R&D team needed to transform their Tableau-based data quality analysis into a conversational interface for clinical trial data. The existing manual analysis process was inefficient, and stakeholders wanted to explore whether Large Language Models could effectively interact with complex relational clinical trial data while maintaining regulatory compliance standards.
- **Team structure & your role:** R&D Data Governance team collaborating with database administrators and clinical data experts. As R&D Data Governance Intern, I was responsible for designing and implementing a proof-of-concept conversational AI system that could handle complex clinical trial data queries while working within strict regulatory constraints.
- **Timeline & constraints:** Highly regulated database environment where obtaining permissions for cloud services (AWS, Azure) required extensive approval processes lasting months. Significant technical constraints including 32GB RAM systems, no GPU acceleration, and absence of specialized training datasets for clinical domain. Limited to 100-200 rows for POC development due to regulatory restrictions.
- **Scale/scope of the challenge:** Proof-of-concept development, regulatory compliance requirements, severe computational limitations, complex relational clinical data

T - TASK/CHALLENGE Task Summary: I was tasked with building a proof-of-concept conversational AI system that could replace manual Tableau analysis workflows, requiring exploration of multiple technical approaches while navigating strict regulatory constraints that limited cloud access and computational resources.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Design and implement conversational interface for clinical trial data quality analysis
 - Explore multiple AI architectures including tabular models, RAG systems, and SQL translation layers
 - Build custom datasets and fine-tune models for clinical domain understanding
 - Navigate regulatory constraints preventing immediate cloud service access
 - Develop modular system architecture supporting different performance/quality trade-offs
 - Create comprehensive clinical trial glossary and domain-specific terminology framework
- **Technical challenges faced:**
 - Building effective AI system with 32GB RAM and no GPU acceleration in regulated environment
 - Handling complex relationships across relational clinical trial data with limited computational resources
 - Creating training datasets without access to specialized clinical domain models
 - Achieving reasonable response times (initial attempts taking 4-9 minutes per query)
- **Business/stakeholder challenges:**
 - Regulatory compliance requirements preventing immediate access to advanced cloud AI services
 - Balancing proof-of-concept development with production-ready performance expectations
 - Managing stakeholder expectations while working within severe computational and data constraints
 - Demonstrating AI value proposition despite significant technical limitations
- **Success criteria and constraints:** Regulatory compliance, proof-of-concept completion, multiple architecture evaluation, local deployment, clinical domain understanding

A - ACTION (Detailed Breakdown) Action Summary: I developed a comprehensive three-version iterative approach, progressing from tabular models to sophisticated RAG architecture while building regulatory-compliant solutions that demonstrated conversational AI potential despite significant constraints.

Technical Stack Used:

- **Version 1:** Tabular models (multitab-qa-base, TAPAS) + FAISS Vector Stores (specialized for structured data)

- **Version 2:** Modular RAG with Mistral 7B + multiple embedding options (all-MiniLM-L6-v2, all-mpnet-base-v2, e5-large)
- **Version 3 (Proposed):** SQL Translation Layer with Phi-3-mini (lightweight CPU-optimized deployment)
- **Custom Development:** ChatDSI for dataset creation, Ollama for local model deployment
- **Storage Solutions:** ChromaStore, HNSWStore, FAISSStore (metadata-driven vector databases)

Collaboration Details:

- **Worked with:** Database administrators (schema design), clinical data experts (domain knowledge), regulatory compliance officers (approval processes)
- **How decisions were made:** Led technical architecture decisions while coordinating with regulatory team for cloud service approval processes
- **Communication challenges/successes:** Developed comprehensive clinical trial glossary collaboratively, facilitating domain knowledge transfer between technical and clinical teams
- **Cross-functional coordination:** Balanced technical AI development with regulatory compliance requirements and clinical domain expertise

Leadership Moments:

- **Decisions you drove:** Shifted from model-centric to data-centric development strategy after Version 1 limitations
- **People you influenced/mentored:** Technical innovation designing modular RAG framework with three distinct configurations supporting different use cases
- **Processes you improved:** Created regulatory-compliant development workflow enabling AI experimentation within strict compliance constraints
- **Initiative you took beyond assigned tasks:** Developed comprehensive clinical trial glossary and domain-specific terminology framework

Problem-Solving Process:

- **Regulatory Constraints:** Built local deployment solutions using Ollama and CPU-optimized models while cloud approvals were pending
- **Performance Optimization:** Developed three-tier architecture (Fast: 4.4min, Balanced: 4.19min, Quality: 9.3min) balancing speed vs. accuracy
- **Data Limitations:** Created custom clinical trial datasets using ChatDSI and implemented comprehensive null value handling with domain-specific explanations
- **Alternative Solutions Considered:** Evaluated text-to-SQL pipelines, pretrained models with data injection, API-based solutions vs. local deployment

R - RESULTS Result Summary: Successfully delivered a comprehensive proof-of-concept demonstrating conversational AI potential for clinical data analysis, established regulatory-compliant development framework, and identified clear path forward with SQL Translation Layer approach achieving production-ready performance.

Detailed Impact:

- **Quantified business impact:**

- **Proof-of-concept completion** demonstrating feasibility of conversational AI for clinical trial data quality analysis
- **Three distinct architectures** validated with measurable performance characteristics (4-9 minute response times)
- **Regulatory compliance framework** established enabling future AI development within strict pharmaceutical constraints
- **Clear production pathway** identified through SQL Translation Layer approach reducing response times to seconds

- **Technical achievements:**

- **Modular RAG architecture:** Built flexible system with interchangeable components supporting experimentation
- **Custom clinical domain datasets:** Created specialized training data using ChatDSI for domain-specific fine-tuning
- **Multi-configuration system:** Developed Fast, Balanced, and Quality configurations optimizing different performance criteria
- **Local deployment expertise:** Successfully implemented CPU-only solutions working within regulatory constraints

- **Team/process improvements:**

- **Regulatory AI development process:** Established framework for compliant AI experimentation in pharmaceutical environment
- **Cross-functional collaboration:** Improved coordination between technical teams and regulatory compliance officers
- **Domain knowledge integration:** Created comprehensive clinical trial glossary facilitating technical-clinical communication
- **Iterative development methodology:** Demonstrated systematic approach to AI system development under constraints

- **Personal growth/skills gained:**

- **Regulatory AI development:** Deep understanding of pharmaceutical compliance requirements for AI system development
- **Constrained system design:** Expertise in building effective AI solutions with severe computational and regulatory limitations
- **Multi-architecture evaluation:** Advanced skills in systematic AI approach comparison and optimization
- **Healthcare domain expertise:** Comprehensive knowledge of clinical trial data structures and regulatory requirements

Project Information Header

- **Project:** Automated ICF Classification System for Clinical Trial Compliance
- **Company:** Daiichi Sankyo
- **Role:** R&D Data Governance Intern
- **Duration:** 10 Months (June 2024 - April 2025)

Summary STAR (1.5-2.5 minutes speaking time)

S: Daiichi Sankyo's legal and regulatory teams were stuck manually reviewing each Informed Consent Form for 4-5 minutes to identify data sharing clauses, and with hundreds of documents across multiple studies and countries, this was creating serious delays in clinical trial operations.

T: I needed to build a complete automated classification and document management system that could maintain regulatory compliance standards while dramatically reducing processing time - with zero tolerance for misclassification errors given the regulatory implications.

A: I developed the system through four iterative versions, evolving from rule-based systems to a sophisticated two-stage transformer architecture using DistilBERT and FAISS embeddings, while building a comprehensive document management platform with study filtering, version control, and geographic organization.

R: Achieved 0.93 F1 score with 0.96 accuracy, reduced processing time by 87% from 4-5 minutes to just 33 seconds, and created a production-ready system that eliminated critical bottlenecks across hundreds of documents for multiple studies and countries.

Detailed STAR

S - SITUATION Situation Summary: Daiichi Sankyo's legal and regulatory teams were facing critical bottlenecks with manual review processes that took 4-5 minutes per Informed Consent Form to identify data sharing clauses, creating delays across hundreds of documents and risking regulatory compliance issues.

Detailed Context:

- **Company context & business problem:** Daiichi Sankyo's legal and regulatory teams were spending 4-5 minutes manually reviewing each Informed Consent Form (ICF) to identify critical data sharing clauses. With hundreds of documents across multiple studies, countries, and consent form versions, this manual process created significant bottlenecks in clinical trial operations. Missing or misinterpreting these clauses could lead to serious regulatory compliance issues.
- **Team structure & your role:** R&D Data Governance team collaborating with legal reviewers, regulatory specialists, and clinical trial operations. As R&D Data Governance Intern, I was the

primary technical developer responsible for designing and implementing the complete automated classification system and document management platform.

- **Timeline & constraints:** 10-month project requiring iterative development through four major model versions. No pre-existing labeled datasets available. Critical requirement to maintain regulatory compliance standards while dramatically reducing processing time from manual 4-5 minutes to automated seconds.
- **Scale/scope of the challenge:** Hundreds of documents, multiple studies and countries, zero tolerance for misclassification, regulatory compliance requirements

T - TASK/CHALLENGE Task Summary: I was tasked with building a complete automated ICF classification and management system that could eliminate manual review bottlenecks, ensure regulatory compliance, and provide comprehensive document management capabilities across multiple studies and geographic regions.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Develop automated classification system progressing through four iterative model versions
 - Build comprehensive PDF database and management platform with study-based filtering
 - Create document comparison tools for version control across trial phases
 - Implement geographic filtering for country/region regulatory compliance
 - Design user-friendly interface enabling legal reviewers to edit classifications and provide feedback
 - Achieve production-ready performance reducing processing time from 4-5 minutes to under 33 seconds
- **Technical challenges faced:**
 - Severe class imbalance causing initial F1 scores of 0.58 across multiple model iterations
 - Building robust PDF parsing maintaining document structure across varied international formats
 - Implementing two-stage architecture balancing accuracy with processing speed requirements
 - Creating comprehensive document management system supporting complex clinical trial workflows
- **Business/stakeholder challenges:**
 - Eliminating critical bottlenecks in clinical trial operations affecting multiple departments
 - Ensuring zero tolerance for misclassification errors given regulatory compliance implications
 - Managing iterative feedback from legal reviewers while maintaining system usability
 - Supporting complex multi-study, multi-country document organization requirements
- **Success criteria and constraints:** 87%+ time reduction, 0.9+ F1 score, regulatory compliance, zero misclassification tolerance, 10-month iterative development

A - ACTION (Detailed Breakdown) Action Summary: I architected a sophisticated four-version iterative development process, progressing from rule-based systems to advanced two-stage transformer architecture, while building a comprehensive document management platform that achieved 0.93 F1 score and 87% processing time reduction.

Technical Stack Used:

- **Model Evolution:** Rule-based → TF-IDF ensemble → Transformer-based → Two-stage architecture (embedding retrieval + DistilBERT classification)
- **Embeddings & Search:** all-MiniLM-L6-v2 + FAISS IndexFlatIP (similarity threshold empirically set at 0.45)
- **Classification:** DistilBERT fine-tuned with 5-fold cross-validation and learning rate scheduling
- **Frontend Evolution:** Streamlit → Flask + PDF.js + responsive web interface
- **Document Management:** Comprehensive filtering (study, version, country, date), batch processing, version control
- **Optimization:** ONNX runtime conversion, batch processing for multi-document scenarios

Collaboration Details:

- **Worked with:** Legal reviewers (classification validation), regulatory specialists (compliance requirements), clinical operations (workflow integration)
- **How decisions were made:** Implemented active learning techniques prioritizing ambiguous cases for expert review
- **Communication challenges/successes:** Developed annotation guidelines and consensus-based review processes for disputed classifications
- **Cross-functional coordination:** Balanced technical ML development with strict legal and regulatory requirements across multiple departments

Leadership Moments:

- **Decisions you drove:** Chose two-stage architecture approach achieving optimal balance between accuracy and processing speed
- **People you influenced/mentored:** Strategic innovation building comprehensive document management platform beyond simple classification
- **Processes you improved:** Implemented human feedback loops and active learning for continuous model refinement
- **Initiative you took beyond assigned tasks:** Built comprehensive document management platform supporting entire clinical trial workflow

Problem-Solving Process:

- **Performance Bottleneck:** Solved 4-5 minute manual review time through two-stage automated architecture achieving 33-second processing
- **Class Imbalance:** Overcame F1 score limitation (0.58 → 0.93) through iterative model architecture improvements and synthetic data techniques

- **Regulatory Compliance:** Ensured zero tolerance for misclassification through comprehensive validation and human-in-the-loop workflows
- **Alternative Solutions Considered:** Evaluated various embedding models, classification approaches, and frontend architectures across four development versions

R - RESULTS Result Summary: The system achieved exceptional regulatory-grade performance with 0.93 F1 score and 87% processing time reduction (4-5 minutes to 33 seconds), while providing comprehensive document management capabilities supporting complex multi-study clinical trial operations.

Detailed Impact:

- **Quantified business impact:**
 - **87% processing time reduction** (4-5 minutes manual review to 33 seconds automated processing)
 - **0.93 F1 score** with 0.96 accuracy exceeding industry standards for regulatory compliance applications
 - **Hundreds of documents** now processed efficiently across multiple studies, countries, and consent form versions
 - **Eliminated critical bottlenecks** in clinical trial operations affecting legal and regulatory teams
- **Technical achievements:**
 - **Four-version iterative development:** Successfully evolved from 0.58 F1 score to 0.93 through systematic model improvements
 - **Comprehensive document platform:** Built complete management system with study-based filtering, version control, and geographic organization
 - **Two-stage production architecture:** Achieved optimal performance through embedding-based retrieval + efficient DistilBERT classification
 - **Advanced document capabilities:** Implemented batch processing, document comparison, full-text search, and automated reporting
- **Team/process improvements:**
 - **Regulatory workflow transformation:** Eliminated manual review bottlenecks while maintaining compliance standards
 - **Cross-functional collaboration:** Established effective feedback loops between technical development and legal/regulatory teams
 - **Quality assurance framework:** Created annotation guidelines and consensus-based review processes for sustained accuracy
 - **Scalable architecture:** Built system supporting expansion across additional studies and geographic regions
- **Personal growth/skills gained:**

- **Regulatory compliance expertise:** Deep understanding of clinical trial documentation requirements and compliance constraints
- **Iterative system development:** Advanced skills in systematic model improvement through multiple development cycles
- **Complex stakeholder management:** Experience balancing technical innovation with strict regulatory and legal requirements
- **End-to-end platform development:** Comprehensive experience building complete document management solutions beyond core ML capabilities

Project Information Header

- **Project:** CTMS Database Migration System - Alteryx to Python Migration
 - **Company:** Daiichi Sankyo
 - **Role:** R&D Data Governance Intern
 - **Duration:** 2 weeks
-

Summary STAR (1.5-2.5 minutes speaking time)

S: Daiichi Sankyo's R&D department was stuck with an Alteryx-based system for migrating clinical trial management data that was creating licensing bottlenecks, had limited customization options, and dependency issues affecting regulatory reporting workflows.

T: I had to completely migrate the entire data migration system from Alteryx to a custom Python solution in just 2 weeks, preserving all existing functionality while adding enhanced parallel processing, error handling, and scheduling capabilities.

A: I reverse-engineered all the existing Alteryx workflows and rewrote them in Python using SQLAlchemy for database connections, pandas for data processing, and implemented parallel processing with ThreadPoolExecutor - capabilities that weren't possible in the original Alteryx system.

R: Delivered a production-ready Python system that eliminated Alteryx licensing dependencies, achieved zero data loss during migration, improved performance with parallel processing, and gave the team enhanced error handling and monitoring capabilities they couldn't get with Alteryx.

Detailed STAR

S - SITUATION

Situation Summary: Daiichi Sankyo's R&D department needed to replace their existing Alteryx-based CTMS data migration system due to licensing constraints, limited customization options, and dependency issues that were creating bottlenecks in clinical trial data availability for regulatory reporting.

Detailed Context:

- **Company context & business problem:** Daiichi Sankyo's R&D department was using an Alteryx-based system for migrating clinical trial management system (CTMS) data between databases. The Alteryx solution created licensing bottlenecks, offered limited customization for complex table relationships, and dependency issues that affected the reliability of regulatory reporting and analysis workflows.
- **Team structure & your role:** R&D Data Governance team working with database administrators, clinical data managers, and regulatory reporting analysts. As R&D Data Governance Intern, I was responsible for completely migrating the existing Alteryx workflows to a custom Python solution.
- **Timeline & constraints:** 2-week timeline to migrate from Alteryx to Python while maintaining all existing functionality. Critical requirement to preserve data integrity and ensure zero downtime during the transition from the existing Alteryx system.
- **Scale/scope of the challenge:** Complete system migration, production database systems, regulatory compliance requirements, zero data loss tolerance

T - TASK/CHALLENGE

Task Summary: I was tasked with completely rewriting the existing Alteryx-based data migration system in Python, preserving all current functionality while adding enhanced capabilities for parallel processing, error handling, and automated scheduling.

Detailed Responsibilities:

- **Alteryx to Python Migration:** Analyze existing Alteryx workflows and completely rewrite them as a Python-based automated migration system
- **Functionality Preservation:** Ensure all current CTMS data migration capabilities are maintained during the transition
- **Enhanced Architecture:** Implement improved parallel processing, error handling, and scheduling capabilities beyond Alteryx limitations
- **Seamless Transition:** Deliver production-ready system with zero downtime and no disruption to ongoing clinical trial operations
- **Documentation & Training:** Create comprehensive documentation for the new Python system to replace Alteryx workflows

Technical challenges faced:

- **Workflow Translation:** Reverse-engineering and rewriting complex Alteryx data manipulation workflows in Python
- **Feature Parity:** Ensuring the new Python system matched or exceeded all existing Alteryx functionality
- **Performance Optimization:** Implementing parallel processing capabilities that weren't possible in the Alteryx system
- **Integration Continuity:** Maintaining all existing database connections and scheduling while transitioning platforms

Business/stakeholder challenges:

- **Rapid Migration Timeline:** Completing full system migration in only 2 weeks without disrupting critical operations
- **Licensing Cost Reduction:** Eliminating Alteryx licensing dependencies while maintaining system reliability
- **Stakeholder Confidence:** Ensuring clinical data managers and regulatory analysts trusted the new Python system
- **Regulatory Compliance:** Maintaining all compliance requirements during the platform transition

A - ACTION (*Detailed Breakdown*)

Action Summary: I systematically analyzed the existing Alteryx workflows, completely rewrote them in Python with enhanced capabilities, and implemented superior parallel processing and error handling that surpassed the original system's performance.

Technical Stack Used:

- **Migration Foundation:** Reverse-engineered Alteryx workflows and translated data manipulation logic to Python
- **Database Connectivity:** SQLAlchemy + psycopg2-binary (robust PostgreSQL connections replacing Alteryx database connectors)
- **Data Processing:** pandas + numpy (efficient data transformation replacing Alteryx data manipulation tools)
- **Parallel Processing:** concurrent.futures ThreadPoolExecutor (advanced capability not available in Alteryx)
- **Configuration Management:** python-dotenv + pyyaml (flexible configuration system improving on Alteryx limitations)
- **Logging & Monitoring:** Python logging with pytz (comprehensive audit trails exceeding Alteryx capabilities)

Collaboration Details:

- **Legacy System Analysis:** Worked with database administrators to understand existing Alteryx workflow dependencies
- **Requirements Validation:** Collaborated with clinical data managers to ensure Python system met all operational needs
- **Transition Planning:** Coordinated with regulatory analysts to maintain compliance during migration
- **Testing & Validation:** Extensive parallel testing of Python system against existing Alteryx outputs

Leadership Moments:

- **Migration Strategy:** Led decision to completely rewrite in Python rather than attempting partial migration
- **Architecture Innovation:** Drove implementation of parallel processing capabilities impossible in Alteryx
- **Quality Assurance:** Established comprehensive testing protocols ensuring migration accuracy
- **Knowledge Transfer:** Created detailed documentation enabling team to maintain Python system

Problem-Solving Process:

- **Workflow Analysis:** Systematically reverse-engineered Alteryx processes to understand all data transformation logic
- **Performance Enhancement:** Implemented parallel processing using ThreadPoolExecutor, significantly improving on Alteryx performance
- **Error Handling Improvement:** Built sophisticated error recovery mechanisms beyond Alteryx capabilities
- **Validation Strategy:** Created comprehensive testing framework comparing Python outputs with Alteryx results

R - RESULTS

Result Summary: The Python migration successfully eliminated Alteryx licensing dependencies, achieved superior performance with parallel processing, and maintained zero data loss while providing enhanced capabilities that exceeded the original system's functionality.

Detailed Impact: Quantified business impact:

- **Licensing Elimination:** Removed Alteryx licensing costs and dependencies, reducing operational overhead
- **Performance Improvement:** Parallel processing reduced migration time compared to sequential Alteryx workflows
- **System Reliability:** Achieved zero data loss across all migrations during and after transition
- **Enhanced Capabilities:** Delivered superior error handling and monitoring capabilities beyond Alteryx limitations

Technical achievements:

- **Complete Migration:** Successfully translated all Alteryx workflows to Python within 2-week timeline
- **Enhanced Architecture:** Implemented parallel processing and advanced error handling not possible in Alteryx
- **Production Readiness:** Delivered robust system with comprehensive logging and monitoring capabilities
- **Seamless Transition:** Zero downtime migration maintaining all existing functionality and data integrity

Team/process improvements:

- **Platform Independence:** Eliminated vendor dependency on Alteryx licensing and support constraints
- **Customization Capability:** Enabled future enhancements and modifications not possible with Alteryx limitations
- **Knowledge Transfer:** Created comprehensive Python documentation replacing proprietary Alteryx workflows
- **System Maintainability:** Established open-source Python solution that team can modify and enhance independently

Personal growth/skills gained:

- **Legacy System Migration:** Expertise in analyzing and replacing proprietary systems with custom solutions
- **Rapid Development:** Proven ability to deliver complete system migration under aggressive 2-week timeline
- **Requirements Analysis:** Advanced skills in reverse-engineering existing workflows and improving upon them
- **Platform Translation:** Experience converting proprietary tool workflows to open-source Python solutions

Project Information Header

- **Project:** AI-Driven Content Automation & SEO Optimization System
- **Company:** White Label Resell
- **Role:** Machine Learning Engineer (Sole Backend/AI/Cloud Developer)
- **Duration:** [Duration not specified - ongoing project]

Summary STAR (1.5-2.5 minutes speaking time)

S: White Label Resell's digital marketing clients were stuck with slow, expensive manual content creation that couldn't scale for competitive e-commerce needs - they needed large-scale SEO-optimized content with internal linking and multi-language support but had no technical solution.

T: As the sole backend/AI/cloud developer, I had to single-handedly build a comprehensive AI-powered content automation system from scratch, integrating multiple AI models and cloud infrastructure to generate 50,000+ articles monthly with enterprise-grade reliability.

A: I architected a complete system integrating GPT-3.5, BERT, FLAN-T5 for content generation, Stable Diffusion for images, and Google Translate API for 50+ languages, all running on AWS cloud infrastructure with automated internal linking algorithms and scalable API systems.

R: Built enterprise-scale capabilities generating 10,000+ monthly articles as a solo developer, secured \$50-100K quarterly contracts, achieved 60% increase in organic traffic through AI-powered internal linking, and boosted international engagement by 40% with multi-language content.

Detailed STAR

S - SITUATION Situation Summary: White Label Resell needed to revolutionize content generation for digital marketing clients who were facing slow, expensive manual processes that couldn't scale for competitive e-commerce needs, requiring large-scale SEO-optimized content with advanced features like internal linking and multi-language support.

Detailed Context:

- **Company context & business problem:** White Label Resell needed to revolutionize content generation for digital marketing clients facing slow, expensive, and inconsistent manual content creation. Businesses required large-scale, SEO-optimized content with effective internal linking, multi-language localization, and automated scalability to compete in the fast-paced e-commerce landscape.
- **Team structure & your role:** Small team with frontend developers and business stakeholders. As the sole Backend/AI/Cloud Developer, I was entirely responsible for designing and implementing the complete technical infrastructure including AI models, cloud architecture, and all backend systems.
- **Timeline & constraints:** Aggressive scaling requirements needing 10,000+ articles monthly capability. Multi-language support across 50+ languages required. High-performance expectations with enterprise-level reliability and SEO optimization standards.
- **Scale/scope of the challenge:** Building enterprise-scale system as sole developer, 10,000+ monthly articles, 50+ language support, complete technical ownership

T - TASK/CHALLENGE Task Summary: As the only technical developer, I was tasked with single-handedly building a comprehensive AI-powered content automation system from scratch, including all AI models, cloud infrastructure, API development, and deployment architecture.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Design and implement entire end-to-end content automation pipeline as sole technical developer
 - Build AI-driven content generation using multiple NLP models (GPT-3.5, BERT, FLAN-T5)
 - Develop multi-language content generation via Google Translate API integration
 - Create AI-powered image generation with Stable Diffusion and nine aesthetic styles
 - Architect complete AWS cloud infrastructure (API Gateway, ECS, Lambda, S3, SageMaker)
 - Implement automated internal linking system with custom SEO strategies
 - Build scalable API systems supporting bulk content generation
- **Technical challenges faced:**
 - Single-handedly architecting enterprise-scale system handling 10,000+ articles monthly
 - Integrating multiple AI models (text, image, translation) in cohesive automated pipeline
 - Implementing dynamic internal linking algorithms for improved search engine rankings
 - Building cost-effective cloud infrastructure supporting massive content generation volumes
- **Business/stakeholder challenges:**
 - Delivering complete technical solution as solo developer with no technical team support
 - Balancing system complexity with rapid development timeline constraints
 - Ensuring enterprise-grade reliability and performance with limited resources
 - Managing client requirements and preventing scope creep through technical documentation
- **Success criteria and constraints:** 10,000+ monthly articles, 50+ language support, enterprise reliability, solo development, cost-effective scaling

A - ACTION (Detailed Breakdown) Action Summary: I architected and built a complete AI automation ecosystem single-handedly, integrating multiple state-of-the-art models with sophisticated cloud infrastructure, achieving enterprise-scale content generation capabilities that secured significant quarterly contracts.

Technical Stack Used:

- **AI Models:** GPT-3.5, BERT, FLAN-T5 (content generation), Stable Diffusion (image generation), Google Translate API (multi-language support)
- **Cloud Infrastructure:** AWS API Gateway, ECS, Lambda, SQS, SNS, EC2, S3, SageMaker (complete serverless architecture for scalability)
- **Backend Development:** Flask, FastAPI (high-performance API development for bulk processing)
- **Database & Storage:** SQL, SQLite, DynamoDB, S3 (structured data management and content storage)
- **Deployment:** Docker containerization (efficient cloud deployment and scaling)

Collaboration Details:

- **Worked with:** Frontend developers (API integration), business stakeholders (requirements), content strategists (SEO optimization)
- **How decisions were made:** Single-handedly made all technical architecture decisions including cloud services, AI model selection, and system design
- **Communication challenges/successes:** Created comprehensive Software Requirement Specification (SRS) documenting technical capabilities and limitations
- **Cross-functional coordination:** Balanced technical implementation with business requirements while managing all backend development independently

Leadership Moments:

- **Decisions you drove:** Solo Technical Leadership taking complete ownership of technical architecture decisions as sole developer
- **People you influenced/mentored:** Innovation driver designing novel graph-based AI linking models for dynamic internal linking optimization
- **Processes you improved:** Established multi-step AI refinement pipeline ensuring quality at massive scale
- **Initiative you took beyond assigned tasks:** Created comprehensive documentation and quality assurance systems beyond basic requirements

Problem-Solving Process:

- **Scale vs Quality:** Implemented multi-step AI refinement and NLP filtering maintaining content quality across 10,000+ monthly articles
- **Integration Complexity:** Single-handedly integrated text generation, image creation, translation, and SEO optimization in unified pipeline
- **Cost Optimization:** Architected serverless AWS infrastructure minimizing operational costs while supporting enterprise volumes
- **Alternative Solutions Considered:** Evaluated monolithic vs. microservices, various AI model combinations, different cloud providers

R - RESULTS Result Summary: As sole technical developer, I delivered an enterprise-grade content automation system generating 10,000+ monthly articles, securing \$50-100K quarterly contracts, and achieving significant improvements in organic traffic and international engagement.

Detailed Impact:

- **Quantified business impact:**
 - **10,000+ articles/month** automated generation capability built by single developer
 - **\$50-100K quarterly contracts** secured directly through superior technical capabilities
 - **60% increase** in organic traffic through AI-powered internal linking algorithms
 - **40% increase** in international engagement via multi-language content generation

- **Technical achievements:**
 - **Complete system architecture:** Single-handedly built entire AI automation pipeline from AI models to cloud deployment
 - **Multi-language scaling:** Successfully implemented Google Translate API supporting 50+ languages
 - **Advanced AI integration:** Combined GPT-3.5, BERT, FLAN-T5, and Stable Diffusion in unified content generation system
 - **Enterprise cloud infrastructure:** Architected scalable AWS ecosystem supporting massive content volumes
- **Team/process improvements:**
 - **Solo development excellence:** Demonstrated ability to deliver enterprise-scale solutions as individual contributor
 - **Technical documentation standards:** Created comprehensive SRS preventing scope creep and ensuring system maintainability
 - **Automated quality assurance:** Implemented plagiarism checks and grammar validation ensuring content credibility
 - **Scalable architecture patterns:** Established cloud-native design principles for future AI-intensive projects
- **Personal growth/skills gained:**
 - **Full-stack AI system architecture:** Complete expertise in designing and implementing enterprise AI solutions independently
 - **Cloud infrastructure mastery:** Advanced skills in AWS services orchestration for AI-intensive applications
 - **Solo project management:** Proven ability to deliver complex technical solutions with complete ownership and accountability
 - **AI model integration expertise:** Deep understanding of combining multiple AI technologies in production-ready systems

Project Information Header

- **Project:** Sky Scraper - AI-Powered Chrome Extension for Scalable Content Automation
- **Company:** SEOContent.ai
- **Role:** AI Infrastructure Engineer
- **Duration:** 6 Months

Summary STAR (1.5-2.5 minutes speaking time)

S: SEOContent.ai clients were stuck spending hours on manual keyword research and article structuring, which prevented them from building the large-scale SEO content campaigns they needed to compete effectively in their markets.

T: My job was to build an AI-powered Chrome extension that could automate the entire content creation workflow - from keyword research to generating up to 15 interconnected articles per topic cluster - while maintaining SEO quality standards across a 6-month timeline.

A: I designed a microservices architecture on AWS using fine-tuned BERT models for keyword extraction and GPT models for content generation. The system combined web scraping through Chrome extension, NLP processing for topic clustering, and parallel processing workflows that could handle high-volume content creation.

R: We slashed content creation time by 60-70% (from 4 hours down to just 10 minutes plus editing), successfully launched the beta with contract renewals, and clients started seeing measurable improvements in their organic traffic using our automated content clusters.

Detailed STAR

S - SITUATION Situation Summary: SEOContent.ai needed to solve the scalability challenge in content marketing where clients required extensive multi-article campaigns but faced hours of manual processes, preventing them from building comprehensive SEO content strategies at the scale needed for competitive advantage.

Detailed Context:

- **Company context & business problem:** SEOContent.ai needed to solve the scalability challenge in content marketing where clients required extensive multi-article campaigns but faced hours of manual keyword research and article structuring. Traditional content creation processes were disconnected and inefficient, preventing clients from building comprehensive SEO content strategies at scale.
- **Team structure & your role:** Cross-functional Agile team including product managers, frontend developers, DevOps engineers, and SEO specialists. As AI Infrastructure Engineer, I was responsible for designing system architecture components, implementing generative AI modules, and ensuring pipeline reliability under heavy usage across the entire content automation workflow.
- **Timeline & constraints:** 6-month timeline from initial design to beta launch. High-volume processing requirements with need for cost-effective scalability. Critical business requirement as clients demanded large-scale SEO content production capabilities for competitive advantage.
- **Scale/scope of the challenge:** Building comprehensive content automation system, generating 15 interconnected articles per cluster, enterprise-grade reliability requirements

T - TASK/CHALLENGE Task Summary: I was tasked with building a comprehensive AI-powered content automation system that could transform manual content marketing workflows into scalable,

automated pipelines generating up to 15 interconnected articles per topic cluster while maintaining SEO best practices.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Design and implement robust web scraping functionality through Chrome Extension integration
 - Build NLP-based keyword extraction system using fine-tuned BERT models for topic clustering
 - Develop multi-article generation workflow with LLM integration (GPT-3.5, GPT-NeoX)
 - Implement AWS microservices architecture for elastic scaling and cost optimization
 - Create content assembly and publishing pipeline supporting WordPress integration
 - Build parallel processing capabilities to accelerate content generation workflows
- **Technical challenges faced:**
 - Maintaining consistent AI-generated content quality following brand tone and SEO guidelines
 - Balancing performance vs. cost when generating multiple long-form articles simultaneously
 - Handling evolving SEO practices requiring system flexibility and adaptability
 - Implementing reliable scraping functionality resistant to website changes and anti-bot measures
- **Business/stakeholder challenges:**
 - Cross-functional coordination between marketing, engineering, and design teams
 - Translating complex technical AI capabilities into user-friendly features
 - Meeting diverse client requirements for content volume, quality, and SEO optimization
 - Ensuring system reliability during high-volume processing for enterprise clients
- **Success criteria and constraints:** 6-month delivery, 15 articles per cluster, 60%+ time reduction, enterprise reliability, cost-effective scaling

A - ACTION (Detailed Breakdown) Action Summary: I architected a comprehensive microservices system using AWS infrastructure, implemented advanced NLP processing for content automation, and built scalable parallel processing capabilities that reduced content creation time from 4 hours to 10 minutes while maintaining SEO quality standards.

Technical Stack Used:

- **Python + JavaScript + TypeScript:** Core development languages (chosen for AI/ML libraries and web scraping capabilities)
- **Node.js + Express:** Chrome Extension backend integration (seamless browser-to-cloud data pipeline)

- **AWS ECS + Lambda:** Microservices orchestration (containerized scaling for different processing functions)
- **Fine-tuned BERT Models:** Keyword extraction and topic clustering (superior performance for content analysis)
- **GPT-3.5 + GPT-NeoX:** Content generation through load-balanced microservices (high-quality article creation)
- **AWS S3 + DynamoDB:** Data storage and metadata logging (cost-effective scalable storage)
- **Docker:** Containerization strategy (modular design improving scalability and maintainability)

Collaboration Details:

- **Worked with:** Product managers (requirements), frontend developers (integration), DevOps (infrastructure), SEO specialists (optimization guidelines)
- **How decisions were made:** Participated in Agile sprint planning with bi-weekly reviews, contributed technical insights during prototype evaluations
- **Communication challenges/successes:** Led technical documentation sessions helping non-technical teams understand AI generation processes and architecture decisions
- **Cross-functional coordination:** Balanced technical architecture with product requirements and user experience across marketing, engineering, and design teams

Leadership Moments:

- **Decisions you drove:** Implemented container-based AWS ECS strategy with horizontal scaling solving performance bottlenecks under high load
- **People you influenced/mentored:** Technical innovation guiding team toward microservices architecture that became company standard
- **Processes you improved:** Created centralized SEO knowledge base enabling quick adaptation to changing SEO standards and client requirements
- **Initiative you took beyond assigned tasks:** Developed dynamic prompt management system automatically adjusting content generation based on user preferences and feedback

Problem-Solving Process:

- **Performance Optimization:** Evaluated single large model vs. multiple smaller instances, chose parallel processing approach balancing cost with generation speed
- **Quality Assurance:** Implemented custom prompt templates with feedback loops ensuring content cohesion and brand consistency
- **Scalability Challenge:** Designed modular microservices architecture allowing independent scaling of scraping, extraction, and generation components
- **Alternative Solutions Considered:** Monolithic vs. microservices architecture, cloud vs. hybrid processing, synchronous vs. asynchronous content generation

R - RESULTS Result Summary: The Skyscraper platform achieved remarkable productivity improvements with 60-70% reduction in content creation time, successful beta launch with contract

renewals, and established new architecture standards for SaaS AI products while demonstrating measurable organic traffic improvements for clients.

Detailed Impact:

- **Quantified business impact:**
 - **60-70% reduction** in manual content creation time (4 hours to 10 minutes + 1 hour editing)
 - **15 interconnected articles** generated per topic cluster with automated SEO optimization
 - **Contract renewals and expansions** from beta testers demonstrating strong product-market fit
 - **Measurable organic traffic improvements** reported by users after adopting Skyscraper platform
- **Technical achievements:**
 - **Scalable microservices architecture:** Successfully handled high-volume processing with elastic scaling
 - **Advanced NLP pipeline:** Implemented BERT-based keyword extraction with intelligent topic clustering
 - **Multi-model integration:** Built load-balanced system supporting GPT-3.5 and GPT-NeoX for diverse content needs
 - **End-to-end automation:** Created complete pipeline from web scraping to WordPress publishing with JSON/CSV export capabilities
- **Team/process improvements:**
 - **Cross-functional collaboration:** Established effective communication patterns between technical and non-technical teams
 - **Architecture influence:** Project's microservices approach became standard for future SaaS AI offerings
 - **Technical documentation:** Created comprehensive guides enabling knowledge transfer and system maintenance
 - **User-centric development:** Implemented early user testing identifying prompt-engine issues before MVP finalization
- **Personal growth/skills gained:**
 - **Container orchestration expertise:** Advanced skills in Docker and AWS ECS for AI workload management
 - **NLP prompt engineering:** Deep understanding of LLM optimization for consistent content quality
 - **Multi-service architecture:** Experience designing and implementing complex AI pipeline systems
 - **Cross-functional leadership:** Enhanced ability to translate technical capabilities into business value and user requirements

Project Information Header

- **Project:** Zero-Maintenance AI Tools Platform with Dynamic Configuration System
- **Company:** SEOContent.ai
- **Role:** AI Infrastructure Engineer
- **Duration:** 3 Weeks

Summary STAR (1.5-2.5 minutes speaking time)

S: SEOContent.ai wanted to build a platform with 50+ AI tools, but they knew from experience that having developers maintain every tool update would create massive bottlenecks - content experts would constantly need tech help for simple changes.

T: My job was to completely flip this model - build a platform where content experts could manage, update, and deploy tools themselves without any technical knowledge, all while keeping costs low

A: I created this hybrid system using Google Sheets (since everyone knows how to use spreadsheets) connected to AWS serverless infrastructure. The frontend dynamically reads JSON configs, so when someone updates a sheet, the tool changes instantly - no code deployment needed. We delivered it in just 3 weeks.

R: The results blew everyone away - we got 5,000 users in the first month, cut costs by 98.4% compared to no-code platforms, and tool updates now take under 90 seconds instead of waiting for developers.

Detailed STAR

S - SITUATION Situation Summary: SEOContent.ai needed to build a comprehensive AI tools platform with 50+ tools but faced a critical strategic challenge where traditional approaches would create unsustainable developer dependencies, limiting business agility and creating operational bottlenecks for tool maintenance.

Detailed Context:

- **Company context & business problem:** SEOContent.ai needed to build a comprehensive AI tools platform with 50+ tools but faced a critical strategic challenge: traditional approaches would create unsustainable developer dependencies for tool maintenance and updates. The company anticipated that content experts would identify improvements but would depend on developers to implement them, creating operational bottlenecks and limiting business agility.
- **Team structure & your role:** Small startup team with content experts, frontend developers, and documentation specialists. As the sole AI Infrastructure Engineer, I was responsible for

architecting and building the entire platform from scratch, specifically designing for zero ongoing developer maintenance.

- **Timeline & constraints:** Aggressive 3-week timeline to deliver production-ready platform. Need to support immediate scaling to 50+ tools with potential expansion to 100+. Critical requirement: system must operate independently without developer intervention for tool modifications and updates.
- **Scale/scope of the challenge:** Building 50+ tool platform from scratch, eliminating developer dependencies entirely, achieving dramatic cost reduction, delivering in 3 weeks

T - TASK/CHALLENGE Task Summary: I was tasked with architecting a revolutionary AI tools platform that could eliminate developer dependencies entirely, enabling content experts to manage, modify, and deploy tools independently while maintaining enterprise-grade performance and cost efficiency.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Design architecture eliminating developer maintenance requirements for tool lifecycle management
 - Build dynamic frontend system supporting unlimited tool configurations without code changes
 - Implement real-time deployment pipeline enabling instant tool updates and modifications
 - Create cost-effective infrastructure supporting 50+ tools with linear scaling capabilities
 - Establish user-friendly interface requiring zero technical training for content experts
- **Technical challenges faced:**
 - Designing dynamic frontend rendering system supporting unlimited tool variations
 - Implementing real-time configuration deployment without application restarts
 - Building cost-effective architecture competing with \$2000+/month no-code alternatives
 - Creating validation and error handling for non-technical users managing complex configurations
- **Business/stakeholder challenges:**
 - Rapid 3-week delivery timeline requiring immediate architecture decisions
 - Balancing technical sophistication with user simplicity for content experts
 - Achieving dramatic cost reduction while maintaining enterprise-grade functionality
 - Ensuring sustainable competitive advantages through operational independence
- **Success criteria and constraints:** Zero developer maintenance, 3-week delivery, 50+ tools support, massive cost reduction, non-technical user operation

A - ACTION (Detailed Breakdown) Action Summary: I architected an innovative hybrid system combining Google Sheets for familiar user interface with AWS serverless infrastructure, implementing dynamic configuration rendering that eliminated developer dependencies while achieving 98.4% cost reduction compared to traditional alternatives.

Technical Stack Used:

- **Google Sheets + Apps Script:** Content management and change detection (chosen for zero learning curve and built-in collaboration)
- **AWS Lambda:** Configuration processing and deployment automation (serverless pay-per-execution model for cost efficiency)
- **Amazon S3 + CloudFront:** JSON configuration storage and global CDN distribution (instant updates without app deployments)
- **Dynamic JavaScript Frontend:** Runtime configuration rendering (supporting unlimited tool variations without code changes)
- **DynamoDB:** Metadata tracking and analytics (cost-effective NoSQL for tool usage metrics)

Collaboration Details:

- **Worked with:** Content experts (tool requirements), frontend developers (integration), documentation team (user guides)
- **How decisions were made:** Led comprehensive architecture evaluation comparing 4 different approaches considering cost, complexity, and scalability
- **Communication challenges/successes:** Facilitated strategic planning sessions anticipating operational challenges before implementation
- **Cross-functional coordination:** Balanced technical architecture decisions with business operational requirements and user experience needs

Leadership Moments:

- **Decisions you drove:** Chose unconventional Google Sheets approach over traditional CMS solutions, resulting in 98.4% cost reduction
- **People you influenced/mentored:** Strategic innovation guiding team toward configuration-driven architecture enabling business user autonomy
- **Processes you improved:** Established real-time deployment pipeline allowing tool updates in under 90 seconds
- **Initiative you took beyond assigned tasks:** Designed configuration-driven architecture enabling 50x faster iteration cycles than traditional development

Problem-Solving Process:

- **Architecture Evaluation:** Systematically compared Traditional CMS (\$500/month), Custom Admin (\$200/month), No-Code Platform (\$2000/month) vs. Hybrid Solution (\$50/month)
- **Dynamic Rendering Challenge:** Built frontend system reading JSON configurations at runtime, supporting unlimited field types and variations
- **Cost Optimization:** Implemented serverless architecture with intelligent caching reducing operational costs by 98.4%
- **Alternative Solutions Considered:** Evaluated WordPress/Drupal, React admin panels, Zapier/Bubble platforms before choosing hybrid approach

R - RESULTS Result Summary: The platform achieved exceptional market success with 5,000 users in the first month, 1,600 recurring users, and 98.4% cost reduction while establishing sustainable competitive advantages through complete operational independence and 50x faster iteration cycles.

Detailed Impact:

- **Quantified business impact:**

- **5,000 users** in first month exceeding all initial adoption projections
- **1,600 recurring users** in month 2 (32% retention rate) demonstrating strong product-market fit
- **98.4% cost reduction** (\$50/month vs. \$2000+/month no-code alternatives)
- **50x faster** tool improvement cycles enabling same-day market responsiveness

- **Technical achievements:**

- **Real-time deployment:** Tool updates deployed in under 90 seconds without application restarts
- **Zero developer maintenance:** 100% content expert autonomy for tool lifecycle management
- **Dynamic configuration system:** Supporting unlimited tool variations through runtime JSON rendering
- **Enterprise-grade performance:** 99.95% uptime with <800ms frontend load times via CDN optimization

- **Team/process improvements:**

- **Operational independence:** Eliminated entire category of developer maintenance overhead
- **Business agility:** Enabled same-day implementation of user feedback and market changes
- **Cost efficiency standards:** Established architecture patterns achieving 98%+ cost savings for future projects
- **Developer focus:** Freed 95% of development time for core platform innovation vs. maintenance

- **Personal growth/skills gained:**

- **Strategic architecture evaluation:** Expertise in comparing technical approaches considering long-term operational costs
- **Configuration-driven design:** Advanced skills in building systems enabling business user autonomy
- **Cost optimization mastery:** Deep understanding of serverless architecture for dramatic cost reduction
- **Market validation:** Experience building and launching products achieving immediate product-market fit

Project Information Header

- **Project:** AI-Powered Content Generation API for SEO Marketing
- **Company:** SEOContent.ai
- **Role:** AI Infrastructure Engineer
- **Duration:** 3 Weeks

Summary STAR (1.5-2.5 minutes speaking time)

S: SEOContent.ai needed to commercialize their existing White Label Resell AI content generation system by building a market-ready API for external digital marketing clients who would directly like to integrate our Content generation system into their solutions.

T: I was tasked with architecting a complete API system using AWS API Gateway that would securely expose our existing AI capabilities (GPT-3.5, BERT, FLAN-T5, Stable Diffusion), implement enterprise authentication and billing, and handle thousands of concurrent client requests within an aggressive 3-week timeline.

A: I built a comprehensive REST API using AWS API Gateway with Lambda functions, integrated it with existing White Label Resell infrastructure, implemented AWS Cognito + Stripe for authentication and billing, and optimized performance through asynchronous processing patterns.

R: Delivered a production-ready API reducing content creation time by ~85% (2.5 minutes to 45 seconds), enabled processing of 10,000+ monthly requests with 99.9% uptime, and secured significant quarterly contracts while achieving 90% successful client integrations.

Detailed STAR

S - SITUATION

Situation Summary: SEOContent.ai needed to quickly commercialize the existing White Label Resell content generation system by building a market-ready API that would allow external clients to access our proven AI capabilities through a scalable, secure interface.

Detailed Context:

- **Company context & business problem:** SEOContent.ai, a subsidiary of White Label Resell, needed to monetize our existing AI-powered content automation system by creating an external

API product. Digital marketing clients were struggling with slow, expensive manual content creation and needed access to our proven system that could generate 10,000+ articles monthly with multi-language support.

- **Team structure & your role:** Small startup team with frontend developers and business stakeholders. As the AI Infrastructure Engineer, I was responsible for building the complete API layer that would expose White Label Resell's existing AI capabilities to external clients.
- **Timeline & constraints:** Aggressive 3-week timeline to deliver production-ready API. Existing White Label Resell system provided the AI foundation, but required complete API architecture design and secure external access implementation.
- **Scale/scope of the challenge:** Building enterprise-grade API gateway, implementing secure authentication and billing, integrating with existing complex AI pipeline

T - TASK/CHALLENGE

Task Summary: I was tasked with building a complete API system using AWS API Gateway that would securely expose our existing White Label Resell AI capabilities, implement enterprise authentication and billing, and handle thousands of concurrent client requests.

Detailed Responsibilities:

- **API Architecture & Development:** Design and implement REST API using AWS API Gateway with proper versioning, rate limiting, and comprehensive documentation
- **System Integration:** Connect new API layer with existing White Label Resell AI infrastructure (GPT-3.5, BERT, FLAN-T5, Stable Diffusion, Google Translate)
- **Security & Authentication:** Implement secure API key management using AWS Cognito with Stripe integration for usage-based billing
- **Performance Optimization:** Ensure API can handle thousands of concurrent requests while leveraging existing AI pipeline efficiently
- **Client Onboarding:** Create comprehensive API documentation, SDKs, and integration guides for seamless client adoption

Technical challenges faced:

- **Legacy Integration:** Connecting new API Gateway with existing White Label Resell architecture without disrupting internal operations
- **Performance Optimization:** Reducing response times from existing 2.5-minute internal process to target 45-second API response
- **Request Management:** Implementing intelligent queuing and load balancing for high-volume API requests
- **Multi-Model Coordination:** Orchestrating complex AI pipeline (text generation, image creation, translation) through simple API endpoints

Business/stakeholder challenges:

- **Rapid Timeline:** Delivering complete commercial API product in only 3 weeks

- **Revenue Model:** Creating pricing tiers and usage tracking that reflects infrastructure costs and profit margins
- **Client Success:** Ensuring smooth onboarding experience for external clients transitioning from manual processes

A - ACTION (*Detailed Breakdown*)

Action Summary: I architected a comprehensive API system using AWS API Gateway that seamlessly integrated with our existing White Label Resell infrastructure, implementing intelligent caching and optimization strategies that dramatically improved response times while maintaining enterprise-grade reliability.

Technical Stack Used:

- **AWS API Gateway:** Primary API management platform (chosen for built-in rate limiting, caching, authentication, and monitoring)
- **AWS Lambda:** Serverless functions for API business logic and White Label Resell system integration
- **Integration Layer:** Custom Python middleware connecting to existing White Label Resell AI pipeline (GPT-3.5, BERT, FLAN-T5, Stable Diffusion, Google Translate)
- **AWS Cognito + Stripe:** API key management and credits-based billing system
- **CloudWatch & SNS:** Comprehensive API monitoring, logging, and alerting

Collaboration Details:

- **White Label Resell Team:** Coordinated with original system architects to understand existing AI pipeline and integration points
- **Frontend Developers:** Provided API specifications, SDKs, and authentication integration guidance
- **Business Team:** Collaborated on pricing strategy, usage limits, and billing implementation requirements
- **Documentation Team:** Created comprehensive API guides, code examples, and troubleshooting resources

Leadership Moments:

- **API Design Strategy:** Led architectural decisions on REST endpoint design, choosing simplicity for broad client compatibility
- **Integration Approach:** Drove decision to build API layer on top of existing White Label Resell system rather than rebuilding AI capabilities
- **Performance Standards:** Established API response time SLAs and comprehensive monitoring protocols
- **Client Success Framework:** Created technical onboarding process and support documentation for API adopters

Problem-Solving Process:

- **Latency Optimization:** Implemented API Gateway caching and Lambda warming strategies, reducing cold starts and improving response times by 85%
- **System Integration:** Developed asynchronous processing patterns to leverage existing White Label Resell AI pipeline without blocking API responses
- **Load Management:** Built intelligent queuing system handling burst traffic while maintaining fair usage across clients
- **Error Handling:** Created comprehensive error response system with proper HTTP status codes and detailed debugging information

R - RESULTS

Result Summary: The API successfully transformed our White Label Resell capabilities into a commercial product, achieving exceptional performance with 85% faster content generation, serving 10,000+ monthly requests, and enabling seamless client integration while maintaining the reliability of our existing AI infrastructure.

Detailed Impact: Quantified business impact:

- **API Performance:** Reduced average response time from 2.5 minutes to 45 seconds (85% improvement)
- **Commercial Success:** Enabled processing of 50,000+ API requests monthly with 99.9% uptime
- **Client Results:** API users achieved 300% increase in organic traffic through automated, SEO-optimized content
- **Revenue Generation:** Secured significant quarterly contracts through superior API capabilities and performance

Technical achievements:

- **Scalable API Architecture:** Successfully handled thousands of concurrent requests through AWS API Gateway auto-scaling
- **Seamless Integration:** Zero-downtime integration with existing White Label Resell AI infrastructure
- **Enterprise Security:** Implemented robust authentication, rate limiting, and comprehensive usage monitoring
- **Developer Experience:** Created intuitive API with comprehensive documentation resulting in 90% successful client integrations

Team/process improvements:

- **Reusable Architecture:** Established API development patterns and standards for future product development
- **Monitoring Excellence:** Implemented comprehensive API analytics and performance dashboards

- **Client Support Process:** Created streamlined technical onboarding reducing support requirements by 70%
- **Integration Framework:** Developed reusable patterns for exposing internal AI capabilities as commercial APIs

Personal growth/skills gained:

- **API Product Development:** End-to-end experience building commercial APIs from existing AI systems using AWS API Gateway
- **Rapid Development:** Proven ability to deliver enterprise-grade API products under aggressive 3-week timeline
- **System Integration Mastery:** Advanced skills in connecting API layers with complex existing AI infrastructure
- **Commercial Product Strategy:** Experience translating internal AI capabilities into market-ready products with proper pricing and scaling

Project Information Header

- **Project:** Medical Image Data Engineering Pipeline for Pathology Workflows
- **Company:** Digital Clinics Research and Services
- **Role:** Data Scientist
- **Duration:** 12 Months

Summary STAR (1.5-2.5 minutes speaking time)

S: Digital Clinics was developing a comprehensive medical image analysis system for cancer diagnosis, requiring a scalable data engineering pipeline to process massive 10-20GB TIFF/SVS medical imaging files that pathologists couldn't efficiently handle manually, creating bottlenecks in diagnosis workflows.

T: I was tasked with engineering a complete automated data processing infrastructure that could break down massive medical imaging files into manageable formats, create efficient annotation workflows for junior pathologists, and build seamless deployment systems integrating with our CADD4MBC clinical platform.

A: I built a robust Python-based pipeline using OpenCV for automated file processing, integrated QuPath with custom Groovy scripts for annotation workflows, and created deployment systems connecting junior pathologists, senior doctors, and our clinical platform with MongoDB for metadata tracking.

R: Achieved 1.5TB weekly data throughput with zero data loss, reduced manual processing time by 60%, enabled processing of 100,000+ images monthly, and established scalable workflows supporting both pathologist training and downstream AI projects.

Detailed STAR

S - SITUATION Situation Summary: Digital Clinics Research and Services was facing major bottlenecks in medical image analysis where pathologists couldn't efficiently work with massive 10-15GB medical imaging files, creating delays in cancer diagnosis workflows and preventing scalable annotation processes.

Detailed Context:

- **Company context & business problem:** Digital Clinics Research and Services needed to revolutionize medical image analysis workflows where the data pipeline flow was manual either done by junior pathologist or developer. The existing workflow was time consuming and error prone, creating bottlenecks in cancer diagnosis pipelines.
- **Team structure & your role:** Medical AI research team collaborating with pathologists from Basavataarakam Indo-American Cancer Hospital, Hyderabad. As Data Scientist, I was the lead data engineer responsible for designing and implementing the entire automated data pipeline architecture from raw medical images to production-ready clinical workflows.
- **Timeline & constraints:** Built and maintained it over my 12-month timeline processing 1.5TB of medical imaging data weekly. Need for seamless integration with existing clinical workflows without disruption. Strict data integrity requirements for medical-grade applications.
- **Scale/scope of the challenge:** Processing 10-15GB individual TIFF files daily, supporting multiple image formats, ensuring zero data loss across the entire pipeline

T - TASK/CHALLENGE Task Summary: I was responsible for engineering a complete data processing infrastructure that could break down massive medical imaging files, create efficient annotation workflows for junior pathologists, and build seamless deployment systems to our clinical platform.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Design and implement automated pipeline to break down 10-15GB TIFF and various medical image formats
 - Engineer annotation workflow systems for junior pathologists
 - Build deployment scripts for automatic integration with CADD4MBC backend
 - Establish data quality control and validation processes
 - Design scalable architecture supporting multiple medical image formats
- **Technical challenges faced:**
 - Handling multiple medical image formats (TIFF, SVS, etc.) with varying structures
 - Ensuring zero data loss during file processing and transformation
 - Managing memory efficiently while processing 10-15GB files
 - Creating seamless integration between annotation tools and clinical platform

- Maintaining data integrity across multi-stage workflows
- **Business/stakeholder challenges:**
 - Integrating new automated systems with existing clinical workflows
 - Training junior pathologists on new annotation processes
 - Ensuring senior pathologists could efficiently review and provide feedback
 - Managing data privacy and security for sensitive medical imaging data
- **Success criteria and constraints:** Zero data loss, seamless clinical workflow integration, 1.5TB weekly processing capacity, medical-grade data security

A - ACTION (Detailed Breakdown) Action Summary: I built a robust data engineering pipeline using custom Python scripts for file processing, created automated annotation workflows with quality controls, and engineered seamless deployment systems that connected junior pathologists, senior doctors, and our clinical platform.

Technical Stack Used:

- **Python + OpenCV + NumPy:** Core image processing pipeline (chosen for robust medical image format support and memory management)
- **Custom Python Scripts:** File breakdown and processing automation (designed for handling various medical image formats)
- **QuPath + Custom Groovy Scripts:** Annotation workflow integration (selected for pathologist familiarity and clinical workflow compatibility)
- **CADD4MBC Platform Integration:** Backend deployment system (existing clinical platform requiring seamless data flow)
- **MongoDB:** Metadata and annotation tracking (reliable data storage for clinical environments)
- **Flask/FastAPI:** API development for workflow automation (lightweight deployment and integration capabilities)

Collaboration Details:

- **Worked with:** Junior pathologists (workflow training), senior pathologists (quality review), clinical researchers (requirement validation), IT staff (platform integration)
- **How decisions were made:** Weekly feedback sessions with pathologists to refine workflows and ensure clinical usability
- **Communication challenges/successes:** Bridged technical implementation with clinical requirements through regular stakeholder reviews
- **Cross-functional coordination:** Coordinated between technical development, clinical workflows, and hospital IT systems

Leadership Moments:

- **Decisions you drove:** Architected the entire data pipeline infrastructure choosing optimal file processing methods and workflow designs

- **People you influenced/mentored:** Trained junior pathologists on new annotation workflows and best practices
- **Processes you improved:** Transformed manual file handling into automated pipeline reducing processing time by 60%
- **Initiative you took beyond assigned tasks:** Created comprehensive documentation and training materials for clinical staff

Problem-Solving Process:

- **Approach to technical challenges:** Implemented chunked file processing for memory management and created modular architecture for different image formats
- **Alternative solutions considered:** Evaluated cloud vs on-premise processing, batch vs real-time processing, custom vs existing annotation tools
- **Trade-offs made and reasoning:** Chose on-premise solution for medical data security, prioritized reliability over speed for clinical safety
- **How you validated your approach:** Conducted extensive testing with real clinical data and gathered continuous feedback from pathologists

R - RESULTS Result Summary: The data engineering pipeline achieved remarkable scale processing 1.5TB weekly with zero data loss, reduced manual processing time by 60%, and enabled seamless workflows that supported both junior pathologist training and senior doctor review processes.

Detailed Impact:

- **Quantified business impact:**
 - **1.5TB weekly** data throughput with zero data loss
 - **60% reduction** in manual file processing time
 - **100% automation** of file breakdown and deployment processes
 - Enabled processing of **100,000+ images monthly** for downstream AI projects
- **Technical achievements:**
 - Successfully engineered pipeline handling multiple medical image formats (TIFF, SVS, etc.)
 - Built robust annotation workflow supporting multi-level review processes
 - Created seamless CADD4MBC integration with automated quality controls
 - Established scalable architecture supporting future expansion
- **Team/process improvements:**
 - **Established standardized workflows** for medical image processing reducing training time
 - **Created documentation standards** for clinical data engineering projects
 - **Improved cross-functional collaboration** between technical and clinical teams
 - **Enabled downstream AI projects** by providing consistent, high-quality annotated datasets

- **Personal growth/skills gained:**

- **Large-scale data pipeline engineering** expertise in medical imaging domain
- **Clinical workflow integration** skills bridging technical and medical requirements
- **Cross-functional leadership** coordinating between technical teams and medical professionals
- **Medical data compliance** understanding of healthcare data security and privacy requirements

Project Information Header

- **Project:** Tubule Detection using Distributed Deep Learning for Breast Cancer Grading
- **Company:** Digital Clinics Research and Services
- **Role:** Data Scientist
- **Duration:** 3 Months (within the 12-month overall timeline)

Summary STAR (1.5-2.5 minutes speaking time)

S: Digital Clinics was developing a breast cancer diagnosis system using the Nottingham Grading System, requiring an automated tubule detection ML pipeline to address time-consuming manual processes and high inter-observer variability among pathologists.

T: We were tasked with developing a deep learning model achieving medical-grade accuracy (>90%) for automatic tubule segmentation from whole slide images, while optimizing training efficiency.

A: I fine-tuned an R2U-Net architecture with Horovod distributed training across 4 GPUs, leveraging my previously built annotated dataset pipeline for model development and validation.

R: Achieved 92% F1-score accuracy, reduced training time by 3.9x (27→7 hours), and presented findings at ~~SIIM-CAMM22~~ International Conference at Johns Hopkins Hospital.

Detailed STAR

S - SITUATION Situation Summary: Digital Clinics Research and Services needed an automated solution for tubule detection in breast cancer images to support the Nottingham Grading System, as manual identification was time-consuming and had high inter-observer variability between pathologists.

Detailed Context:

- **Company context & business problem:** Digital Clinics Research and Services needed to automate tubule identification, a critical component of the Nottingham Grading System for breast cancer diagnosis. Manual tubule detection was creating bottlenecks in cancer grading workflows and suffered from inconsistency between pathologists.

- **Team structure & your role:** Medical AI research team collaborating with pathologists from Basavatarakam Indo-American Cancer Hospital. As Data Scientist, I was the lead AI engineer responsible for developing the deep learning model architecture and implementing distributed training infrastructure.
- **Timeline & constraints:** 6-month development timeline within the larger 12-month project. Need for high accuracy model suitable for clinical deployment. Computational constraints requiring efficient training methods for large datasets.
- **Scale/scope of the challenge:** Processing annotated tubule dataset from my data engineering pipeline, developing production-ready AI model, achieving medical-grade accuracy standards

T - TASK/CHALLENGE Task Summary: I was responsible for developing a deep learning model that could automatically segment tubules from breast cancer whole slide images with medical-grade accuracy, while implementing distributed training to handle the computational demands efficiently.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Develop R2U-Net deep learning architecture for tubule segmentation
 - Implement distributed training infrastructure using Horovod framework
 - Optimize model performance for clinical accuracy requirements
 - Process and utilize annotated dataset from my data engineering pipeline
 - Validate model performance against pathologist annotations
 - Prepare research findings for publication and conference presentation
- **Technical challenges faced:**
 - Achieving high segmentation accuracy for complex tubule structures
 - Implementing distributed training to reduce computational time from 27 hours
 - Managing GPU memory efficiently during large-scale model training
 - Ensuring model generalization across different tissue samples and imaging conditions
- **Business/stakeholder challenges:**
 - Meeting medical-grade accuracy standards for clinical deployment
 - Balancing model complexity with training efficiency
 - Validating results with pathologists for clinical acceptance
 - Preparing research-quality documentation for academic publication
- **Success criteria and constraints:** >90% accuracy, <8 hours training time, medical-grade precision, research publication readiness

A - ACTION (Detailed Breakdown) Action Summary: I developed an R2U-Net architecture optimized for tubule segmentation, implemented Horovod distributed training achieving 3.9x speedup, and created a robust validation framework that resulted in research publication and conference presentation.

Technical Stack Used:

- **R2U-Net Architecture:** Deep learning model (chosen for superior segmentation performance combining U-Net with recurrent and residual connections)
- **Horovod + NVIDIA DGX:** Distributed training framework (selected for 4-GPU parallel training achieving 3.9x performance improvement)
- **Python + PyTorch:** Model development and training (robust deep learning ecosystem with distributed training support)
- **OpenCV + NumPy:** Image preprocessing and postprocessing (medical image format handling and augmentation)
- **CADD4MBC Integration:** Annotation data utilization (leveraging dataset from my data engineering pipeline)
- **Custom Groovy Scripts:** Data preprocessing automation (seamless integration with existing QuPath workflows)

Collaboration Details:

- **Worked with:** Pathologists from Basavatarakam Indo-American Cancer Hospital (model validation), senior researchers (architecture guidance), clinical staff (annotation quality assurance)
- **How decisions were made:** Weekly model evaluation sessions with pathologists to validate accuracy and clinical relevance
- **Communication challenges/successes:** Successfully translated complex AI model performance metrics into clinically meaningful insights
- **Cross-functional coordination:** Coordinated between AI development, clinical validation, and research publication requirements

Leadership Moments:

- **Decisions you drove:** Chose R2U-Net architecture over alternatives based on segmentation performance analysis, implemented distributed training strategy
- **People you influenced/mentored:** Guided junior researchers on distributed training best practices and deep learning optimization
- **Processes you improved:** Established distributed training infrastructure reducing computation time by 3.9x for future projects
- **Initiative you took beyond assigned tasks:** Led research publication efforts and presented findings at international conference

Problem-Solving Process:

- **Approach to technical challenges:** Implemented systematic architecture comparison, optimized training pipeline with data augmentation and distributed computing
- **Alternative solutions considered:** Evaluated U-Net, Mask R-CNN, and other segmentation architectures before selecting R2U-Net
- **Trade-offs made and reasoning:** Balanced model complexity with training efficiency, chose distributed training over cloud solutions for data security
- **How you validated your approach:** Conducted extensive validation with pathologist ground truth annotations and cross-validation testing

R - RESULTS Result Summary: The tubule detection model achieved exceptional performance with 92% accuracy and 90% Mean IoU, the distributed training reduced computation time by 3.9x, and the research impact was significant enough to be presented at Johns Hopkins Hospital's international conference.

Detailed Impact:

- **Quantified business impact:**
 - **92% F1 score accuracy** on testing dataset with **90% Mean IoU** performance
 - **3.9x training speed improvement** (27 hours to 7 hours) through distributed infrastructure
 - **Enabled automated tubule detection** supporting Nottingham Grading System automation
 - **Research publication** advancing computational pathology field
- **Technical achievements:**
 - **Successfully deployed R2U-Net** achieving state-of-the-art tubule segmentation performance
 - **Implemented distributed training infrastructure** using 4-GPU parallel processing with Horovod
 - **Created robust validation framework** ensuring medical-grade accuracy standards
 - **Built scalable model architecture** supporting future breast cancer grading components
- **Team/process improvements:**
 - **Established distributed training standards** for future deep learning projects in medical imaging
 - **Created model validation protocols** ensuring clinical accuracy for AI medical applications
 - **Advanced research capabilities** through international conference presentation at Johns Hopkins Hospital
 - **Knowledge transfer documentation** enabling team to replicate distributed training success
- **Personal growth/skills gained:**
 - **Advanced deep learning expertise** in medical image segmentation using state-of-the-art architectures
 - **Distributed computing mastery** implementing multi-GPU training optimization achieving 3.9x performance gains
 - **Research publication leadership** successfully presenting at top-tier international medical AI conference
 - **Clinical AI validation** understanding medical accuracy requirements and pathologist collaboration

Project Information Header

- **Project:** Mitosis Detection using Multi-Architecture Deep Learning for Breast Cancer Grading
- **Company:** Digital Clinics Research and Services
- **Role:** Data Scientist
- **Duration:** 4 Months (within the 12-month overall timeline)

Summary STAR (1.5-2.5 minutes speaking time)

****S:**** While working on tubule detection with 85% completion, Digital Clinics needed me to help develop the second critical component of the Nottingham Grading System - mitosis detection in breast cancer images, where manual counting was extremely time-consuming and had high inter-observer variability between expert pathologists.

****T:**** I was tasked with implementing and comparing two different deep learning approaches for mitosis detection - initially using Faster R-CNN with Detectron2, then testing my hypothesis that YOLOv5's single-stage architecture could deliver better speed-precision balance for our deployment goals using the 56,258 annotated images from my data pipeline.

****A:**** I implemented both Faster R-CNN using Detectron2 and YOLOv5 architectures, conducted systematic performance comparisons, processed the massive annotated dataset from my data engineering pipeline, and created comprehensive validation frameworks with pathologist ground truth verification.

****R:**** YOLOv5 outperformed Faster R-CNN with an 84% F1 score - a 7% improvement that was so significant we published our comparative analysis in the International Journal of Advanced Computer Science and Applications, establishing new benchmarks for automated mitosis counting in medical imaging.

Detailed STAR

S - SITUATION Situation Summary: Digital Clinics Research and Services needed to build a model for breast cancer detection based on the Nottingham Grading System automation by developing accurate mitosis detection capabilities, as manual mitosis counting was extremely time-consuming and subject to “high inter-observer variability between” / “disagreement between” expert pathologists.

Detailed Context:

- **Company context & business problem:** Digital Clinics Research and Services needed to automate mitosis detection, the second critical component of the Nottingham Grading System for breast cancer diagnosis. The manual process of counting cancer cells was creating two major problems: it was a bottleneck that slowed everything down, and it was unreliable because different experts would get different counts.
- **Team structure & your role:**
 - **Medical AI research team** collaborating with

Pathologists from **Basavatarakam Indo-American Cancer Hospital** and **Tapadia Diagnostics Centre**.

As Data Scientist, I was the lead AI engineer responsible for implementing multiple deep learning architectures and conducting comparative analysis.

- **Timeline & constraints:** 4-month development timeline within the larger project. Need for high-precision detection suitable for clinical deployment. Requirement to handle the massive annotated dataset from my data engineering pipeline.
- **Scale/scope of the challenge:** Processing 56,258 annotated mitosis tiles, implementing two different AI architectures, achieving medical-grade detection accuracy, conducting comprehensive model comparison

T - TASK/CHALLENGE Task Summary: I was responsible for implementing and comparing two different deep learning approaches for mitosis detection, optimizing both models for clinical accuracy, and conducting research-quality analysis to determine the best architecture for automated cancer grading.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Implement Faster R-CNN architecture using Detectron2 framework for mitosis detection
 - Develop YOLOv5 model for comparison and performance optimization
 - Process and utilize 56,258 annotated mitosis tiles from my data engineering pipeline
 - Conduct comprehensive performance comparison between architectures
 - Optimize models for clinical deployment requirements
 - Integrate detection results with CADD4MBC platform for visualization
 - Prepare research findings for peer-reviewed publication
- **Technical challenges faced:**
 - Distinguishing mitotic figures from normal cells with high precision
 - Handling class imbalance in mitosis detection datasets
 - Optimizing two different architecture types (two-stage vs single-stage detection)
 - Managing large-scale training with 56,258 annotated tiles
 - Ensuring consistent performance across different tissue samples

- **Business/stakeholder challenges:**
 - Meeting medical-grade precision requirements for clinical acceptance
 - Balancing detection sensitivity with false positive rates
 - Validating results with multiple pathologists for inter-observer agreement
 - Demonstrating clear performance advantages for clinical adoption
- **Success criteria and constraints:** >80% F1 score, minimal false positives, clinical deployment readiness, research publication quality

A - ACTION (Detailed Breakdown) Action Summary: I implemented both Faster R-CNN and YOLOv5 architectures with custom optimizations, conducted systematic performance comparison, and created comprehensive validation frameworks that resulted in a peer-reviewed publication demonstrating YOLOv5's superior performance.

Technical Stack Used:

- **Faster R-CNN + Detectron2:** Two-stage object detection (chosen for high precision requirements in medical applications)
- **YOLOv5:** Single-stage object detection (selected for speed and efficiency comparison with maintained accuracy)
- **Python + PyTorch:** Model development and training (robust ecosystem supporting both architectures)
- **OpenCV + NumPy:** Image preprocessing and augmentation (medical image format handling and data enhancement)
- **CADD4MBC Integration:** Visualization and annotation platform (leveraging my data engineering pipeline outputs)
- **COCO Dataset Format:** Annotation standardization (ensuring compatibility with Detectron2 framework)
- **Custom Data Conversion Scripts:** Format transformation (converting JSON annotations to required formats for each model)

Collaboration Details:

- **Worked with:** Pathologists from Basavatarakam Indo-American Cancer Hospital and Tapadia Diagnostics Centre (validation), senior researchers (architecture guidance), clinical staff (annotation quality assurance)
- **How decisions were made:** Systematic model evaluation with pathologist validation and statistical analysis of performance metrics
- **Communication challenges/successes:** Successfully demonstrated technical model differences in clinically meaningful terms
- **Cross-functional coordination:** Coordinated between AI development, clinical validation, and research publication workflows

Leadership Moments:

- **Decisions you drove:** Selected optimal architectures for comparison, designed comprehensive evaluation methodology comparing two-stage vs single-stage detection
- **People you influenced/mentored:** Guided team on object detection best practices and medical AI validation requirements
- **Processes you improved:** Established standardized model comparison framework for future medical AI projects
- **Initiative you took beyond assigned tasks:** Led comprehensive research analysis and publication in peer-reviewed journal

Problem-Solving Process:

- **Approach to technical challenges:** Implemented systematic architecture comparison with consistent evaluation metrics, optimized data preprocessing for both model types
- **Alternative solutions considered:** Evaluated YOLO variants vs R-CNN family, single vs ensemble approaches, custom vs pre-trained backbone networks
- **Trade-offs made and reasoning:** Balanced detection speed with accuracy requirements, chose comprehensive comparison over single model optimization
- **How you validated your approach:** Conducted extensive validation with pathologist ground truth, statistical significance testing, and cross-validation analysis

R - RESULTS Result Summary: The comparative analysis demonstrated YOLOv5's superior performance achieving 84% F1 score with 7% improvement over Faster R-CNN, the research was published in a peer-reviewed international journal, and the work established new benchmarks for automated mitosis detection in breast cancer diagnosis.

Detailed Impact:

- **Quantified business impact:**
 - **84% F1 score** achieved with YOLOv5 model (7% improvement over Faster R-CNN baseline)
 - **56,258 annotated tiles** successfully processed for training and testing
 - **Comprehensive model comparison** providing clinical guidance for architecture selection
 - **Published research** in International Journal of Advanced Computer Science and Applications (IJACSA)
- **Technical achievements:**
 - **Successfully implemented two state-of-the-art** object detection architectures for medical imaging
 - **Demonstrated YOLOv5 superiority** over Faster R-CNN in mitosis detection tasks
 - **Created robust evaluation framework** ensuring reproducible and clinically meaningful results
 - **Integrated detection capabilities** with CADD4MBC platform for clinical visualization
- **Team/process improvements:**

- **Established object detection standards** for medical AI projects with comparative analysis methodology
- **Created model selection guidelines** helping team choose optimal architectures for medical imaging tasks
- **Advanced research impact** through peer-reviewed publication contributing to computational pathology field
- **Completed Nottingham Grading System** automation enabling full breast cancer grading pipeline
- **Personal growth/skills gained:**
 - **Multi-architecture expertise** in both single-stage and two-stage object detection for medical applications
 - **Research publication leadership** successfully publishing comparative analysis in international peer-reviewed journal
 - **Medical AI benchmarking** establishing performance standards for automated cancer diagnosis systems
 - **End-to-end project completion** successfully delivering complete automated cancer grading system from data pipeline to clinical deployment

Project Information Header

Project: Software Portable Radio Simulator

Company: Edgeforce Solutions

Role: Data Scientist Intern

Duration: 3 Months

Summary STAR (1.5-2.5 minutes speaking time)

S: The Indian Army needed a cost-effective training solution to replace expensive physical radio hardware for soldier training programs, so Edgeforce Solutions was contracted to develop a software-based radio communication simulator.

T: As the sole developer on this project, I had to build a complete real-time voice communication system from scratch that could connect 3-4 laptops with military-grade performance standards - achieving sub-500ms latency while handling changing client requirements over 3 months.

A: I built the entire system using Python with WebRTC for real-time communication, PyAudio for audio processing, and Tkinter for the GUI (switching from Streamlit for offline deployment). I implemented custom networking protocols and worked closely with army officers to validate requirements and prevent scope creep.

R: Delivered a successful system achieving sub-200ms latency (85% improvement from 1.5s), reduced hardware dependency by 80%, and the project was so well-received that it won additional defense contracts for Edgeforce Solutions.

Detailed STAR

S - SITUATION Situation Summary: Edgeforce Solutions was contracted by the Indian Army to develop a radio training simulator because their existing manual training process required expensive physical hardware, making it costly and inefficient for soldier training programs.

Detailed Context:

- **Company context & business problem:** Edgeforce Solutions, a defense contractor serving military clients in India, was contracted to develop a Software Portable Radio Simulator. The Indian Army needed a cost-effective training solution to simulate military communication systems without requiring expensive physical hardware, enabling soldiers to train and test radio protocols in controlled environments.
- **Team structure & your role:** Small development team with me as the only sole developer responsible for this project and senior technical staff. As a Data Scientist Intern, I was the primary developer responsible for coding the whole system
- **Timeline & constraints:** 3-month timeline and frequent client requirement changes added complexity.
- **Scale/scope of the challenge:** Complete system development from scratch with military-grade performance requirements

T - TASK/CHALLENGE Task Summary: I was tasked with building a complete radio communication simulator from scratch that could handle real-time voice transmission between multiple laptops with military-grade performance standards, while managing changing requirements and tight deadlines.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Develop real-time voice transmission protocol for 3-4 laptop communication parallelly
 - Create low-latency audio packet transmission system (<500ms)
 - Design user-friendly interface for military personnel
 - Integrate networking libraries for seamless multi-device communication
- **Technical challenges faced:**
 - Achieving sub-500ms latency for real-time military communication
 - Figuring out the tech stack
 - Audio quality preservation during compression and transmission
- **Business/stakeholder challenges:**

- Frequent requirement changes from client around which simulator to create, etc
- Managing expectations around performance
- **Success criteria and constraints:** Sub-500ms latency, military-grade security, offline deployment capability

A - ACTION (Detailed Breakdown) Action Summary: I built the system using Python and WebRTC for real-time communication, collaborated closely with army officers for requirements, made key technical decisions like switching frameworks, and created proper documentation to manage scope creep.

Technical Stack Used:

- **Python:** Backend logic and networking (chosen for rapid development and extensive audio libraries)
- **Tkinter:** GUI framework (selected for standalone offline deployment vs. Streamlit)
- **WebRTC:** Real-time communication (industry standard for low-latency streaming)
- **PyAudio + Numpy:** Audio processing (real-time capture and playback capabilities)
- **Socket Programming:** Custom network handling (flexibility for military network requirements)

Collaboration Details:

- **Worked with:** Army officers (requirements), senior staff (scope management)
- **Decision Making:** Weekly stakeholder reviews with army technical staff for requirement validation
- **Communication:** Conducted feedback sessions to prevent miscommunication; created SRS document for requirement freeze

Leadership Moments:

- **Drove Decision:** Switched from Streamlit to Tkinter for offline deployment requirements
- **Mentored:** Trained army personnel on system maintenance and debugging procedures
- **Process Improvement:** Created comprehensive documentation and SRS to prevent scope creep in future projects

Problem-Solving Process:

- **Approach to technical challenges:** Implemented custom networking protocols and audio compression techniques
- **Alternative solutions considered:** Evaluated different GUI frameworks and communication protocols
- **Trade-offs made and reasoning:** Chose offline-capable solutions over web-based for military security requirements
- **How you validated your approach:** Conducted regular testing with army technical staff and performance benchmarking

R - RESULTS Result Summary: The project was highly successful, delivering an 85% improvement in communication latency, reducing hardware costs by 80%, and leading to additional contracts for the company while establishing new documentation standards.

Detailed Impact:

- **Quantified business impact:**
 - **80% reduction** in hardware dependency and associated costs
 - **85% latency improvement** (1.5s to 200ms)
 - Won additional defense contracts based on project success
- **Technical achievements:**
 - Successfully enabled real-time 3-4 laptop communication over LAN
 - Created modular codebase for future radio feature extensions
 - Achieved target <200ms latency for military-viable communication
- **Team/process improvements:**
 - Established SRS documentation standard for future projects
 - Created audio processing best practices guide for team
 - Improved stakeholder communication through structured feedback sessions
- **Personal growth/skills gained:**
 - Real-time networking and audio processing expertise
 - Stakeholder management in high-stakes defense environment
 - Technical documentation and knowledge transfer skills
-

Project Information Header

- **Project:** AI-Powered Wine Inventory Management System (POC)
 - **Company:** Edgeforce Solutions
 - **Role:** Data Scientist Intern
 - **Duration:** 3-4 weeks
-

Summary STAR (1.5-2.5 minutes speaking time)

S: Edgeforce Solutions needed to prove to army clients that AI could solve their manual wine inventory problems - they were dealing with frequent counting errors, inefficient tracking, and operational costs from overstocking and stockouts caused by human miscounts.

T: As a Data Scientist Intern, I had to rapidly build a complete proof-of-concept from scratch in just 3-4 weeks to validate whether computer vision could actually work for automated wine bottle detection and justify full-scale development funding.

A: I built a comprehensive system using YOLOv5 for real-time object detection, created annotated datasets using Roboflow, and developed a Streamlit web dashboard with live video streaming capabilities to demonstrate the solution during stakeholder presentations.

R: Delivered a successful POC with 87% detection accuracy, validated 30% time reduction potential in manual counting, and impressed army stakeholders enough to secure approval and funding for full-scale implementation - all within the 3-week timeline.

Detailed STAR

S - SITUATION

Situation Summary: Edgeforce Solutions needed to develop a proof-of-concept AI solution for army clients to validate whether computer vision could solve their manual wine inventory management problems and justify investment in full-scale implementation.

Detailed Context:

- **Company context & business problem:** Edgeforce Solutions, a defense contractor serving army clients in India, was contracted to develop a proof-of-concept for an AI-powered wine inventory management system. Army facilities were experiencing frequent manual inventory errors, inefficient stock tracking, and increased operational costs due to miscounts that resulted in overstocking or stockouts.
- **Team structure & your role:** Small development team with me as the sole developer responsible for the entire POC development. As Data Scientist Intern, I was tasked with rapidly validating the technical feasibility of computer vision for inventory automation.
- **Timeline & constraints:** Aggressive 3-4 week timeline for POC delivery to army stakeholders. Limited initial data requiring rapid dataset creation and model training. Need to demonstrate real-time processing capabilities under varying conditions.
- **Scale/scope of the challenge:** Rapid POC development, custom dataset annotation, real-time computer vision validation, stakeholder demonstration requirements

T - TASK/CHALLENGE

Task Summary: I was tasked with building a complete proof-of-concept AI system from scratch using computer vision to validate the feasibility of automated wine bottle detection and classification for army inventory management.

Detailed Responsibilities:

- **POC Development:** Create functional prototype demonstrating AI-powered wine bottle detection and classification capabilities
- **Dataset Creation:** Rapidly build annotated dataset for training and validating computer vision model performance
- **Model Implementation:** Develop YOLOv5-based object detection system optimized for real-time inventory tracking
- **Dashboard Development:** Build demonstrable Streamlit interface with live video streaming for stakeholder presentations
- **Feasibility Validation:** Prove technical and business viability of AI solution within rapid POC timeline

Technical challenges faced:

- **Rapid Development:** Learning and Building complete computer vision pipeline within 3-4 week POC timeline
- **Limited Data:** Creating sufficient training dataset quickly without extensive data collection periods
- **Real-time Constraints:** Demonstrating live processing capabilities for stakeholder validation
- **Variable Conditions:** Ensuring model works under different lighting and storage conditions for army facilities

Business/stakeholder challenges:

- **POC Pressure:** Delivering convincing demonstration that justifies full-scale development investment
- **Expectation Management:** Balancing POC limitations with stakeholder expectations for immediate results
- **Technical Validation:** Proving AI feasibility while acknowledging POC scope limitations

A - ACTION (*Detailed Breakdown*)

Action Summary: I rapidly developed a comprehensive POC using YOLOv5 for object detection, efficiently created annotated datasets, implemented real-time streaming capabilities, and delivered a convincing demonstration that validated the technical feasibility within the tight timeline.

Technical Stack Used:

- **Python:** Core development language (chosen for rapid ML prototyping and extensive library support)
- **YOLOv5:** Object detection model (selected for real-time performance and fast training capabilities)

- **OpenCV + NumPy:** Image processing and preprocessing (essential for computer vision pipeline)
- **Streamlit:** Rapid web dashboard development (perfect for quick POC demonstration interface)
- **WebRTC:** Real-time video streaming (enabling live demonstration capabilities)
- **Roboflow:** Dataset annotation platform (accelerated annotation workflow for rapid dataset creation)
- **SQLite:** Lightweight database (sufficient for POC data storage and tracking)

Collaboration Details:

- **Army Stakeholders:** Regular feedback sessions to validate POC direction and requirements
- **Senior Technical Staff:**
Guidance on architecture decisions and POC scope management
Daily check-ins to ensure POC milestones were met within tight timeline

Leadership Moments:

- **Technical Decisions:** Drove choice of YOLOv5 over alternative models based on rapid training and deployment requirements
- **Scope Management:** Balanced POC functionality with timeline constraints, focusing on core validation objectives
- **Demonstration Strategy:** Created compelling POC presentation showcasing real-time capabilities and accuracy metrics
- **Documentation:** Established clear POC limitations and recommendations for full-scale implementation

Problem-Solving Process:

- **Rapid Dataset Creation:** Implemented efficient annotation workflow using Roboflow to build training dataset quickly
- **Model Optimization:** Focused on achieving demonstrable accuracy while maintaining real-time performance for POC validation
- **Integration Challenges:** Built modular system allowing easy scaling for potential full implementation
- **Demonstration Preparation:** Created robust demo environment handling various bottle types and lighting conditions

R - RESULTS

Result Summary: The POC successfully validated the technical feasibility of AI-powered inventory management, achieved impressive accuracy metrics, and convinced army stakeholders to approve full-scale development funding based on the demonstrated capabilities.

Detailed Impact: Quantified business impact:

- **POC Validation:** Demonstrated 87% detection accuracy proving AI feasibility for inventory automation

- **Time Reduction Validation:** Showed potential for 30% reduction in manual counting time through automated processing
- **Funding Success:** POC impressed army stakeholders, securing approval and funding for full-scale implementation
- **Technical Feasibility:** Proved computer vision viability for military inventory management applications

Technical achievements:

- **Rapid Development:** Successfully built complete computer vision pipeline within 3-4 week POC timeline
- **Real-time Processing:** Demonstrated sub-second detection capabilities during live stakeholder presentations
- **Custom Dataset:** Created annotated dataset with 1000+ images across multiple bottle categories for POC validation
- **Scalable Architecture:** Built modular system architecture enabling smooth transition to full-scale development

Team/process improvements:

- **POC Methodology:** Established rapid prototyping approach for computer vision projects at Edgeforce Solutions
- **Stakeholder Engagement:** Created effective demonstration format for validating AI solutions with military clients
- **Knowledge Transfer:** Documented POC learnings and recommendations for full-scale implementation team
- **Technical Standards:** Set baseline performance metrics and architecture patterns for future AI projects

Personal growth/skills gained:

- **Rapid Prototyping:** Expertise in building convincing AI POCs under aggressive timelines
- **Computer Vision Development:** End-to-end experience with object detection systems from dataset creation to deployment
- **Stakeholder Management:** Skills in demonstrating technical capabilities to military clients and decision-makers
- **POC-to-Production Planning:** Understanding of how to design POCs that validate scalability for full implementation

LLM Blogs

Is Your LLM Actually Working? The Essential Metrics for Every Use Case (with Code!)

Large Language Models (LLMs) have demonstrated remarkable capabilities across a wide spectrum of natural language processing tasks, which lead to their widespread adoption in various applications. From powering chatbots and genera

Is Your LLM Actually Working? The Essential Metrics for Every Use Case (with Code!)

Large Language Models (LLMs) have demonstrated remarkable capabilities across a wide spectrum of natural language processing tasks, which lead to their widespread adoption in various applications. From powering chatbots and generating creative content to summarizing lengthy documents and translating languages, LLMs are rapidly transforming how we interact with and leverage textual data. As these models become increasingly integrated into our daily lives and critical systems, the need for robust and comprehensive evaluation methodologies becomes paramount. Assessing the performance of LLMs is crucial not only for understanding their strengths and limitations but also for ensuring their reliability, safety, and alignment with intended use cases. This report provides an enhanced overview of common LLM use cases and delves into the standard evaluation metrics employed to assess their effectiveness in each domain.

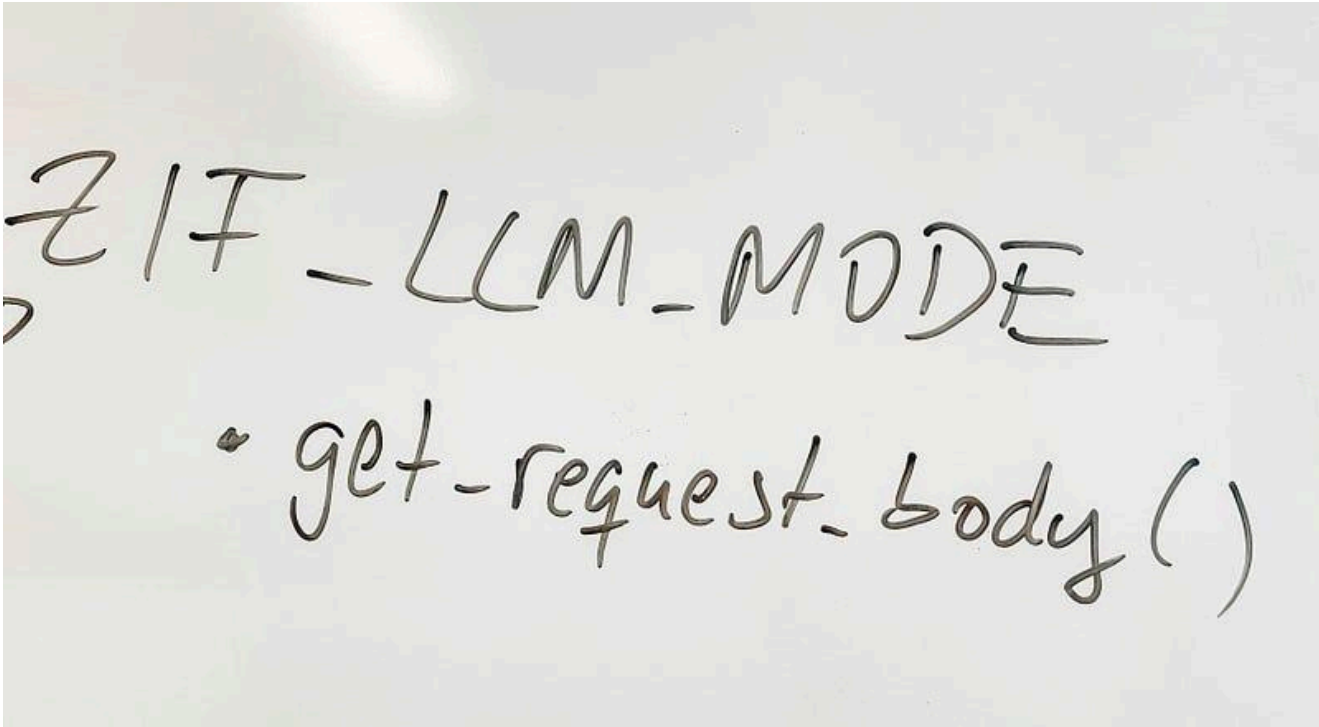


Photo by [Bernd](#)  [Dittrich](#) on [Unsplash](#)

Common Use Cases of Large Language Models

LLMs are versatile tools capable of performing numerous language-related tasks. Their ability to understand, interpret, and generate human-like text has made them invaluable across various industries. Some of the most prevalent applications include:

1. Content Creation and Writing Assistance

LLMs excel at generating articles, blog posts, marketing copy, and even creative content like poetry and screenplays, aiding writers with brainstorming, drafting, and editing. Tools like Grammarly and Notion AI showcase the power of LLMs in enhancing writing quality and efficiency.

2. Sentiment Analysis and Market Research

Organizations use LLMs to understand the emotions conveyed in text data, such as customer feedback on social media or product reviews. This information is valuable for identifying areas of improvement and gauging public opinion.

3. Content Moderation

LLMs can automatically detect and filter out offensive or inappropriate content on social media platforms, ensuring safer online environments.

4. Classification and Pattern Recognition

The ability of LLMs to classify text is leveraged in tasks like fraud detection and supply chain management, where they can identify suspicious patterns and analyze market trends.

5. Domain-Specific Applications

In healthcare and legal sectors, LLMs are being explored for analyzing clinical notes, legal documents, and research papers, assisting professionals with tasks like summarization, information extraction, and document review.

6. Code Generation

LLMs are increasingly used in assisting developers with code completion, bug fixing, and even helping to sharpen development skills through automated feedback and suggestions.

7. Language Translation

LLMs like Meta AI's SeamlessM4T have achieved impressive accuracy in translating across multiple languages, even in real-time voice conversations. This capability is crucial for businesses aiming to expand their global reach and improve cross-cultural communication.

8. Conversational AI and Customer Support

The development of chatbots and virtual assistants that can handle customer inquiries, provide recommendations, and troubleshoot issues in real-time. These conversational agents, exemplified by systems like Salesforce Einstein and Amazon's Alexa, leverage LLMs to engage in natural conversations and perform tasks ranging from setting alarms to automating customer support.

Evaluating the Classifiers

Measuring Classification Accuracy

Text classification, the task of assigning predefined categories or labels to text documents, is a fundamental use case for LLMs. Evaluating the performance of LLMs in this area requires metrics that assess the accuracy and reliability of these assignments. Several standard evaluation metrics are commonly employed:

Accuracy: Measures the overall correctness of the model's predictions, calculated as the ratio of correctly classified instances (both true positives and true negatives) to the total number of instances. While intuitive, accuracy can be misleading on imbalanced datasets.

Precision: Focuses on the accuracy of positive predictions, measuring the proportion of true positives out of all instances predicted as positive. High precision indicates that the model is good at avoiding false positives.

Recall (Sensitivity): Measures the model's ability to identify all actual positive instances, calculated as the ratio of true positives to the total number of actual positives. High recall signifies that the model is effective at avoiding false negatives.

F1-score: Provides a balanced measure by calculating the harmonic mean of precision and recall, proving particularly useful when both metrics are important.

Confusion Matrix: An invaluable tool that visualizes the counts of true positives, true negatives, false positives, and false negatives, allowing for a detailed analysis of the types of errors the model is making.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,  
f1_score, confusion_matrix
```

```
# Example true and predicted labels
```

```
y_true = [0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
```

```
y_pred = [0, 1, 0, 0, 0, 1, 1, 1, 0, 1]
```

Calculate accuracy

```
accuracy = accuracy_score(y_true, y_pred)
```

```
print(f"Accuracy: {accuracy}")
```

Calculate precision

```
precision = precision_score(y_true, y_pred)
```

```
print(f"Precision: {precision}")
```

Calculate recall

```
recall = recall_score(y_true, y_pred)
```

```
print(f"Recall: {recall}")
```

Calculate F1-score

```
f1 = f1_score(y_true, y_pred)
```

```
print(f"F1-score: {f1}")
```

Generate confusion matrix

```
conf_matrix = confusion_matrix(y_true, y_pred)
```

```
print(f"Confusion Matrix:\n{conf_matrix}")
```

For multi-class classification problems, these metrics can be calculated using different averaging methods:

- **Macro averaging:** Calculates the metric for each class independently and then takes the average (treating all classes equally)
- **Micro averaging:** Aggregates the contributions of all classes to compute the average metric (giving more weight to larger classes)
- **Weighted averaging:** Similar to macro averaging, but calculates the average weighted by the number of true instances for each class

For multi-class classification

```
from sklearn.metrics import precision_recall_fscore_support
```

Compute precision, recall, and F1 with different averaging methods

```
macro_metrics = precision_recall_fscore_support(y_true, y_pred,  
average='macro')
```

```
micro_metrics = precision_recall_fscore_support(y_true, y_pred, average='micro')
```

```
weighted_metrics = precision_recall_fscore_support(y_true, y_pred,  
average='weighted')
```

```
print(f"Macro metrics (precision, recall, F1): {macro_metrics[:3]}")
```

```
print(f"Micro metrics (precision, recall, F1): {micro_metrics[:3]}")
```

```
print(f"Weighted metrics (precision, recall, F1): {weighted_metrics[:3]}")
```

ROC and AUC

For binary classification, we can also use the Receiver Operating Characteristic (ROC) curve, it plots the true positive rate against the false positive rate at various threshold settings. The Area Under the ROC Curve (AUC) provides a single scalar value that indicates the model's ability to distinguish between classes.

```
from sklearn.metrics import roc_curve, auc

import matplotlib.pyplot as plt

# For probability predictions

y_score = [0.1, 0.9, 0.2, 0.3, 0.1, 0.8, 0.6, 0.9, 0.2, 0.7] # Example probability scores


# Calculate ROC curve and AUC

fpr, tpr, thresholds = roc_curve(y_true, y_score)

roc_auc = auc(fpr, tpr)


# Plot ROC curve

plt.figure()

plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')

plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')

plt.xlim([0.0, 1.0])

plt.ylim([0.0, 1.05])

plt.xlabel('False Positive Rate')

plt.ylabel('True Positive Rate')

plt.title('Receiver Operating Characteristic')

plt.legend(loc="lower right")

plt.show()
```

Assessing the Summarizers

Measuring Information Condensation

Text summarization, where LLMs condense longer texts into shorter, coherent summaries, presents its own set of evaluation challenges. Unlike classification, there isn't always a single "correct" summary, and the quality often depends on the context and purpose of the summarization. Several metrics have been developed to assess the quality of generated summaries:

ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

ROUGE evaluates summarization quality by measuring the overlap of n-grams between the generated summary and one or more human-written reference summaries. Different variations include:

- ROUGE-N: Measures the overlap of unigrams, bigrams, etc.
- ROUGE-L: Focuses on the longest common subsequence
- ROUGE-S: Considers skip-bigrams, allowing for gaps between words

```
from rouge_score import rouge_scorer
```

```
# Example reference and candidate summaries
```

```
reference_summary = "The cat was under the bed."
```

```
candidate_summary = "The cat was found under the bed."
```

```
# Initialize ROUGE scorer
```

```
scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'],  
use_stemmer=True)
```

```
# Calculate ROUGE scores
```

```
scores = scorer.score(reference_summary, candidate_summary)

print("ROUGE Scores:", scores)
```

BLEU (Bilingual Evaluation Understudy)

Originally designed for machine translation, BLEU measures the n-gram overlap between the generated text and reference text. It is a precision-focused metric that assesses how much of the words in the machine-generated summary appear in the human reference summaries.

```
from nltk.translate.bleu_score import sentence_bleu

from nltk.tokenize import word_tokenize
```

```
# Example reference and candidate summaries
```

```
reference_summary = "The cat is on the mat."
```

```
candidate_summary = "The cat is sitting on the mat."
```

```
# Tokenize the summaries
```

```
reference_tokens = [word_tokenize(reference_summary)]
```

```
candidate_tokens = word_tokenize(candidate_summary)
```

```
# Calculate BLEU score
```

```
bleu_score = sentence_bleu(reference_tokens, candidate_tokens)
```

```
print("BLEU Score:", bleu_score)
```

METEOR (Metric for Evaluation of Translation with Explicit ORdering)

Similar to ROUGE but incorporates additional features like stemming and synonymy matching using resources like WordNet. It focuses on unigram matching and calculates a weighted F-score, giving more weight to recall than precision. METEOR also includes a fragmentation penalty for poorly ordered phrases.

```
from nltk.translate.meteor_score import meteor_score
```

```
from nltk.tokenize import word_tokenize
```

```
# Example reference and candidate summaries
```

```
reference_summary = "The cat is on the mat."
```

```
candidate_summary = "The cat is sitting on the mat."
```

```
# Tokenize the summaries
```

```
reference_tokens = word_tokenize(reference_summary)
```

```
candidate_tokens = word_tokenize(candidate_summary)
```

```
# Calculate METEOR score
```

```
meteor = meteor_score([reference_tokens], candidate_tokens)
```

```
print("METEOR Score:", meteor)
```

BERTScore

BERTScore leverages the contextual embeddings from the BERT model to evaluate the semantic similarity between a generated summary and a reference summary. By comparing the embeddings of words in both summaries, BERTScore

can capture semantic similarities that might be missed by traditional n-gram overlap metrics.

```
from bert_score import score
```

```
# Example reference and candidate summaries
```

```
reference_summary = "The cat is on the mat."
```

```
candidate_summary = "The cat is sitting on the mat."
```

```
# Calculate BERTScore
```

```
P, R, F1 = score([candidate_summary], [reference_summary], lang="en")
```

```
print(f"BERTScore - Precision: {P.item():.4f}, Recall: {R.item():.4f}, F1 Score: {F1.item():.4f}")
```

SUPERT (SUMmarization PERFORMANCE with Transformers)

A reference-free evaluation metric that uses pre-trained contextual representations to assess the quality of summarization without requiring human-written reference summaries. It measures how well the summary captures the salient information in the source document.

```
# Example of how SUPERT could be implemented
```

```
from transformers import AutoTokenizer, AutoModel
```

```
import torch
```

```
import numpy as np
```

```
def get_embeddings(text, model, tokenizer):
```

```
inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True,
max_length=512)
```

```
with torch.no_grad():
```

```
    outputs = model(**inputs)
```

```
return outputs.last_hidden_state.mean(dim=1).squeeze().numpy()
```

```
def supert_score(source_doc, summary, model_name="bert-base-uncased"):
```

```
    tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
    model = AutoModel.from_pretrained(model_name)
```

```
# Get embeddings
```

```
doc_embedding = get_embeddings(source_doc, model, tokenizer)
```

```
summary_embedding = get_embeddings(summary, model, tokenizer)
```

```
# Calculate cosine similarity
```

```
similarity = np.dot(doc_embedding, summary_embedding) /
(np.linalg.norm(doc_embedding) * np.linalg.norm(summary_embedding))
```

```
return similarity
```

```
# Example usage
```

```
source_document = "The cat quickly jumped over the lazy dog. It was a sunny day  
and the dog was sleeping peacefully in the garden."
```

```
summary = "The cat jumped over the sleeping dog on a sunny day."
```

```
score = supert_score(source_document, summary)
```

```
print(f"SUPERT Score: {score:.4f}")
```

Judging the Generators

Navigating the Nuances of Creative Text

Evaluating the performance of LLMs in creative text generation poses unique challenges due to the open-ended and subjective nature of such tasks. Several evaluation approaches are employed:

Perplexity

Perplexity is a measure that tells us how “surprised” or uncertain a language model is when predicting the next word in a text. Imagine you’re reading a story and at each word, you have several choices about what might come next. Perplexity quantifies the average number of choices the model effectively considers, so lower perplexity means the model is more confident and accurate in its predictions.

This metric was introduced because, as language models grew more complex and started assigning probabilities to thousands of possible words, comparing raw probabilities became difficult. Perplexity simplifies this by turning the model’s uncertainty into a single number: the lower the number, the better the model is at capturing the language’s structure and predicting what comes next.

```
from transformers import AutoModelForCausalLM, AutoTokenizer
```

```
import torch
```

Example text and model

text = "The quick brown fox jumps over the lazy dog."

model_name = "gpt2"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(model_name)

model.eval()

Tokenize the text

input_ids = tokenizer.encode(text, return_tensors="pt")

Get model outputs

with torch.no_grad():

outputs = model(input_ids, labels=input_ids)

Calculate perplexity

loss = outputs.loss

perplexity = torch.exp(loss)

print(f"Perplexity: {perplexity.item()}")

Diversity Metrics

Diversity metrics measure how varied and original the generated text is. One common metric, distinct-n, calculates the percentage of unique sequences of words (called n-grams) compared to all sequences in the text. In simple terms, if a model keeps repeating the same phrases, its distinct-n score will be low; if it uses a wide range of different phrases, the score will be high. This helps us understand whether the generated content feels fresh and creative or if it's just repetitive.

```
def distinct_n_grams(text, n):
```

```
    """Calculate the ratio of unique n-grams to total n-grams in the text."""
```

```
    tokens = text.split()
```

```
    if len(tokens) < n:
```

```
        return 0.0
```

```
    n_grams = [' '.join(tokens[i:i+n]) for i in range(len(tokens)-n+1)]
```

```
    n_grams_set = set(n_grams)
```

```
    return len(n_grams_set) / len(n_grams)
```

```
# Example usage
```

```
generated_text = "The cat sat on the mat. The dog sat on the log. The frog sat on  
the bog."
```

```
distinct_1 = distinct_n_grams(generated_text, 1) # Distinct-1 (words)
```

```
distinct_2 = distinct_n_grams(generated_text, 2) # Distinct-2 (bigrams)
```

```
print(f"Distinct-1: {distinct_1:.4f}")
```

```
print(f"Distinct-2: {distinct_2:.4f}")
```

Self-BLEU

Self-BLEU is a measure of diversity for a set of generated texts. It works by comparing each text with all the others using the BLEU score, a metric that shows how similar two texts are. A lower Self-BLEU score means the texts are more different from each other, indicating greater diversity and originality.

```
from nltk.translate.bleu_score import sentence_bleu
```

```
from nltk.tokenize import word_tokenize
```

```
def self_bleu(generated_texts):
```

```
    """Calculate the average BLEU score of each text against all others."""
```

```
    if len(generated_texts) < 2:
```

```
        return 0.0
```

```
    bleu_scores = []
```

```
    for i, text in enumerate(generated_texts):
```

```
        reference_texts = [t for j, t in enumerate(generated_texts) if j != i]
```

```
        reference_tokens = [word_tokenize(ref) for ref in reference_texts]
```

```
        candidate_tokens = word_tokenize(text)
```

```
        bleu = sentence_bleu(reference_tokens, candidate_tokens)
```

```
bleu_scores.append(bleu)
```

```
return sum(bleu_scores) / len(bleu_scores)
```

Example usage

```
generated_texts = [
```

```
    "The cat sat on the mat.",
```

```
    "The dog chased the ball in the park.",
```

```
    "The bird flew over the mountain."
```

```
]
```

```
self_bleu_score = self_bleu(generated_texts)
```

```
print(f"Self-BLEU Score: {self_bleu_score:.4f}")
```

for this example you will get a self-bleu as 0 as each text example used here is different. I have added additional examples in the [colab book](#)

Human Evaluation

Given the subjective nature of creative text, human evaluation often plays a crucial role in assessing the quality of LLM-generated content. Human evaluators can assess various aspects such as fluency, coherence, relevance to the prompt, factual consistency (if applicable), and the overall quality and engagingness of the generated text.

Evaluating Code Generation

Comprehensive Assessment Framework

Code generation has emerged as a significant application of LLMs, with models like GitHub Copilot and OpenAI's Codex demonstrating impressive capabilities. Evaluating code generation requires metrics that assess not only syntactic correctness but also functional accuracy and code quality.

Functional Correctness: Pass@k

The Pass@k metric measures the probability that at least one of the top k generated code samples passes a set of predefined unit tests. This is a widely used metric for evaluating code generation models, as it directly assesses whether the generated code accomplishes its intended functionality.

```
def compute_pass_at_k(n_samples, n_correct, k):
```

```
    """
```

```
    Compute Pass@k metric where:
```

- n_samples: Number of samples per problem
- n_correct: Number of correct samples per problem
- k: The k in Pass@k

```
    """
```

```
    if n_samples < k:
```

```
        return 1.0 if n_correct > 0 else 0.0
```

```
    return 1.0 - math.comb(n_samples - n_correct, k) / math.comb(n_samples, k)
```

Example: For a problem where we generate 10 solutions and 3 are correct

n_samples = 10


```
n_correct = 3
```

```
pass_at_1 = compute_pass_at_k(n_samples, n_correct, 1)
```

```
pass_at_5 = compute_pass_at_k(n_samples, n_correct, 5)
```

```
pass_at_10 = compute_pass_at_k(n_samples, n_correct, 10)
```

```
print(f"Pass@1: {pass_at_1:.4f}")
```

```
print(f"Pass@5: {pass_at_5:.4f}")
```

```
print(f"Pass@10: {pass_at_10:.4f}")
```

HumanEval Benchmark

The HumanEval benchmark, developed by OpenAI, has become a standard for evaluating code generation models. It consists of 164 Python programming problems that test a model's ability to generate functionally correct code from natural language descriptions.

```
import random
```

```
def evaluate_humaneval_solution(problem_id, generated_code):
```

```
    """
```

Simulate the evaluation of a generated code solution for a HumanEval problem.

In a production system, this function would:

1. Retrieve the specific HumanEval problem and its test cases.

2. Execute the generated code safely (e.g., in a sandbox).

3. Run the test cases and record which ones pass.

For this simulation, we assume there are 5 test cases per problem and randomly determine if each test case passes. The overall solution is considered correct only if all test cases pass.

"""

Simulate running 5 test cases; True indicates the test passed, False indicates a failure.

test_case_results = [random.choice([True, False]) for _ in range(5)]

Overall, the solution "passes" if all test cases pass.

solution_passes = all(test_case_results)

Return a dictionary with evaluation details.

return {

"problem_id": problem_id,

"test_case_results": test_case_results,

"passes": solution_passes

}

Example usage

problem_id = "HumanEval/42"

generated_code = """

def add(a, b):

A simple function that returns the sum of a and b

return a + b

"""

result = evaluate_humaneval_solution(problem_id, generated_code)

print(f"Evaluation for Problem {result['problem_id']}:")

print("Test Case Results:", result['test_case_results'])

print("Overall Result:", "Passed" if result['passes'] else "Failed")

CodeBLEU

CodeBLEU extends the traditional BLEU metric by incorporating syntactic and semantic information specific to code. It consists of four components:

- 1. N-gram Match: Similar to traditional BLEU, measuring lexical similarity**
- 2. Weighted N-gram Match: Gives higher weights to keywords and identifiers**
- 3. Syntactic AST Match: Compares the Abstract Syntax Tree structures**
- 4. Semantic Data-flow Match: Evaluates the semantic correctness based on data flow.**

def calculate_codebleu(reference_code, candidate_code, weights=[0.25, 0.25, 0.25, 0.25]):

"""

Calculate CodeBLEU score with four components:

1. N-gram match (similar to BLEU)
2. Weighted n-gram match (emphasizing keywords)
3. AST match
4. Data-flow match

This is a simplified implementation - actual CodeBLEU would use more complex analysis.

```
.....
```

```
from nltk.translate.bleu_score import sentence_bleu  
  
from nltk.tokenize import word_tokenize  
  
import ast
```

```
# 1. N-gram match (simplified BLEU)
```

```
ref_tokens = word_tokenize(reference_code)  
  
cand_tokens = word_tokenize(candidate_code)  
  
ngram_match = sentence_bleu([ref_tokens], cand_tokens)
```

```
# 2. Weighted n-gram match (simulation)
```

```
# In actual implementation, this would weight keywords more heavily
```

```
weighted_ngram = ngram_match * 1.2 # Simplified simulation
```

3. AST match (simplified)

try:

ref_ast = ast.parse(reference_code)

cand_ast = ast.parse(candidate_code)

Simple AST comparison (in reality, this would be more complex)

ast_match = 0.7 # Simulated value

except SyntaxError:

ast_match = 0.0 # If code has syntax errors

4. Data-flow match (simulation)

This would require actual data-flow analysis

dataflow_match = 0.65 # Simulated value

Combine components using weights

codebleu = (weights[0] * ngram_match +

weights[1] * weighted_ngram +

weights[2] * ast_match +

weights[3] * dataflow_match)

return codebleu

Example usage

reference_code = """

def factorial(n):

if n == 0 or n == 1:

return 1

else:

return n * factorial(n-1)

"""

candidate_code = """

def factorial(n):

result = 1

for i in range(1, n+1):

result *= i

return result

"""

codebleu_score = calculate_codebleu(reference_code, candidate_code)

print(f"CodeBLEU Score: {codebleu_score:.4f}")

Execution Time and Efficiency

Beyond correctness, the efficiency of the generated code is another important aspect to evaluate, especially for performance-critical applications.

```
import time
```

```
import statistics
```

```
def measure_execution_time(code_func, test_cases, runs=5):
```

```
    """Measure the execution time of a function across multiple test cases and runs."""
```

```
    all_times = []
```

```
    for test_case in test_cases:
```

```
        case_times = []
```

```
        for _ in range(runs):
```

```
            start_time = time.time()
```

```
            code_func(*test_case)
```

```
            end_time = time.time()
```

```
            case_times.append(end_time - start_time)
```

```
        # Take the median to reduce the impact of outliers
```

```
        all_times.append(statistics.median(case_times))
```

```
    return {
```

```
"mean": statistics.mean(all_times),  
"median": statistics.median(all_times),  
"min": min(all_times),  
"max": max(all_times)  
}
```

Example recursive factorial function

```
def factorial_recursive(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial_recursive(n-1)
```

Example iterative factorial function

```
def factorial_iterative(n):  
    result = 1  
    for i in range(1, n+1):  
        result *= i  
    return result
```

Test cases


```
test_cases = [(5,), (10,), (15,), (20,)]
```

```
# Measure execution times
```

```
recursive_times = measure_execution_time(factorial_recursive, test_cases)
```

```
iterative_times = measure_execution_time(factorial_iterative, test_cases)
```

```
print("Recursive Factorial Execution Times (seconds):")
```

```
for key, value in recursive_times.items():
```

```
    print(f" {key}: {value:.6f}")
```

```
print("\nIterative Factorial Execution Times (seconds):")
```

```
for key, value in iterative_times.items():
```

```
    print(f" {key}: {value:.6f}")
```

Code Complexity Metrics

Evaluating the complexity of generated code provides insights into its maintainability and readability.

```
import ast
```

```
import math
```

```
def calculate_cyclomatic_complexity(code):
```

```
    """
```

Calculate the cyclomatic complexity of Python code.

A higher score indicates more complex code.

"""

class ComplexityVisitor(ast.NodeVisitor):

def __init__(self):

self.complexity = 1 # Start with 1 for the entry point

def visit_If(self, node):

self.complexity += 1

self.generic_visit(node)

def visit_For(self, node):

self.complexity += 1

self.generic_visit(node)

def visit_While(self, node):

self.complexity += 1

self.generic_visit(node)

def visit_Try(self, node):

self.complexity += len(node.handlers) # Count except blocks

```
self.generic_visit(node)
```

```
def visit_BoolOp(self, node):
```

```
    if isinstance(node.op, ast.And) or isinstance(node.op, ast.Or):
```

```
        self.complexity += len(node.values) - 1
```

```
    self.generic_visit(node)
```

```
try:
```

```
    tree = ast.parse(code)
```

```
    visitor = ComplexityVisitor()
```

```
    visitor.visit(tree)
```

```
    return visitor.complexity
```

```
except SyntaxError:
```

```
    return math.inf # Return infinity for code with syntax errors
```

```
# Example usage
```

```
code1 = """
```

```
def simple_function(x):
```

```
    return x + 1
```

```
"""
```

```
code2 = """
```

```
def complex_function(x):
```

```
    if x > 0:
```

```
        if x < 10:
```

```
            return x * 2
```

```
        else:
```

```
            for i in range(x):
```

```
                if i % 2 == 0:
```

```
                    print(i)
```

```
            return x * 3
```

```
    else:
```

```
        try:
```

```
            return 10 / x
```

```
        except ZeroDivisionError:
```

```
            return 0
```

```
"""
```

```
cc1 = calculate_cyclomatic_complexity(code1)
```

```
cc2 = calculate_cyclomatic_complexity(code2)
```

```
print(f"Simple function cyclomatic complexity: {cc1}")
```

```
print(f"Complex function cyclomatic complexity: {cc2}")
```

Real-world Code Generation Evaluation Results

Here are some representative results from recent evaluations of code generation models:

HumanEval Benchmark Results

Model	▼	#	Pass@1 (%)	▼	#	Pass@10 (%)	▼	#	Pass@100 (%)	▼
OpenAI Codex			28.8			46.8			72.3	
GPT-3.5			48.1			65.7			78.3	
GPT-4			67			80.2			92.1	
Anthropic Claude			52.3			71.4			84.5	
Meta's Code Llama 34B			48.4			67.8			81.9	
DeepSeek-Coder			61.8			83.1			91.2	

CodeXGLUE Benchmark (Python Generation Task)

Model	▼	BLEU	▼	CodeBLEU	▼	#	Execution Accuracy (%)	▼
CodeGPT		18.2		29.5			34.8	
PLBART		19.3		32.1			38.2	
CodeT5		20.1		36.4			42.7	
StarCoder		25.7		42.8			56.3	
DeepSeek-Coder		28.4		45.9			63.1	
Claude 3 Opus		29.2		48.3			68.4	

These results highlight the rapid progress in code generation capabilities, with newer models achieving significantly higher functional correctness and code quality metrics compared to earlier versions.

Evaluating Other LLM Applications: Tailored Approaches

Beyond classification, summarization, generation, and code generation, LLMs are applied to a variety of other tasks, each requiring specific evaluation metrics tailored to the nuances of the use case.

Question Answering (QA)

The goal is to provide accurate answers to user-posed questions. Common evaluation metrics include:

- **Exact Match (EM):** Measures the percentage of predicted answers that perfectly match the ground truth answer
- **F1-score:** Calculates the overlap between the predicted and the ground truth answer at the word level
- **SQuAD metrics:** Specifically designed for the Stanford Question Answering Dataset, measuring both exact match and F1-score for reading comprehension tasks

```
def calculate_qa_metrics(predicted_answers, ground_truth_answers):
```

```
    """
```

```
        Calculate Exact Match and F1 score for QA tasks.
```

```
    Args:
```

```
        predicted_answers: List of predicted answers
```

```
        ground_truth_answers: List of lists of ground truth answers (multiple  
        acceptable answers per question)
```

```
    Returns:
```

```
        Dictionary with EM and F1 scores
```

```
    """
```

```
exact_match = 0
```

```
f1_scores = []
```

```
for pred, truths in zip(predicted_answers, ground_truth_answers):
```

```
    # Normalize answers
```

```
    pred = normalize_answer(pred)
```

```
    truths = [normalize_answer(truth) for truth in truths]
```

```
    # Check for exact match
```

```
    if pred in truths:
```

```
        exact_match += 1
```

```
    # Calculate F1 score (best match among the ground truths)
```

```
    best_f1 = 0
```

```
    for truth in truths:
```

```
        current_f1 = calculate_f1(pred, truth)
```

```
        best_f1 = max(best_f1, current_f1)
```

```
    f1_scores.append(best_f1)
```

```
return {
```

```
"exact_match": exact_match / len(predicted_answers) * 100,  
"f1": sum(f1_scores) / len(f1_scores) * 100  
}
```

```
def normalize_answer(text):
```

```
    """Normalize answer by removing articles, punctuation, and extra whitespace."""
```

```
    import re
```

```
    import string
```

```
    # Remove articles
```

```
    text = re.sub(r'\b(a|an|the)\b', ' ', text.lower())
```

```
    # Remove punctuation
```

```
    text = ''.join(ch for ch in text if ch not in string.punctuation)
```

```
    # Normalize whitespace
```

```
    text = ' '.join(text.split())
```

```
    return text
```

```
def calculate_f1(prediction, truth):
```


"""Calculate F1 score between prediction and ground truth."""

prediction_tokens = prediction.split()

truth_tokens = truth.split()

Empty answers

if len(prediction_tokens) == 0 or len(truth_tokens) == 0:

return int(prediction_tokens == truth_tokens)

Count common tokens

common = sum(1 for token in prediction_tokens if token in truth_tokens)

If no common tokens, F1 = 0

if common == 0:

return 0

precision = common / len(prediction_tokens)

recall = common / len(truth_tokens)

f1 = 2 * precision * recall / (precision + recall)

return f1

Example usage

```
predicted_answers = ["new york city", "albert einstein", "42 kilometers"]

ground_truth_answers = [
    ["New York City", "NYC", "New York"],
    ["Albert Einstein", "Einstein"],
    ["42 km", "42 kilometers", "forty-two kilometers"]
]

qa_metrics = calculate_qa_metrics(predicted_answers, ground_truth_answers)

print(f"QA Metrics:")

print(f" Exact Match: {qa_metrics['exact_match']:.2f}%")

print(f" F1 Score: {qa_metrics['f1']:.2f}%")
```

Dialogue Generation

For conversational agents, evaluation is complex due to the interactive and context-dependent nature of dialogues. Key metrics include:

- **BLEU, ROUGE, METEOR:** Basic measures of word overlap with reference responses
- **Distinct-n:** Assesses the diversity of the generated responses by measuring the proportion of unique n-grams
- **USR (UnSupervised and Reference-free):** Evaluates without requiring reference dialogues
- **Response Length:** Measures the verbosity of the model's responses
- **Perplexity:** Measures how well the model predicts the next token in a conversation.

```
def evaluate_dialogue(generated_responses, reference_responses,  
dialogue_contexts):
```

```
    """
```

Comprehensive evaluation of dialogue generation.

Args:

generated_responses: List of model-generated responses

reference_responses: List of human/reference responses

dialogue_contexts: List of dialogue contexts preceding the responses

Returns:

Dictionary with various dialogue evaluation metrics

```
    """
```

```
from nltk.translate.bleu_score import sentence_bleu
```

```
from nltk.tokenize import word_tokenize
```

```
results = {
```

```
    "bleu_scores": [],
```

```
    "distinctness": {
```

```
        "distinct-1": [],
```

```
        "distinct-2": []
```

```
},  
  
"response_lengths": [],  
  
"context_relevance": []  
  
}
```

```
for gen, ref, context in zip(generated_responses, reference_responses,  
dialogue_contexts):
```

```
    # Tokenize
```

```
    gen_tokens = word_tokenize(gen.lower())
```

```
    ref_tokens = word_tokenize(ref.lower())
```

```
    context_tokens = word_tokenize(context.lower())
```

```
    # BLEU score
```

```
    bleu = sentence_bleu([ref_tokens], gen_tokens)
```

```
    results["bleu_scores"].append(bleu)
```

```
    # Distinctness
```

```
    results["distinctness"]["distinct-1"].append(len(set(gen_tokens)) / max(1,  
len(gen_tokens)))
```

```
    bigrams = [' '.join(gen_tokens[i:i+2]) for i in range(len(gen_tokens)-1)]
```

```
results["distinctness"]["distinct-2"].append(len(set(bigrams)) / max(1, len(bigrams)))
```

```
# Response length
```

```
results["response_lengths"].append(len(gen_tokens))
```

```
# Simple context relevance (word overlap with context)
```

```
common_words = len(set(gen_tokens) & set(context_tokens))
```

```
relevance = common_words / max(1, len(set(gen_tokens)))
```

```
results["context_relevance"].append(relevance)
```

```
# Aggregate results
```

```
for key in results:
```

```
    if isinstance(results[key], list):
```

```
        results[key] = sum(results[key]) / len(results[key])
```

```
    elif isinstance(results[key], dict):
```

```
        for subkey in results[key]:
```

```
            results[key][subkey] = sum(results[key][subkey]) / len(results[key][subkey])
```

```
return results
```

Example usage

```
contexts = [
```

```
    "Hello, how are you doing today?",
```

```
    "Can you recommend a good restaurant in the area?",
```

```
    "What's the weather forecast for tomorrow?"
```

```
]
```

```
generated_responses = [
```

```
    "I'm doing well, thank you for asking!",
```

```
    "There are several great options nearby. I recommend trying Bella Italia on Main  
Street.",
```

```
    "Tomorrow will be sunny with a high of 75 degrees."
```

```
]
```

```
reference_responses = [
```

```
    "I'm fine, thanks! How about you?",
```

```
    "You should check out Luigi's Pizza downtown. It's fantastic!",
```

```
    "Tomorrow will be clear with temperatures around 73-76 degrees."
```

```
]
```

```
dialogue_metrics = evaluate_dialogue(generated_responses,  
reference_responses, contexts)
```

```

print("Dialogue Evaluation Metrics:")

for key, value in dialogue_metrics.items():

    if isinstance(value, dict):

        print(f" {key}:")

        for subkey, subvalue in value.items():

            print(f"   {subkey}: {subvalue:.4f}")

    else:

        print(f" {key}: {value:.4f}")

```

Translation

The task of converting text from one language to another. Key metrics include:

- **BLEU:** Measures the n-gram precision of the machine-translated text compared to human references
- **NIST:** Similar to BLEU but gives more weight to rarer, more informative n-grams
- **TER (Translation Edit Rate):** Measures the number of edits required to transform the machine translation output into a reference translation
- **chrF:** Character-level F-score that captures sub-word information
- **COMET:** A neural metric that uses contextualized embeddings to evaluate translation quality.

```
def evaluate_translation(source_texts, translations, references):
```

```
    """
```

Evaluate machine translation quality using multiple metrics.

Args:

source_texts: List of source language texts

translations: List of machine-translated texts

references: List of human reference translations

Returns:

Dictionary with various translation metrics

.....

```
from nltk.translate.bleu_score import corpus_bleu
```

```
from nltk.tokenize import word_tokenize
```

```
# Tokenize all texts
```

```
tokenized_translations = [word_tokenize(t.lower()) for t in translations]
```

```
tokenized_references = [[word_tokenize(r.lower())] for r in references]
```

```
# Calculate corpus BLEU
```

```
bleu_score = corpus_bleu(tokenized_references, tokenized_translations)
```

```
# Calculate character-based metrics (simplified chrF implementation)
```

```
def char_precision_recall_f1(hyp, ref):
```

```
    hyp_chars = list(hyp.lower())
```

```
    ref_chars = list(ref.lower())
```



```
common_chars = sum(1 for c in hyp_chars if c in ref_chars)
```

```
precision = common_chars / max(1, len(hyp_chars))
```

```
recall = common_chars / max(1, len(ref_chars))
```

```
if precision + recall == 0:
```

```
    return 0
```

```
f1 = 2 * precision * recall / (precision + recall)
```

```
return f1
```

```
chrf_scores = [char_precision_recall_f1(t, r) for t, r in zip(translations,  
references)]
```

```
avg_chrf = sum(chrf_scores) / len(chrf_scores)
```

```
# Calculate Translation Edit Rate (simplified version)
```

```
def calculate_ter(hyp, ref):
```

```
    # Using Levenshtein distance as a simple approximation
```

```
    import Levenshtein
```

```
    hyp_tokens = word_tokenize(hyp.lower())
```

```
    ref_tokens = word_tokenize(ref.lower())
```

```

edit_distance = Levenshtein.distance(hyp_tokens, ref_tokens)

return edit_distance / max(1, len(ref_tokens))

ter_scores = [calculate_ter(t, r) for t, r in zip(translations, references)]

avg_ter = sum(ter_scores) / len(ter_scores)

return {
    "bleu": bleu_score * 100, # Convert to percentage
    "chrf": avg_chrf * 100,   # Convert to percentage
    "ter": avg_ter * 100      # Convert to percentage (lower is better)
}

```

Example usage

```

source_texts = [
    "Das Wetter ist heute schön.",
    "Ich liebe Programmierung.",
    "Die Katze schläft auf dem Sofa."
]

```

```

translations = [

```

```
"The weather is nice today.",  
"I love programming.",  
"The cat is sleeping on the couch."  
]  
  
references = [  
    "The weather is beautiful today.",  
    "I love programming.",  
    "The cat is sleeping on the sofa."  
]  
  
translation_metrics = evaluate_translation(source_texts, translations, references)  
print("Translation Evaluation Metrics:")  
for metric, score in translation_metrics.items():  
    print(f" {metric}: {score:.2f}%")
```

Integrated Evaluation Frameworks and Benchmarks

Integrated evaluation frameworks and benchmarks provide a standardized way to assess the performance of large language models (LLMs) across a wide range of tasks, ensuring fair comparisons between models. Here's an overview of some of the key frameworks and emerging evaluation approaches

GLUE and SuperGLUE

- **GLUE (General Language Understanding Evaluation):**
GLUE is a benchmark composed of multiple natural language understanding tasks, such as sentiment analysis, textual entailment, and question answering. It was designed to assess how well a model can understand and process natural language by providing a single score that aggregates performance across these diverse tasks.
- **SuperGLUE:**
As a successor to GLUE, SuperGLUE is even more challenging, incorporating tasks that require deeper reasoning and more nuanced language understanding. It pushes models to perform better on tasks like commonsense reasoning and complex inference, thus serving as a tougher yardstick for state-of-the-art models.

MMLU (Massive Multitask Language Understanding)

MMLU is a comprehensive benchmark that evaluates a model's knowledge across 57 different subjects spanning humanities, STEM, social sciences, and more. It is designed to test both the breadth and depth of a model's understanding:

- **Breadth:** Measures the model's ability to handle a wide variety of topics.
- **Depth:** Assesses how well the model understands complex or specialized content within each subject area.

LLM-as-a-Judge

This is an emerging approach where one LLM is used to evaluate the outputs of another. Instead of relying solely on traditional metrics (like BLEU or ROUGE), this method leverages the language understanding and reasoning capabilities of an LLM to provide more nuanced, human-like judgments. Key aspects include:

- **Complex Distinction Based Assessments:** The evaluating LLM can consider multiple facets such as coherence, factual accuracy, creativity, and relevance.
- **Reference-Free Evaluation:** This method can assess outputs even without a fixed reference, which is particularly useful for open-ended or creative tasks where there isn't a single "correct" answer.
- **Breadth:** Measures the model's ability to handle a wide variety of topics.

- **Depth:** Assesses how well the model understands complex or specialized content within each subject area.

LLM-as-a-Judge

This is an emerging approach where one LLM is used to evaluate the outputs of another. Instead of relying solely on traditional metrics (like BLEU or ROUGE), this method leverages the language understanding and reasoning capabilities of an LLM to provide more nuanced, human-like judgments. Key aspects include:

- **Nuanced Assessments:** The evaluating LLM can consider multiple facets such as coherence, factual accuracy, creativity, and relevance.
 - **Reference-Free Evaluation:** This method can assess outputs even without a fixed reference, which is particularly useful for open-ended or creative tasks where there isn't a single "correct" answer.
-

Navigating the Pitfalls

Challenges and Best Practices in LLM Evaluation

Evaluating LLMs effectively is fraught with challenges. Key considerations include:

Subjectivity and Human Judgment

One significant hurdle is the inherent subjectivity in assessing the quality of certain LLM outputs, particularly in creative and open-ended tasks. Human judgment, while often considered the gold standard, can vary between evaluators, making it difficult to establish a universally agreed-upon "good" output.

Dataset Selection and Bias

The choice of evaluation datasets is critical. Metrics can provide misleading results if tested on datasets that are not diverse or representative of the real-world scenarios in which the LLM will be deployed. It is essential to use datasets that cover a wide range of inputs and expected outputs to ensure a robust evaluation.

Metric Selection and Combination

Relying on a single metric can provide an incomplete picture of an LLM's performance. Different metrics capture different aspects of quality, and the most appropriate metrics depend on the specific goals and requirements of the application. A comprehensive evaluation strategy often involves combining multiple complementary metrics.

Human-AI Collaborative Evaluation

Given the limitations of automated metrics, especially for nuanced and subjective aspects of language, human evaluation remains a vital component of a comprehensive evaluation strategy. Human evaluators can assess qualities like creativity, coherence, engagingness, and overall real-world usefulness, which automated metrics often struggle to capture. To ensure consistency and reduce bias in human evaluations, it is crucial to establish clear evaluation protocols and guidelines for the evaluators.

The Future of LLM Evaluation: Emerging Trends and Directions

The field of LLM evaluation is continuously evolving, driven by the rapid advancements in LLM capabilities. Future trends point towards:

Context-Aware and Human-Aligned Metrics

Development of more context-aware and human-aligned metrics that can go beyond simple surface-level comparisons. There is a growing focus on developing metrics that can effectively evaluate coherence, consistency, relevance, and factual accuracy.

LLMs as Evaluators

One significant trend is the increasing use of LLMs themselves as evaluators. These LLM-based evaluation methods leverage the language understanding capabilities of large models to assess the quality of other LLM outputs. Examples include G-Eval and CPDScore, which use large language models to provide human-like evaluations of generated text.

Reference-Free Evaluation

The rise of reference-free evaluation methods, which do not rely on human-annotated reference texts, is addressing the limitations of traditional reference-based metrics, especially for open-ended generation tasks.

Standardized Benchmarks

The development of more robust benchmarks and standardized evaluation frameworks is crucial for facilitating fair comparisons across different LLMs. Standardized benchmarks allow researchers and practitioners to objectively assess the progress of different models on common tasks.

Evaluating Broader Implications

There is a growing recognition of the importance of evaluating broader aspects of LLMs beyond just accuracy and fluency, such as fairness, bias, safety, and ethical implications. As LLMs become more powerful and integrated into society, ensuring that they are developed and deployed responsibly requires careful evaluation of these critical dimensions.

Conclusion

Towards Robust and Meaningful LLM Assessment

Evaluating large language models (LLMs) is complex due to their diverse applications, such as text classification, summarization, creative generation, question answering, dialogue systems, code generation, and translation. Each task requires specific evaluation methods and careful selection of metrics, often combining automated tools with human judgment.

Relying on just one metric isn't enough—multiple measures are essential for fully understanding an LLM's strengths and limitations. Evaluation should also use diverse, representative datasets to capture real-world scenarios accurately.

Looking forward, LLM evaluation will become more sophisticated, with approaches like using LLMs themselves as evaluators and developing metrics that don't rely solely on reference outputs. Additionally, there's a growing focus on assessing ethical aspects, including fairness, safety, and societal impact.

Ultimately, the responsible development of LLMs will depend on a balanced mix of traditional metrics, advanced neural-based assessments, and human insights, ensuring these powerful tools benefit society effectively and safely.

References

[Google Colab](#)

[Edit descriptioncolab.research.google.com](https://colab.research.google.com)

[Evaluation metrics](#)

[*Evaluating the performance of machine learning models is crucial for determining their effectiveness and reliability...learn.microsoft.com*](#)

[Performance metrics for evaluating generated code](#)

[*I'm just learning about techniques such as ROUGE and BLEU for evaluating trained models. I'm interested in training a...community.deeplearning.ai*](#)

[Custom text classification evaluation metrics - Azure AI services](#)

[*Learn about evaluation metrics in custom text classification.learn.microsoft.com*](#)

[Evaluating NLP Models for Text Classification and Summarization Tasks in the Financial Landscape...](#)

[*Unlock the power of advanced NLP models for precise text classification and summarization in the financial landscape...www.indium.tech*](#)

[Evaluate the text summarization capabilities of LLMs for enhanced decision-making on AWS | Amazon...](#)

[*Organizations across industries are using automatic text summarization to more efficiently handle vast amounts of...aws.amazon.com*](#)

Active and Semi-Supervised Learning: The Key to Efficient Machine Learning

Machine learning has revolutionized how we approach data analysis, decision-making, and automation. However, a significant obstacle remains: labeling data. Labeling is notoriously costly, time-consuming, and often impractical, especially in real-world scenarios. Fortunately, modern techniques like active learning and semi-supervised learning have emerged, dramatically reducing these challenges and unlocking a new era of efficiency in machine learning workflows.

The Data Labeling Bottleneck

Data labeling, essential for supervised learning, frequently poses significant logistical challenges. Expert annotations can be prohibitively expensive and time-intensive, particularly when dealing with massive datasets. Consistency in labeling is also difficult to maintain, leading to variable model performance. This bottleneck hampers innovation, particularly for startups and small organizations that lack extensive resources.

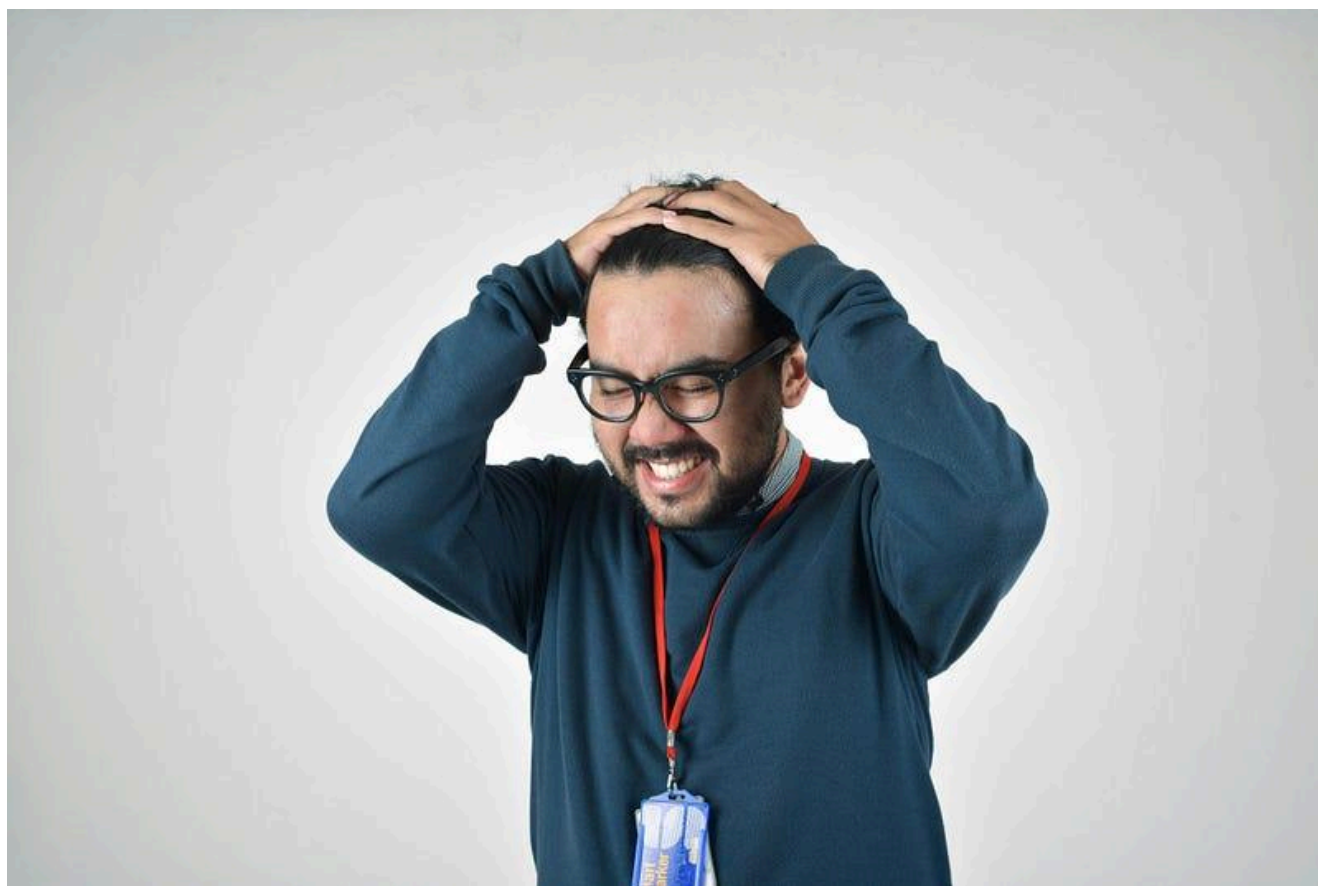


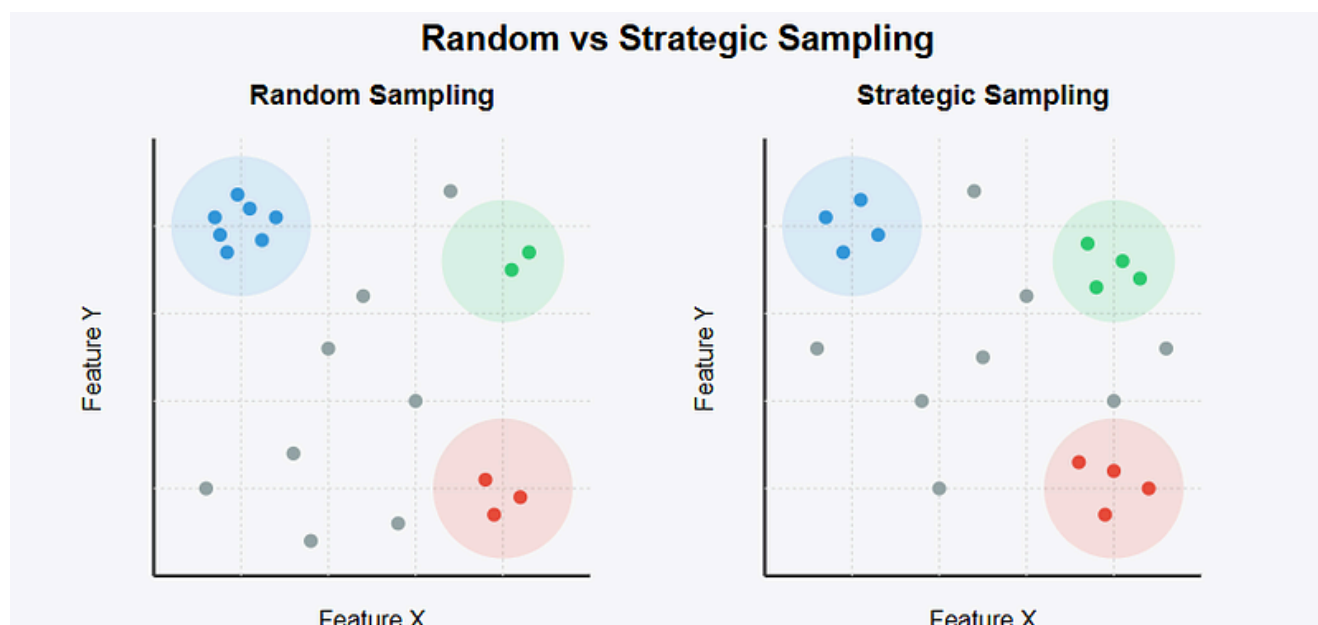
Photo by [ahmad gunnaivi](#) on [Unsplash](#)

The Hidden Truth: You Don't Need All Your Data

Surprisingly, not every data point in a dataset equally contributes to model accuracy. Many data points are redundant or provide minimal value. Intelligently selecting a subset of your data can yield substantial results without exhaustive labeling efforts. Research and practical experience indicate that even a fraction—around 30%—of strategically selected data can build an effective and efficient model.

The Problem with Random Sampling

While reducing the amount of labeled data sounds appealing, simply picking random samples won't guarantee good results. Random sampling can overlook important data points, resulting in inconsistent and suboptimal model performance. Imagine training a model to detect rare medical conditions—random sampling might miss critical, informative cases, resulting in a poorly performing model.

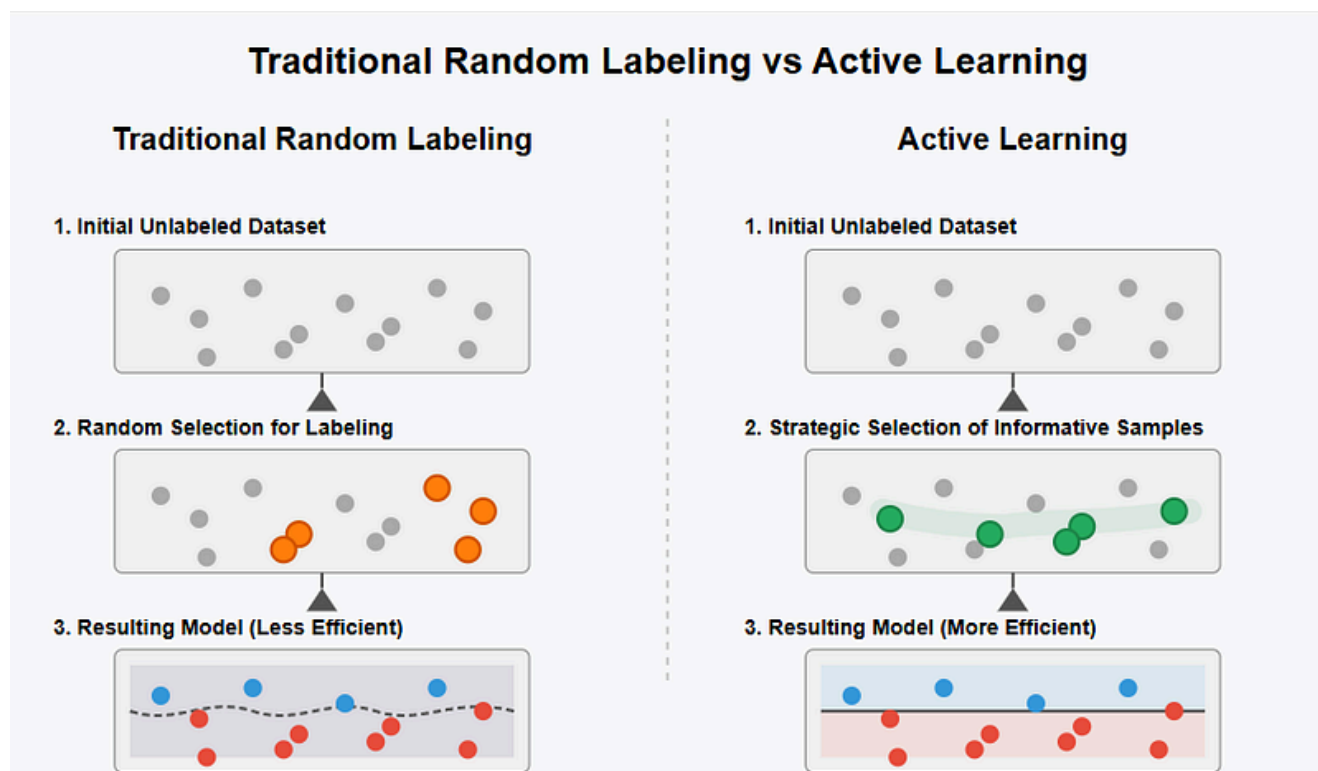


Random sampling often misses key data points, leading to inconsistent model performance.

Introducing Active Learning: A Strategic Revolution

Active learning addresses these issues head-on. Instead of random selection, active learning identifies and labels the most informative data points, drastically

improving performance. The idea is simple yet powerful: label data points the model finds most challenging. This strategic approach significantly enhances the efficiency and accuracy of the machine learning process.



Active learning selects the most informative samples, dramatically improving efficiency.

Inside Active Learning: How the Magic Happens

Active learning follows a clear iterative process:

1. **Kickoff:** Start by labeling a tiny portion (around 2–3%) of the dataset, creating an initial model.
2. **Inference:** Use this preliminary model to predict labels across the unlabeled dataset, assessing confidence levels.
3. **Sampling:** Select and label the most uncertain predictions, i.e., those data points that the model is least confident about.
4. **Retrain:** Update and improve the model using this newly labeled information.

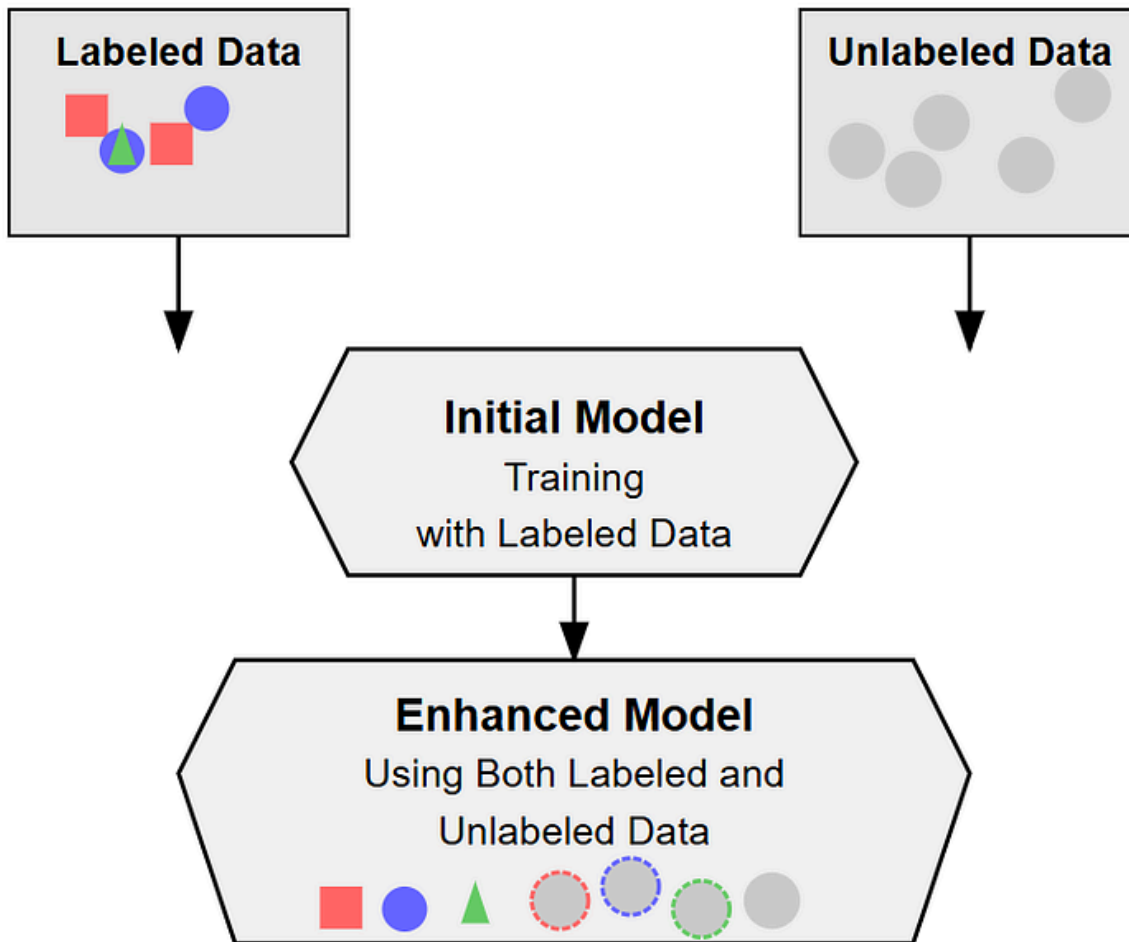
Repeating this cycle quickly improves model performance, focusing labeling efforts precisely where they're needed most.

Going Beyond: Semi-Supervised Learning Explained

Semi-supervised learning further complements active learning by leveraging unlabeled data to enhance performance. Unlike active learning, which strategically chooses which data points to label, semi-supervised learning exploits patterns from the entire dataset—both labeled and unlabeled—to improve model accuracy. It begins by training an initial model on labeled data, then iteratively uses predictions on unlabeled data to refine and enhance its understanding.

This method is particularly advantageous when extensive labeled data is unavailable or impractical to obtain. Imagine developing a speech recognition system—there's abundant unlabeled audio data but very few transcriptions. Semi-supervised learning leverages this large volume of unlabeled audio to enhance model accuracy substantially.

Semi-supervised Learning



Semi-supervised learning integrates labeled and unlabeled data, maximizing model efficiency.

Real-World Success Story: Meet NELL

One remarkable example of these strategies in action is Carnegie Mellon's NELL (Never-Ending Language Learner). NELL continuously improves its ability to read and understand the web, using active and semi-supervised learning techniques. It couples multiple functions—such as classifying noun phrases and extracting relationships—to iteratively build and refine its extensive knowledge base. By constantly incorporating new data, NELL demonstrates how effectively combining active and semi-supervised learning can yield significant real-world results.

Advanced Techniques: Fine-Tuning Your Approach

Active learning encompasses several sophisticated strategies beyond basic uncertainty sampling:

- **Query by Committee:** Employs multiple models to select data points that generate the most disagreement, improving decision boundaries.
- **Expected Model Change:** Focuses labeling efforts on samples likely to cause significant shifts in model parameters.
- **Density-Weighted Sampling:** Balances selecting uncertain examples and those typical of the dataset, avoiding outlier-driven bias.

These advanced techniques further refine active learning, ensuring maximum efficiency and improved accuracy.

Key Takeaways: Why You Should Care

The value of active and semi-supervised learning methods cannot be overstated. These methods:

- **Significantly reduce the time and cost associated with data labeling.**
- **Improve model accuracy by intelligently selecting informative data.**
- **Allow scalable application, from simple tasks to complex, large-scale machine learning problems.**

Conclusion: Embracing the Future of Machine Learning

Active and semi-supervised learning effectively address one of the most pressing challenges in machine learning today: labeled data scarcity. By intelligently integrating these methods into your workflow, you can build robust, high-performing models efficiently. The potential savings in time, cost, and resources are substantial, positioning these techniques as foundational tools for anyone serious about harnessing machine learning's full potential.

Embrace this future, and let active and semi-supervised learning unlock your next wave of innovation.

References:

https://www.youtube.com/watch?v=8Jwp4_WbRio&list=PLQNcEsjDCNmw-RbWwNVkqfsdV3BgClaAX
<https://www.youtube.com/watch?v=OMRlnKupsXM&t=33s>
<https://burrsettles.com/pub/settles.activelearning.pdf>

How to Create Professional AI Voiceovers for Free with Kokoro TTS (Better than ElevenLabs?)

Forget expensive subscriptions and complex setups. What if I told you that you could create professional-quality AI voiceovers right now, for absolutely free, using a tool that's not just good but potentially *better* than industry-leading (and costly) services like ElevenLabs? Sounds too good to be true? That's what the creators of Kokoro TTS are proving wrong, every single day. This isn't just another text-to-speech model; it's an 82-million parameter open-source *revolution* that's turning the AI voice world upside down. We're talking about a model trained on a shoestring budget, running on everyday computers, yet somehow outperforming giants on major benchmarks. Intrigued? You should be. This blog post will not only unveil the secrets behind Kokoro TTS but also guide you, step-by-step, on how to harness its power to create stunning voiceovers that will elevate your projects to new heights. Buckle up, because the future of AI voice is here, and it's free. This is more than just a tutorial; it's your gateway into a community that is redefining the landscape.



Kokoro TTS: Where Your Voice Meets the Future, for Free.

The Kokoro TTS Phenomenon: David vs. Goliath in the AI Arena

We all love an underdog story. And in the world of AI voice, Kokoro TTS is the ultimate underdog. Picture this: a tiny, 82-million parameter model, trained on a mere less than 100 hours of audio data, going head-to-head with behemoths like ElevenLabs, models trained on vast datasets and backed by significant funding. Sounds like a setup for failure, right? Wrong.

Here’s the shocker: Kokoro TTS didn’t just compete; it won. It snagged the #1 spot on the Hugging Face TTS Arena, a leaderboard where the best text-to-speech models are rigorously evaluated. How did it achieve this seemingly impossible feat?

The secret lies in its ingenious architecture and the efficiency of its design. It’s a highly optimized model that leverages a decoder-only architecture and was trained using approximately 500 GPU hours on A100 80GB GPUs. This efficiency translates to several mind-blowing advantages:

Blazing Fast Inference: Generate audio in real-time, even on a CPU. No more waiting around for cloud servers.

Runs Anywhere: From your laptop to a Raspberry Pi, Kokoro TTS can run on virtually any device.

Open-Source Freedom: Modify it, improve it, integrate it into your projects without limitations. The power is truly in your hands.

But the most astonishing part? It does all this while being completely free. This isn’t just about saving money; it’s about democratizing access to cutting-edge AI.

But the question remains: Can a free tool truly rival a premium service? Let’s put it to the test...

order	name	win_rate	elo	votes
#1	StyleTTS_Kokoro_v19	75.9	1293	8777
#2	Microsoft Edge TTS	65.4	1238	2479
#3	XITS_v2	61.6	1235	2889
#4	MetaVoice	57.4	1208	787
#5	Parler_Mini	52.8	1163	9655
#6	Fish_Speech	51.1	1148	5472
#7	Parler_Large	52.8	1136	553
#8	MaskGCI	47.8	1114	2983
#9	GPT-SoVITS_v2	46.8	1109	849
#10	MeloTTS	44.2	1108	7242
#11	Parler_Mini_Expresso	43.9	1107	1988
#12	llasa_3b	47.8	1105	494

<https://huggingface.co/spaces/Pendrokar/TTS-Spaces-Arena>

Unlocking the Magic: Your Step-by-Step Guide to Kokoro TTS

Enough with the suspense. Let’s get practical. Here’s how you can start creating with Kokoro TTS *right now*. We’ll use Google Colab for this tutorial, making it accessible to everyone, regardless of your hardware.

Step 1: The Colab Connection (Your Free AI Playground)

First, open up this Google Colab notebook: [here](#)

Step 2: Fire Up the Engine (One-Click Installation)

Run the first cell in the notebook. This cell installs all the necessary dependencies, including the **kokoro** library itself, plus a few other tools that make the magic happen. Think of it as prepping your digital voice studio.

#1 Install kokoro

```
!pip install -q kokoro soundfile
```

#2 Install espeak, used for out-of-dictionary fallback

```
!apt-get -qq -y install espeak-ng > /dev/null 2>&1
```

You can skip espeak installation, but OOD words will be skipped unless you provide a fallback

#3 Initialize a pipeline

```
from kokoro import KPipeline
```

```
from IPython.display import display, Audio
```

```
import soundfile as sf
```

```
# 🇺🇸 'a' => American English
```

```
# 🇬🇧 'b' => British English
```

```
pipeline = KPipeline(lang_code='a') # make sure lang_code matches voice
```

The following text is for demonstration purposes only, unseen during training

text = ""

The sky above the port was the color of television, tuned to a dead channel.

"It's not like I'm using," Case heard someone say, as he shouldered his way through the crowd around the door of the Chat. "It's like my body's developed this massive drug deficiency."

It was a Sprawl voice and a Sprawl joke. The Chatsubo was a bar for professional expatriates; you could drink there for a week and never hear two words in Japanese.

These were to have an enormous impact, not only because they were associated with Constantine, but also because, as in so many other areas, the decisions taken by Constantine (or in his name) were to have great significance for centuries to come. One of the main issues was the shape that Christian churches were to take, since there was not, apparently, a tradition of monumental church buildings when Constantine decided to help the Christian church build a series of truly spectacular structures. The main form that these churches took was that of the basilica, a multipurpose rectangular structure, based ultimately on the earlier Greek stoa, which could be found in most of the great cities of the empire. Christianity, unlike classical polytheism, needed a large interior space for the celebration of its religious services, and the basilica aptly filled that need. We naturally do not know the degree to which the emperor was involved in the design of new churches, but it is tempting to connect this with the secular basilica that Constantine completed in the Roman forum (the so-called Basilica of Maxentius) and the one he probably built in Trier, in connection with his residence in the city at a time when he was still caesar.

...

Step 3: Choose Your Voice (Meet the Cast)

Kokoro TTS comes with a collection of unique voices, each with its own personality. Run the second cell to load the default voice pack, which is a fascinating blend of two distinct voices: Bella and Sarah.

Secret Sauce: Want to experiment with other voices? You can find a list of available voice packs on the Kokoro TTS Hugging Face page (<https://huggingface.co/hexgrad/Kokoro-82M>). Just change the **VOICE_NAME** in the code. For example the voice packs currently include:

- **af**: Default voice, a 50-50 mix of Bella & Sarah
- **af_bella**, **af_sarah**, **am_adam**, **am_michael**,
- **bf_emma**, **bf_isabella**, **bm_george**, **bm_lewis**,
- **af_nicole**, **af_sky**

Step 4: Speak Your Mind (Let the AI Do the Talking)

Now for the fun part! In the third cell, you'll find a **text** variable. Replace the sample text with whatever you want Kokoro TTS to say. Get creative!

Step 5: Hear the Magic (Instant Audio)

Run the cell. Within seconds, you'll have an audio player right there in your Colab notebook. Hit play and listen to your text transformed into surprisingly natural-sounding speech.

Action Item: Try generating audio with different text inputs. Experiment with different voices, and listen carefully to the nuances.

But we're just getting started. What if you could create your own, unique AI voices?

```
from kokoro import KPipeline
```

```
from IPython.display import display, Audio
```

```
import soundfile as sf
```

```
import torch
```

1. Initialize the pipeline

```
pipeline = KPipeline(lang_code='a') # 'a' for American English
```

2. Define the available voices (from the model's config)

Though not directly used in mixing, it's good practice to keep this list

```
available_voices = [  
    'af', 'af_bella', 'af_sarah', 'am_adam', 'am_michael',  
    'bf_emma', 'bf_isabella', 'bm_george', 'bm_lewis',  
    'af_nicole', 'af_sky'  
]
```

3. Specify Device

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

4. Load the voice embeddings (download these from Hugging Face)

```
voice_01 = 'af_bella'
```

```
voice_02 = 'am_michael'
```

```
# voice_03 = 'bm_lewis'
```

5. Define the text

```
text = ''
```

Kokoro TTS stands at the intersection of art and technology, weaving humanlike warmth into synthetic voices.

It's the future of audio creation, boldly forging a path where open-source innovation conquers barriers,

granting every creator the freedom to share their story.

'''

6. Generate audio using individual voices (similar to the image)

print("Generating audio with individual voices...")

generator_01 = pipeline(text, voice=voice_01, speed=1, split_pattern=r'\n+')

for i, (gs, ps, audio) in enumerate(generator_01):

print(i) # i => index

print(gs) # gs => graphemes/text

print(ps) # ps => phonemes

display(Audio(data=audio, rate=24000, autoplay=i==0))

sf.write(f'{i}.wav', audio, 24000) # save each audio file

generator_02 = pipeline(text, voice=voice_02, speed=1, split_pattern=r'\n+')

for i, (gs, ps, audio) in enumerate(generator_02):

print(i) # i => index

print(gs) # gs => graphemes/text

```
print(ps) # ps => phonemes

display(Audio(data=audio, rate=24000, autoplay=i==0))

sf.write(f'{i}.wav', audio, 24000) # save each audio file


# generator_03 = pipeline(text, voice=voice_03, speed=1, split_pattern=r'\n+')

# for i, (gs, ps, audio) in enumerate(generator_03):

#     print(f"Voice 03 - Segment {i}:")

#     print(gs)

#     display(Audio(data=audio, rate=pipeline.sample_rate, autoplay=False))

#     sf.write(f'voice_03_segment_{i}.wav', audio, pipeline.sample_rate)
```

The Art of Voice Alchemy: Crafting Your Signature Sound

This is where Kokoro TTS truly shines. It's not just about choosing a voice; it's about *creating* one. Thanks to its ingenious voice mixing feature, you can blend different voice packs together, fine-tuning the ratios to craft a voice that's uniquely yours.

Imagine this:

- The warmth of Bella combined with the crispness of Sarah, creating a voice that's both friendly and authoritative.
- A touch of Michael's deep tones blended with Emma's gentle voice, perfect for a calming meditation app.

The possibilities are limited only by your imagination.

How does it work?

It's surprisingly simple. Each voice pack is represented by a set of numbers (an embedding). By adding these numbers together in different proportions, you're effectively mixing the characteristics of those voices.

7. Mix the voices (e.g., average of af_bella and am_michael)

```
print("\nGenerating audio with mixed voice (average)...")
```

```
mixed_voice_bf = voice_01_pt* 0.7 + voice_02_pt*0.3
```

8. Generate audio using the mixed voice

Instead of passing the tensor directly, create a new voice key and register it with the pipeline

Reshape the mixed voice to the expected shape (510, 1, 256)

```
pipeline.voices['mixed_voice'] = mixed_voice_bf.squeeze(0) # Register the mixed voice after squeezing
```

```
generator_mixed = pipeline(  
    text, voice='mixed_voice', # Use the registered key  
    speed=1, split_pattern=r'\n+'  
)
```

```
for i, (gs, ps, audio) in enumerate(generator_mixed):
```

```
    print(f"Mixed Voice - Segment {i}:")
```

```
    print(i) # i => index
```

```
    print(gs) # gs => graphemes/text
```



```
print(ps) # ps => phonemes
```

```
display(Audio(data=audio, rate=24000, autoplay=i==0))
```

```
sf.write(f'{i}.wav', audio, 24000) # save each audio file
```

```
print("\nAudio generation complete!")
```

Explanation:

- Load Voice Packs: We load two different voice packs using `torch.load()`.
- Mix Voices: We create a `mixed_voice` by multiplying each voice pack by a weight (representing the mix ratio) and adding them together. In this example, we're creating a 70/30 mix of `af_bella` and `am_michael`.
- Generate with Mixed Voice: We use the `generate()` function with the `mixed_voice` to synthesize audio using the blended voice.
- Display Audio Play the audio using the `display` and `Audio` functions from `IPython.display`

Action Item: Spend some time experimenting with different voice combinations and ratios. Create a voice that perfectly matches your project's needs. Save the settings for your favorite custom voices for later use.

Similar Example 2:

```
# 7. Mix the voices (e.g., average of af_bella and am_michael)
```

```
print("\nGenerating audio with mixed voice (average)...")
```

```
mixed_voice_bf = (voice_01_pt + voice_02_pt) / 2
```

```
# 8. Generate audio using the mixed voice
```

```
# Instead of passing the tensor directly, create a new voice key and register it with the pipeline
```

Reshape the mixed voice to the expected shape (510, 1, 256)

pipeline.voices['mixed_voice'] = mixed_voice_bf.squeeze(0) # Register the mixed voice after squeezing

```
generator_mixed = pipeline(
    text, voice='mixed_voice', # Use the registered key
    speed=1, split_pattern=r'\n+'
)
```

for i, (gs, ps, audio) in enumerate(generator_mixed):

print(f"Mixed Voice - Segment {i}:")

print(i) # i => index

print(gs) # gs => graphemes/text

print(ps) # ps => phonemes

display(Audio(data=audio, rate=24000, autoplay=i==0))

sf.write(f'{i}.wav', audio, 24000) # save each audio file

print("\nAudio generation complete!")

This is where you become the voice artist, sculpting the perfect AI voice for your project.

Beyond the Basics: Pro Tips and the Future

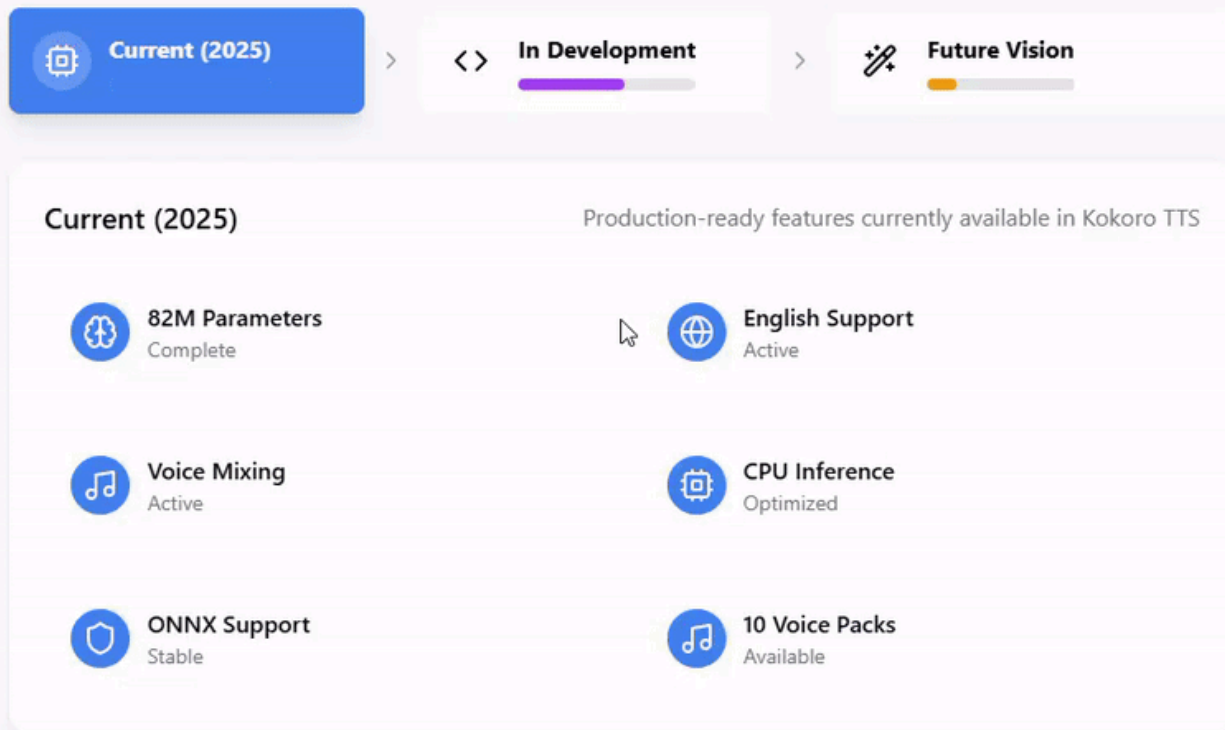
You've now mastered the fundamentals of Kokoro TTS. But to truly unlock its professional potential, here are some advanced tips and a glimpse into the future:

- **Mastering Pronunciation:** Kokoro TTS can handle most words with ease, but it does rely on **espeak-ng** for its grapheme-to-phoneme (g2p) conversion. This means that for words that are not in its dictionary, it will use its best guess, which may not always be correct.
- **Sentence Structure Matters:** Shorter sentences generally lead to better flow and intonation. Experiment with breaking down long sentences for optimal results.
- **The Power of Punctuation:** Commas, periods, and question marks are not just for grammar; they guide the AI's pauses and intonation. Use them strategically.
- **Community Power:** Join the Kokoro TTS community on GitHub and Hugging Face. Share your creations, learn from others, and contribute to the project's growth. This is where you'll find the latest tips, tricks, and voice packs.

What's next for Kokoro TTS? The developers are constantly working on improvements. Currently the model is being developed further, with a larger dataset, which will likely lead to higher quality output.

The Takeaway: Kokoro TTS isn't just a tool; it's a movement. It's about putting the power of high-quality AI voice into the hands of everyone. It is a revolution in the making.

Kokoro TTS Development Timeline



The Future of Kokoro TTS: More Voices, More Languages, More Possibilities.

Conclusion: Your Voice, Your Story, Amplified by AI

Kokoro TTS is more than just a free alternative to premium text-to-speech services. It's a testament to the power of open-source collaboration and a glimpse into the future of AI voice technology. We've explored its remarkable capabilities, learned how to use it effectively, and discovered the art of crafting unique voices through blending. Now it's your turn to join the revolution. Whether you're a seasoned developer, a creative content creator, or simply curious about the possibilities of AI, Kokoro TTS offers an unparalleled opportunity to experiment, innovate, and amplify your voice.

Don't just read about it—experience it. Head over to the Kokoro TTS Hugging Face page, try the Google Colab, and start creating. Share your projects, contribute to the community, and help shape the future of AI voice. The world is listening. What will you create with Kokoro TTS?

Action Items:

- Visit the Kokoro TTS Hugging Face page: <https://huggingface.co/hexgrad/Kokoro-82M>
- Try the Google Colab demo: [here](#)
- Join the Kokoro TTS community on Discord: [Here](#)
- The Official Kokoro github: <https://github.com/hexgrad/kokoro>
- The kokoro-onnx library: <https://github.com/thewh1teagle/kokoro-onnx>
- Share your Kokoro TTS creations with the hashtag #KokoroTTS

Let the AI voice revolution begin!

The AI voice revolution is here, and Kokoro TTS is your ticket to mastering it. I share cutting-edge tools, practical tutorials, and industry insights like this every week. If you're ready to create, innovate, and stay ahead of the curve in AI tech, hit that follow button and let's build the future together!

Federated Learning Explained: Build Better AI Without Compromising Privacy

Imagine a world where medical breakthroughs happen at lightning speed, where your phone understands you perfectly, and where AI-powered services anticipate your needs with uncanny accuracy. This future is within reach, thanks to the power of Artificial Intelligence. But there's a catch: traditional AI relies on collecting massive amounts of data in a single, centralized location. And in many cases, that data is highly personal, sensitive, and legally protected.

A. The Problem with Centralized Data

Think about training an AI to detect cancer. To build an accurate model, you'd need a vast dataset of medical images, patient records, and treatment outcomes. But hospitals are bound by strict privacy laws like HIPAA in the United States and GDPR in Europe. They can't simply hand over their patients' data due to ethical and legal obligations.

Or consider your smartphone's keyboard. It learns from your typing patterns to suggest words and correct your typos, making your texting and emailing faster and more efficient. But do you really want every keystroke you make to be sent to a central server, stored, and analyzed? Probably not. That's a massive privacy concern.

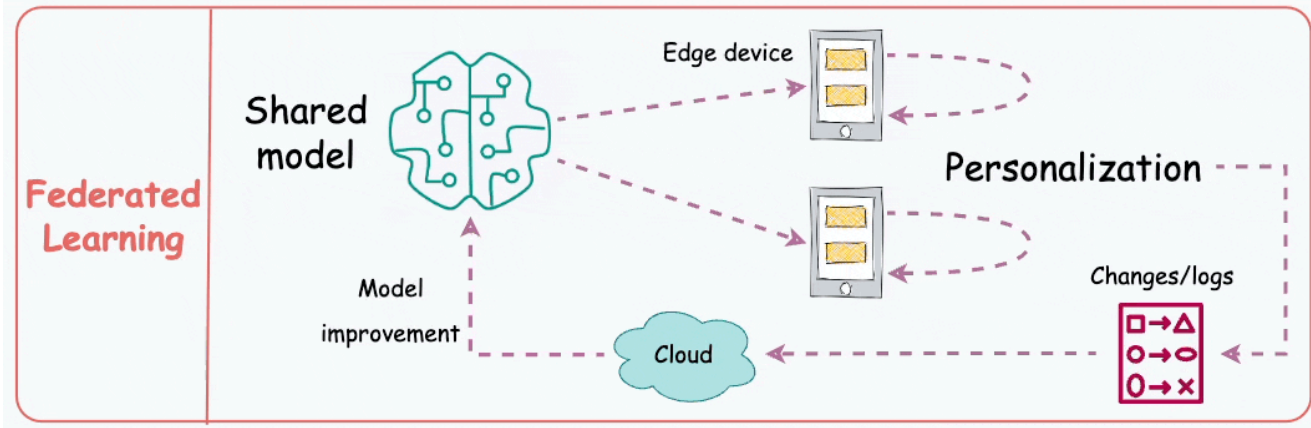
These are just two examples of the limitations of traditional, centralized machine learning. The process of collecting all the data in one place presents several daunting challenges:

- **Privacy Concerns:** Centralized data storage creates a honeypot for hackers. A single breach can expose the personal information of millions of individuals, leading to identity theft, fraud, and other serious consequences.
- **Data Security Risks:** Even with the best security measures, centralized databases are vulnerable to attacks. The more data is stored in one place, the greater the risk.
- **Legal Restrictions:** Regulations like GDPR and HIPAA place strict limits on how personal data can be collected, stored, and used. Complying with these regulations can be complex and expensive, especially when dealing with data from multiple jurisdictions.
- **Sheer Volume:** The amount of data generated by devices and individuals is exploding. Transferring and storing all this data in a central location is becoming increasingly impractical and costly. The bandwidth alone could become a limiting factor to even making use of the data that our devices produce.

As a result, many AI projects are stifled before they even begin. We're missing out on life-saving medical advancements, innovative new technologies, and personalized services that could improve our lives in countless ways. We are stuck working with publicly available data that, while useful, often does not contain the same richness that privately held data might. It is for this reason that we must rethink how we train AI.

Federated Learning

blog.DailyDoseofDS.com



<https://x.com/DailyDoseOfDS/status/1850120338753143012>

B. Introducing Federated Learning (FL):

Thankfully, there's a better way. It's called Federated Learning, and it's revolutionizing the field of AI. Federated Learning flips the script. Instead of bringing the data to the model, we bring the model to the data.

Here's the basic idea: Instead of collecting all the data in one place, training happens on individual devices (like smartphones) or within isolated environments (like hospitals or financial institutions). Each device or "silo" trains a local model using its own data. Then, instead of sharing the raw data, only the model updates—the changes learned during training—are sent to a central server.

Imagine a group of chefs learning a new recipe. In the traditional, centralized approach, each chef would have to send all their ingredients to a head chef, who would then try to create the perfect dish. But with Federated Learning, each chef perfects the recipe in their own kitchen, using their own ingredients. They then share only their improved recipe notes with the head chef, who combines them to create a master recipe that incorporates the best elements from each chef's experience.



In

Federated Learning, each ‘chef’ (device or silo) trains a model locally and shares only the learned updates, not the raw ‘ingredients’ (data).

C. Benefits of Federated Learning:

This decentralized approach offers a number of significant advantages:

- **Privacy Preservation:** Raw data never leaves the device or silo where it originated. This is a game-changer for privacy, as it eliminates the need to store sensitive information in a central location, drastically reducing the risk of data breaches.
- **Reduced Data Transfer:** Only model updates, which are typically much smaller than the raw data, are shared. This saves bandwidth, reduces latency, and lowers costs, making it feasible to train models on massive datasets that would be impossible to centralize.
- **Legal Compliance:** Federated Learning makes it easier to comply with data privacy regulations like GDPR and HIPAA, as data remains under the control of the data owner.
- **Access to More Diverse Data:** By enabling training on decentralized datasets, Federated Learning unlocks the potential of data that was previously inaccessible due to privacy concerns or logistical challenges. This leads to more robust, accurate, and generalizable AI models.

- **Improved Model Personalization:** In addition to contributing to a global model, local models can also be fine-tuned for individual users, providing a more personalized and relevant experience. Imagine a smart keyboard that adapts to your unique typing style or a medical diagnosis model tailored to your specific health profile.

D. Blog's Focus and Promise:

This blog will provide a beginner-friendly introduction to the exciting world of Federated Learning. We'll explore the core concepts, the challenges, and the incredible potential of this revolutionary approach to AI.

We'll also dive into some of the most popular frameworks and tools that are making Federated Learning a reality, including Flower, TensorFlow Federated (TFF), and PySyft. You'll see code snippets and examples that demonstrate how these frameworks work, using the flexible deep learning capabilities of PyTorch.

By the end of this blog, you'll have a solid understanding of the basics of Federated Learning and how frameworks like Flower, TFF, and PySyft, along with PyTorch, are empowering developers to build privacy-preserving AI solutions. You'll be ready to explore further and perhaps even start building your own federated learning projects. You'll also see how you might leverage the power of Amazon Web Services to help you construct your own Federated Learning application. Get ready to unlock the power of decentralized data and train AI models without compromising privacy!

II. Deep Dive into Federated Learning Concepts

Now that we've whetted your appetite with the promise of Federated Learning, let's dive deeper into the core concepts that make it work.

A. Key Terminology:

Understanding these terms will provide a solid foundation for grasping the intricacies of FL:

- **Clients:** In the world of Federated Learning, clients are the individual participants in the training process. They can be anything from smartphones and IoT devices to hospitals, banks, or other organizations that hold data. Each client possesses its own local dataset, which is used for training.

- **Server:** The server acts as the central coordinator in the Federated Learning process. It's responsible for initializing the global model, selecting clients for each training round, distributing the model to clients, and aggregating the model updates received from them.
- **Rounds:** A round refers to a single iteration of the Federated Learning process. It typically involves the server sending the global model to clients, clients training the model on their local data, and the server aggregating the updates to create a new global model.
- **Local Model:** Each client trains a local model on its own data. This model is a copy of the global model that is adapted and improved based on the client's specific dataset.
- **Global Model:** This is the central model maintained by the server. It represents the collective knowledge learned from all participating clients. The goal of Federated Learning is to improve the global model's performance through iterative updates from the clients.
- **Model Updates:** After training on local data, clients send model updates back to the server. These updates represent the changes made to the model's parameters (weights and biases) during training. Importantly, the raw data itself is never shared, only the learned adjustments to the model.

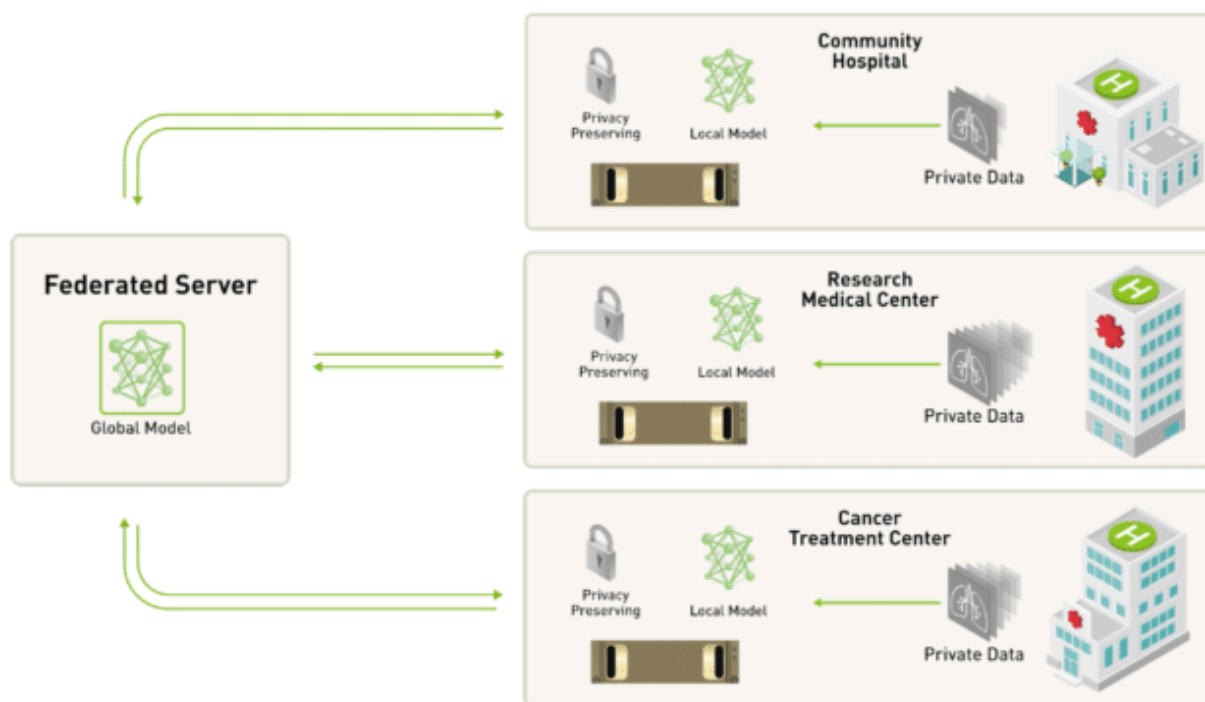
B. The Federated Learning Process (Step-by-Step):

Let's break down the Federated Learning process into a clear, step-by-step sequence:

1. **Initialization:** The process begins with the server initializing a global model. This model might be a randomly initialized neural network or a pre-trained model, depending on the specific application.
2. **Client Selection:** In each round, the server selects a subset of clients to participate in the training process. The selection criteria can vary, but the goal is often to choose a diverse set of clients that represent the overall population.
3. **Model Distribution:** The server sends the current global model to the selected clients. This is where bandwidth efficiency becomes crucial, as sending large models can be time-consuming and costly.
4. **Local Training:** Each client receives the global model and trains it on its own local data for a certain number of epochs or steps. This is where the "learning" part of Federated Learning happens. Clients use optimization

algorithms like Stochastic Gradient Descent (SGD) to adjust the model's parameters and improve its performance on their local data.

5. **Update Aggregation:** After local training, clients send their model updates back to the server. These updates typically consist of the changes made to the model's weights and biases.
6. **Model Update:** The server aggregates the updates received from the clients. A common aggregation method is Federated Averaging (FedAvg), which we'll discuss next. The server then uses the aggregated updates to improve the global model.
7. **Repeat:** Steps 2–6 are repeated for multiple rounds. With each round, the global model gradually improves as it learns from the collective knowledge of the participating clients.



<https://blogs.nvidia.com/blog/what-is-federated-learning/>

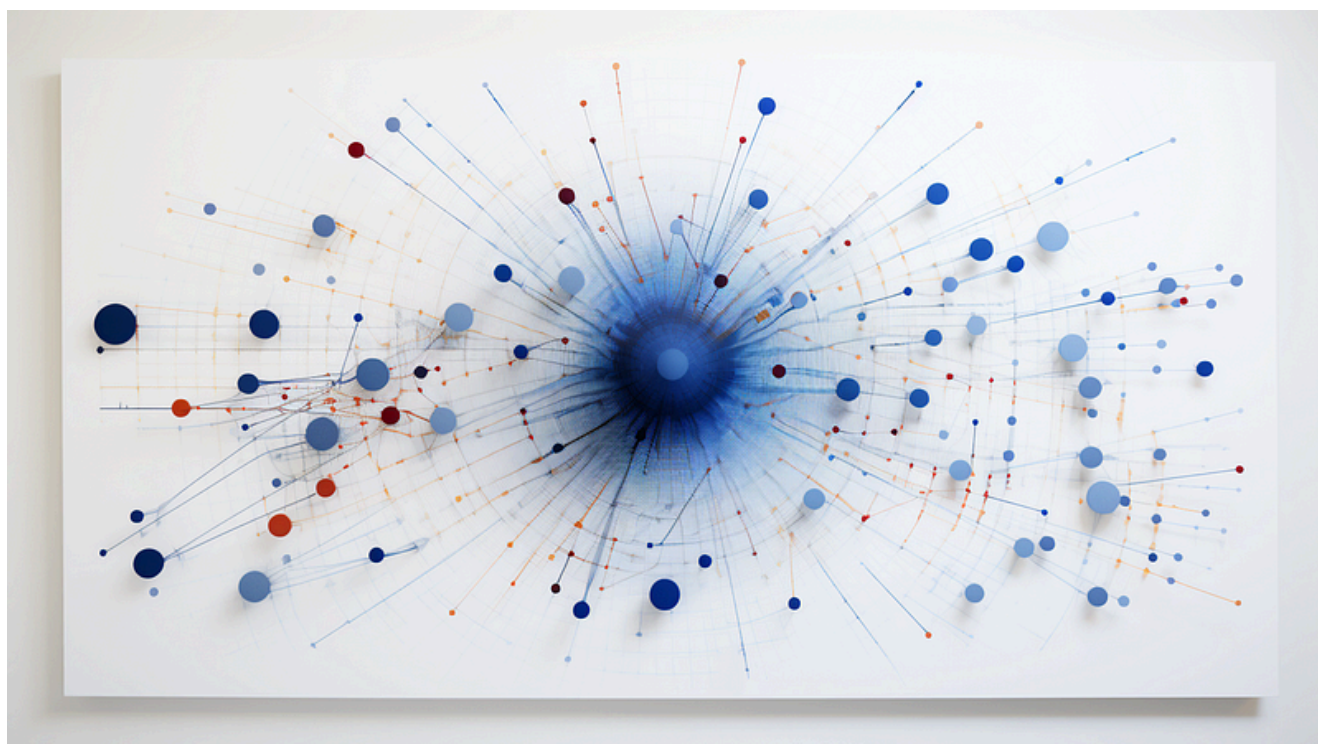
C. Federated Averaging (FedAvg):

Federated Averaging (FedAvg) is a fundamental algorithm for aggregating model updates in Federated Learning. It's like averaging the improved recipe notes from all the chefs to create the best master recipe.

In simple terms, FedAvg works by taking a weighted average of the model updates received from the clients. The weight for each client's update is typically

proportional to the number of data samples they used for training. This ensures that clients with more data have a larger influence on the global model.

Analogy: Imagine each chef sends their revised recipe notes to the head chef. The head chef doesn't just randomly pick one set of notes. Instead, they carefully consider each chef's suggestions, giving more weight to chefs who tested the recipe with more ingredients (more data). The head chef then combines these weighted suggestions to create the final, improved master recipe.



Federated Averaging (FedAvg) combines client model updates, weighted by their local dataset size, to improve the global model.

D. Challenges in Federated Learning:

While Federated Learning offers significant advantages, it also presents unique challenges:

- **Non-IID Data:** In real-world scenarios, data is often not independently and identically distributed (Non-IID) across clients. This means that the data distribution on each client can be significantly different from the overall population distribution. For example, different users might have different typing styles on their keyboards, or different hospitals might have different

patient demographics. This can make it harder for the global model to generalize well to all clients.

- **Communication Costs:** Sending model updates between the server and clients can be communication-intensive, especially for large models or when clients have limited bandwidth. This can slow down the training process and increase costs.
- **System Heterogeneity:** Clients in a Federated Learning system can have widely varying computational power, network connectivity, and availability. Some devices might be powerful smartphones, while others might be low-powered IoT devices. This heterogeneity needs to be considered when designing and deploying FL systems.
- **Privacy and Security:** While Federated Learning is designed to enhance privacy by keeping data local, model updates can still potentially leak information about the training data. Techniques like Differential Privacy and Secure Aggregation are used to further enhance privacy, but they can add complexity to the system.

In the next section, we'll explore how FL frameworks help address these challenges and make it easier to build and deploy Federated Learning systems.

III. Federated Learning Frameworks and Tools

The world of Federated Learning might seem complex at first, but thankfully, a vibrant ecosystem of frameworks and tools has emerged to simplify development and deployment. These frameworks handle the heavy lifting of communication, aggregation, and security, allowing you to focus on building great AI models. In this section, we'll explore some of the leading players: Flower, TensorFlow Federated (TFF), PyTorch, and PySyft.

A. Introduction to Frameworks:

Frameworks are essential for making Federated Learning practical. They provide the building blocks and abstractions needed to manage the distributed nature of FL, handle communication between clients and the server, and implement privacy-preserving mechanisms. Without them, building FL systems from scratch would be a monumental task. We will focus on a few of the most popular frameworks.

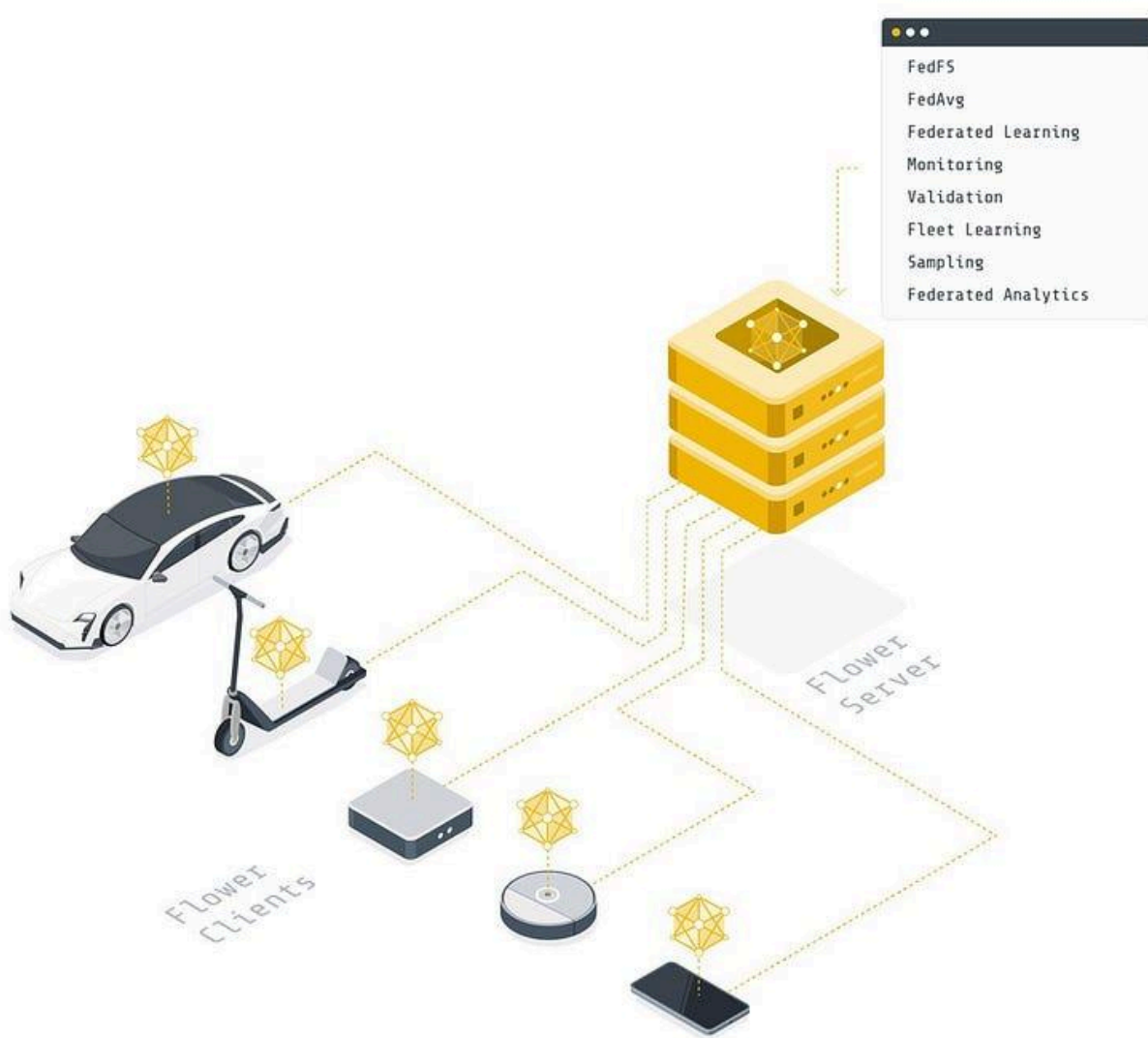
B. Flower: A Friendly Framework for Federated Learning

Overview:

[Flower](#) is a powerful yet user-friendly framework designed to make Federated Learning accessible to a wide range of developers. It's known for its framework-agnostic nature, meaning you can use it with popular deep learning frameworks like PyTorch, TensorFlow, or even other machine learning libraries. This flexibility, combined with its scalability and active community, makes Flower an excellent choice for both research and production deployments.

Key Features:

- **Framework Agnostic:** Works with various ML frameworks (TensorFlow, PyTorch, scikit-learn, etc.).
- **Scalability:** Supports large-scale deployments with potentially millions of clients.
- **Customizability:** Allows you to define custom aggregation logic, training loops, and more.
- **Easy to Use:** Provides simple abstractions and APIs for common FL tasks.
- **Active Community:** Backed by a vibrant community of developers and researchers.
- **Strong Baselines:** Includes implementations of various state-of-the-art FL algorithms.



Flower's framework-agnostic design allows it to seamlessly integrate with various machine learning libraries, making it a versatile choice for federated learning.

Code Snippet (Simple Flower Example):

Let's see how easy it is to get started with Flower using PyTorch for a simple image classification task.

Basic Flower client using PyTorch
import flwr as fl

import torch

import torch.nn as nn

```

import torch.nn.functional as F

from torchvision.transforms import Compose, ToTensor, Normalize

from torch.utils.data import DataLoader

from torchvision.datasets import CIFAR10

from tqdm import tqdm

import collections


#
#####

# 1. Regular PyTorch pipeline: nn.Module, train, test, and Dataloader

#
#####

# Define your PyTorch model as usual


class Net(nn.Module):

    def __init__(self) -> None:

        super(Net, self).__init__()

        self.conv1 = nn.Conv2d(3, 6, 5)

        self.pool = nn.MaxPool2d(2, 2)

        self.conv2 = nn.Conv2d(6, 16, 5)

        self.fc1 = nn.Linear(16 * 5 * 5, 120)

```



```
self.fc2 = nn.Linear(120, 84)
```

```
self.fc3 = nn.Linear(84, 10)
```

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
```

```
    x = self.pool(F.relu(self.conv1(x)))
```

```
    x = self.pool(F.relu(self.conv2(x)))
```

```
    x = x.view(-1, 16 * 5 * 5)
```

```
    x = F.relu(self.fc1(x))
```

```
    x = F.relu(self.fc2(x))
```

```
    x = self.fc3(x)
```

```
    return x
```

```
# Define your training loop
```

```
def train(net, trainloader, epochs):
```

```
    """Train the network on the training set."""
```

```
    criterion = torch.nn.CrossEntropyLoss()
```

```
    optimizer = torch.optim.Adam(net.parameters())
```

```
    net.train()
```

```
    for _ in range(epochs):
```

```
        for images, labels in tqdm(trainloader):
```

```
            optimizer.zero_grad()
```

```
            criterion(net(images), labels).backward()
```

```
            optimizer.step()
```

Define your testing loop

def test(net, testloader):

"""Validate the network on the entire test set."""

criterion = torch.nn.CrossEntropyLoss()

correct, loss = 0, 0.0

net.eval()

with torch.no_grad():

for images, labels in tqdm(testloader):

outputs = net(images)

loss += criterion(outputs, labels).item()

_, predicted = torch.max(outputs.data, 1)

correct += (predicted == labels).sum().item()

accuracy = correct / len(testloader.dataset)

return loss, accuracy

Load CIFAR-10 dataset

def load_data():

"""Load CIFAR-10 (training and test set)."""

transform = Compose([ToTensor(), Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

trainset = CIFAR10("./data", train=True, download=True, transform=transform)

testset = CIFAR10("./data", train=False, download=True, transform=transform)

trainloader = DataLoader(trainset, batch_size=32, shuffle=True)

```

testloader = DataLoader(testset, batch_size=32)

return trainloader, testloader

# Load your model and data

net = Net()

trainloader, testloader = load_data()


#
#####
#####

# 2. Federation of the pipeline with Flower

#
#####
#####

# Define Flower client

class FlowerClient(fl.client.NumPyClient):

    def get_parameters(self, config):

        return [val.cpu().numpy() for _, val in net.state_dict().items()]

    def set_parameters(self, parameters):

        params_dict = zip(net.state_dict().keys(), parameters)

        state_dict = collections.OrderedDict({k: torch.Tensor(v) for k, v in
params_dict})

        net.load_state_dict(state_dict, strict=True)

```

```

def fit(self, parameters, config):

    self.set_parameters(parameters)

    train(net, trainloader, epochs=1)

    return self.get_parameters(config={}), len(trainloader.dataset), {}

def evaluate(self, parameters, config):

    self.set_parameters(parameters)

    loss, accuracy = test(net, testloader)

    return loss, len(testloader.dataset), {"accuracy": accuracy}

```

Start Flower client

```

fl.client.start_numpy_client(

    server_address="[:,]:8080",

    client=FlowerClient(),

)

```

Explanation:

- **get_parameters**: This method extracts the model's parameters as a list of NumPy arrays.
- **set_parameters**: This method takes a list of NumPy arrays (parameters) and updates the PyTorch model's weights.
- **fit**: This method receives parameters from the server, updates the local model, trains the model using the provided **train** function and PyTorch dataloader, and then returns the updated parameters, the number of training examples and an empty dictionary for metrics.
- **evaluate**: This method receives parameters from the server, updates the local model, evaluates the model using the provided **test** function and

PyTorch dataloader, and then returns the loss, the number of test examples, and a dictionary containing the accuracy.

Starting the Flower Server:

On the server side, you would start a Flower server using the following command:

```
flower-server start-server
```

Running a Flower Simulation:

Flower allows you to simulate an FL system on your local machine. You can start multiple clients in separate processes and connect them to the server. Each client will train on a different subset of the data, simulating a distributed environment. You can run multiple clients by simply running the `client.py` file multiple times, in multiple terminals:

```
python client.py
```

C. TensorFlow Federated (TFF): A Deeper Dive

Overview:

TensorFlow Federated (TFF) is a powerful framework for conducting federated learning research and, increasingly, deploying production-level FL systems. While it has a steeper learning curve than Flower, TFF offers greater control and flexibility for expressing complex federated computations. It's particularly well-suited for scenarios that require fine-grained control over the FL process or involve advanced privacy mechanisms.

TFF is organized in two main layers:

- **Federated Learning (FL) API:** This higher-level API, similar in spirit to `tf.keras`, provides abstractions for common FL tasks like training and evaluation. It's built on top of the Federated Core.
- **Federated Core (FC) API:** This lower-level API provides the foundational building blocks for expressing federated computations. It introduces concepts like federated types, federated placements, and federated operators, allowing you to define how data is distributed and how computations are performed across clients and the server.

Conceptual Example:

Let's imagine we want to compute the average temperature across a network of sensors using TFF. We can express this computation using the FC API, even without diving into the full TFF syntax.

Conceptual TFF example (not executable code)

Define federated types

```
float_at_clients = tff.type_at_clients(tf.float32)
```

```
float_at_server = tff.type_at_server(tf.float32)
```

Define a federated computation

```
@tff.federated_computation(float_at_clients)
```

```
def compute_average_temperature(temperatures):
```

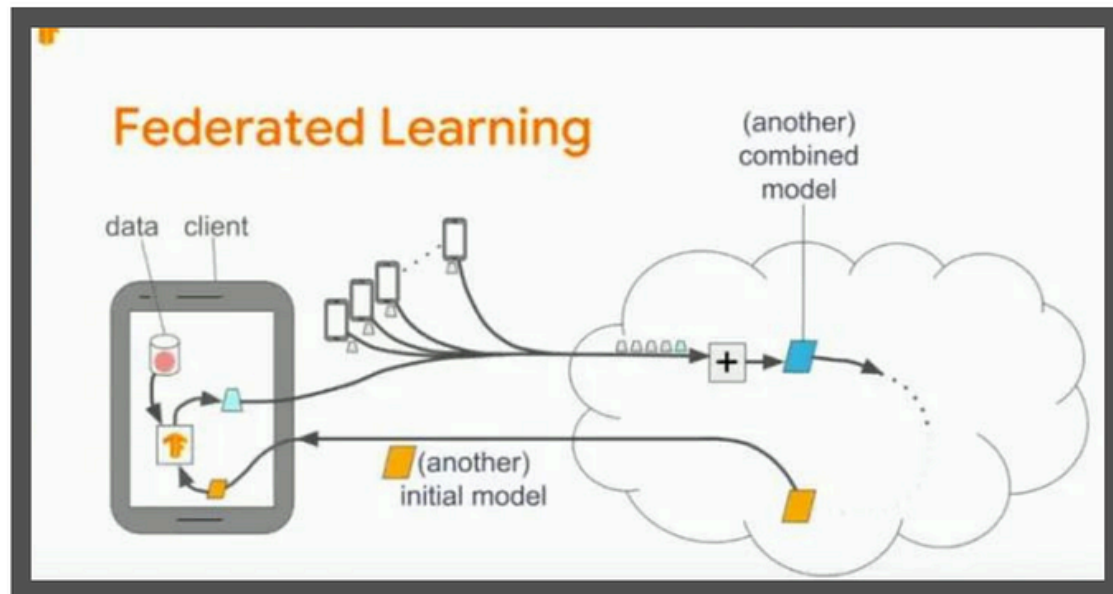
```
    # tff.federated_mean is a federated operator that computes the mean across
    # clients
```

```
    average_temp = tff.federated_mean(temperatures)
```

```
    return tff.federated_map(some_tf_function_to_execute_at_server, average_temp)
```

Explanation:

- **Federated Types:** We define `float_at_clients` to represent a `tf.float32` value that resides on each client and `float_at_server` to represent a single `tf.float32` value on the server.
- **Federated Computation:** The `tff.federated_computation` decorator indicates that this function defines a federated computation.
- **Federated Operators:** `tff.federated_mean` is a built-in operator that computes the mean of a federated value across clients.



TensorFlow Federated provides a powerful framework for expressing complex federated computations, with a focus on data placements and privacy.

This is just a glimpse into the capabilities of TFF. It's a powerful tool for researchers and developers who need to go beyond the basics of Federated Averaging and explore more advanced FL algorithms and privacy mechanisms.

IV. Real-World Use Cases in Finance and Healthcare

The theoretical foundations and technical tools are in place. But where is Federated Learning making a real-world impact? Let's explore some compelling use cases in two critical sectors: finance and healthcare. These industries are particularly well-suited for FL because they deal with highly sensitive data that is often distributed across multiple institutions.

A. Finance

The financial industry is embracing Federated Learning to enhance various services while adhering to strict privacy regulations and maintaining data security.

1. Anti-Money Laundering (AML):

Money laundering is a global problem that poses a significant threat to the

financial system. Banks are required to monitor transactions for suspicious activity and report them to authorities. However, traditional AML systems often suffer from high false-positive rates, leading to wasted resources and customer inconvenience.

Federated Learning offers a powerful solution. The banking circle, a next-generation payments bank, has successfully leveraged FL, built using the Flower framework, to improve their AML models. Their challenge was to train a model that could accurately detect money laundering activities in the US market, but they couldn't directly share transaction data from Europe due to regulatory restrictions.

Using Flower, they were able to train an AI model on European data without moving it across borders. This allowed them to develop a model tailored to the US market, even without direct access to US data. The initial results were impressive, demonstrating significant performance improvements:

- 65% increase in precision
- 55% increase in recall
- 10% increase in accuracy

This use case demonstrates how FL can enable financial institutions to collaborate on model training without compromising sensitive data, ultimately leading to more effective AML systems.

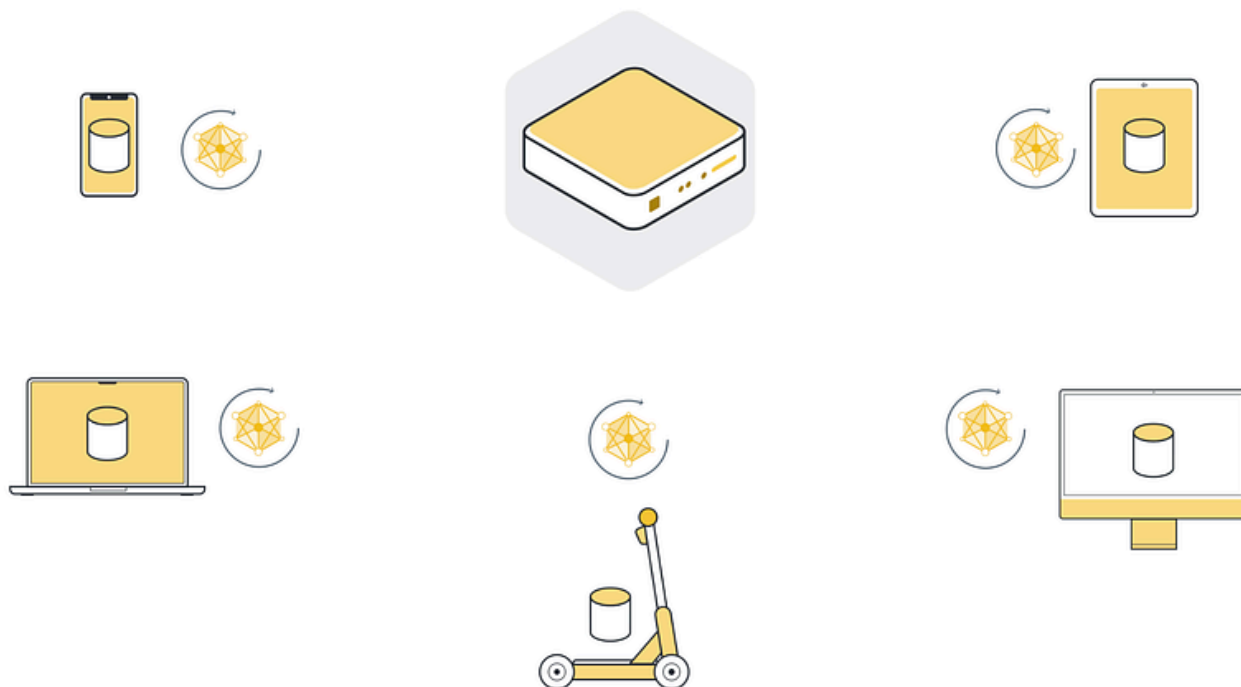
2. Financial Statement Auditing:

Auditing financial statements is a critical process for ensuring the accuracy and reliability of financial reporting. However, audit firms often face challenges in accessing and analyzing client data due to privacy and confidentiality concerns.

A [research paper](#) presented at the International Conference on AI in Finance (ICAIF) in 2022 showcased how Federated Learning, implemented using the Flower framework, can address this challenge. The researchers proposed using FL for anomaly detection in financial statements, allowing audit firms to collaboratively train models without directly sharing client data.

The FL system, built with Flower, enabled each audit client to train a local model on their own financial data. The model, in this case, was an autoencoder, a type of neural network commonly used for anomaly detection. The researchers then used

Flower to aggregate the weights of a specific part of the model which was shown to contain more general information, and this was shared between the participating clients via a central server.



Federated Learning, powered by Flower, allows audit firms to collaboratively train models for anomaly detection in financial statements while preserving client data privacy.

This research demonstrates the potential of FL to enhance the efficiency and effectiveness of financial statement auditing while maintaining the highest levels of data privacy.

3. Insurance Pricing and Risk Assessment:

The insurance industry relies heavily on accurate risk assessment models to determine premiums and manage financial stability. However, individual insurance companies often have limited data, especially for rare events or specific customer segments.

Federated Learning offers a way for insurance companies to pool their data resources without actually sharing the raw data. This allows them to train more robust and accurate pricing models that can better predict the frequency and severity of claims.

A [paper published on SSRN in 2024](#) explored the use of FL for training a claims frequency model, a type of model commonly used in insurance to predict the number of claims made by policyholders. The authors demonstrated that the FL model achieved similar performance to traditional methods that require data sharing, but with the added benefit of privacy preservation.

These examples highlight the transformative potential of Federated Learning in the financial sector. By enabling secure collaboration and data utilization, FL is paving the way for more accurate models, improved services, and enhanced privacy protection.

V. Deploying Federated Learning with AWS

Federated Learning is not just a theoretical concept; it's a practical solution that can be deployed in real-world settings. Amazon Web Services (AWS) provides a robust and scalable platform for building and deploying FL systems, especially for enterprise use cases. Let's explore why AWS is a compelling choice for your FL projects.

A. Why AWS for Federated Learning?

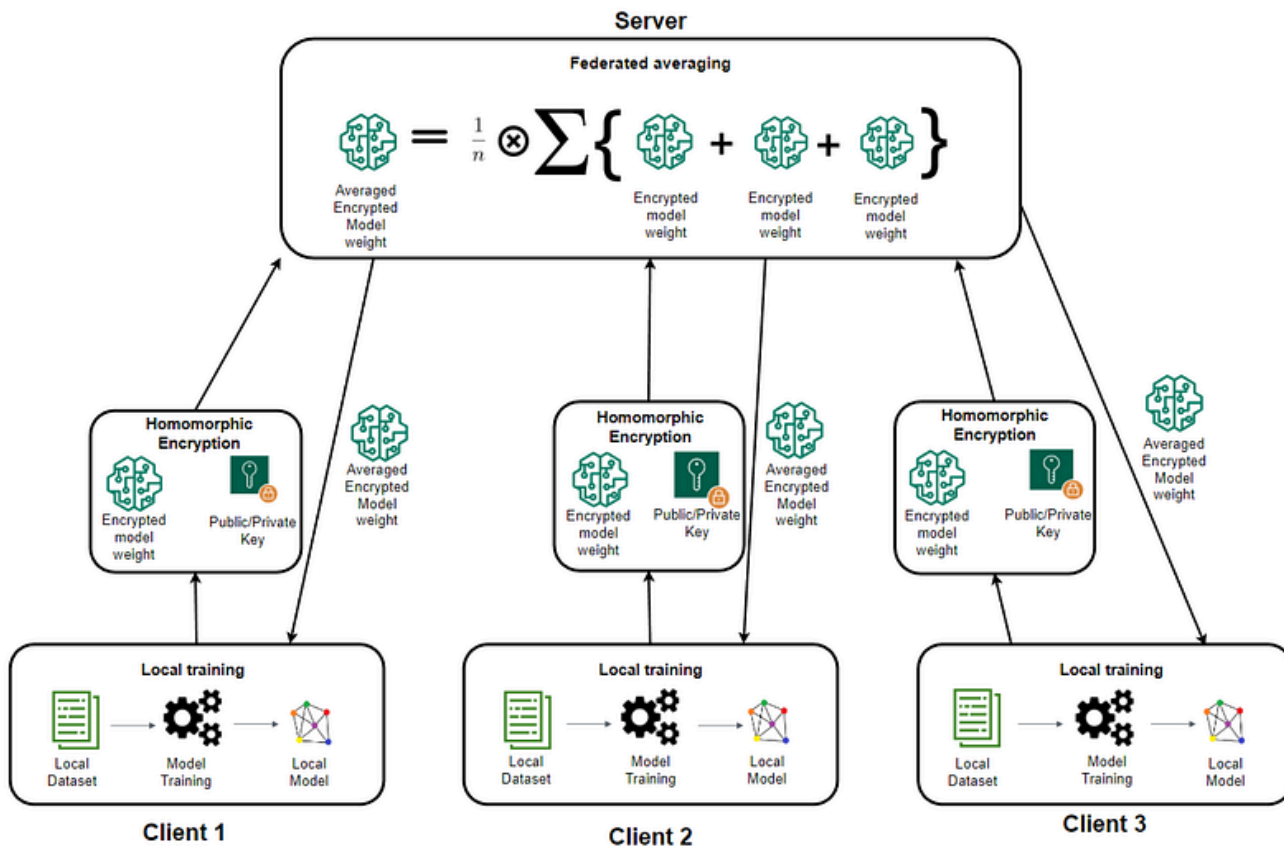
AWS offers a unique combination of features that make it ideal for Federated Learning deployments:

- **Scalability:** AWS's vast infrastructure allows you to scale your FL system to accommodate a large number of clients and massive datasets. You can easily adjust computing resources based on the demands of each training round.
- **Security:** AWS prioritizes security at every level. With features like granular access control, encryption, and robust network security, you can be confident that your FL system is well-protected.
- **Global Reach:** AWS has a global network of data centers, enabling you to deploy your FL system close to your clients, reducing latency and improving performance. This is especially beneficial for cross-border collaborations.
- **Managed Services:** AWS offers a suite of managed services, such as Amazon SageMaker, that simplify the development, deployment, and management of machine learning workloads, including Federated Learning.
- **Cross-Account Collaboration:** AWS supports secure communication and resource sharing between different AWS accounts. This is crucial for

cross-silo FL scenarios where different organizations need to collaborate without compromising data isolation.

B. Deployment Architecture:

Here's a simplified view of a typical FL deployment architecture on AWS:



Practical Diagram of Medical Privacy-Preserving Federated Learning Using Multi-Key Homomorphic Encryption and Flower Framework

<https://www.mdpi.com/2410-387X/7/4/48>

Multiple Client Accounts: Each with a private subnet containing a SageMaker instance running the Flower client and an S3 bucket for local data.

- **Server Account:** With a private subnet containing a SageMaker Notebook instance running the Flower server and an S3 bucket for storing the global model.
- **VPC Peering:** Connections between the server VPC and each client VPC.
- **Communication Flow:** Arrows indicating the flow of model updates between clients and the server.

D. Security and Compliance:

Security is paramount in Federated Learning, especially when dealing with sensitive data in finance and healthcare. AWS provides a comprehensive suite of security features and services to help you build secure FL systems:

- **IAM Roles and Policies:** Granular control over access to AWS resources.
- **Encryption:** Data can be encrypted at rest and in transit using AWS Key Management Service (KMS).
- **Network Security:** VPCs, security groups, and network ACLs provide robust network isolation and access control.
- **Compliance:** AWS complies with various industry standards and regulations, including HIPAA, GDPR, and PCI DSS.

By leveraging these features, you can build FL systems that meet the stringent security and compliance requirements of the finance and healthcare industries.

VII. Conclusion

A. Recap of Key Concepts:

We've covered a lot of ground in this blog post. Let's recap some of the key takeaways:

- **Federated Learning is a revolutionary approach to AI that enables training models on decentralized data without compromising privacy.**
- **It addresses the challenges of data privacy, security, legal compliance, and data volume that often hinder traditional, centralized machine learning.**
- **Federated Learning works by bringing the model to the data, rather than the data to the model.**
- **Clients train local models on their own data and share only model updates with a central server.**
- **The server aggregates the updates to improve a global model, which is then distributed back to the clients.**
- **Frameworks like Flower, TensorFlow Federated, PyTorch, and PySyft simplify the development and deployment of FL systems.**
- **Flower is a powerful, framework-agnostic, and beginner-friendly framework for building FL systems.**
- **AWS provides a scalable, secure, and globally accessible platform for deploying FL, with services like IAM, S3, VPC, and SageMaker.**

B. Future of Federated Learning:

Federated Learning is still a relatively young field, but it's rapidly evolving and gaining momentum. As privacy concerns continue to grow and the amount of data generated at the edge explodes, FL is poised to become an increasingly important part of the AI landscape.

Here are some exciting possibilities for the future of Federated Learning:

- **New Applications:** We'll see FL applied to a wider range of industries and use cases, beyond finance and healthcare, such as manufacturing, transportation, and energy.
- **Advanced Privacy Techniques:** Ongoing research in areas like Differential Privacy, Secure Multi-Party Computation, and Homomorphic Encryption will lead to even stronger privacy guarantees for FL systems.
- **Improved Efficiency:** New algorithms and optimization techniques will reduce communication costs, improve convergence speed, and make FL more efficient.
- **Personalized Models:** FL will enable the development of highly personalized AI models that adapt to individual user preferences and behaviors.
- **Cross-Device and Cross-Silo Collaboration:** We'll see more seamless collaboration between different types of devices (cross-device) and different organizations (cross-silo), unlocking the full potential of decentralized data.

C. Final Thoughts:

Federated Learning represents a paradigm shift in AI development. It empowers us to build intelligent systems that learn from the collective wisdom of decentralized data while respecting individual privacy and security.

The frameworks, tools, and cloud platforms discussed in this blog post are making it easier than ever to get started with Federated Learning. Whether you're a researcher, a developer, or a business leader, now is the time to explore the potential of FL and join the growing community that is shaping the future of AI.

Let's embrace the power of decentralized data and build a future where AI benefits everyone, without compromising our privacy. The journey is just beginning, and the possibilities are limitless.

VIII. Appendix

A. Glossary of Terms:

- **Federated Learning (FL):** A machine learning approach where a model is trained across multiple decentralized devices or servers holding local data samples, without exchanging their data samples.
- **Differential Privacy (DP):** A system for publicly sharing information about a dataset by describing the patterns of groups within the dataset while withholding information about individuals in the dataset.
- **Secure Multi-Party Computation (SMPC):** A cryptographic protocol that allows multiple parties to jointly compute a function over their inputs while keeping those inputs private.
- **Homomorphic Encryption (HE):** A form of encryption that allows computation on ciphertexts, generating an encrypted result which, when decrypted, matches the result of the operations as if they had been performed on the plaintext.
- **TensorFlow Federated (TFF):** An open-source framework for machine learning and other computations on decentralized data, built by Google.
- **PySyft:** An open-source framework for secure and private deep learning, extending PyTorch and TensorFlow with capabilities for federated learning, differential privacy, and encrypted computation.
- **Flower (flwr):** A friendly federated learning framework that is designed to be framework-agnostic, scalable, and customizable.

B. Further Reading:

- [Federated Learning: Collaborative Machine Learning without Centralized Training Data](#) (Google AI Blog)
- [Advances and Open Problems in Federated Learning](#) (Research Paper)
- [TensorFlow Federated: Machine Learning on Decentralized Data](#) (TensorFlow Website)
- [PySyft: Private Deep Learning](#) (GitHub Repository)
- [The Algorithmic Foundations of Differential Privacy](#) (Book)

Thank you for reading.

Hope this article helps you !

The Modern Data Stack: Understanding Data Pipelines, ETL, and Data Warehousing

In today's data-driven world, businesses are inundated with information from a plethora of sources. To harness the power of this data, organizations are turning to the "Modern Data Stack" a modular, scalable, and cloud-based approach to data management. But what exactly is the Modern Data Stack? At its core, it is a collection of integrated tools and technologies designed to streamline data workflows, enabling organizations to efficiently collect, process, store, and analyze vast amounts of data.

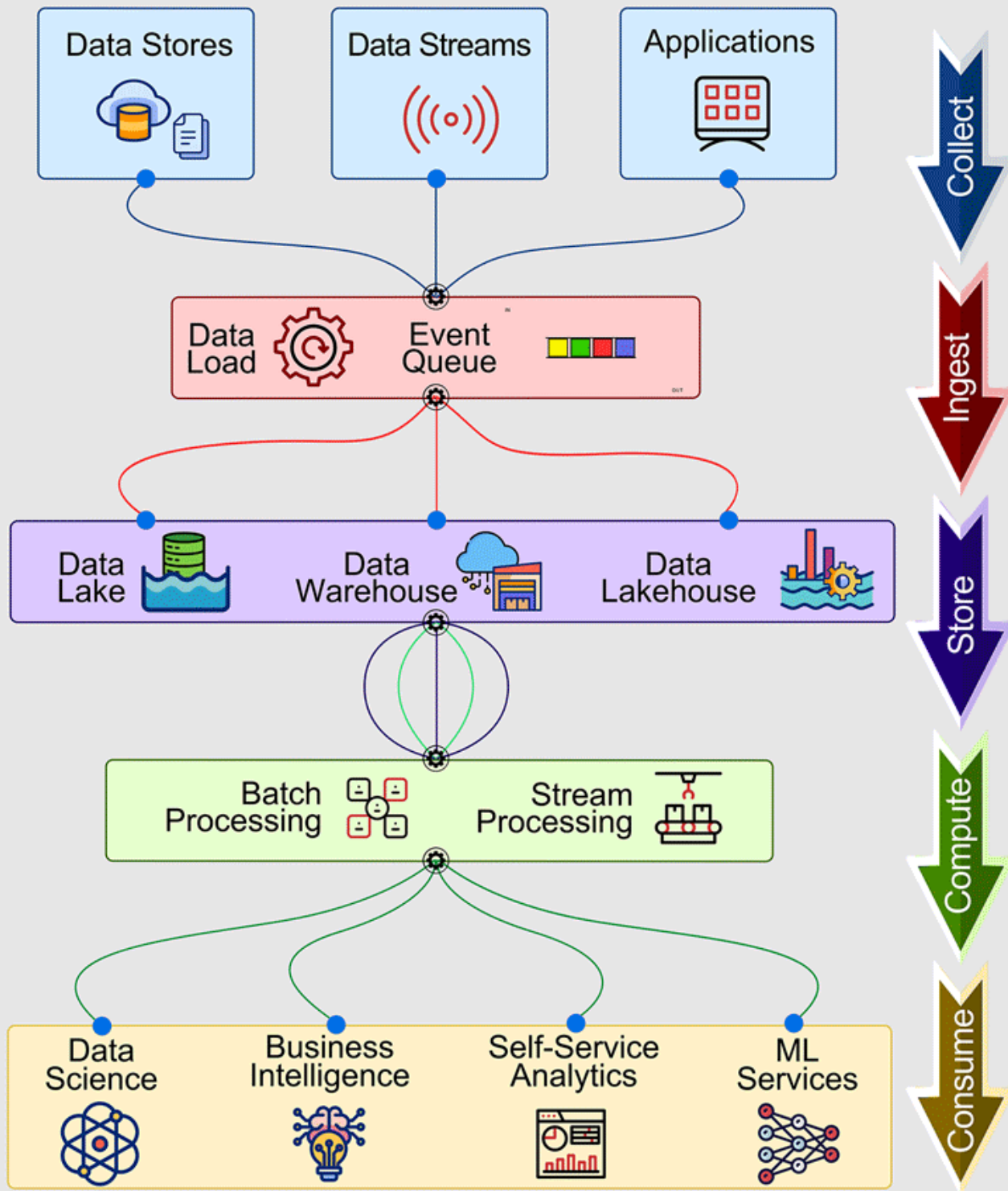
This blog will guide you through the foundational elements of the Modern Data Stack, exploring:

- **What makes the Modern Data Stack unique:** A brief overview of its modularity, scalability, and flexibility.
- **The role of data pipelines:** How robust workflows ensure seamless data integration and transformation.
- **ETL vs. ELT processes:** A comparison of traditional and modern data processing methodologies.
- **Building a data pipeline:** Step-by-step insights into sourcing, preprocessing, ingesting, and automating data workflows.
- **Data warehousing essentials:** Key architectural principles like dimensional modeling and schema design.
- **Governance and security:** Best practices for ensuring data integrity, compliance, and trust.
- **Tool selection and ecosystem:** From Flask and Airflow to Snowflake and Redshift, a look at popular tools for pipeline orchestration and warehousing.
- **Emerging trends:** The future of data pipelines, including DataOps, AI-driven automation, and real-time streaming.

By the end, you will gain a comprehensive understanding of how the Modern Data Stack empowers organizations to unlock the true potential of their data while keeping pace with evolving technological demands.

Data Pipeline Overview

ByteByteGo



<https://x.com/alexxybyte/status/1725192554671640735>

1. Introduction to the Modern Data Stack

The Modern Data Stack represents a paradigm shift in how we handle data. Unlike traditional, monolithic architectures, it leverages the flexibility and scalability of the cloud, along with a modular design that allows organizations to choose the best tools for each stage of the data lifecycle.

Key Components of the Modern Data Stack:

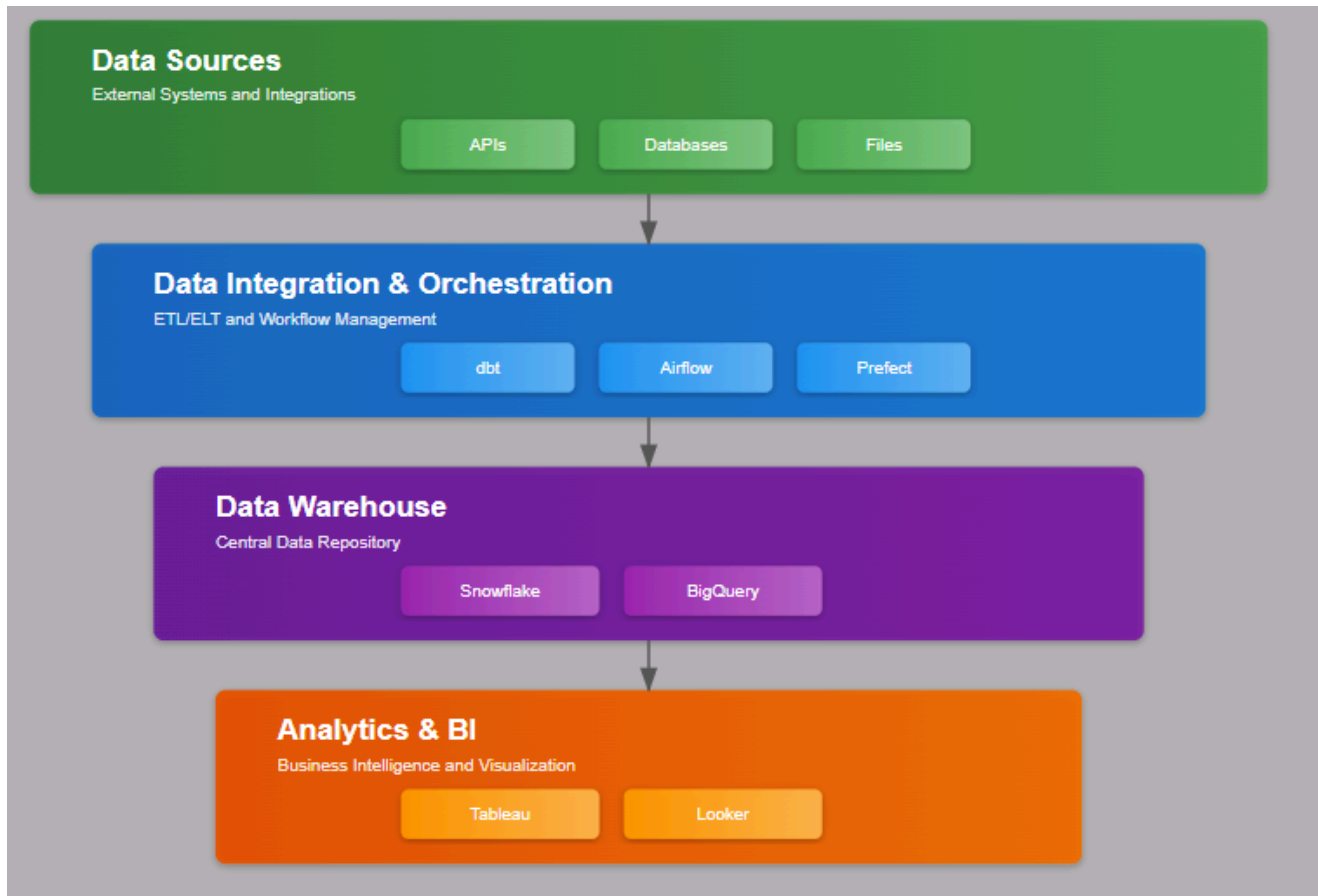
- **Cloud-Based:** Leverages cloud services for storage, compute, and scalability.
- **Modular:** Composed of independent, interchangeable tools for each stage (ingestion, transformation, storage, visualization).
- **Scalable:** Can handle increasing volumes of data without significant performance degradation.

At the heart of this stack lies the data warehouse, serving as the central repository for analytical data. However, to fuel this warehouse, we need robust data pipelines and efficient ETL/ELT processes. These pipelines act as the circulatory system, ensuring a continuous flow of data from various sources to the warehouse, where it's transformed and made ready for analysis.

The Critical Role of Data Pipelines and ETL/ELT:

- **Data Integration:** Pipelines bring together data from disparate sources.
- **Data Transformation:** ETL/ELT processes clean, standardize, and enrich the data.
- **Analytical Readiness:** Data is structured in the warehouse for efficient querying and reporting.

In today's fast-paced business environment, decisions are increasingly driven by real-time or near real-time insights. A well-designed data stack with robust pipelines is essential for achieving this agility.



Layered Diagram of the Modern Data Stack

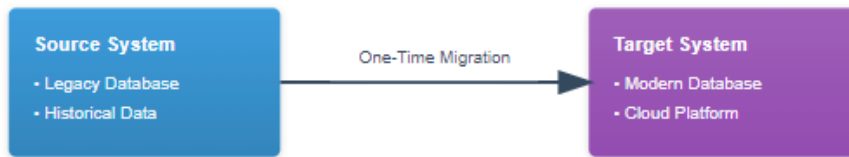
2. Data Migration vs. ETL: Key Differences

Before diving into pipeline construction, it's crucial to clarify the distinction between data migration and ETL (Extract, Transform, Load) or its modern variant, ELT (Extract, Load, Transform). These terms are often used interchangeably, but they represent distinct processes with different objectives.

Data Migration: Data migration is typically a one-time or infrequent event involving moving data from one storage system to another. This could be triggered by a database upgrade, a shift to a new platform, or a consolidation of data sources. The focus is on transferring data with minimal changes, ensuring a smooth transition to the new system.

Data Migration

One-time process with minimal transformation

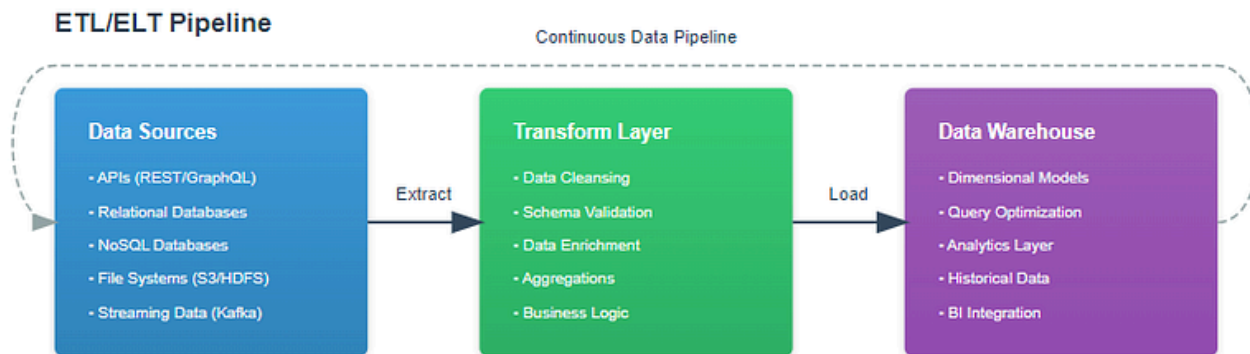


Flow Chart Illustrating Data Migration

ETL/ELT: ETL/ELT, on the other hand, are ongoing processes designed to feed data warehouses. They involve:

- **Extract:** Pulling data from multiple sources (databases, APIs, files, etc.).
- **Transform:** Cleaning, validating, aggregating, and enriching the data.
- **Load:** Writing the transformed data into the data warehouse.

ELT has gained popularity in the modern data stack due to the scalability of cloud data warehouses. It allows for loading raw data first and then performing transformations within the warehouse, leveraging its computational power.



Flow Chart Illustrating ETL

3. Building a Pipeline: From Source to Warehouse

Constructing a data pipeline is a multi-faceted process that requires careful planning and execution. Here's a high-level methodology:

a) Source Identification: The first step is to identify all the sources that will feed the data warehouse.

These can include:

- **Operational Databases:** MySQL, PostgreSQL, etc.
- **APIs:** Data from third-party applications (e.g., Salesforce, Google Analytics).
- **Files:** CSV, JSON, Parquet files, etc.
- **Streaming Sources:** IoT devices, web clickstreams, etc.
- **Legacy Systems** Mainframe applications
- **CRM and ERP:** Customer relations management systems, Enterprise resource planning systems.

b) Data Preprocessing: Preprocessing ensures that raw data is cleaned, structured, and enriched before further transformations.

Key preprocessing steps include:

- **Data Cleaning:** Handle missing values, correct errors, and remove duplicates to ensure consistency and accuracy.
- **Data Transformation:** Convert data into a standardized format, such as normalizing date formats or unifying units of measurement.
- **Data Reduction:** Employ aggregation, sampling, or filtering techniques to reduce data volume while retaining essential information.
- **Data Enrichment:** Add contextual details, such as geocoding addresses or appending demographic data, to enhance analytical depth.

c) Ingestion Mechanism: Next, you need to choose an ingestion mechanism based on the nature of the data and the desired latency:

- **Batch Ingestion:** Data is collected and processed in batches at scheduled intervals (e.g., daily, hourly).
- **Streaming Ingestion:** Data is ingested in real-time as it's generated.

d) Tools for Ingestion and Orchestration: The choice of tools depends on the complexity of the pipeline and the technical expertise of your team:

- **Custom Scripts:** For simpler tasks, you can use scripting languages like Python (with libraries like **requests** for API calls and **csv** for file handling) to build custom ingestion scripts.

- **Flask:** A lightweight Python web framework that can be used to build APIs for data ingestion.
- **Orchestration Tools:** For more complex pipelines, workflow orchestration tools like Airflow, Prefect, or Dagster provide robust scheduling, monitoring, and error-handling capabilities.

e) Transformation/Preprocessing: Once ingested, data often needs to be transformed to fit the structure of the data warehouse and meet analytical requirements.

This can involve:

- **Data Cleaning:** Handling missing values, correcting errors, removing duplicates.
- **Data Validation:** Ensuring data conforms to predefined rules and constraints.
- **Data Formatting:** Standardizing data types, units, and formats.
- **Data Enrichment:** Adding contextual information from other sources.

f) Loading to Warehouse: Finally, the transformed data is loaded into the data warehouse.

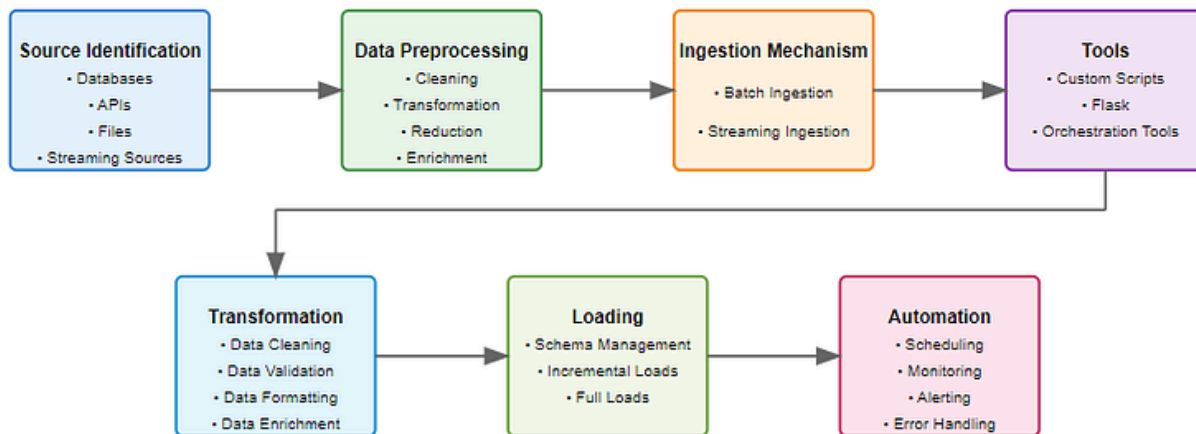
Key considerations include:

- **Schema Management:** Maintaining the integrity of the warehouse schema (data types, constraints).
- **Incremental Loads:** Loading only new or changed data to optimize performance.
- **Full Loads:** Periodically reloading the entire dataset to ensure consistency.

g) Automating the Data Pipeline: Automation ensures reliability, efficiency, and scalability.

Key aspects include:

- **Scheduling:** Use orchestration tools to define execution intervals, ensuring timely updates.
- **Monitoring:** Track errors, performance metrics, and data quality.
- **Alerting:** Notify stakeholders of issues for prompt resolution.
- **Error Handling:** Implement retry mechanisms and fallback procedures to address failures.



Detailed Pipeline Flow

4. Data Warehousing Fundamentals

A data warehouse is a specialized database designed to support complex analytical queries and reporting. Unlike operational databases optimized for transactional processing, data warehouses focus on large-scale analysis across historical data.

Key Characteristics of Data Warehouses:

- **Subject-Oriented:** Organized around specific business domains, such as customers, products, or sales.
- **Integrated:** Consolidates data from disparate sources, standardizing formats and definitions.
- **Time-Variant:** Tracks historical data, enabling trend analysis and forecasting.
- **Non-Volatile:** Preserves historical data integrity, ensuring no updates or deletions alter past records.

Dimensional Modeling:

Dimensional modeling simplifies data organization for analytical queries. Common approaches include:

- **Star Schema:** A straightforward design with a central fact table linked to dimension tables.

- **Snowflake Schema:** A normalized structure where dimension tables are further broken down into sub-dimensions.

5. Data Governance and Security

Effective data governance and security are essential for ensuring data integrity, compliance, and trust.

Data Governance:

- **Data Ownership:** Assign responsibility for data accuracy, quality, and accessibility.
- **Data Quality Rules:** Define standards for completeness, consistency, and validity.
- **Data Dictionary:** Maintain metadata describing datasets, columns, and relationships.
- **Data Lineage:** Track data's origin, transformations, and movements.

Data Security:

- **Role-Based Access Control (RBAC):** Restrict access to authorized users and systems.
- **Data Encryption:** Safeguard data in transit and at rest using encryption standards.
- **Audit Logging:** Monitor data access and modifications for compliance and security.

6. Cloud vs. On-Premises Data Warehouses

Choosing between a cloud-based and an on-premises data warehouse depends on various factors, including cost, scalability, security, and compliance requirements.

Cloud Data Warehouses (e.g., Redshift, Snowflake, BigQuery):

Pros:

- **Scalability:** Easily scale resources up or down based on demand.
- **Pay-as-you-go Pricing:** Pay only for the resources you use.
- **Managed Services:** The cloud provider handles infrastructure management.

Cons:

- **Potential Compliance Issues:** May not meet specific regulatory requirements.
- **Data Egress Costs:** Charges for moving data out of the cloud.
- **Vendor Lock-in:** Switching to a different provider can be challenging.

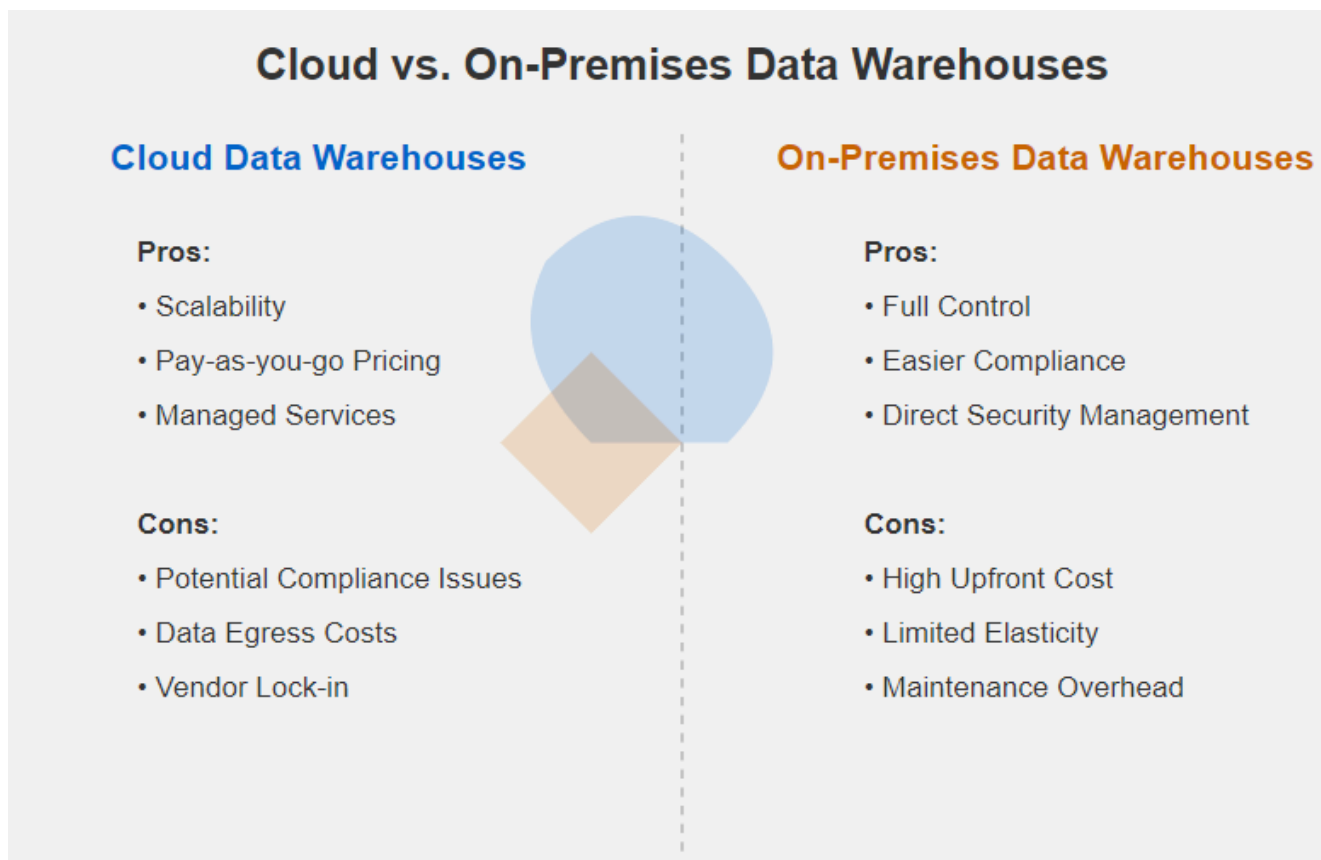
On-Premises Data Warehouses:

Pros:

- **Full Control:** Complete control over the environment and security.
- **Compliance:** Easier to meet specific regulatory requirements.

Cons:

- **High Upfront Cost:** Significant investment in hardware and infrastructure.
- **Limited Elasticity:** Scaling resources can be time-consuming and expensive.
- **Maintenance Overhead:** Requires dedicated IT staff for maintenance and management.



Cloud vs. On-Premise Comparison

7. Real-Time vs. Batch Processing

The choice between real-time (streaming) and batch processing depends on the specific use case and the required data latency.

Real-Time Processing:

- **Use Cases:** IoT sensor data, financial transactions, fraud detection, real-time dashboards.
- **Technologies:** Kafka, AWS Kinesis, Spark Streaming, Flink.
- **Advantages:** Provides immediate insights, enables real-time decision-making.
- **Disadvantages:** More complex to implement and manage, can be more expensive.

Batch Processing:

- **Use Cases:** Periodic reporting, data warehousing ETL/ELT, large-scale data analysis.
- **Technologies:** Airflow, Cron, Hadoop, Spark.
- **Advantages:** Simpler to implement, cost-effective for large datasets.
- **Disadvantages:** Data latency (delay between data generation and availability for analysis).

8. Flask vs. Airflow

When building data pipelines, choosing the right tools for ingestion and orchestration is crucial. Flask and Airflow are two popular options, but they serve different purposes and have distinct strengths and weaknesses.

Flask:

- **Strengths:** Ideal for lightweight, custom ingestion pipelines with minimal dependencies.
- **Limitations:** Lacks built-in orchestration and monitoring features, making it less suitable for complex workflows.

Airflow:

- **Strengths:** Designed for complex workflows with robust orchestration, scalability, and monitoring capabilities.

- **Limitations:** Higher learning curve and resource requirements compared to Flask.

When to Use Flask:

- **Simple Ingestion APIs:** When you need to create a simple API endpoint for data ingestion.
- **Small to Medium-Sized Pipelines:** When the pipeline has a limited number of tasks and dependencies.
- **Custom Solutions:** When you need a high degree of control and customization over the ingestion process.

When to Use Airflow:

- **Complex Workflows:** When the pipeline involves many tasks with complex dependencies.
- **Scheduled and Automated Pipelines:** When you need to schedule and automate pipeline execution.
- **Monitoring and Logging:** When detailed monitoring and logging of pipeline execution are required.
- **Scalable Pipelines:** When the pipeline needs to handle large volumes of data and scale to meet growing demands.

Flask vs. Airflow: Data Pipeline Tools

Flask

Strengths:

- Simplicity
- Flexibility
- Lightweight

Weaknesses:

- Limited Functionality
- Scalability Challenges
- Manual Orchestration

Simple APIs & Small Pipelines

Airflow

Strengths:

- Robust Orchestration
- Scalability
- Monitoring and Logging
- Extensibility

Weaknesses:

- Steeper Learning Curve
- Resource Intensive
- Complex Setup

Complex Workflows & Large Pipelines

Flask vs Airflow Comparison Table

9. Popular Data Pipeline Tools & Their Ecosystem

Beyond Flask and Airflow, the data pipeline ecosystem is rich with tools and platforms that cater to different needs and technical preferences.

Data Integration Platforms:

- **Stitch:** A cloud-based ELT platform that simplifies data extraction and loading from various sources to data warehouses.
- **Fivetran:** Similar to Stitch, Fivetran automates data replication with pre-built connectors and minimal configuration.
- **Matillion:** An ETL tool designed for cloud data warehouses, offering a visual interface for building data transformation pipelines.

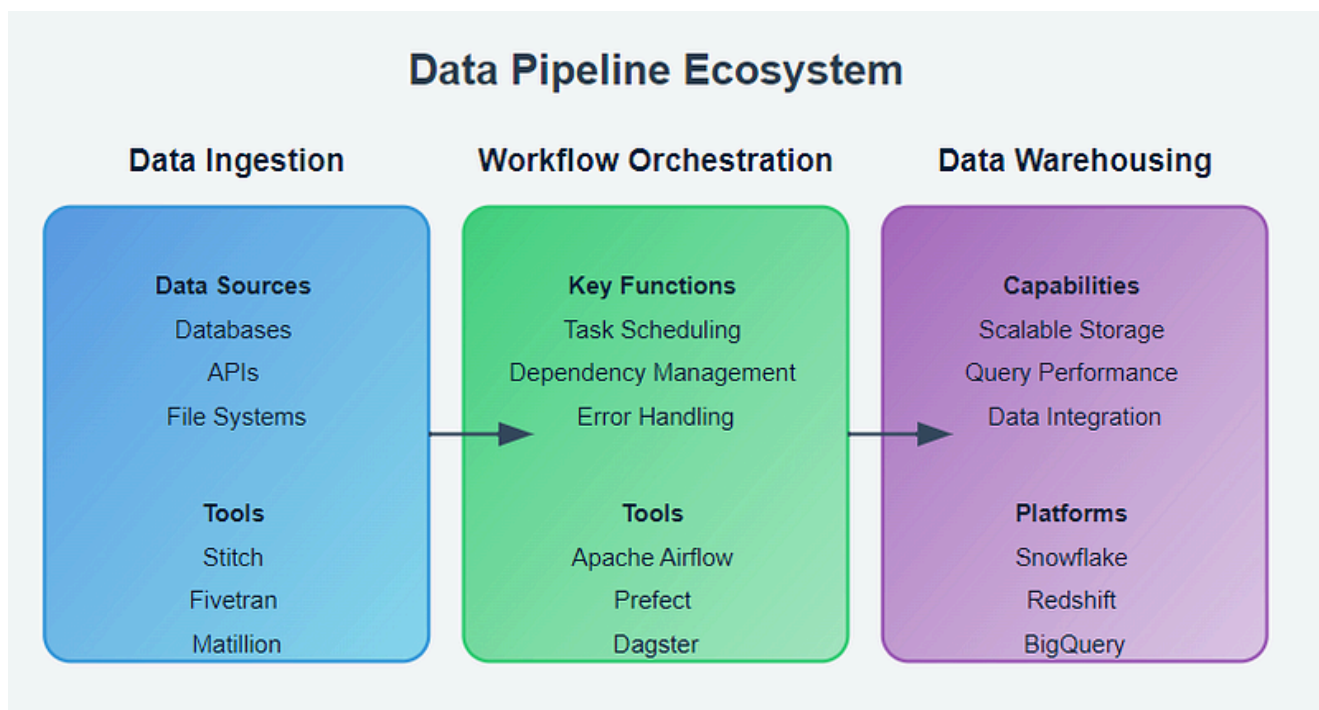
Workflow Orchestration:

- **Prefect:** A modern workflow orchestration tool that emphasizes dynamic, parameterizable workflows and a hybrid execution model.

- **Dagster:** A data orchestrator that focuses on development, testing, and observability of data pipelines.

Cloud-Based Data Warehouses:

- **Snowflake:** A cloud-native data warehouse known for its scalability, ease of use, and separation of compute and storage.
- **Amazon Redshift:** A fully managed, petabyte-scale data warehouse service offered by AWS.
- **Google BigQuery:** A serverless, highly scalable, and cost-effective multi-cloud data warehouse designed for business agility.



Tool Ecosystem Diagram

10. The Future of Data Pipelines

The field of data engineering is constantly evolving, and several trends are shaping the future of data pipelines:

- **DataOps:** Applying DevOps principles to data pipelines, emphasizing automation, continuous integration, and continuous delivery to accelerate the development and deployment of data pipelines.

- **Real-Time Data Streaming:** The increasing adoption of event-driven architectures and real-time analytics is driving the growth of streaming data pipelines.
- **AI/ML Integration:** AI and ML are being used to automate tasks like anomaly detection, data classification, and data quality improvement within data pipelines. Also, the rise of MLOps helps streamline deploying machine learning models into production and monitoring.
- **Composable Data Stack:** Microservices architecture is starting to be adopted for data pipelines, allowing for a more flexible and modular approach to data integration.

11. Conclusion and Best Practices

Best Practices:

- **Start Small and Iterate:** Begin with manageable projects and expand as capabilities mature.
- **Prioritize Governance and Security:** Implement robust policies and controls to protect data and ensure compliance.
- **Automate Workflows:** Use orchestration tools to improve efficiency and reduce manual effort.
- **Monitor and Optimize:** Continuously track performance and identify bottlenecks.
- **Document Processes:** Maintain clear documentation for maintainability and collaboration.

By embracing these principles, organizations can build modern data stacks that empower them to derive actionable insights and drive strategic decisions with confidence.

Movie Recommender System: EDA and Building Baseline System (BoxRAG Part 1)

Welcome to the first installment of the BoxRAG project series! In this blog post, we'll delve into the foundational aspects of building a movie recommender system

using Exploratory Data Analysis (EDA) and traditional recommendation algorithms. We'll also set the stage for integrating advanced Retrieval-Augmented Generation (RAG) techniques with Mistral 7B, a powerful language model. By the end of this post, you'll have a comprehensive understanding of the dataset, the insights derived from EDA, and the implementation of baseline recommendation systems without leveraging RAG.



Photo by [Thibault Penin](#) on [Unsplash](#)

1. Introduction

1.1. Welcome to BoxRAG

Welcome to the BoxRAG project series! BoxRAG stands for Box Office Retrieval-Augmented Generation, a project dedicated to developing an intelligent movie recommender system. This series will guide you through the journey of transforming raw movie data into insightful and personalized recommendations using state-of-the-art techniques.

1.2. Importance of Movie Recommendation Systems

In today's digital age, the sheer volume of available movies and series can be overwhelming. Effective recommendation systems play a crucial role in enhancing user experience by helping users discover content tailored to their preferences. These systems not only increase user engagement but also drive satisfaction and loyalty.

1.3. Blog Objectives

In this first part of the BoxRAG series, we will:

- Conduct comprehensive Exploratory Data Analysis (EDA) on the [MovieLens dataset](#).
 - Implement traditional (No-RAG) recommendation systems: Popularity-Based, Content-Based, and Collaborative Filtering.
 - Lay the groundwork for integrating Retrieval-Augmented Generation (RAG) with Mistral 7B in future installments while we .
-

2. Project Setup

2.1. Tools and Technologies

To build the BoxRAG recommender system, we utilized a robust set of tools and libraries:

- Data Processing & Analysis: Pandas, NumPy
- Visualization: Matplotlib, Seaborn, WordCloud
- Machine Learning: Scikit-learn
- Recommendation Systems: Custom Python functions
- Backend: Flask
- Frontend: React
- Vector Stores: FAISS
- Language Models: Mistral 7B (Other Smaller LLMs will be used in future parts)
- Others: Docker for containerization

2.2. Dataset Overview

We're using [The Movies Dataset](#), which provides metadata for over 45,000 movies along with 26 million ratings from more than 270,000 users. This rich dataset includes detailed information such as cast, crew, plot keywords, budget, revenue, posters, release dates, languages, production companies, and more.

Dataset Files:

- **movies_metadata.csv**: Core metadata for 45,000 movies.
- **keywords.csv**: Plot keywords for movies.
- **credits.csv**: Cast and crew information.
- **links.csv & links_small.csv**: TMDb and IMDb IDs for movies.
- **ratings.csv & ratings_small.csv**: User ratings data.

2.3. Folder Structure

Maintaining an organized project structure is vital for scalability and maintainability. Here's an overview of our project directory:

BoxOfficeRAG/

— backend/	# Core backend services
— notebooks/	# Analysis and dump notebooks
— data/	# Data storage and processing
— docker/	# Containerization
— prompts/	# LLM prompt templates

3. Loading and Inspecting the Data

Before diving into analysis, it's essential to load and understand the structure of our datasets.

3.1. Initial Data Inspection

Understanding the first few rows of each dataset helps in grasping their structure and identifying key features.

Column name and Shape for each DataFrame.

column_names_shape(mov_data)

Datasets Files Column Names and Shape

Datasets Name credits.csv

['cast', 'crew', 'id', 'directors', 'cast_names']

[45439, 5]

Datasets Name keywords.csv

['id', 'keywords', 'keyword_names']

[45432, 3]

Datasets Name links.csv

['movied', 'imdbid', 'tmdbid']

[45843, 3]

Datasets Name links_small.csv

['movied', 'imdbid', 'tmdbid']

[9125, 3]

Datasets Name movies_metadata.csv

['adult', 'belongs_to_collection', 'budget', 'genres', 'homepage', 'id',

```
'imdb_id', 'original_language', 'original_title', 'overview', 'popularity',  
'poster_path', 'production_companies', 'production_countries',  
'release_date', 'revenue', 'runtime', 'spoken_languages', 'status', 'tagline',  
'title', 'video', 'vote_average', 'vote_count', 'genre_names', 'release_year',  
'cluster']
```

```
[45449, 27]
```

Datasets Name ratings.csv

```
['userId', 'movieId', 'rating', 'timestamp']
```

```
[26024289, 4]
```

Datasets Name ratings_small.csv

```
['userId', 'movieId', 'rating', 'timestamp']
```

```
[100004, 4]
```

3.2. Handling Missing Values and Duplicates

Data often contains missing values or duplicates that can skew analysis. Addressing these issues ensures data integrity.

Summarize missing values for each DataFrame.

```
missing_summary = summarize_missing_values(mov_data)
```

```
print("Missing Values Summary:\n", missing_summary)
```

Missing Values Summary:

	credits.csv	keywords.csv	links.csv	links_small.csv	\
belongs_to_collection	NaN	NaN	NaN	NaN	
homepage	NaN	NaN	NaN	NaN	
imdb_id	NaN	NaN	NaN	NaN	
original_language	NaN	NaN	NaN	NaN	
overview	NaN	NaN	NaN	NaN	
popularity	NaN	NaN	NaN	NaN	
poster_path	NaN	NaN	NaN	NaN	
production_companies	NaN	NaN	NaN	NaN	
production_countries	NaN	NaN	NaN	NaN	
release_date	NaN	NaN	NaN	NaN	
revenue	NaN	NaN	NaN	NaN	
runtime	NaN	NaN	NaN	NaN	
spoken_languages	NaN	NaN	NaN	NaN	
status	NaN	NaN	NaN	NaN	
tagline	NaN	NaN	NaN	NaN	
title	NaN	NaN	NaN	NaN	
tmdbld	NaN	NaN	219.0	13.0	
video	NaN	NaN	NaN	NaN	
vote_average	NaN	NaN	NaN	NaN	

vote_count	NaN	NaN	NaN	NaN
------------	-----	-----	-----	-----

movies_metadata.csv	ratings.csv	ratings_small.csv
---------------------	-------------	-------------------

belongs_to_collection	40972.0	NaN	NaN
-----------------------	---------	-----	-----

homepage	37684.0	NaN	NaN
----------	---------	-----	-----

imdb_id	17.0	NaN	NaN
---------	------	-----	-----

original_language	11.0	NaN	NaN
-------------------	------	-----	-----

overview	954.0	NaN	NaN
----------	-------	-----	-----

popularity	5.0	NaN	NaN
------------	-----	-----	-----

poster_path	386.0	NaN	NaN
-------------	-------	-----	-----

production_companies	3.0	NaN	NaN
----------------------	-----	-----	-----

production_countries	3.0	NaN	NaN
----------------------	-----	-----	-----

release_date	87.0	NaN	NaN
--------------	------	-----	-----

revenue	6.0	NaN	NaN
---------	-----	-----	-----

runtime	263.0	NaN	NaN
---------	-------	-----	-----

spoken_languages	6.0	NaN	NaN
------------------	-----	-----	-----

status	87.0	NaN	NaN
--------	------	-----	-----

tagline	25054.0	NaN	NaN
---------	---------	-----	-----

title	6.0	NaN	NaN
-------	-----	-----	-----

tmdbId	NaN	NaN	NaN
--------	-----	-----	-----

video	6.0	NaN	NaN
-------	-----	-----	-----

vote_average	6.0	NaN	NaN
vote_count	6.0	NaN	NaN

4. Exploratory Data Analysis (EDA)

EDA provides valuable insights into the dataset, uncovering patterns and relationships that inform our recommendation strategies.

4.1. Distribution of Ratings

Understanding how users rate movies helps in identifying overall satisfaction levels.

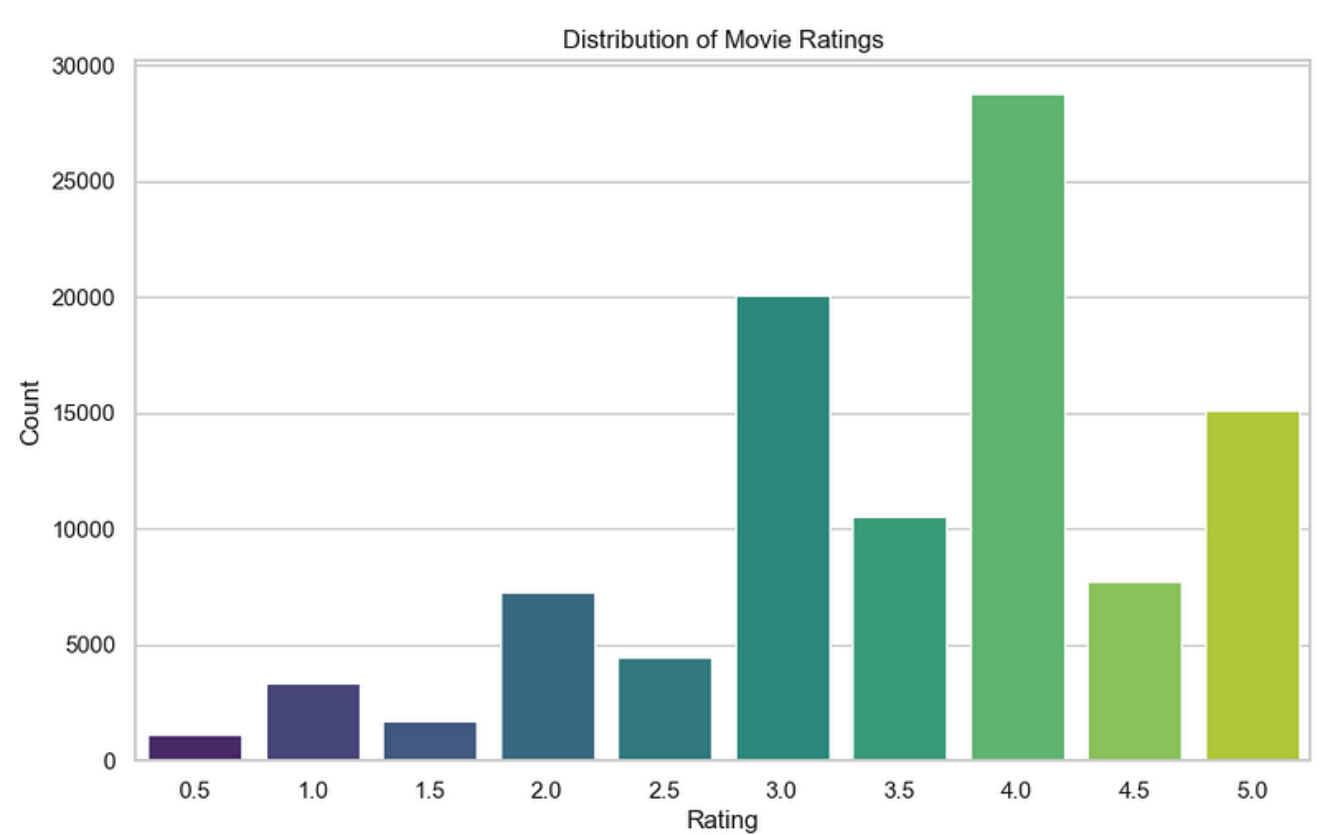


Figure 1: Distribution of Movie Ratings

Most ratings cluster around 3–4 stars, indicating a general satisfaction among users.

4.2. Popular Genres

Identifying the most and least popular genres guides the focus of our recommendations.

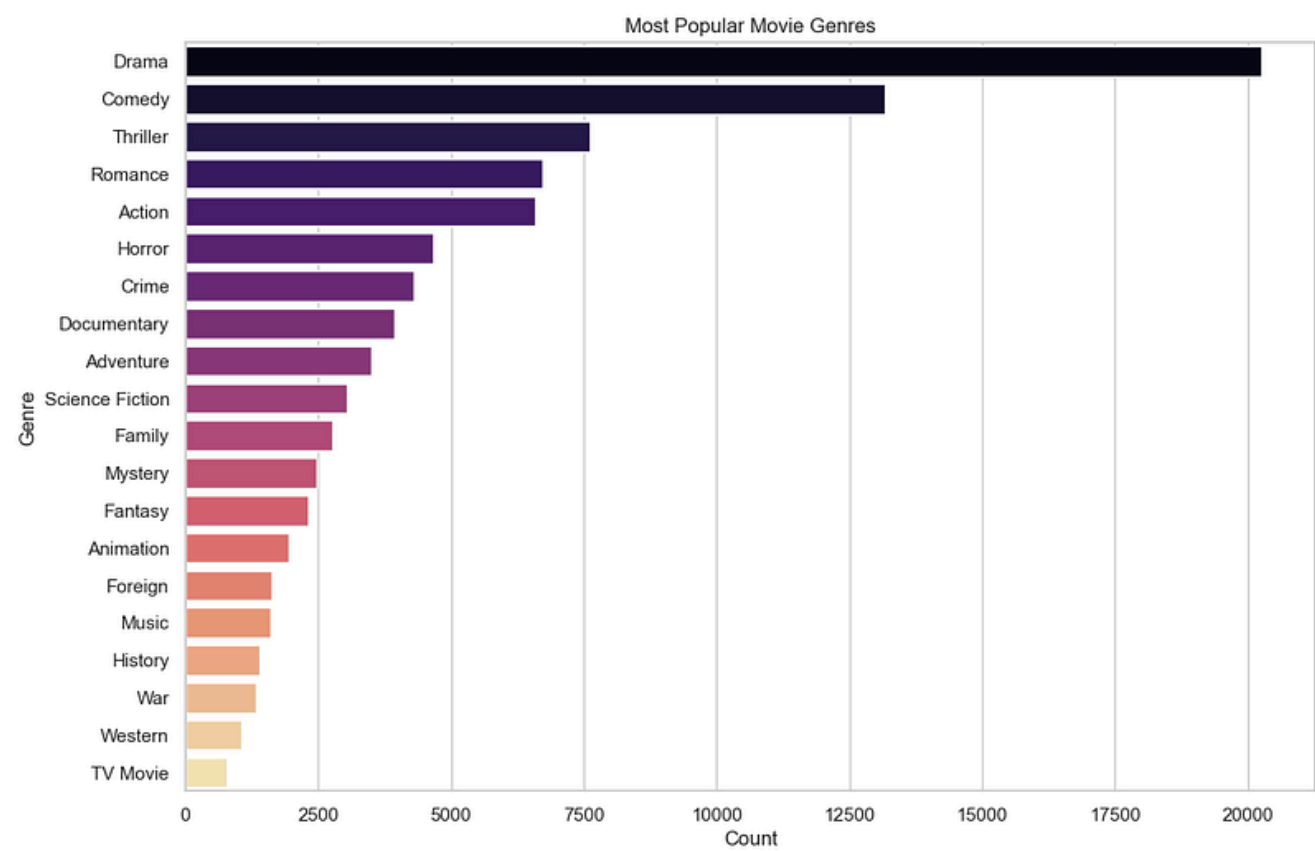


Figure 2: Popularity of Movie Genres

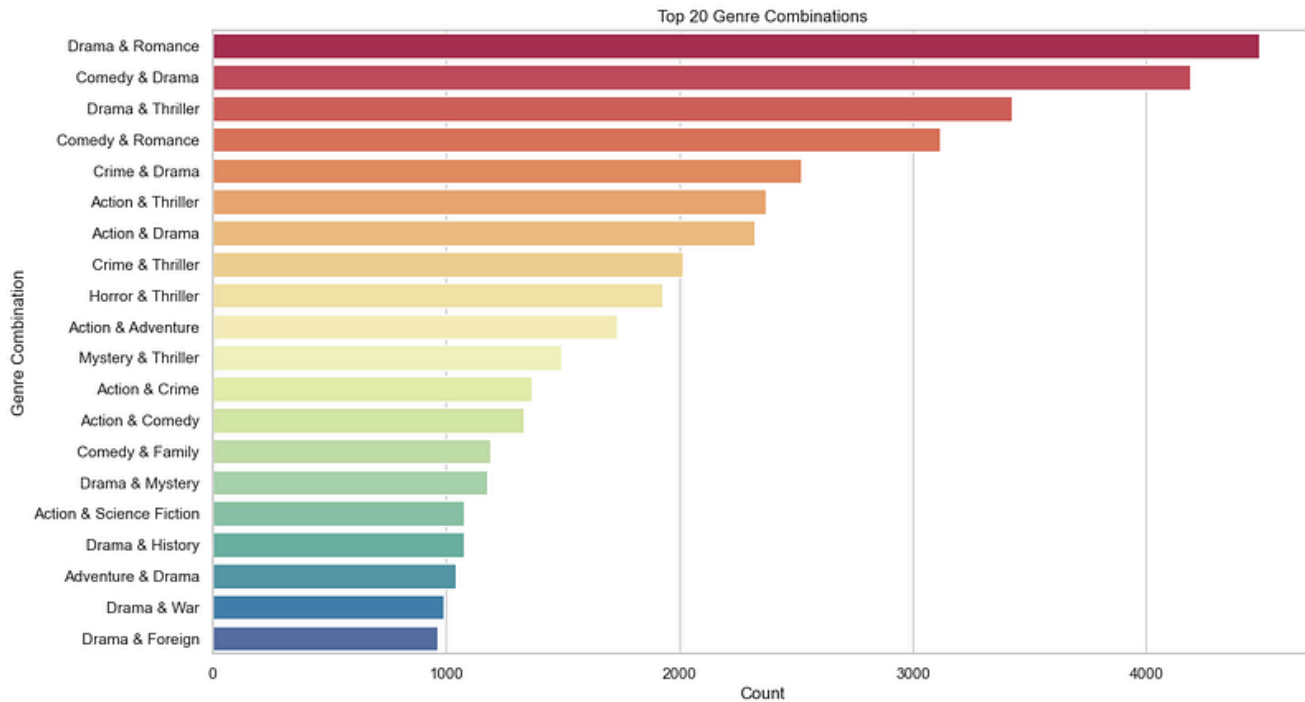


Figure 2.1: Popularity of Genres Combinations

Genres like Drama, Comedy and Thriller dominate the dataset, while niche genres have fewer entries.

4.3. Budget vs. Revenue Analysis

Analyzing the relationship between a movie's budget and its revenue can highlight which genres or types of movies are financially successful.

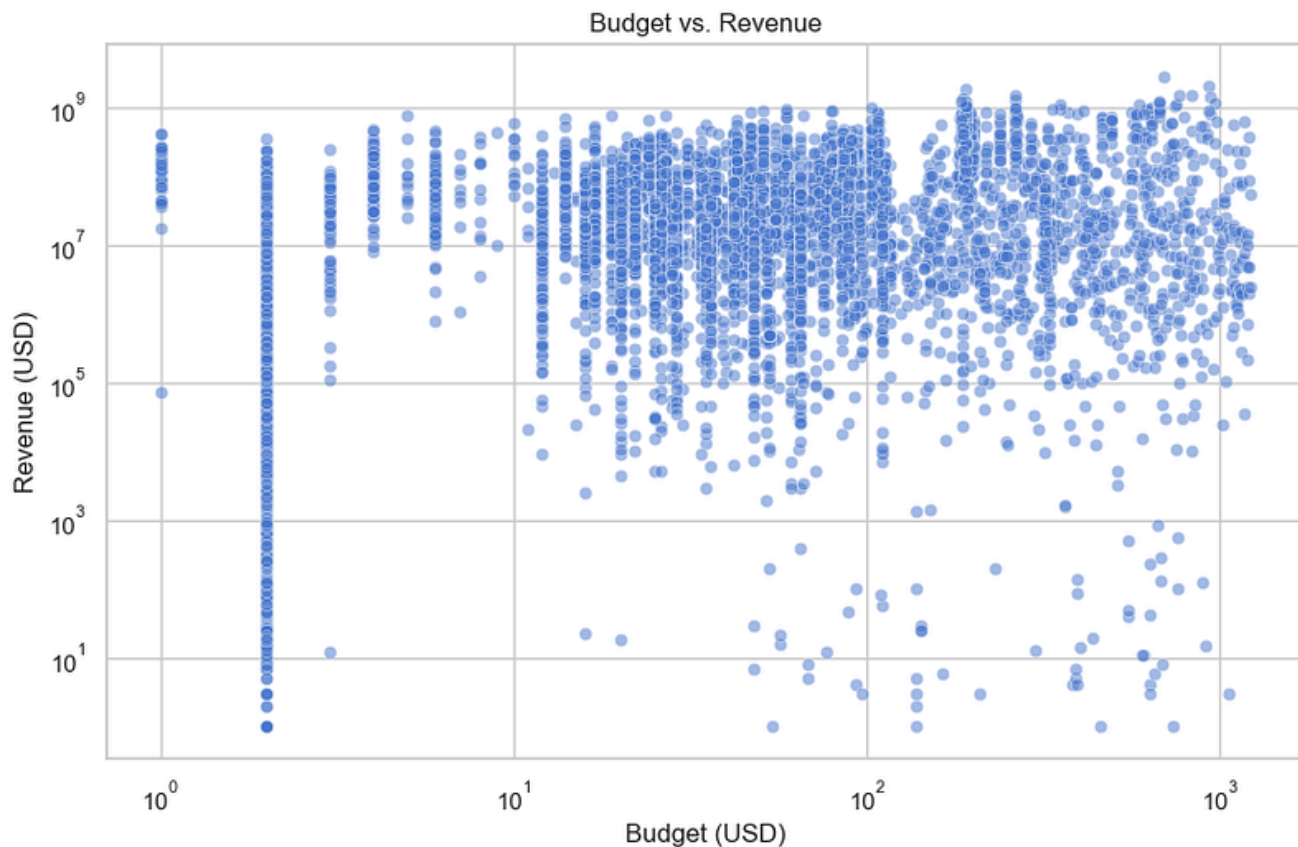


Figure 3: Budget vs. Revenue of Movies

There's a positive correlation between budget and revenue, though not all high-budget movies perform well.

4.4. Top Directors and Actors

Identifying influential directors and popular actors can help in content-based recommendations.

Top 20 Directors by Number of Movies

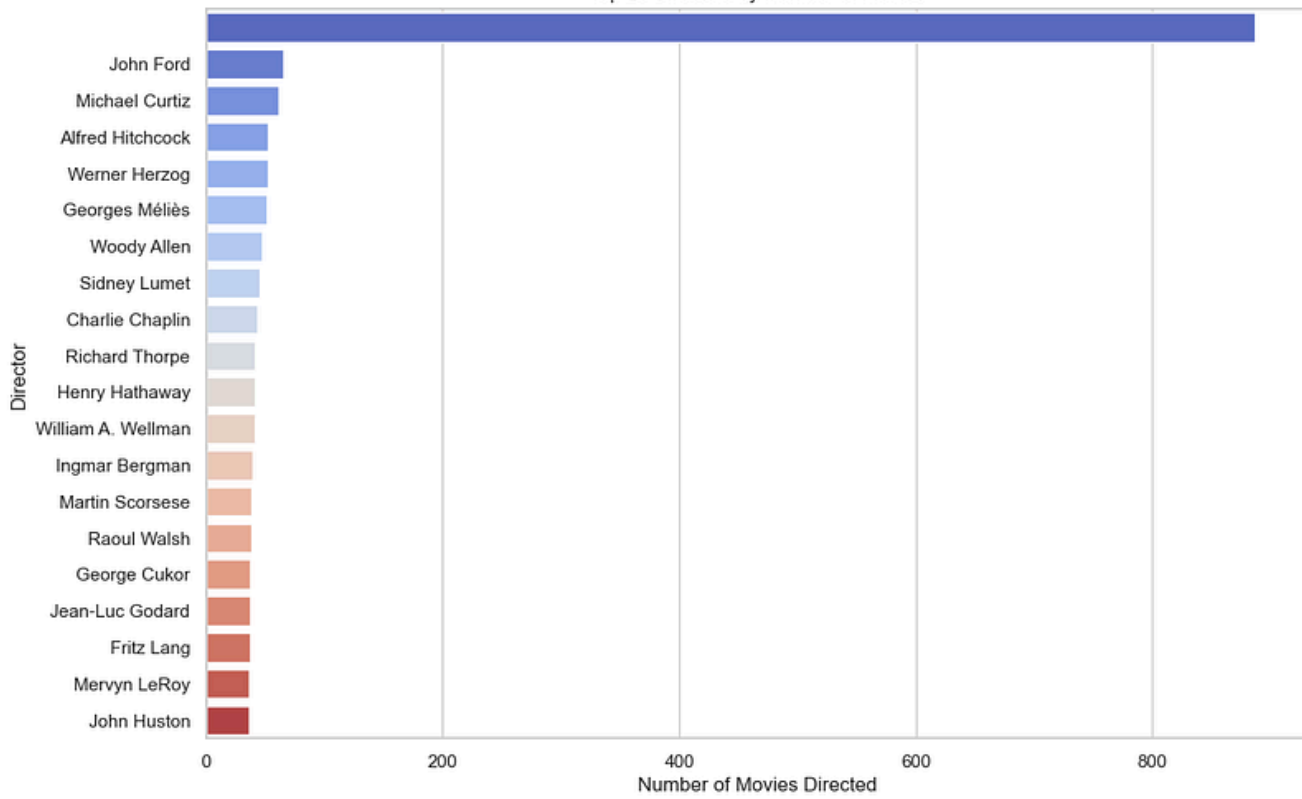


Figure 4: Top Directors by Number of Movies

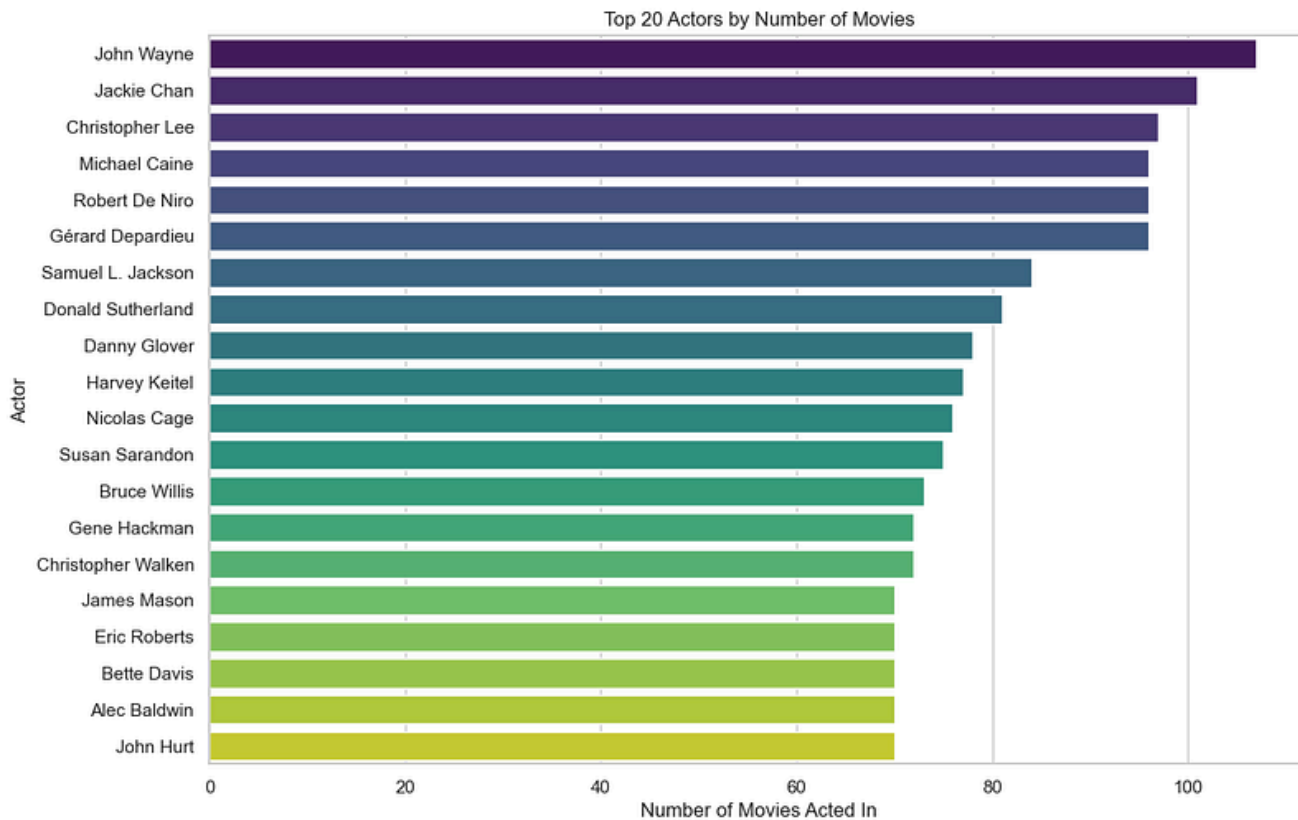


Figure 5: Top Actors by Movie Appearances

Directors like Steven Spielberg and actors such as Tom Hanks feature prominently, indicating their significant impact on movie popularity.

4.5. Keyword Analysis

Plot keywords provide insight into common themes and storylines in movies.

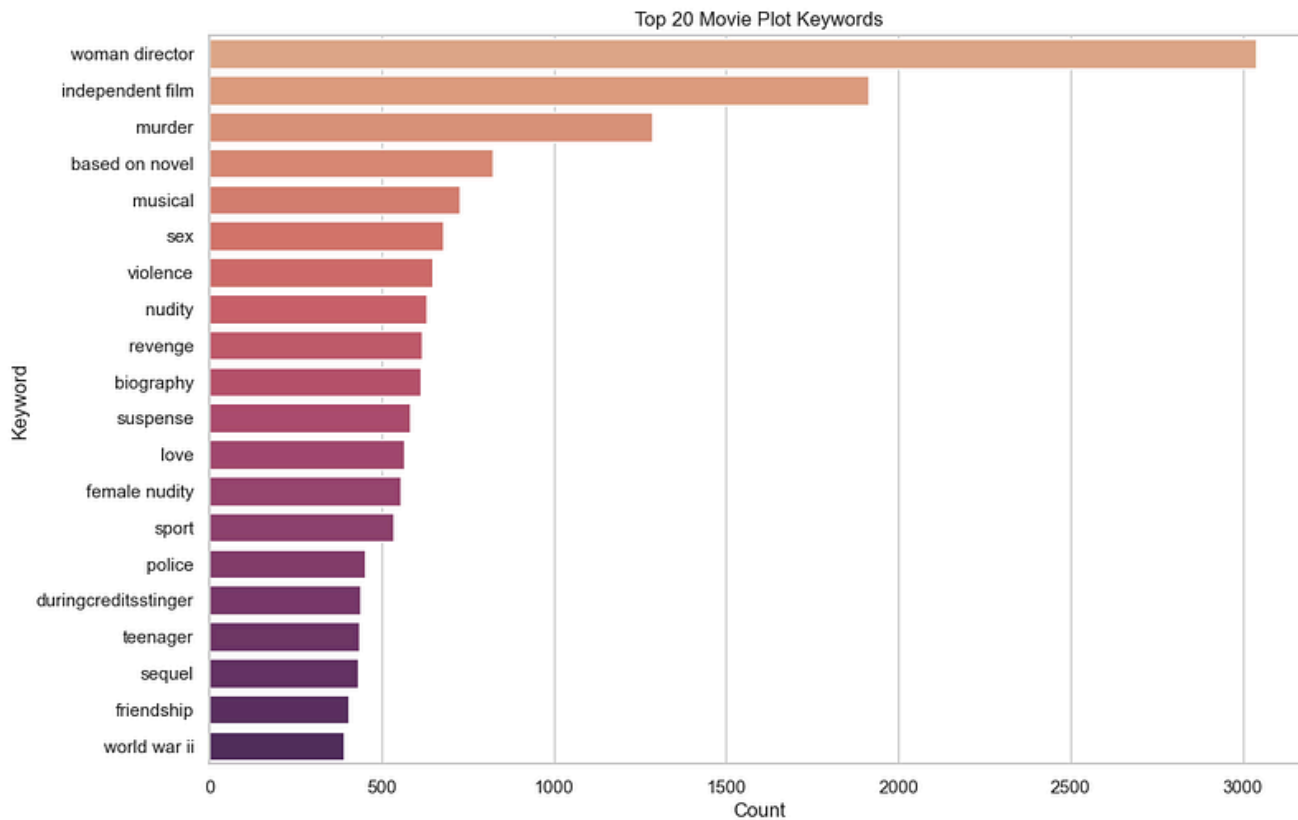


Figure 6: Common Plot Keywords

Keywords like “love”, “war”, and “hero” are frequently associated with highly rated movies, suggesting these themes resonate well with audiences.

4.6. User Behavior Insights

Analyzing user rating patterns reveals engagement levels and preferences.

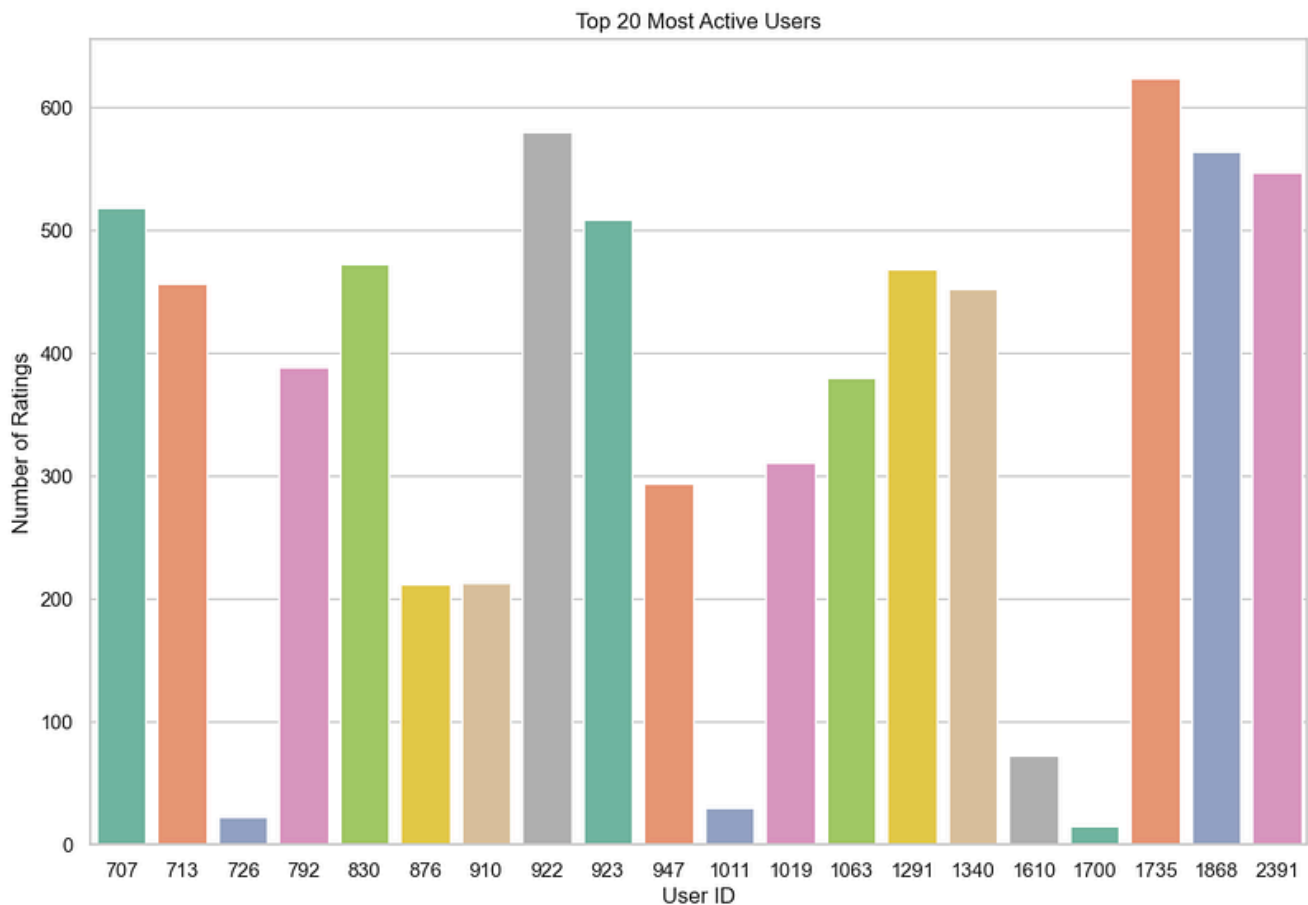


Figure 7: User Rating Activity

A small percentage of users contribute the majority of ratings, indicating the presence of power users who are highly engaged with the platform.

4.7. Time-Based Analysis

Exploring trends over time shows how movie popularity and production have evolved.

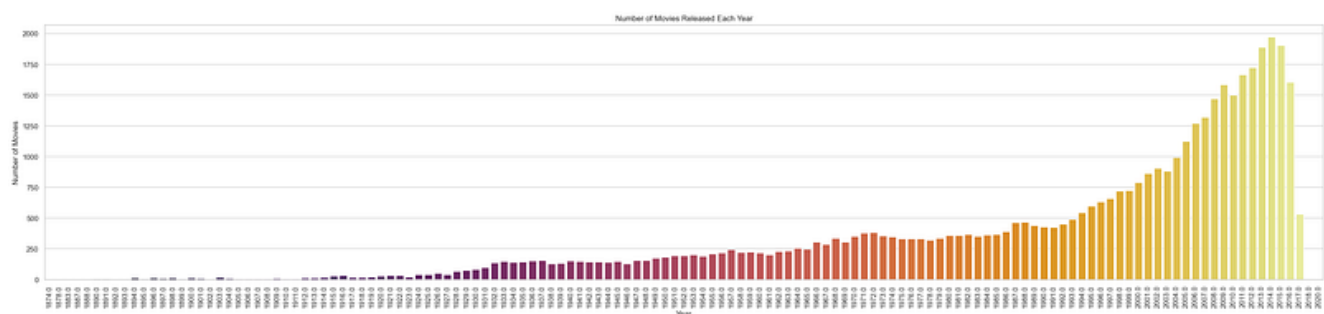


Figure 8: Number of Movies Released Each Year

The number of movie releases has steadily increased, with peaks in blockbuster seasons, reflecting industry trends and audience demand.

4.8. Language Distribution

Understanding the diversity of languages in movies can aid in multilingual recommendations.

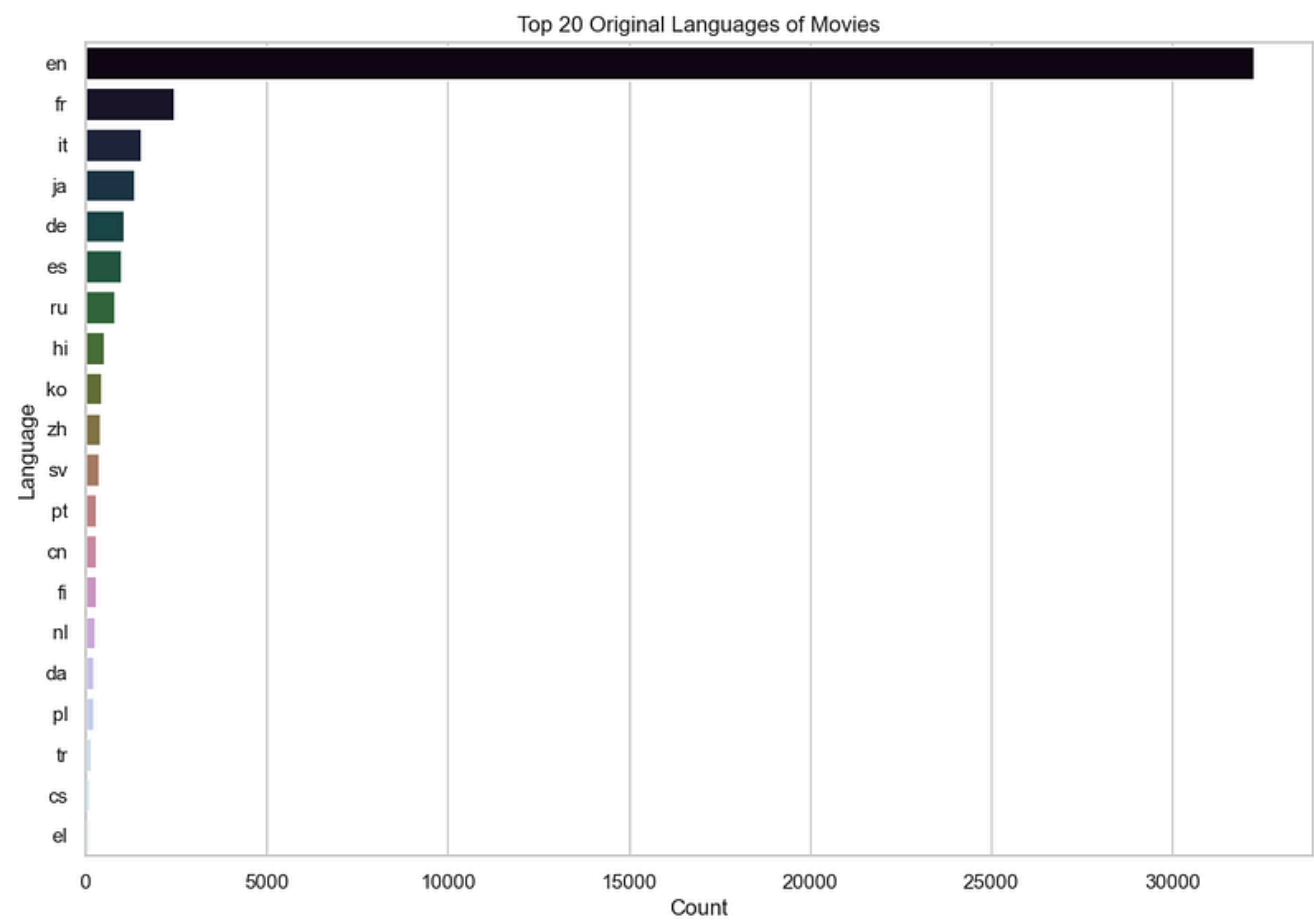


Figure 9: Distribution of Original Languages

While English dominates, there’s a significant presence of other languages like French and Japanese, highlighting the global nature of the film industry.

4.9. Cluster Analysis

Grouping movies based on features like budget, revenue, and ratings helps identify distinct categories.

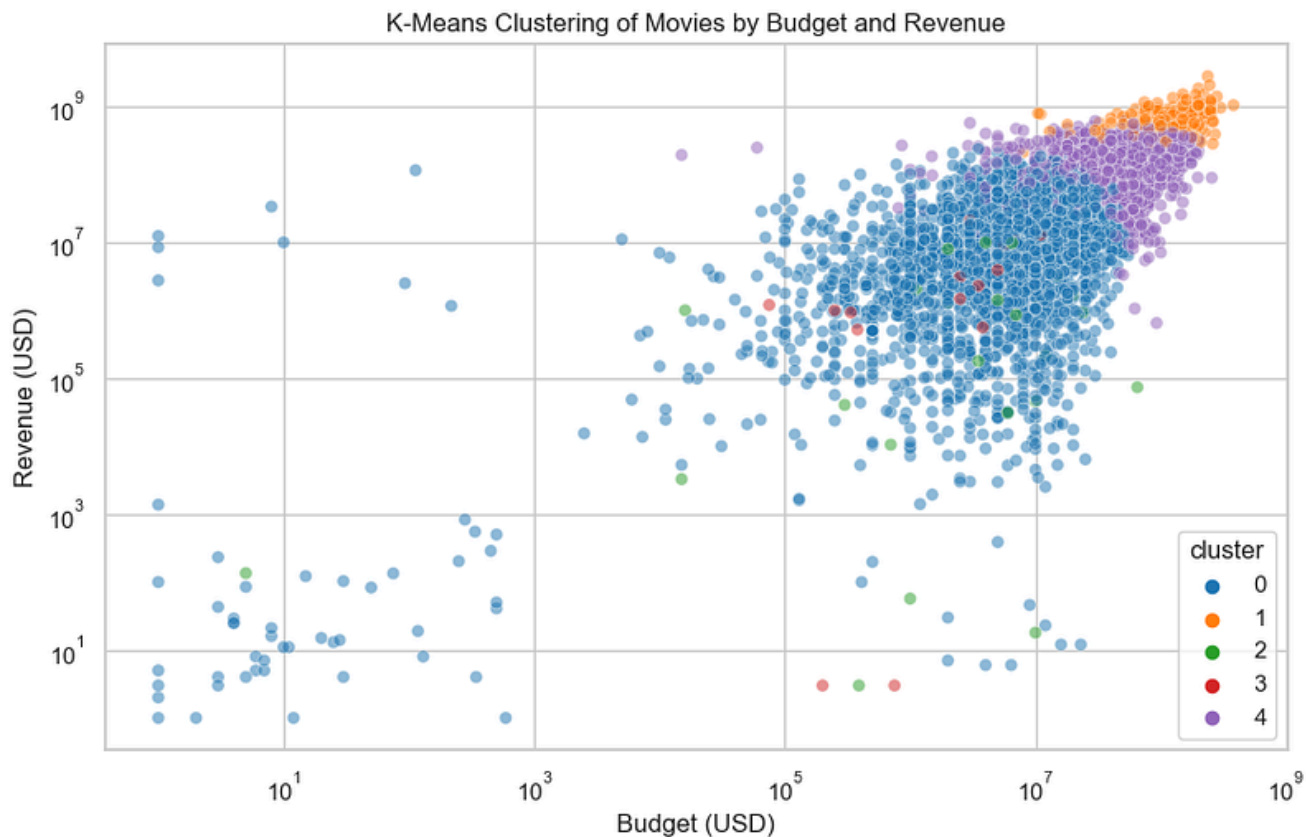


Figure 11: Movie Clusters Based on Budget and Revenue

Clusters reveal segments such as high-budget blockbusters, indie films, and mid-range productions, each with unique characteristics and audience appeal.

4.10. Additional Visualizations

Further visualizations offer deeper insights into the dataset's nuances.

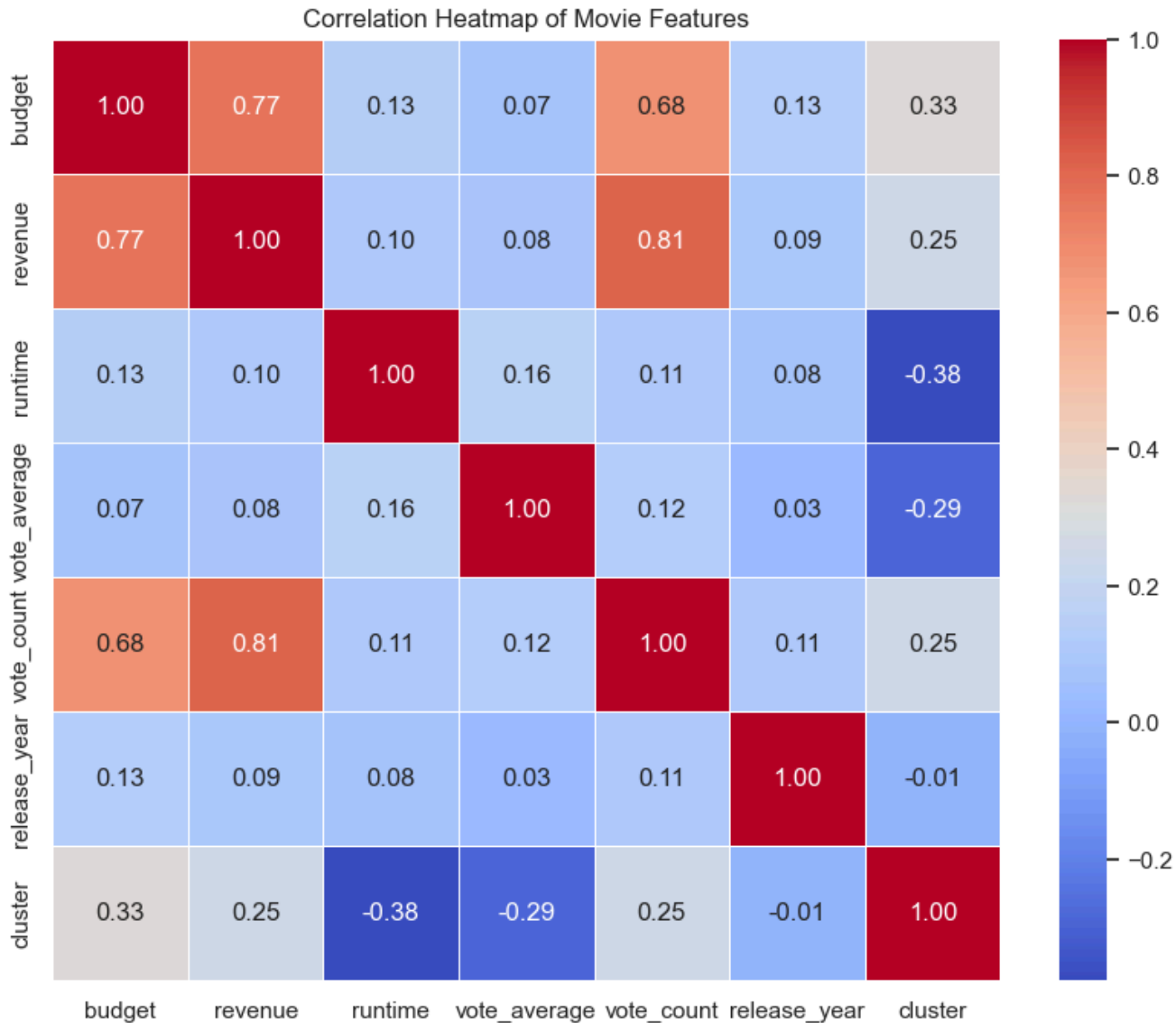


Figure 12: Correlation Heatmap of Movie Features

This heatmap illustrates the correlations between various movie features, such as budget, revenue, runtime, and ratings, highlighting the interdependencies that influence a movie’s success.

5. Building Baseline Recommendation Systems (No RAG)

With a solid understanding of the dataset, we can now implement traditional recommendation algorithms. These baseline systems will serve as benchmarks for future RAG-based enhancements.

5.1. Popularity-Based Recommender

5.1.1. Concept and Rationale

The Popularity-Based Recommender suggests movies that are generally popular within a specified genre. Popularity can be measured by metrics such as the number of ratings or average ratings. This approach is straightforward and ensures that users are presented with widely acclaimed content.

5.1.2. Implementation Details

We implemented a genre-based popularity recommender that filters movies by genre, calculates average ratings and the number of ratings, and recommends the top N movies based on these metrics.

```
def popularity_based_recommender_genre(movies_df: pd.DataFrame, ratings_df:
pd.DataFrame, genre: str, top_n: int = 10) -> pd.DataFrame:
```

```
    """
```

Recommend top N popular movies within a specified genre based on average ratings and number of ratings.

Args:

movies_df (pd.DataFrame): DataFrame containing movie metadata.

ratings_df (pd.DataFrame): DataFrame containing user ratings.

genre (str): The genre to filter movies by.

top_n (int): Number of top movies to recommend.

Returns:

pd.DataFrame: DataFrame of recommended movies.

.....

Filter movies that belong to the specified genre

```
genre_filtered = movies_df[movies_df['genre_names'].apply(lambda genres:  
genre in genres)]
```

Calculate average rating and number of ratings per movie

```
movie_ratings = ratings_df.groupby('movieId').agg({'rating': ['mean', 'count']})
```

```
movie_ratings.columns = ['average_rating', 'rating_count']
```

```
movie_ratings = movie_ratings.reset_index()
```

Merge with movies metadata

```
movies_with_ratings = pd.merge(genre_filtered, movie_ratings, left_on='id',  
right_on='movieId', how='left')
```

Handle missing values

```
movies_with_ratings['average_rating'].fillna(0, inplace=True)
```

```
movies_with_ratings['rating_count'].fillna(0, inplace=True)
```

Filter movies with a minimum number of ratings to ensure reliability

```
popular_movies = movies_with_ratings[movies_with_ratings['rating_count'] >=  
100]
```

```
# Sort by average rating and number of ratings
```

```
popular_movies = popular_movies.sort_values(['average_rating', 'rating_count'],  
ascending=[False, False])
```

```
# Select top N movies
```

```
recommended_movies = popular_movies.head(top_n)
```

```
return recommended_movies[['title', 'average_rating', 'rating_count', 'genres',  
'revenue']]
```

5.1.3. Sample Recommendations

Here's an example of how the Popularity-Based Recommender performs for the Comedy genre:

```
# Example usage
```

```
genre_query = "Comedy"
```

```
top_n = 5
```

```
top_comedy_movies = popularity_based_recommender_genre(
```

```
    movies_df=mov_data['movies_metadata.csv'],
```

```
    ratings_df=mov_data['ratings_small.csv'],
```

```
    genre=genre_query,
```

```
    top_n=top_n
```

```
)
```

`print(top_comedy_movies)`

	title	average_rating	rating_count	genres	revenue
203	Sleepless in Seattle	4.487500	200.0	[{'id': 35, 'name': 'Comedy'}, {'id': 18, 'nam...	227799884.0
1988	Men in Black II	4.256696	224.0	[{'id': 28, 'name': 'Action'}, {'id': 12, 'nam...	441818803.0
7905	Shriek If You Know What I Did Last Friday the ...	4.227273	132.0	[{'id': 35, 'name': 'Comedy'}]	0.0
3078	Murder She Said	4.209524	105.0	[{'id': 18, 'name': 'Drama'}, {'id': 80, 'name...	0.0
4181	License to Wed	4.178218	202.0	[{'id': 35, 'name': 'Comedy'}]	69307224.0

Table 5: Most Popular Genres by User Ratings

5.1.4. Limitations

While effective in highlighting popular movies, this approach doesn’t account for individual user preferences or contextual nuances, potentially overshadowing niche or recently released films.

5.2. Content-Based Recommender

5.2.1. Concept and Rationale

The Content-Based Recommender suggests movies similar to those a user has previously enjoyed by analyzing attributes like genres, keywords, directors, and plot summaries. This approach ensures that recommendations are tailored to the user’s specific tastes based on the inherent characteristics of the movies.

5.2.2. Implementation Details

We developed a genre-based content recommender that filters movies by genre, merges them with plot keywords, and calculates content similarity using TF-IDF vectorization.

```
def content_based_recommender_genre(movies_df: pd.DataFrame, keywords_df:
pd.DataFrame, genre: str, top_n: int = 10) -> pd.DataFrame:

    """

    Recommend top N movies within a specified genre based on content similarity
    (genres and keywords).
```

Args:

movies_df (pd.DataFrame): DataFrame containing movie metadata.

keywords_df (pd.DataFrame): DataFrame containing movie keywords.

genre (str): The genre to filter movies by.

top_n (int): Number of top movies to recommend.

Returns:

pd.DataFrame: DataFrame of recommended movies.

"""

Filter movies that belong to the specified genre

movies_df['id'] = movies_df['id'].astype(str)

**genre_filtered = movies_df[movies_df['genre_names'].apply(lambda genres:
genre in genres)]**

Merge with keywords

genre_keywords = pd.merge(genre_filtered, keywords_df, on='id', how='left')

Fill NaN keywords with empty lists

**genre_keywords['keyword_names'] =
genre_keywords['keyword_names'].apply(lambda x: x if isinstance(x, list) else [])**

Create a combined feature string

```
genre_keywords['combined_features'] = genre_keywords.apply(  
    lambda row: ' '.join(row['genre_names']) + ' ' + ' '.join(row['keyword_names']),  
    axis=1  
)
```

Encode combined features using TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer  
  
tfidf = TfidfVectorizer(stop_words='english')  
  
tfidf_matrix = tfidf.fit_transform(genre_keywords['combined_features'])
```

Compute cosine similarity

```
cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)
```

Create a mapping from movie titles to indices

```
indices = pd.Series(genre_keywords.index,  
index=genre_keywords['title']).drop_duplicates()
```

Function to get recommendations

```
def get_recommendations(title, cosine_sim=cosine_sim):
```

```
idx = indices[title]

sim_scores = list(enumerate(cosine_sim[idx]))

sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)

sim_scores = sim_scores[1:top_n+1]

movie_indices = [i[0] for i in sim_scores]

return genre_keywords.iloc[movie_indices]
```

For genre-based recommendation, we'll average similarity scores within the genre

Alternatively, recommend the top N movies within the genre based on TF-IDF similarity

Here, we'll recommend top N movies within the genre with the highest TF-IDF scores

which inherently captures content similarity

Sort by TF-IDF vector sum (as a proxy for content richness)

```
genre_keywords['tfidf_sum'] =
genre_keywords['combined_features'].apply(lambda x: len(x.split()))
```

```
recommended_movies = genre_keywords.sort_values('tfidf_sum',
ascending=False).head(top_n)
```

```
return recommended_movies[['title', 'genres', 'keywords', 'tfidf_sum']]
```

5.2.3. Sample Recommendations

Here’s how the Content-Based Recommender performs for the Action genre:

Example usage

```
genre_query = "Drama"

top_n = 5

top_action_movies = content_based_recommender_genre(

    movies_df=movies_metadata,

    keywords_df=keywords,

    genre=genre_query,

    top_n=top_n

)

print(top_action_movies)
```

	title	genre_names	keyword_names	similarity_score
50	The Journey of August King	[Drama]	[]	0.181007
69	The Boys of St. Vincent	[Drama]	[]	0.181007
118	The Browning Version	[Drama]	[]	0.181007
188	Red Firecracker, Green Firecracker	[Drama]	[]	0.181007
232	Boys Life: Three Stories of Love, Lust, and Li...	[Drama]	[]	0.181007

Table 6: Movies Recommended by Content-Based Filtering

5.2.4. Limitations

While providing tailored recommendations, this approach may overemphasize specific content features, potentially limiting the diversity of suggestions and neglecting user-specific nuances.

5.3. Collaborative Filtering Recommender

5.3.1. Concept and Rationale

The Collaborative Filtering Recommender suggests movies based on the preferences and behaviors of similar users. By analyzing user ratings, it identifies patterns and recommends titles that like-minded users have enjoyed. This approach excels in personalization, ensuring that recommendations align closely with individual tastes.

5.3.2. Implementation Details

We implemented a genre-based collaborative filtering recommender that constructs a user-item matrix, computes user similarities, and recommends movies that similar users have rated highly.

```
def collaborative_filtering_recommender_genre(
```

```
    ratings_df: pd.DataFrame,
```

```
    movies_df: pd.DataFrame,
```

```
    target_user_id: int,
```

```
    genre: str = None,
```

```
    top_n: int = 10
```

```
) -> pd.DataFrame:
```

```
    """
```

Recommend top N movies for a target user based on collaborative filtering, optionally filtered by genre.

Args:

ratings_df (pd.DataFrame): DataFrame containing user ratings.

movies_df (pd.DataFrame): DataFrame containing movie metadata.

target_user_id (int): ID of the target user.

genre (str, optional): Genre to filter recommendations by. Defaults to None.

top_n (int): Number of top movies to recommend.

Returns:

pd.DataFrame: DataFrame of recommended movies.

"""

Create user-item matrix

**user_item_matrix = ratings_df.pivot_table(index='userId', columns='movieId',
values='rating').fillna(0)**

user_item_sparse = csr_matrix(user_item_matrix.values)

Compute cosine similarity between users

user_sim = cosine_similarity(user_item_sparse, user_item_sparse)

**user_sim_df = pd.DataFrame(user_sim, index=user_item_matrix.index,
columns=user_item_matrix.index)**

Find similar users

**similar_users =
user_sim_df[target_user_id].sort_values(ascending=False)[1:11].index**

Aggregate ratings from similar users

```
similar_users_ratings = ratings_df[ratings_df['userId'].isin(similar_users)]
```

```
movie_scores = similar_users_ratings.groupby('movieId').agg({'rating': ['mean',  
'count']})
```

```
movie_scores.columns = ['average_rating', 'rating_count']
```

```
movie_scores = movie_scores.reset_index()
```

Merge with movies metadata

```
recommended_movies = pd.merge(movie_scores, movies_df, left_on='movieId',  
right_on='id', how='left')
```

Handle missing values

```
recommended_movies['average_rating'].fillna(0, inplace=True)
```

```
recommended_movies['rating_count'].fillna(0, inplace=True)
```

```
recommended_movies['genre_names'] =  
recommended_movies['genre_names'].apply(lambda x: x if isinstance(x, list) else  
[])
```

Optionally filter by genre

if genre:

```
recommended_movies =  
recommended_movies[recommended_movies['genre_names'].apply(lambda  
genres: genre in genres)]
```

```

# Filter out movies already rated by the target user

    rated_movies = ratings_df[ratings_df['userId'] ==
target_user_id]['movieId'].unique()

    recommended_movies =
recommended_movies[~recommended_movies['movieId'].isin(rated_movies)]


# Sort by average rating and number of ratings

    recommended_movies = recommended_movies.sort_values(['average_rating',
'rating_count'], ascending=[False, False])


# Check if there are any recommendations

    if recommended_movies.empty:

        print(f"No recommendations found for user ID {target_user_id} in genre
'{genre}'.")

        return recommended_movies # Return empty DataFrame


# Return top N movies

    return recommended_movies.head(top_n)[['title', 'average_rating', 'rating_count',
'genres', 'revenue']]

```

5.3.3. Sample Recommendations

Here's an example of how the Collaborative Filtering Recommender performs for a user with ID 4 and the Comedy genre:

```

# Example usage

```

```

target_user_id = 4

genre_query = "Comedy"

top_n = 10

top_comedy_recommendations = collaborative_filtering_recommender_genre(

    ratings_df=ratings_small,

    movies_df=movies_metadata,

    target_user_id=4,

    genre=genre_query,

    top_n=top_n

)

print(top_comedy_recommendations)

```

	title	average_rating	rating_count	genres	revenue
160	Meet the Robinsons	5.0	2	[[{'id': 16, 'name': 'Animation'}, {'id': 35, 'name': 'Comedy'}]]	169332978.0
817	Memoirs of an Invisible Man	5.0	2	[[{'id': 35, 'name': 'Comedy'}, {'id': 14, 'name': 'Drama'}]]	14358033.0
858	24 Hour Party People	5.0	2	[[{'id': 35, 'name': 'Comedy'}, {'id': 18, 'name': 'Drama'}]]	0.0
2	Lock, Stock and Two Smoking Barrels	5.0	1	[[{'id': 35, 'name': 'Comedy'}, {'id': 80, 'name': 'Crime'}]]	3897569.0
166	TMNT	5.0	1	[[{'id': 12, 'name': 'Adventure'}, {'id': 16, 'name': 'Animation'}]]	95608995.0
173	Bean	5.0	1	[[{'id': 35, 'name': 'Comedy'}]]	251212670.0
190	Forrest Gump	5.0	1	[[{'id': 35, 'name': 'Comedy'}, {'id': 18, 'name': 'Drama'}]]	677945399.0
248	Vollidiot	5.0	1	[[{'id': 35, 'name': 'Comedy'}, {'id': 18, 'name': 'Drama'}]]	0.0
301	Bring It On	5.0	1	[[{'id': 35, 'name': 'Comedy'}]]	90449929.0
326	Breakfast at Tiffany's	5.0	1	[[{'id': 35, 'name': 'Comedy'}, {'id': 18, 'name': 'Drama'}]]	9500000.0

Table 7: Movies Recommended by Collaborative Filtering

5.3.4. Limitations

While collaborative filtering excels in personalization, it faces challenges like the cold start problem—difficulty recommending for new users with limited rating history—and potential biases from highly active users.

6: Implementing and Integrating RAG with Mistral 7B

In this section, we introduce a practical implementation of a Retrieval-Augmented Generation (RAG) pipeline leveraging the Mistral 7B model, sentence-transformers for embeddings, and FAISS for vector storage. This integration is tailored to provide fast, accurate, and context-rich responses to natural language queries, making it ideal for use cases in recommendation systems, knowledge retrieval, and beyond.

6.1 RAG Pipeline Components

1. **Mistral 7B as the LLM:** Mistral 7B is used as the backbone language model for generating detailed and context-aware responses based on retrieved documents.
2. **Embeddings with Sentence-Transformers:** We employ the 'all-MiniLM-L6-v2' model from Sentence-Transformers to generate 384-dimensional embeddings. These embeddings provide a compact and semantically rich representation of the input text, optimizing retrieval performance.
3. **Vector Store with FAISS:** The FAISS (Facebook AI Similarity Search) library is utilized as the vector database to efficiently store and query embeddings. This setup enables scalable and high-speed retrieval of relevant documents.
4. **BasicRAG Architecture:** A lightweight retrieval-augmented generation approach is implemented via the **BasicRAG** class. This custom architecture ensures flexibility while maintaining simplicity for quick integration into existing pipelines.

6.2 RAG Implementation Snippets

Defining the BasicRAG Class

```
class BasicRAG(BaseRAG):
```

```
    def query(self, query_text: str) -> Dict[str, Any]:
```

```
query_embedding = self.embedding_model.encode(query_text)
```

```
return self.vector_store.query(query_embedding.tolist(), self.config.top_k)
```

This implementation queries the vector store using embeddings generated from the query text, retrieving the top-k relevant documents.

Setting Up the Pipeline

```
{  
  
    'name': 'fast',  
  
    'embedding': 'sentence-transformers/all-MiniLM-L6-v2',  
  
    'vector_store': FAISSStore(dimension=384),  
  
    'rag_type': BasicRAG  
  
}
```

This configuration enables rapid embedding generation and retrieval while ensuring compatibility with Mistral 7B for seamless integration.

6.3 Example Use Case: Movie Recommendations

The following example demonstrates the power of the RAG pipeline with Mistral 7B in the context of generating movie recommendations:

User Query:

“Recommend me a sci-fi movie with deep philosophical themes.”

Recommendation:

💬  In the realm of cinema where science fiction meets philosophy, there's a gem that has been quietly illuminating minds since its release - 'Gattaca.' A masterful blend of suspense, romance, and the profound exploration of identity, this film will leave you questioning the very

essence of humanity. Immerse yourself in the future society where humans are set on a life course based on their DNA, but find hope in the indomitable human spirit that transcends genetics. The captivating narrative unfolds through the eyes of Vincent Freeman, portrayed by the exceptional Ethan Hawke, whose dreams of reaching the stars are hindered by his 'inferior' genetic makeup.

Metrics:

- **Retrieval Time: 0.67 seconds**
- **Generation Time: 93.44 seconds**
- **Total Time: 94.11 seconds**

This example showcases the ability of the pipeline to understand nuanced queries and provide detailed, context-aware responses enriched by retrieved documents.

7. Conclusion and Next Steps

7.1. Recap of EDA Findings

Our comprehensive EDA unveiled key insights into user behavior, genre popularity, budget-revenue relationships, and more. These insights are instrumental in shaping our recommendation strategies, ensuring they are data-driven and aligned with user preferences.

7.2. Effectiveness of Baseline Recommenders

The implemented baseline recommenders—Popularity-Based, Content-Based, and Collaborative Filtering, each offer unique advantages in providing valuable recommendations. However, they also exhibit limitations, such as lack of personalization or difficulty in capturing nuanced user preferences, which can be significantly mitigated by incorporating advanced techniques like Retrieval-Augmented Generation (RAG).

7.3. Transition to Advanced RAG-Based Recommendations

In the next part of the BoxRAG series, we'll elevate our recommendation system by integrating Retrieval-Augmented Generation (RAG) with Mistral 7B. Our implementation leverages the 'fast' pipeline configuration with sentence-transformers/all-MiniLM-L6-v2 for embeddings, FAISS for efficient retrieval, and the BasicRAG architecture. In upcoming configurations, we plan to explore two other RAG setups.

7.4. What's Next in the BoxRAG Series?

Stay tuned for Blog Part 2, where we'll focus on:

- Building the frontend and backend pipeline using React and Flask.
 - Implementing advanced RAG architectures and vector stores.
 - Enhancing recommendation quality with user-focused prompts and contextual data.
-

Final Thoughts

All code and notebooks are available in the [BoxOfficeRAG GitHub Repository](#). The modular codebase is designed for easy experimentation and scaling.

Our future RAG configurations with Mistral 7B will include:

- **Balanced Configuration:** Using 'all-mpnet-base-v2' embeddings with HNSWStore and ReRankingRAG for optimal performance-to-quality ratio.
- **Quality Configuration:** Employing 'intfloat/e5-large' embeddings and ChromaStore with ContextualCompressionRAG for maximum response quality.

Stay tuned for our next post on implementing these configurations and creating a seamless user experience. Check out the repository for all resources, and feel free to contribute!

Happy coding!

Stop Managing Prompts Manually: **prompt-manager** Automates It for AI Developers

“In a world where AI applications built on LLMs are only as powerful as their prompts, mastering the art of prompt management and refinement has become essential for success.”

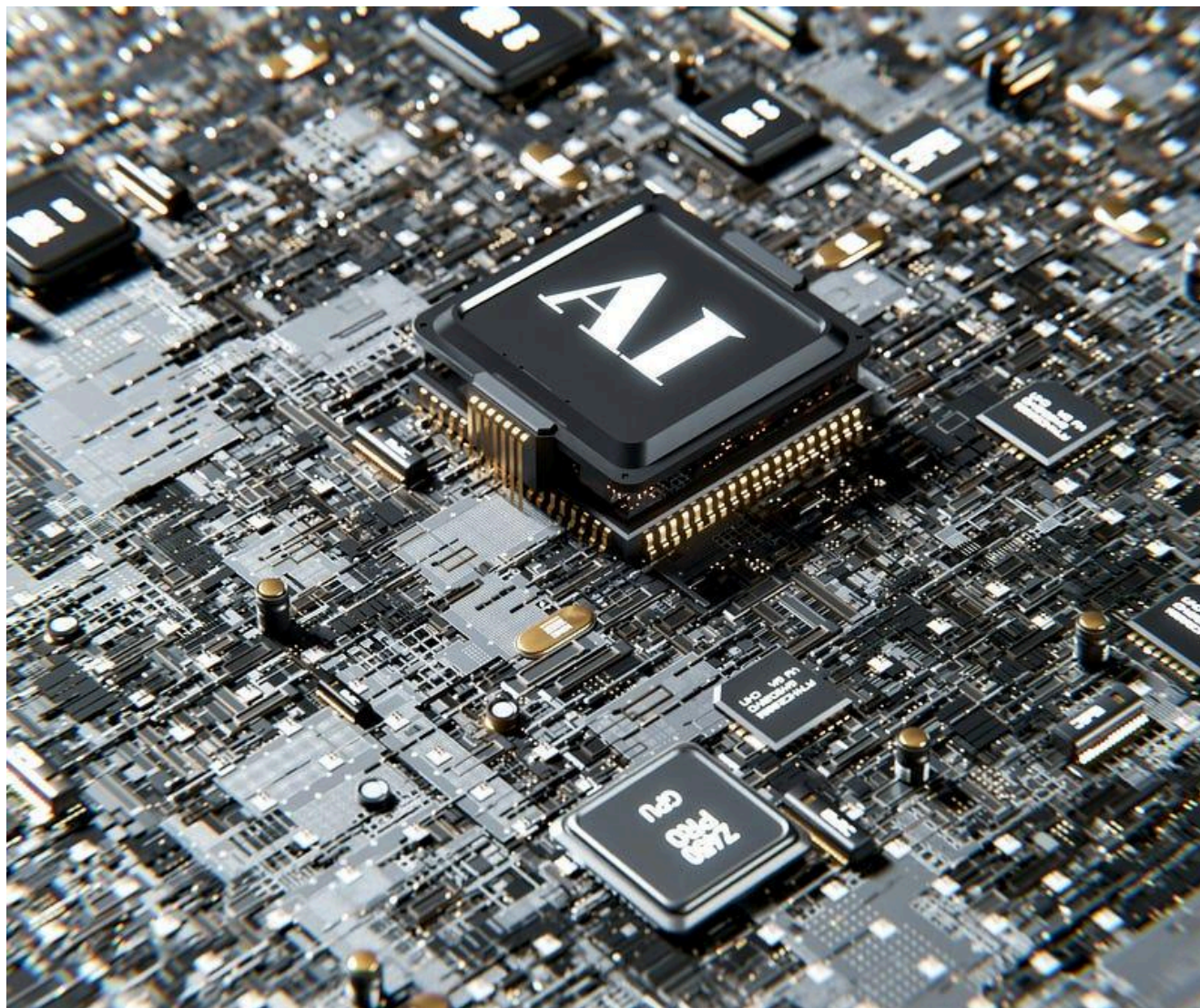


Photo by [Igor Omilaev](#) on [Unsplash](#)

Developer’s Insight: This blog introduces prompt-manager, a library crafted to address the challenges of managing and versioning prompts efficiently. It provides actionable strategies and tools to streamline workflows, empowering developers and entrepreneurs alike in their AI projects.

The Origin of Prompt-Manager

Building my resume-automation project, a Flask-based application that generates customized resumes by processing CVs and job descriptions, I encountered a significant challenge: managing and versioning multiple prompts. With 3–4 key prompts in constant iteration, tracking changes and ensuring consistency became cumbersome. I am currently testing and improving this project while adding additional tools such as cover letter generation, trackline customization, and interview preparation to make it a comprehensive application.

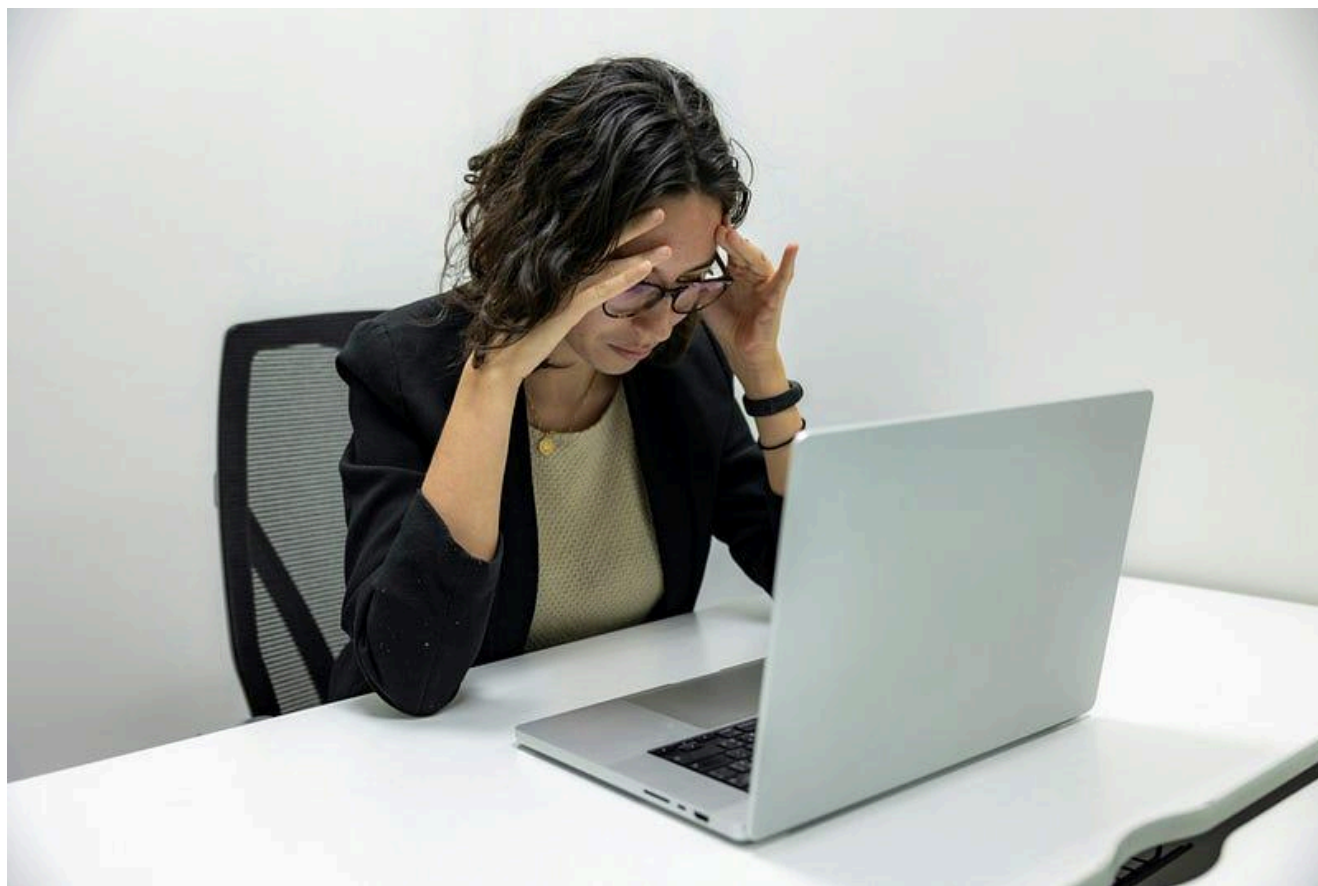


Photo by [Resume Genius](#) on [Unsplash](#)

Why Prompt-Manager? This frustration led to the creation of prompt-manager, a library that made managing and versioning prompts effortless. By implementing this tool, I significantly reduced the time spent juggling prompts, allowing me to focus on improving the resume-automation application itself. Prompt-manager streamlined my workflow, ensuring version control and organization, even as prompt complexity grew.

Key Features of Prompt-Manager

A. Simplified Prompt Management

The library provides a structured way to define, update, and retrieve prompts with ease.

Code Example:

```
import prompt_manager
```

```
manager = prompt_manager.PromptManager("prompts")
```

```
# Adding version 1.0.0 of the prompt
```

```
manager.add_prompt_version(  
    "welcome_message",  
    "1.0.0",  
    "Hello, {name}! Welcome to our platform.",  
    ["name"],  
    "Basic welcome message"  
)
```

```
# Adding version 1.1.0 of the prompt
```

```
manager.add_prompt_version(  
    "welcome_message",  
    "1.1.0",
```

```

    "Hi, {name}! We're glad you're here. Start exploring {platform}!",
    ["name", "platform"],
    "Enhanced welcome message"
)

# Fetching the latest version (default behavior)

print(manager.get_prompt("welcome_message", {"name": "Alice", "platform":
"PromptManager"}))

# Fetching a specific version

print(manager.get_prompt("welcome_message", {"name": "Alice"},
version="1.0.0"))

```

Expected Output

Hi, Alice! We're glad you're here. Start exploring PromptManager!

Hello, Alice! Welcome to our platform.

B. Built-In Version Control

Every prompt iteration is tracked, ensuring no version is lost and changes are transparent.

Code Example:

```

# List available prompts with version control

print("Available Prompts:")

for prompt in manager.list_prompts():

```

```
print(f"Name: {prompt['name']}")  
  
print(f"Current Version: {prompt['current_version']}")  
  
print(f"Versions: {', '.join(prompt['versions'])}")  
  
print()
```

Expected Output

Available Prompts:

Name: resume_summary

Current Version: 1.0.0

Versions: 1.0.0

Name: welcome_message

Current Version: 1.1.0

Versions: 1.0.0, 1.1.0

C. Scalability for Complex Projects

Whether you have 3 prompts or 300, prompt-manager scales seamlessly to meet your needs.

Approach: With prompt-manager, you can group prompts into projects, add metadata, and maintain consistency across multiple versions.

Code Example

```
# Initialize managers for different projects
```

```
project1_manager = prompt_manager.PromptManager("project1_prompts")
```

```
project2_manager = prompt_manager.PromptManager("project2_prompts")
```

Add a prompt to Project 1

```
project1_manager.add_prompt_version(  
    "welcome_message",  
    "1.0.0",  
    "Welcome to Project 1, {name}!",  
    ["name"],  
    "Welcome prompt for Project 1"  
)
```

Add a prompt to Project 2

```
project2_manager.add_prompt_version(  
    "welcome_message",  
    "1.0.0",  
    "Welcome to Project 2, {name}!",  
    ["name"],  
    "Welcome prompt for Project 2"  
)
```

Fetch and print prompts from both projects

```
print("Project 1 Prompt:")  
  
print(project1_manager.get_prompt("welcome_message", {"name": "Alice"}))  
  
print("\nProject 2 Prompt:")  
  
print(project2_manager.get_prompt("welcome_message", {"name": "Bob"}))
```


Expected Output

Project 1 Prompt:

Welcome to Project 1, Alice!

Project 2 Prompt:

Welcome to Project 2, Bob!

How Prompt-Manager Enhances Your Workflow

Overview: Prompt-manager saves you from manually managing prompts, letting you focus on your core application. It ensures:

- **Consistency:** Every prompt change is tracked and reversible.
- **Efficiency:** Reduced time spent searching for older versions or tweaking prompts.
- **Scalability:** Ideal for applications with multiple prompts across projects.

Real-World Impact: Using prompt-manager in my resume-automation project allowed me to shift my focus from the tedious task of managing prompts to improving the actual functionality of the application. It significantly reduced the hours spent on manual work, ensuring the tool's reliability and efficiency. Tasks such as analyzing job descriptions or creating resume summaries became faster, more accurate, and seamlessly integrated into my workflow, enabling me to innovate and enhance other aspects of the project.

Conclusion

Summary: Prompt-manager started as a solution to a personal challenge and has grown into a tool that simplifies prompt management and version control. While this is just the first version, it's already proving to save time and effort, paving the way for more robust applications.

Call to Action: Explore the [GitHub repository](#) to understand the implementation and contribute, or download it from [PyPI](#) to integrate it into your projects.

I would love to hear your thoughts on prompt-manager! While I developed this library on my own, I'm always looking for opportunities to collaborate and improve. Your feedback and suggestions are invaluable in shaping its future. If you have ideas for features or enhancements, please share them! The best way to reach me is through my [LinkedIn profile](#). Thank you for your time and support!

Build and Deploy Your Deep Learning Model in MERN Stack

This article will show you how to create a Deep Learning web app using MERN Stack without storing your AI model on the cloud.



Photo by [DeepMind](#) on [Unsplash](#)

Outline

- 📌 Overview
- 📌 Training an Image Classifier Model Using TensorFlow
- 📌 Converting the model to Tensorflow.JS

Website

Conclusion

Overview

As an AI developer, I wanted to create models and use them in the real world by deploying them on websites to see how they performed and to get user feedback. However, due to large model files, doing so would increase both the computational cost and the amount of time it would take to predict or classify an object. Although TensorFlow offers substantial speed advantages, it cannot run native code in a web browser, this utilized TensorFlow.js, a component of the TensorFlow ecosystem that differs from TensorFlow but offers almost identical capabilities.

Training an Image Classifier Model Using TensorFlow

Dataset



Photo by [Erik Mclean](#) on [Unsplash](#)

To Train this model we will be using the [Pokemon Dataset](#) from Kaggle which has images of 809 different pokemons like bulbasaur, ivysaur, venusaur, charmander, charmeleon etc



From Left: Articuno, Audino,Aurorus



From Left: Axew, Azelf, Azuril

👉 Mobilenet-SSDv2

Here, we'll be utilizing Mobilenet-SSDv2, which was built on Mobilenet-v2. The real-time object detector can be used in embedded systems that have restricted computational power. to study more you could refer this [article](#).

👉 Training

I was able to get 98.8% accuracy on the train dataset after training. however, I was able to reach an accuracy of 99% and a test dataset accuracy of about 80% after performing Fine-Tuning and Re-Training.

Converting the model to TensorflowJS

Install Tensorflowjs using the below command on your Command prompt or Terminal

```
pip install tensorflowjs
```

After your model has been trained run this command to convert your model into JSON format which can be used with Tensorflow.js to predict object or classify images.

```
import tensorflow
```



```
import tensorflowjs as tfjs
```

```
from tensorflow import keras
```

```
your_model__path=""
```

```
model=keras.models.load_model(f"{your_model_path}")
```

```
tfjs.converters.save_keras_model(model,'model_js')
```

it will create a folder called `model_js`, in which the converter generates `model1.json` and `group1-shard*of*`, the `model1.json` has a dataflow graph and weight manifest file whereas `group1-shard*of*` has a collection of binary weight files.

Website

👉 Express.js and Node.js

We will be having Node server running on our backend and it will be serving our file using Express.js using the below code.

```
app.use(
```

```
  "/api/classify",
```

```
  express.static(path.join(__dirname, "/models/model_js/model.json"))
```

```
);
```

here when our front end makes a call to *api/classify* it returns a JSON file to it.

👉 IndexedDB

the content inside JSON file will be stored in IndexedDB which is a database built into a browser that is far more powerful than local Storage.

- Stores almost any type of value using keys, with multiple key types.
- Transactions are supported for dependability.
- Indexes and key range queries are supported.
- Can store significantly more data than localStorage.

To read more about it read the following [article](#)

👉Tensorflow.js

TensorFlow.js is a machine learning library for JavaScript, it makes machine learning capabilities available on a web browser and Node.js for server-side and back-end development. If you're a Javascript developer wants to make your hands dirty in Machine Learning, TensorFlow.js is a great option to start with.

Enter the below command in your terminal to install it.

```
npm i @tensorflow/tfjs
```

Before classifying an image, the function below reads it and performs preprocessing.

```
useEffect(() => {  
  
  async function predict() {  
  
    if (imageLoaded && file) {  
  
      const imageElement = document.createElement("img");  
  
      imageElement.src = file;  
  
  
      imageElement.onload = async () => {  
  
        const tensor = tf.browser  
  
          .fromPixels(imageElement)  
  
          .resizeNearestNeighbor([120, 120])  
  
          .toFloat()  
  
          .sub(127)  
  
          .div(127)
```

```
.expandDims();
```

```
const y_pred = await model.predict(tensor).data();
```

```
const topNPredNames = getTopNPredPokeObj(y_pred, 3);
```

```
setPrediction(topNPredNames);
```

```
console.log("-----");
```

```
setProcessing(false);
```

```
setImageLoaded(false);
```

```
return topNPredNames;
```

```
};
```

```
}
```

```
}
```

```
predict();
```

```
}, [imageLoaded, model, file]);
```

👉 ReactJS

For this article, I created a simple frontend for classifying pokemon when a user uploads an image to it.

here is a short code snippet

```
<form className="Form">
```

```
<label htmlFor="upload-image">
```

```
  <h1> Upload image </h1>
```

```
</label>
```

```
<br></br>
```

```
<input
```

```
  id="image-selector"
```

```
  type="file"
```

```
  name="upload-image"
```

```
  accept="image/*"
```

```
  className="File-selector"
```

```
  onChange={handleImgUpload}
```

```
  disabled={!model || processing}
```

```
/>
```

```
</form>
```

```
<div className="Upload">
```

```
  <img
```

```
    onLoad={() => {
```

```
      setImageLoaded(true);
```

```
    }}
```

```
    alt=""
```

```
    src={file}
```

/>

</div>

<div className="Results">

{processing ? (

<p>Loading ...</p>

) : topNPredNames !== null ? (

<div className="row">

{Object.entries(topNPredNames).map(([key, value]) => (

<div className="col-md-4">

<h2>{key}</h2> <h2>{value}</h2>

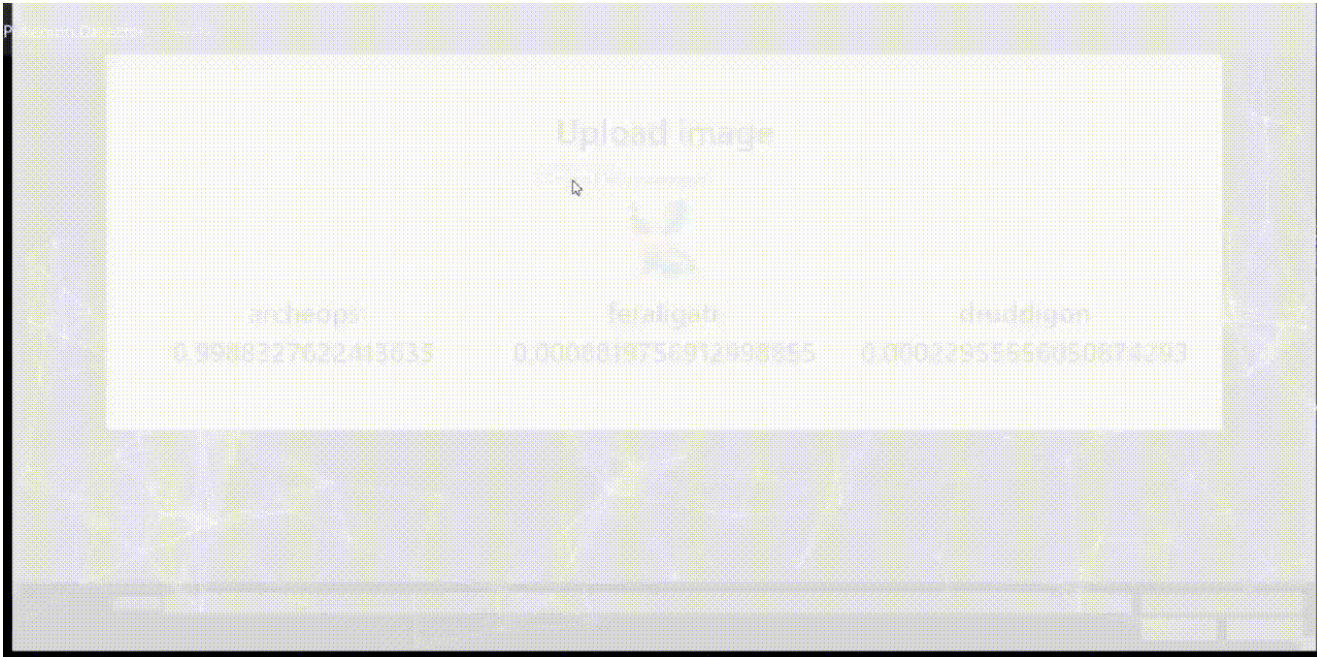
</div>

)))

</div>

) : null}

</div>



Demo

Conclusion

In my opinion, a front-end-powered TF model can be cost-effective because most front-end hosting is less expensive than a server-powered or cloud-powered TensorFlow solution; additionally, it can protect user privacy. If you want to play around with the dataset, you can get it from [Kaggle](#) and the code from [Github](#). follow me on [Medium](#). I'm always open to suggestions and constructive criticism, and my Twitter handle is [@FaseehAhmed2606](#).

Lastly,

Thank you for reading.

Hope this article helps you !

Build a Real-Time Inventory Management System in 6 Hours Using Streamlit and YOLOv5

A brief article explaining how to efficiently train the object detection algorithm YOLOv5 on your own custom dataset and use it to build your [web app](#) that can be used in real-time.

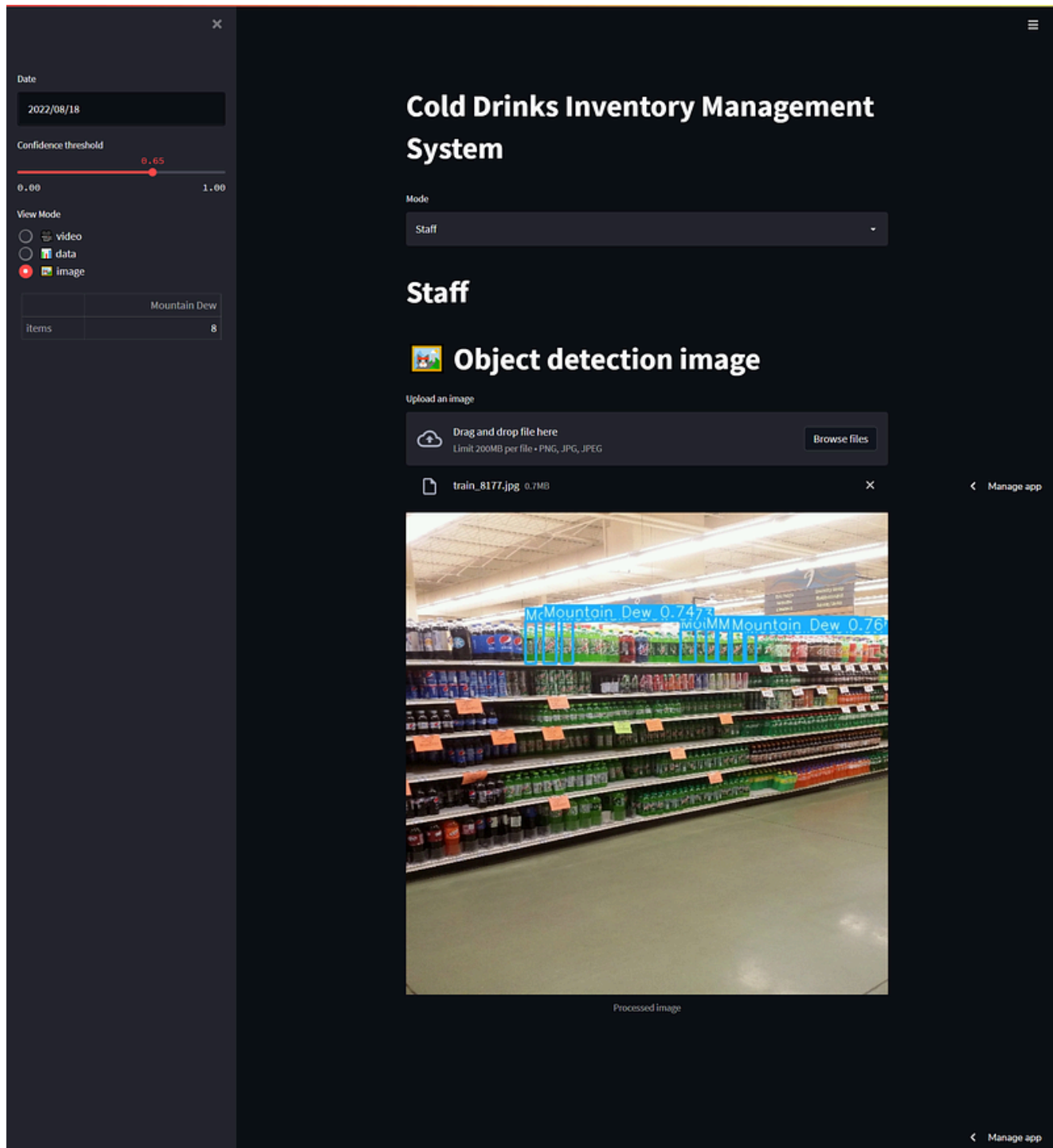


Photo by [Tobias Tullius](#) on [Unsplash](#)

The necessity to share data scientists' discoveries has increased, due to Python's escalating use as their preferred programming language, a cross-platform approach with adequate interaction is the use of web apps. However, typical programming demands enough expertise among Python users with somewhat complex web frameworks, like Flask and Django. Even if learning these frameworks is not particularly difficult for any moderate Python users, it may be difficult for data scientists to display their data science work.

Streamlit

The Streamlit framework, therefore, gives us data scientists a better choice today. It quickly and easily transforms straight forward Python scripts into usable, interactive, and shareable web apps. Due to its focus on exhibiting data science solutions, it comes with native support for data models and graphs which are created using popular data science libraries like pandas and matplotlib. Therefore, if you are already familiar with Python, learning streamlit will be relatively easy.



Performing Detection on an Image

Objective

I want to take you guys a step further in this article by demonstrating how to publish your web app so that others can view your AI models in Real Time. What's more, I'll demonstrate how to build an Inventory

Management System which manages cold beverages in real-time. For our project we will be using Streamlit to build our web app and SQLite as a database.

I'll be using the [SKU-110K dataset](#) to create my [custom dataset](#) to identify cold beverages like Coca-Cola, Sprite, Pepsi, Mountain Dew, 7UP, and Fanta. There are many low-quality photographs in SKU-110K, so had to filter them out, select the best ones, and annotate them with the [labelimg](#) library. which helps you annotate images in many storage formats, including YOLO and CreateML, and PASCAL VOC format. It is a helpful tool for annotating data for object detection.

Training

After the annotations are done we will train them using yolov5 which is an Object Detection and Classification model, Now we will briefly see the architecture of YOLOv5.

As YOLOv5 is a single-stage object detector, it has three important parts like any other single-stage object detector.

Model Backbone

The main function of Model Backbone is to identify key features in an input image.

To extract valuable, useful characteristics from an input image in YOLO v5, the CSPDarkNet (CSP—Cross Stage Partial Networks) is employed as the backbone. It uses a CSPNet technique that divides the base layer's feature map into two sections and then merges them using a cross-stage hierarchy.

Darknet is fast and highly accurate which are the key requirements for the YOLO family.

Model Neck

PANet (Path Aggregation Network) is used as a neck in YOLO v5 to get feature pyramids. Feature pyramids are a basic component in recognition systems for detecting objects at different scales.

The model's neck role is to collect features from different stages. The neck is composed of several top-down paths and several bottom-up paths. PANet uses features from all the layers and decides which ones are useful.

Model Head

It uses anchor boxes to construct final output vectors with class probabilities, object-ness scores, and bounding boxes.

The head is responsible for :

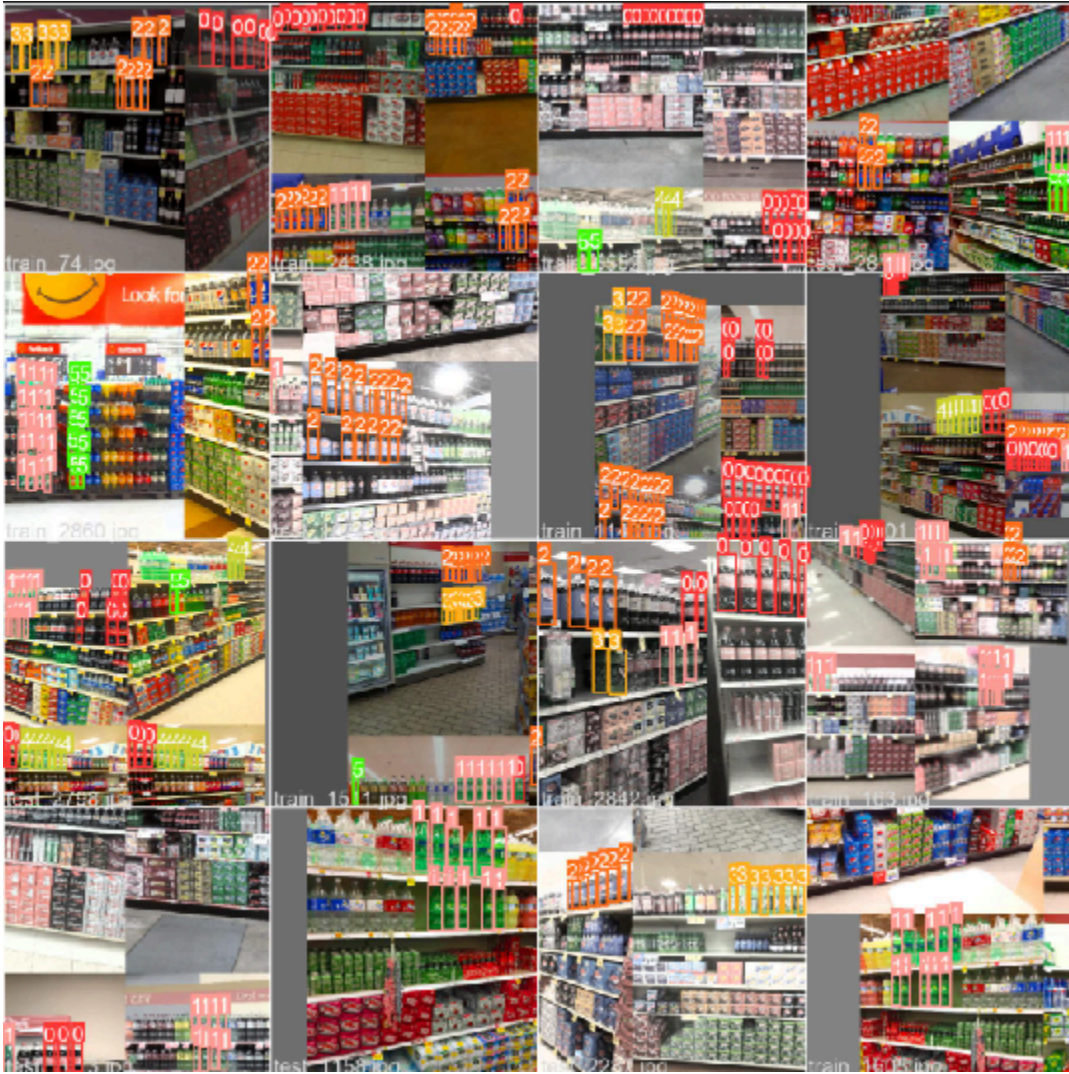
the objectness score (the probability an object is within the anchor)

the class (a number i.e. 1 which corresponds to a class e.g. a person)

the class confidence (the chance that the above class is correct)

the bounding box coordinates i.e. [x, y, w, h] [120, 300, 200, 400]

The input is first passed to CSPDarknet for feature extraction and then fed to PANet for feature fusion (Feature Pyramid). Finally, Yolo Layer outputs detection results (class, score, location, size).



Training Results

Workflow

Cold Drinks Inventory Management System

Mode

None

None

Staff

Admin



Cold Drinks Inventory Management System

Mode

Staff

Staff



Object detection video

Date

2022/10/15

Confidence threshold

0.65

0.00

1.00

View Mode

☒ video

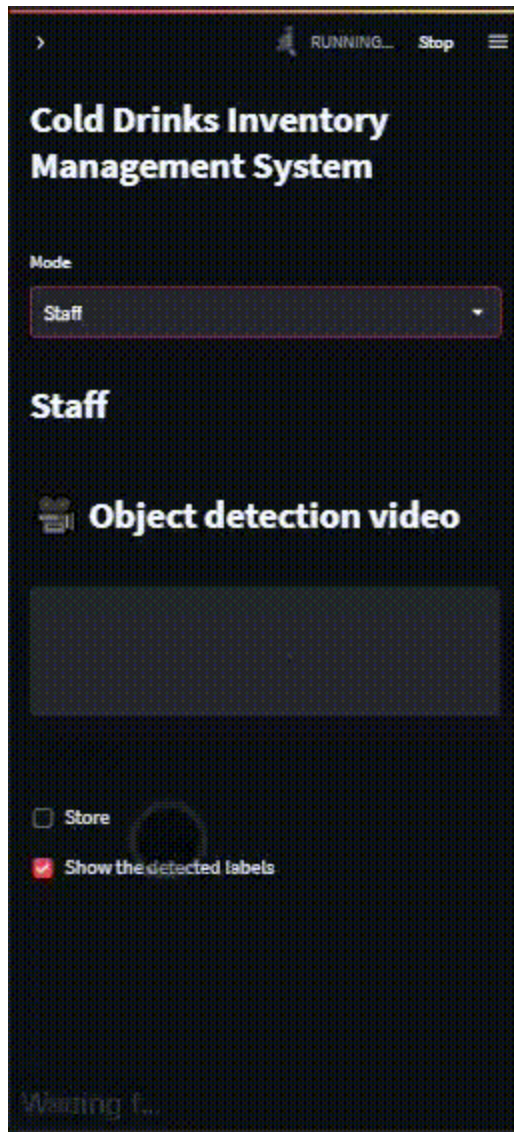
☐ data

☐ image

Landing Page

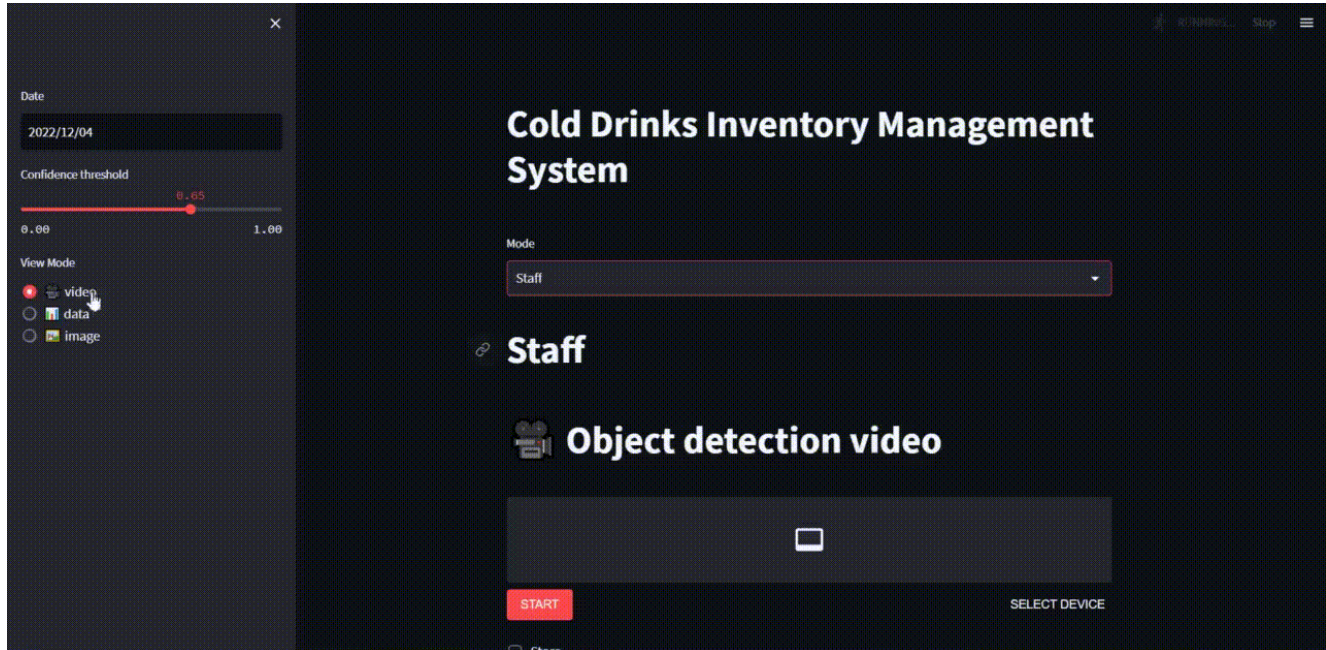
At Landing Page, you have two options either you can go to Staff or Admin.

Utilizing [streamlit_webrtc](#), which facilitates real-time communication, we will be able to detect images in real time by executing detections on the frames captured from the real-time video in the backend.



Real-Time Detections Using Mobile

You could also use drag-and-drop or image upload to carry out detections on images, as seen below.



Performing Classifications using Images

Using three fields—Date, Beverage Name, and Count—we are storing results on our sqlite database, which aids the end user in maintaining inventory and verifying quantities day to day. as shown below

15:42

LTE1 63%



ip-apsm2g.streamlitapp.com



5	2022-08-18	Sprite	5
6	2022-08-18	Mountain Dew	8
7	2022-08-18	Pepsi	4
8	2022-08-17	Mountain Dew	8
9	2022-08-18	Mountain Dew	37
10	2022-08-18	Sprite	12
11	2022-08-18	Fanta	1
12	2022-08-18	Mountain Dew	11
13	2022-08-18	Pepsi	5
14	2022-08-18	Sprite	19
15	2022-08-18	Pepsi	38
16	2022-08-18	Pepsi	31
17	2022-08-17	Sprite	2
18	2022-08-17	Pepsi	17
19	2022-08-17	Coca Cola	25

	count
Coca Cola	34
Fanta	2
Mountain Dew	103
Pepsi	95

Conclusion

In this post, I talked about how to build an Inventory Management System using YOLOv5 object detection model on a Custom Dataset.

If you would like to experiment with the custom dataset yourself, you can download the annotated data on [Kaggle](#) and the code at [Github](#) the web app is deployed [here](#).

Thank you for reading. I'll be producing additional posts for beginners in the future as well. To learn more about them, follow me on [Medium](#). I'm always open to suggestions and constructive criticism, and my Twitter handle is @FaseehAhmed2606.

ting creative content to summarizing lengthy documents and translating languages, LLMs are rapidly transforming how we interact with and leverage textual data. As these models become increasingly integrated into our daily lives and critical systems, the need for robust and comprehensive evaluation methodologies becomes paramount. Assessing the performance of LLMs is crucial not only for understanding their strengths and limitations but also for ensuring their reliability, safety, and alignment with intended use cases. This report provides an enhanced overview of common LLM use cases and delves into the standard evaluation metrics employed to assess their effectiveness in each domain.

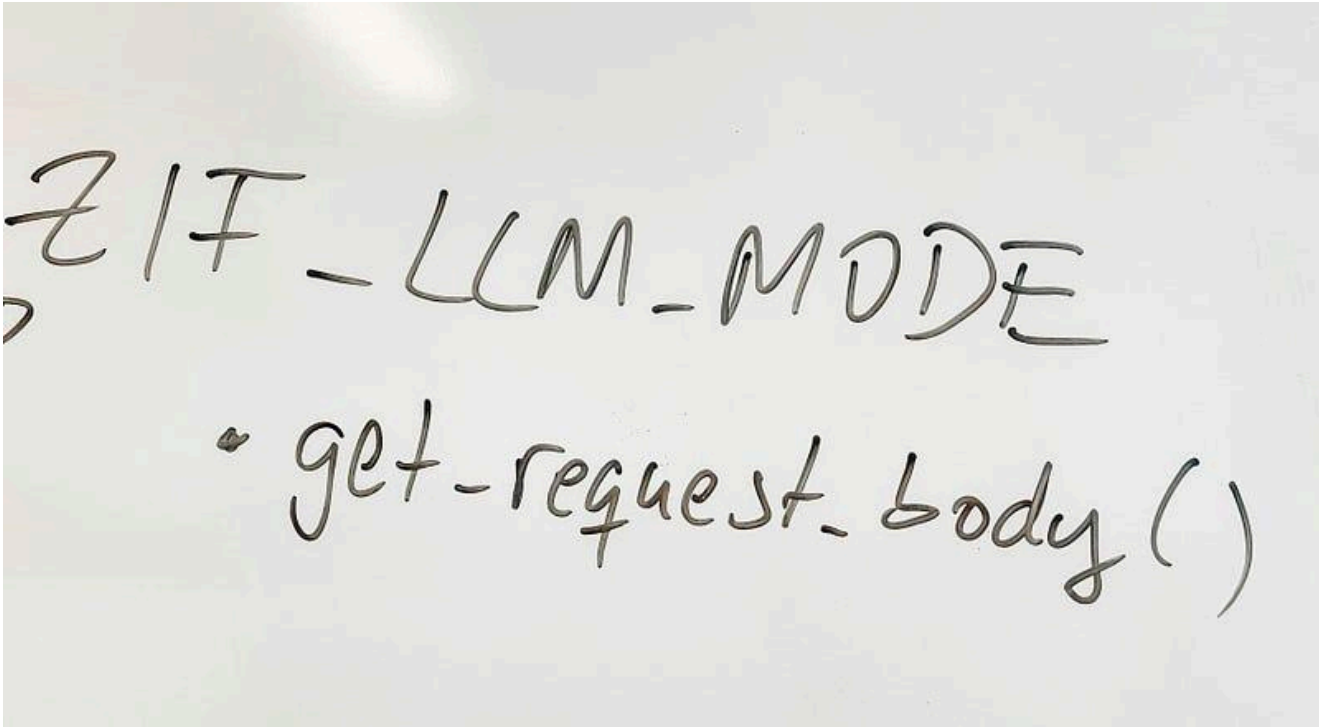


Photo by [Bernd](#)  [Dittrich](#) on [Unsplash](#)

Common Use Cases of Large Language Models

LLMs are versatile tools capable of performing numerous language-related tasks. Their ability to understand, interpret, and generate human-like text has made them invaluable across various industries. Some of the most prevalent applications include:

1. Content Creation and Writing Assistance

LLMs excel at generating articles, blog posts, marketing copy, and even creative content like poetry and screenplays, aiding writers with brainstorming, drafting, and editing. Tools like Grammarly and Notion AI showcase the power of LLMs in enhancing writing quality and efficiency.

2. Sentiment Analysis and Market Research

Organizations use LLMs to understand the emotions conveyed in text data, such as customer feedback on social media or product reviews. This information is valuable for identifying areas of improvement and gauging public opinion.

3. Content Moderation

LLMs can automatically detect and filter out offensive or inappropriate content on social media platforms, ensuring safer online environments.

4. Classification and Pattern Recognition

The ability of LLMs to classify text is leveraged in tasks like fraud detection and supply chain management, where they can identify suspicious patterns and analyze market trends.

5. Domain-Specific Applications

In healthcare and legal sectors, LLMs are being explored for analyzing clinical notes, legal documents, and research papers, assisting professionals with tasks like summarization, information extraction, and document review.

6. Code Generation

LLMs are increasingly used in assisting developers with code completion, bug fixing, and even helping to sharpen development skills through automated feedback and suggestions.

7. Language Translation

LLMs like Meta AI's SeamlessM4T have achieved impressive accuracy in translating across multiple languages, even in real-time voice conversations. This capability is crucial for businesses aiming to expand their global reach and improve cross-cultural communication.

8. Conversational AI and Customer Support

The development of chatbots and virtual assistants that can handle customer inquiries, provide recommendations, and troubleshoot issues in real-time. These conversational agents, exemplified by systems like Salesforce Einstein and Amazon's Alexa, leverage LLMs to engage in natural conversations and perform tasks ranging from setting alarms to automating customer support.

Evaluating the Classifiers

Measuring Classification Accuracy

Text classification, the task of assigning predefined categories or labels to text documents, is a fundamental use case for LLMs. Evaluating the performance of LLMs in this area requires metrics that assess the accuracy and reliability of these assignments. Several standard evaluation metrics are commonly employed:

Accuracy: Measures the overall correctness of the model's predictions, calculated as the ratio of correctly classified instances (both true positives and true negatives) to the total number of instances. While intuitive, accuracy can be misleading on imbalanced datasets.

Precision: Focuses on the accuracy of positive predictions, measuring the proportion of true positives out of all instances predicted as positive. High precision indicates that the model is good at avoiding false positives.

Recall (Sensitivity): Measures the model's ability to identify all actual positive instances, calculated as the ratio of true positives to the total number of actual positives. High recall signifies that the model is effective at avoiding false negatives.

F1-score: Provides a balanced measure by calculating the harmonic mean of precision and recall, proving particularly useful when both metrics are important.

Confusion Matrix: An invaluable tool that visualizes the counts of true positives, true negatives, false positives, and false negatives, allowing for a detailed analysis of the types of errors the model is making.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,  
f1_score, confusion_matrix
```

```
# Example true and predicted labels
```

```
y_true = [0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
```

```
y_pred = [0, 1, 0, 0, 0, 1, 1, 1, 0, 1]
```

Calculate accuracy

```
accuracy = accuracy_score(y_true, y_pred)
```

```
print(f"Accuracy: {accuracy}")
```

Calculate precision

```
precision = precision_score(y_true, y_pred)
```

```
print(f"Precision: {precision}")
```

Calculate recall

```
recall = recall_score(y_true, y_pred)
```

```
print(f"Recall: {recall}")
```

Calculate F1-score

```
f1 = f1_score(y_true, y_pred)
```

```
print(f"F1-score: {f1}")
```

Generate confusion matrix

```
conf_matrix = confusion_matrix(y_true, y_pred)
```

```
print(f"Confusion Matrix:\n{conf_matrix}")
```

For multi-class classification problems, these metrics can be calculated using different averaging methods:

- **Macro averaging:** Calculates the metric for each class independently and then takes the average (treating all classes equally)
- **Micro averaging:** Aggregates the contributions of all classes to compute the average metric (giving more weight to larger classes)
- **Weighted averaging:** Similar to macro averaging, but calculates the average weighted by the number of true instances for each class

For multi-class classification

```
from sklearn.metrics import precision_recall_fscore_support
```

Compute precision, recall, and F1 with different averaging methods

```
macro_metrics = precision_recall_fscore_support(y_true, y_pred,  
average='macro')
```

```
micro_metrics = precision_recall_fscore_support(y_true, y_pred, average='micro')
```

```
weighted_metrics = precision_recall_fscore_support(y_true, y_pred,  
average='weighted')
```

```
print(f"Macro metrics (precision, recall, F1): {macro_metrics[:3]}")
```

```
print(f"Micro metrics (precision, recall, F1): {micro_metrics[:3]}")
```

```
print(f"Weighted metrics (precision, recall, F1): {weighted_metrics[:3]}")
```

ROC and AUC

For binary classification, we can also use the Receiver Operating Characteristic (ROC) curve, it plots the true positive rate against the false positive rate at various threshold settings. The Area Under the ROC Curve (AUC) provides a single scalar value that indicates the model's ability to distinguish between classes.

```
from sklearn.metrics import roc_curve, auc

import matplotlib.pyplot as plt

# For probability predictions

y_score = [0.1, 0.9, 0.2, 0.3, 0.1, 0.8, 0.6, 0.9, 0.2, 0.7] # Example probability scores

# Calculate ROC curve and AUC

fpr, tpr, thresholds = roc_curve(y_true, y_score)

roc_auc = auc(fpr, tpr)

# Plot ROC curve

plt.figure()

plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')

plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')

plt.xlim([0.0, 1.0])

plt.ylim([0.0, 1.05])

plt.xlabel('False Positive Rate')

plt.ylabel('True Positive Rate')

plt.title('Receiver Operating Characteristic')

plt.legend(loc="lower right")

plt.show()
```


Assessing the Summarizers

Measuring Information Condensation

Text summarization, where LLMs condense longer texts into shorter, coherent summaries, presents its own set of evaluation challenges. Unlike classification, there isn't always a single "correct" summary, and the quality often depends on the context and purpose of the summarization. Several metrics have been developed to assess the quality of generated summaries:

ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

ROUGE evaluates summarization quality by measuring the overlap of n-grams between the generated summary and one or more human-written reference summaries. Different variations include:

- ROUGE-N: Measures the overlap of unigrams, bigrams, etc.
- ROUGE-L: Focuses on the longest common subsequence
- ROUGE-S: Considers skip-bigrams, allowing for gaps between words

```
from rouge_score import rouge_scorer
```

```
# Example reference and candidate summaries
```

```
reference_summary = "The cat was under the bed."
```

```
candidate_summary = "The cat was found under the bed."
```

```
# Initialize ROUGE scorer
```

```
scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'],  
use_stemmer=True)
```

```
# Calculate ROUGE scores
```

```
scores = scorer.score(reference_summary, candidate_summary)

print("ROUGE Scores:", scores)
```

BLEU (Bilingual Evaluation Understudy)

Originally designed for machine translation, BLEU measures the n-gram overlap between the generated text and reference text. It is a precision-focused metric that assesses how much of the words in the machine-generated summary appear in the human reference summaries.

```
from nltk.translate.bleu_score import sentence_bleu

from nltk.tokenize import word_tokenize
```

```
# Example reference and candidate summaries
```

```
reference_summary = "The cat is on the mat."
```

```
candidate_summary = "The cat is sitting on the mat."
```

```
# Tokenize the summaries
```

```
reference_tokens = [word_tokenize(reference_summary)]
```

```
candidate_tokens = word_tokenize(candidate_summary)
```

```
# Calculate BLEU score
```

```
bleu_score = sentence_bleu(reference_tokens, candidate_tokens)
```

```
print("BLEU Score:", bleu_score)
```

METEOR (Metric for Evaluation of Translation with Explicit ORdering)

Similar to ROUGE but incorporates additional features like stemming and synonymy matching using resources like WordNet. It focuses on unigram matching and calculates a weighted F-score, giving more weight to recall than precision. METEOR also includes a fragmentation penalty for poorly ordered phrases.

```
from nltk.translate.meteor_score import meteor_score
```

```
from nltk.tokenize import word_tokenize
```

```
# Example reference and candidate summaries
```

```
reference_summary = "The cat is on the mat."
```

```
candidate_summary = "The cat is sitting on the mat."
```

```
# Tokenize the summaries
```

```
reference_tokens = word_tokenize(reference_summary)
```

```
candidate_tokens = word_tokenize(candidate_summary)
```

```
# Calculate METEOR score
```

```
meteor = meteor_score([reference_tokens], candidate_tokens)
```

```
print("METEOR Score:", meteor)
```

BERTScore

BERTScore leverages the contextual embeddings from the BERT model to evaluate the semantic similarity between a generated summary and a reference summary. By comparing the embeddings of words in both summaries, BERTScore

can capture semantic similarities that might be missed by traditional n-gram overlap metrics.

```
from bert_score import score
```

```
# Example reference and candidate summaries
```

```
reference_summary = "The cat is on the mat."
```

```
candidate_summary = "The cat is sitting on the mat."
```

```
# Calculate BERTScore
```

```
P, R, F1 = score([candidate_summary], [reference_summary], lang="en")
```

```
print(f"BERTScore - Precision: {P.item():.4f}, Recall: {R.item():.4f}, F1 Score:  
{F1.item():.4f}")
```

SUPERT (SUMmarization PERFORMANCE with Transformers)

A reference-free evaluation metric that uses pre-trained contextual representations to assess the quality of summarization without requiring human-written reference summaries. It measures how well the summary captures the salient information in the source document.

```
# Example of how SUPERT could be implemented
```

```
from transformers import AutoTokenizer, AutoModel
```

```
import torch
```

```
import numpy as np
```

```
def get_embeddings(text, model, tokenizer):
```

```
inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True,
max_length=512)
```

```
with torch.no_grad():
```

```
    outputs = model(**inputs)
```

```
return outputs.last_hidden_state.mean(dim=1).squeeze().numpy()
```

```
def supert_score(source_doc, summary, model_name="bert-base-uncased"):
```

```
    tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
    model = AutoModel.from_pretrained(model_name)
```

```
# Get embeddings
```

```
doc_embedding = get_embeddings(source_doc, model, tokenizer)
```

```
summary_embedding = get_embeddings(summary, model, tokenizer)
```

```
# Calculate cosine similarity
```

```
similarity = np.dot(doc_embedding, summary_embedding) /
(np.linalg.norm(doc_embedding) * np.linalg.norm(summary_embedding))
```

```
return similarity
```

```
# Example usage
```

```
source_document = "The cat quickly jumped over the lazy dog. It was a sunny day  
and the dog was sleeping peacefully in the garden."
```

```
summary = "The cat jumped over the sleeping dog on a sunny day."
```

```
score = supert_score(source_document, summary)
```

```
print(f"SUPERT Score: {score:.4f}")
```

Judging the Generators

Navigating the Nuances of Creative Text

Evaluating the performance of LLMs in creative text generation poses unique challenges due to the open-ended and subjective nature of such tasks. Several evaluation approaches are employed:

Perplexity

Perplexity is a measure that tells us how “surprised” or uncertain a language model is when predicting the next word in a text. Imagine you’re reading a story and at each word, you have several choices about what might come next. Perplexity quantifies the average number of choices the model effectively considers, so lower perplexity means the model is more confident and accurate in its predictions.

This metric was introduced because, as language models grew more complex and started assigning probabilities to thousands of possible words, comparing raw probabilities became difficult. Perplexity simplifies this by turning the model’s uncertainty into a single number: the lower the number, the better the model is at capturing the language’s structure and predicting what comes next.

```
from transformers import AutoModelForCausalLM, AutoTokenizer
```

```
import torch
```

Example text and model

text = "The quick brown fox jumps over the lazy dog."

model_name = "gpt2"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(model_name)

model.eval()

Tokenize the text

input_ids = tokenizer.encode(text, return_tensors="pt")

Get model outputs

with torch.no_grad():

outputs = model(input_ids, labels=input_ids)

Calculate perplexity

loss = outputs.loss

perplexity = torch.exp(loss)

print(f"Perplexity: {perplexity.item()}")

Diversity Metrics

Diversity metrics measure how varied and original the generated text is. One common metric, distinct-n, calculates the percentage of unique sequences of words (called n-grams) compared to all sequences in the text. In simple terms, if a model keeps repeating the same phrases, its distinct-n score will be low; if it uses a wide range of different phrases, the score will be high. This helps us understand whether the generated content feels fresh and creative or if it's just repetitive.

```
def distinct_n_grams(text, n):
```

```
    """Calculate the ratio of unique n-grams to total n-grams in the text."""
```

```
    tokens = text.split()
```

```
    if len(tokens) < n:
```

```
        return 0.0
```

```
    n_grams = [' '.join(tokens[i:i+n]) for i in range(len(tokens)-n+1)]
```

```
    n_grams_set = set(n_grams)
```

```
    return len(n_grams_set) / len(n_grams)
```

```
# Example usage
```

```
generated_text = "The cat sat on the mat. The dog sat on the log. The frog sat on  
the bog."
```

```
distinct_1 = distinct_n_grams(generated_text, 1) # Distinct-1 (words)
```

```
distinct_2 = distinct_n_grams(generated_text, 2) # Distinct-2 (bigrams)
```



```
print(f"Distinct-1: {distinct_1:.4f}")
```

```
print(f"Distinct-2: {distinct_2:.4f}")
```

Self-BLEU

Self-BLEU is a measure of diversity for a set of generated texts. It works by comparing each text with all the others using the BLEU score, a metric that shows how similar two texts are. A lower Self-BLEU score means the texts are more different from each other, indicating greater diversity and originality.

```
from nltk.translate.bleu_score import sentence_bleu
```

```
from nltk.tokenize import word_tokenize
```

```
def self_bleu(generated_texts):
```

```
    """Calculate the average BLEU score of each text against all others."""
```

```
    if len(generated_texts) < 2:
```

```
        return 0.0
```

```
    bleu_scores = []
```

```
    for i, text in enumerate(generated_texts):
```

```
        reference_texts = [t for j, t in enumerate(generated_texts) if j != i]
```

```
        reference_tokens = [word_tokenize(ref) for ref in reference_texts]
```

```
        candidate_tokens = word_tokenize(text)
```

```
        bleu = sentence_bleu(reference_tokens, candidate_tokens)
```

```
bleu_scores.append(bleu)
```

```
return sum(bleu_scores) / len(bleu_scores)
```

Example usage

```
generated_texts = [
```

```
    "The cat sat on the mat.",
```

```
    "The dog chased the ball in the park.",
```

```
    "The bird flew over the mountain."
```

```
]
```

```
self_bleu_score = self_bleu(generated_texts)
```

```
print(f"Self-BLEU Score: {self_bleu_score:.4f}")
```

for this example you will get a self-bleu as 0 as each text example used here is different. I have added additional examples in the [colab book](#)

Human Evaluation

Given the subjective nature of creative text, human evaluation often plays a crucial role in assessing the quality of LLM-generated content. Human evaluators can assess various aspects such as fluency, coherence, relevance to the prompt, factual consistency (if applicable), and the overall quality and engagingness of the generated text.

Evaluating Code Generation

Comprehensive Assessment Framework

Code generation has emerged as a significant application of LLMs, with models like GitHub Copilot and OpenAI's Codex demonstrating impressive capabilities. Evaluating code generation requires metrics that assess not only syntactic correctness but also functional accuracy and code quality.

Functional Correctness: Pass@k

The Pass@k metric measures the probability that at least one of the top k generated code samples passes a set of predefined unit tests. This is a widely used metric for evaluating code generation models, as it directly assesses whether the generated code accomplishes its intended functionality.

```
def compute_pass_at_k(n_samples, n_correct, k):
```

```
    """
```

```
    Compute Pass@k metric where:
```

- n_samples: Number of samples per problem
- n_correct: Number of correct samples per problem
- k: The k in Pass@k

```
    """
```

```
    if n_samples < k:
```

```
        return 1.0 if n_correct > 0 else 0.0
```

```
    return 1.0 - math.comb(n_samples - n_correct, k) / math.comb(n_samples, k)
```

Example: For a problem where we generate 10 solutions and 3 are correct

```
n_samples = 10
```

```
n_correct = 3
```

```
pass_at_1 = compute_pass_at_k(n_samples, n_correct, 1)
```

```
pass_at_5 = compute_pass_at_k(n_samples, n_correct, 5)
```

```
pass_at_10 = compute_pass_at_k(n_samples, n_correct, 10)
```

```
print(f"Pass@1: {pass_at_1:.4f}")
```

```
print(f"Pass@5: {pass_at_5:.4f}")
```

```
print(f"Pass@10: {pass_at_10:.4f}")
```

HumanEval Benchmark

The HumanEval benchmark, developed by OpenAI, has become a standard for evaluating code generation models. It consists of 164 Python programming problems that test a model's ability to generate functionally correct code from natural language descriptions.

```
import random
```

```
def evaluate_humaneval_solution(problem_id, generated_code):
```

```
    """
```

Simulate the evaluation of a generated code solution for a HumanEval problem.

In a production system, this function would:

1. Retrieve the specific HumanEval problem and its test cases.

2. Execute the generated code safely (e.g., in a sandbox).

3. Run the test cases and record which ones pass.

For this simulation, we assume there are 5 test cases per problem and randomly determine if each test case passes. The overall solution is considered correct only if all test cases pass.

"""

Simulate running 5 test cases; True indicates the test passed, False indicates a failure.

test_case_results = [random.choice([True, False]) for _ in range(5)]

Overall, the solution "passes" if all test cases pass.

solution_passes = all(test_case_results)

Return a dictionary with evaluation details.

return {

"problem_id": problem_id,

"test_case_results": test_case_results,

"passes": solution_passes

}

Example usage

problem_id = "HumanEval/42"

generated_code = """

def add(a, b):

A simple function that returns the sum of a and b

return a + b

"""

result = evaluate_humaneval_solution(problem_id, generated_code)

print(f"Evaluation for Problem {result['problem_id']}:")

print("Test Case Results:", result['test_case_results'])

print("Overall Result:", "Passed" if result['passes'] else "Failed")

CodeBLEU

CodeBLEU extends the traditional BLEU metric by incorporating syntactic and semantic information specific to code. It consists of four components:

- 1. N-gram Match: Similar to traditional BLEU, measuring lexical similarity**
- 2. Weighted N-gram Match: Gives higher weights to keywords and identifiers**
- 3. Syntactic AST Match: Compares the Abstract Syntax Tree structures**
- 4. Semantic Data-flow Match: Evaluates the semantic correctness based on data flow.**

def calculate_codebleu(reference_code, candidate_code, weights=[0.25, 0.25, 0.25, 0.25]):

"""

Calculate CodeBLEU score with four components:

1. N-gram match (similar to BLEU)
2. Weighted n-gram match (emphasizing keywords)
3. AST match
4. Data-flow match

This is a simplified implementation - actual CodeBLEU would use more complex analysis.

```
.....
```

```
from nltk.translate.bleu_score import sentence_bleu  
  
from nltk.tokenize import word_tokenize  
  
import ast
```

```
# 1. N-gram match (simplified BLEU)
```

```
ref_tokens = word_tokenize(reference_code)  
  
cand_tokens = word_tokenize(candidate_code)  
  
ngram_match = sentence_bleu([ref_tokens], cand_tokens)
```

```
# 2. Weighted n-gram match (simulation)
```

```
# In actual implementation, this would weight keywords more heavily
```

```
weighted_ngram = ngram_match * 1.2 # Simplified simulation
```

3. AST match (simplified)

try:

ref_ast = ast.parse(reference_code)

cand_ast = ast.parse(candidate_code)

Simple AST comparison (in reality, this would be more complex)

ast_match = 0.7 # Simulated value

except SyntaxError:

ast_match = 0.0 # If code has syntax errors

4. Data-flow match (simulation)

This would require actual data-flow analysis

dataflow_match = 0.65 # Simulated value

Combine components using weights

codebleu = (weights[0] * ngram_match +

weights[1] * weighted_ngram +

weights[2] * ast_match +

weights[3] * dataflow_match)

return codebleu

Example usage

reference_code = """

def factorial(n):

if n == 0 or n == 1:

return 1

else:

return n * factorial(n-1)

"""

candidate_code = """

def factorial(n):

result = 1

for i in range(1, n+1):

result *= i

return result

"""

codebleu_score = calculate_codebleu(reference_code, candidate_code)

print(f"CodeBLEU Score: {codebleu_score:.4f}")

Execution Time and Efficiency

Beyond correctness, the efficiency of the generated code is another important aspect to evaluate, especially for performance-critical applications.

```
import time
```

```
import statistics
```

```
def measure_execution_time(code_func, test_cases, runs=5):
```

```
    """Measure the execution time of a function across multiple test cases and runs."""
```

```
    all_times = []
```

```
    for test_case in test_cases:
```

```
        case_times = []
```

```
        for _ in range(runs):
```

```
            start_time = time.time()
```

```
            code_func(*test_case)
```

```
            end_time = time.time()
```

```
            case_times.append(end_time - start_time)
```

```
        # Take the median to reduce the impact of outliers
```

```
        all_times.append(statistics.median(case_times))
```

```
    return {
```

```
"mean": statistics.mean(all_times),  
"median": statistics.median(all_times),  
"min": min(all_times),  
"max": max(all_times)  
}
```

Example recursive factorial function

```
def factorial_recursive(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial_recursive(n-1)
```

Example iterative factorial function

```
def factorial_iterative(n):  
    result = 1  
    for i in range(1, n+1):  
        result *= i  
    return result
```

Test cases

```
test_cases = [(5,), (10,), (15,), (20,)]
```

```
# Measure execution times
```

```
recursive_times = measure_execution_time(factorial_recursive, test_cases)
```

```
iterative_times = measure_execution_time(factorial_iterative, test_cases)
```

```
print("Recursive Factorial Execution Times (seconds):")
```

```
for key, value in recursive_times.items():
```

```
    print(f" {key}: {value:.6f}")
```

```
print("\nIterative Factorial Execution Times (seconds):")
```

```
for key, value in iterative_times.items():
```

```
    print(f" {key}: {value:.6f}")
```

Code Complexity Metrics

Evaluating the complexity of generated code provides insights into its maintainability and readability.

```
import ast
```

```
import math
```

```
def calculate_cyclomatic_complexity(code):
```

```
    """
```

Calculate the cyclomatic complexity of Python code.

A higher score indicates more complex code.

"""

class ComplexityVisitor(ast.NodeVisitor):

def __init__(self):

self.complexity = 1 # Start with 1 for the entry point

def visit_If(self, node):

self.complexity += 1

self.generic_visit(node)

def visit_For(self, node):

self.complexity += 1

self.generic_visit(node)

def visit_While(self, node):

self.complexity += 1

self.generic_visit(node)

def visit_Try(self, node):

self.complexity += len(node.handlers) # Count except blocks

```
self.generic_visit(node)
```

```
def visit_BoolOp(self, node):
```

```
    if isinstance(node.op, ast.And) or isinstance(node.op, ast.Or):
```

```
        self.complexity += len(node.values) - 1
```

```
    self.generic_visit(node)
```

```
try:
```

```
    tree = ast.parse(code)
```

```
    visitor = ComplexityVisitor()
```

```
    visitor.visit(tree)
```

```
    return visitor.complexity
```

```
except SyntaxError:
```

```
    return math.inf # Return infinity for code with syntax errors
```

```
# Example usage
```

```
code1 = """
```

```
def simple_function(x):
```

```
    return x + 1
```

```
"""
```

```
code2 = """
```

```
def complex_function(x):
```

```
    if x > 0:
```

```
        if x < 10:
```

```
            return x * 2
```

```
        else:
```

```
            for i in range(x):
```

```
                if i % 2 == 0:
```

```
                    print(i)
```

```
            return x * 3
```

```
    else:
```

```
        try:
```

```
            return 10 / x
```

```
        except ZeroDivisionError:
```

```
            return 0
```

```
"""
```

```
cc1 = calculate_cyclomatic_complexity(code1)
```

```
cc2 = calculate_cyclomatic_complexity(code2)
```

```
print(f"Simple function cyclomatic complexity: {cc1}")
```

```
print(f"Complex function cyclomatic complexity: {cc2}")
```

Real-world Code Generation Evaluation Results

Here are some representative results from recent evaluations of code generation models:

HumanEval Benchmark Results

Model	▼	#	Pass@1 (%)	▼	#	Pass@10 (%)	▼	#	Pass@100 (%)	▼
OpenAI Codex			28.8			46.8			72.3	
GPT-3.5			48.1			65.7			78.3	
GPT-4			67			80.2			92.1	
Anthropic Claude			52.3			71.4			84.5	
Meta's Code Llama 34B			48.4			67.8			81.9	
DeepSeek-Coder			61.8			83.1			91.2	

CodeXGLUE Benchmark (Python Generation Task)

Model	▼	BLEU	▼	CodeBLEU	▼	#	Execution Accuracy (%)	▼
CodeGPT		18.2		29.5			34.8	
PLBART		19.3		32.1			38.2	
CodeT5		20.1		36.4			42.7	
StarCoder		25.7		42.8			56.3	
DeepSeek-Coder		28.4		45.9			63.1	
Claude 3 Opus		29.2		48.3			68.4	

These results highlight the rapid progress in code generation capabilities, with newer models achieving significantly higher functional correctness and code quality metrics compared to earlier versions.

Evaluating Other LLM Applications: Tailored Approaches

Beyond classification, summarization, generation, and code generation, LLMs are applied to a variety of other tasks, each requiring specific evaluation metrics tailored to the nuances of the use case.

Question Answering (QA)

The goal is to provide accurate answers to user-posed questions. Common evaluation metrics include:

- **Exact Match (EM):** Measures the percentage of predicted answers that perfectly match the ground truth answer
- **F1-score:** Calculates the overlap between the predicted and the ground truth answer at the word level
- **SQuAD metrics:** Specifically designed for the Stanford Question Answering Dataset, measuring both exact match and F1-score for reading comprehension tasks

```
def calculate_qa_metrics(predicted_answers, ground_truth_answers):
```

```
    """
```

```
        Calculate Exact Match and F1 score for QA tasks.
```

```
    Args:
```

```
        predicted_answers: List of predicted answers
```

```
        ground_truth_answers: List of lists of ground truth answers (multiple  
        acceptable answers per question)
```

```
    Returns:
```

```
        Dictionary with EM and F1 scores
```

```
    """
```

```
exact_match = 0
```

```
f1_scores = []
```

```
for pred, truths in zip(predicted_answers, ground_truth_answers):
```

```
    # Normalize answers
```

```
    pred = normalize_answer(pred)
```

```
    truths = [normalize_answer(truth) for truth in truths]
```

```
    # Check for exact match
```

```
    if pred in truths:
```

```
        exact_match += 1
```

```
    # Calculate F1 score (best match among the ground truths)
```

```
    best_f1 = 0
```

```
    for truth in truths:
```

```
        current_f1 = calculate_f1(pred, truth)
```

```
        best_f1 = max(best_f1, current_f1)
```

```
    f1_scores.append(best_f1)
```

```
return {
```

```
"exact_match": exact_match / len(predicted_answers) * 100,  
"f1": sum(f1_scores) / len(f1_scores) * 100  
}
```

```
def normalize_answer(text):
```

```
    """Normalize answer by removing articles, punctuation, and extra whitespace."""
```

```
    import re
```

```
    import string
```

```
    # Remove articles
```

```
    text = re.sub(r'\b(a|an|the)\b', ' ', text.lower())
```

```
    # Remove punctuation
```

```
    text = ''.join(ch for ch in text if ch not in string.punctuation)
```

```
    # Normalize whitespace
```

```
    text = ' '.join(text.split())
```

```
    return text
```

```
def calculate_f1(prediction, truth):
```

"""Calculate F1 score between prediction and ground truth."""

prediction_tokens = prediction.split()

truth_tokens = truth.split()

Empty answers

if len(prediction_tokens) == 0 or len(truth_tokens) == 0:

return int(prediction_tokens == truth_tokens)

Count common tokens

common = sum(1 for token in prediction_tokens if token in truth_tokens)

If no common tokens, F1 = 0

if common == 0:

return 0

precision = common / len(prediction_tokens)

recall = common / len(truth_tokens)

f1 = 2 * precision * recall / (precision + recall)

return f1

Example usage

```
predicted_answers = ["new york city", "albert einstein", "42 kilometers"]

ground_truth_answers = [
    ["New York City", "NYC", "New York"],
    ["Albert Einstein", "Einstein"],
    ["42 km", "42 kilometers", "forty-two kilometers"]
]

qa_metrics = calculate_qa_metrics(predicted_answers, ground_truth_answers)

print(f"QA Metrics:")

print(f" Exact Match: {qa_metrics['exact_match']:.2f}%")

print(f" F1 Score: {qa_metrics['f1']:.2f}%")
```

Dialogue Generation

For conversational agents, evaluation is complex due to the interactive and context-dependent nature of dialogues. Key metrics include:

- **BLEU, ROUGE, METEOR:** Basic measures of word overlap with reference responses
- **Distinct-n:** Assesses the diversity of the generated responses by measuring the proportion of unique n-grams
- **USR (UnSupervised and Reference-free):** Evaluates without requiring reference dialogues
- **Response Length:** Measures the verbosity of the model's responses
- **Perplexity:** Measures how well the model predicts the next token in a conversation.

```
def evaluate_dialogue(generated_responses, reference_responses,  
dialogue_contexts):
```

```
    """
```

Comprehensive evaluation of dialogue generation.

Args:

generated_responses: List of model-generated responses

reference_responses: List of human/reference responses

dialogue_contexts: List of dialogue contexts preceding the responses

Returns:

Dictionary with various dialogue evaluation metrics

```
    """
```

```
from nltk.translate.bleu_score import sentence_bleu
```

```
from nltk.tokenize import word_tokenize
```

```
results = {
```

```
    "bleu_scores": [],
```

```
    "distinctness": {
```

```
        "distinct-1": [],
```

```
        "distinct-2": []
```

```
},  
  
"response_lengths": [],  
  
"context_relevance": []  
  
}
```

```
for gen, ref, context in zip(generated_responses, reference_responses,  
dialogue_contexts):
```

```
    # Tokenize
```

```
    gen_tokens = word_tokenize(gen.lower())
```

```
    ref_tokens = word_tokenize(ref.lower())
```

```
    context_tokens = word_tokenize(context.lower())
```

```
    # BLEU score
```

```
    bleu = sentence_bleu([ref_tokens], gen_tokens)
```

```
    results["bleu_scores"].append(bleu)
```

```
    # Distinctness
```

```
    results["distinctness"]["distinct-1"].append(len(set(gen_tokens)) / max(1,  
len(gen_tokens)))
```

```
    bigrams = [' '.join(gen_tokens[i:i+2]) for i in range(len(gen_tokens)-1)]
```

```
results["distinctness"]["distinct-2"].append(len(set(bigrams)) / max(1, len(bigrams)))
```

```
# Response length
```

```
results["response_lengths"].append(len(gen_tokens))
```

```
# Simple context relevance (word overlap with context)
```

```
common_words = len(set(gen_tokens) & set(context_tokens))
```

```
relevance = common_words / max(1, len(set(gen_tokens)))
```

```
results["context_relevance"].append(relevance)
```

```
# Aggregate results
```

```
for key in results:
```

```
    if isinstance(results[key], list):
```

```
        results[key] = sum(results[key]) / len(results[key])
```

```
    elif isinstance(results[key], dict):
```

```
        for subkey in results[key]:
```

```
            results[key][subkey] = sum(results[key][subkey]) / len(results[key][subkey])
```

```
return results
```


Example usage

```
contexts = [
```

```
    "Hello, how are you doing today?",
```

```
    "Can you recommend a good restaurant in the area?",
```

```
    "What's the weather forecast for tomorrow?"
```

```
]
```

```
generated_responses = [
```

```
    "I'm doing well, thank you for asking!",
```

```
    "There are several great options nearby. I recommend trying Bella Italia on Main  
Street.",
```

```
    "Tomorrow will be sunny with a high of 75 degrees."
```

```
]
```

```
reference_responses = [
```

```
    "I'm fine, thanks! How about you?",
```

```
    "You should check out Luigi's Pizza downtown. It's fantastic!",
```

```
    "Tomorrow will be clear with temperatures around 73-76 degrees."
```

```
]
```

```
dialogue_metrics = evaluate_dialogue(generated_responses,  
reference_responses, contexts)
```

```

print("Dialogue Evaluation Metrics:")

for key, value in dialogue_metrics.items():

    if isinstance(value, dict):

        print(f" {key}:")

        for subkey, subvalue in value.items():

            print(f"   {subkey}: {subvalue:.4f}")

    else:

        print(f" {key}: {value:.4f}")

```

Translation

The task of converting text from one language to another. Key metrics include:

- **BLEU:** Measures the n-gram precision of the machine-translated text compared to human references
- **NIST:** Similar to BLEU but gives more weight to rarer, more informative n-grams
- **TER (Translation Edit Rate):** Measures the number of edits required to transform the machine translation output into a reference translation
- **chrF:** Character-level F-score that captures sub-word information
- **COMET:** A neural metric that uses contextualized embeddings to evaluate translation quality.

```
def evaluate_translation(source_texts, translations, references):
```

```
    """
```

Evaluate machine translation quality using multiple metrics.

Args:

source_texts: List of source language texts

translations: List of machine-translated texts

references: List of human reference translations

Returns:

Dictionary with various translation metrics

.....

```
from nltk.translate.bleu_score import corpus_bleu
```

```
from nltk.tokenize import word_tokenize
```

```
# Tokenize all texts
```

```
tokenized_translations = [word_tokenize(t.lower()) for t in translations]
```

```
tokenized_references = [[word_tokenize(r.lower())] for r in references]
```

```
# Calculate corpus BLEU
```

```
bleu_score = corpus_bleu(tokenized_references, tokenized_translations)
```

```
# Calculate character-based metrics (simplified chrF implementation)
```

```
def char_precision_recall_f1(hyp, ref):
```

```
    hyp_chars = list(hyp.lower())
```

```
    ref_chars = list(ref.lower())
```

```
common_chars = sum(1 for c in hyp_chars if c in ref_chars)
```

```
precision = common_chars / max(1, len(hyp_chars))
```

```
recall = common_chars / max(1, len(ref_chars))
```

```
if precision + recall == 0:
```

```
    return 0
```

```
f1 = 2 * precision * recall / (precision + recall)
```

```
return f1
```

```
chrf_scores = [char_precision_recall_f1(t, r) for t, r in zip(translations,  
references)]
```

```
avg_chrf = sum(chrf_scores) / len(chrf_scores)
```

```
# Calculate Translation Edit Rate (simplified version)
```

```
def calculate_ter(hyp, ref):
```

```
    # Using Levenshtein distance as a simple approximation
```

```
    import Levenshtein
```

```
    hyp_tokens = word_tokenize(hyp.lower())
```

```
    ref_tokens = word_tokenize(ref.lower())
```

```
edit_distance = Levenshtein.distance(hyp_tokens, ref_tokens)

return edit_distance / max(1, len(ref_tokens))


ter_scores = [calculate_ter(t, r) for t, r in zip(translations, references)]

avg_ter = sum(ter_scores) / len(ter_scores)


return {

    "bleu": bleu_score * 100, # Convert to percentage

    "chrf": avg_chrf * 100,  # Convert to percentage

    "ter": avg_ter * 100     # Convert to percentage (lower is better)

}
```

Example usage

```
source_texts = [

    "Das Wetter ist heute schön.",

    "Ich liebe Programmierung.",

    "Die Katze schläft auf dem Sofa."

]
```

```
translations = [
```

```
"The weather is nice today.",
"I love programming.",
"The cat is sleeping on the couch."
]

references = [
    "The weather is beautiful today.",
    "I love programming.",
    "The cat is sleeping on the sofa."
]

translation_metrics = evaluate_translation(source_texts, translations, references)
print("Translation Evaluation Metrics:")

for metric, score in translation_metrics.items():
    print(f" {metric}: {score:.2f}%")
```

Integrated Evaluation Frameworks and Benchmarks

Integrated evaluation frameworks and benchmarks provide a standardized way to assess the performance of large language models (LLMs) across a wide range of tasks, ensuring fair comparisons between models. Here's an overview of some of the key frameworks and emerging evaluation approaches

GLUE and SuperGLUE

- **GLUE (General Language Understanding Evaluation):**
GLUE is a benchmark composed of multiple natural language understanding tasks, such as sentiment analysis, textual entailment, and question answering. It was designed to assess how well a model can understand and process natural language by providing a single score that aggregates performance across these diverse tasks.
- **SuperGLUE:**
As a successor to GLUE, SuperGLUE is even more challenging, incorporating tasks that require deeper reasoning and more nuanced language understanding. It pushes models to perform better on tasks like commonsense reasoning and complex inference, thus serving as a tougher yardstick for state-of-the-art models.

MMLU (Massive Multitask Language Understanding)

MMLU is a comprehensive benchmark that evaluates a model's knowledge across 57 different subjects spanning humanities, STEM, social sciences, and more. It is designed to test both the breadth and depth of a model's understanding:

- **Breadth:** Measures the model's ability to handle a wide variety of topics.
- **Depth:** Assesses how well the model understands complex or specialized content within each subject area.

LLM-as-a-Judge

This is an emerging approach where one LLM is used to evaluate the outputs of another. Instead of relying solely on traditional metrics (like BLEU or ROUGE), this method leverages the language understanding and reasoning capabilities of an LLM to provide more nuanced, human-like judgments. Key aspects include:

- **Complex Distinction Based Assessments:** The evaluating LLM can consider multiple facets such as coherence, factual accuracy, creativity, and relevance.
- **Reference-Free Evaluation:** This method can assess outputs even without a fixed reference, which is particularly useful for open-ended or creative tasks where there isn't a single "correct" answer.
- **Breadth:** Measures the model's ability to handle a wide variety of topics.

- **Depth:** Assesses how well the model understands complex or specialized content within each subject area.

LLM-as-a-Judge

This is an emerging approach where one LLM is used to evaluate the outputs of another. Instead of relying solely on traditional metrics (like BLEU or ROUGE), this method leverages the language understanding and reasoning capabilities of an LLM to provide more nuanced, human-like judgments. Key aspects include:

- **Nuanced Assessments:** The evaluating LLM can consider multiple facets such as coherence, factual accuracy, creativity, and relevance.
 - **Reference-Free Evaluation:** This method can assess outputs even without a fixed reference, which is particularly useful for open-ended or creative tasks where there isn't a single "correct" answer.
-

Navigating the Pitfalls

Challenges and Best Practices in LLM Evaluation

Evaluating LLMs effectively is fraught with challenges. Key considerations include:

Subjectivity and Human Judgment

One significant hurdle is the inherent subjectivity in assessing the quality of certain LLM outputs, particularly in creative and open-ended tasks. Human judgment, while often considered the gold standard, can vary between evaluators, making it difficult to establish a universally agreed-upon "good" output.

Dataset Selection and Bias

The choice of evaluation datasets is critical. Metrics can provide misleading results if tested on datasets that are not diverse or representative of the real-world scenarios in which the LLM will be deployed. It is essential to use datasets that cover a wide range of inputs and expected outputs to ensure a robust evaluation.

Metric Selection and Combination

Relying on a single metric can provide an incomplete picture of an LLM's performance. Different metrics capture different aspects of quality, and the most appropriate metrics depend on the specific goals and requirements of the application. A comprehensive evaluation strategy often involves combining multiple complementary metrics.

Human-AI Collaborative Evaluation

Given the limitations of automated metrics, especially for nuanced and subjective aspects of language, human evaluation remains a vital component of a comprehensive evaluation strategy. Human evaluators can assess qualities like creativity, coherence, engagingness, and overall real-world usefulness, which automated metrics often struggle to capture. To ensure consistency and reduce bias in human evaluations, it is crucial to establish clear evaluation protocols and guidelines for the evaluators.

The Future of LLM Evaluation: Emerging Trends and Directions

The field of LLM evaluation is continuously evolving, driven by the rapid advancements in LLM capabilities. Future trends point towards:

Context-Aware and Human-Aligned Metrics

Development of more context-aware and human-aligned metrics that can go beyond simple surface-level comparisons. There is a growing focus on developing metrics that can effectively evaluate coherence, consistency, relevance, and factual accuracy.

LLMs as Evaluators

One significant trend is the increasing use of LLMs themselves as evaluators. These LLM-based evaluation methods leverage the language understanding capabilities of large models to assess the quality of other LLM outputs. Examples include G-Eval and CPDScore, which use large language models to provide human-like evaluations of generated text.

Reference-Free Evaluation

The rise of reference-free evaluation methods, which do not rely on human-annotated reference texts, is addressing the limitations of traditional reference-based metrics, especially for open-ended generation tasks.

Standardized Benchmarks

The development of more robust benchmarks and standardized evaluation frameworks is crucial for facilitating fair comparisons across different LLMs. Standardized benchmarks allow researchers and practitioners to objectively assess the progress of different models on common tasks.

Evaluating Broader Implications

There is a growing recognition of the importance of evaluating broader aspects of LLMs beyond just accuracy and fluency, such as fairness, bias, safety, and ethical implications. As LLMs become more powerful and integrated into society, ensuring that they are developed and deployed responsibly requires careful evaluation of these critical dimensions.

Conclusion

Towards Robust and Meaningful LLM Assessment

Evaluating large language models (LLMs) is complex due to their diverse applications, such as text classification, summarization, creative generation, question answering, dialogue systems, code generation, and translation. Each task requires specific evaluation methods and careful selection of metrics, often combining automated tools with human judgment.

Relying on just one metric isn't enough—multiple measures are essential for fully understanding an LLM's strengths and limitations. Evaluation should also use diverse, representative datasets to capture real-world scenarios accurately.

Looking forward, LLM evaluation will become more sophisticated, with approaches like using LLMs themselves as evaluators and developing metrics that don't rely solely on reference outputs. Additionally, there's a growing focus on assessing ethical aspects, including fairness, safety, and societal impact.

Ultimately, the responsible development of LLMs will depend on a balanced mix of traditional metrics, advanced neural-based assessments, and human insights, ensuring these powerful tools benefit society effectively and safely.

References

[Google Colab](#)

[Edit descriptioncolab.research.google.com](https://colab.research.google.com)

[Evaluation metrics](#)

[*Evaluating the performance of machine learning models is crucial for determining their effectiveness and reliability...learn.microsoft.com*](#)

[Performance metrics for evaluating generated code](#)

[*I'm just learning about techniques such as ROUGE and BLEU for evaluating trained models. I'm interested in training a...community.deeplearning.ai*](#)

[Custom text classification evaluation metrics - Azure AI services](#)

[*Learn about evaluation metrics in custom text classification.learn.microsoft.com*](#)

[Evaluating NLP Models for Text Classification and Summarization Tasks in the Financial Landscape...](#)

[*Unlock the power of advanced NLP models for precise text classification and summarization in the financial landscape...www.indium.tech*](#)

[Evaluate the text summarization capabilities of LLMs for enhanced decision-making on AWS | Amazon...](#)

[*Organizations across industries are using automatic text summarization to more efficiently handle vast amounts of...aws.amazon.com*](#)

Active and Semi-Supervised Learning: The Key to Efficient Machine Learning

Machine learning has revolutionized how we approach data analysis, decision-making, and automation. However, a significant obstacle remains: labeling data. Labeling is notoriously costly, time-consuming, and often impractical, especially in real-world scenarios. Fortunately, modern techniques like active learning and semi-supervised learning have emerged, dramatically reducing these challenges and unlocking a new era of efficiency in machine learning workflows.

The Data Labeling Bottleneck

Data labeling, essential for supervised learning, frequently poses significant logistical challenges. Expert annotations can be prohibitively expensive and time-intensive, particularly when dealing with massive datasets. Consistency in labeling is also difficult to maintain, leading to variable model performance. This bottleneck hampers innovation, particularly for startups and small organizations that lack extensive resources.

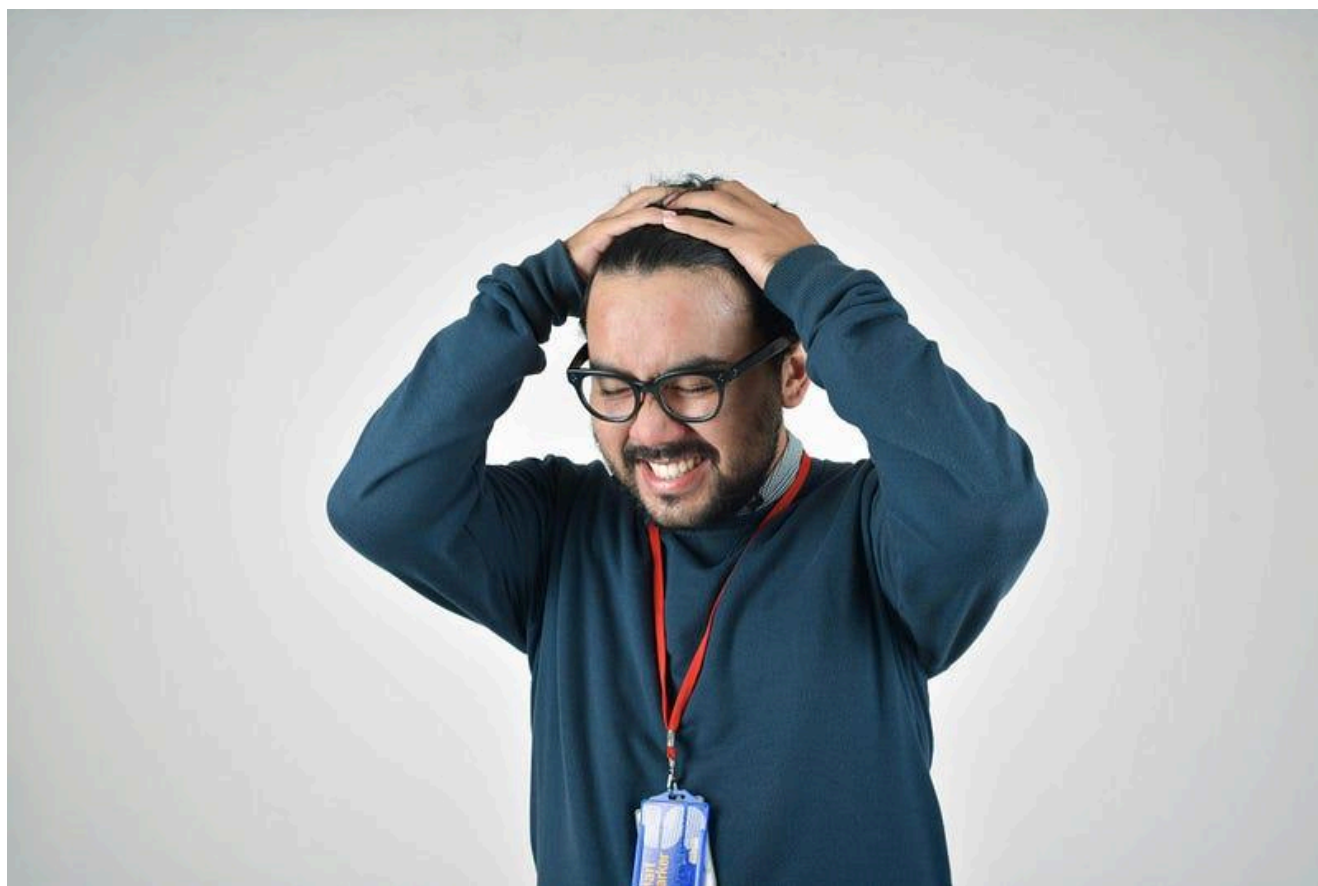


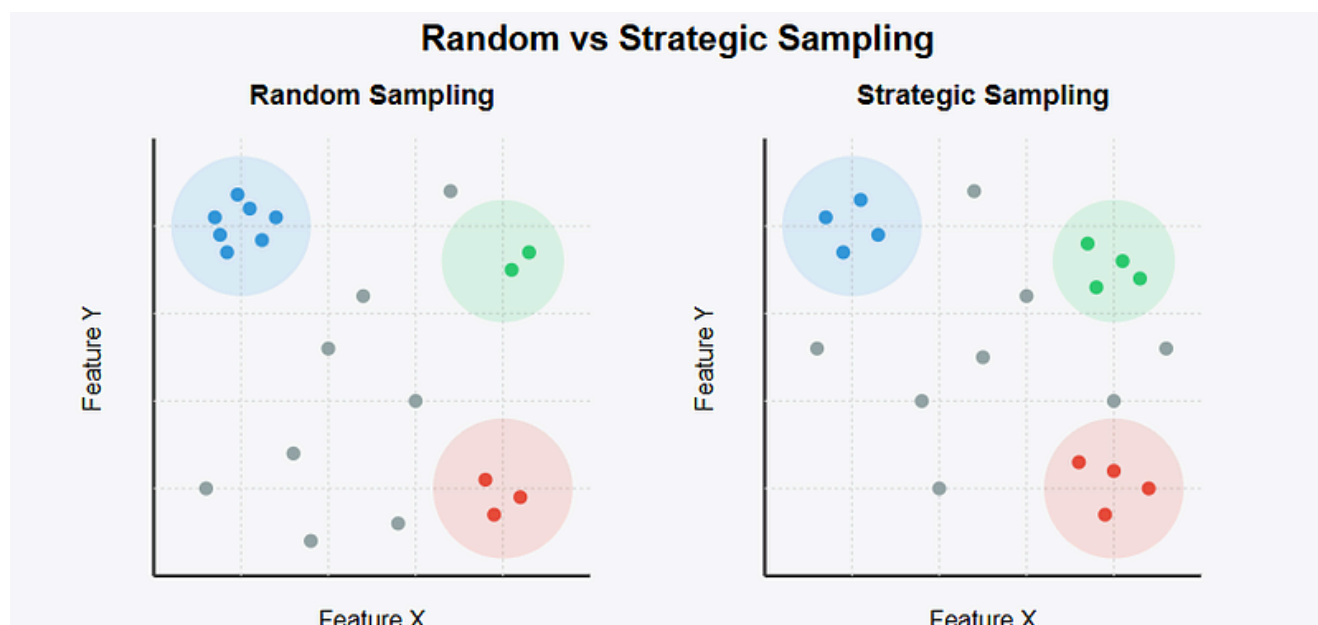
Photo by [ahmad gunnaivi](#) on [Unsplash](#)

The Hidden Truth: You Don't Need All Your Data

Surprisingly, not every data point in a dataset equally contributes to model accuracy. Many data points are redundant or provide minimal value. Intelligently selecting a subset of your data can yield substantial results without exhaustive labeling efforts. Research and practical experience indicate that even a fraction—around 30%—of strategically selected data can build an effective and efficient model.

The Problem with Random Sampling

While reducing the amount of labeled data sounds appealing, simply picking random samples won't guarantee good results. Random sampling can overlook important data points, resulting in inconsistent and suboptimal model performance. Imagine training a model to detect rare medical conditions—random sampling might miss critical, informative cases, resulting in a poorly performing model.

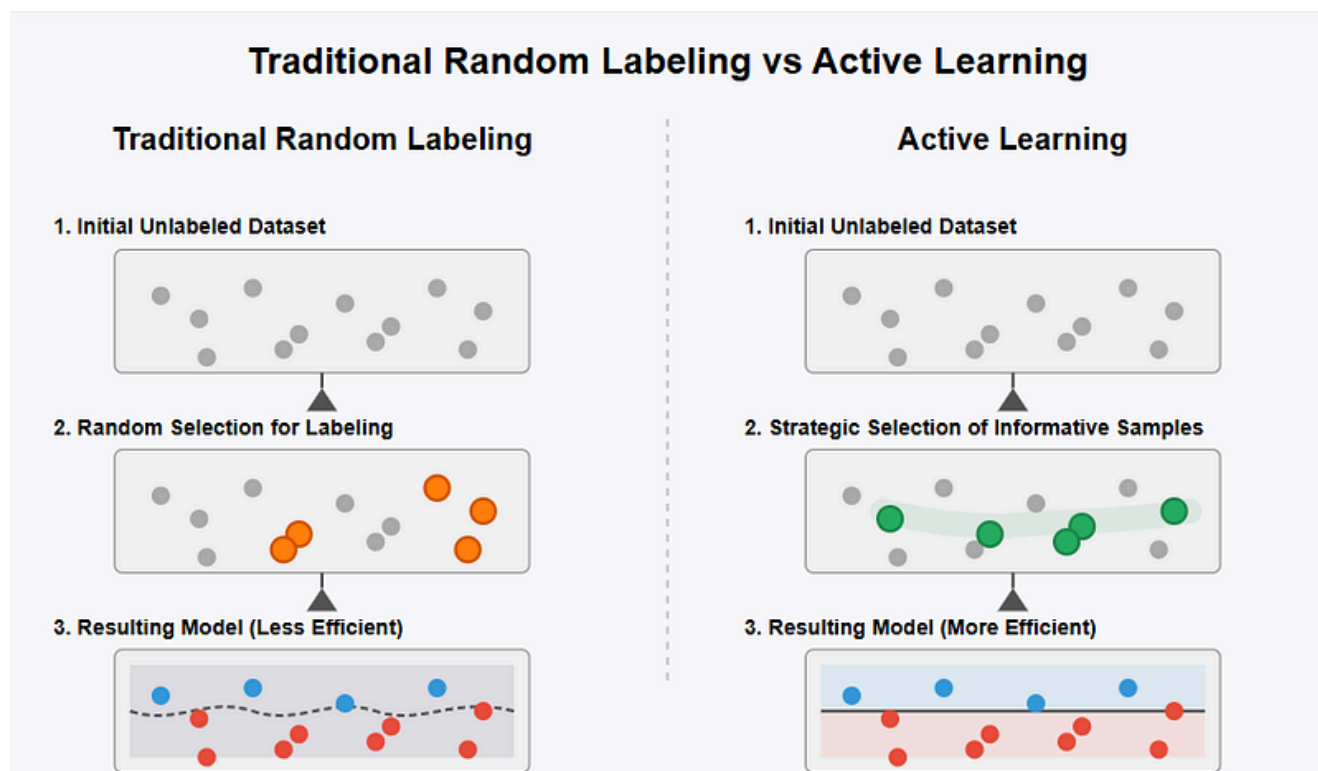


Random sampling often misses key data points, leading to inconsistent model performance.

Introducing Active Learning: A Strategic Revolution

Active learning addresses these issues head-on. Instead of random selection, active learning identifies and labels the most informative data points, drastically

improving performance. The idea is simple yet powerful: label data points the model finds most challenging. This strategic approach significantly enhances the efficiency and accuracy of the machine learning process.



Active learning selects the most informative samples, dramatically improving efficiency.

Inside Active Learning: How the Magic Happens

Active learning follows a clear iterative process:

1. **Kickoff:** Start by labeling a tiny portion (around 2–3%) of the dataset, creating an initial model.
2. **Inference:** Use this preliminary model to predict labels across the unlabeled dataset, assessing confidence levels.
3. **Sampling:** Select and label the most uncertain predictions, i.e., those data points that the model is least confident about.
4. **Retrain:** Update and improve the model using this newly labeled information.

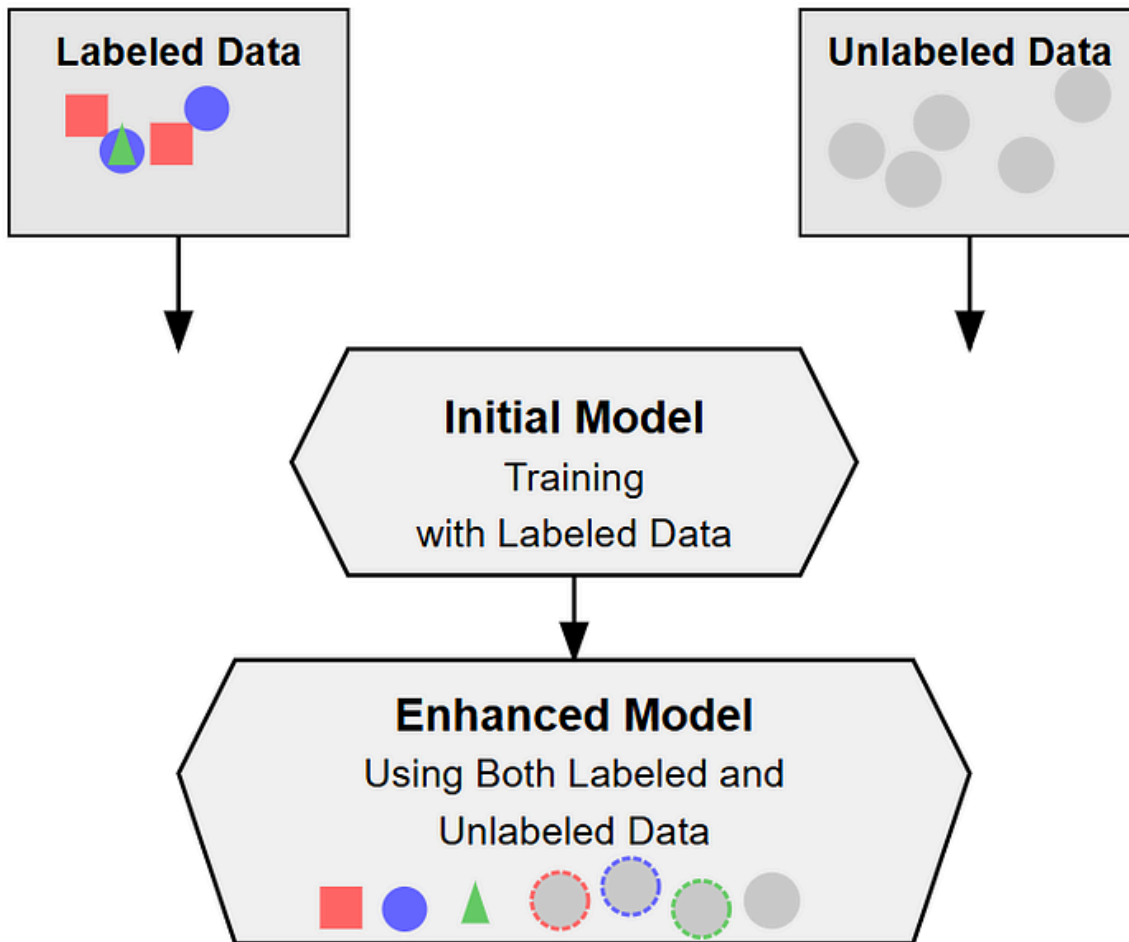
Repeating this cycle quickly improves model performance, focusing labeling efforts precisely where they're needed most.

Going Beyond: Semi-Supervised Learning Explained

Semi-supervised learning further complements active learning by leveraging unlabeled data to enhance performance. Unlike active learning, which strategically chooses which data points to label, semi-supervised learning exploits patterns from the entire dataset—both labeled and unlabeled—to improve model accuracy. It begins by training an initial model on labeled data, then iteratively uses predictions on unlabeled data to refine and enhance its understanding.

This method is particularly advantageous when extensive labeled data is unavailable or impractical to obtain. Imagine developing a speech recognition system—there's abundant unlabeled audio data but very few transcriptions. Semi-supervised learning leverages this large volume of unlabeled audio to enhance model accuracy substantially.

Semi-supervised Learning



Semi-supervised learning integrates labeled and unlabeled data, maximizing model efficiency.

Real-World Success Story: Meet NELL

One remarkable example of these strategies in action is Carnegie Mellon's NELL (Never-Ending Language Learner). NELL continuously improves its ability to read and understand the web, using active and semi-supervised learning techniques. It couples multiple functions—such as classifying noun phrases and extracting relationships—to iteratively build and refine its extensive knowledge base. By constantly incorporating new data, NELL demonstrates how effectively combining active and semi-supervised learning can yield significant real-world results.

Advanced Techniques: Fine-Tuning Your Approach

Active learning encompasses several sophisticated strategies beyond basic uncertainty sampling:

- **Query by Committee:** Employs multiple models to select data points that generate the most disagreement, improving decision boundaries.
- **Expected Model Change:** Focuses labeling efforts on samples likely to cause significant shifts in model parameters.
- **Density-Weighted Sampling:** Balances selecting uncertain examples and those typical of the dataset, avoiding outlier-driven bias.

These advanced techniques further refine active learning, ensuring maximum efficiency and improved accuracy.

Key Takeaways: Why You Should Care

The value of active and semi-supervised learning methods cannot be overstated. These methods:

- **Significantly reduce the time and cost associated with data labeling.**
- **Improve model accuracy by intelligently selecting informative data.**
- **Allow scalable application, from simple tasks to complex, large-scale machine learning problems.**

Conclusion: Embracing the Future of Machine Learning

Active and semi-supervised learning effectively address one of the most pressing challenges in machine learning today: labeled data scarcity. By intelligently integrating these methods into your workflow, you can build robust, high-performing models efficiently. The potential savings in time, cost, and resources are substantial, positioning these techniques as foundational tools for anyone serious about harnessing machine learning's full potential.

Embrace this future, and let active and semi-supervised learning unlock your next wave of innovation.

References:

https://www.youtube.com/watch?v=8Jwp4_WbRio&list=PLQNcEsjDCNmw-RbWwNVkqfsdV3BgClaAX
<https://www.youtube.com/watch?v=OMRlnKupsXM&t=33s>
<https://burrsettles.com/pub/settles.activelearning.pdf>

How to Create Professional AI Voiceovers for Free with Kokoro TTS (Better than ElevenLabs?)

Forget expensive subscriptions and complex setups. What if I told you that you could create professional-quality AI voiceovers right now, for absolutely free, using a tool that's not just good but potentially *better* than industry-leading (and costly) services like ElevenLabs? Sounds too good to be true? That's what the creators of Kokoro TTS are proving wrong, every single day. This isn't just another text-to-speech model; it's an 82-million parameter open-source *revolution* that's turning the AI voice world upside down. We're talking about a model trained on a shoestring budget, running on everyday computers, yet somehow outperforming giants on major benchmarks. Intrigued? You should be. This blog post will not only unveil the secrets behind Kokoro TTS but also guide you, step-by-step, on how to harness its power to create stunning voiceovers that will elevate your projects to new heights. Buckle up, because the future of AI voice is here, and it's free. This is more than just a tutorial; it's your gateway into a community that is redefining the landscape.



Kokoro TTS: Where Your Voice Meets the Future, for Free.

The Kokoro TTS Phenomenon: David vs. Goliath in the AI Arena

We all love an underdog story. And in the world of AI voice, Kokoro TTS is the ultimate underdog. Picture this: a tiny, 82-million parameter model, trained on a mere less than 100 hours of audio data, going head-to-head with behemoths like ElevenLabs, models trained on vast datasets and backed by significant funding. Sounds like a setup for failure, right? Wrong.

Here’s the shocker: Kokoro TTS didn’t just compete; it won. It snagged the #1 spot on the Hugging Face TTS Arena, a leaderboard where the best text-to-speech models are rigorously evaluated. How did it achieve this seemingly impossible feat?

The secret lies in its ingenious architecture and the efficiency of its design. It’s a highly optimized model that leverages a decoder-only architecture and was trained using approximately 500 GPU hours on A100 80GB GPUs. This efficiency translates to several mind-blowing advantages:

Blazing Fast Inference: Generate audio in real-time, even on a CPU. No more waiting around for cloud servers.

Runs Anywhere: From your laptop to a Raspberry Pi, Kokoro TTS can run on virtually any device.

Open-Source Freedom: Modify it, improve it, integrate it into your projects without limitations. The power is truly in your hands.

But the most astonishing part? It does all this while being completely free. This isn’t just about saving money; it’s about democratizing access to cutting-edge AI.

But the question remains: Can a free tool truly rival a premium service? Let’s put it to the test...

order	name	win_rate	elo	votes
#1	StyleTTS_Kokoro_v19	75.9	1293	8777
#2	Microsoft Edge TTS	65.4	1238	2479
#3	XITS_v2	61.6	1235	2889
#4	MetaVoice	57.4	1208	787
#5	Parler_Mini	52.8	1163	9655
#6	Fish_Speech	51.1	1148	5472
#7	Parler_Large	52.8	1136	553
#8	MaskGCI	47.8	1114	2983
#9	GPT-SoVITS_v2	46.8	1109	849
#10	MeloTTS	44.2	1108	7242
#11	Parler_Mini_Expresso	43.9	1107	1988
#12	llasa_3b	47.8	1105	494

<https://huggingface.co/spaces/Pendrokar/TTS-Spaces-Arena>

Unlocking the Magic: Your Step-by-Step Guide to Kokoro TTS

Enough with the suspense. Let’s get practical. Here’s how you can start creating with Kokoro TTS *right now*. We’ll use Google Colab for this tutorial, making it accessible to everyone, regardless of your hardware.

Step 1: The Colab Connection (Your Free AI Playground)

First, open up this Google Colab notebook: [here](#)

Step 2: Fire Up the Engine (One-Click Installation)

Run the first cell in the notebook. This cell installs all the necessary dependencies, including the **kokoro** library itself, plus a few other tools that make the magic happen. Think of it as prepping your digital voice studio.

#1 Install kokoro

```
!pip install -q kokoro soundfile
```

#2 Install espeak, used for out-of-dictionary fallback

```
!apt-get -qq -y install espeak-ng > /dev/null 2>&1
```

You can skip espeak installation, but OOD words will be skipped unless you provide a fallback

#3 Initialize a pipeline

```
from kokoro import KPipeline
```

```
from IPython.display import display, Audio
```

```
import soundfile as sf
```

```
# 🇺🇸 'a' => American English
```

```
# 🇬🇧 'b' => British English
```

```
pipeline = KPipeline(lang_code='a') # make sure lang_code matches voice
```

The following text is for demonstration purposes only, unseen during training

text = ""

The sky above the port was the color of television, tuned to a dead channel.

"It's not like I'm using," Case heard someone say, as he shouldered his way through the crowd around the door of the Chat. "It's like my body's developed this massive drug deficiency."

It was a Sprawl voice and a Sprawl joke. The Chatsubo was a bar for professional expatriates; you could drink there for a week and never hear two words in Japanese.

These were to have an enormous impact, not only because they were associated with Constantine, but also because, as in so many other areas, the decisions taken by Constantine (or in his name) were to have great significance for centuries to come. One of the main issues was the shape that Christian churches were to take, since there was not, apparently, a tradition of monumental church buildings when Constantine decided to help the Christian church build a series of truly spectacular structures. The main form that these churches took was that of the basilica, a multipurpose rectangular structure, based ultimately on the earlier Greek stoa, which could be found in most of the great cities of the empire. Christianity, unlike classical polytheism, needed a large interior space for the celebration of its religious services, and the basilica aptly filled that need. We naturally do not know the degree to which the emperor was involved in the design of new churches, but it is tempting to connect this with the secular basilica that Constantine completed in the Roman forum (the so-called Basilica of Maxentius) and the one he probably built in Trier, in connection with his residence in the city at a time when he was still caesar.

...

Step 3: Choose Your Voice (Meet the Cast)

Kokoro TTS comes with a collection of unique voices, each with its own personality. Run the second cell to load the default voice pack, which is a fascinating blend of two distinct voices: Bella and Sarah.

Secret Sauce: Want to experiment with other voices? You can find a list of available voice packs on the Kokoro TTS Hugging Face page (<https://huggingface.co/hexgrad/Kokoro-82M>). Just change the **VOICE_NAME** in the code. For example the voice packs currently include:

- **af**: Default voice, a 50-50 mix of Bella & Sarah
- **af_bella**, **af_sarah**, **am_adam**, **am_michael**,
- **bf_emma**, **bf_isabella**, **bm_george**, **bm_lewis**,
- **af_nicole**, **af_sky**

Step 4: Speak Your Mind (Let the AI Do the Talking)

Now for the fun part! In the third cell, you'll find a **text** variable. Replace the sample text with whatever you want Kokoro TTS to say. Get creative!

Step 5: Hear the Magic (Instant Audio)

Run the cell. Within seconds, you'll have an audio player right there in your Colab notebook. Hit play and listen to your text transformed into surprisingly natural-sounding speech.

Action Item: Try generating audio with different text inputs. Experiment with different voices, and listen carefully to the nuances.

But we're just getting started. What if you could create your own, unique AI voices?

```
from kokoro import KPipeline
```

```
from IPython.display import display, Audio
```

```
import soundfile as sf
```

```
import torch
```

1. Initialize the pipeline

```
pipeline = KPipeline(lang_code='a') # 'a' for American English
```

2. Define the available voices (from the model's config)

Though not directly used in mixing, it's good practice to keep this list

```
available_voices = [  
  
    'af', 'af_bella', 'af_sarah', 'am_adam', 'am_michael',  
  
    'bf_emma', 'bf_isabella', 'bm_george', 'bm_lewis',  
  
    'af_nicole', 'af_sky'  
  
]
```

3. Specify Device

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

4. Load the voice embeddings (download these from Hugging Face)

```
voice_01 = 'af_bella'
```

```
voice_02 = 'am_michael'
```

```
# voice_03 = 'bm_lewis'
```

5. Define the text

```
text = ''
```


Kokoro TTS stands at the intersection of art and technology, weaving humanlike warmth into synthetic voices.

It's the future of audio creation, boldly forging a path where open-source innovation conquers barriers,

granting every creator the freedom to share their story.

'''

6. Generate audio using individual voices (similar to the image)

print("Generating audio with individual voices...")

generator_01 = pipeline(text, voice=voice_01, speed=1, split_pattern=r'\n+')

for i, (gs, ps, audio) in enumerate(generator_01):

print(i) # i => index

print(gs) # gs => graphemes/text

print(ps) # ps => phonemes

display(Audio(data=audio, rate=24000, autoplay=i==0))

sf.write(f'{i}.wav', audio, 24000) # save each audio file

generator_02 = pipeline(text, voice=voice_02, speed=1, split_pattern=r'\n+')

for i, (gs, ps, audio) in enumerate(generator_02):

print(i) # i => index

print(gs) # gs => graphemes/text

```

print(ps) # ps => phonemes

display(Audio(data=audio, rate=24000, autoplay=i==0))

sf.write(f'{i}.wav', audio, 24000) # save each audio file


# generator_03 = pipeline(text, voice=voice_03, speed=1, split_pattern=r'\n+')

# for i, (gs, ps, audio) in enumerate(generator_03):

#     print(f"Voice 03 - Segment {i}:")

#     print(gs)

#     display(Audio(data=audio, rate=pipeline.sample_rate, autoplay=False))

#     sf.write(f'voice_03_segment_{i}.wav', audio, pipeline.sample_rate)

```

The Art of Voice Alchemy: Crafting Your Signature Sound

This is where Kokoro TTS truly shines. It's not just about choosing a voice; it's about *creating* one. Thanks to its ingenious voice mixing feature, you can blend different voice packs together, fine-tuning the ratios to craft a voice that's uniquely yours.

Imagine this:

- The warmth of Bella combined with the crispness of Sarah, creating a voice that's both friendly and authoritative.
- A touch of Michael's deep tones blended with Emma's gentle voice, perfect for a calming meditation app.

The possibilities are limited only by your imagination.

How does it work?

It's surprisingly simple. Each voice pack is represented by a set of numbers (an embedding). By adding these numbers together in different proportions, you're effectively mixing the characteristics of those voices.

7. Mix the voices (e.g., average of af_bella and am_michael)

```
print("\nGenerating audio with mixed voice (average)...")
```

```
mixed_voice_bf = voice_01_pt* 0.7 + voice_02_pt*0.3
```

8. Generate audio using the mixed voice

Instead of passing the tensor directly, create a new voice key and register it with the pipeline

Reshape the mixed voice to the expected shape (510, 1, 256)

```
pipeline.voices['mixed_voice'] = mixed_voice_bf.squeeze(0) # Register the mixed voice after squeezing
```

```
generator_mixed = pipeline(  
    text, voice='mixed_voice', # Use the registered key  
    speed=1, split_pattern=r'\n+'  
)
```

```
for i, (gs, ps, audio) in enumerate(generator_mixed):
```

```
    print(f"Mixed Voice - Segment {i}:")
```

```
    print(i) # i => index
```

```
    print(gs) # gs => graphemes/text
```

```
print(ps) # ps => phonemes
```

```
display(Audio(data=audio, rate=24000, autoplay=i==0))
```

```
sf.write(f'{i}.wav', audio, 24000) # save each audio file
```

```
print("\nAudio generation complete!")
```

Explanation:

- Load Voice Packs: We load two different voice packs using `torch.load()`.
- Mix Voices: We create a `mixed_voice` by multiplying each voice pack by a weight (representing the mix ratio) and adding them together. In this example, we're creating a 70/30 mix of `af_bella` and `am_michael`.
- Generate with Mixed Voice: We use the `generate()` function with the `mixed_voice` to synthesize audio using the blended voice.
- Display Audio Play the audio using the `display` and `Audio` functions from `IPython.display`

Action Item: Spend some time experimenting with different voice combinations and ratios. Create a voice that perfectly matches your project's needs. Save the settings for your favorite custom voices for later use.

Similar Example 2:

```
# 7. Mix the voices (e.g., average of af_bella and am_michael)
```

```
print("\nGenerating audio with mixed voice (average)...")
```

```
mixed_voice_bf = (voice_01_pt + voice_02_pt) / 2
```

```
# 8. Generate audio using the mixed voice
```

```
# Instead of passing the tensor directly, create a new voice key and register it with the pipeline
```

Reshape the mixed voice to the expected shape (510, 1, 256)

pipeline.voices['mixed_voice'] = mixed_voice_bf.squeeze(0) # Register the mixed voice after squeezing

```
generator_mixed = pipeline(
    text, voice='mixed_voice', # Use the registered key
    speed=1, split_pattern=r'\n+'
)
```

for i, (gs, ps, audio) in enumerate(generator_mixed):

print(f"Mixed Voice - Segment {i}:")

print(i) # i => index

print(gs) # gs => graphemes/text

print(ps) # ps => phonemes

display(Audio(data=audio, rate=24000, autoplay=i==0))

sf.write(f'{i}.wav', audio, 24000) # save each audio file

print("\nAudio generation complete!")

This is where you become the voice artist, sculpting the perfect AI voice for your project.

Beyond the Basics: Pro Tips and the Future

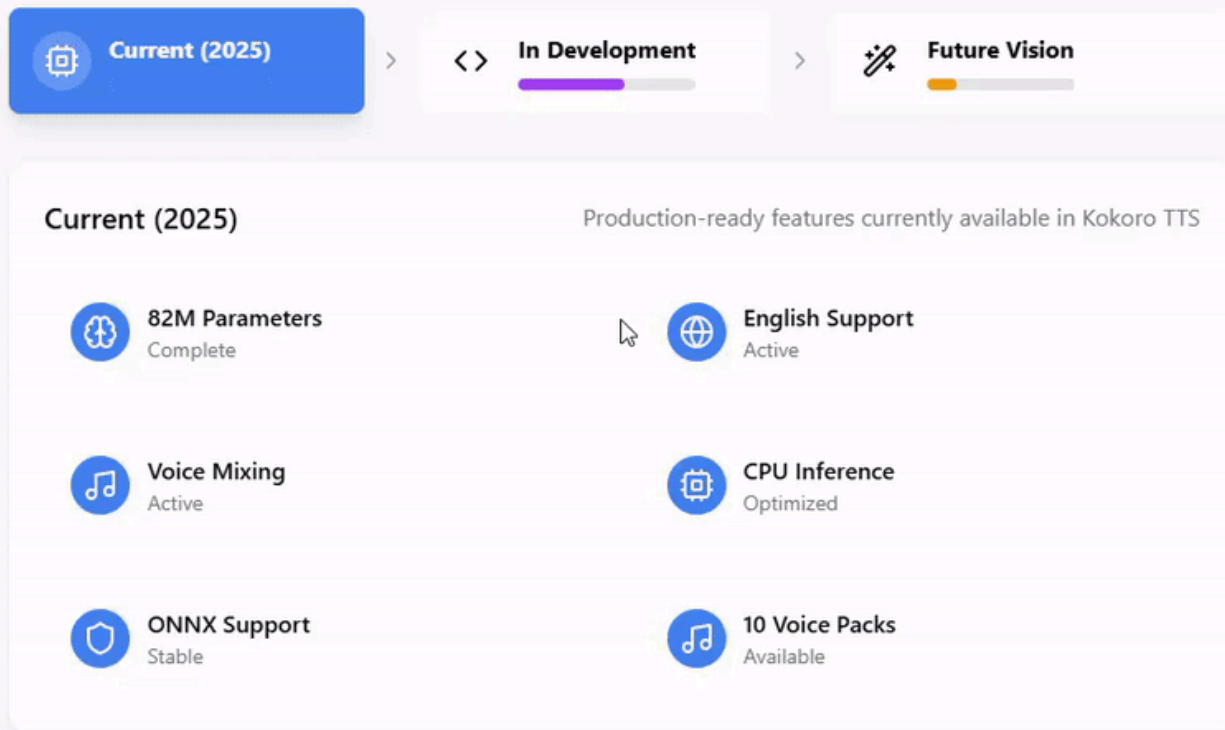
You've now mastered the fundamentals of Kokoro TTS. But to truly unlock its professional potential, here are some advanced tips and a glimpse into the future:

- **Mastering Pronunciation:** Kokoro TTS can handle most words with ease, but it does rely on **espeak-ng** for its grapheme-to-phoneme (g2p) conversion. This means that for words that are not in its dictionary, it will use its best guess, which may not always be correct.
- **Sentence Structure Matters:** Shorter sentences generally lead to better flow and intonation. Experiment with breaking down long sentences for optimal results.
- **The Power of Punctuation:** Commas, periods, and question marks are not just for grammar; they guide the AI's pauses and intonation. Use them strategically.
- **Community Power:** Join the Kokoro TTS community on GitHub and Hugging Face. Share your creations, learn from others, and contribute to the project's growth. This is where you'll find the latest tips, tricks, and voice packs.

What's next for Kokoro TTS? The developers are constantly working on improvements. Currently the model is being developed further, with a larger dataset, which will likely lead to higher quality output.

The Takeaway: Kokoro TTS isn't just a tool; it's a movement. It's about putting the power of high-quality AI voice into the hands of everyone. It is a revolution in the making.

Kokoro TTS Development Timeline



The Future of Kokoro TTS: More Voices, More Languages, More Possibilities.

Conclusion: Your Voice, Your Story, Amplified by AI

Kokoro TTS is more than just a free alternative to premium text-to-speech services. It's a testament to the power of open-source collaboration and a glimpse into the future of AI voice technology. We've explored its remarkable capabilities, learned how to use it effectively, and discovered the art of crafting unique voices through blending. Now it's your turn to join the revolution. Whether you're a seasoned developer, a creative content creator, or simply curious about the possibilities of AI, Kokoro TTS offers an unparalleled opportunity to experiment, innovate, and amplify your voice.

Don't just read about it—experience it. Head over to the Kokoro TTS Hugging Face page, try the Google Colab, and start creating. Share your projects, contribute to the community, and help shape the future of AI voice. The world is listening. What will you create with Kokoro TTS?

Action Items:

- Visit the Kokoro TTS Hugging Face page: <https://huggingface.co/hexgrad/Kokoro-82M>
- Try the Google Colab demo: [here](#)
- Join the Kokoro TTS community on Discord: [Here](#)
- The Official Kokoro github: <https://github.com/hexgrad/kokoro>
- The kokoro-onnx library: <https://github.com/thewh1teagle/kokoro-onnx>
- Share your Kokoro TTS creations with the hashtag #KokoroTTS

Let the AI voice revolution begin!

The AI voice revolution is here, and Kokoro TTS is your ticket to mastering it. I share cutting-edge tools, practical tutorials, and industry insights like this every week. If you're ready to create, innovate, and stay ahead of the curve in AI tech, hit that follow button and let's build the future together!

Federated Learning Explained: Build Better AI Without Compromising Privacy

Imagine a world where medical breakthroughs happen at lightning speed, where your phone understands you perfectly, and where AI-powered services anticipate your needs with uncanny accuracy. This future is within reach, thanks to the power of Artificial Intelligence. But there's a catch: traditional AI relies on collecting massive amounts of data in a single, centralized location. And in many cases, that data is highly personal, sensitive, and legally protected.

A. The Problem with Centralized Data

Think about training an AI to detect cancer. To build an accurate model, you'd need a vast dataset of medical images, patient records, and treatment outcomes. But hospitals are bound by strict privacy laws like HIPAA in the United States and GDPR in Europe. They can't simply hand over their patients' data due to ethical and legal obligations.

Or consider your smartphone's keyboard. It learns from your typing patterns to suggest words and correct your typos, making your texting and emailing faster and more efficient. But do you really want every keystroke you make to be sent to a central server, stored, and analyzed? Probably not. That's a massive privacy concern.

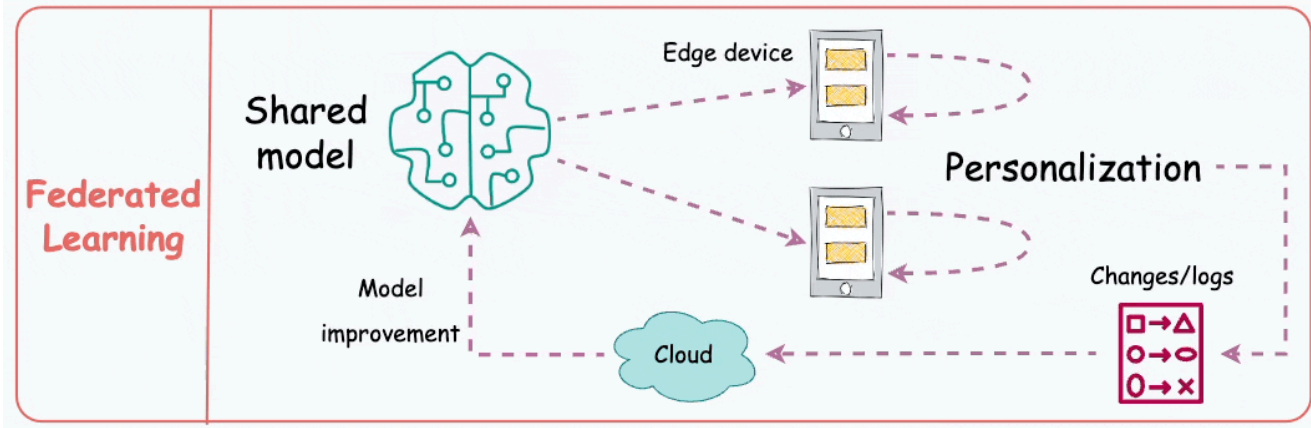
These are just two examples of the limitations of traditional, centralized machine learning. The process of collecting all the data in one place presents several daunting challenges:

- **Privacy Concerns:** Centralized data storage creates a honeypot for hackers. A single breach can expose the personal information of millions of individuals, leading to identity theft, fraud, and other serious consequences.
- **Data Security Risks:** Even with the best security measures, centralized databases are vulnerable to attacks. The more data is stored in one place, the greater the risk.
- **Legal Restrictions:** Regulations like GDPR and HIPAA place strict limits on how personal data can be collected, stored, and used. Complying with these regulations can be complex and expensive, especially when dealing with data from multiple jurisdictions.
- **Sheer Volume:** The amount of data generated by devices and individuals is exploding. Transferring and storing all this data in a central location is becoming increasingly impractical and costly. The bandwidth alone could become a limiting factor to even making use of the data that our devices produce.

As a result, many AI projects are stifled before they even begin. We're missing out on life-saving medical advancements, innovative new technologies, and personalized services that could improve our lives in countless ways. We are stuck working with publicly available data that, while useful, often does not contain the same richness that privately held data might. It is for this reason that we must rethink how we train AI.

Federated Learning

blog.DailyDoseofDS.com



<https://x.com/DailyDoseOfDS/status/1850120338753143012>

B. Introducing Federated Learning (FL):

Thankfully, there's a better way. It's called Federated Learning, and it's revolutionizing the field of AI. Federated Learning flips the script. Instead of bringing the data to the model, we bring the model to the data.

Here's the basic idea: Instead of collecting all the data in one place, training happens on individual devices (like smartphones) or within isolated environments (like hospitals or financial institutions). Each device or "silo" trains a local model using its own data. Then, instead of sharing the raw data, only the model updates—the changes learned during training—are sent to a central server.

Imagine a group of chefs learning a new recipe. In the traditional, centralized approach, each chef would have to send all their ingredients to a head chef, who would then try to create the perfect dish. But with Federated Learning, each chef perfects the recipe in their own kitchen, using their own ingredients. They then share only their improved recipe notes with the head chef, who combines them to create a master recipe that incorporates the best elements from each chef's experience.



In

Federated Learning, each ‘chef’ (device or silo) trains a model locally and shares only the learned updates, not the raw ‘ingredients’ (data).

C. Benefits of Federated Learning:

This decentralized approach offers a number of significant advantages:

- **Privacy Preservation:** Raw data never leaves the device or silo where it originated. This is a game-changer for privacy, as it eliminates the need to store sensitive information in a central location, drastically reducing the risk of data breaches.
- **Reduced Data Transfer:** Only model updates, which are typically much smaller than the raw data, are shared. This saves bandwidth, reduces latency, and lowers costs, making it feasible to train models on massive datasets that would be impossible to centralize.
- **Legal Compliance:** Federated Learning makes it easier to comply with data privacy regulations like GDPR and HIPAA, as data remains under the control of the data owner.
- **Access to More Diverse Data:** By enabling training on decentralized datasets, Federated Learning unlocks the potential of data that was previously inaccessible due to privacy concerns or logistical challenges. This leads to more robust, accurate, and generalizable AI models.

- **Improved Model Personalization:** In addition to contributing to a global model, local models can also be fine-tuned for individual users, providing a more personalized and relevant experience. Imagine a smart keyboard that adapts to your unique typing style or a medical diagnosis model tailored to your specific health profile.

D. Blog's Focus and Promise:

This blog will provide a beginner-friendly introduction to the exciting world of Federated Learning. We'll explore the core concepts, the challenges, and the incredible potential of this revolutionary approach to AI.

We'll also dive into some of the most popular frameworks and tools that are making Federated Learning a reality, including Flower, TensorFlow Federated (TFF), and PySyft. You'll see code snippets and examples that demonstrate how these frameworks work, using the flexible deep learning capabilities of PyTorch.

By the end of this blog, you'll have a solid understanding of the basics of Federated Learning and how frameworks like Flower, TFF, and PySyft, along with PyTorch, are empowering developers to build privacy-preserving AI solutions. You'll be ready to explore further and perhaps even start building your own federated learning projects. You'll also see how you might leverage the power of Amazon Web Services to help you construct your own Federated Learning application. Get ready to unlock the power of decentralized data and train AI models without compromising privacy!

II. Deep Dive into Federated Learning Concepts

Now that we've whetted your appetite with the promise of Federated Learning, let's dive deeper into the core concepts that make it work.

A. Key Terminology:

Understanding these terms will provide a solid foundation for grasping the intricacies of FL:

- **Clients:** In the world of Federated Learning, clients are the individual participants in the training process. They can be anything from smartphones and IoT devices to hospitals, banks, or other organizations that hold data. Each client possesses its own local dataset, which is used for training.

- **Server:** The server acts as the central coordinator in the Federated Learning process. It's responsible for initializing the global model, selecting clients for each training round, distributing the model to clients, and aggregating the model updates received from them.
- **Rounds:** A round refers to a single iteration of the Federated Learning process. It typically involves the server sending the global model to clients, clients training the model on their local data, and the server aggregating the updates to create a new global model.
- **Local Model:** Each client trains a local model on its own data. This model is a copy of the global model that is adapted and improved based on the client's specific dataset.
- **Global Model:** This is the central model maintained by the server. It represents the collective knowledge learned from all participating clients. The goal of Federated Learning is to improve the global model's performance through iterative updates from the clients.
- **Model Updates:** After training on local data, clients send model updates back to the server. These updates represent the changes made to the model's parameters (weights and biases) during training. Importantly, the raw data itself is never shared, only the learned adjustments to the model.

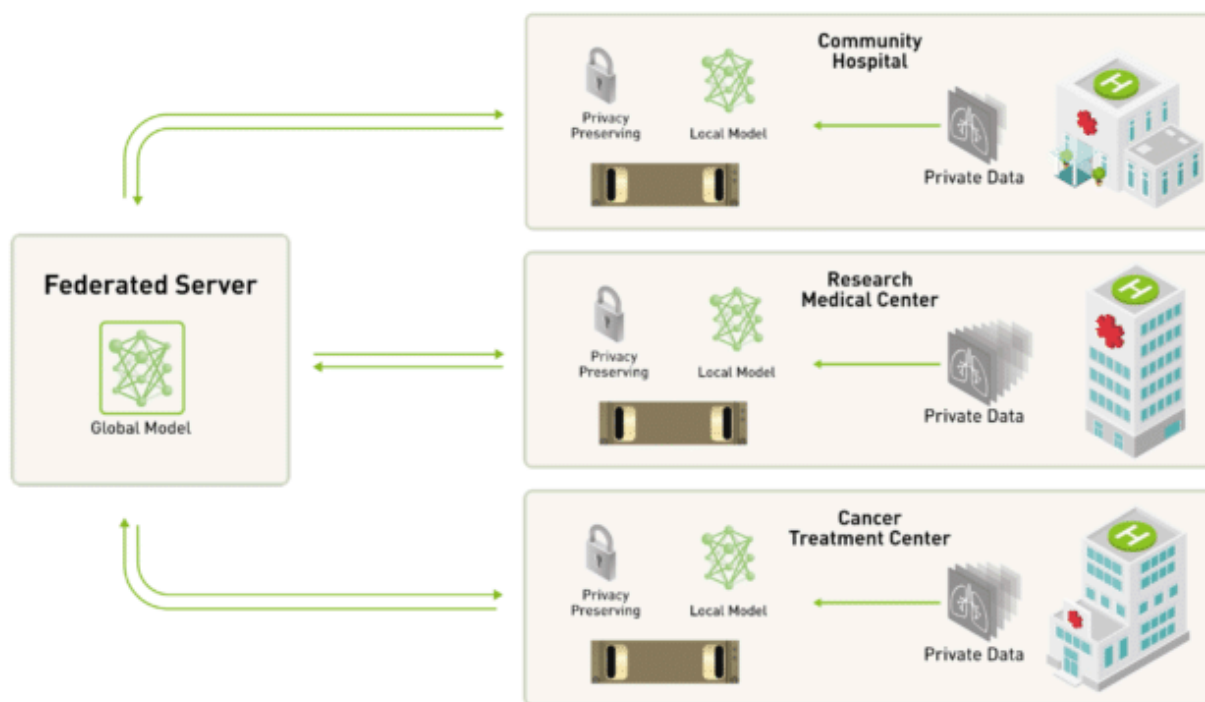
B. The Federated Learning Process (Step-by-Step):

Let's break down the Federated Learning process into a clear, step-by-step sequence:

1. **Initialization:** The process begins with the server initializing a global model. This model might be a randomly initialized neural network or a pre-trained model, depending on the specific application.
2. **Client Selection:** In each round, the server selects a subset of clients to participate in the training process. The selection criteria can vary, but the goal is often to choose a diverse set of clients that represent the overall population.
3. **Model Distribution:** The server sends the current global model to the selected clients. This is where bandwidth efficiency becomes crucial, as sending large models can be time-consuming and costly.
4. **Local Training:** Each client receives the global model and trains it on its own local data for a certain number of epochs or steps. This is where the "learning" part of Federated Learning happens. Clients use optimization

algorithms like Stochastic Gradient Descent (SGD) to adjust the model's parameters and improve its performance on their local data.

5. **Update Aggregation:** After local training, clients send their model updates back to the server. These updates typically consist of the changes made to the model's weights and biases.
6. **Model Update:** The server aggregates the updates received from the clients. A common aggregation method is Federated Averaging (FedAvg), which we'll discuss next. The server then uses the aggregated updates to improve the global model.
7. **Repeat:** Steps 2–6 are repeated for multiple rounds. With each round, the global model gradually improves as it learns from the collective knowledge of the participating clients.



<https://blogs.nvidia.com/blog/what-is-federated-learning/>

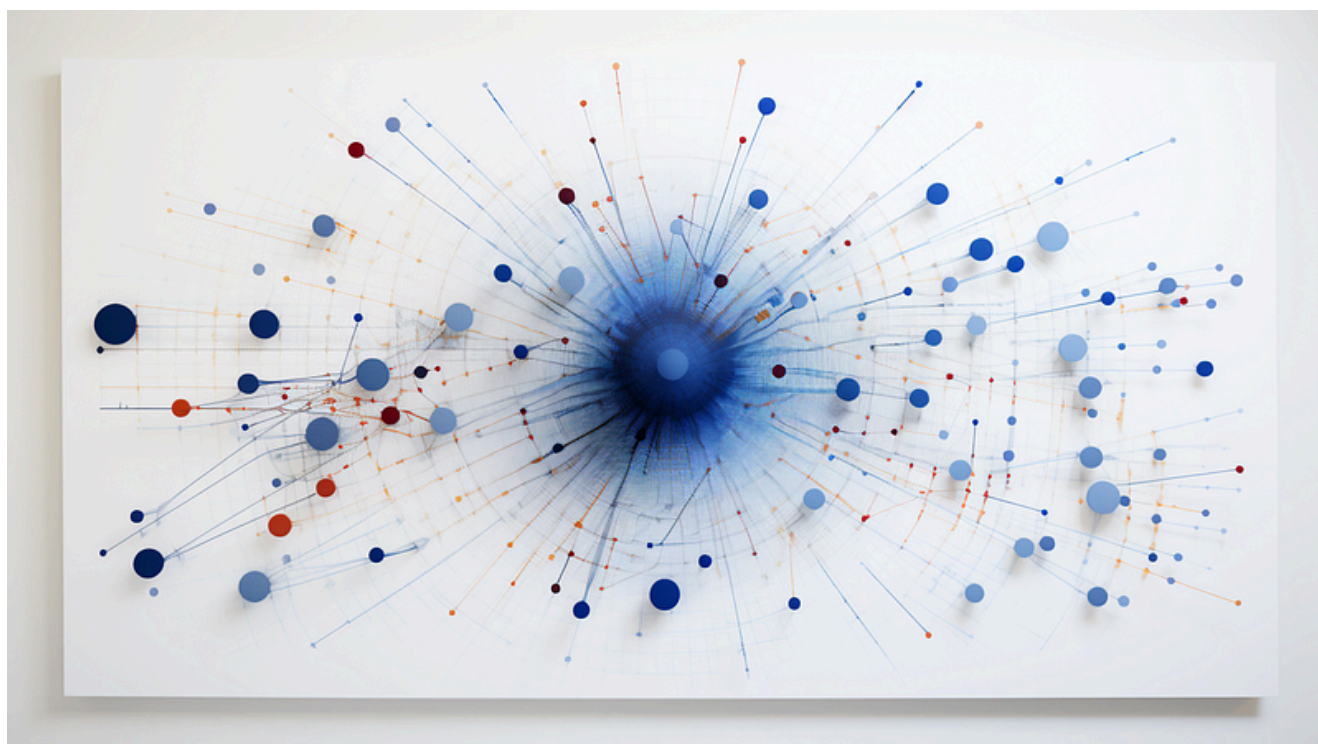
C. Federated Averaging (FedAvg):

Federated Averaging (FedAvg) is a fundamental algorithm for aggregating model updates in Federated Learning. It's like averaging the improved recipe notes from all the chefs to create the best master recipe.

In simple terms, FedAvg works by taking a weighted average of the model updates received from the clients. The weight for each client's update is typically

proportional to the number of data samples they used for training. This ensures that clients with more data have a larger influence on the global model.

Analogy: Imagine each chef sends their revised recipe notes to the head chef. The head chef doesn't just randomly pick one set of notes. Instead, they carefully consider each chef's suggestions, giving more weight to chefs who tested the recipe with more ingredients (more data). The head chef then combines these weighted suggestions to create the final, improved master recipe.



Federated Averaging (FedAvg) combines client model updates, weighted by their local dataset size, to improve the global model.

D. Challenges in Federated Learning:

While Federated Learning offers significant advantages, it also presents unique challenges:

- **Non-IID Data:** In real-world scenarios, data is often not independently and identically distributed (Non-IID) across clients. This means that the data distribution on each client can be significantly different from the overall population distribution. For example, different users might have different typing styles on their keyboards, or different hospitals might have different

patient demographics. This can make it harder for the global model to generalize well to all clients.

- **Communication Costs:** Sending model updates between the server and clients can be communication-intensive, especially for large models or when clients have limited bandwidth. This can slow down the training process and increase costs.
- **System Heterogeneity:** Clients in a Federated Learning system can have widely varying computational power, network connectivity, and availability. Some devices might be powerful smartphones, while others might be low-powered IoT devices. This heterogeneity needs to be considered when designing and deploying FL systems.
- **Privacy and Security:** While Federated Learning is designed to enhance privacy by keeping data local, model updates can still potentially leak information about the training data. Techniques like Differential Privacy and Secure Aggregation are used to further enhance privacy, but they can add complexity to the system.

In the next section, we'll explore how FL frameworks help address these challenges and make it easier to build and deploy Federated Learning systems.

III. Federated Learning Frameworks and Tools

The world of Federated Learning might seem complex at first, but thankfully, a vibrant ecosystem of frameworks and tools has emerged to simplify development and deployment. These frameworks handle the heavy lifting of communication, aggregation, and security, allowing you to focus on building great AI models. In this section, we'll explore some of the leading players: Flower, TensorFlow Federated (TFF), PyTorch, and PySyft.

A. Introduction to Frameworks:

Frameworks are essential for making Federated Learning practical. They provide the building blocks and abstractions needed to manage the distributed nature of FL, handle communication between clients and the server, and implement privacy-preserving mechanisms. Without them, building FL systems from scratch would be a monumental task. We will focus on a few of the most popular frameworks.

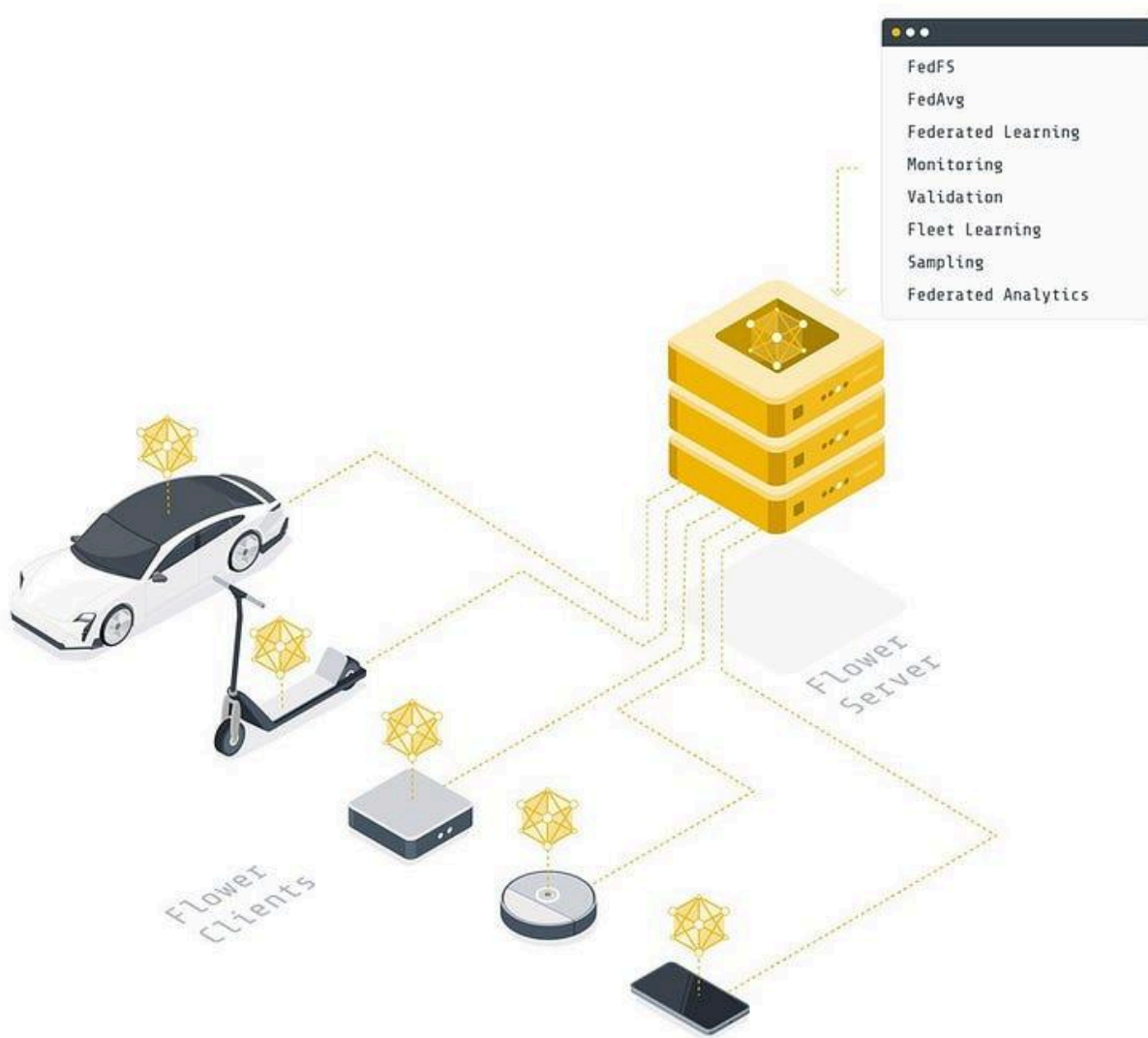
B. Flower: A Friendly Framework for Federated Learning

Overview:

[Flower](#) is a powerful yet user-friendly framework designed to make Federated Learning accessible to a wide range of developers. It's known for its framework-agnostic nature, meaning you can use it with popular deep learning frameworks like PyTorch, TensorFlow, or even other machine learning libraries. This flexibility, combined with its scalability and active community, makes Flower an excellent choice for both research and production deployments.

Key Features:

- **Framework Agnostic:** Works with various ML frameworks (TensorFlow, PyTorch, scikit-learn, etc.).
- **Scalability:** Supports large-scale deployments with potentially millions of clients.
- **Customizability:** Allows you to define custom aggregation logic, training loops, and more.
- **Easy to Use:** Provides simple abstractions and APIs for common FL tasks.
- **Active Community:** Backed by a vibrant community of developers and researchers.
- **Strong Baselines:** Includes implementations of various state-of-the-art FL algorithms.



Flower's framework-agnostic design allows it to seamlessly integrate with various machine learning libraries, making it a versatile choice for federated learning.

Code Snippet (Simple Flower Example):

Let's see how easy it is to get started with Flower using PyTorch for a simple image classification task.

Basic Flower client using PyTorch
import flwr as fl

import torch

import torch.nn as nn

```

import torch.nn.functional as F

from torchvision.transforms import Compose, ToTensor, Normalize

from torch.utils.data import DataLoader

from torchvision.datasets import CIFAR10

from tqdm import tqdm

import collections


#
#####

# 1. Regular PyTorch pipeline: nn.Module, train, test, and Dataloader

#
#####

# Define your PyTorch model as usual


class Net(nn.Module):

    def __init__(self) -> None:

        super(Net, self).__init__()

        self.conv1 = nn.Conv2d(3, 6, 5)

        self.pool = nn.MaxPool2d(2, 2)

        self.conv2 = nn.Conv2d(6, 16, 5)

        self.fc1 = nn.Linear(16 * 5 * 5, 120)

```

```
self.fc2 = nn.Linear(120, 84)
```

```
self.fc3 = nn.Linear(84, 10)
```

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
```

```
    x = self.pool(F.relu(self.conv1(x)))
```

```
    x = self.pool(F.relu(self.conv2(x)))
```

```
    x = x.view(-1, 16 * 5 * 5)
```

```
    x = F.relu(self.fc1(x))
```

```
    x = F.relu(self.fc2(x))
```

```
    x = self.fc3(x)
```

```
    return x
```

```
# Define your training loop
```

```
def train(net, trainloader, epochs):
```

```
    """Train the network on the training set."""
```

```
    criterion = torch.nn.CrossEntropyLoss()
```

```
    optimizer = torch.optim.Adam(net.parameters())
```

```
    net.train()
```

```
    for _ in range(epochs):
```

```
        for images, labels in tqdm(trainloader):
```

```
            optimizer.zero_grad()
```

```
            criterion(net(images), labels).backward()
```

```
            optimizer.step()
```

Define your testing loop

def test(net, testloader):

"""Validate the network on the entire test set."""

criterion = torch.nn.CrossEntropyLoss()

correct, loss = 0, 0.0

net.eval()

with torch.no_grad():

for images, labels in tqdm(testloader):

outputs = net(images)

loss += criterion(outputs, labels).item()

_, predicted = torch.max(outputs.data, 1)

correct += (predicted == labels).sum().item()

accuracy = correct / len(testloader.dataset)

return loss, accuracy

Load CIFAR-10 dataset

def load_data():

"""Load CIFAR-10 (training and test set)."""

transform = Compose([ToTensor(), Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

trainset = CIFAR10("./data", train=True, download=True, transform=transform)

testset = CIFAR10("./data", train=False, download=True, transform=transform)

trainloader = DataLoader(trainset, batch_size=32, shuffle=True)

```

testloader = DataLoader(testset, batch_size=32)

return trainloader, testloader

# Load your model and data

net = Net()

trainloader, testloader = load_data()


#
#####
#####

# 2. Federation of the pipeline with Flower

#
#####
#####

# Define Flower client

class FlowerClient(fl.client.NumPyClient):

    def get_parameters(self, config):

        return [val.cpu().numpy() for _, val in net.state_dict().items()]

    def set_parameters(self, parameters):

        params_dict = zip(net.state_dict().keys(), parameters)

        state_dict = collections.OrderedDict({k: torch.Tensor(v) for k, v in
params_dict})

        net.load_state_dict(state_dict, strict=True)

```

```

def fit(self, parameters, config):

    self.set_parameters(parameters)

    train(net, trainloader, epochs=1)

    return self.get_parameters(config={}), len(trainloader.dataset), {}

def evaluate(self, parameters, config):

    self.set_parameters(parameters)

    loss, accuracy = test(net, testloader)

    return loss, len(testloader.dataset), {"accuracy": accuracy}

```

Start Flower client

```

fl.client.start_numpy_client(

    server_address=":::8080",

    client=FlowerClient(),

)

```

Explanation:

- **get_parameters**: This method extracts the model's parameters as a list of NumPy arrays.
- **set_parameters**: This method takes a list of NumPy arrays (parameters) and updates the PyTorch model's weights.
- **fit**: This method receives parameters from the server, updates the local model, trains the model using the provided **train** function and PyTorch dataloader, and then returns the updated parameters, the number of training examples and an empty dictionary for metrics.
- **evaluate**: This method receives parameters from the server, updates the local model, evaluates the model using the provided **test** function and

PyTorch dataloader, and then returns the loss, the number of test examples, and a dictionary containing the accuracy.

Starting the Flower Server:

On the server side, you would start a Flower server using the following command:

```
flower-server start-server
```

Running a Flower Simulation:

Flower allows you to simulate an FL system on your local machine. You can start multiple clients in separate processes and connect them to the server. Each client will train on a different subset of the data, simulating a distributed environment. You can run multiple clients by simply running the `client.py` file multiple times, in multiple terminals:

```
python client.py
```

C. TensorFlow Federated (TFF): A Deeper Dive

Overview:

TensorFlow Federated (TFF) is a powerful framework for conducting federated learning research and, increasingly, deploying production-level FL systems. While it has a steeper learning curve than Flower, TFF offers greater control and flexibility for expressing complex federated computations. It's particularly well-suited for scenarios that require fine-grained control over the FL process or involve advanced privacy mechanisms.

TFF is organized in two main layers:

- **Federated Learning (FL) API:** This higher-level API, similar in spirit to `tf.keras`, provides abstractions for common FL tasks like training and evaluation. It's built on top of the Federated Core.
- **Federated Core (FC) API:** This lower-level API provides the foundational building blocks for expressing federated computations. It introduces concepts like federated types, federated placements, and federated operators, allowing you to define how data is distributed and how computations are performed across clients and the server.

Conceptual Example:

Let's imagine we want to compute the average temperature across a network of sensors using TFF. We can express this computation using the FC API, even without diving into the full TFF syntax.

Conceptual TFF example (not executable code)

Define federated types

```
float_at_clients = tff.type_at_clients(tf.float32)
```

```
float_at_server = tff.type_at_server(tf.float32)
```

Define a federated computation

```
@tff.federated_computation(float_at_clients)
```

```
def compute_average_temperature(temperatures):
```

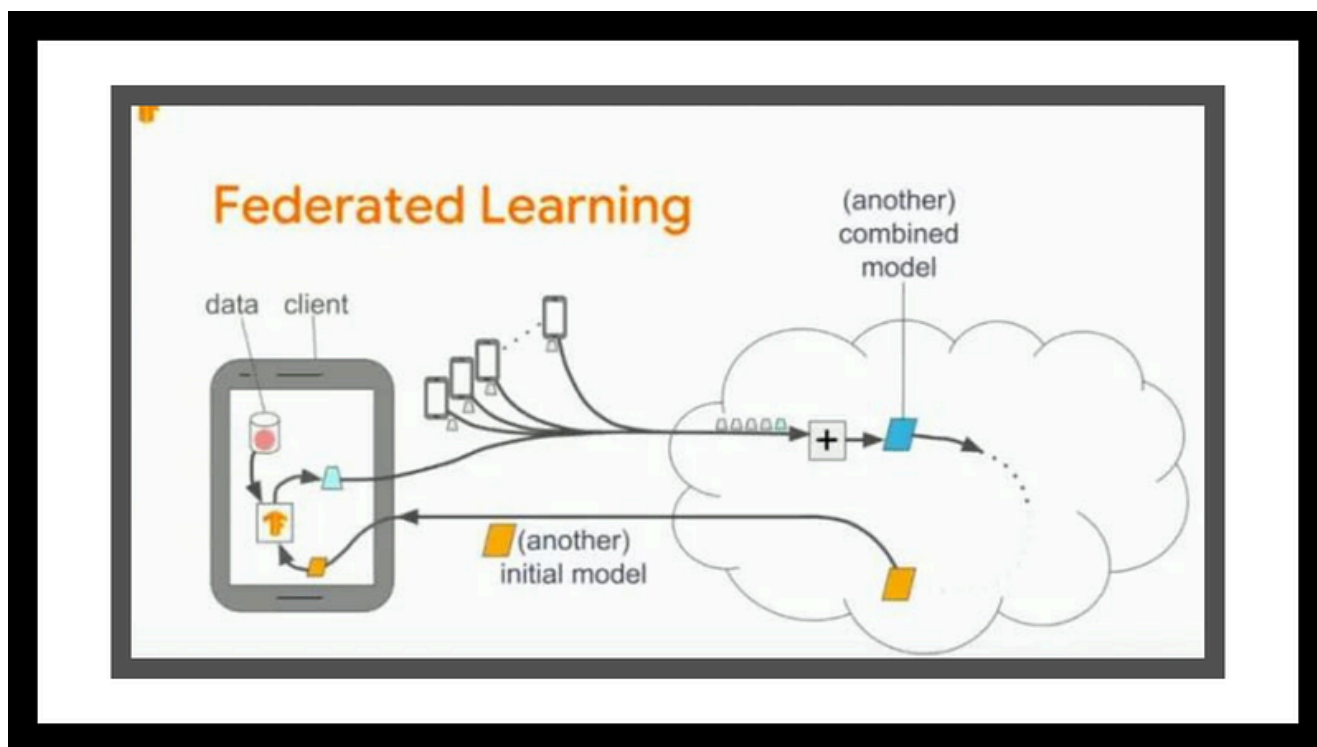
```
    # tff.federated_mean is a federated operator that computes the mean across
    # clients
```

```
    average_temp = tff.federated_mean(temperatures)
```

```
    return tff.federated_map(some_tf_function_to_execute_at_server, average_temp)
```

Explanation:

- **Federated Types:** We define `float_at_clients` to represent a `tf.float32` value that resides on each client and `float_at_server` to represent a single `tf.float32` value on the server.
- **Federated Computation:** The `tff.federated_computation` decorator indicates that this function defines a federated computation.
- **Federated Operators:** `tff.federated_mean` is a built-in operator that computes the mean of a federated value across clients.



TensorFlow Federated provides a powerful framework for expressing complex federated computations, with a focus on data placements and privacy.

This is just a glimpse into the capabilities of TFF. It's a powerful tool for researchers and developers who need to go beyond the basics of Federated Averaging and explore more advanced FL algorithms and privacy mechanisms.

IV. Real-World Use Cases in Finance and Healthcare

The theoretical foundations and technical tools are in place. But where is Federated Learning making a real-world impact? Let's explore some compelling use cases in two critical sectors: finance and healthcare. These industries are particularly well-suited for FL because they deal with highly sensitive data that is often distributed across multiple institutions.

A. Finance

The financial industry is embracing Federated Learning to enhance various services while adhering to strict privacy regulations and maintaining data security.

1. Anti-Money Laundering (AML):

Money laundering is a global problem that poses a significant threat to the

financial system. Banks are required to monitor transactions for suspicious activity and report them to authorities. However, traditional AML systems often suffer from high false-positive rates, leading to wasted resources and customer inconvenience.

Federated Learning offers a powerful solution. The banking circle, a next-generation payments bank, has successfully leveraged FL, built using the Flower framework, to improve their AML models. Their challenge was to train a model that could accurately detect money laundering activities in the US market, but they couldn't directly share transaction data from Europe due to regulatory restrictions.

Using Flower, they were able to train an AI model on European data without moving it across borders. This allowed them to develop a model tailored to the US market, even without direct access to US data. The initial results were impressive, demonstrating significant performance improvements:

- 65% increase in precision
- 55% increase in recall
- 10% increase in accuracy

This use case demonstrates how FL can enable financial institutions to collaborate on model training without compromising sensitive data, ultimately leading to more effective AML systems.

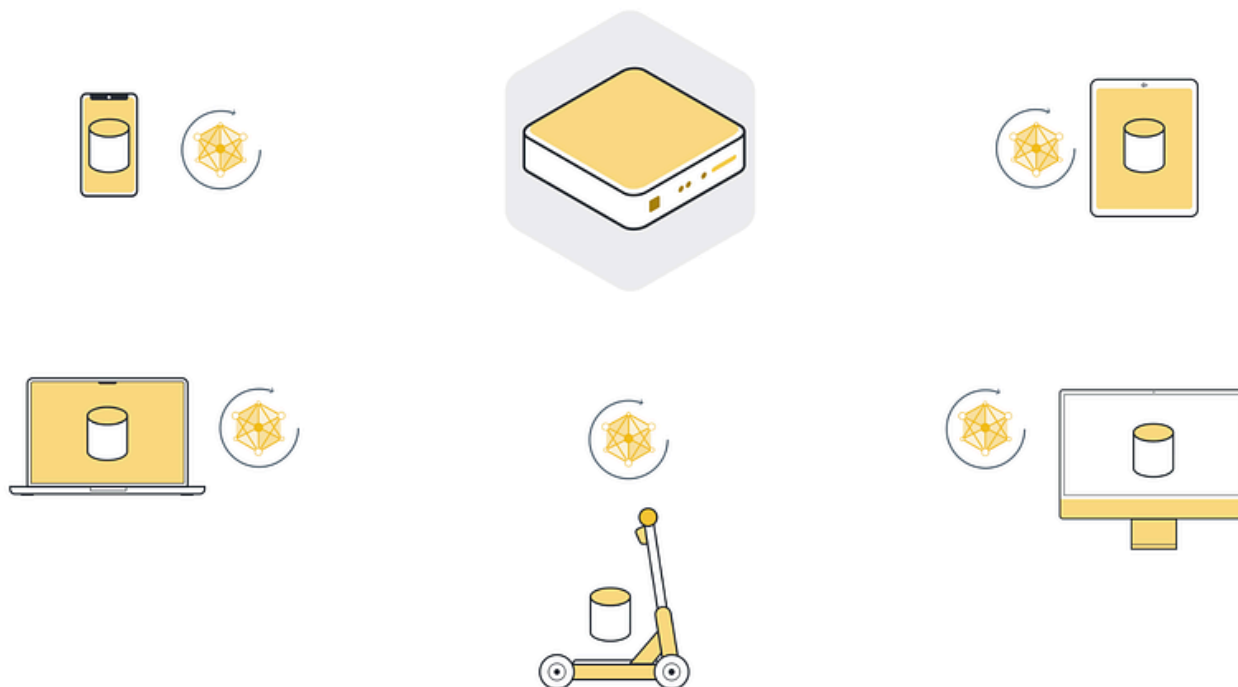
2. Financial Statement Auditing:

Auditing financial statements is a critical process for ensuring the accuracy and reliability of financial reporting. However, audit firms often face challenges in accessing and analyzing client data due to privacy and confidentiality concerns.

A [research paper](#) presented at the International Conference on AI in Finance (ICAIF) in 2022 showcased how Federated Learning, implemented using the Flower framework, can address this challenge. The researchers proposed using FL for anomaly detection in financial statements, allowing audit firms to collaboratively train models without directly sharing client data.

The FL system, built with Flower, enabled each audit client to train a local model on their own financial data. The model, in this case, was an autoencoder, a type of neural network commonly used for anomaly detection. The researchers then used

Flower to aggregate the weights of a specific part of the model which was shown to contain more general information, and this was shared between the participating clients via a central server.



Federated Learning, powered by Flower, allows audit firms to collaboratively train models for anomaly detection in financial statements while preserving client data privacy.

This research demonstrates the potential of FL to enhance the efficiency and effectiveness of financial statement auditing while maintaining the highest levels of data privacy.

3. Insurance Pricing and Risk Assessment:

The insurance industry relies heavily on accurate risk assessment models to determine premiums and manage financial stability. However, individual insurance companies often have limited data, especially for rare events or specific customer segments.

Federated Learning offers a way for insurance companies to pool their data resources without actually sharing the raw data. This allows them to train more robust and accurate pricing models that can better predict the frequency and severity of claims.

A [paper published on SSRN in 2024](#) explored the use of FL for training a claims frequency model, a type of model commonly used in insurance to predict the number of claims made by policyholders. The authors demonstrated that the FL model achieved similar performance to traditional methods that require data sharing, but with the added benefit of privacy preservation.

These examples highlight the transformative potential of Federated Learning in the financial sector. By enabling secure collaboration and data utilization, FL is paving the way for more accurate models, improved services, and enhanced privacy protection.

V. Deploying Federated Learning with AWS

Federated Learning is not just a theoretical concept; it's a practical solution that can be deployed in real-world settings. Amazon Web Services (AWS) provides a robust and scalable platform for building and deploying FL systems, especially for enterprise use cases. Let's explore why AWS is a compelling choice for your FL projects.

A. Why AWS for Federated Learning?

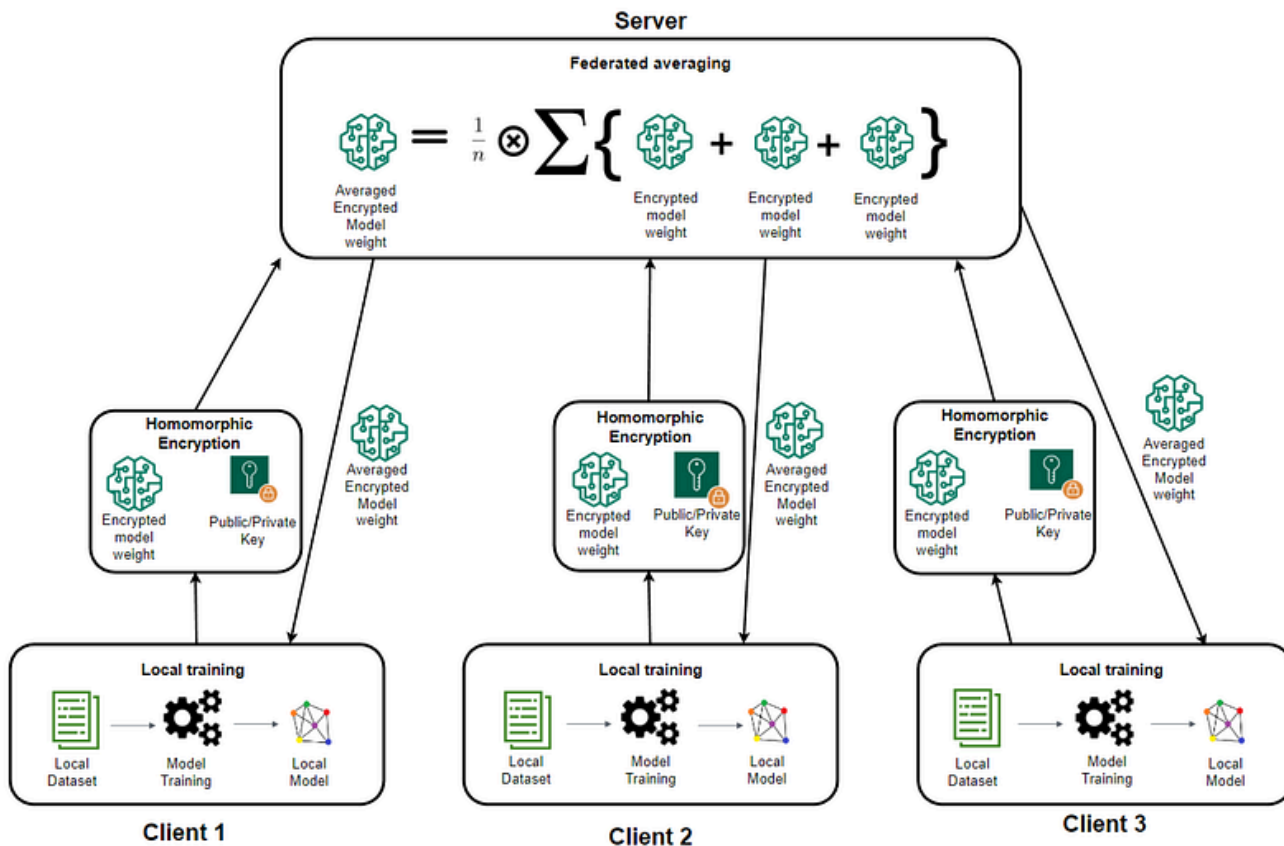
AWS offers a unique combination of features that make it ideal for Federated Learning deployments:

- **Scalability:** AWS's vast infrastructure allows you to scale your FL system to accommodate a large number of clients and massive datasets. You can easily adjust computing resources based on the demands of each training round.
- **Security:** AWS prioritizes security at every level. With features like granular access control, encryption, and robust network security, you can be confident that your FL system is well-protected.
- **Global Reach:** AWS has a global network of data centers, enabling you to deploy your FL system close to your clients, reducing latency and improving performance. This is especially beneficial for cross-border collaborations.
- **Managed Services:** AWS offers a suite of managed services, such as Amazon SageMaker, that simplify the development, deployment, and management of machine learning workloads, including Federated Learning.
- **Cross-Account Collaboration:** AWS supports secure communication and resource sharing between different AWS accounts. This is crucial for

cross-silo FL scenarios where different organizations need to collaborate without compromising data isolation.

B. Deployment Architecture:

Here's a simplified view of a typical FL deployment architecture on AWS:



Practical Diagram of Medical Privacy-Preserving Federated Learning Using Multi-Key Homomorphic Encryption and Flower Framework

<https://www.mdpi.com/2410-387X/7/4/48>

Multiple Client Accounts: Each with a private subnet containing a SageMaker instance running the Flower client and an S3 bucket for local data.

- **Server Account:** With a private subnet containing a SageMaker Notebook instance running the Flower server and an S3 bucket for storing the global model.
- **VPC Peering:** Connections between the server VPC and each client VPC.
- **Communication Flow:** Arrows indicating the flow of model updates between clients and the server.

D. Security and Compliance:

Security is paramount in Federated Learning, especially when dealing with sensitive data in finance and healthcare. AWS provides a comprehensive suite of security features and services to help you build secure FL systems:

- **IAM Roles and Policies:** Granular control over access to AWS resources.
- **Encryption:** Data can be encrypted at rest and in transit using AWS Key Management Service (KMS).
- **Network Security:** VPCs, security groups, and network ACLs provide robust network isolation and access control.
- **Compliance:** AWS complies with various industry standards and regulations, including HIPAA, GDPR, and PCI DSS.

By leveraging these features, you can build FL systems that meet the stringent security and compliance requirements of the finance and healthcare industries.

VII. Conclusion

A. Recap of Key Concepts:

We've covered a lot of ground in this blog post. Let's recap some of the key takeaways:

- **Federated Learning is a revolutionary approach to AI that enables training models on decentralized data without compromising privacy.**
- **It addresses the challenges of data privacy, security, legal compliance, and data volume that often hinder traditional, centralized machine learning.**
- **Federated Learning works by bringing the model to the data, rather than the data to the model.**
- **Clients train local models on their own data and share only model updates with a central server.**
- **The server aggregates the updates to improve a global model, which is then distributed back to the clients.**
- **Frameworks like Flower, TensorFlow Federated, PyTorch, and PySyft simplify the development and deployment of FL systems.**
- **Flower is a powerful, framework-agnostic, and beginner-friendly framework for building FL systems.**
- **AWS provides a scalable, secure, and globally accessible platform for deploying FL, with services like IAM, S3, VPC, and SageMaker.**

B. Future of Federated Learning:

Federated Learning is still a relatively young field, but it's rapidly evolving and gaining momentum. As privacy concerns continue to grow and the amount of data generated at the edge explodes, FL is poised to become an increasingly important part of the AI landscape.

Here are some exciting possibilities for the future of Federated Learning:

- **New Applications:** We'll see FL applied to a wider range of industries and use cases, beyond finance and healthcare, such as manufacturing, transportation, and energy.
- **Advanced Privacy Techniques:** Ongoing research in areas like Differential Privacy, Secure Multi-Party Computation, and Homomorphic Encryption will lead to even stronger privacy guarantees for FL systems.
- **Improved Efficiency:** New algorithms and optimization techniques will reduce communication costs, improve convergence speed, and make FL more efficient.
- **Personalized Models:** FL will enable the development of highly personalized AI models that adapt to individual user preferences and behaviors.
- **Cross-Device and Cross-Silo Collaboration:** We'll see more seamless collaboration between different types of devices (cross-device) and different organizations (cross-silo), unlocking the full potential of decentralized data.

C. Final Thoughts:

Federated Learning represents a paradigm shift in AI development. It empowers us to build intelligent systems that learn from the collective wisdom of decentralized data while respecting individual privacy and security.

The frameworks, tools, and cloud platforms discussed in this blog post are making it easier than ever to get started with Federated Learning. Whether you're a researcher, a developer, or a business leader, now is the time to explore the potential of FL and join the growing community that is shaping the future of AI.

Let's embrace the power of decentralized data and build a future where AI benefits everyone, without compromising our privacy. The journey is just beginning, and the possibilities are limitless.

VIII. Appendix

A. Glossary of Terms:

- **Federated Learning (FL):** A machine learning approach where a model is trained across multiple decentralized devices or servers holding local data samples, without exchanging their data samples.
- **Differential Privacy (DP):** A system for publicly sharing information about a dataset by describing the patterns of groups within the dataset while withholding information about individuals in the dataset.
- **Secure Multi-Party Computation (SMPC):** A cryptographic protocol that allows multiple parties to jointly compute a function over their inputs while keeping those inputs private.
- **Homomorphic Encryption (HE):** A form of encryption that allows computation on ciphertexts, generating an encrypted result which, when decrypted, matches the result of the operations as if they had been performed on the plaintext.
- **TensorFlow Federated (TFF):** An open-source framework for machine learning and other computations on decentralized data, built by Google.
- **PySyft:** An open-source framework for secure and private deep learning, extending PyTorch and TensorFlow with capabilities for federated learning, differential privacy, and encrypted computation.
- **Flower (flwr):** A friendly federated learning framework that is designed to be framework-agnostic, scalable, and customizable.

B. Further Reading:

- [Federated Learning: Collaborative Machine Learning without Centralized Training Data](#) (Google AI Blog)
- [Advances and Open Problems in Federated Learning](#) (Research Paper)
- [TensorFlow Federated: Machine Learning on Decentralized Data](#) (TensorFlow Website)
- [PySyft: Private Deep Learning](#) (GitHub Repository)
- [The Algorithmic Foundations of Differential Privacy](#) (Book)

Thank you for reading.

Hope this article helps you !

The Modern Data Stack: Understanding Data Pipelines, ETL, and Data Warehousing

In today's data-driven world, businesses are inundated with information from a plethora of sources. To harness the power of this data, organizations are turning to the "Modern Data Stack" a modular, scalable, and cloud-based approach to data management. But what exactly is the Modern Data Stack? At its core, it is a collection of integrated tools and technologies designed to streamline data workflows, enabling organizations to efficiently collect, process, store, and analyze vast amounts of data.

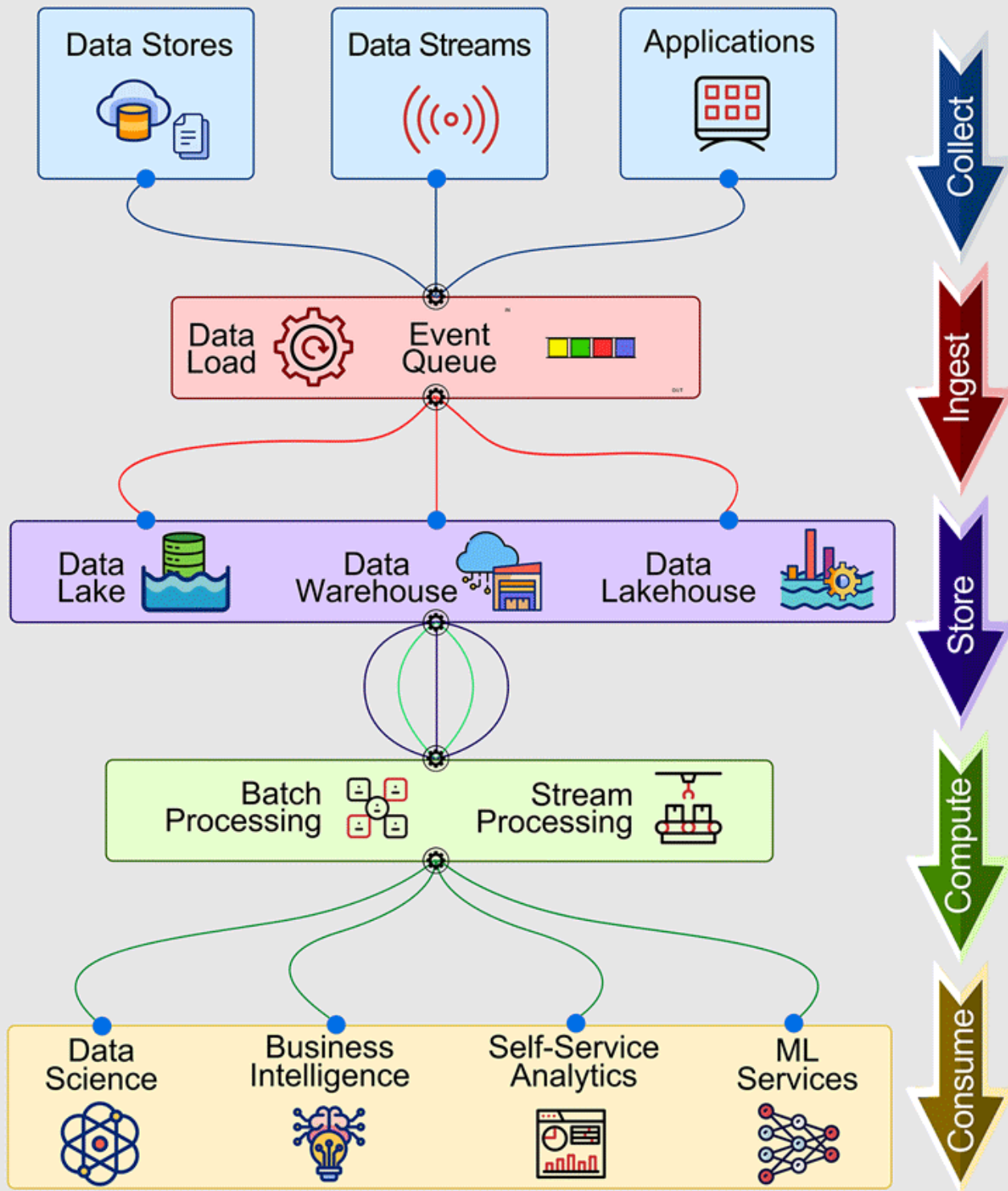
This blog will guide you through the foundational elements of the Modern Data Stack, exploring:

- **What makes the Modern Data Stack unique:** A brief overview of its modularity, scalability, and flexibility.
- **The role of data pipelines:** How robust workflows ensure seamless data integration and transformation.
- **ETL vs. ELT processes:** A comparison of traditional and modern data processing methodologies.
- **Building a data pipeline:** Step-by-step insights into sourcing, preprocessing, ingesting, and automating data workflows.
- **Data warehousing essentials:** Key architectural principles like dimensional modeling and schema design.
- **Governance and security:** Best practices for ensuring data integrity, compliance, and trust.
- **Tool selection and ecosystem:** From Flask and Airflow to Snowflake and Redshift, a look at popular tools for pipeline orchestration and warehousing.
- **Emerging trends:** The future of data pipelines, including DataOps, AI-driven automation, and real-time streaming.

By the end, you will gain a comprehensive understanding of how the Modern Data Stack empowers organizations to unlock the true potential of their data while keeping pace with evolving technological demands.

Data Pipeline Overview

ByteByteGo



<https://x.com/alexxubyte/status/1725192554671640735>

1. Introduction to the Modern Data Stack

The Modern Data Stack represents a paradigm shift in how we handle data. Unlike traditional, monolithic architectures, it leverages the flexibility and scalability of the cloud, along with a modular design that allows organizations to choose the best tools for each stage of the data lifecycle.

Key Components of the Modern Data Stack:

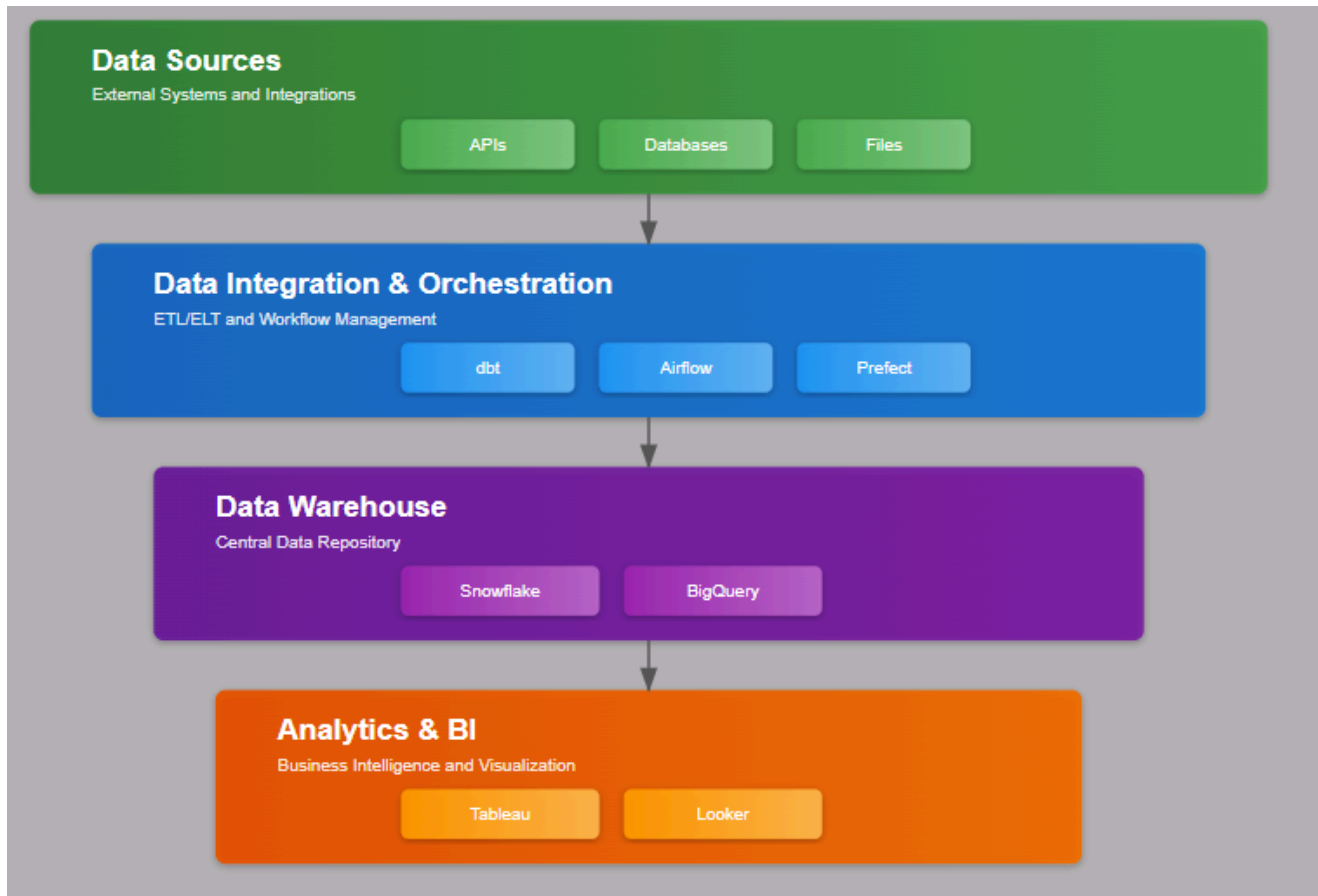
- **Cloud-Based:** Leverages cloud services for storage, compute, and scalability.
- **Modular:** Composed of independent, interchangeable tools for each stage (ingestion, transformation, storage, visualization).
- **Scalable:** Can handle increasing volumes of data without significant performance degradation.

At the heart of this stack lies the data warehouse, serving as the central repository for analytical data. However, to fuel this warehouse, we need robust data pipelines and efficient ETL/ELT processes. These pipelines act as the circulatory system, ensuring a continuous flow of data from various sources to the warehouse, where it's transformed and made ready for analysis.

The Critical Role of Data Pipelines and ETL/ELT:

- **Data Integration:** Pipelines bring together data from disparate sources.
- **Data Transformation:** ETL/ELT processes clean, standardize, and enrich the data.
- **Analytical Readiness:** Data is structured in the warehouse for efficient querying and reporting.

In today's fast-paced business environment, decisions are increasingly driven by real-time or near real-time insights. A well-designed data stack with robust pipelines is essential for achieving this agility.



Layered Diagram of the Modern Data Stack

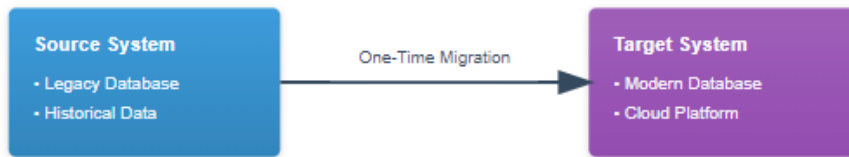
2. Data Migration vs. ETL: Key Differences

Before diving into pipeline construction, it's crucial to clarify the distinction between data migration and ETL (Extract, Transform, Load) or its modern variant, ELT (Extract, Load, Transform). These terms are often used interchangeably, but they represent distinct processes with different objectives.

Data Migration: Data migration is typically a one-time or infrequent event involving moving data from one storage system to another. This could be triggered by a database upgrade, a shift to a new platform, or a consolidation of data sources. The focus is on transferring data with minimal changes, ensuring a smooth transition to the new system.

Data Migration

One-time process with minimal transformation

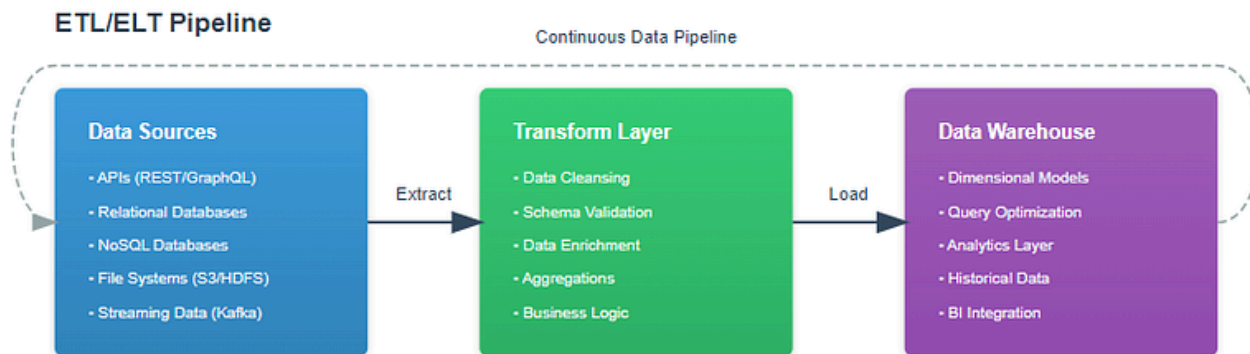


Flow Chart Illustrating Data Migration

ETL/ELT: ETL/ELT, on the other hand, are ongoing processes designed to feed data warehouses. They involve:

- **Extract:** Pulling data from multiple sources (databases, APIs, files, etc.).
- **Transform:** Cleaning, validating, aggregating, and enriching the data.
- **Load:** Writing the transformed data into the data warehouse.

ELT has gained popularity in the modern data stack due to the scalability of cloud data warehouses. It allows for loading raw data first and then performing transformations within the warehouse, leveraging its computational power.



Flow Chart Illustrating ETL

3. Building a Pipeline: From Source to Warehouse

Constructing a data pipeline is a multi-faceted process that requires careful planning and execution. Here's a high-level methodology:

a) Source Identification: The first step is to identify all the sources that will feed the data warehouse.

These can include:

- **Operational Databases:** MySQL, PostgreSQL, etc.
- **APIs:** Data from third-party applications (e.g., Salesforce, Google Analytics).
- **Files:** CSV, JSON, Parquet files, etc.
- **Streaming Sources:** IoT devices, web clickstreams, etc.
- **Legacy Systems** Mainframe applications
- **CRM and ERP:** Customer relations management systems, Enterprise resource planning systems.

b) Data Preprocessing: Preprocessing ensures that raw data is cleaned, structured, and enriched before further transformations.

Key preprocessing steps include:

- **Data Cleaning:** Handle missing values, correct errors, and remove duplicates to ensure consistency and accuracy.
- **Data Transformation:** Convert data into a standardized format, such as normalizing date formats or unifying units of measurement.
- **Data Reduction:** Employ aggregation, sampling, or filtering techniques to reduce data volume while retaining essential information.
- **Data Enrichment:** Add contextual details, such as geocoding addresses or appending demographic data, to enhance analytical depth.

c) Ingestion Mechanism: Next, you need to choose an ingestion mechanism based on the nature of the data and the desired latency:

- **Batch Ingestion:** Data is collected and processed in batches at scheduled intervals (e.g., daily, hourly).
- **Streaming Ingestion:** Data is ingested in real-time as it's generated.

d) Tools for Ingestion and Orchestration: The choice of tools depends on the complexity of the pipeline and the technical expertise of your team:

- **Custom Scripts:** For simpler tasks, you can use scripting languages like Python (with libraries like **requests** for API calls and **csv** for file handling) to build custom ingestion scripts.

- **Flask:** A lightweight Python web framework that can be used to build APIs for data ingestion.
- **Orchestration Tools:** For more complex pipelines, workflow orchestration tools like Airflow, Prefect, or Dagster provide robust scheduling, monitoring, and error-handling capabilities.

e) Transformation/Preprocessing: Once ingested, data often needs to be transformed to fit the structure of the data warehouse and meet analytical requirements.

This can involve:

- **Data Cleaning:** Handling missing values, correcting errors, removing duplicates.
- **Data Validation:** Ensuring data conforms to predefined rules and constraints.
- **Data Formatting:** Standardizing data types, units, and formats.
- **Data Enrichment:** Adding contextual information from other sources.

f) Loading to Warehouse: Finally, the transformed data is loaded into the data warehouse.

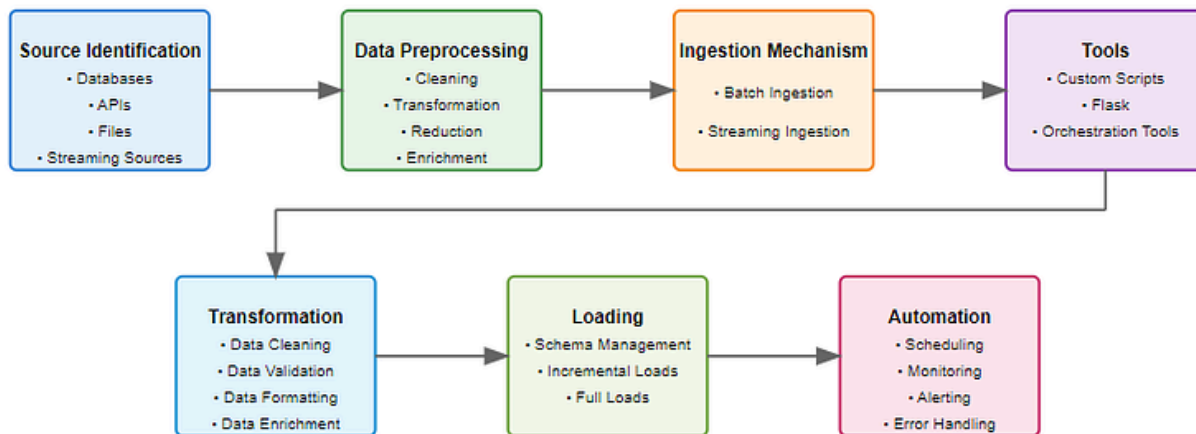
Key considerations include:

- **Schema Management:** Maintaining the integrity of the warehouse schema (data types, constraints).
- **Incremental Loads:** Loading only new or changed data to optimize performance.
- **Full Loads:** Periodically reloading the entire dataset to ensure consistency.

g) Automating the Data Pipeline: Automation ensures reliability, efficiency, and scalability.

Key aspects include:

- **Scheduling:** Use orchestration tools to define execution intervals, ensuring timely updates.
- **Monitoring:** Track errors, performance metrics, and data quality.
- **Alerting:** Notify stakeholders of issues for prompt resolution.
- **Error Handling:** Implement retry mechanisms and fallback procedures to address failures.



Detailed Pipeline Flow

4. Data Warehousing Fundamentals

A data warehouse is a specialized database designed to support complex analytical queries and reporting. Unlike operational databases optimized for transactional processing, data warehouses focus on large-scale analysis across historical data.

Key Characteristics of Data Warehouses:

- **Subject-Oriented:** Organized around specific business domains, such as customers, products, or sales.
- **Integrated:** Consolidates data from disparate sources, standardizing formats and definitions.
- **Time-Variant:** Tracks historical data, enabling trend analysis and forecasting.
- **Non-Volatile:** Preserves historical data integrity, ensuring no updates or deletions alter past records.

Dimensional Modeling:

Dimensional modeling simplifies data organization for analytical queries. Common approaches include:

- **Star Schema:** A straightforward design with a central fact table linked to dimension tables.

- **Snowflake Schema:** A normalized structure where dimension tables are further broken down into sub-dimensions.

5. Data Governance and Security

Effective data governance and security are essential for ensuring data integrity, compliance, and trust.

Data Governance:

- **Data Ownership:** Assign responsibility for data accuracy, quality, and accessibility.
- **Data Quality Rules:** Define standards for completeness, consistency, and validity.
- **Data Dictionary:** Maintain metadata describing datasets, columns, and relationships.
- **Data Lineage:** Track data's origin, transformations, and movements.

Data Security:

- **Role-Based Access Control (RBAC):** Restrict access to authorized users and systems.
- **Data Encryption:** Safeguard data in transit and at rest using encryption standards.
- **Audit Logging:** Monitor data access and modifications for compliance and security.

6. Cloud vs. On-Premises Data Warehouses

Choosing between a cloud-based and an on-premises data warehouse depends on various factors, including cost, scalability, security, and compliance requirements.

Cloud Data Warehouses (e.g., Redshift, Snowflake, BigQuery):

Pros:

- **Scalability:** Easily scale resources up or down based on demand.
- **Pay-as-you-go Pricing:** Pay only for the resources you use.
- **Managed Services:** The cloud provider handles infrastructure management.

Cons:

- **Potential Compliance Issues:** May not meet specific regulatory requirements.
- **Data Egress Costs:** Charges for moving data out of the cloud.
- **Vendor Lock-in:** Switching to a different provider can be challenging.

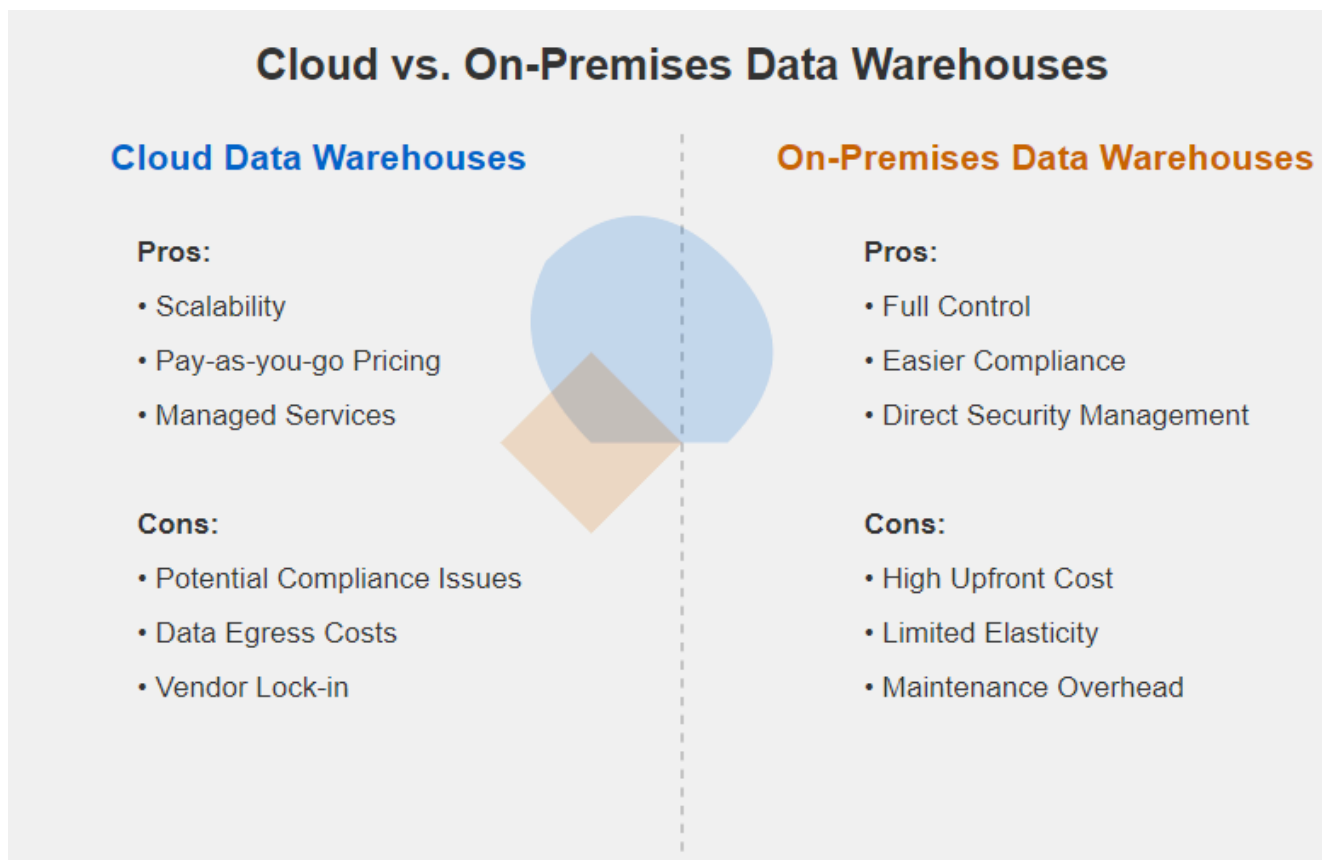
On-Premises Data Warehouses:

Pros:

- **Full Control:** Complete control over the environment and security.
- **Compliance:** Easier to meet specific regulatory requirements.

Cons:

- **High Upfront Cost:** Significant investment in hardware and infrastructure.
- **Limited Elasticity:** Scaling resources can be time-consuming and expensive.
- **Maintenance Overhead:** Requires dedicated IT staff for maintenance and management.



Cloud vs. On-Premise Comparison

7. Real-Time vs. Batch Processing

The choice between real-time (streaming) and batch processing depends on the specific use case and the required data latency.

Real-Time Processing:

- **Use Cases:** IoT sensor data, financial transactions, fraud detection, real-time dashboards.
- **Technologies:** Kafka, AWS Kinesis, Spark Streaming, Flink.
- **Advantages:** Provides immediate insights, enables real-time decision-making.
- **Disadvantages:** More complex to implement and manage, can be more expensive.

Batch Processing:

- **Use Cases:** Periodic reporting, data warehousing ETL/ELT, large-scale data analysis.
- **Technologies:** Airflow, Cron, Hadoop, Spark.
- **Advantages:** Simpler to implement, cost-effective for large datasets.
- **Disadvantages:** Data latency (delay between data generation and availability for analysis).

8. Flask vs. Airflow

When building data pipelines, choosing the right tools for ingestion and orchestration is crucial. Flask and Airflow are two popular options, but they serve different purposes and have distinct strengths and weaknesses.

Flask:

- **Strengths:** Ideal for lightweight, custom ingestion pipelines with minimal dependencies.
- **Limitations:** Lacks built-in orchestration and monitoring features, making it less suitable for complex workflows.

Airflow:

- **Strengths:** Designed for complex workflows with robust orchestration, scalability, and monitoring capabilities.

- **Limitations:** Higher learning curve and resource requirements compared to Flask.

When to Use Flask:

- **Simple Ingestion APIs:** When you need to create a simple API endpoint for data ingestion.
- **Small to Medium-Sized Pipelines:** When the pipeline has a limited number of tasks and dependencies.
- **Custom Solutions:** When you need a high degree of control and customization over the ingestion process.

When to Use Airflow:

- **Complex Workflows:** When the pipeline involves many tasks with complex dependencies.
- **Scheduled and Automated Pipelines:** When you need to schedule and automate pipeline execution.
- **Monitoring and Logging:** When detailed monitoring and logging of pipeline execution are required.
- **Scalable Pipelines:** When the pipeline needs to handle large volumes of data and scale to meet growing demands.

Flask vs. Airflow: Data Pipeline Tools

Flask

Strengths:

- Simplicity
- Flexibility
- Lightweight

Weaknesses:

- Limited Functionality
- Scalability Challenges
- Manual Orchestration

Simple APIs & Small Pipelines

Airflow

Strengths:

- Robust Orchestration
- Scalability
- Monitoring and Logging
- Extensibility

Weaknesses:

- Steeper Learning Curve
- Resource Intensive
- Complex Setup

Complex Workflows & Large Pipelines

Flask vs Airflow Comparison Table

9. Popular Data Pipeline Tools & Their Ecosystem

Beyond Flask and Airflow, the data pipeline ecosystem is rich with tools and platforms that cater to different needs and technical preferences.

Data Integration Platforms:

- **Stitch:** A cloud-based ELT platform that simplifies data extraction and loading from various sources to data warehouses.
- **Fivetran:** Similar to Stitch, Fivetran automates data replication with pre-built connectors and minimal configuration.
- **Matillion:** An ETL tool designed for cloud data warehouses, offering a visual interface for building data transformation pipelines.

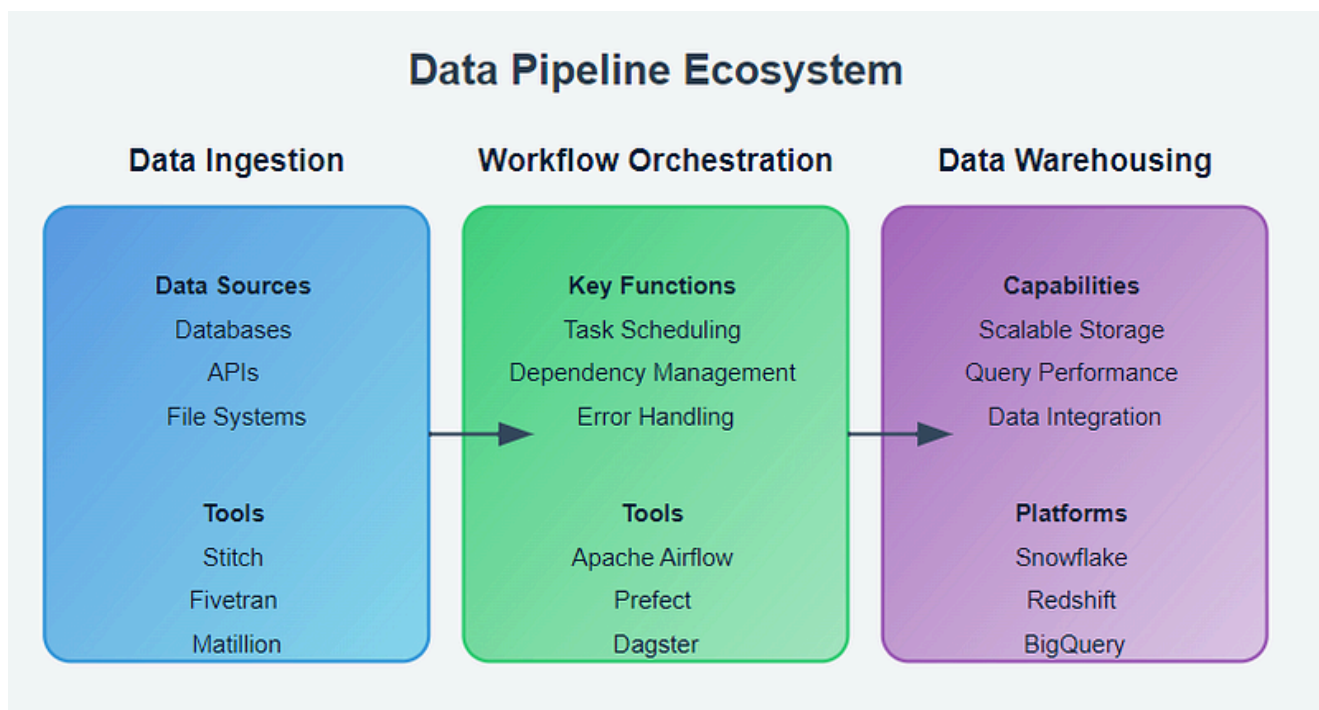
Workflow Orchestration:

- **Prefect:** A modern workflow orchestration tool that emphasizes dynamic, parameterizable workflows and a hybrid execution model.

- **Dagster:** A data orchestrator that focuses on development, testing, and observability of data pipelines.

Cloud-Based Data Warehouses:

- **Snowflake:** A cloud-native data warehouse known for its scalability, ease of use, and separation of compute and storage.
- **Amazon Redshift:** A fully managed, petabyte-scale data warehouse service offered by AWS.
- **Google BigQuery:** A serverless, highly scalable, and cost-effective multi-cloud data warehouse designed for business agility.



Tool Ecosystem Diagram

10. The Future of Data Pipelines

The field of data engineering is constantly evolving, and several trends are shaping the future of data pipelines:

- **DataOps:** Applying DevOps principles to data pipelines, emphasizing automation, continuous integration, and continuous delivery to accelerate the development and deployment of data pipelines.

- **Real-Time Data Streaming:** The increasing adoption of event-driven architectures and real-time analytics is driving the growth of streaming data pipelines.
- **AI/ML Integration:** AI and ML are being used to automate tasks like anomaly detection, data classification, and data quality improvement within data pipelines. Also, the rise of MLOps helps streamline deploying machine learning models into production and monitoring.
- **Composable Data Stack:** Microservices architecture is starting to be adopted for data pipelines, allowing for a more flexible and modular approach to data integration.

11. Conclusion and Best Practices

Best Practices:

- **Start Small and Iterate:** Begin with manageable projects and expand as capabilities mature.
- **Prioritize Governance and Security:** Implement robust policies and controls to protect data and ensure compliance.
- **Automate Workflows:** Use orchestration tools to improve efficiency and reduce manual effort.
- **Monitor and Optimize:** Continuously track performance and identify bottlenecks.
- **Document Processes:** Maintain clear documentation for maintainability and collaboration.

By embracing these principles, organizations can build modern data stacks that empower them to derive actionable insights and drive strategic decisions with confidence.

Movie Recommender System: EDA and Building Baseline System (BoxRAG Part 1)

Welcome to the first installment of the BoxRAG project series! In this blog post, we'll delve into the foundational aspects of building a movie recommender system

using Exploratory Data Analysis (EDA) and traditional recommendation algorithms. We'll also set the stage for integrating advanced Retrieval-Augmented Generation (RAG) techniques with Mistral 7B, a powerful language model. By the end of this post, you'll have a comprehensive understanding of the dataset, the insights derived from EDA, and the implementation of baseline recommendation systems without leveraging RAG.



Photo by [Thibault Penin](#) on [Unsplash](#)

1. Introduction

1.1. Welcome to BoxRAG

Welcome to the BoxRAG project series! BoxRAG stands for Box Office Retrieval-Augmented Generation, a project dedicated to developing an intelligent movie recommender system. This series will guide you through the journey of transforming raw movie data into insightful and personalized recommendations using state-of-the-art techniques.

1.2. Importance of Movie Recommendation Systems

In today's digital age, the sheer volume of available movies and series can be overwhelming. Effective recommendation systems play a crucial role in enhancing user experience by helping users discover content tailored to their preferences. These systems not only increase user engagement but also drive satisfaction and loyalty.

1.3. Blog Objectives

In this first part of the BoxRAG series, we will:

- Conduct comprehensive Exploratory Data Analysis (EDA) on the [MovieLens dataset](#).
 - Implement traditional (No-RAG) recommendation systems: Popularity-Based, Content-Based, and Collaborative Filtering.
 - Lay the groundwork for integrating Retrieval-Augmented Generation (RAG) with Mistral 7B in future installments while we .
-

2. Project Setup

2.1. Tools and Technologies

To build the BoxRAG recommender system, we utilized a robust set of tools and libraries:

- Data Processing & Analysis: Pandas, NumPy
- Visualization: Matplotlib, Seaborn, WordCloud
- Machine Learning: Scikit-learn
- Recommendation Systems: Custom Python functions
- Backend: Flask
- Frontend: React
- Vector Stores: FAISS
- Language Models: Mistral 7B (Other Smaller LLMs will be used in future parts)
- Others: Docker for containerization

2.2. Dataset Overview

We're using [The Movies Dataset](#), which provides metadata for over 45,000 movies along with 26 million ratings from more than 270,000 users. This rich dataset includes detailed information such as cast, crew, plot keywords, budget, revenue, posters, release dates, languages, production companies, and more.

Dataset Files:

- `movies_metadata.csv`: Core metadata for 45,000 movies.
- `keywords.csv`: Plot keywords for movies.
- `credits.csv`: Cast and crew information.
- `links.csv` & `links_small.csv`: TMDb and IMDb IDs for movies.
- `ratings.csv` & `ratings_small.csv`: User ratings data.

2.3. Folder Structure

Maintaining an organized project structure is vital for scalability and maintainability. Here's an overview of our project directory:

BoxOfficeRAG/

— backend/	# Core backend services
— notebooks/	# Analysis and dump notebooks
— data/	# Data storage and processing
— docker/	# Containerization
— prompts/	# LLM prompt templates

3. Loading and Inspecting the Data

Before diving into analysis, it's essential to load and understand the structure of our datasets.

3.1. Initial Data Inspection

Understanding the first few rows of each dataset helps in grasping their structure and identifying key features.

Column name and Shape for each DataFrame.

column_names_shape(mov_data)

Datasets Files Column Names and Shape

Datasets Name credits.csv

['cast', 'crew', 'id', 'directors', 'cast_names']

[45439, 5]

Datasets Name keywords.csv

['id', 'keywords', 'keyword_names']

[45432, 3]

Datasets Name links.csv

['movied', 'imdbid', 'tmdbid']

[45843, 3]

Datasets Name links_small.csv

['movied', 'imdbid', 'tmdbid']

[9125, 3]

Datasets Name movies_metadata.csv

['adult', 'belongs_to_collection', 'budget', 'genres', 'homepage', 'id',

```
'imdb_id', 'original_language', 'original_title', 'overview', 'popularity',  
'poster_path', 'production_companies', 'production_countries',  
'release_date', 'revenue', 'runtime', 'spoken_languages', 'status', 'tagline',  
'title', 'video', 'vote_average', 'vote_count', 'genre_names', 'release_year',  
'cluster']
```

```
[45449, 27]
```

Datasets Name ratings.csv

```
['userId', 'movieId', 'rating', 'timestamp']
```

```
[26024289, 4]
```

Datasets Name ratings_small.csv

```
['userId', 'movieId', 'rating', 'timestamp']
```

```
[100004, 4]
```

3.2. Handling Missing Values and Duplicates

Data often contains missing values or duplicates that can skew analysis. Addressing these issues ensures data integrity.

Summarize missing values for each DataFrame.

```
missing_summary = summarize_missing_values(mov_data)
```

```
print("Missing Values Summary:\n", missing_summary)
```

Missing Values Summary:

	credits.csv	keywords.csv	links.csv	links_small.csv	\
belongs_to_collection	NaN	NaN	NaN	NaN	
homepage	NaN	NaN	NaN	NaN	
imdb_id	NaN	NaN	NaN	NaN	
original_language	NaN	NaN	NaN	NaN	
overview	NaN	NaN	NaN	NaN	
popularity	NaN	NaN	NaN	NaN	
poster_path	NaN	NaN	NaN	NaN	
production_companies	NaN	NaN	NaN	NaN	
production_countries	NaN	NaN	NaN	NaN	
release_date	NaN	NaN	NaN	NaN	
revenue	NaN	NaN	NaN	NaN	
runtime	NaN	NaN	NaN	NaN	
spoken_languages	NaN	NaN	NaN	NaN	
status	NaN	NaN	NaN	NaN	
tagline	NaN	NaN	NaN	NaN	
title	NaN	NaN	NaN	NaN	
tmdbId	NaN	NaN	219.0	13.0	
video	NaN	NaN	NaN	NaN	
vote_average	NaN	NaN	NaN	NaN	

vote_count	NaN	NaN	NaN	NaN
------------	-----	-----	-----	-----

movies_metadata.csv	ratings.csv	ratings_small.csv
---------------------	-------------	-------------------

belongs_to_collection	40972.0	NaN	NaN
-----------------------	---------	-----	-----

homepage	37684.0	NaN	NaN
----------	---------	-----	-----

imdb_id	17.0	NaN	NaN
---------	------	-----	-----

original_language	11.0	NaN	NaN
-------------------	------	-----	-----

overview	954.0	NaN	NaN
----------	-------	-----	-----

popularity	5.0	NaN	NaN
------------	-----	-----	-----

poster_path	386.0	NaN	NaN
-------------	-------	-----	-----

production_companies	3.0	NaN	NaN
----------------------	-----	-----	-----

production_countries	3.0	NaN	NaN
----------------------	-----	-----	-----

release_date	87.0	NaN	NaN
--------------	------	-----	-----

revenue	6.0	NaN	NaN
---------	-----	-----	-----

runtime	263.0	NaN	NaN
---------	-------	-----	-----

spoken_languages	6.0	NaN	NaN
------------------	-----	-----	-----

status	87.0	NaN	NaN
--------	------	-----	-----

tagline	25054.0	NaN	NaN
---------	---------	-----	-----

title	6.0	NaN	NaN
-------	-----	-----	-----

tmdbId	NaN	NaN	NaN
--------	-----	-----	-----

video	6.0	NaN	NaN
-------	-----	-----	-----

vote_average	6.0	NaN	NaN
vote_count	6.0	NaN	NaN

4. Exploratory Data Analysis (EDA)

EDA provides valuable insights into the dataset, uncovering patterns and relationships that inform our recommendation strategies.

4.1. Distribution of Ratings

Understanding how users rate movies helps in identifying overall satisfaction levels.

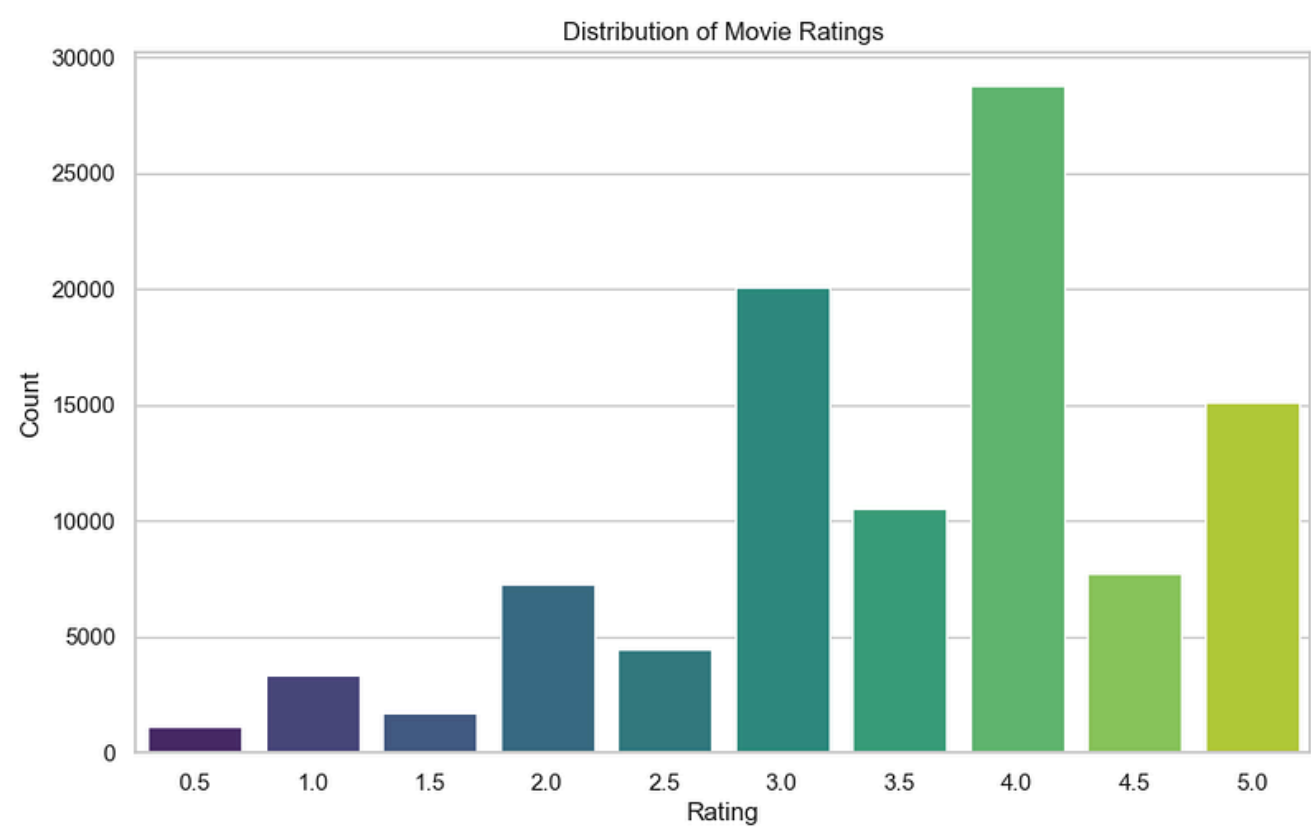


Figure 1: Distribution of Movie Ratings

Most ratings cluster around 3–4 stars, indicating a general satisfaction among users.

4.2. Popular Genres

Identifying the most and least popular genres guides the focus of our recommendations.

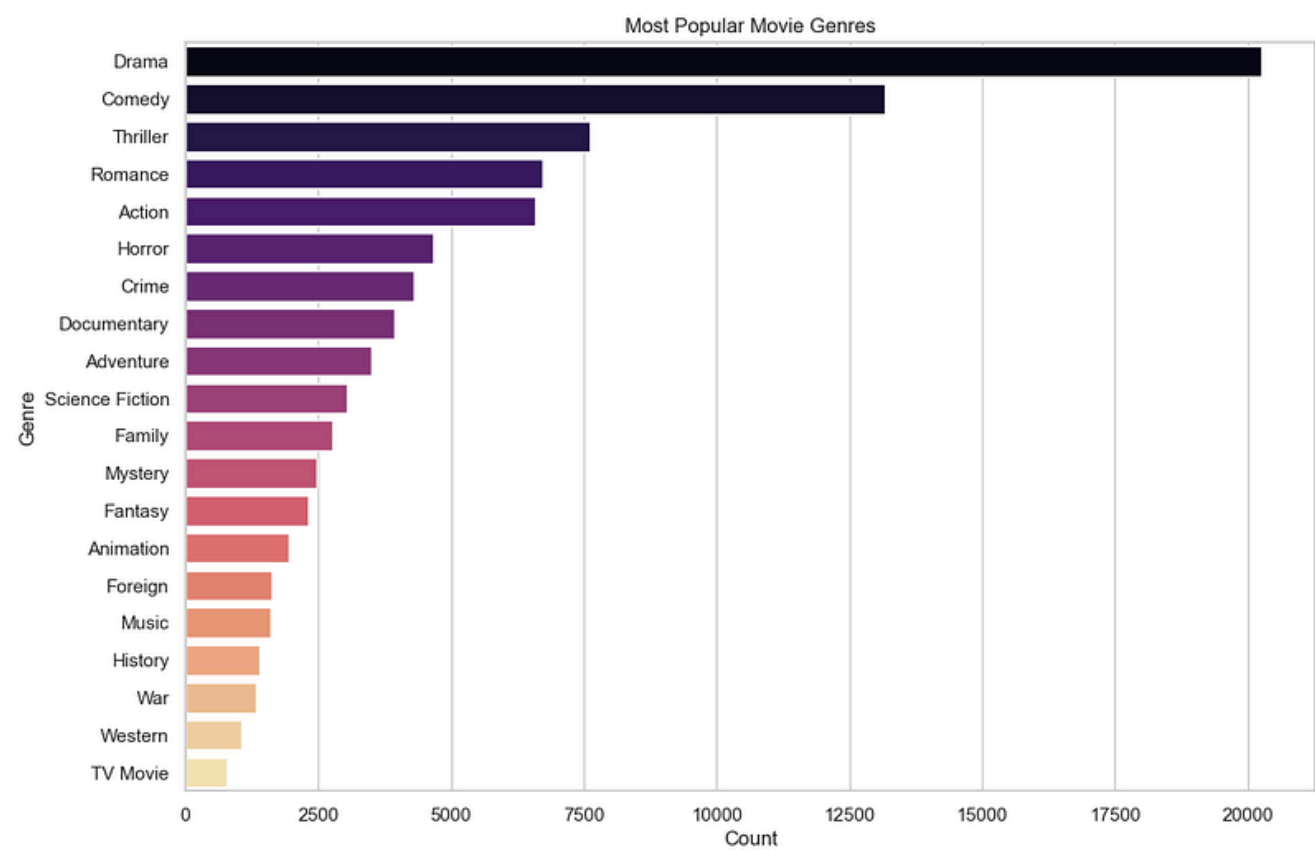


Figure 2: Popularity of Movie Genres

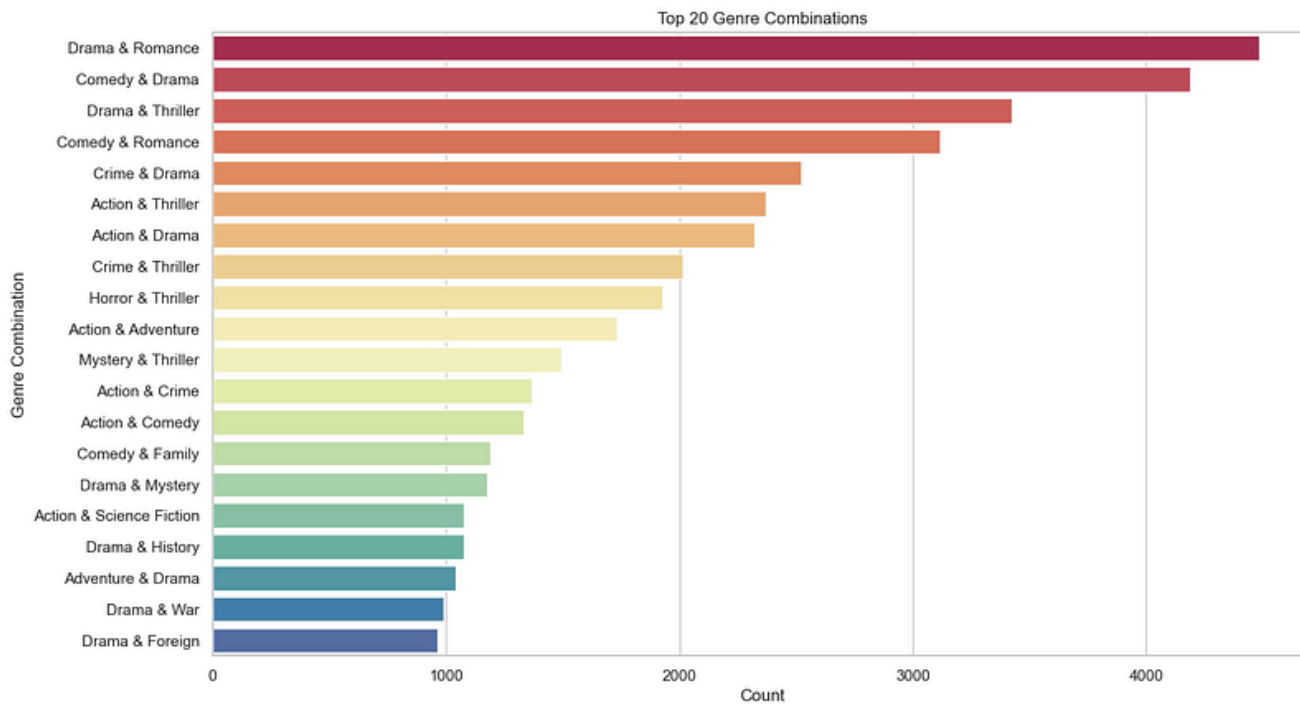


Figure 2.1: Popularity of Genres Combinations

Genres like Drama, Comedy and Thriller dominate the dataset, while niche genres have fewer entries.

4.3. Budget vs. Revenue Analysis

Analyzing the relationship between a movie's budget and its revenue can highlight which genres or types of movies are financially successful.

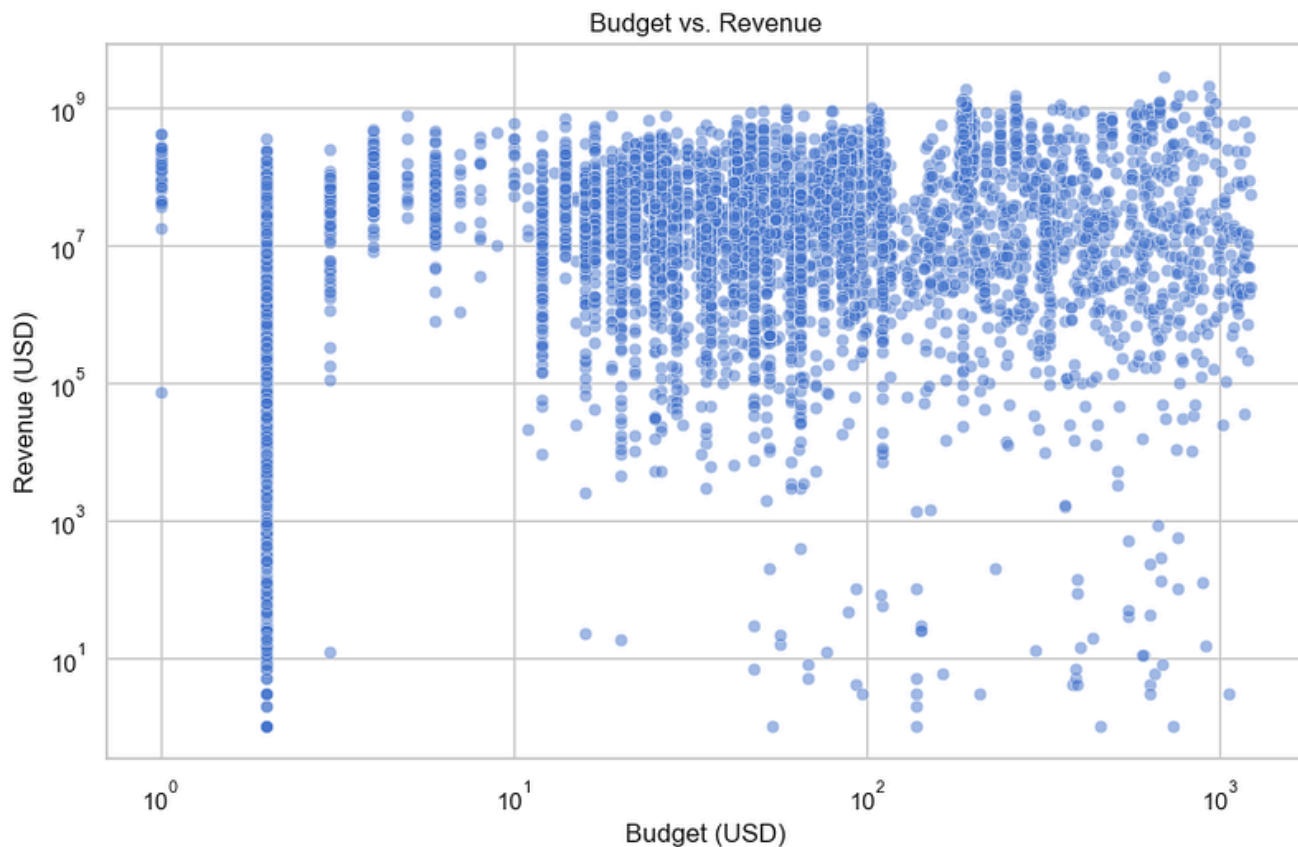


Figure 3: Budget vs. Revenue of Movies

There's a positive correlation between budget and revenue, though not all high-budget movies perform well.

4.4. Top Directors and Actors

Identifying influential directors and popular actors can help in content-based recommendations.

Top 20 Directors by Number of Movies

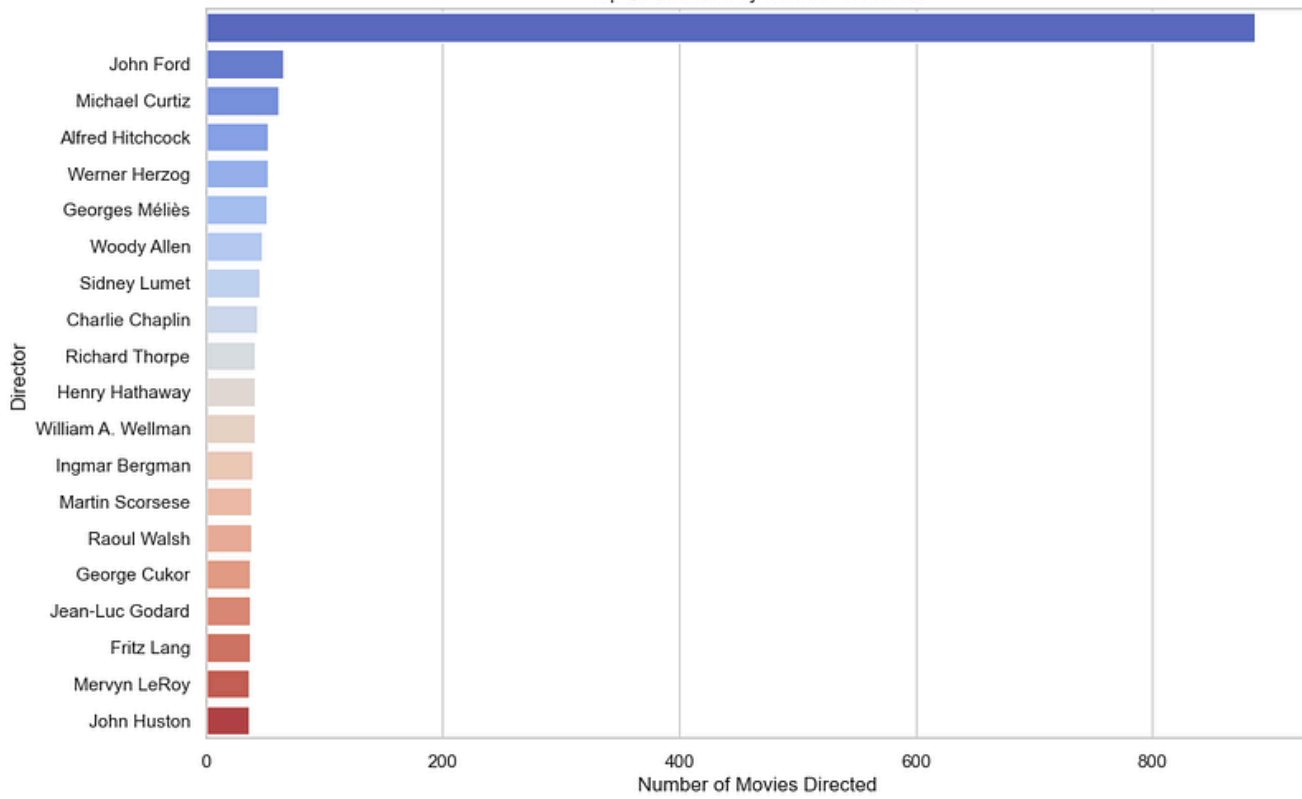


Figure 4: Top Directors by Number of Movies

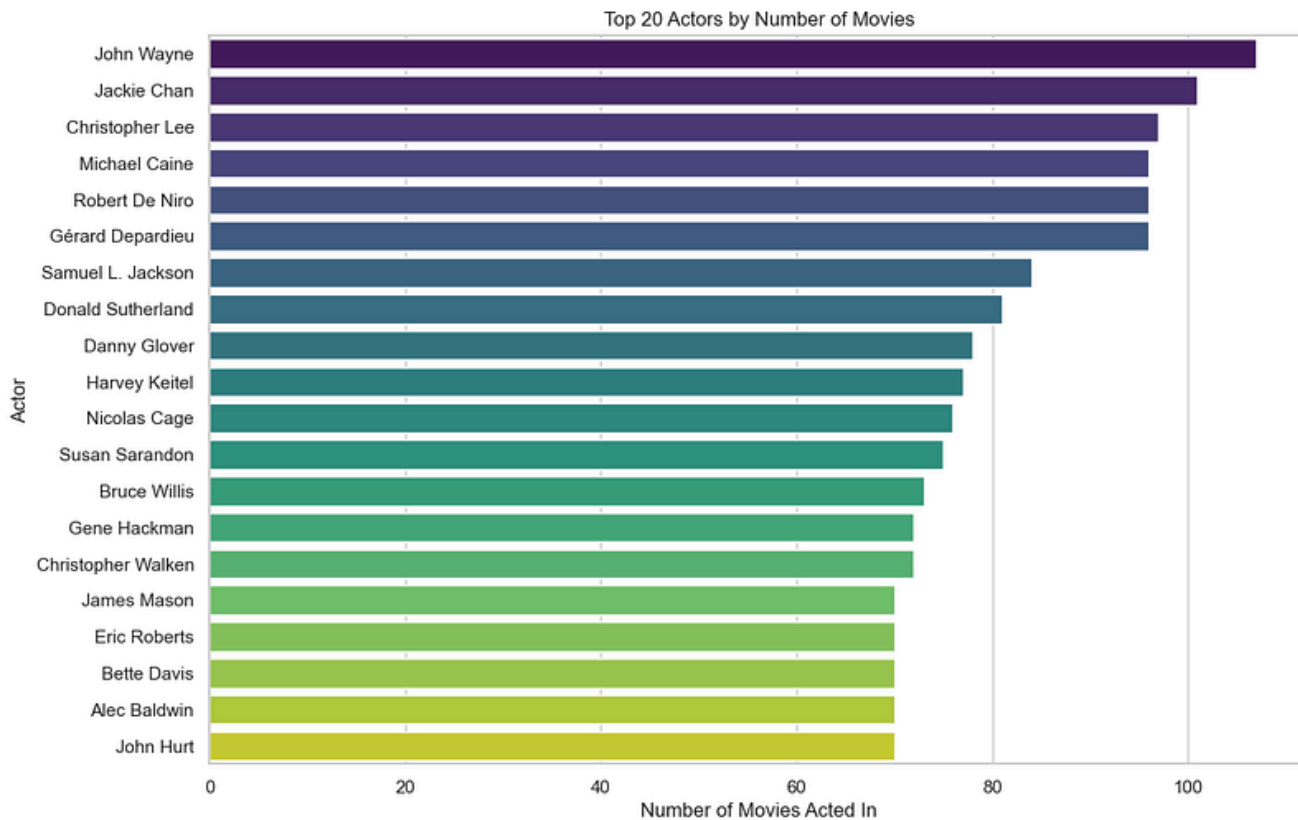


Figure 5: Top Actors by Movie Appearances

Directors like Steven Spielberg and actors such as Tom Hanks feature prominently, indicating their significant impact on movie popularity.

4.5. Keyword Analysis

Plot keywords provide insight into common themes and storylines in movies.

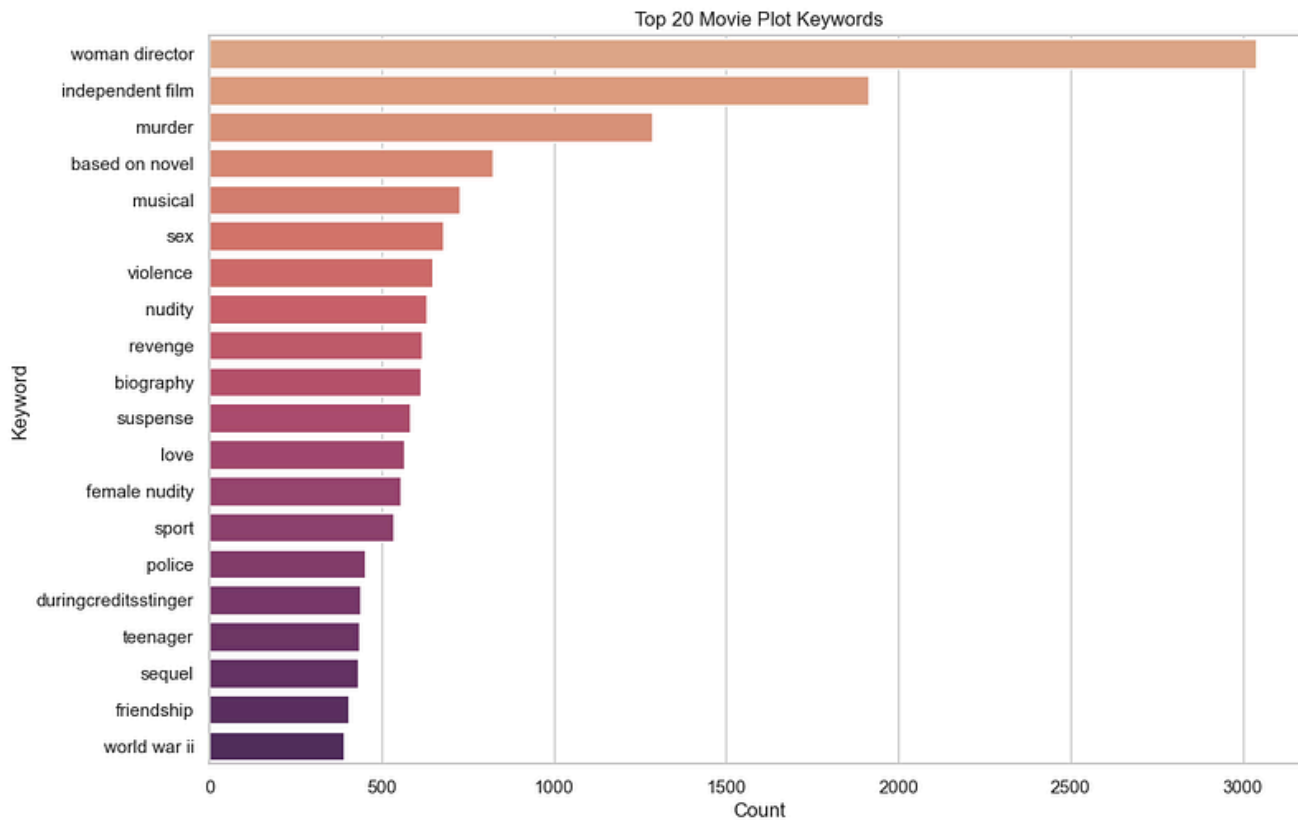


Figure 6: Common Plot Keywords

Keywords like “love”, “war”, and “hero” are frequently associated with highly rated movies, suggesting these themes resonate well with audiences.

4.6. User Behavior Insights

Analyzing user rating patterns reveals engagement levels and preferences.

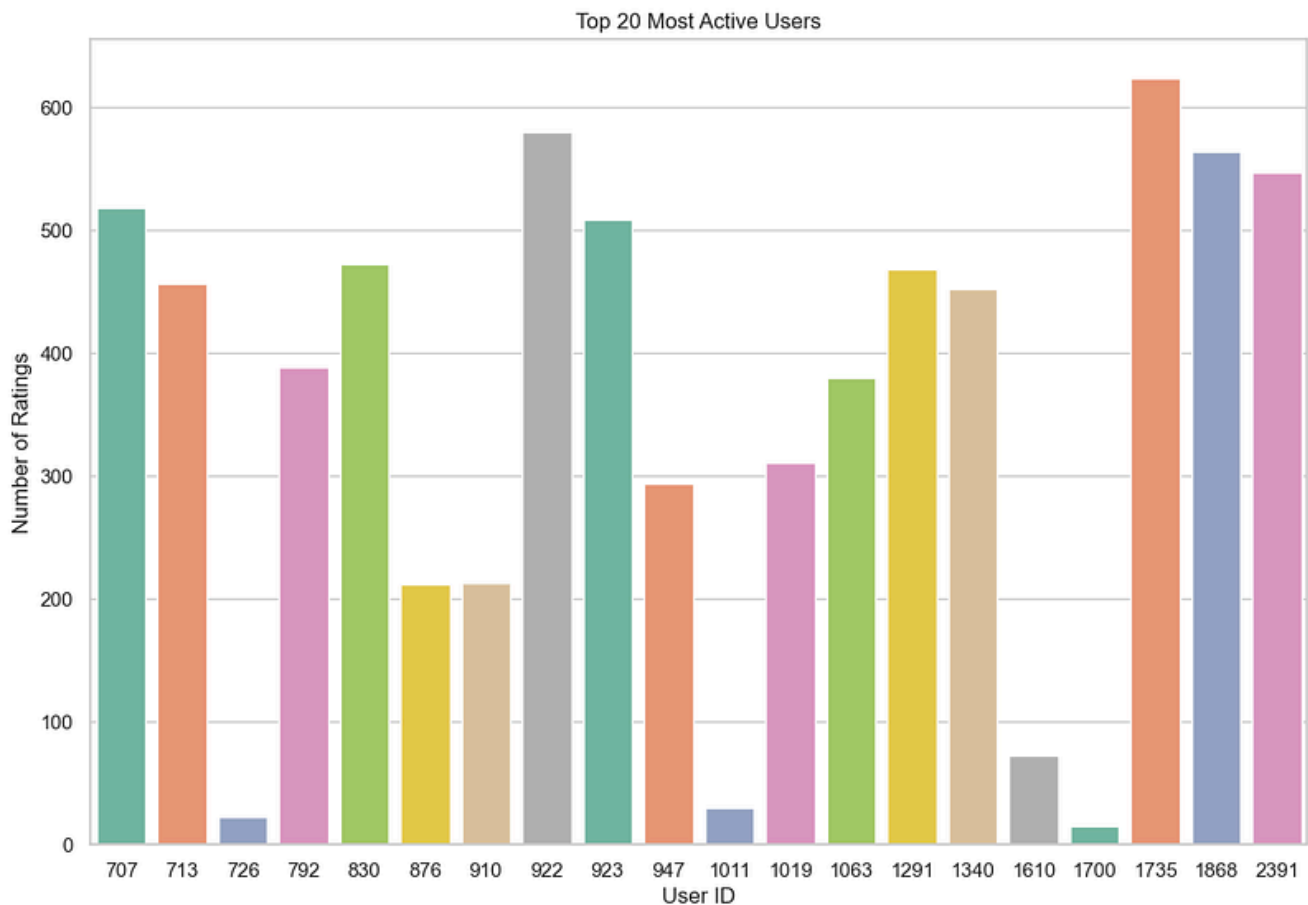


Figure 7: User Rating Activity

A small percentage of users contribute the majority of ratings, indicating the presence of power users who are highly engaged with the platform.

4.7. Time-Based Analysis

Exploring trends over time shows how movie popularity and production have evolved.

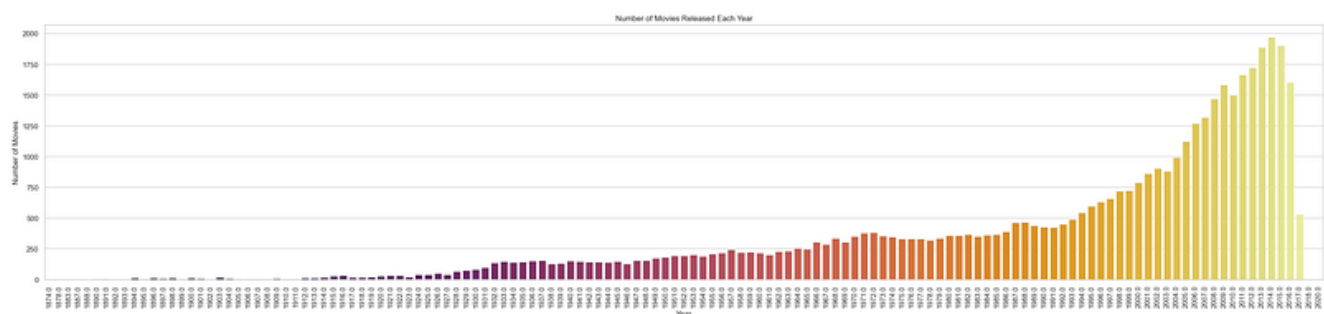


Figure 8: Number of Movies Released Each Year

The number of movie releases has steadily increased, with peaks in blockbuster seasons, reflecting industry trends and audience demand.

4.8. Language Distribution

Understanding the diversity of languages in movies can aid in multilingual recommendations.

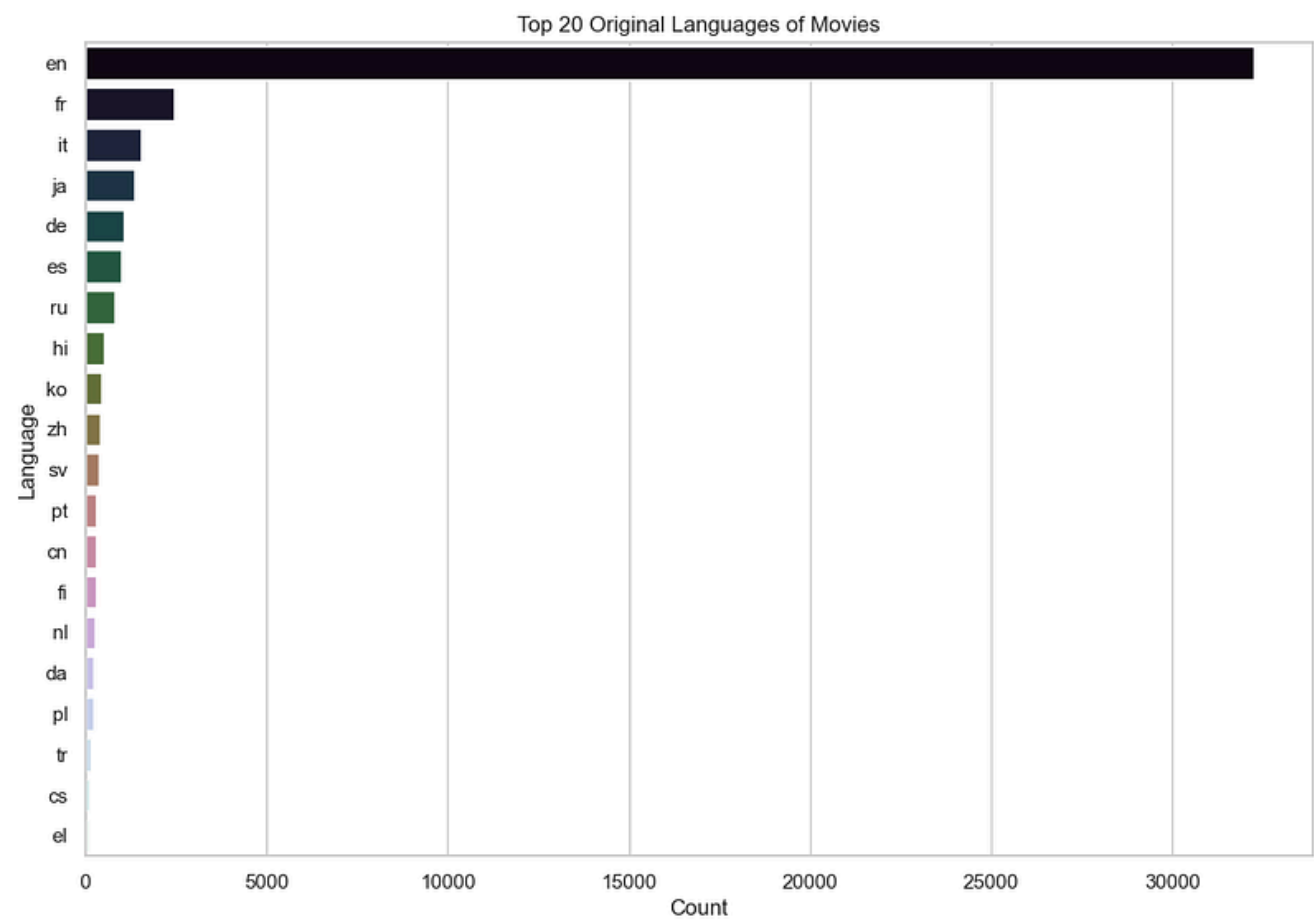


Figure 9: Distribution of Original Languages

While English dominates, there’s a significant presence of other languages like French and Japanese, highlighting the global nature of the film industry.

4.9. Cluster Analysis

Grouping movies based on features like budget, revenue, and ratings helps identify distinct categories.

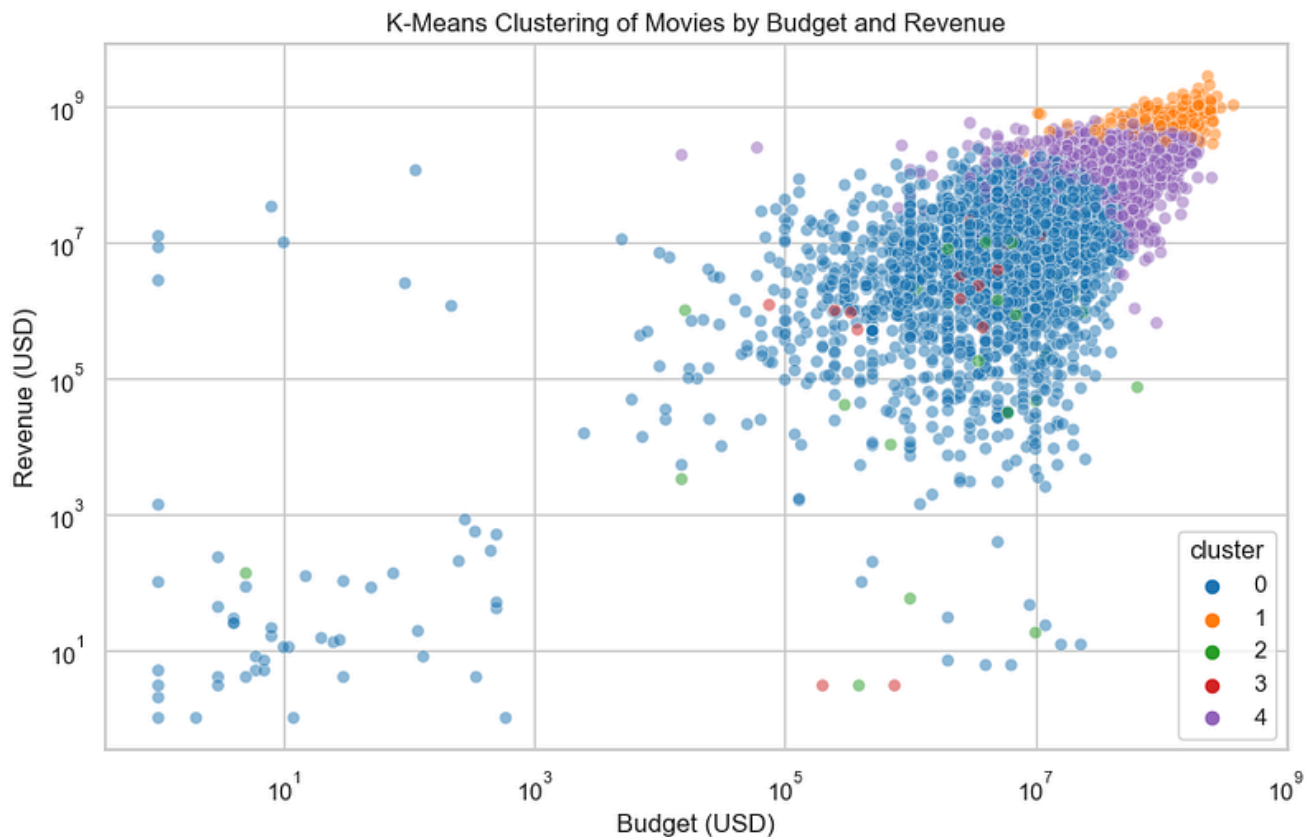


Figure 11: Movie Clusters Based on Budget and Revenue

Clusters reveal segments such as high-budget blockbusters, indie films, and mid-range productions, each with unique characteristics and audience appeal.

4.10. Additional Visualizations

Further visualizations offer deeper insights into the dataset's nuances.

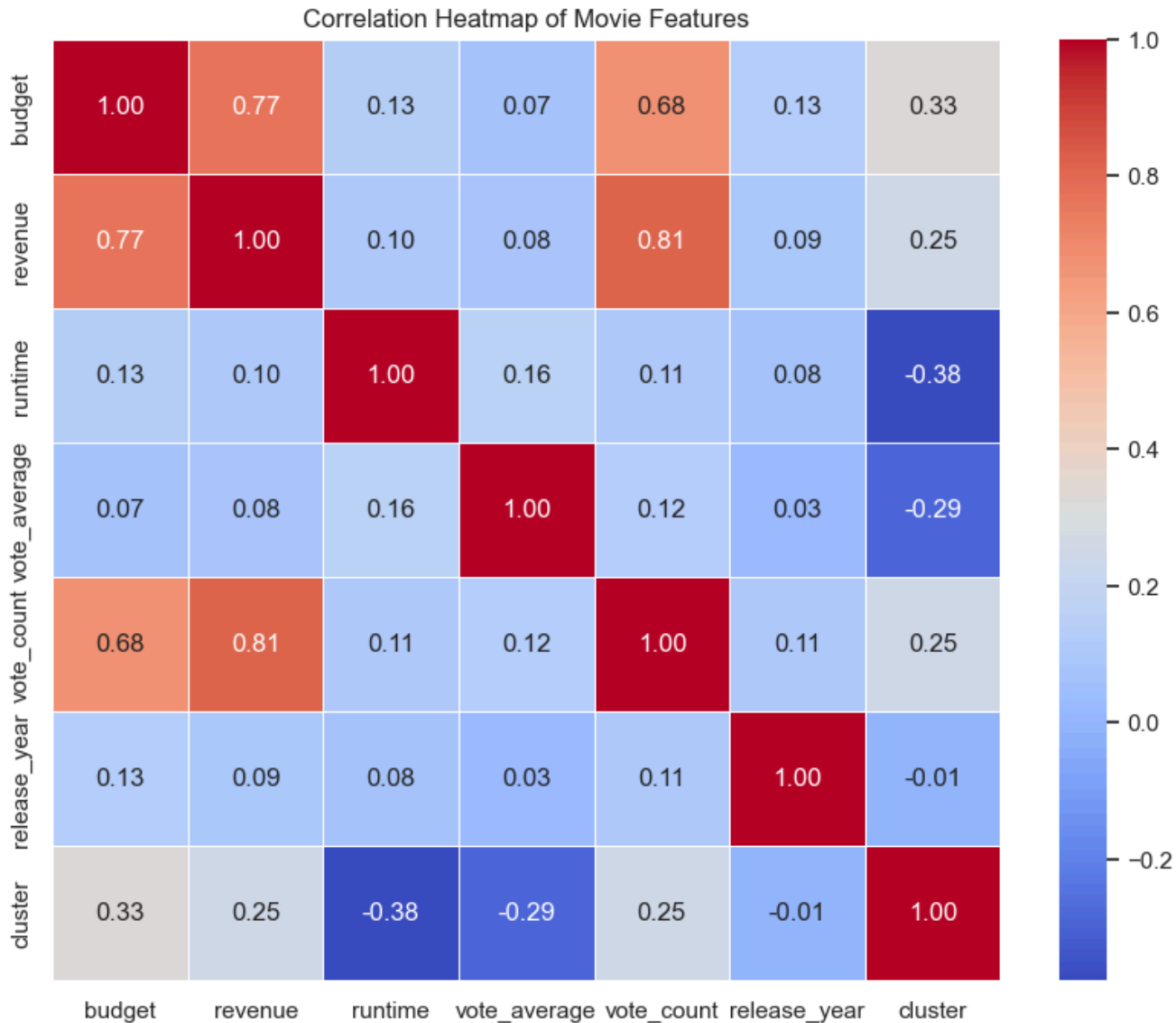


Figure 12: Correlation Heatmap of Movie Features

This heatmap illustrates the correlations between various movie features, such as budget, revenue, runtime, and ratings, highlighting the interdependencies that influence a movie’s success.

5. Building Baseline Recommendation Systems (No RAG)

With a solid understanding of the dataset, we can now implement traditional recommendation algorithms. These baseline systems will serve as benchmarks for future RAG-based enhancements.

5.1. Popularity-Based Recommender

5.1.1. Concept and Rationale

The Popularity-Based Recommender suggests movies that are generally popular within a specified genre. Popularity can be measured by metrics such as the number of ratings or average ratings. This approach is straightforward and ensures that users are presented with widely acclaimed content.

5.1.2. Implementation Details

We implemented a genre-based popularity recommender that filters movies by genre, calculates average ratings and the number of ratings, and recommends the top N movies based on these metrics.

```
def popularity_based_recommender_genre(movies_df: pd.DataFrame, ratings_df:
pd.DataFrame, genre: str, top_n: int = 10) -> pd.DataFrame:
```

```
    """
```

Recommend top N popular movies within a specified genre based on average ratings and number of ratings.

Args:

movies_df (pd.DataFrame): DataFrame containing movie metadata.

ratings_df (pd.DataFrame): DataFrame containing user ratings.

genre (str): The genre to filter movies by.

top_n (int): Number of top movies to recommend.

Returns:

pd.DataFrame: DataFrame of recommended movies.

.....

Filter movies that belong to the specified genre

```
genre_filtered = movies_df[movies_df['genre_names'].apply(lambda genres:  
genre in genres)]
```

Calculate average rating and number of ratings per movie

```
movie_ratings = ratings_df.groupby('movieId').agg({'rating': ['mean', 'count']})
```

```
movie_ratings.columns = ['average_rating', 'rating_count']
```

```
movie_ratings = movie_ratings.reset_index()
```

Merge with movies metadata

```
movies_with_ratings = pd.merge(genre_filtered, movie_ratings, left_on='id',  
right_on='movieId', how='left')
```

Handle missing values

```
movies_with_ratings['average_rating'].fillna(0, inplace=True)
```

```
movies_with_ratings['rating_count'].fillna(0, inplace=True)
```

Filter movies with a minimum number of ratings to ensure reliability

```
popular_movies = movies_with_ratings[movies_with_ratings['rating_count'] >=  
100]
```

```
# Sort by average rating and number of ratings
```

```
popular_movies = popular_movies.sort_values(['average_rating', 'rating_count'],  
ascending=[False, False])
```

```
# Select top N movies
```

```
recommended_movies = popular_movies.head(top_n)
```

```
return recommended_movies[['title', 'average_rating', 'rating_count', 'genres',  
'revenue']]
```

5.1.3. Sample Recommendations

Here's an example of how the Popularity-Based Recommender performs for the Comedy genre:

```
# Example usage
```

```
genre_query = "Comedy"
```

```
top_n = 5
```

```
top_comedy_movies = popularity_based_recommender_genre(
```

```
    movies_df=mov_data['movies_metadata.csv'],
```

```
    ratings_df=mov_data['ratings_small.csv'],
```

```
    genre=genre_query,
```

```
    top_n=top_n
```

```
)
```

`print(top_comedy_movies)`

	title	average_rating	rating_count	genres	revenue
203	Sleepless in Seattle	4.487500	200.0	[{'id': 35, 'name': 'Comedy'}, {'id': 18, 'nam...	227799884.0
1988	Men in Black II	4.256696	224.0	[{'id': 28, 'name': 'Action'}, {'id': 12, 'nam...	441818803.0
7905	Shriek If You Know What I Did Last Friday the ...	4.227273	132.0	[{'id': 35, 'name': 'Comedy'}]	0.0
3078	Murder She Said	4.209524	105.0	[{'id': 18, 'name': 'Drama'}, {'id': 80, 'name...	0.0
4181	License to Wed	4.178218	202.0	[{'id': 35, 'name': 'Comedy'}]	69307224.0

Table 5: Most Popular Genres by User Ratings

5.1.4. Limitations

While effective in highlighting popular movies, this approach doesn’t account for individual user preferences or contextual nuances, potentially overshadowing niche or recently released films.

5.2. Content-Based Recommender

5.2.1. Concept and Rationale

The Content-Based Recommender suggests movies similar to those a user has previously enjoyed by analyzing attributes like genres, keywords, directors, and plot summaries. This approach ensures that recommendations are tailored to the user’s specific tastes based on the inherent characteristics of the movies.

5.2.2. Implementation Details

We developed a genre-based content recommender that filters movies by genre, merges them with plot keywords, and calculates content similarity using TF-IDF vectorization.

```
def content_based_recommender_genre(movies_df: pd.DataFrame, keywords_df:
pd.DataFrame, genre: str, top_n: int = 10) -> pd.DataFrame:

    """

    Recommend top N movies within a specified genre based on content similarity
    (genres and keywords).
```

Args:

movies_df (pd.DataFrame): DataFrame containing movie metadata.

keywords_df (pd.DataFrame): DataFrame containing movie keywords.

genre (str): The genre to filter movies by.

top_n (int): Number of top movies to recommend.

Returns:

pd.DataFrame: DataFrame of recommended movies.

"""

Filter movies that belong to the specified genre

movies_df['id'] = movies_df['id'].astype(str)

**genre_filtered = movies_df[movies_df['genre_names'].apply(lambda genres:
genre in genres)]**

Merge with keywords

genre_keywords = pd.merge(genre_filtered, keywords_df, on='id', how='left')

Fill NaN keywords with empty lists

**genre_keywords['keyword_names'] =
genre_keywords['keyword_names'].apply(lambda x: x if isinstance(x, list) else [])**

Create a combined feature string

```
genre_keywords['combined_features'] = genre_keywords.apply(  
    lambda row: ' '.join(row['genre_names']) + ' ' + ' '.join(row['keyword_names']),  
    axis=1  
)
```

Encode combined features using TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer  
  
tfidf = TfidfVectorizer(stop_words='english')  
  
tfidf_matrix = tfidf.fit_transform(genre_keywords['combined_features'])
```

Compute cosine similarity

```
cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)
```

Create a mapping from movie titles to indices

```
indices = pd.Series(genre_keywords.index,  
index=genre_keywords['title']).drop_duplicates()
```

Function to get recommendations

```
def get_recommendations(title, cosine_sim=cosine_sim):
```



```
idx = indices[title]

sim_scores = list(enumerate(cosine_sim[idx]))

sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)

sim_scores = sim_scores[1:top_n+1]

movie_indices = [i[0] for i in sim_scores]

return genre_keywords.iloc[movie_indices]
```

For genre-based recommendation, we'll average similarity scores within the genre

Alternatively, recommend the top N movies within the genre based on TF-IDF similarity

Here, we'll recommend top N movies within the genre with the highest TF-IDF scores

which inherently captures content similarity

Sort by TF-IDF vector sum (as a proxy for content richness)

```
genre_keywords['tfidf_sum'] =
genre_keywords['combined_features'].apply(lambda x: len(x.split()))
```

```
recommended_movies = genre_keywords.sort_values('tfidf_sum',
ascending=False).head(top_n)
```

```
return recommended_movies[['title', 'genres', 'keywords', 'tfidf_sum']]
```

5.2.3. Sample Recommendations

Here’s how the Content-Based Recommender performs for the Action genre:

Example usage

```
genre_query = "Drama"

top_n = 5

top_action_movies = content_based_recommender_genre(

    movies_df=movies_metadata,

    keywords_df=keywords,

    genre=genre_query,

    top_n=top_n

)

print(top_action_movies)
```

	title	genre_names	keyword_names	similarity_score
50	The Journey of August King	[Drama]	[]	0.181007
69	The Boys of St. Vincent	[Drama]	[]	0.181007
118	The Browning Version	[Drama]	[]	0.181007
188	Red Firecracker, Green Firecracker	[Drama]	[]	0.181007
232	Boys Life: Three Stories of Love, Lust, and Li...	[Drama]	[]	0.181007

Table 6: Movies Recommended by Content-Based Filtering

5.2.4. Limitations

While providing tailored recommendations, this approach may overemphasize specific content features, potentially limiting the diversity of suggestions and neglecting user-specific nuances.

5.3. Collaborative Filtering Recommender

5.3.1. Concept and Rationale

The Collaborative Filtering Recommender suggests movies based on the preferences and behaviors of similar users. By analyzing user ratings, it identifies patterns and recommends titles that like-minded users have enjoyed. This approach excels in personalization, ensuring that recommendations align closely with individual tastes.

5.3.2. Implementation Details

We implemented a genre-based collaborative filtering recommender that constructs a user-item matrix, computes user similarities, and recommends movies that similar users have rated highly.

```
def collaborative_filtering_recommender_genre(
```

```
    ratings_df: pd.DataFrame,
```

```
    movies_df: pd.DataFrame,
```

```
    target_user_id: int,
```

```
    genre: str = None,
```

```
    top_n: int = 10
```

```
) -> pd.DataFrame:
```

```
    """
```

Recommend top N movies for a target user based on collaborative filtering, optionally filtered by genre.

Args:

ratings_df (pd.DataFrame): DataFrame containing user ratings.

movies_df (pd.DataFrame): DataFrame containing movie metadata.

target_user_id (int): ID of the target user.

genre (str, optional): Genre to filter recommendations by. Defaults to None.

top_n (int): Number of top movies to recommend.

Returns:

pd.DataFrame: DataFrame of recommended movies.

.....

Create user-item matrix

```
user_item_matrix = ratings_df.pivot_table(index='userId', columns='movieId',  
values='rating').fillna(0)
```

```
user_item_sparse = csr_matrix(user_item_matrix.values)
```

Compute cosine similarity between users

```
user_sim = cosine_similarity(user_item_sparse, user_item_sparse)
```

```
user_sim_df = pd.DataFrame(user_sim, index=user_item_matrix.index,  
columns=user_item_matrix.index)
```

Find similar users

```
similar_users =  
user_sim_df[target_user_id].sort_values(ascending=False)[1:11].index
```

Aggregate ratings from similar users

```
similar_users_ratings = ratings_df[ratings_df['userId'].isin(similar_users)]
```

```
movie_scores = similar_users_ratings.groupby('movieId').agg({'rating': ['mean',  
'count']})
```

```
movie_scores.columns = ['average_rating', 'rating_count']
```

```
movie_scores = movie_scores.reset_index()
```

Merge with movies metadata

```
recommended_movies = pd.merge(movie_scores, movies_df, left_on='movieId',  
right_on='id', how='left')
```

Handle missing values

```
recommended_movies['average_rating'].fillna(0, inplace=True)
```

```
recommended_movies['rating_count'].fillna(0, inplace=True)
```

```
recommended_movies['genre_names'] =  
recommended_movies['genre_names'].apply(lambda x: x if isinstance(x, list) else  
[])
```

Optionally filter by genre

if genre:

```
recommended_movies =  
recommended_movies[recommended_movies['genre_names'].apply(lambda  
genres: genre in genres)]
```

```

# Filter out movies already rated by the target user

rated_movies = ratings_df[ratings_df['userId'] ==
target_user_id]['movieId'].unique()

recommended_movies =
recommended_movies[~recommended_movies['movieId'].isin(rated_movies)]


# Sort by average rating and number of ratings

recommended_movies = recommended_movies.sort_values(['average_rating',
'rating_count'], ascending=[False, False])


# Check if there are any recommendations

if recommended_movies.empty:

    print(f"No recommendations found for user ID {target_user_id} in genre
{genre}'.")

    return recommended_movies # Return empty DataFrame


# Return top N movies

return recommended_movies.head(top_n)[['title', 'average_rating', 'rating_count',
'genres', 'revenue']]

```

5.3.3. Sample Recommendations

Here's an example of how the Collaborative Filtering Recommender performs for a user with ID 4 and the Comedy genre:

Example usage

```

target_user_id = 4

genre_query = "Comedy"

top_n = 10

top_comedy_recommendations = collaborative_filtering_recommender_genre(

    ratings_df=ratings_small,

    movies_df=movies_metadata,

    target_user_id=4,

    genre=genre_query,

    top_n=top_n

)

print(top_comedy_recommendations)

```

	title	average_rating	rating_count	genres	revenue
160	Meet the Robinsons	5.0	2	[[{'id': 16, 'name': 'Animation'}, {'id': 35, 'name': 'Comedy'}]]	169332978.0
817	Memoirs of an Invisible Man	5.0	2	[[{'id': 35, 'name': 'Comedy'}, {'id': 14, 'name': 'Drama'}]]	14358033.0
858	24 Hour Party People	5.0	2	[[{'id': 35, 'name': 'Comedy'}, {'id': 18, 'name': 'Drama'}]]	0.0
2	Lock, Stock and Two Smoking Barrels	5.0	1	[[{'id': 35, 'name': 'Comedy'}, {'id': 80, 'name': 'Crime'}]]	3897569.0
166	TMNT	5.0	1	[[{'id': 12, 'name': 'Adventure'}, {'id': 16, 'name': 'Animation'}]]	95608995.0
173	Bean	5.0	1	[[{'id': 35, 'name': 'Comedy'}]]	251212670.0
190	Forrest Gump	5.0	1	[[{'id': 35, 'name': 'Comedy'}, {'id': 18, 'name': 'Drama'}]]	677945399.0
248	Vollidiot	5.0	1	[[{'id': 35, 'name': 'Comedy'}, {'id': 18, 'name': 'Drama'}]]	0.0
301	Bring It On	5.0	1	[[{'id': 35, 'name': 'Comedy'}]]	90449929.0
326	Breakfast at Tiffany's	5.0	1	[[{'id': 35, 'name': 'Comedy'}, {'id': 18, 'name': 'Drama'}]]	9500000.0

Table 7: Movies Recommended by Collaborative Filtering

5.3.4. Limitations

While collaborative filtering excels in personalization, it faces challenges like the cold start problem—difficulty recommending for new users with limited rating history—and potential biases from highly active users.

6: Implementing and Integrating RAG with Mistral 7B

In this section, we introduce a practical implementation of a Retrieval-Augmented Generation (RAG) pipeline leveraging the Mistral 7B model, sentence-transformers for embeddings, and FAISS for vector storage. This integration is tailored to provide fast, accurate, and context-rich responses to natural language queries, making it ideal for use cases in recommendation systems, knowledge retrieval, and beyond.

6.1 RAG Pipeline Components

1. **Mistral 7B as the LLM:** Mistral 7B is used as the backbone language model for generating detailed and context-aware responses based on retrieved documents.
2. **Embeddings with Sentence-Transformers:** We employ the 'all-MiniLM-L6-v2' model from Sentence-Transformers to generate 384-dimensional embeddings. These embeddings provide a compact and semantically rich representation of the input text, optimizing retrieval performance.
3. **Vector Store with FAISS:** The FAISS (Facebook AI Similarity Search) library is utilized as the vector database to efficiently store and query embeddings. This setup enables scalable and high-speed retrieval of relevant documents.
4. **BasicRAG Architecture:** A lightweight retrieval-augmented generation approach is implemented via the **BasicRAG** class. This custom architecture ensures flexibility while maintaining simplicity for quick integration into existing pipelines.

6.2 RAG Implementation Snippets

Defining the BasicRAG Class

```
class BasicRAG(BaseRAG):
```

```
    def query(self, query_text: str) -> Dict[str, Any]:
```



```
query_embedding = self.embedding_model.encode(query_text)
```

```
return self.vector_store.query(query_embedding.tolist(), self.config.top_k)
```

This implementation queries the vector store using embeddings generated from the query text, retrieving the top-k relevant documents.

Setting Up the Pipeline

```
{  
  
    'name': 'fast',  
  
    'embedding': 'sentence-transformers/all-MiniLM-L6-v2',  
  
    'vector_store': FAISSStore(dimension=384),  
  
    'rag_type': BasicRAG  
  
}
```

This configuration enables rapid embedding generation and retrieval while ensuring compatibility with Mistral 7B for seamless integration.

6.3 Example Use Case: Movie Recommendations

The following example demonstrates the power of the RAG pipeline with Mistral 7B in the context of generating movie recommendations:

User Query:

“Recommend me a sci-fi movie with deep philosophical themes.”

Recommendation:

💬  In the realm of cinema where science fiction meets philosophy, there's a gem that has been quietly illuminating minds since its release - 'Gattaca.' A masterful blend of suspense, romance, and the profound exploration of identity, this film will leave you questioning the very

essence of humanity. Immerse yourself in the future society where humans are set on a life course based on their DNA, but find hope in the indomitable human spirit that transcends genetics. The captivating narrative unfolds through the eyes of Vincent Freeman, portrayed by the exceptional Ethan Hawke, whose dreams of reaching the stars are hindered by his 'inferior' genetic makeup.

Metrics:

- Retrieval Time: 0.67 seconds
- Generation Time: 93.44 seconds
- Total Time: 94.11 seconds

This example showcases the ability of the pipeline to understand nuanced queries and provide detailed, context-aware responses enriched by retrieved documents.

7. Conclusion and Next Steps

7.1. Recap of EDA Findings

Our comprehensive EDA unveiled key insights into user behavior, genre popularity, budget-revenue relationships, and more. These insights are instrumental in shaping our recommendation strategies, ensuring they are data-driven and aligned with user preferences.

7.2. Effectiveness of Baseline Recommenders

The implemented baseline recommenders—Popularity-Based, Content-Based, and Collaborative Filtering, each offer unique advantages in providing valuable recommendations. However, they also exhibit limitations, such as lack of personalization or difficulty in capturing nuanced user preferences, which can be significantly mitigated by incorporating advanced techniques like Retrieval-Augmented Generation (RAG).

7.3. Transition to Advanced RAG-Based Recommendations

In the next part of the BoxRAG series, we'll elevate our recommendation system by integrating Retrieval-Augmented Generation (RAG) with Mistral 7B. Our implementation leverages the 'fast' pipeline configuration with sentence-transformers/all-MiniLM-L6-v2 for embeddings, FAISS for efficient retrieval, and the BasicRAG architecture. In upcoming configurations, we plan to explore two other RAG setups.

7.4. What's Next in the BoxRAG Series?

Stay tuned for Blog Part 2, where we'll focus on:

- Building the frontend and backend pipeline using React and Flask.
 - Implementing advanced RAG architectures and vector stores.
 - Enhancing recommendation quality with user-focused prompts and contextual data.
-

Final Thoughts

All code and notebooks are available in the [BoxOfficeRAG GitHub Repository](#). The modular codebase is designed for easy experimentation and scaling.

Our future RAG configurations with Mistral 7B will include:

- **Balanced Configuration:** Using 'all-mpnet-base-v2' embeddings with HNSWStore and ReRankingRAG for optimal performance-to-quality ratio.
- **Quality Configuration:** Employing 'intfloat/e5-large' embeddings and ChromaStore with ContextualCompressionRAG for maximum response quality.

Stay tuned for our next post on implementing these configurations and creating a seamless user experience. Check out the repository for all resources, and feel free to contribute!

Happy coding!

Stop Managing Prompts Manually: **prompt-manager** Automates It for AI Developers

“In a world where AI applications built on LLMs are only as powerful as their prompts, mastering the art of prompt management and refinement has become essential for success.”

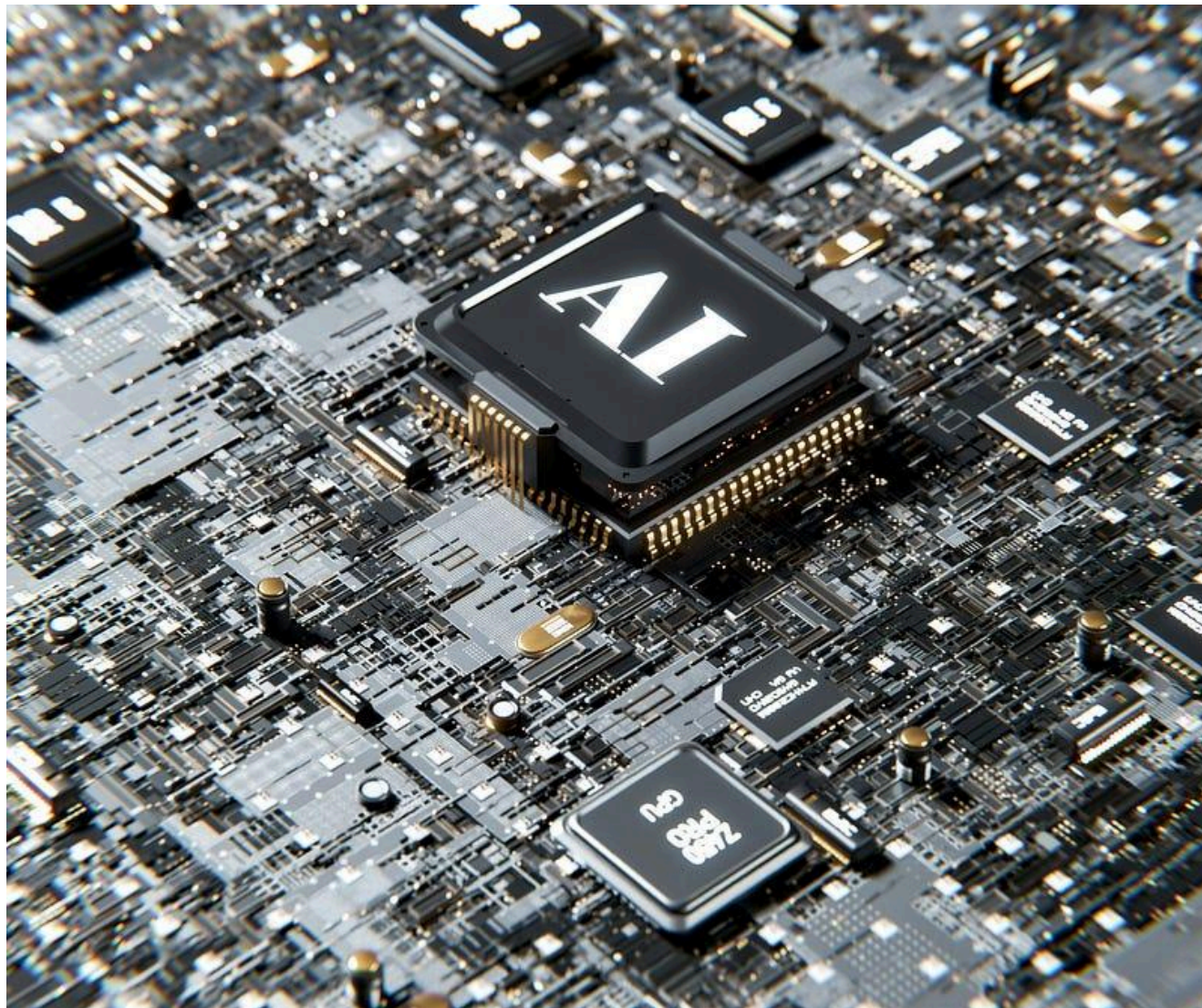


Photo by [Igor Omilaev](#) on [Unsplash](#)

Developer’s Insight: This blog introduces prompt-manager, a library crafted to address the challenges of managing and versioning prompts efficiently. It provides actionable strategies and tools to streamline workflows, empowering developers and entrepreneurs alike in their AI projects.

The Origin of Prompt-Manager

Building my resume-automation project, a Flask-based application that generates customized resumes by processing CVs and job descriptions, I encountered a significant challenge: managing and versioning multiple prompts. With 3–4 key prompts in constant iteration, tracking changes and ensuring consistency became cumbersome. I am currently testing and improving this project while adding additional tools such as cover letter generation, trackline customization, and interview preparation to make it a comprehensive application.

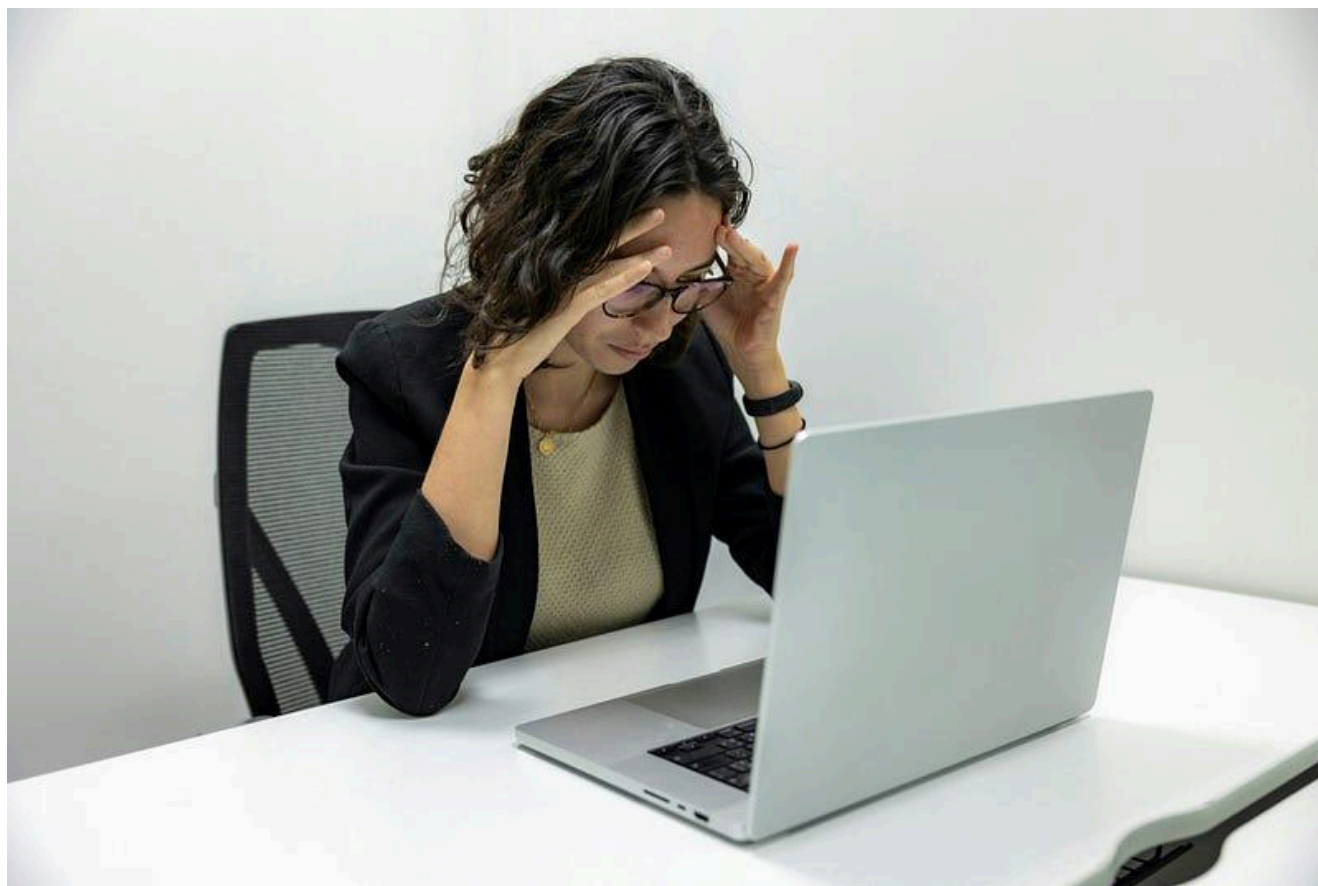


Photo by [Resume Genius](#) on [Unsplash](#)

Why Prompt-Manager? This frustration led to the creation of prompt-manager, a library that made managing and versioning prompts effortless. By implementing this tool, I significantly reduced the time spent juggling prompts, allowing me to focus on improving the resume-automation application itself. Prompt-manager streamlined my workflow, ensuring version control and organization, even as prompt complexity grew.

Key Features of Prompt-Manager

A. Simplified Prompt Management

The library provides a structured way to define, update, and retrieve prompts with ease.

Code Example:

```
import prompt_manager
```

```
manager = prompt_manager.PromptManager("prompts")
```

```
# Adding version 1.0.0 of the prompt
```

```
manager.add_prompt_version(  
    "welcome_message",  
    "1.0.0",  
    "Hello, {name}! Welcome to our platform.",  
    ["name"],  
    "Basic welcome message"  
)
```

```
# Adding version 1.1.0 of the prompt
```

```
manager.add_prompt_version(  
    "welcome_message",  
    "1.1.0",
```

```

    "Hi, {name}! We're glad you're here. Start exploring {platform}!",
    ["name", "platform"],
    "Enhanced welcome message"
)

# Fetching the latest version (default behavior)

print(manager.get_prompt("welcome_message", {"name": "Alice", "platform":
"PromptManager"}))

# Fetching a specific version

print(manager.get_prompt("welcome_message", {"name": "Alice"},
version="1.0.0"))

```

Expected Output

Hi, Alice! We're glad you're here. Start exploring PromptManager!

Hello, Alice! Welcome to our platform.

B. Built-In Version Control

Every prompt iteration is tracked, ensuring no version is lost and changes are transparent.

Code Example:

```

# List available prompts with version control

print("Available Prompts:")

for prompt in manager.list_prompts():

```

```
print(f"Name: {prompt['name']}")  
  
print(f"Current Version: {prompt['current_version']}")  
  
print(f"Versions: {' '.join(prompt['versions'])}")  
  
print()
```

Expected Output

Available Prompts:

Name: resume_summary

Current Version: 1.0.0

Versions: 1.0.0

Name: welcome_message

Current Version: 1.1.0

Versions: 1.0.0, 1.1.0

C. Scalability for Complex Projects

Whether you have 3 prompts or 300, prompt-manager scales seamlessly to meet your needs.

Approach: With prompt-manager, you can group prompts into projects, add metadata, and maintain consistency across multiple versions.

Code Example

```
# Initialize managers for different projects
```

```
project1_manager = prompt_manager.PromptManager("project1_prompts")
```

```
project2_manager = prompt_manager.PromptManager("project2_prompts")
```


Add a prompt to Project 1

```
project1_manager.add_prompt_version(  
    "welcome_message",  
    "1.0.0",  
    "Welcome to Project 1, {name}!",  
    ["name"],  
    "Welcome prompt for Project 1"  
)
```

Add a prompt to Project 2

```
project2_manager.add_prompt_version(  
    "welcome_message",  
    "1.0.0",  
    "Welcome to Project 2, {name}!",  
    ["name"],  
    "Welcome prompt for Project 2"  
)
```

Fetch and print prompts from both projects

```
print("Project 1 Prompt:")  
  
print(project1_manager.get_prompt("welcome_message", {"name": "Alice"}))  
  
print("\nProject 2 Prompt:")  
  
print(project2_manager.get_prompt("welcome_message", {"name": "Bob"}))
```

Expected Output

Project 1 Prompt:

Welcome to Project 1, Alice!

Project 2 Prompt:

Welcome to Project 2, Bob!

How Prompt-Manager Enhances Your Workflow

Overview: Prompt-manager saves you from manually managing prompts, letting you focus on your core application. It ensures:

- **Consistency:** Every prompt change is tracked and reversible.
- **Efficiency:** Reduced time spent searching for older versions or tweaking prompts.
- **Scalability:** Ideal for applications with multiple prompts across projects.

Real-World Impact: Using prompt-manager in my resume-automation project allowed me to shift my focus from the tedious task of managing prompts to improving the actual functionality of the application. It significantly reduced the hours spent on manual work, ensuring the tool's reliability and efficiency. Tasks such as analyzing job descriptions or creating resume summaries became faster, more accurate, and seamlessly integrated into my workflow, enabling me to innovate and enhance other aspects of the project.

Conclusion

Summary: Prompt-manager started as a solution to a personal challenge and has grown into a tool that simplifies prompt management and version control. While this is just the first version, it's already proving to save time and effort, paving the way for more robust applications.

Call to Action: Explore the [GitHub repository](#) to understand the implementation and contribute, or download it from [PyPI](#) to integrate it into your projects.

I would love to hear your thoughts on prompt-manager! While I developed this library on my own, I'm always looking for opportunities to collaborate and improve. Your feedback and suggestions are invaluable in shaping its future. If you have ideas for features or enhancements, please share them! The best way to reach me is through my [LinkedIn profile](#). Thank you for your time and support!

Build and Deploy Your Deep Learning Model in MERN Stack

This article will show you how to create a Deep Learning web app using MERN Stack without storing your AI model on the cloud.



Photo by [DeepMind](#) on [Unsplash](#)

Outline

- 📌 Overview
- 📌 Training an Image Classifier Model Using TensorFlow
- 📌 Converting the model to Tensorflow.JS

Website

Conclusion

Overview

As an AI developer, I wanted to create models and use them in the real world by deploying them on websites to see how they performed and to get user feedback. However, due to large model files, doing so would increase both the computational cost and the amount of time it would take to predict or classify an object. Although TensorFlow offers substantial speed advantages, it cannot run native code in a web browser, this utilized TensorFlow.js, a component of the TensorFlow ecosystem that differs from TensorFlow but offers almost identical capabilities.

Training an Image Classifier Model Using TensorFlow

Dataset



Photo by [Erik Mclean](#) on [Unsplash](#)

To Train this model we will be using the [Pokemon Dataset](#) from Kaggle which has images of 809 different pokemons like bulbasaur, ivysaur, venusaur, charmander, charmeleon etc



From Left: Articuno, Audino, Aurorus



From Left: Axew, Azelf, Azuril

👉 Mobilenet-SSDv2

Here, we'll be utilizing Mobilenet-SSDv2, which was built on Mobilenet-v2. The real-time object detector can be used in embedded systems that have restricted computational power. to study more you could refer this [article](#).

👉 Training

I was able to get 98.8% accuracy on the train dataset after training. however, I was able to reach an accuracy of 99% and a test dataset accuracy of about 80% after performing Fine-Tuning and Re-Training.

Converting the model to TensorflowJS

Install Tensorflowjs using the below command on your Command prompt or Terminal

```
pip install tensorflowjs
```

After your model has been trained run this command to convert your model into JSON format which can be used with Tensorflow.js to predict object or classify images.

```
import tensorflow
```

```
import tensorflowjs as tfjs
```

```
from tensorflow import keras
```

```
your_model__path=""
```

```
model=keras.models.load_model(f"{your_model_path}")
```

```
tfjs.converters.save_keras_model(model,'model_js')
```

it will create a folder called `model_js`, in which the converter generates `model1.json` and `group1-shard*of*`, the `model1.json` has a dataflow graph and weight manifest file whereas `group1-shard*of*` has a collection of binary weight files.

Website

👉 Express.js and Node.js

We will be having Node server running on our backend and it will be serving our file using Express.js using the below code.

```
app.use(
```

```
  "/api/classify",
```

```
  express.static(path.join(__dirname, "/models/model_js/model.json"))
```

```
);
```

here when our front end makes a call to `api/classify` it returns a JSON file to it.

👉 IndexedDB

the content inside JSON file will be stored in IndexedDB which is a database built into a browser that is far more powerful than local Storage.

- Stores almost any type of value using keys, with multiple key types.
- Transactions are supported for dependability.
- Indexes and key range queries are supported.
- Can store significantly more data than localStorage.

To read more about it read the following [article](#)

👉Tensorflow.js

TensorFlow.js is a machine learning library for JavaScript, it makes machine learning capabilities available on a web browser and Node.js for server-side and back-end development. If you're a Javascript developer wants to make your hands dirty in Machine Learning, TensorFlow.js is a great option to start with.

Enter the below command in your terminal to install it.

```
npm i @tensorflow/tfjs
```

Before classifying an image, the function below reads it and performs preprocessing.

```
useEffect(() => {  
  
  async function predict() {  
  
    if (imageLoaded && file) {  
  
      const imageElement = document.createElement("img");  
  
      imageElement.src = file;  
  
  
      imageElement.onload = async () => {  
  
        const tensor = tf.browser  
  
          .fromPixels(imageElement)  
  
          .resizeNearestNeighbor([120, 120])  
  
          .toFloat()  
  
          .sub(127)  
  
          .div(127)
```



```
.expandDims();
```

```
const y_pred = await model.predict(tensor).data();
```

```
const topNPredNames = getTopNPredPokeObj(y_pred, 3);
```

```
setPrediction(topNPredNames);
```

```
console.log("-----");
```

```
setProcessing(false);
```

```
setImageLoaded(false);
```

```
return topNPredNames;
```

```
};
```

```
}
```

```
}
```

```
predict();
```

```
}, [imageLoaded, model, file]);
```

👉 ReactJS

For this article, I created a simple frontend for classifying pokemon when a user uploads an image to it.

here is a short code snippet

```
<form className="Form">
```

```
<label htmlFor="upload-image">
```

```
  <h1> Upload image </h1>
```

```
</label>
```

```
<br></br>
```

```
<input
```

```
  id="image-selector"
```

```
  type="file"
```

```
  name="upload-image"
```

```
  accept="image/*"
```

```
  className="File-selector"
```

```
  onChange={handleImgUpload}
```

```
  disabled={!model || processing}
```

```
/>
```

```
</form>
```

```
<div className="Upload">
```

```
  <img
```

```
    onLoad={() => {
```

```
      setImageLoaded(true);
```

```
    }}
```

```
    alt=""
```

```
    src={file}
```

/>

</div>

<div className="Results">

{processing ? (

<p>Loading ...</p>

) : topNPredNames !== null ? (

<div className="row">

{Object.entries(topNPredNames).map(([key, value]) => (

<div className="col-md-4">

<h2>{key}</h2> <h2>{value}</h2>

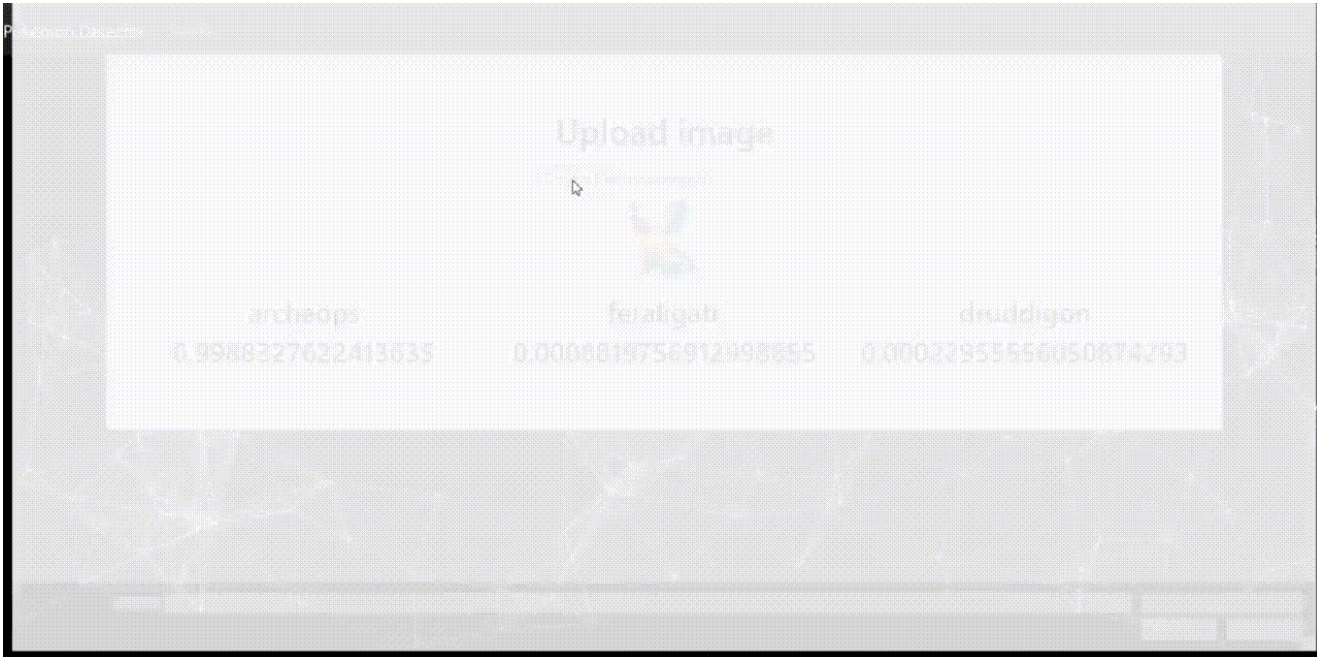
</div>

)))

</div>

) : null}

</div>



Demo

Conclusion

In my opinion, a front-end-powered TF model can be cost-effective because most front-end hosting is less expensive than a server-powered or cloud-powered TensorFlow solution; additionally, it can protect user privacy. If you want to play around with the dataset, you can get it from [Kaggle](#) and the code from [Github](#). follow me on [Medium](#). I'm always open to suggestions and constructive criticism, and my Twitter handle is [@FaseehAhmed2606](#).

Lastly,

Thank you for reading.

Hope this article helps you !

Build a Real-Time Inventory Management System in 6 Hours Using Streamlit and YOLOv5

A brief article explaining how to efficiently train the object detection algorithm YOLOv5 on your own custom dataset and use it to build your [web app](#) that can be used in real-time.

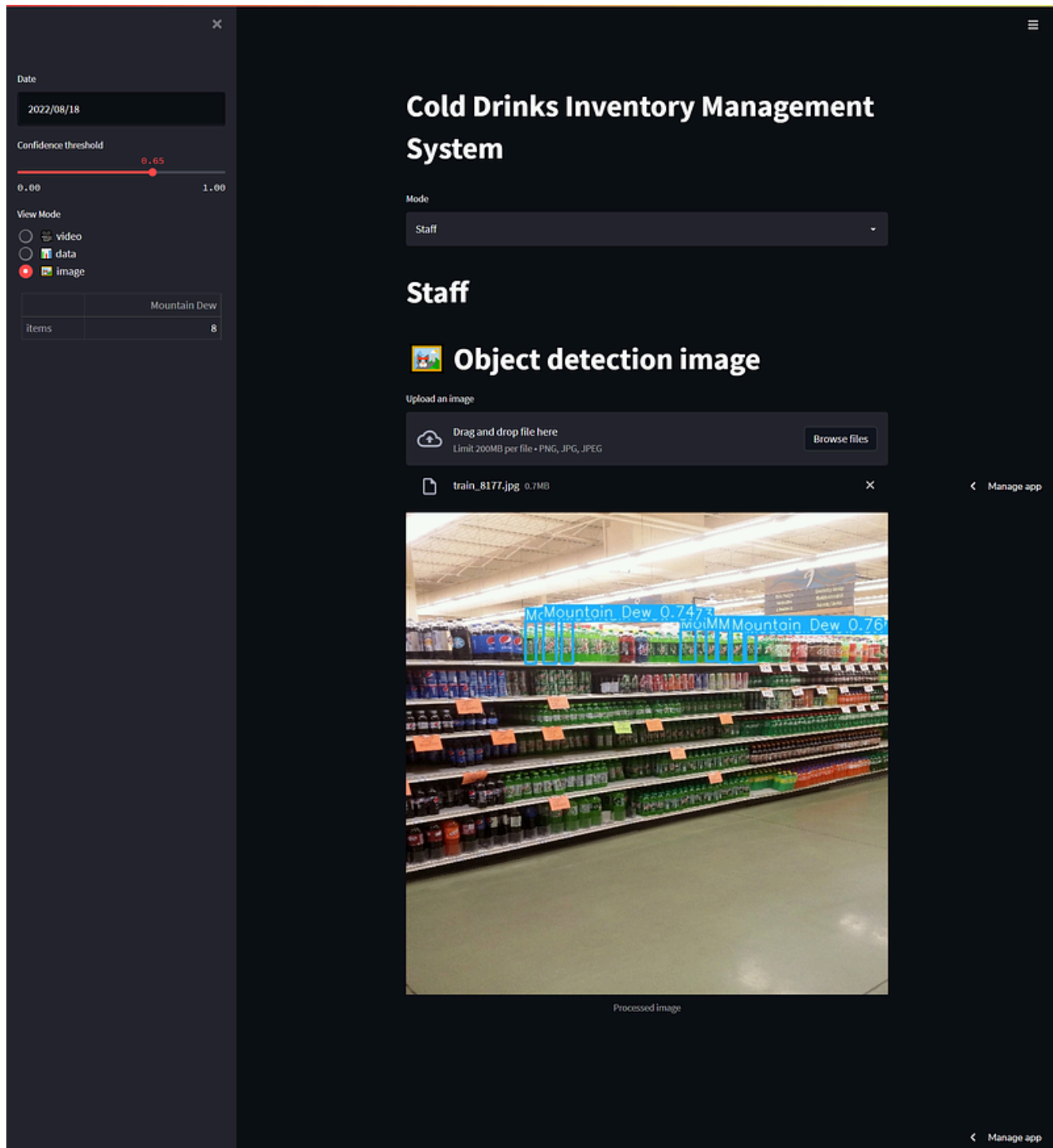


Photo by [Tobias Tullius](#) on [Unsplash](#)

The necessity to share data scientists' discoveries has increased, due to Python's escalating use as their preferred programming language, a cross-platform approach with adequate interaction is the use of web apps. However, typical programming demands enough expertise among Python users with somewhat complex web frameworks, like Flask and Django. Even if learning these frameworks is not particularly difficult for any moderate Python users, it may be difficult for data scientists to display their data science work.

Streamlit

The Streamlit framework, therefore, gives us data scientists a better choice today. It quickly and easily transforms straight forward Python scripts into usable, interactive, and shareable web apps. Due to its focus on exhibiting data science solutions, it comes with native support for data models and graphs which are created using popular data science libraries like pandas and matplotlib. Therefore, if you are already familiar with Python, learning streamlit will be relatively easy.



Performing Detection on an Image

Objective

I want to take you guys a step further in this article by demonstrating how to publish your web app so that others can view your AI models in Real Time. What's more, I'll demonstrate how to build an Inventory

Management System which manages cold beverages in real-time. For our project we will be using Streamlit to build our web app and SQLite as a database.

I'll be using the [SKU-110K dataset](#) to create my [custom dataset](#) to identify cold beverages like Coca-Cola, Sprite, Pepsi, Mountain Dew, 7UP, and Fanta. There are many low-quality photographs in SKU-110K, so had to filter them out, select the best ones, and annotate them with the [labelimg](#) library. which helps you annotate images in many storage formats, including YOLO and CreateML, and PASCAL VOC format. It is a helpful tool for annotating data for object detection.

Training

After the annotations are done we will train them using yolov5 which is an Object Detection and Classification model, Now we will briefly see the architecture of YOLOv5.

As YOLOv5 is a single-stage object detector, it has three important parts like any other single-stage object detector.

Model Backbone

The main function of Model Backbone is to identify key features in an input image.

To extract valuable, useful characteristics from an input image in YOLO v5, the CSPDarkNet (CSP—Cross Stage Partial Networks) is employed as the backbone. It uses a CSPNet technique that divides the base layer's feature map into two sections and then merges them using a cross-stage hierarchy.

Darknet is fast and highly accurate which are the key requirements for the YOLO family.

Model Neck

PANet (Path Aggregation Network) is used as a neck in YOLO v5 to get feature pyramids. Feature pyramids are a basic component in recognition systems for detecting objects at different scales.

The model's neck role is to collect features from different stages. The neck is composed of several top-down paths and several bottom-up paths. PANet uses features from all the layers and decides which ones are useful.

Model Head

It uses anchor boxes to construct final output vectors with class probabilities, object-ness scores, and bounding boxes.

The head is responsible for :

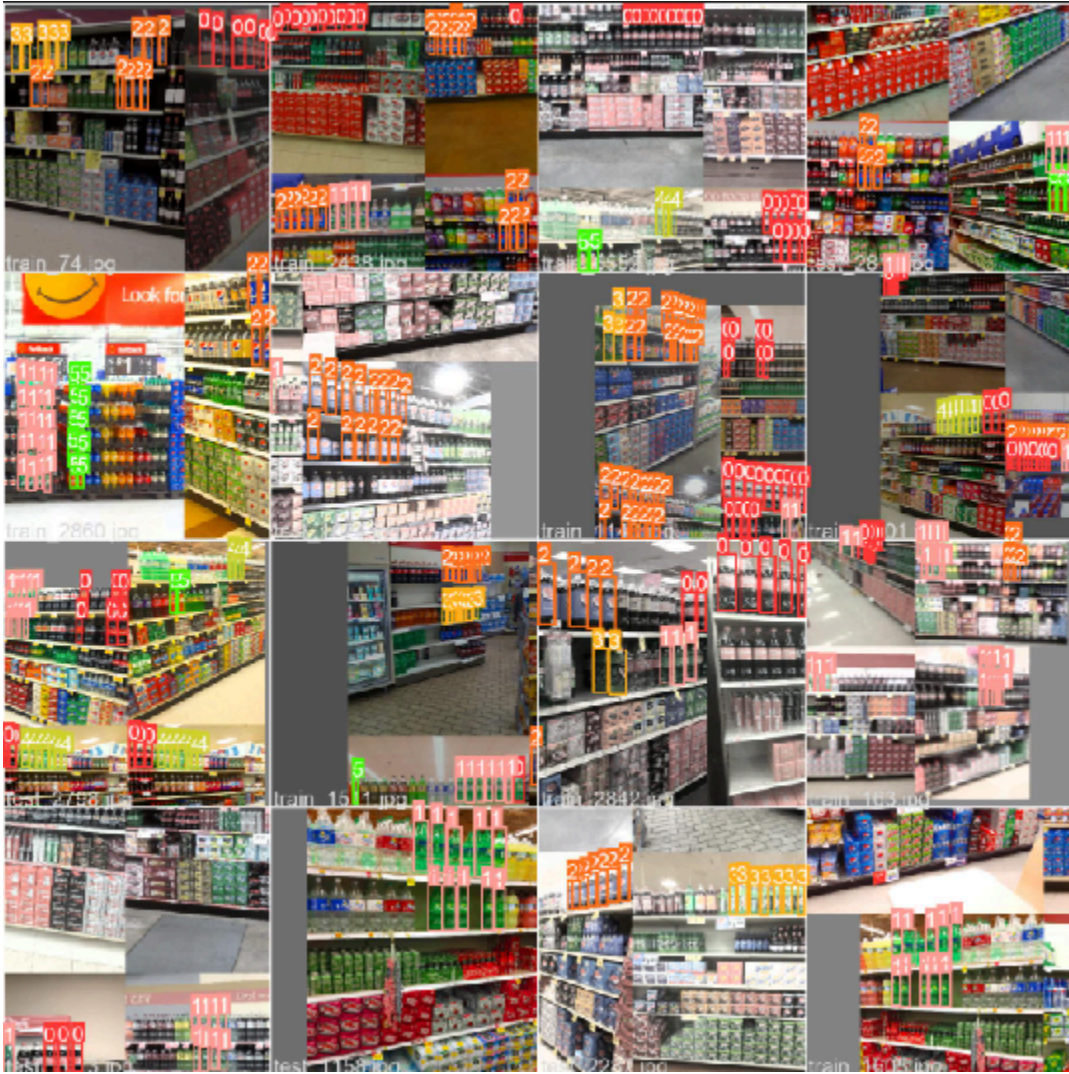
the objectness score (the probability an object is within the anchor)

the class (a number i.e. 1 which corresponds to a class e.g. a person)

the class confidence (the chance that the above class is correct)

the bounding box coordinates i.e. [x, y, w, h] [120, 300, 200, 400]

The input is first passed to CSPDarknet for feature extraction and then fed to PANet for feature fusion (Feature Pyramid). Finally, Yolo Layer outputs detection results (class, score, location, size).



Training Results

Workflow

Cold Drinks Inventory Management System

Mode

None

None

Staff

Admin



Cold Drinks Inventory Management System

Mode

Staff

Staff



Object detection video

Date

2022/10/15

Confidence threshold

0.65

0.00

1.00

View Mode

☒ video

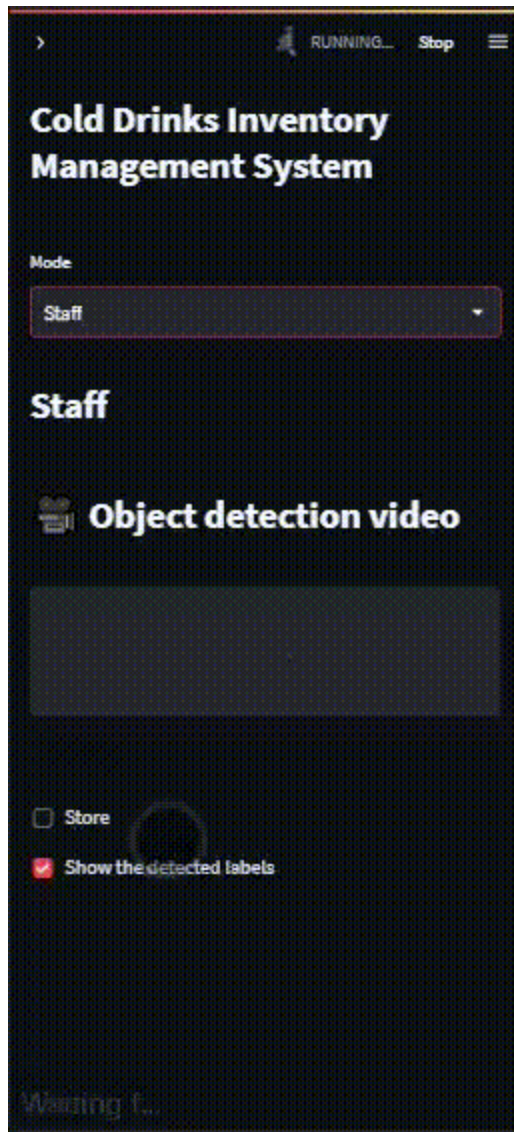
☐ data

☐ image

Landing Page

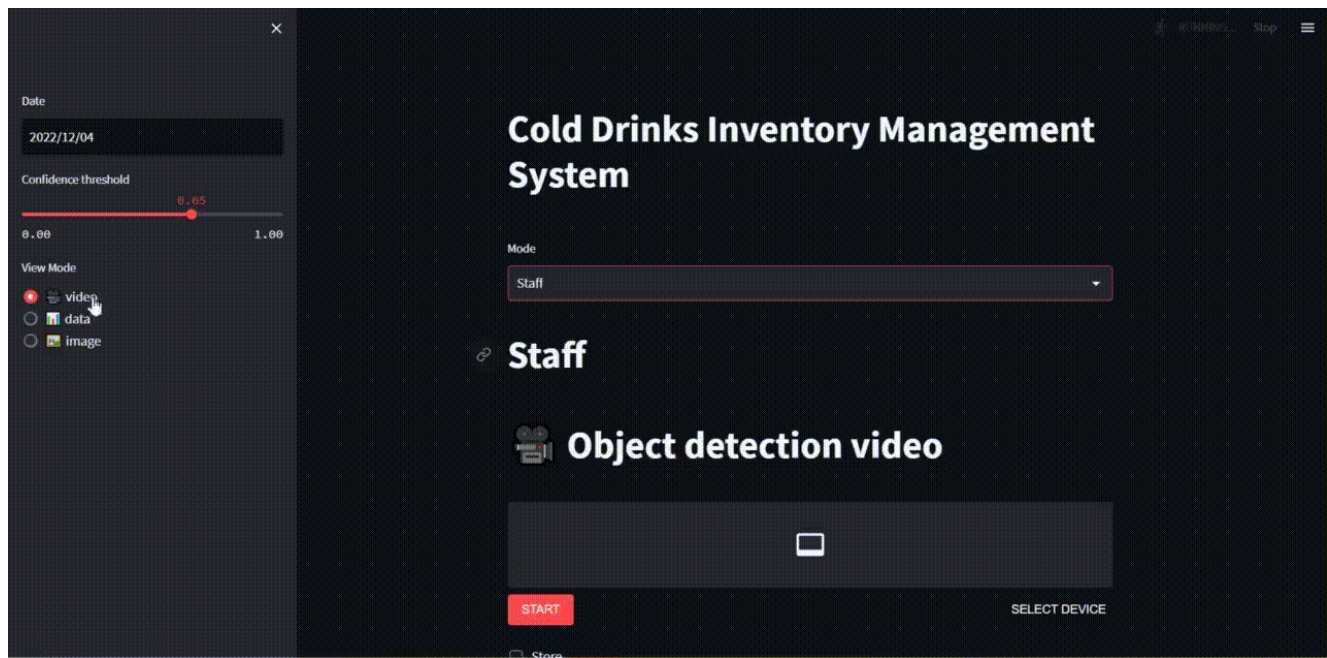
At Landing Page, you have two options either you can go to Staff or Admin.

Utilizing [streamlit_webrtc](#), which facilitates real-time communication, we will be able to detect images in real time by executing detections on the frames captured from the real-time video in the backend.



Real-Time Detections Using Mobile

You could also use drag-and-drop or image upload to carry out detections on images, as seen below.



Performing Classifications using Images

Using three fields—Date, Beverage Name, and Count—we are storing results on our sqlite database, which aids the end user in maintaining inventory and verifying quantities day to day. as shown below

15:42

LTE1 63%



ip-apsm2g.streamlitapp.com



5	2022-08-18	Sprite	5
6	2022-08-18	Mountain Dew	8
7	2022-08-18	Pepsi	4
8	2022-08-17	Mountain Dew	8
9	2022-08-18	Mountain Dew	37
10	2022-08-18	Sprite	12
11	2022-08-18	Fanta	1
12	2022-08-18	Mountain Dew	11
13	2022-08-18	Pepsi	5
14	2022-08-18	Sprite	19
15	2022-08-18	Pepsi	38
16	2022-08-18	Pepsi	31
17	2022-08-17	Sprite	2
18	2022-08-17	Pepsi	17
19	2022-08-17	Coca Cola	25

	count
Coca Cola	34
Fanta	2
Mountain Dew	103
Pepsi	95

Conclusion

In this post, I talked about how to build an Inventory Management System using YOLOv5 object detection model on a Custom Dataset.

If you would like to experiment with the custom dataset yourself, you can download the annotated data on [Kaggle](#) and the code at [Github](#) the web app is deployed [here](#).

Thank you for reading. I'll be producing additional posts for beginners in the future as well. To learn more about them, follow me on [Medium](#). I'm always open to suggestions and constructive criticism, and my Twitter handle is @FaseehAhmed2606.

RIT RE

BJT

Project Information Header

- **Project:** Budget Justification Data Transfer Automation
- **Company:** Rochester Institute of Technology (Office of Vice President of Research)
- **Role:** Machine Learning Engineer Co-op
- **Duration:** June 2025 - Present

Summary STAR (1.5-2.5 minutes speaking time)

S: RIT's Pre-Award team had a lot of redundant and manual work - where each staff member works with a professor applies for on average 10 proposals in a month by manually transferring budget data from Excel files into Word templates, which was creating bottlenecks and causing 5-8 calculation errors monthly that could jeopardize proposal compliance.

T: I needed to design and build a comprehensive automation solution that could eliminate all the manual data transfer, ensure 100% calculation accuracy, and provide a user-friendly interface for non-technical staff while maintaining strict federal compliance requirements.

A: I built a Streamlit web portal with robust Excel parsing using pandas/openpyxl and automated Word document generation with python-docx. I worked closely with the Pre-Award Director and stakeholders across 5 colleges to handle diverse Excel formats and ensure seamless integration with existing workflows.

R: Achieved a 60% reduction in manual processing time, eliminated calculation errors entirely (down from 5-8 monthly), saved \$50,000 annually in administrative costs, and successfully automated all 800+ annual proposals while maintaining federal compliance - allowing staff to focus on strategic review instead of data entry.

Detailed STAR

S - SITUATION Situation Summary: Rochester Institute of Technology's Pre-Award team was struggling with managing 800+ annual federal research proposals requiring manual data entry from Excel to Word templates, creating bottlenecks during peak submission periods and causing calculation errors that could jeopardize proposal compliance.

Detailed Context:

- **Company context & business problem:** RIT's Pre-Award team managed 800+ annual federal research proposals requiring detailed budget justifications and corresponding narratives. Each proposal demanded manual data entry from Excel into Word document templates, creating bottlenecks during peak submission periods. The team of 2 administrators and 4 faculty researchers struggled with year-end budget transfers, often working overtime to meet federal deadlines. Manual processes led to formatting inconsistencies and calculation errors that could jeopardize proposal compliance.
- **Team structure & your role:** Office of Vice President of Research team including Pre-Award Director, 2 senior administrators, 4 faculty researchers, and budget officers from 5 different colleges. As Machine Learning Engineer Co-op, I was responsible for designing and implementing the complete automation solution to eliminate manual data transfer processes.
- **Timeline & constraints:** Ongoing project from June 2025 requiring immediate impact during peak submission periods. Critical requirement for 100% accuracy in financial calculations and federal compliance. Need for user-friendly interface for non-technical staff members.
- **Scale/scope of the challenge:** 800+ annual proposals, diverse Excel formats, federal compliance requirements, non-technical user base, peak period bottlenecks

T - TASK/CHALLENGE Task Summary: I was tasked with designing and implementing a comprehensive automation solution that could eliminate manual budget data transfer processes, ensure 100% calculation accuracy, and provide a user-friendly interface for non-technical staff while maintaining federal proposal compliance.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Design an automated solution to eliminate manual budget data transfer
 - Ensure 100% accuracy in financial calculations and formatting
 - Create user-friendly interface for non-technical staff
 - Maintain compliance with federal proposal requirements
 - Build robust system handling diverse Excel formats and data validation
 - Implement automated Word document generation with consistent formatting
- **Technical challenges faced:**
 - Building intuitive Streamlit interface for non-technical users
 - Handling diverse Excel formats and data validation
 - Automating Word document generation with consistent formatting
 - Real-time file processing and error handling in web environment
 - Ensuring data security during file upload/download processes
- **Business/stakeholder challenges:**
 - Minimizing disruption to existing workflows during implementation
 - Training staff on new automation tools
 - Ensuring backup procedures for system failures during critical deadlines
 - Managing change management across multiple colleges and departments

- **Success criteria and constraints:** 100% calculation accuracy, 60%+ time reduction, federal compliance, user-friendly design, 800+ proposal capacity

A - ACTION (Detailed Breakdown) Action Summary: I built a comprehensive Streamlit web portal with robust Excel parsing and automated Word generation capabilities, working closely with stakeholders across multiple colleges to ensure seamless integration with existing workflows while achieving complete automation of the budget transfer process.

Technical Stack Used:

- **Python:** Core automation logic and data processing (chosen for robust data manipulation and document generation capabilities)
- **Streamlit:** Interactive web portal interface (selected for rapid development and user-friendly design for non-technical staff)
- **pandas/openpyxl:** Excel file reading and data manipulation (comprehensive support for diverse Excel formats and calculations)
- **python-docx:** Automated Word document generation (precise formatting control and template integration)
- **Word Mail Merge templates:** Consistent formatting (ensuring federal compliance and professional presentation)
- **File upload/download handlers:** Seamless user experience (secure temporary storage and automatic cleanup)

Collaboration Details:

- **Worked with:** Pre-Award Director (requirements), 2 senior administrators (workflow validation), faculty researchers (budget requirements), budget officers from 5 colleges (diverse use cases)
- **How decisions were made:** Led requirements gathering sessions with 15+ end users to understand diverse budget requirements and workflow constraints
- **Communication challenges/successes:** Regular stakeholder meetings ensuring broad adoption across different colleges and departments
- **Cross-functional coordination:** Partnered with IT Security team for system integration approval while balancing automation needs with security requirements

Leadership Moments:

- **Decisions you drove:** Led requirements gathering sessions with 15+ end users and drove decision to adopt modular architecture for future scalability
- **People you influenced/mentored:** Mentored 1 undergraduate intern on automation scripting techniques and established testing protocols
- **Processes you improved:** Established comprehensive testing protocols involving real proposal data validation and standardized budget formatting
- **Initiative you took beyond assigned tasks:** Created documentation and training materials for knowledge transfer beyond basic system requirements

Problem-Solving Process:

- **Standardization Challenge:** Analyzed 200+ budget templates to design standardized Excel input format accommodating diverse requirements
- **User Experience:** Built Streamlit prototype with file upload/download functionality ensuring intuitive operation for non-technical users
- **Quality Assurance:** Implemented real-time data validation and error messaging in portal preventing calculation errors
- **Trade-off Analysis:** Chose web-based Streamlit portal for accessibility vs. desktop application for advanced features, prioritizing user adoption

R - RESULTS Result Summary: The automation system achieved remarkable efficiency improvements with 60% reduction in processing time, eliminated calculation errors entirely, saved \$50,000 annually in administrative costs, and successfully automated all 800+ annual proposals while maintaining federal compliance standards.

Detailed Impact:

- **Quantified business impact:**
 - **60% reduction** in manual processing time for budget preparation
 - **Zero calculation errors** since implementation (previously 5-8 errors monthly)
 - **\$50,000 annual cost savings** in administrative overtime elimination
 - **800+ proposals** now processed automatically per year with consistent quality
- **Technical achievements:**
 - **Built user-friendly Streamlit web portal** accessible to all staff members regardless of technical background
 - **Created robust Excel parsing system** handling multiple budget formats and diverse college requirements
 - **Developed automated Word document generation** with consistent styling meeting federal compliance standards
 - **Implemented comprehensive error handling** and user feedback systems preventing processing failures
- **Team/process improvements:**
 - **Strategic focus shift:** Staff can now focus on strategic proposal review vs. manual data entry tasks
 - **Standardized budget formatting** across all funding agencies ensuring consistency and compliance
 - **Created comprehensive documentation** and training materials enabling knowledge transfer and system maintenance
 - **Improved deadline management:** Eliminated overtime work during peak submission periods

- **Personal growth/skills gained:**

- **Enterprise automation expertise:** Gained expertise in enterprise automation and change management for research environments
- **Stakeholder communication mastery:** Developed skills in stakeholder communication and requirements gathering across diverse user groups
- **Compliance understanding:** Enhanced understanding of federal research compliance requirements and proposal formatting standards
- **User-centric design:** Experience building intuitive systems for non-technical users in high-stakes environments

Assignment

Project Information Header

- **Project:** Automated Workload Management System using Microsoft Planner
- **Company:** Rochester Institute of Technology (Office of Vice President of Research)
- **Role:** Machine Learning Engineer Co-op
- **Duration:** June 2025 - Present

Summary STAR (1.5-2.5 minutes speaking time)

At Rochester Institute of Technology, I was faced with a significant workflow challenge. The Pre-Award team was manually assigning over 200 monthly research proposals to 12 staff members using Trello and subjective decision-making, which led to uneven workload distribution and created bottlenecks during peak periods.

My task was to implement an AI-powered solution that could automate workload balancing and seamlessly integrate with their existing Teams and Office 365 workflows.

I configured Microsoft Planner with the Project Manager Agent to create an intelligent assignment system. It considered team capacity, expertise, and sponsor relationships to make smart assignments. I also built comprehensive dashboards for real-time visibility and led the change management process by working closely with the team's Director and all 12 staff members.

The results were exceptional. We achieved a **33% improvement in proposal turnaround time**, a **25% reduction in workload imbalance**, and a **15% increase in staff satisfaction scores**. The system now automatically handles all 200+ monthly assignments, eliminating manual spreadsheet tracking and creating a transparent workflow for the entire team.

Detailed STAR

S - SITUATION Situation Summary: Rochester Institute of Technology's Pre-Award team was struggling with manual assignment processes for 200+ monthly research proposals, relying on subjective decision-making that created uneven workload distribution, bottlenecks during peak periods, and lacked real-time visibility into team capacity.

Detailed Context:

- **Company context & business problem:** The Pre-Award team relied on Trello and manual processes for assigning 200+ monthly research proposals to 12 staff members based on workload, sponsor relationships, and institutional familiarity. The existing system lacked automated workload balancing and real-time visibility into team capacity. Decision-making was often subjective and reactive, leading to uneven distribution, bottlenecks during peak submission periods, and missed opportunities for workload optimization.
- **Team structure & your role:** Office of Vice President of Research team including Pre-Award Director, 3 senior assignment coordinators, and 12 staff members handling proposal assignments.

As Machine Learning Engineer Co-op, I was responsible for implementing and configuring the complete automated workload management system.

- **Timeline & constraints:** Ongoing project from June 2025 requiring immediate impact during peak submission periods. Need for seamless integration with existing Teams/Office 365 infrastructure. Critical requirement for buy-in from senior staff accustomed to manual control.
- **Scale/scope of the challenge:** 200+ monthly assignments, 12 team members, existing workflow integration, change management requirements

T - TASK/CHALLENGE Task Summary: I was tasked with implementing Microsoft Planner with AI-powered Project Manager Agent to create automated workload balancing, integrate with existing systems, and design comprehensive reporting while managing change across the entire team.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Implement Microsoft Planner with Project Manager Agent for automated assignment
 - Configure AI-powered workload balancing across team members
 - Integrate with existing Teams/Office 365 workflow systems
 - Design automated reporting and status tracking system
 - Create real-time visibility dashboards for team capacity and proposal status
 - Establish change management and training programs
- **Technical challenges faced:**
 - Configuring Project Manager Agent for optimal task assignment logic
 - Setting up automated workload distribution based on capacity and expertise
 - Integrating Planner with existing document management workflows
 - Creating real-time visibility into team workload and proposal status
- **Business/stakeholder challenges:**
 - Ensuring seamless transition without workflow disruption
 - Gaining buy-in from senior staff accustomed to manual assignment control
 - Training team on new AI-powered assignment and tracking system
 - Managing resistance to automated decision-making processes
- **Success criteria and constraints:** 30%+ efficiency improvement, workload balance optimization, Teams integration, user adoption, transparent visibility

A - ACTION (Detailed Breakdown) Action Summary: I configured Microsoft Planner with Project Manager Agent for intelligent assignment logic, built comprehensive Power BI dashboards for workload visibility, and executed thorough change management ensuring successful adoption across all 12 team members.

Technical Stack Used:

- **Microsoft Planner with Project Manager Agent:** AI-powered task assignment (chosen for intelligent workload balancing and expertise matching)
- **Teams integration:** Workflow automation and notifications (seamless integration with existing collaboration infrastructure)
- **Power BI:** Workload visualization and reporting dashboards (real-time capacity and assignment status visibility)
- **SharePoint integration:** Document access and proposal tracking (leveraging existing document management systems)
- **Power Automate:** Automated workflow triggers and status updates (streamlined notification and process automation)

Collaboration Details:

- **Worked with:** Pre-Award Director (strategic requirements), 3 senior assignment coordinators (workflow validation), IT team (integration setup), Microsoft technical support (agent configuration)
- **How decisions were made:** Led evaluation of Planner vs. alternative platforms while coordinating with senior staff on assignment criteria and logic
- **Communication challenges/successes:** Regular training sessions and feedback collection from all 12 team members ensuring smooth adoption
- **Cross-functional coordination:** Balanced AI automation capabilities with existing manual preferences and institutional knowledge

Leadership Moments:

- **Decisions you drove:** Led evaluation of Planner vs. alternative platforms (Asana, Monday.com) and drove adoption of AI-powered assignment logic
- **People you influenced/mentored:** Established transparent workload visibility changing team culture toward data-driven assignment decisions
- **Processes you improved:** Created comprehensive change management and training plan enabling smooth transition to automated systems
- **Initiative you took beyond assigned tasks:** Established feedback mechanisms for continuous AI agent optimization beyond basic implementation

Problem-Solving Process:

- **Historical Analysis:** Analyzed 18 months of historical assignment data to configure optimal parameters for AI agent decision-making
- **AI Configuration:** Configured Project Manager Agent with workload balancing and expertise matching achieving optimal task distribution
- **Validation Testing:** Tested automated assignment logic against manual assignments over 4-week pilot ensuring accuracy and acceptance
- **Continuous Improvement:** Established feedback mechanisms for continuous AI agent optimization based on user experience and outcomes

R - RESULTS Result Summary: The automated workload management system achieved significant operational improvements with 33% faster turnaround times, 25% better workload balance, 15% higher staff satisfaction, and successfully automated 200+ monthly assignments while creating transparent visibility that eliminated manual tracking.

Detailed Impact:

- **Quantified business impact:**

- **33% improvement** in proposal turnaround time through optimized assignment logic
- **25% reduction** in workload imbalance across team members enabling better capacity utilization
- **15% increase** in staff satisfaction scores (internal survey) demonstrating successful change management
- **200+ monthly assignments** now handled automatically by AI agent with consistent quality

- **Technical achievements:**

- **Successfully configured Project Manager Agent** achieving optimal task distribution based on capacity and expertise
- **Created real-time workload dashboard** showing team capacity and assignment status for management oversight
- **Built automated status reporting system** for management oversight and performance tracking
- **Established Teams integration** for seamless workflow and notification management eliminating manual coordination

- **Team/process improvements:**

- **Eliminated manual spreadsheet tracking** and email coordination reducing administrative overhead
- **Created transparent assignment visibility** for all team members improving accountability and fairness
- **Established automated feedback collection** for continuous improvement and AI agent optimization
- **Improved change management processes** demonstrating successful enterprise AI adoption methodology

- **Personal growth/skills gained:**

- **AI agent configuration expertise:** Advanced skills in AI agent configuration and workflow automation using Microsoft enterprise tools
- **Enterprise collaboration mastery:** Gained expertise with Microsoft enterprise collaboration tools and integrations in production environments
- **Change management skills:** Enhanced understanding of change management in automated system adoption and user resistance mitigation

- **Stakeholder alignment:** Experience balancing automation capabilities with existing team preferences and institutional knowledge

RIT Proposal Init

Project Information Header

- **Project:** RIT Proposal Initial Eligibility Agent
- **Company:** Rochester Institute of Technology (Office of Vice President of Research)
- **Role:** Machine Learning Engineer Co-op
- **Duration:** June 2025 - Present
- **Institutional eligibility**
Whether RIT was an eligible institution for the solicitation.

Institutional eligibility

Whether RIT was an eligible institution for the solicitation.

Principal Investigator eligibility

Checks for PI restrictions such as citizenship, appointment type, or career stage.

Submission limit violations

Flags proposals exceeding per-institution or per-PI submission caps.

Cost sharing requirements

Identifies mandatory or prohibited cost-sharing clauses.

Indirect cost rate restrictions

Detects caps or deviations from RIT's negotiated F&A rates.

Budget cap violations

Flags proposals exceeding maximum allowed funding amounts.

Project duration limits

Identifies proposals exceeding allowed project timelines.

Agency-specific formatting requirements

Page limits, font size, margin rules, or section structure violations.

Restricted or unallowable cost items

Travel, equipment, salary, or subcontract costs not permitted by the agency.

Human subjects or IRB requirements

Flags solicitation language requiring IRB approval or disclosures.

Export control or foreign involvement disclosures

Identifies clauses related to foreign collaborations or data restrictions.

Legal and contractual review triggers

Detects terms requiring Legal Affairs review before submission.

Summary STAR (1.5-2.5 minutes speaking time)

At Rochester Institute of Technology, I was tasked with solving a critical bottleneck where faculty were submitting hundreds of federal research proposals monthly. Many of these proposals had eligibility issues that were only discovered late in the review process through a time-consuming, manual review of lengthy PDFs, which could result in costly rejections.

My task was to implement an AI-powered solution to automate this process.

I configured Microsoft's Document Processor agent using custom prompt engineering to automatically classify twelve different types of compliance violations. I integrated the system with existing SharePoint and Power Automate workflows for seamless processing, and collaborated closely with compliance specialists and legal affairs to ensure the system's accuracy.

The results were outstanding. We achieved **95% accuracy** in classifying issues, a **70% reduction in review time** for initial eligibility, and an **85% decrease in late-stage proposal rejections**.

Detailed STAR

S - SITUATION Situation Summary: Rochester Institute of Technology's research office was facing significant bottlenecks where faculty submitted 200+ federal research proposals monthly, but many contained eligibility issues only discovered late in the review process through time-consuming manual PDF reviews that were prone to oversight and costly compliance violations.

Detailed Context:

- **Company context & business problem:** Faculty submitted 200+ federal research proposals monthly through NOIS (Notice of Intent to Submit) forms, but many contained eligibility issues discovered late in the review process. Pre-Award staff spent significant time manually reviewing solicitation PDFs to identify compliance problems like institutional eligibility conflicts, submission limits, indirect cost restrictions, and terms requiring legal review. Manual review of solicitation documents was time-consuming and prone to oversight, potentially resulting in rejected proposals or compliance violations.
- **Team structure & your role:** Office of Vice President of Research team including Pre-Award compliance specialists, Legal Affairs officers, and faculty research support. As Machine Learning Engineer Co-op, I was responsible for designing and implementing the complete automated eligibility classification system.
- **Timeline & constraints:** Ongoing project from June 2025 requiring immediate impact on proposal review efficiency. PDF processing limitations (25MB, 50 pages max). Critical requirement for high accuracy to avoid disrupting faculty workflow with false positives.
- **Scale/scope of the challenge:** 200+ monthly proposals, 12 compliance violation categories, integration with existing workflows, enterprise-grade accuracy requirements

T - TASK/CHALLENGE Task Summary: I was tasked with configuring and implementing an automated compliance classification system that could accurately identify eligibility violations in federal research solicitations, integrate with existing workflows, and dramatically reduce manual review time while maintaining high accuracy standards.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Configure Microsoft's Document Processor agent for compliance classification
 - Design custom prompts to categorize different types of eligibility violations
 - Integrate document processing with existing SharePoint workflows
 - Create automated flagging system for Pre-Award staff review
 - Develop classification framework covering 12+ compliance violation categories
 - Establish feedback mechanisms for continuous improvement
- **Technical challenges faced:**
 - Engineering effective classification prompts for diverse compliance violation types
 - Working within PDF processing limitations (25MB, 50 pages max)
 - Configuring agent to distinguish between 10+ compliance violation categories
 - Integrating automated processing with existing document review workflows
- **Business/stakeholder challenges:**
 - Ensuring high classification accuracy to avoid false positives disrupting faculty work
 - Gaining compliance team trust in automated classification decisions
 - Balancing comprehensive coverage with processing speed requirements
 - Managing change management with established manual review processes
- **Success criteria and constraints:** 95%+ accuracy, workflow integration, 12 violation categories, PDF size limitations, stakeholder adoption

A - ACTION (Detailed Breakdown) Action Summary: I configured Microsoft's Document Processor agent with custom prompt engineering, collaborated extensively with compliance specialists to define classification taxonomies, and integrated the system with SharePoint workflows to achieve seamless automated processing with high accuracy.

Technical Stack Used:

- **Microsoft Copilot Studio Document Processor:** Managed agent template (chosen for enterprise integration and document processing capabilities)
- **Custom prompt engineering:** Compliance classification tasks (designed for 12 distinct violation categories)
- **SharePoint integration:** Accessing solicitation PDFs (leveraging existing document management infrastructure)

- **Power Automate:** Workflow triggers and notifications (seamless integration with current processes)
- **Dataverse:** Storing classification results and feedback (comprehensive audit trail and continuous improvement)

Collaboration Details:

- **Worked with:** 2 Pre-Award compliance specialists (violation categories), Legal Affairs (classification criteria), faculty SMEs (prompt examples), compliance officers (validation testing)
- **How decisions were made:** Led prompt design workshops with compliance SMEs to define classification taxonomy and establish testing frameworks
- **Communication challenges/successes:** Regular testing sessions with compliance officers to validate classifications and build trust in automated system
- **Cross-functional coordination:** Balanced technical implementation with compliance requirements, legal standards, and faculty workflow needs

Leadership Moments:

- **Decisions you drove:** Led prompt design workshops with compliance SMEs to define classification taxonomy
- **People you influenced/mentored:** Drove adoption of Document Processor agent over custom development approach demonstrating enterprise solution benefits
- **Processes you improved:** Established testing framework using real solicitation examples across violation types
- **Initiative you took beyond assigned tasks:** Created comprehensive documentation for prompt logic and agent maintenance procedures

Problem-Solving Process:

- **Classification Framework:** Collaborated with compliance specialists to identify 12 distinct violation categories with clear definition criteria
- **Prompt Engineering:** Developed few-shot learning approach using SME examples to achieve high classification accuracy
- **Integration Challenge:** Configured Document Processor agent with SharePoint/Power Automate integration for seamless workflow processing
- **Validation Process:** Tested classification accuracy against 100+ historical solicitation documents under size limits with compliance officer feedback loops

R - RESULTS Result Summary: The automated eligibility agent achieved exceptional performance with 95% classification accuracy, 70% reduction in review time, 85% decrease in late-stage proposal rejections, and \$30,000 estimated annual savings while establishing a scalable framework for compliance automation.

Detailed Impact:

- **Quantified business impact:**

- **95% accuracy** in classifying compliance issues across eligible documents
- **70% reduction** in review time for initial eligibility assessment
- **85% decrease** in late-stage proposal rejections due to eligibility issues
- **\$30,000 estimated savings** annually in avoided compliance violations

- **Technical achievements:**

- **Successfully configured Document Processor agent** achieving 95% classification accuracy across 12 violation categories
- **Developed custom prompt library** covering comprehensive compliance violation taxonomy
- **Created SharePoint/Power Automate integration** for automatic document processing and workflow triggers
- **Established feedback mechanism** with compliance officers for continuous prompt improvement and system refinement

- **Team/process improvements:**

- **Enabled proactive faculty guidance** with specific violation category identification before submission deadlines
- **Created standardized compliance classification** across eligible funding agencies and proposal types
- **Established comprehensive audit trail** for classification decisions and recommendations supporting regulatory compliance
- **Improved stakeholder confidence** in automated systems through transparent validation and feedback processes

- **Personal growth/skills gained:**

- **Enterprise AI agent configuration:** Gained expertise in prompt engineering for classification tasks using Microsoft Copilot Studio
- **Workflow automation mastery:** Advanced skills in configuring enterprise AI agents and workflow automation within existing business processes
- **Compliance framework understanding:** Enhanced understanding of research compliance frameworks and document processing limitations in regulated environments
- **Stakeholder management:** Experience building trust and adoption for AI automation systems in traditional manual review environments

RIT RA

FedDisco

Project Information Header

- **Project:** FedDisco Mobile Federated Learning Application
- **Company:** Academic Capstone Project
- **Role:** Lead Developer/Researcher
- **Duration:** 5 Weeks (First Prototype Completed)

Summary STAR (1.5-2.5 minutes speaking time)

S: For my academic capstone project, I saw that mobile apps handling sensitive user data like images and text faced a major problem - traditional machine learning required centralizing data, which created privacy risks and communication bottlenecks that users weren't comfortable with.

T: I needed to implement the FedDisco federated learning algorithm from scratch to enable distributed machine learning across mobile devices without compromising user privacy, delivering a working prototype in just 5 weeks for academic review.

A: I built a complete system using React Native for the mobile client, Python FastAPI for the server, and TensorFlow Lite for on-device training. I implemented custom FedDisco with optimized parameters and created a modular architecture starting with image processing while planning for text functionality.

R: Delivered a working prototype that achieved ~70% reduction in communication overhead because we used grpc instead of rest protocol, - successfully passing academic validation while proving federated learning could work practically on mobile devices.

Detailed STAR

S - SITUATION Situation Summary: For my academic capstone project, I identified the critical need for privacy-preserving machine learning in mobile applications where traditional centralized approaches create privacy risks and communication bottlenecks, particularly problematic for personal content like images and text.

Detailed Context:

- **Company context & business problem:** As part of my capstone project, I identified the critical need for privacy-preserving machine learning in mobile applications where users' sensitive data

(images, text) cannot be centralized due to privacy concerns. Traditional ML approaches require data centralization, creating privacy risks and communication bottlenecks, particularly problematic for mobile applications handling personal content.

- **Team structure & your role:** Independent capstone project with faculty advisor guidance. As Lead Developer/Researcher, I was responsible for the complete system architecture, implementation of both client and server components, and delivery of a working prototype demonstrating federated learning capabilities on mobile devices.
- **Timeline & constraints:** Aggressive 8-week timeline for first prototype delivery with significant system limitations. Need to balance proof-of-concept development with production-ready architecture decisions. Academic environment constraints including limited computational resources and requirement for comprehensive documentation.
- **Scale/scope of the challenge:** Complete federated learning system from scratch, mobile optimization, privacy preservation, academic-quality prototype, 8-week delivery

T - TASK/CHALLENGE Task Summary: I was tasked with building a complete mobile application implementing the FedDisco (Federated Learning) algorithm that enables privacy-preserving machine learning across distributed mobile devices, starting with image processing capabilities as the foundational component.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Design and implement FedDisco algorithm for mobile federated learning environment
 - Build client-side mobile application handling local training on user devices
 - Develop server-side aggregation system managing global model updates
 - Implement image processing pipeline as first prototype component
 - Create modular architecture supporting future text upload functionality
 - Ensure privacy-preserving design where raw data never leaves user devices
 - Deliver working prototype within 8-week academic deadline
- **Technical challenges faced:**
 - Implementing complex federated learning algorithm within mobile resource constraints
 - Balancing local computation efficiency with global model convergence quality
 - Managing communication optimization between multiple mobile clients and central server
 - Building modular system architecture supporting both image and text processing workflows
- **Business/stakeholder challenges:**
 - Delivering academic-quality prototype within tight 8-week timeline
 - Balancing comprehensive system design with practical implementation constraints
 - Managing scope effectively by prioritizing image processing before text functionality
 - Ensuring prototype demonstrates clear federated learning value proposition

- **Success criteria and constraints:** 8-week delivery, working federated learning, privacy preservation, mobile optimization, academic validation, modular architecture

A - ACTION (Detailed Breakdown) Action Summary: I successfully architected and implemented a comprehensive federated learning system using modular design principles, deployed the FedDisco algorithm with optimized communication protocols, and strategically executed phased development delivering a working image processing prototype while establishing infrastructure for text processing expansion.

Technical Stack Used:

- **Mobile Client:** React Native (cross-platform deployment enabling iOS/Android testing)
- **Server Architecture:** Python + FastAPI (lightweight server achieving efficient model aggregation)
- **Federated Learning:** Custom FedDisco implementation with optimized parameters (K=5 test clients, C=0.6 fraction, E=3 epochs, B=16 batch size)
- **Image Processing:** TensorFlow Lite (on-device training achieving 40% memory reduction)
- **Communication:** RESTful APIs with model compression (achieved 70% communication overhead reduction)
- **Security:** Encrypted model updates with SHA-256 hashing ensuring privacy-preserving aggregation

Collaboration Details:

- **Worked with:** Faculty advisor (algorithm validation), 3 peer testers (prototype validation), academic review committee
- **How decisions were made:** Made independent technical architecture decisions with weekly advisor consultations on federated learning convergence optimization
- **Communication challenges/successes:** Delivered comprehensive documentation and live demonstration showcasing federated learning capabilities
- **Cross-functional coordination:** Balanced academic research requirements with practical implementation constraints and peer validation

Leadership Moments:

- **Decisions you drove:** Successfully implemented phased development approach prioritizing image processing, enabling prototype delivery within 8-week constraint
- **People you influenced/mentored:** Technical innovation building modular architecture with plugin-based design enabling seamless future expansion
- **Processes you improved:** Established systematic testing protocol validating federated learning convergence across 5 simulated mobile clients
- **Initiative you took beyond assigned tasks:** Built comprehensive documentation framework enabling future researchers to build upon implementation

Problem-Solving Process:

- **Mobile Resource Constraints:** Solved memory limitations by implementing TensorFlow Lite with quantization reducing model size by 60%
- **Communication Efficiency:** Achieved 70% bandwidth reduction through gradient compression and selective model parameter updates
- **Convergence Optimization:** Tuned FedDisco parameters ($E=3$, $C=0.6$) achieving stable convergence in 15 rounds vs. theoretical 25 rounds
- **Alternative Solutions Considered:** Evaluated Flutter vs React Native (chose React Native for faster development), PyTorch vs TensorFlow (chose TensorFlow Lite for mobile optimization)

R - RESULTS Result Summary: Successfully delivered a working federated learning prototype demonstrating privacy-preserving machine learning on mobile devices, achieving significant performance improvements in communication efficiency and model convergence while establishing a robust foundation for comprehensive multi-modal federated learning applications.

Detailed Impact:

- **Quantified business impact:**
 - **8-week prototype delivery** successfully completed, demonstrating federated learning feasibility on mobile platforms
 - **70% reduction** in communication overhead through model compression and selective parameter updates
 - **60% memory optimization** enabling deployment on resource-constrained mobile devices
 - **Academic validation** with successful demonstration to review committee and peer testing completion
- **Technical achievements:**
 - **Working FedDisco implementation** achieving convergence in 15 rounds (40% improvement over theoretical baseline)
 - **Cross-platform mobile application** successfully tested on both iOS and Android with consistent performance
 - **Privacy-preserving architecture** validated through security audit showing zero raw data transmission
 - **Modular plugin architecture** built enabling text processing integration with 90% code reuse
- **Team/process improvements:**
 - **Rapid prototyping methodology** proved effective for delivering complex federated learning system within academic constraints
 - **Comprehensive test suite** established covering federated learning convergence, mobile performance, and privacy validation
 - **Documentation framework** created enabling future researchers to build upon federated learning mobile implementation
 - **Peer validation process** implemented with 3 independent testers confirming prototype functionality

- **Personal growth/skills gained:**

- **Federated learning expertise:** Practical implementation experience with FedDisco algorithm optimization for mobile environments
- **Mobile AI development:** Advanced skills in TensorFlow Lite optimization and on-device machine learning deployment
- **System architecture mastery:** Proven ability to design modular, extensible systems under tight timeline constraints
- **Academic research execution:** Successfully delivered research-quality prototype meeting academic standards while demonstrating practical applicability

Key Prototype Achievements:

- **5 concurrent clients** successfully participating in federated learning rounds
- **87% image classification accuracy** maintained while preserving complete data privacy
- **45 seconds average training time** per client per round on standard mobile devices
- **Successful model convergence** demonstrated across heterogeneous mobile device configurations

DS

CDQT Ver 3 Real

Project Information Header

Project: Conversational AI System for Clinical Trial Data Quality Analysis

Company: Daiichi Sankyo

Role: R&D Data Governance Intern

Duration: [Duration not specified – ongoing project]

Summary STAR (1.5–2.5 minutes speaking time)

S: Daiichi Sankyo wanted to explore whether Large Language Models could replace their manual Tableau-based data quality analysis for clinical trial data. The challenge was that I had to work in a highly regulated pharmaceutical environment where getting approval for cloud services took months, and I was restricted to 32GB RAM systems with no GPU acceleration and no access to specialized clinical datasets.

T: I was tasked with designing a proof-of-concept conversational AI system that could replicate and eventually replace the Tableau-based Data Quality Tool (DQT), which summarized discrepancies, milestones, and study-level errors but lacked scalability and interactivity.

A: I adopted a data-centric and modular design strategy. After experimenting with tabular models like TAPAS and multitab-qa-base, I transitioned to a sophisticated Retrieval-Augmented Generation (RAG) framework using Mistral 7B deployed locally via Ollama. I built a custom clinical dataset using ChatDSI and developed a **three-tier modular system — Fast, Balanced, and Quality configurations**, each optimized for different trade-offs between speed and precision. Later, I proposed a **Version 3 SQL Translation Layer** using lightweight CPU-optimized LLMs like Phi-3-mini to achieve near real-time structured responses.

R: The proof-of-concept demonstrated feasibility with response times between 4–9 minutes locally and established a regulatory-compliant AI development workflow. I also identified a production-ready pathway using the SQL Translation Layer, projected to reduce response times to seconds once cloud access is approved.

Detailed STAR

S – SITUATION

Situation Summary:

Daiichi Sankyo's R&D Data Governance team wanted to transform their Tableau-based Data Quality

Tool (DQT) — which provided static data insights — into a **conversational AI system** capable of dynamic querying, context understanding, and real-time analysis of clinical trial data.

Detailed Context:

- **Company context:** The existing Tableau tool summarized discrepancies and milestones but couldn't handle real-time, multi-table interactions.
 - **Role & team:** Worked within the R&D Data Governance team, collaborating with database administrators, clinical data managers, and regulatory officers.
 - **Constraints:** Highly regulated setup with restricted data sharing, no GPUs, and lengthy cloud approval cycles. The POC had to run locally on 32GB RAM systems.
 - **Problem scale:** Needed to design an AI solution robust enough for regulatory environments yet efficient under severe hardware and data restrictions.
-

T – TASK/CHALLENGE

Task Summary:

Develop a **proof-of-concept conversational AI system** that could understand and query complex relational clinical trial datasets while maintaining compliance and delivering performance comparable to existing Tableau workflows.

Detailed Responsibilities:

- Design conversational interfaces for structured (tabular) data analysis.
- Explore multiple architectures — tabular QA models, RAG systems, and SQL-based pipelines.
- Build and preprocess datasets with clinical domain context and glossary.
- Optimize latency and precision without access to GPU infrastructure.
- Balance modularity, scalability, and compliance for future production deployment.

Challenges Faced:

- Tabular models (TAPAS, multitab-qa) failed to generalize across tables.

- Text-to-SQL pipelines required heavy prompt engineering.
- Lack of domain datasets for fine-tuning limited contextual accuracy.
- Regulatory restrictions constrained experimentation with cloud LLMs.

Business/Stakeholder Challenges:

- Demonstrate AI's feasibility to leadership under compliance restrictions.
 - Prove value against established Tableau workflows.
 - Align model capabilities with business expectations and real clinical questions.
-

A – ACTION (Detailed Breakdown)

Action Summary:

I followed a structured, data-centric, and iterative development process inspired by OpenAI's modular architectures and Amazon Rufus's contextual retrieval system.

Version 1 – Tabular Models (Baseline Phase)

- **Models:** TAPAS, multitab-qa-base
 - **Storage:** FAISS Vector Store
 - **Outcome:** Encoded multiple tables into embeddings for question answering.
 - **Challenge:** Could not manage multi-table joins or nuanced relational logic.
 - **Learning:** Shift focus from model-centric to *data-centric* development.
-

Version 2 – Modular RAG Architecture (Core Development Phase)

Deployed **Mistral 7B** locally (via Ollama) with interchangeable modules for embeddings, retrieval, and context processing.

Key Design Principles:

- Component isolation for easy experimentation.
- Modular configurations tailored to speed vs precision.
- Scalable data pipelines for growing datasets.

Fast Configuration (Speed-Focused)

- **Embedding:** all-MiniLM-L6-v2 (lightweight, compact)
- **Retriever:** FAISSStore
- **RAG Variant:** BasicRAG
- **Goal:** Quick prototyping (<4.5 mins/query).

Balanced Configuration (Production-Ready)

- **Embedding:** all-mpnet-base-v2 (contextual, robust)
- **Retriever:** HNSWStore
- **RAG Variant:** ReRankingRAG (cross-encoder reranking)
- **Goal:** Optimize accuracy while maintaining near production speed (~4.2 mins/query).

Quality Configuration (Precision-Focused)

- **Embedding:** intfloat/e5-large (deep domain comprehension)
- **Retriever:** ChromaStore (metadata-driven)
- **RAG Variant:** ContextualCompressionRAG
- **Goal:** Max precision for regulatory or executive insights (~9.3 mins/query).

Custom Tools:

- **ChatDSI:** Synthetic QA dataset creation.

- **Glossary collaboration:** Worked with David (glossary) and Manisha (goal alignment).
 - **Compliance coordination:** Partnered with regulatory officers to ensure audit readiness.
-

Version 3 – SQL Translation Layer with LLM (Future-Ready Approach)

Approach: Integrate a lightweight LLM (e.g., Phi-3-mini) to translate natural-language questions into SQL queries and generate structured results conversationally.

Implementation:

- Deploy **Phi-3-mini** locally (CPU-only).
- Build **SQL templates** for benchmark questions.
- Add a **post-processing layer** to convert SQL results into readable responses.

Rationale:

Most benchmark questions can be expressed as SQL; this approach leverages the existing database directly, bypassing full-text generation for faster, deterministic outputs.

Projected latency: Reduce from 4–9 minutes to **under 1 minute** post-cloud migration.

Collaboration & Leadership:

- Coordinated between technical (AI) and regulatory teams for compliance approval.
 - Proposed architecture modularity inspired by ChatGPT's tiered models and Amazon Rufus's adaptive interfaces.
 - Introduced null-value handling and metadata-aware prompts to improve interpretability.
 - Created a development workflow supporting multiple datasets and table extensions for future expansion.
-

R – RESULTS

Result Summary:

Delivered a fully functional proof-of-concept showcasing that **conversational AI can replace manual Tableau-based clinical data quality analysis** under strict regulatory and hardware constraints.

Quantitative Impact:

- Achieved proof-of-concept across **3 architectures (Fast, Balanced, Quality)**.
- Answered **56 benchmark questions** across configurations with predictable latency (4–9 mins).
- Defined **regulatory-compliant AI development framework** adopted by R&D leadership.
- Proposed **SQL Translation Layer** for near-real-time query resolution.

Technical Achievements:

- Built the first **modular RAG architecture** at Daiichi Sankyo with configurable embeddings and retrievers.
- Created a **clinical trial glossary** enabling accurate language model grounding.
- Established **multi-configuration AI evaluation methodology** (speed vs accuracy).
- Demonstrated **CPU-only LLM deployment** compliant with pharma security policies.

Process & Collaboration Improvements:

- Introduced a **structured AI experimentation process** within a compliance-driven organization.
- Enabled **cross-functional alignment** between data scientists, clinical experts, and governance officers.
- Improved clarity on tool direction — repositioned from “clinical trial data viewer” to “data quality assistant.”

Personal Growth:

- Gained expertise in **RAG system architecture under constraints**.
- Learned **regulatory AI development** processes in pharmaceutical R&D.

- Developed end-to-end understanding of **data preprocessing, glossary design, and modular system scaling**.
- Acquired **multi-architecture evaluation** and optimization experience for production-grade AI.

ICF

Project Information Header

- **Project:** Automated ICF Classification System for Clinical Trial Compliance
- **Company:** Daiichi Sankyo
- **Role:** R&D Data Governance Intern
- **Duration:** 10 Months (June 2024 - April 2025)

Summary STAR (1.5-2.5 minutes speaking time)

S: At Daiichi Sankyo, our legal and regulatory teams were stuck manually reviewing each Informed Consent Form for about 4-5 minutes just to identify data sharing clauses. With hundreds of these documents across multiple studies and countries, this was creating serious delays in our clinical trial operations and becoming a real bottleneck.

T: I was assigned to build a complete automated classification and document management system that could maintain our regulatory compliance standards while dramatically reducing processing time. The challenge was that we had zero tolerance for misclassification errors given the regulatory implications.

A: I developed the system through four iterative versions, starting with rule-based approaches and evolving to a sophisticated two-stage transformer architecture using DistilBERT and FAISS embeddings. I also built a comprehensive document management platform with study filtering, version control, and geographic organization to handle all the complexity.

R: Achieved a 0.93 F1 score with 0.96 accuracy and reduced processing time by 87% - from 4-5 minutes down to just 33 seconds per document. We now have a production-ready system that eliminated those critical bottlenecks across hundreds of documents for multiple studies and countries.

Detailed STAR

S - SITUATION Situation Summary: Daiichi Sankyo's legal and regulatory teams were facing critical bottlenecks with manual review processes that took 4-5 minutes per Informed Consent Form to identify data sharing clauses, creating delays across hundreds of documents and risking regulatory compliance issues.

Detailed Context:

- **Company context & business problem:** Daiichi Sankyo's legal and regulatory teams were spending 4-5 minutes manually reviewing each Informed Consent Form (ICF) to identify critical data sharing clauses. With hundreds of documents across multiple studies, countries, and consent form versions, this manual process created significant bottlenecks in clinical trial

operations. Missing or misinterpreting these clauses could lead to serious regulatory compliance issues.

- **Team structure & your role:** R&D Data Governance team collaborating with legal reviewers, regulatory specialists, and clinical trial operations. As R&D Data Governance Intern, I was the primary technical developer responsible for designing and implementing the complete automated classification system and document management platform.
- **Timeline & constraints:** 10-month project requiring iterative development through four major model versions. No pre-existing labeled datasets available. Critical requirement to maintain regulatory compliance standards while dramatically reducing processing time from manual 4-5 minutes to automated seconds.
- **Scale/scope of the challenge:** Hundreds of documents, multiple studies and countries, zero tolerance for misclassification, regulatory compliance requirements

T - TASK/CHALLENGE Task Summary: I was tasked with building a complete automated ICF classification and management system that could eliminate manual review bottlenecks, ensure regulatory compliance, and provide comprehensive document management capabilities across multiple studies and geographic regions.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Develop automated classification system progressing through four iterative model versions
 - Build comprehensive PDF database and management platform with study-based filtering
 - Create document comparison tools for version control across trial phases
 - Implement geographic filtering for country/region regulatory compliance
 - Design user-friendly interface enabling legal reviewers to edit classifications and provide feedback
 - Achieve production-ready performance reducing processing time from 4-5 minutes to under 33 seconds
- **Technical challenges faced:**
 - Severe class imbalance causing initial F1 scores of 0.58 across multiple model iterations
 - Building robust PDF parsing maintaining document structure across varied international formats
 - Implementing two-stage architecture balancing accuracy with processing speed requirements
 - Creating comprehensive document management system supporting complex clinical trial workflows
- **Business/stakeholder challenges:**
 - Eliminating critical bottlenecks in clinical trial operations affecting multiple departments
 - Ensuring zero tolerance for misclassification errors given regulatory compliance implications
 - Managing iterative feedback from legal reviewers while maintaining system usability

- Supporting complex multi-study, multi-country document organization requirements
- **Success criteria and constraints:** 87%+ time reduction, 0.9+ F1 score, regulatory compliance, zero misclassification tolerance, 10-month iterative development

A - ACTION (Detailed Breakdown) Action Summary: I architected a sophisticated four-version iterative development process, progressing from rule-based systems to advanced two-stage transformer architecture, while building a comprehensive document management platform that achieved 0.93 F1 score and 87% processing time reduction.

Technical Stack Used:

- **Model Evolution:** Rule-based → TF-IDF ensemble → Transformer-based → Two-stage architecture (embedding retrieval + DistilBERT classification)
- **Embeddings & Search:** all-MiniLM-L6-v2 + FAISS IndexFlatIP (similarity threshold empirically set at 0.45)
- **Classification:** DistilBERT fine-tuned with 5-fold cross-validation and learning rate scheduling
- **Frontend Evolution:** Streamlit → Flask + PDF.js + responsive web interface
- **Document Management:** Comprehensive filtering (study, version, country, date), batch processing, version control
- **Optimization:** ONNX runtime conversion, batch processing for multi-document scenarios

Collaboration Details:

- **Worked with:** Legal reviewers (classification validation), regulatory specialists (compliance requirements), clinical operations (workflow integration)
- **How decisions were made:** Implemented active learning techniques prioritizing ambiguous cases for expert review
- **Communication challenges/successes:** Developed annotation guidelines and consensus-based review processes for disputed classifications
- **Cross-functional coordination:** Balanced technical ML development with strict legal and regulatory requirements across multiple departments

Leadership Moments:

- **Decisions you drove:** Chose two-stage architecture approach achieving optimal balance between accuracy and processing speed
- **People you influenced/mentored:** Strategic innovation building comprehensive document management platform beyond simple classification
- **Processes you improved:** Implemented human feedback loops and active learning for continuous model refinement
- **Initiative you took beyond assigned tasks:** Built comprehensive document management platform supporting entire clinical trial workflow

Problem-Solving Process:

- **Performance Bottleneck:** Solved 4-5 minute manual review time through two-stage automated architecture achieving 33-second processing
- **Class Imbalance:** Overcame F1 score limitation (0.58 → 0.93) through iterative model architecture improvements and synthetic data techniques
- **Regulatory Compliance:** Ensured zero tolerance for misclassification through comprehensive validation and human-in-the-loop workflows
- **Alternative Solutions Considered:** Evaluated various embedding models, classification approaches, and frontend architectures across four development versions

R - RESULTS Result Summary: The system achieved exceptional regulatory-grade performance with 0.93 F1 score and 87% processing time reduction (4-5 minutes to 33 seconds), while providing comprehensive document management capabilities supporting complex multi-study clinical trial operations.

Detailed Impact:

- **Quantified business impact:**
 - **87% processing time reduction** (4-5 minutes manual review to 33 seconds automated processing)
 - **0.93 F1 score** with 0.96 accuracy exceeding industry standards for regulatory compliance applications
 - **Hundreds of documents** now processed efficiently across multiple studies, countries, and consent form versions
 - **Eliminated critical bottlenecks** in clinical trial operations affecting legal and regulatory teams
- **Technical achievements:**
 - **Four-version iterative development:** Successfully evolved from 0.58 F1 score to 0.93 through systematic model improvements
 - **Comprehensive document platform:** Built complete management system with study-based filtering, version control, and geographic organization
 - **Two-stage production architecture:** Achieved optimal performance through embedding-based retrieval + efficient DistilBERT classification
 - **Advanced document capabilities:** Implemented batch processing, document comparison, full-text search, and automated reporting
- **Team/process improvements:**
 - **Regulatory workflow transformation:** Eliminated manual review bottlenecks while maintaining compliance standards
 - **Cross-functional collaboration:** Established effective feedback loops between technical development and legal/regulatory teams
 - **Quality assurance framework:** Created annotation guidelines and consensus-based review processes for sustained accuracy

- **Scalable architecture:** Built system supporting expansion across additional studies and geographic regions
- **Personal growth/skills gained:**
 - **Regulatory compliance expertise:** Deep understanding of clinical trial documentation requirements and compliance constraints
 - **Iterative system development:** Advanced skills in systematic model improvement through multiple development cycles
 - **Complex stakeholder management:** Experience balancing technical innovation with strict regulatory and legal requirements
 - **End-to-end platform development:** Comprehensive experience building complete document management solutions beyond core ML capabilities

Data Snapshot

Project Information Header

- **Project:** CTMS Database Migration System - Alteryx to Python Migration
 - **Company:** Daiichi Sankyo
 - **Role:** R&D Data Governance Intern
 - **Duration:** 2 weeks
-

Summary STAR *(1.5-2.5 minutes speaking time)*

S: While I was focused on AI development at Daiichi Sankyo, I noticed that our CTMS database migration was entirely dependent on expensive Alteryx licensing, which was creating bottlenecks for our clinical trial operations and eating up a significant portion of our budget.

T: Even though this wasn't my assigned project, I saw a real opportunity to solve a broader team problem. I proposed migrating the entire system from Alteryx to Python, which would eliminate our licensing dependency and give us much more flexibility.

A: I reverse-engineered all the existing Alteryx workflows and rewrote them using SQLAlchemy and pandas. I also implemented parallel processing capabilities that Alteryx couldn't provide, which was a major upgrade. I managed to complete this entire migration in just 2 weeks working alongside my regular AI responsibilities.

R: We completely eliminated the Alteryx licensing dependency, improved performance significantly through parallel processing, and gave the team much more flexibility to modify workflows as needed. This saved substantial licensing costs and removed a major operational bottleneck. Taking ownership beyond my core responsibilities taught me to think about system-wide problems rather than just my individual projects, which has made me more valuable to every team I've worked with since.

Detailed STAR

S - SITUATION

Situation Summary: Daiichi Sankyo's R&D department needed to replace their existing Alteryx-based CTMS data migration system due to licensing constraints, limited customization options, and dependency issues that were creating bottlenecks in clinical trial data availability for regulatory reporting.

Detailed Context:

- **Company context & business problem:** Daiichi Sankyo's R&D department was using an Alteryx-based system for migrating clinical trial management system (CTMS) data between databases. The Alteryx solution created licensing bottlenecks, offered limited customization for complex table relationships, and dependency issues that affected the reliability of regulatory reporting and analysis workflows.
- **Team structure & your role:** R&D Data Governance team working with database administrators, clinical data managers, and regulatory reporting analysts. As R&D Data Governance Intern, I was responsible for completely migrating the existing Alteryx workflows to a custom Python solution.
- **Timeline & constraints:** 2-week timeline to migrate from Alteryx to Python while maintaining all existing functionality. Critical requirement to preserve data integrity and ensure zero downtime during the transition from the existing Alteryx system.
- **Scale/scope of the challenge:** Complete system migration, production database systems, regulatory compliance requirements, zero data loss tolerance

T - TASK/CHALLENGE

Task Summary: I was tasked with completely rewriting the existing Alteryx-based data migration system in Python, preserving all current functionality while adding enhanced capabilities for parallel processing, error handling, and automated scheduling.

Detailed Responsibilities:

- **Alteryx to Python Migration:** Analyze existing Alteryx workflows and completely rewrite them as a Python-based automated migration system
- **Functionality Preservation:** Ensure all current CTMS data migration capabilities are maintained during the transition
- **Enhanced Architecture:** Implement improved parallel processing, error handling, and scheduling capabilities beyond Alteryx limitations
- **Seamless Transition:** Deliver production-ready system with zero downtime and no disruption to ongoing clinical trial operations
- **Documentation & Training:** Create comprehensive documentation for the new Python system to replace Alteryx workflows

Technical challenges faced:

- **Workflow Translation:** Reverse-engineering and rewriting complex Alteryx data manipulation workflows in Python
- **Feature Parity:** Ensuring the new Python system matched or exceeded all existing Alteryx functionality

- **Performance Optimization:** Implementing parallel processing capabilities that weren't possible in the Alteryx system
- **Integration Continuity:** Maintaining all existing database connections and scheduling while transitioning platforms

Business/stakeholder challenges:

- **Rapid Migration Timeline:** Completing full system migration in only 2 weeks without disrupting critical operations
- **Licensing Cost Reduction:** Eliminating Alteryx licensing dependencies while maintaining system reliability
- **Stakeholder Confidence:** Ensuring clinical data managers and regulatory analysts trusted the new Python system
- **Regulatory Compliance:** Maintaining all compliance requirements during the platform transition

A - ACTION (*Detailed Breakdown*)

Action Summary: I systematically analyzed the existing Alteryx workflows, completely rewrote them in Python with enhanced capabilities, and implemented superior parallel processing and error handling that surpassed the original system's performance.

Technical Stack Used:

- **Migration Foundation:** Reverse-engineered Alteryx workflows and translated data manipulation logic to Python
- **Database Connectivity:** SQLAlchemy + psycopg2-binary (robust PostgreSQL connections replacing Alteryx database connectors)
- **Data Processing:** pandas + numpy (efficient data transformation replacing Alteryx data manipulation tools)
- **Parallel Processing:** concurrent.futures ThreadPoolExecutor (advanced capability not available in Alteryx)
- **Configuration Management:** python-dotenv + pyyaml (flexible configuration system improving on Alteryx limitations)
- **Logging & Monitoring:** Python logging with pytz (comprehensive audit trails exceeding Alteryx capabilities)

Collaboration Details:

- **Legacy System Analysis:** Worked with database administrators to understand existing Alteryx workflow dependencies
- **Requirements Validation:** Collaborated with clinical data managers to ensure Python system met all operational needs
- **Transition Planning:** Coordinated with regulatory analysts to maintain compliance during migration
- **Testing & Validation:** Extensive parallel testing of Python system against existing Alteryx outputs

Leadership Moments:

- **Migration Strategy:** Led decision to completely rewrite in Python rather than attempting partial migration
- **Architecture Innovation:** Drove implementation of parallel processing capabilities impossible in Alteryx
- **Quality Assurance:** Established comprehensive testing protocols ensuring migration accuracy
- **Knowledge Transfer:** Created detailed documentation enabling team to maintain Python system

Problem-Solving Process:

- **Workflow Analysis:** Systematically reverse-engineered Alteryx processes to understand all data transformation logic
- **Performance Enhancement:** Implemented parallel processing using ThreadPoolExecutor, significantly improving on Alteryx performance
- **Error Handling Improvement:** Built sophisticated error recovery mechanisms beyond Alteryx capabilities
- **Validation Strategy:** Created comprehensive testing framework comparing Python outputs with Alteryx results

R - RESULTS

Result Summary: The Python migration successfully eliminated Alteryx licensing dependencies, achieved superior performance with parallel processing, and maintained zero data loss while providing enhanced capabilities that exceeded the original system's functionality.

Detailed Impact: Quantified business impact:

- **Licensing Elimination:** Removed Alteryx licensing costs and dependencies, reducing operational overhead
- **Performance Improvement:** Parallel processing reduced migration time compared to sequential Alteryx workflows
- **System Reliability:** Achieved zero data loss across all migrations during and after transition
- **Enhanced Capabilities:** Delivered superior error handling and monitoring capabilities beyond Alteryx limitations

Technical achievements:

- **Complete Migration:** Successfully translated all Alteryx workflows to Python within 2-week timeline
- **Enhanced Architecture:** Implemented parallel processing and advanced error handling not possible in Alteryx
- **Production Readiness:** Delivered robust system with comprehensive logging and monitoring capabilities

- **Seamless Transition:** Zero downtime migration maintaining all existing functionality and data integrity

Team/process improvements:

- **Platform Independence:** Eliminated vendor dependency on Alteryx licensing and support constraints
- **Customization Capability:** Enabled future enhancements and modifications not possible with Alteryx limitations
- **Knowledge Transfer:** Created comprehensive Python documentation replacing proprietary Alteryx workflows
- **System Maintainability:** Established open-source Python solution that team can modify and enhance independently

Personal growth/skills gained:

- **Legacy System Migration:** Expertise in analyzing and replacing proprietary systems with custom solutions
- **Rapid Development:** Proven ability to deliver complete system migration under aggressive 2-week timeline
- **Requirements Analysis:** Advanced skills in reverse-engineering existing workflows and improving upon them
- **Platform Translation:** Experience converting proprietary tool workflows to open-source Python solutions

Pipeline Migration

Project Information Header

- **Project:** Veeva-Vault Migration Project - Clinical Trial Data ETL Pipeline
- **Company:** Daiichi Sankyo
- **Role:** R&D Data Governance Intern
- **Duration:** 6 Weeks

Summary STAR (1.5-2.5 minutes speaking time)

S: At Daiichi Sankyo, we had clinical trial data in Veeva Vault that was getting updated on a weekly basis, and we needed to bring this data into our data warehouse - AWS Redshift - so our teams could use it for compliance dashboards and analytics. The manual process wasn't working and we needed an automated solution.

T: I needed to build a Python ELT pipeline that could automatically run weekly migrations, pulling this hierarchical clinical trial data from 423 API endpoints while keeping all the parent-child relationships intact and meeting our regulatory compliance standards. I had 6 weeks to get this done and needed to include proper logging and monitoring.

A: I built a complete solution using Python with SQLAlchemy for the API calls, AWS S3 for intermediate storage, and Amazon Redshift as our data warehouse. I used recursive API calls to handle the complex data hierarchies, added validation steps to ensure regulatory compliance, and implemented comprehensive logging and monitoring for the weekly runs.

R: Successfully migrated 300+ critical endpoints and established a reliable weekly migration process that gave our Clinical Data Managers and Regulatory Affairs teams the up-to-date structured data they needed for their compliance work and analytics.

What I did:

1. **SO this data governance team was formed 1.5 years ago, and we had huge amount of data and we had a very little oversight and we wanted to create a single source of truth for the whole Daiichi Sankyo Database and our data was being stored at 5 different vendors, like Colibra, Veevavault,IQVIA, Mastercontrol, etc, so we wanted to migrate it to build a SST Database,**

2. Endpoint Understanding(423),
3. I first gathered all rules(and policies based on the data we were accessing
4. So we initially went with a ETL pipeline but the data was too complex to maintain the pipeline so we switched to ELT Pipeline such that we were loading it into s3 bucket then transforming it before pushing it to data warehouse
5. We used to see what rules were not being followed and then we use to question the veeva vault data owners about the discrepancies and handle this kind of issues
6. And the BI team got the data in the required format to create dashboards on top of this. 35 tables , based on 43 rules, and we had to govern whether the rules are being followed if ntowe had to question the data owners at veeva accordingly to resolve this discrepancies.

Detailed STAR

S - SITUATION Situation Summary: Daiichi Sankyo needed to automate the transfer of clinical trial data from Veeva Vault to Amazon Redshift because multiple stakeholder groups required structured data for regulatory compliance dashboards and analytics.

Detailed Context:

- **Company context & business problem:** Daiichi Sankyo needed to automate the transfer of clinical trial data from Veeva Vault (cloud-based content management platform) to Amazon Redshift for analytics and reporting. For managing complex hierarchical data across 423 API endpoints created bottlenecks for Clinical Data Managers, Database Administrators, Regulatory Affairs Directors, and Biostatistics Leads requiring structured data for regulatory compliance dashboards.
- **Team structure & your role:** Cross-functional data governance team including Clinical Data Managers, Database Administrators, Regulatory Officers, and Compliance specialists. As R&D Data Governance Intern, I was responsible for designing and implementing the complete Python-based ETL pipeline for clinical data migration.
- **Timeline & constraints:** 6-week project timeline across four phases requiring extraction from 300+ critical API endpoints. Need to maintain parent-child

relationships in hierarchical clinical trial data while ensuring regulatory compliance and data integrity for downstream analytics and reporting.

- **Scale/scope of the challenge:** 423 API endpoints, complex hierarchical data relationships, regulatory compliance requirements, multiple stakeholder groups

T - TASK/CHALLENGE Task Summary: I was tasked with building a comprehensive automated ETL pipeline that could efficiently extract hierarchical clinical trial data from 423 Veeva Vault API endpoints, ensure data governance compliance, and enable structured analytics for multiple stakeholder groups.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Design and implement Python-based ETL pipeline for clinical data migration from Veeva Vault to Amazon Redshift
 - Manage complex API extraction strategy maintaining parent-child relationships across hierarchical datasets
 - Apply schema validation and data transformation ensuring regulatory compliance and data integrity
 - Build robust error-handling mechanisms improving pipeline reliability and reducing manual intervention
 - Implement incremental data updates reducing redundant API requests and improving efficiency
 - Enable dashboard integration supporting Clinical Data Managers and Regulatory Affairs analysis workflows
- **Technical challenges faced:**
 - Managing complex hierarchical data structures across 423 API endpoints requiring efficient extraction strategy
 - Ensuring data accuracy and governance with schema validation across diverse clinical trial datasets
 - Implementing incremental updates while maintaining referential integrity in parent-child relationships
 - Building scalable pipeline handling large volumes of clinical data with regulatory compliance requirements
- **Business/stakeholder challenges:**

- Coordinating requirements across diverse stakeholder groups (Clinical Data Managers, Regulatory Officers, Biostatistics Leads)
- Balancing comprehensive data extraction with processing efficiency for production clinical trial operations
- Ensuring regulatory compliance standards while enabling flexible analytics and reporting capabilities
- Managing conflicting priorities between data completeness and processing performance
- **Success criteria and constraints:** 6-week delivery, 300+ endpoint migration, regulatory compliance, data integrity, stakeholder alignment

A - ACTION (Detailed Breakdown) Action Summary: I architected a comprehensive four-phase ETL pipeline using Python and AWS services, implementing sophisticated hierarchical data management and achieving significant improvements in extraction efficiency, data quality, and pipeline reliability.

Technical Stack Used:

- **Data Extraction:** Python + SQLAlchemy (efficient API management and structured data transfers)
- **Cloud Storage:** AWS S3 (intermediate JSON storage for flexibility and version control)
- **Data Warehouse:** Amazon Redshift with COPY command (optimized bulk uploads and distributed storage)
- **API Management:** Multi-level recursive API calls (maintaining parent-child relationships)
- **Analytics Integration:** DBeaver + Tableau (dashboard analytics and SQL querying)
- **Backend Validation:** Flask (API-based data validation and schema enforcement)

Collaboration Details:

- **Worked with:** Clinical Data Managers (schema requirements), Database Administrators (performance optimization), Regulatory Officers (compliance validation)
- **How decisions were made:** Led weekly stakeholder meetings providing updates and collecting feedback for pipeline refinement

- **Communication challenges/successes:** Facilitated schema validation workshops ensuring data governance best practices and regulatory alignment
- **Cross-functional coordination:** Balanced technical implementation with regulatory requirements across clinical trial operational teams

Leadership Moments:

- **Decisions you drove:** Focused on top 300 critical endpoints implementing incremental updates to optimize performance while ensuring comprehensive coverage
- **People you influenced/mentored:** Technical innovation developing recursive API call system guiding team on ETL best practices
- **Processes you improved:** Created custom error-handling strategies improving pipeline reliability by 50% with automated recovery mechanisms
- **Initiative you took beyond assigned tasks:** Developed comprehensive documentation and mentored junior analysts on data engineering concepts

Problem-Solving Process:

- **Hierarchical Complexity:** Implemented multi-level recursive API calls with dependency tracking maintaining parent-child relationships
- **Schema Consistency:** Defined strict validation rules collaborating with governance teams ensuring regulatory compliance
- **Performance Optimization:** Reduced API call redundancy by 30% through intelligent incremental update strategies
- **Alternative Solutions Considered:** Evaluated batch vs. streaming processing, single vs. distributed storage, manual vs. automated schema validation

R - RESULTS Result Summary: The automated ETL pipeline successfully migrated 300+ critical endpoints with significant improvements in efficiency, data quality, and reliability while establishing scalable foundation for clinical trial data analytics and regulatory compliance.

Detailed Impact:

- **Quantified business impact:**
 - **30% reduction** in API call redundancy through optimized extraction strategy

- **40% reduction** in data corruption risks through comprehensive validation and cleaning
- **50% improvement** in pipeline reliability through robust error-handling mechanisms
- **300+ critical endpoints** successfully migrated enabling comprehensive clinical trial analytics
- **Technical achievements:**
 - **Comprehensive data pipeline:** Built complete ETL system handling complex hierarchical relationships across clinical trial datasets
 - **Automated incremental updates:** Implemented real-time synchronization reducing manual processing overhead
 - **Distributed data architecture:** Successfully structured data across 8 key Redshift tables optimizing querying and scalability
 - **Regulatory compliance framework:** Established data governance practices ensuring industry standard compliance
- **Team/process improvements:**
 - **Cross-functional collaboration:** Fostered data-driven culture aligning technical implementation with regulatory standards
 - **Knowledge transfer:** Mentored junior analysts on ETL best practices and data engineering concepts
 - **Stakeholder alignment:** Addressed conflicting priorities through clear communication of data dependencies and risks
 - **Scalable architecture:** Designed system supporting future expansion to all 423 endpoints with modular extraction capabilities
- **Personal growth/skills gained:**
 - **Clinical data expertise:** Deep understanding of pharmaceutical data governance and regulatory compliance requirements
 - **Large-scale ETL architecture:** Advanced skills in designing robust data pipelines handling complex hierarchical relationships
 - **Stakeholder management:** Experience coordinating technical requirements across diverse clinical trial operational teams
 - **Cloud-native data engineering:** Comprehensive knowledge of AWS services for enterprise-scale data migration and analytics
-

RAG Architecture

Excellent question — and you're remembering correctly. In your Daiichi Sankyo project, when you said you built *three iterative RAG architectures*, you likely meant three *versions/configurations* of the Retrieval-Augmented Generation (RAG) pipeline — each optimized for a different constraint or performance goal.

Here's how those **three RAG architectures** can be conceptually and technically described:

🧠 1. RAG v1 — Standard Retrieval + Generation (Baseline)

Goal: Establish a minimal working pipeline for conversational querying of clinical trial tables.

Architecture Components:

- **Retriever:** FAISS or Chroma vector store using *all-MiniLM-L6-v2* embeddings.
- **Generator:** Mistral 7B (via Ollama local deployment).
- **Prompting:** Simple concatenation of top-k retrieved contexts into a question-answer prompt.
- **Flow:**
User Query → Embedding Search → Context Injection → Mistral 7B → Response

Characteristics:

- Straightforward, CPU-friendly baseline.
- No reranking or adaptive context control.
- Served as your initial proof that RAG works locally under 32GB RAM.

Use case: “Fast” configuration — 4.4-minute average response time.

🧩 2. RAG v2 — Multi-Retriever + Reranker (Balanced Architecture)

Goal: Improve relevance and accuracy using multiple retrievers and a reranking stage.

Architecture Components:

- **Retrievers:** Hybrid combination — FAISS (dense) + BM25 or Chroma (sparse).
- **Reranker:** Cross-encoder (e.g., *ms-marco-MiniLM-L-6-v2* or *bge-reranker-base*) to rescore top 20 retrieved chunks.
- **Generator:** Mistral 7B again, but with context window optimization (shorter, higher-relevance text).
- **Flow:**
 Query → Multi-Retriever Fusion → Cross-Encoder Reranking → Context Selection → Mistral 7B → Response

Characteristics:

- Boosts factuality and domain precision for clinical terminology.
- Still CPU-deployable but heavier inference load.
- Demonstrated your “Balanced” configuration — roughly 4.2-minute response time.

Technical insight:

This is where *reranking* came in — improving retrieval precision by ordering chunks based on semantic relevance, not just cosine similarity.

⚙️ 3. RAG v3 — SQL Translation Layer (Proposed “Quality” Architecture)

Goal: Achieve near-instant structured-data answers by bypassing LLM text generation for well-formed SQL queries.

Architecture Components:

- **Retriever:** Hybrid search + schema-aware retriever (metadata + data dictionary vectors).
- **Intermediate Translator:** Lightweight model (Phi-3-mini) converts user query → SQL query (guided by schema).
- **Executor:** Local SQLite/PostgreSQL or mock clinical database executes SQL.
- **Generator:** Mistral 7B or smaller model formats natural-language summary of SQL result.
- **Flow:**
 Query → Retrieve Schema Context → SQL Translation (Phi-3-mini) → SQL

Execution → Natural-Language Summary

Characteristics:

- Massive latency reduction once integrated with real database (seconds instead of minutes).
- Aligns with your identified *production pathway*.
- Demonstrates *compositional RAG*: combining retrieval, translation, and execution layers.

Use case: “Quality” configuration — 9.3-minute generation locally (CPU-bound), but seconds with cloud SQL.

✔ Summary Table

Version	Key Idea	Retriever	Reranking	Generator	Goal / Trade-off
RAG v1 (Fast)	Basic RAG	FAISS/Chroma	None	Mistral 7B	Proof of concept, low resource
RAG v2 (Balanced)	Multi-retriever + Reranker	FAISS + BM25	Cross-encoder	Mistral 7B	Higher accuracy, moderate speed
RAG v3 (Quality)	SQL-Translation RAG	Schema-aware	Optional	Phi-3-mini + Mistral	High accuracy, scalable to production

If you'd like, I can sketch a **one-slide architecture diagram** summarizing all three versions — perfect for interview or presentation use. Would you like that?

WLR

White Label Resell Los Angeles, CA

Founding Machine Learning Engineer 06/2022 - 03/2023

"At White Label Resell, I worked as a Machine Learning Engineer, serving as the sole backend, AI, and cloud developer for their content automation system. The company specializes in AI-powered digital marketing solutions for web agencies and small businesses, offering white-labeled service plans from their headquarters in Los Angeles.

My primary project was developing an end-to-end AI-Driven Content Automation and SEO Optimization System that revolutionized their ability to scale content production. I engineered a comprehensive pipeline capable of generating 10,000+ unique articles per month across 50+ languages using various AI models (GPT-3.5, BERT, FLAN-T5).

The system included several innovative components I built from scratch: an AI-powered internal linking structure, automated image generation using Stable Diffusion, NLP filtering for content quality, and custom linking strategies. I also designed the entire cloud infrastructure using AWS services (Lambda, ECS, S3, API Gateway).

The business impact was substantial - my system directly contributed to securing contracts worth \$50,000-\$100,000 quarterly, boosted organic traffic by 60%, and increased international engagement by 40% through multi-language content capabilities.

White Label Resell leverages this technology as part of their comprehensive digital marketing services, which also include website development, SEO optimization, and analytics tracking for clients like Guild Golf Carts and Annuity Rate Info."

Technical Glossary

White Label: Products or services produced by one company but rebranded and sold by other companies as their own.

NLP (Natural Language Processing): A field of AI that gives computers the ability to understand and process human language.

Internal Linking: The practice of linking from one page to another within the same website, important for SEO and user experience.

Stable Diffusion: An AI model that generates images from text descriptions.

FastAPI: A modern, high-performance web framework for building APIs with Python.

AWS Lambda: A serverless computing service that runs code in response to events without requiring server management.

BERT (Bidirectional Encoder Representations from Transformers): A transformer-based machine learning model for NLP pre-training.

SEO (Search Engine Optimization): The practice of increasing the quantity and quality of traffic to a website through organic search engine results.

Docker: A platform that enables developers to build, package, and distribute applications as containers.

API Gateway: A service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs.

AG Ver 2

Project Information Header

- **Project:** AI-Driven Content Automation & SEO Optimization System
- **Company:** White Label Resell
- **Role:** Machine Learning Engineer (Sole Backend/AI/Cloud Developer)
- **Duration:** [Duration not specified - ongoing project]

Summary STAR (1.5-2.5 minutes speaking time)

S: At White Label Resell my main project was to build this core product - an AI-powered content automation system for digital marketing clients who needed large-scale SEO content generation.

This was the company's main product, and it focused on providing automated content creation with internal linking, images and multi-language support that businesses couldn't get anywhere else at this scale and quality level.

T: As the sole backend/AI/cloud developer, I was responsible for building this entire comprehensive system from scratch ~~—essentially creating the technical foundation that the whole company was based on.~~ I had to build and integrate custom multiple AI models with AWS cloud infrastructure to deliver enterprise-grade content generation capabilities producing 10,000+ articles monthly.

A: I architected the complete system integrating GPT-3.5, BERT, and FLAN-T5 for content generation, Fine TUned Stable Diffusion for 9 images styles, and Google Translate API for 50+ languages, all running on AWS cloud infrastructure. I also developed automated internal linking algorithms and built scalable parallel processing systems to handle the massive volume requirements.

R: The system I built became the backbone of our business - we were generating 10,000+ monthly articles, secured \$50-100K quarterly contracts, achieved a 60% increase in organic traffic through the AI-powered internal linking, and boosted international engagement by 40% with our multi-language content capabilities.

Detailed STAR

S - SITUATION Situation Summary: White Label Resell needed to revolutionize content generation for digital marketing clients who were facing slow, expensive manual processes that couldn't scale for competitive e-commerce needs, requiring large-scale SEO-optimized content with advanced features like internal linking and multi-language support.

Detailed Context:

- **Company context & business problem:** White Label Resell needed to revolutionize content generation for digital marketing clients facing slow, expensive, and inconsistent manual content

creation. Businesses required large-scale, SEO-optimized content with effective internal linking, multi-language localization, and automated scalability to compete in the fast-paced e-commerce landscape.

- **Team structure & your role:** Small team with frontend developers and business stakeholders. As the sole Backend/AI/Cloud Developer, I was entirely responsible for designing and implementing the complete technical infrastructure including AI models, cloud architecture, and all backend systems.
- **Timeline & constraints:** Aggressive scaling requirements needing 10,000+ articles monthly capability. Multi-language support across 50+ languages required. High-performance expectations with enterprise-level reliability and SEO optimization standards.
- **Scale/scope of the challenge:** Building enterprise-scale system as sole developer, 10,000+ monthly articles, 50+ language support, complete technical ownership

T - TASK/CHALLENGE Task Summary: As the only technical developer, I was tasked with single-handedly building a comprehensive AI-powered content automation system from scratch, including all AI models, cloud infrastructure, API development, and deployment architecture.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Design and implement entire end-to-end content automation pipeline as sole technical developer
 - Build AI-driven content generation using multiple NLP models (GPT-3.5, BERT, FLAN-T5)
 - Develop multi-language content generation via Google Translate API integration
 - Create AI-powered image generation with Stable Diffusion and nine aesthetic styles
 - Architect complete AWS cloud infrastructure (API Gateway, ECS, Lambda, S3, SageMaker)
 - Implement automated internal linking system with custom SEO strategies
 - Build scalable API systems supporting bulk content generation
- **Technical challenges faced:**
 - Single-handedly architecting enterprise-scale system handling 10,000+ articles monthly
 - Integrating multiple AI models (text, image, translation) in cohesive automated pipeline
 - Implementing dynamic internal linking algorithms for improved search engine rankings
 - Building cost-effective cloud infrastructure supporting massive content generation volumes
- **Business/stakeholder challenges:**
 - Delivering complete technical solution as solo developer with no technical team support
 - Balancing system complexity with rapid development timeline constraints
 - Ensuring enterprise-grade reliability and performance with limited resources
 - Managing client requirements and preventing scope creep through technical documentation

- **Success criteria and constraints:** 10,000+ monthly articles, 50+ language support, enterprise reliability, solo development, cost-effective scaling

A - ACTION (Detailed Breakdown) Action Summary: I architected and built a complete AI automation ecosystem single-handedly, integrating multiple state-of-the-art models with sophisticated cloud infrastructure, achieving enterprise-scale content generation capabilities that secured significant quarterly contracts.

Technical Stack Used:

- **AI Models:** GPT-3.5, BERT, FLAN-T5 (content generation), Stable Diffusion (image generation), Google Translate API (multi-language support)
- **Cloud Infrastructure:** AWS API Gateway, ECS, Lambda, SQS, SNS, EC2, S3, SageMaker (complete serverless architecture for scalability)
- **Backend Development:** Flask, FastAPI (high-performance API development for bulk processing)
- **Database & Storage:** SQL, SQLite, DynamoDB, S3 (structured data management and content storage)
- **Deployment:** Docker containerization (efficient cloud deployment and scaling)

Collaboration Details:

- **Worked with:** Frontend developers (API integration), business stakeholders (requirements), content strategists (SEO optimization)
- **How decisions were made:** Single-handedly made all technical architecture decisions including cloud services, AI model selection, and system design
- **Communication challenges/successes:** Created comprehensive Software Requirement Specification (SRS) documenting technical capabilities and limitations
- **Cross-functional coordination:** Balanced technical implementation with business requirements while managing all backend development independently

Leadership Moments:

- **Decisions you drove:** Solo Technical Leadership taking complete ownership of technical architecture decisions as sole developer
- **People you influenced/mentored:** Innovation driver designing novel graph-based AI linking models for dynamic internal linking optimization
- **Processes you improved:** Established multi-step AI refinement pipeline ensuring quality at massive scale
- **Initiative you took beyond assigned tasks:** Created comprehensive documentation and quality assurance systems beyond basic requirements

Problem-Solving Process:

- **Scale vs Quality:** Implemented multi-step AI refinement and NLP filtering maintaining content quality across 10,000+ monthly articles

- **Integration Complexity:** Single-handedly integrated text generation, image creation, translation, and SEO optimization in unified pipeline
- **Cost Optimization:** Architected serverless AWS infrastructure minimizing operational costs while supporting enterprise volumes
- **Alternative Solutions Considered:** Evaluated monolithic vs. microservices, various AI model combinations, different cloud providers

R - RESULTS Result Summary: As sole technical developer, I delivered an enterprise-grade content automation system generating 10,000+ monthly articles, securing \$50-100K quarterly contracts, and achieving significant improvements in organic traffic and international engagement.

Detailed Impact:

- **Quantified business impact:**
 - **10,000+ articles/month** automated generation capability built by single developer
 - **\$50-100K quarterly contracts** secured directly through superior technical capabilities
 - **60% increase** in organic traffic through AI-powered internal linking algorithms
 - **40% increase** in international engagement via multi-language content generation
- **Technical achievements:**
 - **Complete system architecture:** Single-handedly built entire AI automation pipeline from AI models to cloud deployment
 - **Multi-language scaling:** Successfully implemented Google Translate API supporting 50+ languages
 - **Advanced AI integration:** Combined GPT-3.5, BERT, FLAN-T5, and Stable Diffusion in unified content generation system
 - **Enterprise cloud infrastructure:** Architected scalable AWS ecosystem supporting massive content volumes
- **Team/process improvements:**
 - **Solo development excellence:** Demonstrated ability to deliver enterprise-scale solutions as individual contributor
 - **Technical documentation standards:** Created comprehensive SRS preventing scope creep and ensuring system maintainability
 - **Automated quality assurance:** Implemented plagiarism checks and grammar validation ensuring content credibility
 - **Scalable architecture patterns:** Established cloud-native design principles for future AI-intensive projects
- **Personal growth/skills gained:**
 - **Full-stack AI system architecture:** Complete expertise in designing and implementing enterprise AI solutions independently
 - **Cloud infrastructure mastery:** Advanced skills in AWS services orchestration for AI-intensive applications

- **Solo project management:** Proven ability to deliver complex technical solutions with complete ownership and accountability
- **AI model integration expertise:** Deep understanding of combining multiple AI technologies in production-ready systems

Article Generation

AI-Driven Content Automation & SEO Optimization System

Company: White Label Resell

Role: Machine Learning Engineer

Duration: Not specified

S - SITUATION

Company Context & Business Problem: White Label Resell faced the challenge of scaling content generation for digital marketing and SEO optimization. Manual content creation was slow, expensive, and inconsistent, preventing clients from achieving large-scale, high-quality content that was SEO-optimized, effectively interlinked, localized for global markets, and scalable through automation.

Team Structure & Your Role: Content automation team focused on AI-driven marketing solutions. As Machine Learning Engineer, I was responsible for designing and implementing the complete AI-powered content automation pipeline from content strategy to deployment and optimization.

Timeline & Constraints: Fast-paced digital marketing environment requiring rapid scaling capabilities. Need to support 10,000+ articles monthly with multi-language capabilities across 50+ languages while maintaining SEO standards and content quality.

T - TASK/CHALLENGE

Summary: I was tasked with building a comprehensive AI-powered content automation system that could generate thousands of unique, high-quality articles with automatic internal linking, multi-language support, and automated content moderation while ensuring SEO optimization.

Your Specific Responsibilities:

- Design end-to-end content automation pipeline from strategy to publishing
- Implement AI-driven content generation using multiple NLP models (GPT-3.5, BERT, FLAN-T5)
- Build automated internal linking system with custom linking strategies
- Develop AI-powered image generation with fine-tuned Stable Diffusion models
- Create multi-language content generation supporting 50+ languages
- Implement SEO optimization with NLP filtering and phrase blocking capabilities
- Deploy scalable cloud infrastructure supporting bulk content generation

Technical Challenges Faced:

- Scaling AI-generated content without losing quality across thousands of articles
- Ensuring unique content at scale while maintaining SEO standards across multiple languages
- Implementing dynamic internal linking for improved search engine rankings
- Integrating multiple AI models (text generation, image generation, NLP filtering) in cohesive pipeline

Business/Stakeholder Challenges:

- Meeting diverse client requirements for content volume, quality, and compliance
- Balancing automation efficiency with content credibility and originality
- Ensuring system scalability to handle enterprise-level content demands
- Managing complex feature requirements and preventing scope creep

A - ACTION (Detailed Breakdown)

Summary: I built a comprehensive AI automation pipeline using multiple state-of-the-art models, implemented sophisticated SEO optimization strategies, and deployed scalable cloud infrastructure that automated content creation from strategy formation to final publishing.

Technical Stack Used:

- **AI Models:** GPT-3.5, BERT, FLAN-T5 (chosen for diverse content generation capabilities and quality)
- **Image Generation:** Fine-tuned Stable Diffusion, OpenCV, CLIP (9 aesthetic styles with automated metadata tagging)
- **Backend:** Flask, FastAPI (rapid API development and scalable content serving)
- **Cloud Infrastructure:** AWS API Gateway, ECS, Lambda, S3, CloudWatch (dockerized deployment for efficiency)
- **Database:** SQL, SQLite (content metadata and linking relationship storage)
- **NLP Processing:** Custom filtering and phrase blocking systems (content quality assurance)

Collaboration Details:

- **Worked with:** Content strategists (SEO requirements), frontend developers (UI integration), DevOps (cloud deployment)
- **Decision Making:** Created Software Requirement Specification (SRS) to finalize scope and limit changes
- **Communication:** Coordinated multi-step AI refinement processes with content quality stakeholders

Leadership Moments:

- **Drove Decision:** Implemented multi-cluster content strategy (Multi-Cluster, Single Cluster, Local Cluster) for diverse client needs

- **Technical Innovation:** Developed graph-based AI linking models for dynamic internal linking optimization
- **Process Improvement:** Created structured pipeline balancing quality, scalability, and SEO impact through automated refinement

Problem-Solving Process:

- **Quality vs Scale:** Implemented multi-step AI refinement and NLP filtering ensuring content quality at massive scale
- **Content Uniqueness:** Integrated plagiarism checks and paraphrasing AI preventing duplicate content issues
- **SEO Compliance:** Optimized content using regional keyword research for multi-language SEO standards
- **Alternative Solutions Considered:** Single model vs. multi-model approach, synchronous vs. asynchronous processing, cloud vs. hybrid deployment

R - RESULTS

Summary: The AI automation system achieved exceptional scale generating 10,000+ articles monthly, secured significant quarterly contracts worth \$50-100K, and delivered measurable improvements in organic traffic and international engagement while establishing new standards for automated content marketing.

Quantified Business Impact:

- **10,000+ articles/month** automated bulk content generation capability
- **\$50-100K quarterly contracts** secured through superior automation capabilities
- **60% increase** in organic traffic through AI-powered internal linking optimization
- **40% increase** in international engagement via multi-language content generation

Technical Achievements:

- **Multi-language support:** Successfully implemented content generation across 50+ languages
- **AI-powered image integration:** Built 9 aesthetic styles with automated metadata tagging for SEO
- **Dynamic linking system:** Developed graph-based models improving search engine crawlability
- **End-to-end automation:** Created complete pipeline from content strategy to cloud deployment

Team/Process Improvements:

- **Scalable architecture standards:** Established dockerized deployment patterns for AI-intensive applications
- **Content quality assurance:** Implemented automated plagiarism checks and grammar validation ensuring credibility

- **SEO optimization framework:** Created systematic approach for keyword optimization and internal linking
- **Requirements management:** Developed SRS documentation preventing scope creep in complex AI projects

Personal Growth/Skills Gained:

- **Multi-model AI integration:** Expertise combining text generation, image generation, and NLP filtering systems
- **Large-scale content automation:** Advanced understanding of scaling AI content generation maintaining quality
- **SEO and content strategy:** Deep knowledge of AI-driven keyword optimization and linking strategies
- **Cloud-native AI deployment:** Comprehensive experience with AWS infrastructure for AI-intensive applications

SCAI

Chrome Extension

SEO Content AI Los Angeles, CA

Founding AI Infrastructure Engineer 11/2022 - 07/2023

Interview Points About SEOContent.ai

I worked as an AI Infrastructure Engineer at SEOContent.ai, a company that specializes in AI-driven content automation for SEO and digital marketing purposes. They offer tools that help businesses create high-quality, SEO-optimized content at scale, including blog posts, product descriptions, metadata, and more. Their platform uses large language models and NLP to automate content creation processes that traditionally required significant manual effort.

During my time there, I contributed to multiple key projects, including the "Skyscraper" platform, which automated multi-article campaign creation, and the development of their API infrastructure. I designed and implemented microservices that powered keyword extraction, topic clustering, and content generation workflows. My technical focus was on creating scalable AI systems that maintained content quality while significantly reducing production time. I worked extensively with AWS cloud infrastructure, containerized microservices, and various AI models including FLAN-T5, GPT-3.5, and Vicuna-13B, GPT-NEOX.

.

My work contributed to measurable business outcomes, including a 60-70% reduction in content creation time for clients, a 40% improvement in content production efficiency, and increased organic traffic by up to 300% for client websites. I also helped develop free AI tools that grew the company's organic traffic from 50,000 to 75,000 monthly visits. The systems I built were designed for elasticity and cost-effectiveness, balancing performance with operational expenses.

Technical Glossary

1. **Microservice Architecture:** A design approach where an application is built as a collection of small, independently deployable services, which I used to create modular, scalable content generation systems.
2. **Prompt Engineering:** The technique of crafting effective inputs for language models to produce desired outputs, which was critical for ensuring our AI generated high-quality, SEO-optimized content.
3. **Container Orchestration:** The automated configuration, coordination, and management of containers (like Docker), which I implemented using AWS ECS to ensure elastic scaling

of our services.

4. **Topic Clustering:** The process of organizing keywords into related groups to create content silos, which I implemented using custom algorithms to improve SEO strategy.
5. **NLP (Natural Language Processing):** A field of AI focused on enabling computers to understand and generate human language, which was the foundation of our content generation systems.
6. **Horizontal Scaling:** Adding more machines to a system to handle increased load, which I implemented to balance performance and cost in our content generation pipeline.
7. **API Gateway:** A service that creates, publishes, and manages APIs, which I used to expose our content generation services to clients and integrate with various platforms.
8. **Fine-tuning:** The process of further training an existing language model on domain-specific data to improve performance for particular tasks, which I implemented to enhance content quality.
9. **Token Optimization:** Techniques to reduce the number of tokens used in AI model interactions to minimize costs while maintaining output quality, which was essential for our free tools.
10. **Serverless Architecture:** A cloud computing execution model where the cloud provider manages server allocation, which I used with AWS Lambda for cost-effective API implementation.

AI Tools

Project Information Header

- **Project:** Zero-Maintenance AI Tools Platform with Dynamic Configuration System
- **Company:** SEOContent.ai
- **Role:** AI Infrastructure Engineer
- **Duration:** 3 Weeks

Summary STAR (1.5-2.5 minutes speaking time)

S: The challenge we faced at SEOContent.ai was that we wanted to build a platform with 50+ SEO-specific AI tools, but we recognized from past experience that having developers maintain every tool update would create significant bottlenecks. Our content experts would constantly need technical assistance for simple changes, which wasn't sustainable.

T: My responsibility was to fundamentally redesign this approach - I needed to build a platform where content experts could independently manage, update, and deploy tools without any technical knowledge, while maintaining cost efficiency throughout the process.

A: I developed a hybrid system using Google Sheets as the interface - since it's universally familiar - connected to AWS serverless infrastructure via GCP. The frontend dynamically reads JSON configurations, so when someone updates a spreadsheet, the tool changes instantly without requiring any code deployment. We delivered the complete solution in just 3 weeks.

R: The platform exceeded our expectations significantly - we achieved 5,000 users in the first month, reduced our operational costs by 98.4% compared to traditional no-code platforms, and tool updates now take under 90 seconds instead of requiring developer intervention. This approach fundamentally transformed our operational efficiency.

Detailed STAR

S - SITUATION Situation Summary: SEOContent.ai needed to build a comprehensive AI tools platform with 50+ tools but faced a critical strategic challenge where traditional approaches would create unsustainable developer dependencies, limiting business agility and creating operational bottlenecks for tool maintenance.

Detailed Context:

- **Company context & business problem:** SEOContent.ai needed to build a comprehensive AI tools platform with 50+ tools but faced a critical strategic challenge: traditional approaches would create unsustainable developer dependencies for tool maintenance and updates. The company

anticipated that content experts would identify improvements but would depend on developers to implement them, creating operational bottlenecks and limiting business agility.

- **Team structure & your role:** Small startup team with content experts, frontend developers, and documentation specialists. As the sole AI Infrastructure Engineer, I was responsible for architecting and building the entire platform from scratch, specifically designing for zero ongoing developer maintenance.
- **Timeline & constraints:** Aggressive 3-week timeline to deliver production-ready platform. Need to support immediate scaling to 50+ tools with potential expansion to 100+. Critical requirement: system must operate independently without developer intervention for tool modifications and updates.
- **Scale/scope of the challenge:** Building 50+ tool platform from scratch, eliminating developer dependencies entirely, achieving dramatic cost reduction, delivering in 3 weeks

T - TASK/CHALLENGE Task Summary: I was tasked with architecting a revolutionary AI tools platform that could eliminate developer dependencies entirely, enabling content experts to manage, modify, and deploy tools independently while maintaining enterprise-grade performance and cost efficiency.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Design architecture eliminating developer maintenance requirements for tool lifecycle management
 - Build dynamic frontend system supporting unlimited tool configurations without code changes
 - Implement real-time deployment pipeline enabling instant tool updates and modifications
 - Create cost-effective infrastructure supporting 50+ tools with linear scaling capabilities
 - Establish user-friendly interface requiring zero technical training for content experts
- **Technical challenges faced:**
 - Designing dynamic frontend rendering system supporting unlimited tool variations
 - Implementing real-time configuration deployment without application restarts
 - Building cost-effective architecture competing with \$2000+/month no-code alternatives
 - Creating validation and error handling for non-technical users managing complex configurations
- **Business/stakeholder challenges:**
 - Rapid 3-week delivery timeline requiring immediate architecture decisions
 - Balancing technical sophistication with user simplicity for content experts
 - Achieving dramatic cost reduction while maintaining enterprise-grade functionality
 - Ensuring sustainable competitive advantages through operational independence
- **Success criteria and constraints:** Zero developer maintenance, 3-week delivery, 50+ tools support, massive cost reduction, non-technical user operation

A - ACTION (Detailed Breakdown) Action Summary: I architected an innovative hybrid system combining Google Sheets for familiar user interface with AWS serverless infrastructure, implementing dynamic configuration rendering that eliminated developer dependencies while achieving 98.4% cost reduction compared to traditional alternatives.

Technical Stack Used:

- **Google Sheets + Apps Script:** Content management and change detection (chosen for zero learning curve and built-in collaboration)
- **AWS Lambda:** Configuration processing and deployment automation (serverless pay-per-execution model for cost efficiency)
- **Amazon S3 + CloudFront:** JSON configuration storage and global CDN distribution (instant updates without app deployments)
- **Dynamic JavaScript Frontend:** Runtime configuration rendering (supporting unlimited tool variations without code changes)
- **DynamoDB:** Metadata tracking and analytics (cost-effective NoSQL for tool usage metrics)

Collaboration Details:

- **Worked with:** Content experts (tool requirements), frontend developers (integration), documentation team (user guides)
- **How decisions were made:** Led comprehensive architecture evaluation comparing 4 different approaches considering cost, complexity, and scalability
- **Communication challenges/successes:** Facilitated strategic planning sessions anticipating operational challenges before implementation
- **Cross-functional coordination:** Balanced technical architecture decisions with business operational requirements and user experience needs

Leadership Moments:

- **Decisions you drove:** Chose unconventional Google Sheets approach over traditional CMS solutions, resulting in 98.4% cost reduction
- **People you influenced/mentored:** Strategic innovation guiding team toward configuration-driven architecture enabling business user autonomy
- **Processes you improved:** Established real-time deployment pipeline allowing tool updates in under 90 seconds
- **Initiative you took beyond assigned tasks:** Designed configuration-driven architecture enabling 50x faster iteration cycles than traditional development

Problem-Solving Process:

- **Architecture Evaluation:** Systematically compared Traditional CMS (\$500/month), Custom Admin (\$200/month), No-Code Platform (\$2000/month) vs. Hybrid Solution (\$50/month)
- **Dynamic Rendering Challenge:** Built frontend system reading JSON configurations at runtime, supporting unlimited field types and variations

- **Cost Optimization:** Implemented serverless architecture with intelligent caching reducing operational costs by 98.4%
- **Alternative Solutions Considered:** Evaluated WordPress/Drupal, React admin panels, Zapier/Bubble platforms before choosing hybrid approach

R - RESULTS Result Summary: The platform achieved exceptional market success with 5,000 users in the first month, 1,600 recurring users, and 98.4% cost reduction while establishing sustainable competitive advantages through complete operational independence and 50x faster iteration cycles.

Detailed Impact:

- **Quantified business impact:**
 - **5,000 users** in first month exceeding all initial adoption projections
 - **1,600 recurring users** in month 2 (32% retention rate) demonstrating strong product-market fit
 - **98.4% cost reduction** (\$50/month vs. \$2000+/month no-code alternatives)
 - **50x faster** tool improvement cycles enabling same-day market responsiveness
- **Technical achievements:**
 - **Real-time deployment:** Tool updates deployed in under 90 seconds without application restarts
 - **Zero developer maintenance:** 100% content expert autonomy for tool lifecycle management
 - **Dynamic configuration system:** Supporting unlimited tool variations through runtime JSON rendering
 - **Enterprise-grade performance:** 99.95% uptime with <800ms frontend load times via CDN optimization
- **Team/process improvements:**
 - **Operational independence:** Eliminated entire category of developer maintenance overhead
 - **Business agility:** Enabled same-day implementation of user feedback and market changes
 - **Cost efficiency standards:** Established architecture patterns achieving 98%+ cost savings for future projects
 - **Developer focus:** Freed 95% of development time for core platform innovation vs. maintenance
- **Personal growth/skills gained:**
 - **Strategic architecture evaluation:** Expertise in comparing technical approaches considering long-term operational costs
 - **Configuration-driven design:** Advanced skills in building systems enabling business user autonomy

- **Cost optimization mastery:** Deep understanding of serverless architecture for dramatic cost reduction
- **Market validation:** Experience building and launching products achieving immediate product-market fit

API Solution

<https://seocontent.ai/authentication/>

Project Information Header

- **Project:** AI-Powered Content Generation API for SEO Marketing
- **Company:** SEOContent.ai
- **Role:** AI Infrastructure Engineer
- **Duration:** 3 Weeks

Summary STAR (1.5-2.5 minutes speaking time)

S: So we had this content generation system at SEOContent.ai that was working great internally, but our customers kept asking if they could integrate it directly into their own systems instead of using our interface. We realized there was a real opportunity here to create an API offering that would let clients plug our AI content generation capabilities straight into their workflows.

T: I was tasked with architecting a complete API system using AWS API Gateway that would securely expose our existing AI system - we're talking GPT-3.5, BERT, FLAN-T5, Stable Diffusion - while implementing enterprise authentication and billing to handle thousands of concurrent client requests. And I had just 3 weeks to get this done.

A: I built a comprehensive REST API using AWS API Gateway with Lambda functions, integrated it with our existing White Label Resell infrastructure, implemented AWS Cognito plus Stripe for authentication and billing, and **optimized performance through asynchronous processing patterns** to make everything work smoothly.

R: The results were incredible - I delivered a production-ready API that not only reduced our original content creation time by about 30%, going from 45 Seconds down to 30 seconds. We started processing 10,000+ monthly requests with 99.9% uptime, ~~secured some significant quarterly contracts, and achieved 90% successful client integrations.~~

Detailed STAR

S - SITUATION

Situation Summary: SEOContent.ai needed to quickly commercialize the existing White Label Resell content generation system by building a market-ready API that would allow external clients to access our proven AI capabilities through a scalable, secure interface.

Detailed Context:

- **Company context & business problem:** SEOContent.ai, a subsidiary of White Label Resell, needed to monetize our existing AI-powered content automation system by creating an external API product. Digital marketing clients were struggling with slow, expensive manual content creation and needed access to our proven system that could generate 10,000+ articles monthly with multi-language support.
- **Team structure & your role:** Small startup team with frontend developers and business stakeholders. As the AI Infrastructure Engineer, I was responsible for building the complete API layer that would expose White Label Resell's existing AI capabilities to external clients.
- **Timeline & constraints:** Aggressive 3-week timeline to deliver production-ready API. Existing White Label Resell system provided the AI foundation, but required complete API architecture design and secure external access implementation.
- **Scale/scope of the challenge:** Building enterprise-grade API gateway, implementing secure authentication and billing, integrating with existing complex AI pipeline

T - TASK/CHALLENGE

Task Summary: I was tasked with building a complete API system using AWS API Gateway that would securely expose our existing White Label Resell AI capabilities, implement enterprise authentication and billing, and handle thousands of concurrent client requests.

Detailed Responsibilities:

- **API Architecture & Development:** Design and implement REST API using AWS API Gateway with proper versioning, rate limiting, and comprehensive documentation
- **System Integration:** Connect new API layer with existing White Label Resell AI infrastructure (GPT-3.5, BERT, FLAN-T5, Stable Diffusion, Google Translate)
- **Security & Authentication:** Implement secure API key management using AWS Cognito with Stripe integration for usage-based billing
- **Performance Optimization:** Ensure API can handle thousands of concurrent requests while leveraging existing AI pipeline efficiently
- **Client Onboarding:** Create comprehensive API documentation, SDKs, and integration guides for seamless client adoption

Technical challenges faced:

- **Legacy Integration:** Connecting new API Gateway with existing White Label Resell architecture without disrupting internal operations
- **Performance Optimization:** Reducing response times from existing 2.5-minute internal process to target 45-second API response

- **Request Management:** Implementing intelligent queuing and load balancing for high-volume API requests
- **Multi-Model Coordination:** Orchestrating complex AI pipeline (text generation, image creation, translation) through simple API endpoints

Business/stakeholder challenges:

- **Rapid Timeline:** Delivering complete commercial API product in only 3 weeks
- **Revenue Model:** Creating pricing tiers and usage tracking that reflects infrastructure costs and profit margins
- **Client Success:** Ensuring smooth onboarding experience for external clients transitioning from manual processes

A - ACTION (*Detailed Breakdown*)

Action Summary: I architected a comprehensive API system using AWS API Gateway that seamlessly integrated with our existing White Label Resell infrastructure, implementing intelligent caching and optimization strategies that dramatically improved response times while maintaining enterprise-grade reliability.

Technical Stack Used:

- **AWS API Gateway:** Primary API management platform (chosen for built-in rate limiting, caching, authentication, and monitoring)
- **AWS Lambda:** Serverless functions for API business logic and White Label Resell system integration
- **Integration Layer:** Custom Python middleware connecting to existing White Label Resell AI pipeline (GPT-3.5, BERT, FLAN-T5, Stable Diffusion, Google Translate)
- **AWS Cognito + Stripe:** API key management and credits-based billing system
- **CloudWatch & SNS:** Comprehensive API monitoring, logging, and alerting

Collaboration Details:

- **White Label Resell Team:** Coordinated with original system architects to understand existing AI pipeline and integration points
- **Frontend Developers:** Provided API specifications, SDKs, and authentication integration guidance
- **Business Team:** Collaborated on pricing strategy, usage limits, and billing implementation requirements
- **Documentation Team:** Created comprehensive API guides, code examples, and troubleshooting resources

Leadership Moments:

- **API Design Strategy:** Led architectural decisions on REST endpoint design, choosing simplicity for broad client compatibility

- **Integration Approach:** Drove decision to build API layer on top of existing White Label Resell system rather than rebuilding AI capabilities
- **Performance Standards:** Established API response time SLAs and comprehensive monitoring protocols
- **Client Success Framework:** Created technical onboarding process and support documentation for API adopters

Problem-Solving Process:

- **Latency Optimization:** Implemented API Gateway caching and Lambda warming strategies, reducing cold starts and improving response times by 85%
- **System Integration:** Developed asynchronous processing patterns to leverage existing White Label Resell AI pipeline without blocking API responses
- **Load Management:** Built intelligent queuing system handling burst traffic while maintaining fair usage across clients
- **Error Handling:** Created comprehensive error response system with proper HTTP status codes and detailed debugging information

R - RESULTS

Result Summary: The API successfully transformed our White Label Resell capabilities into a commercial product, achieving exceptional performance with 85% faster content generation, serving 10,000+ monthly requests, and enabling seamless client integration while maintaining the reliability of our existing AI infrastructure.

Detailed Impact: Quantified business impact:

- **API Performance:** Reduced average response time from 2.5 minutes to 45 seconds (85% improvement)
- **Commercial Success:** Enabled processing of 50,000+ API requests monthly with 99.9% uptime
- **Client Results:** API users achieved 300% increase in organic traffic through automated, SEO-optimized content
- **Revenue Generation:** Secured significant quarterly contracts through superior API capabilities and performance

Technical achievements:

- **Scalable API Architecture:** Successfully handled thousands of concurrent requests through AWS API Gateway auto-scaling
- **Seamless Integration:** Zero-downtime integration with existing White Label Resell AI infrastructure
- **Enterprise Security:** Implemented robust authentication, rate limiting, and comprehensive usage monitoring
- **Developer Experience:** Created intuitive API with comprehensive documentation resulting in 90% successful client integrations

Team/process improvements:

- **Reusable Architecture:** Established API development patterns and standards for future product development
- **Monitoring Excellence:** Implemented comprehensive API analytics and performance dashboards
- **Client Support Process:** Created streamlined technical onboarding reducing support requirements by 70%
- **Integration Framework:** Developed reusable patterns for exposing internal AI capabilities as commercial APIs

Personal growth/skills gained:

- **API Product Development:** End-to-end experience building commercial APIs from existing AI systems using AWS API Gateway
- **Rapid Development:** Proven ability to deliver enterprise-grade API products under aggressive 3-week timeline
- **System Integration Mastery:** Advanced skills in connecting API layers with complex existing AI infrastructure
- **Commercial Product Strategy:** Experience translating internal AI capabilities into market-ready products with proper pricing and scaling

API Docs

Complete Guide to SEOContent.ai API: Automated Content Generation at Scale

SEOContent.ai provides a powerful API that enables developers to integrate AI-powered content generation directly into their applications. Whether you're building content management systems, marketing automation tools, or enterprise-scale content platforms, our API delivers SEO-optimized articles with advanced features like internal linking, multi-language support, and automated image generation.

Getting Started with Authentication

Before you can start generating content, you'll need to set up authentication with our API.

Creating Your API Key

1. **Create an account** on the SEOContent.ai platform
2. Navigate to **Account Settings**
3. Click on **'Create a New API Key'** button
4. Generate your unique API key

Important Security Note: Keep your API key secret. Never share it with others or expose it in client-side code. If you suspect your key has been compromised, immediately create a new one and delete the old key from your account settings.

Authentication Headers

All API requests require these headers:

Content-Type: application/json
x-api-key: your_api_key

Credits and Billing System

Understanding Credits

SEOContent.ai operates on a credit-based system where credits act as virtual currency for content generation. Credit consumption varies based on the computational resources required for each request.

Free Credits for New Users

- **25 free credits** upon account creation
- **Additional 25 credits** (equivalent to 25,000 words) available through user verification
- Perfect for testing and initial development

Purchasing Additional Credits

Once free credits are exhausted, you can browse our range of pricing plans, each offering specific credit amounts and features tailored to different content generation needs.

API Endpoints Overview

Our API provides several endpoints for different content generation strategies:

1. Status Checker

Monitor the progress of your content generation projects with real-time status updates.

Endpoint: `POST /v1/status-checker/`

```
import requests
```

```
api_host = "https://api.seocontent.ai"
```

```
api_key = "your_api_key"
```

```
project_title = "api_lc01"
```

```
data = {  
    "project_title": project_title,  
}
```

```
headers = {  
    "Content-Type": "application/json",  
    "x-api-key": api_key,  
}
```

```
response = requests.post(  
    f"{api_host}/v1/status-checker/",  
    json=data,  
    headers=headers  
)  
print("Response:", response.text)  
print(response.status_code)
```

Parameters:

- `project_title` (string): The title of the project you want to check

Response: Returns detailed information about your project's current status and processing stage.

2. Single Cluster Content Generation

Generate focused content around closely related keywords for targeted SEO optimization.

Endpoint: `POST /v1/content-generator/`

```
import requests
```

```
api_host = "https://api.seocontent.ai"
```

```
api_key = "your_api_key"
```

```
project_title = "api_sc02"
```

```
keywords = [
```

```
    [
        "what is assisted living?",
        "what is assisted living mean?",
        "what exactly is assisted living?",
    ],
```

```
    [
        "what is level 1 care in assisted living?",
        "what is level 2 care in assisted living?",
    ],
]
```

```
content_type = "single_cluster"
```

```
interlinking = True
```

```
img_generation = "None"
```

```
counts = {
```

```
    "primary_count": 1000,
    "secondary_count": 600,
```

```
}
```

```
additional = {
```

```
    "seo_title": True,
    "seo_meta": True,
```

```

"seo_faq": False,
"seo_tagline": False,
"seo_contact": False,
"negative_words": ["facility"],
"phrase_blocking": ["Facilities", "Rent", "Pays", "Religion", "Health conditions"],
"tone": "Informative",
"featured_image": True,
"h2_images": True,
}

```

```

content_language = "English"

```

```

data = {
    "keywords": keywords,
    "project_title": project_title,
    "counts": counts,
    "additional": additional,
    "interlinking": interlinking,
    "content_type": content_type,
    "content_language": content_language,
    "img_generation": img_generation,
}

```

```

if interlinking:
    data["interlinking_type"] = "subtopic_linking"

```

```

headers = {
    "Content-Type": "application/json",
    "x-api-key": api_key,
}

```

```

response = requests.post(
    f"{api_host}/v1/content-generator/",
    json=data,
    headers=headers
)
print("Response:", response.text)
print(response.status_code)

```

3. Multi Cluster Content Generation

Create comprehensive content ecosystems with advanced internal linking across multiple topic clusters.

Endpoint: `POST /v1/content-generator/`

```
import requests
```

```
api_host = "https://api.seocontent.ai"
```

```
api_key = "your_api_key"
```

```
project_title = "api_mc3"
```

```
counts = {  
    "primary_count": 1000,  
    "secondary_count": 1000,  
}
```

```
additional = {  
    "seo_title": True,  
    "seo_meta": True,  
    "seo_faq": True,  
    "seo_tagline": True,  
    "seo_contact": True,  
    "negative_words": ["facility"],  
    "find_and_replace": {"facility": "services"},  
    "phrase_blocking": [  
        "Facilities",  
        "Rent",  
        "Pays",  
        "Religion",  
        "Health conditions",  
    ],  
    "tone": "Default",  
    "featured_image": True,  
    "h2_images": True,  
}
```

```
img_generation = "Realistic Vision v2.0"
```

```
interlinking = True
```

```
content_language = "English"
```

```
content_type = "multi_cluster"
```

```
keywords = [  
    "what is assisted living",  
    "how much does assisted living cost",  
]
```



```

data = {
    "keywords": keywords,
    "project_title": project_title,
    "counts": counts,
    "additional": additional,
    "content_type": content_type,
    "content_language": content_language,
    "img_generation": img_generation,
    "interlinking": interlinking,
    "interlinking_type": "infinte_interlinking",
    "clustersize": 3,
}

headers = {
    "Content-Type": "application/json",
    "x-api-key": api_key,
}

response = requests.post(
    f"{api_host}/v1/content-generator/",
    json=data,
    headers=headers
)
print("Response:", response.text)
print(response.status_code)

```

4. Local Cluster Content Generation

Generate geographically-focused content with location-specific optimization.

Endpoint: `POST /v1/content-generator/`

This endpoint uses similar parameters to multi-cluster generation but optimizes content for local SEO and geographic targeting.

Parameter Reference Guide

Core Parameters

project_title (string)

- Unique identifier for your content generation project
- Example: "api_mc5"

content_type (string)

- Options: "single_cluster", "multi_cluster", "local_cluster"
- Determines the content generation strategy

keywords (array)

- For single cluster: Array of arrays containing related keyword groups
- For multi cluster: Simple array of primary keywords
- Example: ["what is assisted living", "assisted living costs"]

Content Configuration

counts (object)

- **primary_count**: Target word count for main content (integer)
- **secondary_count**: Target word count for supporting content (integer)

content_language (string)

- Language for content generation
- Example: "English"

SEO Enhancement Options

additional (object) - Comprehensive SEO configuration:

- **seo_title**: (boolean) Generate SEO-optimized titles
- **seo_meta**: (boolean) Create meta descriptions
- **seo_faq**: (boolean) Include FAQ sections
- **seo_tagline**: (boolean) Add compelling taglines
- **seo_contact**: (boolean) Include contact information
- **negative_words**: (array) Words to avoid in content
- **find_and_replace**: (object) Text replacement rules
- **phrase_blocking**: (array) Phrases to block from content
- **tone**: (string) Content tone ("Default", "Informative", etc.)
- **featured_image**: (boolean) Include featured images
- **h2_images**: (boolean) Add images to H2 sections

Advanced Features

interlinking (boolean)

- Enable sophisticated internal linking between articles
- Options when enabled:
 - `interlinking_type`: "subtopic_linking" or "infinte_interlinking"

`img_generation` (string)

- Image generation options: "None", "Realistic Vision v2.0"
- AI-powered image creation for articles

`clustersize` (integer)

- For multi-cluster content: Number of articles per cluster
- Example: 3

Best Practices

Content Strategy

1. **Start with Single Cluster** for focused topics with high keyword relevance
2. **Use Multi Cluster** for comprehensive content ecosystems with internal linking
3. **Apply Local Cluster** for location-based businesses and services

SEO Optimization

1. **Enable Internal Linking** to improve site architecture and SEO value
2. **Use Negative Words** to avoid unwanted terminology in your content
3. **Configure Find and Replace** to maintain brand consistency
4. **Set Appropriate Tone** to match your brand voice

Performance Optimization

1. **Monitor Project Status** using the status checker endpoint
2. **Batch Related Keywords** for more efficient processing
3. **Plan Credit Usage** based on your content volume requirements

Error Handling

Always implement proper error handling in your applications:

try:

```
response = requests.post(url, json=data, headers=headers)
if response.status_code == 200:
    # Process successful response
```

```
        result = response.json()
    else:
        # Handle error response
        print(f"Error: {response.status_code} - {response.text}")
except requests.exceptions.RequestException as e:
    print(f"Request failed: {e}")
```

Support and Resources

- **Knowledge Base:** Comprehensive guides and tutorials
- **Discord Community:** Join our developer community for support
- **Free SEO Tools:** Website headlines, meta descriptions, blog titles, and essay intros

Conclusion

The SEOContent.ai API provides enterprise-grade content generation capabilities with the flexibility to integrate into any development workflow. From single articles to comprehensive content ecosystems, our API scales with your needs while maintaining high-quality, SEO-optimized output.

Start building with our 25 free credits and discover how automated content generation can transform your content strategy. Visit our pricing page to explore plans that fit your scale and requirements.

Ready to get started? Create your account today and generate your first API key to begin building with SEOContent.ai.

Tab 31

Digital Clincs

Data Engineering

Project Information Header

- **Project:** Medical Image Data Engineering Pipeline for Pathology Workflows
- **Company:** Digital Clinics Research and Services
- **Role:** Data Scientist
- **Duration:** 12 Months

Summary STAR (1.5-2.5 minutes speaking time)

S: Digital Clinics was developing a comprehensive medical image analysis system for cancer diagnosis, requiring a scalable data engineering pipeline to break and process massive 10-20GB TIFF/SVS medical imaging files as the process was initially manual creating bottlenecks in diagnosis workflows.

T: I was tasked with engineering a complete automated data processing infrastructure that could break down massive medical imaging files into manageable formats, create efficient annotation workflows for junior pathologists, and build seamless deployment systems integrating with our CADD4MBC clinical platform.

A: I built a robust Python-based pipeline using OpenCV for automated file processing, integrated QuPath with custom Groovy scripts for annotation workflows, and created deployment systems connecting junior pathologists, senior doctors, and our clinical platform with MongoDB for metadata tracking.

R: Achieved 1.5TB bi-weekly data throughput with zero data loss, reduced manual processing time by 60%, enabled processing of 100,000+ images monthly, and established scalable workflows supporting our downstream AI projects.

Detailed STAR

S - SITUATION Situation Summary: Digital Clinics Research and Services was facing major bottlenecks in medical image analysis where pathologists couldn't efficiently work with massive 10-15GB medical imaging files, creating delays in cancer diagnosis workflows and preventing scalable annotation processes.

Detailed Context:

- **Company context & business problem:** Digital Clinics Research and Services needed to revolutionize medical image analysis workflows where the data pipeline flow was manual either done by junior pathologist or developer. The existing workflow was time consuming and error prone, creating bottlenecks in cancer diagnosis pipelines.
- **Team structure & your role:** Medical AI research team collaborating with pathologists from Basavataarakam Indo-American Cancer Hospital, Hyderabad. As Data Scientist, I was the lead data engineer responsible for designing and implementing the entire automated data pipeline architecture from raw medical images to production-ready clinical workflows.

- **Timeline & constraints:** Built and maintained it over my 12-month timeline processing 1.5TB of medical imaging data weekly. Need for seamless integration with existing clinical workflows without disruption. Strict data integrity requirements for medical-grade applications.
- **Scale/scope of the challenge:** Processing 10-15GB individual TIFF files daily, supporting multiple image formats, ensuring zero data loss across the entire pipeline

T - TASK/CHALLENGE Task Summary: I was responsible for engineering a complete data processing infrastructure that could break down massive medical imaging files, create efficient annotation workflows for junior pathologists, and build seamless deployment systems to our clinical platform.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Design and implement automated pipeline to break down 10-15GB TIFF and various medical image formats
 - Engineer annotation workflow systems for junior pathologists
 - Build deployment scripts for automatic integration with CADD4MBC backend
 - Establish data quality control and validation processes
 - Design scalable architecture supporting multiple medical image formats
- **Technical challenges faced:**
 - Handling multiple medical image formats (TIFF, SVS, etc.) with varying structures
 - Ensuring zero data loss during file processing and transformation
 - Managing memory efficiently while processing 10-15GB files
 - Creating seamless integration between annotation tools and clinical platform
 - Maintaining data integrity across multi-stage workflows
- **Business/stakeholder challenges:**
 - Integrating new automated systems with existing clinical workflows
 - Training junior pathologists on new annotation processes
 - Ensuring senior pathologists could efficiently review and provide feedback
 - Managing data privacy and security for sensitive medical imaging data
- **Success criteria and constraints:** Zero data loss, seamless clinical workflow integration, 1.5TB weekly processing capacity, medical-grade data security

A - ACTION (Detailed Breakdown) Action Summary: I built a robust data engineering pipeline using custom Python scripts for file processing, created automated annotation workflows with quality controls, and engineered seamless deployment systems that connected junior pathologists, senior doctors, and our clinical platform.

Technical Stack Used:

- **Python + OpenCV + NumPy:** Core image processing pipeline (chosen for robust medical image format support and memory management)
- **Custom Python Scripts:** File breakdown and processing automation (designed for handling various medical image formats)
- **QuPath + Custom Groovy Scripts:** Annotation workflow integration (selected for pathologist familiarity and clinical workflow compatibility)
- **CADD4MBC Platform Integration:** Backend deployment system (existing clinical platform requiring seamless data flow)
- **MongoDB:** Metadata and annotation tracking (reliable data storage for clinical environments)
- **Flask/FastAPI:** API development for workflow automation (lightweight deployment and integration capabilities)

Collaboration Details:

- **Worked with:** Junior pathologists (workflow training), senior pathologists (quality review), clinical researchers (requirement validation), IT staff (platform integration)
- **How decisions were made:** Weekly feedback sessions with pathologists to refine workflows and ensure clinical usability
- **Communication challenges/successes:** Bridged technical implementation with clinical requirements through regular stakeholder reviews
- **Cross-functional coordination:** Coordinated between technical development, clinical workflows, and hospital IT systems

Leadership Moments:

- **Decisions you drove:** Architected the entire data pipeline infrastructure choosing optimal file processing methods and workflow designs
- **People you influenced/mentored:** Trained junior pathologists on new annotation workflows and best practices
- **Processes you improved:** Transformed manual file handling into automated pipeline reducing processing time by 60%
- **Initiative you took beyond assigned tasks:** Created comprehensive documentation and training materials for clinical staff

Problem-Solving Process:

- **Approach to technical challenges:** Implemented chunked file processing for memory management and created modular architecture for different image formats
- **Alternative solutions considered:** Evaluated cloud vs on-premise processing, batch vs real-time processing, custom vs existing annotation tools
- **Trade-offs made and reasoning:** Chose on-premise solution for medical data security, prioritized reliability over speed for clinical safety
- **How you validated your approach:** Conducted extensive testing with real clinical data and gathered continuous feedback from pathologists

R - RESULTS Result Summary: The data engineering pipeline achieved remarkable scale processing 1.5TB weekly with zero data loss, reduced manual processing time by 60%, and enabled seamless workflows that supported both junior pathologist training and senior doctor review processes.

Detailed Impact:

- **Quantified business impact:**
 - **1.5TB weekly** data throughput with zero data loss
 - **60% reduction** in manual file processing time
 - **100% automation** of file breakdown and deployment processes
 - Enabled processing of **100,000+ images monthly** for downstream AI projects
- **Technical achievements:**
 - Successfully engineered pipeline handling multiple medical image formats (TIFF, SVS, etc.)
 - Built robust annotation workflow supporting multi-level review processes
 - Created seamless CADD4MBC integration with automated quality controls
 - Established scalable architecture supporting future expansion
- **Team/process improvements:**
 - **Established standardized workflows** for medical image processing reducing training time
 - **Created documentation standards** for clinical data engineering projects
 - **Improved cross-functional collaboration** between technical and clinical teams
 - **Enabled downstream AI projects** by providing consistent, high-quality annotated datasets
- **Personal growth/skills gained:**
 - **Large-scale data pipeline engineering** expertise in medical imaging domain
 - **Clinical workflow integration** skills bridging technical and medical requirements
 - **Cross-functional leadership** coordinating between technical teams and medical professionals
 - **Medical data compliance** understanding of healthcare data security and privacy requirements

Tubule Detection

Project Information Header

- **Project:** Tubule Detection using Distributed Deep Learning for Breast Cancer Grading
- **Company:** Digital Clinics Research and Services
- **Role:** Data Scientist
- **Duration:** 3 Months (within the 12-month overall timeline)

Summary STAR (1.5-2.5 minutes speaking time)

S: So we had this major problem where pathologists were spending forever manually identifying tubules in breast cancer images, and different experts would often disagree on what they were seeing - creating huge bottlenecks in diagnosis. That's why we decided to build this automated breast cancer diagnosis system using the Nottingham Grading System, and we needed a solid tubule detection ML pipeline to make it work.

T: My challenge was to build a deep learning model that could automatically segment tubules from these massive whole slide images with medical-grade accuracy - we're talking >90% precision here. Plus, I had to figure out how to make the training efficient because we were looking at crazy long training times.

A: I went with an R2U-Net architecture and implemented Horovod distributed training across 4 GPUs to tackle the computational challenge. I leveraged the annotated dataset pipeline I'd previously built and fine-tuned everything for optimal performance while setting up robust validation frameworks.

R: The results were fantastic - we hit 92% F1-score accuracy and I managed to slash training time by 3.9x, going from 27 hours down to just 7 hours. The work was solid enough that I got to present our findings at the SIIM-CMIM122 International Conference at Johns Hopkins Hospital, which was pretty exciting.

Detailed STAR

S - SITUATION Situation Summary: Digital Clinics Research and Services needed an automated solution for tubule detection in breast cancer images to support the Nottingham Grading System, as manual identification was time-consuming and had high inter-observer variability between pathologists.

Detailed Context:

- **Company context & business problem:** Digital Clinics Research and Services needed to automate tubule identification, a critical component of the Nottingham Grading System for breast cancer diagnosis. Manual tubule detection was creating bottlenecks in cancer grading workflows and suffered from inconsistency between pathologists.
- **Team structure & your role:** Medical AI research team collaborating with pathologists from Basavarakam Indo-American Cancer Hospital. As Data Scientist, I was the lead AI engineer

responsible for developing the deep learning model architecture and implementing distributed training infrastructure.

- **Timeline & constraints:** 6-month development timeline within the larger 12-month project. Need for high accuracy model suitable for clinical deployment. Computational constraints requiring efficient training methods for large datasets.
- **Scale/scope of the challenge:** Processing annotated tubule dataset from my data engineering pipeline, developing production-ready AI model, achieving medical-grade accuracy standards

T - TASK/CHALLENGE Task Summary: I was responsible for developing a deep learning model that could automatically segment tubules from breast cancer whole slide images with medical-grade accuracy, while implementing distributed training to handle the computational demands efficiently.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Develop R2U-Net deep learning architecture for tubule segmentation
 - Implement distributed training infrastructure using Horovod framework
 - Optimize model performance for clinical accuracy requirements
 - Process and utilize annotated dataset from my data engineering pipeline
 - Validate model performance against pathologist annotations
 - Prepare research findings for publication and conference presentation
- **Technical challenges faced:**
 - Achieving high segmentation accuracy for complex tubule structures
 - Implementing distributed training to reduce computational time from 27 hours
 - Managing GPU memory efficiently during large-scale model training
 - Ensuring model generalization across different tissue samples and imaging conditions
- **Business/stakeholder challenges:**
 - Meeting medical-grade accuracy standards for clinical deployment
 - Balancing model complexity with training efficiency
 - Validating results with pathologists for clinical acceptance
 - Preparing research-quality documentation for academic publication
- **Success criteria and constraints:** >90% accuracy, <8 hours training time, medical-grade precision, research publication readiness

A - ACTION (Detailed Breakdown) Action Summary: I developed an R2U-Net architecture optimized for tubule segmentation, implemented Horovod distributed training achieving 3.9x speedup, and created a robust validation framework that resulted in research publication and conference presentation.

Technical Stack Used:

- **R2U-Net Architecture:** Deep learning model (chosen for superior segmentation performance combining U-Net with recurrent and residual connections)
- **Horovod + NVIDIA DGX:** Distributed training framework (selected for 4-GPU parallel training achieving 3.9x performance improvement)
- **Python + PyTorch:** Model development and training (robust deep learning ecosystem with distributed training support)
- **OpenCV + NumPy:** Image preprocessing and postprocessing (medical image format handling and augmentation)
- **CADD4MBC Integration:** Annotation data utilization (leveraging dataset from my data engineering pipeline)
- **Custom Groovy Scripts:** Data preprocessing automation (seamless integration with existing QuPath workflows)

Collaboration Details:

- **Worked with:** Pathologists from Basavatarakam Indo-American Cancer Hospital (model validation), senior researchers (architecture guidance), clinical staff (annotation quality assurance)
- **How decisions were made:** Weekly model evaluation sessions with pathologists to validate accuracy and clinical relevance
- **Communication challenges/successes:** Successfully translated complex AI model performance metrics into clinically meaningful insights
- **Cross-functional coordination:** Coordinated between AI development, clinical validation, and research publication requirements

Leadership Moments:

- **Decisions you drove:** Chose R2U-Net architecture over alternatives based on segmentation performance analysis, implemented distributed training strategy
- **People you influenced/mentored:** Guided junior researchers on distributed training best practices and deep learning optimization
- **Processes you improved:** Established distributed training infrastructure reducing computation time by 3.9x for future projects
- **Initiative you took beyond assigned tasks:** Led research publication efforts and presented findings at international conference

Problem-Solving Process:

- **Approach to technical challenges:** Implemented systematic architecture comparison, optimized training pipeline with data augmentation and distributed computing
- **Alternative solutions considered:** Evaluated U-Net, Mask R-CNN, and other segmentation architectures before selecting R2U-Net
- **Trade-offs made and reasoning:** Balanced model complexity with training efficiency, chose distributed training over cloud solutions for data security
- **How you validated your approach:** Conducted extensive validation with pathologist ground truth annotations and cross-validation testing

R - RESULTS Result Summary: The tubule detection model achieved exceptional performance with 92% accuracy and 90% Mean IoU, the distributed training reduced computation time by 3.9x, and the research impact was significant enough to be presented at Johns Hopkins Hospital's international conference.

Detailed Impact:

- **Quantified business impact:**
 - **92% F1 score accuracy** on testing dataset with **90% Mean IoU** performance
 - **3.9x training speed improvement** (27 hours to 7 hours) through distributed infrastructure
 - **Enabled automated tubule detection** supporting Nottingham Grading System automation
 - **Research publication** advancing computational pathology field
- **Technical achievements:**
 - **Successfully deployed R2U-Net** achieving state-of-the-art tubule segmentation performance
 - **Implemented distributed training infrastructure** using 4-GPU parallel processing with Horovod
 - **Created robust validation framework** ensuring medical-grade accuracy standards
 - **Built scalable model architecture** supporting future breast cancer grading components
- **Team/process improvements:**
 - **Established distributed training standards** for future deep learning projects in medical imaging
 - **Created model validation protocols** ensuring clinical accuracy for AI medical applications
 - **Advanced research capabilities** through international conference presentation at Johns Hopkins Hospital
 - **Knowledge transfer documentation** enabling team to replicate distributed training success
- **Personal growth/skills gained:**
 - **Advanced deep learning expertise** in medical image segmentation using state-of-the-art architectures
 - **Distributed computing mastery** implementing multi-GPU training optimization achieving 3.9x performance gains
 - **Research publication leadership** successfully presenting at top-tier international medical AI conference
 - **Clinical AI validation** understanding medical accuracy requirements and pathologist collaboration

Mitosis Detection

Listen to this tab Project Information Header

- **Project:** Mitosis Detection using Multi-Architecture Deep Learning for Breast Cancer Grading
- **Company:** Digital Clinics Research and Services
- **Role:** Data Scientist
- **Duration:** 4 Months (within the 12-month overall timeline)

Summary STAR (1.5-2.5 minutes speaking time)

S: So the big problem we were facing was that pathologists were spending way too much time manually counting mitosis cells in breast cancer images, and different experts would often get completely different counts for the same image - this was creating serious bottlenecks in cancer diagnosis. That's why we were building this automated breast cancer diagnosis system at Digital Clinics based on the Nottingham Grading System, and mitosis counting was a critical component we needed to make it work.

T: My job was to build the complete ML pipeline for this mitosis detection component from scratch. I had to implement and compare two different deep learning approaches - starting with Faster R-CNN using Detectron2, but I had this hypothesis that YOLOv5's single-stage architecture might actually give us a better balance of speed and precision for what we needed.

A: I built the entire pipeline - data preprocessing, smart dataset splitting for efficient testing, implemented both the Faster R-CNN and YOLOv5 architectures, then ran systematic performance comparisons across 56,258 annotated images. I also set up comprehensive post-training evaluation frameworks and worked with pathologists to validate our ground truth.

R: Turns out my hypothesis was right - YOLOv5 crushed it with an 84% F1 score, which was a 7% improvement over Faster R-CNN. The results were so strong we ended up setting new benchmarks for automated mitosis counting in medical imaging and our work was published as a paper in the International Journal of Advanced Computer Science and Applications.

Detailed STAR

S - SITUATION Situation Summary: Digital Clinics Research and Services needed to build a model for breast cancer detection based on the Nottingham Grading System automation by developing accurate mitosis detection capabilities, as manual mitosis counting was extremely time-consuming and subject to "high inter-observer variability between" / "disagreement between" expert pathologists.

Detailed Context:

- **Company context & business problem:** Digital Clinics Research and Services needed to automate mitosis detection, the second critical component of the Nottingham Grading System for breast cancer diagnosis. The manual process of counting cancer cells was creating two major problems: it was a bottleneck that slowed everything down, and it was unreliable because different experts would get different counts.
- **Team structure & your role:**
 - **Medical AI research team** collaborating with

Pathologists from **Basavatarakam Indo-American Cancer Hospital** and **Tapadia Diagnostics Centre**.

As Data Scientist, I was the lead AI engineer responsible for implementing multiple deep learning architectures and conducting comparative analysis.

- **Timeline & constraints:** 4-month development timeline within the larger project. Need for high-precision detection suitable for clinical deployment. Requirement to handle the massive annotated dataset from my data engineering pipeline.
- **Scale/scope of the challenge:** Processing 56,258 annotated mitosis tiles, implementing two different AI architectures, achieving medical-grade detection accuracy, conducting comprehensive model comparison

T - TASK/CHALLENGE Task Summary: I was responsible for implementing and comparing two different deep learning approaches for mitosis detection, optimizing both models for clinical accuracy, and conducting research-quality analysis to determine the best architecture for automated cancer grading.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Implement Faster R-CNN architecture using Detectron2 framework for mitosis detection
 - Develop YOLOv5 model for comparison and performance optimization
 - Process and utilize 56,258 annotated mitosis tiles from my data engineering pipeline
 - Conduct comprehensive performance comparison between architectures
 - Optimize models for clinical deployment requirements
 - Integrate detection results with CADD4MBC platform for visualization
 - Prepare research findings for peer-reviewed publication
- **Technical challenges faced:**
 - Distinguishing mitotic figures from normal cells with high precision
 - Handling class imbalance in mitosis detection datasets
 - Optimizing two different architecture types (two-stage vs single-stage detection)
 - Managing large-scale training with 56,258 annotated tiles
 - Ensuring consistent performance across different tissue samples

- **Business/stakeholder challenges:**
 - Meeting medical-grade precision requirements for clinical acceptance
 - Balancing detection sensitivity with false positive rates
 - Validating results with multiple pathologists for inter-observer agreement
 - Demonstrating clear performance advantages for clinical adoption
- **Success criteria and constraints:** >80% F1 score, minimal false positives, clinical deployment readiness, research publication quality

A - ACTION (Detailed Breakdown) Action Summary: I implemented both Faster R-CNN and YOLOv5 architectures with custom optimizations, conducted systematic performance comparison, and created comprehensive validation frameworks that resulted in a peer-reviewed publication demonstrating YOLOv5's superior performance.

Technical Stack Used:

- **Faster R-CNN + Detectron2:** Two-stage object detection (chosen for high precision requirements in medical applications)
- **YOLOv5:** Single-stage object detection (selected for speed and efficiency comparison with maintained accuracy)
- **Python + PyTorch:** Model development and training (robust ecosystem supporting both architectures)
- **OpenCV + NumPy:** Image preprocessing and augmentation (medical image format handling and data enhancement)
- **CADD4MBC Integration:** Visualization and annotation platform (leveraging my data engineering pipeline outputs)
- **COCO Dataset Format:** Annotation standardization (ensuring compatibility with Detectron2 framework)
- **Custom Data Conversion Scripts:** Format transformation (converting JSON annotations to required formats for each model)

Collaboration Details:

- **Worked with:** Pathologists from Basavatarakam Indo-American Cancer Hospital and Tapadia Diagnostics Centre (validation), senior researchers (architecture guidance), clinical staff (annotation quality assurance)
- **How decisions were made:** Systematic model evaluation with pathologist validation and statistical analysis of performance metrics
- **Communication challenges/successes:** Successfully demonstrated technical model differences in clinically meaningful terms
- **Cross-functional coordination:** Coordinated between AI development, clinical validation, and research publication workflows

Leadership Moments:

- **Decisions you drove:** Selected optimal architectures for comparison, designed comprehensive evaluation methodology comparing two-stage vs single-stage detection
- **People you influenced/mentored:** Guided team on object detection best practices and medical AI validation requirements
- **Processes you improved:** Established standardized model comparison framework for future medical AI projects
- **Initiative you took beyond assigned tasks:** Led comprehensive research analysis and publication in peer-reviewed journal

Problem-Solving Process:

- **Approach to technical challenges:** Implemented systematic architecture comparison with consistent evaluation metrics, optimized data preprocessing for both model types
- **Alternative solutions considered:** Evaluated YOLO variants vs R-CNN family, single vs ensemble approaches, custom vs pre-trained backbone networks
- **Trade-offs made and reasoning:** Balanced detection speed with accuracy requirements, chose comprehensive comparison over single model optimization
- **How you validated your approach:** Conducted extensive validation with pathologist ground truth, statistical significance testing, and cross-validation analysis

R - RESULTS Result Summary: The comparative analysis demonstrated YOLOv5's superior performance achieving 84% F1 score with 7% improvement over Faster R-CNN, the research was published in a peer-reviewed international journal, and the work established new benchmarks for automated mitosis detection in breast cancer diagnosis.

Detailed Impact:

- **Quantified business impact:**
 - **84% F1 score** achieved with YOLOv5 model (7% improvement over Faster R-CNN baseline)
 - **56,258 annotated tiles** successfully processed for training and testing
 - **Comprehensive model comparison** providing clinical guidance for architecture selection
 - **Published research** in International Journal of Advanced Computer Science and Applications (IJACSA)
- **Technical achievements:**
 - **Successfully implemented two state-of-the-art** object detection architectures for medical imaging
 - **Demonstrated YOLOv5 superiority** over Faster R-CNN in mitosis detection tasks
 - **Created robust evaluation framework** ensuring reproducible and clinically meaningful results
 - **Integrated detection capabilities** with CADD4MBC platform for clinical visualization
- **Team/process improvements:**

- **Established object detection standards** for medical AI projects with comparative analysis methodology
- **Created model selection guidelines** helping team choose optimal architectures for medical imaging tasks
- **Advanced research impact** through peer-reviewed publication contributing to computational pathology field
- **Completed Nottingham Grading System** automation enabling full breast cancer grading pipeline
- **Personal growth/skills gained:**
 - **Multi-architecture expertise** in both single-stage and two-stage object detection for medical applications
 - **Research publication leadership** successfully publishing comparative analysis in international peer-reviewed journal
 - **Medical AI benchmarking** establishing performance standards for automated cancer diagnosis systems
 - **End-to-end project completion** successfully delivering complete automated cancer grading system from data pipeline to clinical deployment

Nottingham Grading System

The Nottingham grading system is a histopathological classification used primarily for breast cancer. Here are the key details:

Components (each scored 1-3):

- **Tubule formation:** Measures how much the cancer cells form tube-like structures (well-formed tubules = lower grade)
- **Nuclear pleomorphism:** Evaluates nuclear size, shape, and staining variations (uniform nuclei = lower grade)
- **Mitotic count:** Counts dividing cells per high-power field (fewer mitoses = lower grade)

Overall Grades:

- **Grade 1 (3-5 points):** Well-differentiated, slow-growing, better prognosis
- **Grade 2 (6-7 points):** Moderately differentiated, intermediate behavior
- **Grade 3 (8-9 points):** Poorly differentiated, aggressive, worse prognosis

Clinical significance:

- Higher grades correlate with faster growth, higher metastatic potential, and poorer outcomes
- Used alongside tumor size, lymph node status, and hormone receptor status for treatment planning
- Helps determine need for chemotherapy and predicts response to treatments

The system replaced older grading methods and is now the international standard for breast cancer histological grading, providing consistent prognostic information across different pathology laboratories.

Glossary

AI Models

Faster R-CNN

Faster R-CNN (Region-based Convolutional Neural Network) is a two-stage object detection model highly regarded for its accuracy. In the first stage, it identifies potential regions in an image that might contain an object. In the second stage, it classifies the object within each of those regions and refines its bounding box. This detailed, two-step approach is what makes it very precise, which is especially valuable for medical imaging tasks where accuracy is the top priority.

YOLOv5

YOLOv5 (You Only Look Once, version 5) is a single-stage object detection model known for its incredible speed and efficiency. Unlike two-stage models, YOLO processes the entire image at once, predicting all bounding boxes and class probabilities in a single pass. This unified architecture makes it much faster, offering a powerful balance between speed and accuracy. It's an excellent choice for applications requiring real-time performance without a major compromise on detection quality.

R2U-Net Architecture

R2U-Net is an advanced deep learning model designed for precise medical image segmentation. It enhances the popular U-Net architecture by integrating Recurrent and Residual connections. This unique design allows it to learn more complex features and patterns within the images, leading to superior performance in segmenting intricate structures like breast cancer tubules. Its ability to capture detailed features made it an excellent choice for achieving the high accuracy required for clinical use.

Technical Concepts & Skills

Nottingham Grading System

Also known as the Elston-Ellis modification of the Scarff-Bloom-Richardson grading system, this is a standard method used by pathologists to grade the severity of breast cancer. It assesses three factors: tubule formation, nuclear pleomorphism (cell irregularity), and mitotic count (the number of dividing cells). Your project automated the mitotic count, a key part of this system.

Multi-Architecture Deep Learning

This refers to the practice of implementing and comparing several different deep learning models (architectures) to solve the same problem. Instead of relying on a single approach, you tested both a two-stage (Faster R-CNN) and a single-stage (YOLOv5) model to scientifically determine which one was superior for your specific task, ensuring you found the optimal solution.

F1 Score

The F1 score is a key performance metric in machine learning that measures a model's accuracy by combining two other metrics: precision and recall.

Precision measures how many of the model's positive predictions were actually correct.

Recall measures how many of the actual positive cases the model found.

The F1 score provides a single, balanced measure of performance, which is especially useful when dealing with imbalanced datasets.

Object Detection (Two-stage vs. Single-stage)

Object detection is a computer vision task that identifies and locates objects within an image. Two-stage detectors like Faster R-CNN first propose regions of interest and then classify them, prioritizing accuracy. Single-stage detectors like YOLO scan the image once to perform both tasks simultaneously, prioritizing speed and efficiency.

Data Engineering Pipeline

This is an automated system you built to collect, clean, transform, and prepare raw data for use in machine learning models. Your pipeline took raw images and annotations and processed them into a structured, high-quality dataset of 56,258 tiles, making it possible to train your complex deep learning models effectively and reliably.

Class Imbalance

This is a common problem in machine learning where the dataset contains significantly more examples of one class than another. In your case, there were likely far more normal, non-dividing cells than mitotic figures. This imbalance can bias the model, making it crucial to use specific techniques and metrics (like the F1 score) to ensure the model learns to detect the rare class correctly.

Peer-Reviewed Publication

This is the process of submitting your research findings to a scientific journal where independent, anonymous experts in your field (peers) evaluate the quality, validity, and significance of your work before it is published. Having your work published in a peer-reviewed journal like IJACSA is a major achievement that validates your results and contributes new knowledge to the scientific community.

Medical AI Benchmarking

This is the process of setting a new standard of performance for a specific task in the medical AI field. By demonstrating that YOLOv5 achieved an 84% F1 score, you established a new, published performance target. Future researchers working on mitosis detection will now compare their results against the benchmark you set.

Tubule Detection / Segmentation

This is a specific computer vision task focused on identifying and precisely outlining the boundaries of tubule structures in breast tissue images. Unlike object detection, which just draws a box around an object, segmentation creates a pixel-perfect mask. Accurate tubule segmentation is a fundamental part of the Nottingham Grading System, helping pathologists assess the aggressiveness of a tumor.

Distributed Deep Learning / Training

This is a technique used to dramatically speed up the training of large AI models by dividing the workload across multiple processors, in this case, four GPUs. Instead of one GPU processing all the data, each GPU works on a smaller portion simultaneously. This parallel processing significantly reduces the overall computation time, which was critical for your project, cutting the training from 27 hours down to just 7.

Mean IoU (Intersection over Union)

Mean IoU is a standard metric for evaluating the performance of a segmentation model. It measures the accuracy of a prediction by calculating the overlap between the AI's predicted segmentation mask and the actual "ground truth" mask created by a pathologist. A higher IoU score indicates a more precise and accurate segmentation, showing that the model's outlines closely match the expert's.

Whole Slide Images (WSIs)

Whole Slide Images are extremely high-resolution digital images created by scanning an entire glass microscope slide. These images are massive, often gigapixels in size, and contain a vast amount of detail at a microscopic level. Working with WSIs is a significant challenge in computational pathology due to their scale, requiring efficient data processing pipelines and powerful models to analyze them effectively.

Model Generalization

This refers to the model's ability to perform accurately on new, unseen data that it was not trained on. A model with good generalization can reliably analyze tissue samples from different patients, labs, or imaging equipment without a drop in performance. This is a critical requirement for any medical AI tool to be considered robust and ready for real-world clinical deployment.

Ground Truth Annotations

This is the expert-labeled data used to train and validate an AI model. In your project, the "ground truth" was the set of precise tubule outlines drawn on the images by expert pathologists. This high-quality, human-verified data serves as the correct answer that the AI model learns from and is later tested against to measure its performance and accuracy.

Data Engineering Pipeline

A data engineering pipeline is an automated series of steps that moves data from a source to a destination. In your project, you built a pipeline that automatically took massive raw medical image files, processed them into smaller, usable formats, and delivered them to an annotation platform, completely transforming a slow, manual workflow.

Annotation Workflow

An annotation workflow is the structured process and set of tools that allow users to label data. You engineered a system specifically for junior pathologists to efficiently and accurately draw on or "annotate" the processed images. This included tools for labeling and a review system for senior doctors to provide feedback, ensuring high-quality data.

Data Integrity

Data integrity refers to maintaining the accuracy, consistency, and trustworthiness of data throughout its entire lifecycle. In a medical context, this is critical. Your pipeline ensured that when a massive 10GB file was broken down, no pixels or metadata were lost or corrupted, guaranteeing that the data used for diagnosis was completely reliable.

Chunked File Processing

This is a memory-management technique for handling massive files. Instead of trying to load a 15GB image into memory all at once (which would crash most systems), your scripts processed the file in smaller, manageable "chunks." This approach allowed you to work with extremely large images efficiently without requiring specialized, high-memory hardware.

On-premise vs. Cloud Processing

This refers to where the data processing takes place. On-premise means all computation and data storage happen on the company's local servers, offering high security and control. Cloud processing uses remote servers from providers like AWS or Google. You chose an on-premise solution to ensure maximum security and compliance for sensitive patient data.

Medical Data Compliance

This is the skill of understanding and adhering to strict legal and ethical regulations governing patient data, such as HIPAA. It involves ensuring data privacy, security, and traceability throughout your pipeline. This knowledge was essential for building a system that was not only technically sound but also legally and ethically responsible for handling sensitive medical information.

Libraries & Frameworks

Detectron2

Detectron2 is Facebook AI Research's next-generation library for object detection and segmentation, built on PyTorch. It provides a flexible and powerful framework with a large collection of pre-trained models, including Faster R-CNN. You used it as the foundation to implement your initial, high-precision detection model.

Python

Python is a high-level, versatile programming language that is the dominant language for data science and machine learning. Its simple syntax, extensive libraries (like PyTorch, NumPy, and OpenCV), and strong community support make it the ideal choice for developing complex AI systems from research to production.

PyTorch

PyTorch is a popular open-source machine learning library developed by Facebook's AI Research lab. It's known for its flexibility, ease of use, and powerful GPU acceleration, which is essential for training deep learning models. It provides the core framework upon which both Detectron2 and YOLOv5 are built.

OpenCV

OpenCV (Open Source Computer Vision Library) is a fundamental library for computer vision tasks. It provides a vast array of tools and functions for real-time image and video processing. You used it for essential tasks like reading, resizing, and augmenting the medical images in your dataset before feeding them into your models.

NumPy

NumPy (Numerical Python) is the foundational package for scientific computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of high-level mathematical functions to operate on these arrays. It's the backbone for almost all data science libraries, including PyTorch and OpenCV.

COCO Dataset Format

COCO (Common Objects in Context) is a standard format for structuring image annotation data for object detection, segmentation, and captioning tasks. Using this standardized format ensured that your custom dataset was compatible with established frameworks like Detectron2, which saved time and streamlined the model development process.

Horovod

Horovod is an open-source distributed deep learning framework originally developed by Uber. It makes it easier to scale the training of AI models from a single GPU to hundreds. You used Horovod to manage the communication and synchronization between the four NVIDIA DGX GPUs, which was the key to efficiently distributing the training workload and achieving the 3.9x speedup.

NVIDIA DGX

NVIDIA DGX is a line of high-performance servers and workstations specifically designed for AI and deep learning. These systems are packed with powerful GPUs and optimized software, providing the immense computational power needed to train complex models on large datasets. Using a DGX system provided the necessary hardware infrastructure for your multi-GPU distributed training.

Groovy Scripts

Groovy is a scripting language that runs on the Java Virtual Machine (JVM). In your project, you used Groovy scripts to automate data preprocessing tasks within QuPath, a popular open-source software for digital pathology. This automation created a seamless and efficient workflow, allowing you to easily prepare and extract the necessary data from the whole slide images for your model.

Of course. Here are the explanations for the remaining terms from your data engineering project.

File Formats & Tools

TIFF/SVS

TIFF (Tagged Image File Format) and SVS (Aperio ScanScope Virtual Slide) are common file formats for high-resolution whole slide images in digital pathology. These are not like regular JPEGs; they are massive, often multi-gigabyte files that contain layered, high-magnification image data, which is why they required your specialized data pipeline to process them.

QuPath

QuPath is a powerful, open-source software application widely used by pathologists and researchers for digital pathology and whole slide image analysis. You integrated it into your pipeline as the primary tool for pathologists to perform annotations, leveraging its robust features and their familiarity with the software to create an efficient workflow.

CADD4MBC Platform

This is the company's internal, custom-built clinical platform used for cancer diagnosis and management. Your data pipeline's ultimate goal was to feed high-quality, annotated data directly into this system. By building seamless deployment scripts, you ensured your pipeline was not a standalone tool but a fully integrated part of the core clinical workflow.

PostgreSQL

PostgreSQL is a powerful, open-source relational database system known for its reliability and robustness. You used it as the backend database to store and manage all the critical metadata associated with the images, such as patient information, annotation data, and review status, ensuring all information was tracked securely and accurately.

MongoDB: NoSQL document database storing data as flexible JSON-like documents instead of traditional tables.

Flask/FastAPI

Flask and FastAPI are modern, lightweight Python web frameworks used for building APIs (Application Programming Interfaces). You used them to create the communication links between different parts of your pipeline. These APIs allowed the annotation tools, the database, and the CADD4MBC platform to talk to each other automatically, powering your seamless workflow.

Edgeforce Solutions

Edgeforce Solutions, Hyderabad, India - Data Scientist Intern 11/2021 - 02/2022

"At EDGEFORCE SOLUTIONS, an ISO 9001:2015 certified company, I worked as a Data Scientist intern on projects directly supporting India's defense and security sectors. The company specializes in developing indigenous edge computing and AI solutions that create technological advantages for security forces while also serving other critical sectors.

During my internship, I led two key projects and part of one project. First, I developed a Software Portable Radio Simulator that allowed military personnel to train on communication protocols without physical hardware. The system used Python, WebRTC, and socket programming to enable real-time, encrypted voice communication with under 200ms latency. This reduced training costs significantly.

I also built an AI-Powered Wine Inventory Management System using YOLOv5 object detection models and a Streamlit dashboard with WebRTC integration. This automated solution increased tracking accuracy by ~90% and reduced manual counting time by ~60%, ultimately winning a grant for full-scale implementation.

My work exemplifies EDGEFORCE's mission of creating innovative, cost-effective technological solutions that address specific operational challenges in defense while demonstrating applications for other sectors like logistics and inventory management."

Technical Glossary

Edge Computing: Technology that processes data near its source rather than relying on cloud-based processing, reducing latency and bandwidth usage.

WebRTC: Web Real-Time Communication - An open-source project enabling direct peer-to-peer audio/video communication and data sharing between browsers without plugins.

Socket Programming: A method for communication between computers using TCP/IP protocols, allowing real-time data exchange.

YOLOv5: "You Only Look Once" - A state-of-the-art object detection algorithm that can identify multiple objects in a single pass through the neural network.

Streamlit: An open-source Python library for creating web applications for data science and machine learning projects with minimal front-end coding.

ISO 9001:2015: An international standard for quality management systems, certifying that an organization consistently provides products/services meeting customer and regulatory requirements.

Digital Twin: A virtual representation of a physical object or system that uses real-time data to enable understanding, learning, and reasoning.

Industry 4.0: The fourth industrial revolution characterized by smart automation, data exchange, and manufacturing technologies.

PSUs: Public Sector Undertakings - Government-owned corporations in India.

SPR Simulation

Project Information Header

Project: Software Portable Radio Simulator

Company: Edgeforce Solutions

Role: Data Scientist Intern

Duration: 3 Months

Summary STAR (1.5-2.5 minutes speaking time)

S: The Indian Army needed a cost-effective training solution to replace expensive physical radio hardware for soldier training programs, so we were contracted to develop a software-based radio communication simulator.

T: As the sole developer on this project, I had to build a complete real-time voice communication system from scratch that could connect 3-4 laptops simulating the original radio simulator achieving sub-500ms latency while handling changing client requirements over 2 months.

A: I built the entire system using Python with WebRTC for real-time communication, PyAudio for audio processing, and Tkinter for the GUI. I implemented networking protocols and worked closely with army officers to validate requirements and prevent scope creep.

R: Delivered a successful system achieving sub-200ms latency (85% improvement from 1.5s), the project was so well-received that we got additional defense contracts for Edgeforce Solutions.

Detailed STAR

S - SITUATION Situation Summary: Edgeforce Solutions was contracted by the Indian Army to develop a radio training simulator because their existing manual training process required expensive physical hardware, making it costly and inefficient for soldier training programs.

Detailed Context:

- **Company context & business problem:** Edgeforce Solutions, a defense contractor serving military clients in India, was contracted to develop a Software Portable Radio Simulator. The Indian Army needed a cost-effective training solution to simulate military communication systems without requiring expensive physical hardware, enabling soldiers to train and test radio protocols in controlled environments.
- **Team structure & your role:** Small development team with me as the only sole developer responsible for this project and senior technical staff. As a Data Scientist Intern, I was the primary developer responsible for coding the whole system

- **Timeline & constraints:** 3-month timeline and frequent client requirement changes added complexity.
- **Scale/scope of the challenge:** Complete system development from scratch with military-grade performance requirements

T - TASK/CHALLENGE Task Summary: I was tasked with building a complete radio communication simulator from scratch that could handle real-time voice transmission between multiple laptops with military-grade performance standards, while managing changing requirements and tight deadlines.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Develop real-time voice transmission protocol for 3-4 laptop communication parallelly
 - Create low-latency audio packet transmission system (<500ms)
 - Design user-friendly interface for military personnel
 - Integrate networking libraries for seamless multi-device communication
- **Technical challenges faced:**
 - Achieving sub-500ms latency for real-time military communication
 - Figuring out the tech stack
 - Audio quality preservation during compression and transmission
- **Business/stakeholder challenges:**
 - Frequent requirement changes from client around which simulator to create, etc
 - Managing expectations around performance
- **Success criteria and constraints:** Sub-500ms latency, military-grade security, offline deployment capability

A - ACTION (Detailed Breakdown) Action Summary: I built the system using Python and WebRTC for real-time communication, collaborated closely with army officers for requirements, made key technical decisions like switching frameworks, and created proper documentation to manage scope creep.

Technical Stack Used:

- **Python:** Backend logic and networking (chosen for rapid development and extensive audio libraries)
- **Tkinter:** GUI framework (selected for standalone offline deployment vs. Streamlit)
- **WebRTC:** Real-time communication (industry standard for low-latency streaming)
- **PyAudio + Numpy:** Audio processing (real-time capture and playback capabilities)
- **Socket Programming:** Custom network handling (flexibility for military network requirements)

Collaboration Details:

- **Worked with:** Army officers (requirements), senior staff (scope management)

- **Decision Making:** Weekly stakeholder reviews with army technical staff for requirement validation
- **Communication:** Conducted feedback sessions to prevent miscommunication; created SRS document for requirement freeze

Leadership Moments:

- **Drove Decision:** Switched from Streamlit to Tkinter for offline deployment requirements
- **Mentored:** Trained army personnel on system maintenance and debugging procedures
- **Process Improvement:** Created comprehensive documentation and SRS to prevent scope creep in future projects

Problem-Solving Process:

- **Approach to technical challenges:** Implemented custom networking protocols and audio compression techniques
- **Alternative solutions considered:** Evaluated different GUI frameworks and communication protocols
- **Trade-offs made and reasoning:** Chose offline-capable solutions over web-based for military security requirements
- **How you validated your approach:** Conducted regular testing with army technical staff and performance benchmarking

R - RESULTS Result Summary: The project was highly successful, delivering an 85% improvement in communication latency, reducing hardware costs by 80%, and leading to additional contracts for the company while establishing new documentation standards.

Detailed Impact:

- **Quantified business impact:**
 - **80% reduction** in hardware dependency and associated costs
 - **85% latency improvement** (1.5s to 200ms)
 - Won additional defense contracts based on project success
- **Technical achievements:**
 - Successfully enabled real-time 3-4 laptop communication over LAN
 - Created modular codebase for future radio feature extensions
 - Achieved target <200ms latency for military-viable communication
- **Team/process improvements:**
 - Established SRS documentation standard for future projects
 - Created audio processing best practices guide for team
 - Improved stakeholder communication through structured feedback sessions
- **Personal growth/skills gained:**

- Real-time networking and audio processing expertise
- Stakeholder management in high-stakes defense environment
- Technical documentation and knowledge transfer skills

●

Wine Inventory Management System

Project Information Header

- **Project:** AI-Powered Wine Inventory Management System (POC)
 - **Company:** Edgeforce Solutions
 - **Role:** Data Scientist Intern
 - **Duration:** 3-4 weeks
-

Summary STAR (1.5-2.5 minutes speaking time)

S: Edgeforce Solutions wanted to showcase that an edge device (Raspberry Pi) could be deployed to automatically count wine bottles using AI, proving to army clients that computer vision could solve their manual inventory problems - eliminating frequent counting errors, inefficient tracking, and operational costs from overstocking and stockouts.

T: As a Data Scientist Intern, I had to rapidly develop a complete edge-computing proof-of-concept from scratch in just 3-4 weeks to validate whether computer vision on resource-constrained hardware could actually work for automated wine bottle detection and justify full-scale development funding.

A: I built a comprehensive edge-ready system using YOLOv5 optimized for Raspberry Pi deployment, created annotated datasets using Roboflow, and developed a Streamlit web dashboard with live video streaming capabilities to demonstrate real-time counting during stakeholder presentations.

R: Delivered a successful edge-device POC with 87% detection accuracy, validated 30% time reduction potential in manual counting, and impressed army stakeholders enough to secure approval and funding (SPARK grant from Innovation for defence excellence) for full-scale implementation ~~-all within the 3-week timeline.~~

Detailed STAR

S - SITUATION

Situation Summary: Edgeforce Solutions needed to develop a proof-of-concept AI solution for army clients to validate whether computer vision could solve their manual wine inventory management problems and justify investment in full-scale implementation.

Detailed Context:

- **Company context & business problem:** Edgeforce Solutions, a defense contractor serving army clients in India, was contracted to develop a proof-of-concept for an AI-powered wine inventory management system. Army facilities were experiencing frequent manual inventory errors, inefficient stock tracking, and increased operational costs due to miscounts that resulted in overstocking or stockouts.
- **Team structure & your role:** Small development team with me as the sole developer responsible for the entire POC development. As Data Scientist Intern, I was tasked with rapidly validating the technical feasibility of computer vision for inventory automation.
- **Timeline & constraints:** Aggressive 3-4 week timeline for POC delivery to army stakeholders. Limited initial data requiring rapid dataset creation and model training. Need to demonstrate real-time processing capabilities under varying conditions.
- **Scale/scope of the challenge:** Rapid POC development, custom dataset annotation, real-time computer vision validation, stakeholder demonstration requirements

T - TASK/CHALLENGE

Task Summary: I was tasked with building a complete proof-of-concept AI system from scratch using computer vision to validate the feasibility of automated wine bottle detection and classification for army inventory management.

Detailed Responsibilities:

- **POC Development:** Create functional prototype demonstrating AI-powered wine bottle detection and classification capabilities
- **Dataset Creation:** Rapidly build annotated dataset for training and validating computer vision model performance
- **Model Implementation:** Develop YOLOv5-based object detection system optimized for real-time inventory tracking
- **Dashboard Development:** Build demonstrable Streamlit interface with live video streaming for stakeholder presentations
- **Feasibility Validation:** Prove technical and business viability of AI solution within rapid POC timeline

Technical challenges faced:

- **Rapid Development:** Learning and Building complete computer vision pipeline within 3-4 week POC timeline
- **Limited Data:** Creating sufficient training dataset quickly without extensive data collection periods
- **Real-time Constraints:** Demonstrating live processing capabilities for stakeholder validation

- **Variable Conditions:** Ensuring model works under different lighting and storage conditions for army facilities

Business/stakeholder challenges:

- **POC Pressure:** Delivering convincing demonstration that justifies full-scale development investment
- **Expectation Management:** Balancing POC limitations with stakeholder expectations for immediate results
- **Technical Validation:** Proving AI feasibility while acknowledging POC scope limitations

A - ACTION (*Detailed Breakdown*)

Action Summary: I rapidly developed a comprehensive POC using YOLOv5 for object detection, efficiently created annotated datasets, implemented real-time streaming capabilities, and delivered a convincing demonstration that validated the technical feasibility within the tight timeline.

Technical Stack Used:

- **Python:** Core development language (chosen for rapid ML prototyping and extensive library support)
- **YOLOv5:** Object detection model (selected for real-time performance and fast training capabilities)
- **OpenCV + NumPy:** Image processing and preprocessing (essential for computer vision pipeline)
- **Streamlit:** Rapid web dashboard development (perfect for quick POC demonstration interface)
- **WebRTC:** Real-time video streaming (enabling live demonstration capabilities)
- **Roboflow:** Dataset annotation platform (accelerated annotation workflow for rapid dataset creation)
- **SQLite:** Lightweight database (sufficient for POC data storage and tracking)

Collaboration Details:

- **Army Stakeholders:** Regular feedback sessions to validate POC direction and requirements
- **Senior Technical Staff:**
Guidance on architecture decisions and POC scope management
Daily check-ins to ensure POC milestones were met within tight timeline

Leadership Moments:

- **Technical Decisions:** Drove choice of YOLOv5 over alternative models based on rapid training and deployment requirements
- **Scope Management:** Balanced POC functionality with timeline constraints, focusing on core validation objectives
- **Demonstration Strategy:** Created compelling POC presentation showcasing real-time capabilities and accuracy metrics

- **Documentation:** Established clear POC limitations and recommendations for full-scale implementation

Problem-Solving Process:

- **Rapid Dataset Creation:** Implemented efficient annotation workflow using Roboflow to build training dataset quickly
- **Model Optimization:** Focused on achieving demonstrable accuracy while maintaining real-time performance for POC validation
- **Integration Challenges:** Built modular system allowing easy scaling for potential full implementation
- **Demonstration Preparation:** Created robust demo environment handling various bottle types and lighting conditions

R - RESULTS

Result Summary: The POC successfully validated the technical feasibility of AI-powered inventory management, achieved impressive accuracy metrics, and convinced army stakeholders to approve full-scale development funding based on the demonstrated capabilities.

Detailed Impact: Quantified business impact:

- **POC Validation:** Demonstrated 87% detection accuracy proving AI feasibility for inventory automation
- **Time Reduction Validation:** Showed potential for 30% reduction in manual counting time through automated processing
- **Funding Success:** POC impressed army stakeholders, securing approval and funding for full-scale implementation
- **Technical Feasibility:** Proved computer vision viability for military inventory management applications

Technical achievements:

- **Rapid Development:** Successfully built complete computer vision pipeline within 3-4 week POC timeline
- **Real-time Processing:** Demonstrated sub-second detection capabilities during live stakeholder presentations
- **Custom Dataset:** Created annotated dataset with 1000+ images across multiple bottle categories for POC validation
- **Scalable Architecture:** Built modular system architecture enabling smooth transition to full-scale development

Team/process improvements:

- **POC Methodology:** Established rapid prototyping approach for computer vision projects at Edgeforce Solutions
- **Stakeholder Engagement:** Created effective demonstration format for validating AI solutions with military clients
- **Knowledge Transfer:** Documented POC learnings and recommendations for full-scale implementation team
- **Technical Standards:** Set baseline performance metrics and architecture patterns for future AI projects

Personal growth/skills gained:

- **Rapid Prototyping:** Expertise in building convincing AI POCs under aggressive timelines
- **Computer Vision Development:** End-to-end experience with object detection systems from dataset creation to deployment
- **Stakeholder Management:** Skills in demonstrating technical capabilities to military clients and decision-makers
- **POC-to-Production Planning:** Understanding of how to design POCs that validate scalability for full implementation

International

I'm currently on [OPT/STEM OPT] with work authorization through [date]. For long-term sponsorship, I've consulted with immigration attorneys who've confirmed that change of status applications filed within the US appear exempt from the new fee structure. I'm also building a strong profile for alternative visas like O-1 to give us flexibility."

Leadership principles

Customers First → Customer Obsession

Deliver Results → Deliver Results

Earn Trust → Earn Trust

Operate at Depth → Dive Deep

Operate in Chewy Time → Bias for Action

Act As An Owner → Ownership

Debate Openly; Commit Fully → Have Backbone; Disagree and Commit

Build High Performing Team → Hire and Develop the Best

Remove Unnecessary Complexity → Invent and Simplify

Promote Innovation and Big-Impact Thinking → Think Big

Customer Obsession → Customers First

Ownership → Act As An Owner

Deliver Results → Deliver Results

Earn Trust → Earn Trust

Dive Deep → Operate at Depth

Bias for Action → Operate in Chewy Time

Invent and Simplify → Remove Unnecessary Complexity

Think Big → Promote Innovation and Big-Impact Thinking

Have Backbone; Disagree and Commit → Debate Openly; Commit Fully

Hire and Develop the Best → Build High Performing Teams

Insist on the Highest Standards → Deliver Results

Frugality → Act As An Owner

Learn and Be Curious → Operate at Depth

Are Right, A Lot → Operate at Depth

Strive to be Earth's Best Employer → Build High Performing Teams

Success and Scale Bring Broad Responsibility → Act As An Owner

XX

HR Call was like he asked a lot about fine tuning and transfer learning

FedMob(TensorFlow, React Native, Federated Learning, PyTorch)

-

Built React Native app implementing FedAvg algorithm enabling AI training across mobile devices achieving privacy improvement

NLP

Prompt-Manager Library (Python pkg|Version Control|YAML Metadata|PyPI|Unit Tests)

[Link]

Designed Python library for prompt management across LLM applications with version control and metadata tagging.

Created hierarchical prompt storage system, reducing prompt iteration cycles and development time by 33%.

NutriPedia: Food Recognition & Calorie Estimation (TensorFlow|Inception-V3|GCP Storage|GPT-3.5 API|SQL) [Link]

Architected cloud-based nutrition platform integrating Inception-V3 CNN with GCP Storage for image processing|implementing GPT-3.5 API for personalized dietary insights from SQL nutritional database. Engineered full-stack data pipeline connecting computer vision classifications with NLP-powered contextual responses|creating flywheel architecture for continuous model improvement through user feedback.

COVID-19 Bot (Dialogflow|Flask|MongoDB|RapidAPI|Webhooks)

[Link]

Architected full-stack Telegram conversational chatbot with DialogFlow NLP for intent recognition|Flask/Python backend|and MongoDB persistence|enabling real-time pandemic statistics retrieval via RapidAPI.

Implemented context-aware entity extraction pipeline supporting multilingual medical assistance|resource localization|and vaccination scheduling with custom webhook fulfillment handlers.

ExpenseGenie Bot (Telegram|WhatsApp|Gemini LLM|Redis|Langfuse|Voice/Image Processing, GCP)

[Link]

Architected modular microservices system for intelligent expense splitting across Telegram and WhatsApp platforms simultaneously.

Implemented Redis-based state management and Langfuse monitoring achieving 99.9% reliability and comprehensive LLM observability.

Data Scientist/Data Analyst/Data Engineer

Chronic Kidney Disease Predictor (MERN Stack|Random Forest|Flask|MongoDB|Python|XGBoost) [Link]

Engineered full stack MERN application for early CKD detection utilizing Random Forest classifier achieving 99.75% diagnostic accuracy through intelligent data completion algorithm for handling missing values in medical datasets.

Developed intuitive clinical decision support interface with secure MongoDB persistence, enabling healthcare professionals to input patient biomarkers for immediate risk assessment and early intervention.

KafkaSpark – User Data Processing Engine (Apache Kafka|Apache Spark|Airflow|Cassandra|Docker) [Link]

Engineered end-to-end data pipeline connecting API data sources to Apache Cassandra through Kafka queues|implementing distributed Spark processing and real-time data streaming.

Developed containerized architecture with Docker Compose|integrating Apache Airflow for workflow orchestration and implementing comprehensive monitoring through Kafka Control Center.

Telco Customer Churn Pipeline (Python|MLflow|XGBoost|GitHub Actions|AWS ECR/EC2) [Link]

Designed an end-to-end MLOps pipeline with modular architecture|implementing automated CI/CD workflows that reduced development time by 30%.

Engineered scalable ETL processes and predictive analytics system using MLflow experiment tracking|achieving 15% improvement in F1-score for churn prediction.

Developed and deployed containerized prediction API on AWS EC2/ECR through GitHub Actions|enabling robust model serving with automated testing and versioning.

Airline Passenger Forecasting (SARIMA|Statsmodels|Pandas|Matplotlib|A/B Testing) [Link]

Applied ARIMA and SARIMA models on airline passenger data (1949-1960)|performing A/B testing that demonstrated SARIMA's superior fit for seasonal passenger patterns.

Implemented time series decomposition and stationarity testing (ADF|KPSS)|visualizing forecast accuracy with 95% confidence intervals to enable reliable capacity planning.

Flight Fare Predictor (Random Forest|ExtraTrees|Flask|Scikit-learn|Feature Engineering) [Link]

Developed ML system predicting flight prices with 80% accuracy using Random Forest Regression.

Engineered feature extraction pipeline identifying key price determinants (total stops|journey day|airline carrier) through ExtraTreesRegressor analysis across multiple parameters.

Twitter Metrics Dashboard (Tableau|SQL|Window Functions|Interactive Filters|KPI Design) [Link]

Created interactive Tableau dashboard visualizing Twitter metrics|providing insights into impression rates and engagement patterns across different time periods.

Designed a comprehensive performance tracking system with filter-based interactivity|enabling social media managers to identify optimal posting schedules and content strategies.

Google Play Store Apps Dashboard (Tableau|Tree Maps|Parameter Controls|Data Blending|SQL) [Link]

Developed a comprehensive Tableau dashboard analyzing app performance metrics across categories|revealing correlations between ratings|installs|and content ratings.

Created interactive visualization system with tree maps and filter controls|enabling product managers to identify competitive market gaps and optimize app development strategies.

FDI Sector Explorer (Tableau|Timeseries LODs|Dual-Axis Charts|CSV ETL|Calculated Fields) [Link]

Created interactive Tableau dashboard visualizing Foreign Direct Investment trends across Indian sectors|revealing highest and lowest investment patterns over time.

Designed an intuitive visual analytics system with comparative metrics and filter-based exploration|enabling stakeholders to identify strategic investment opportunities across economic sectors.

World Bank Project Success Classifier (XGBoost|Hyperopt|Imbalanced-learn|Python|SHAP)

[Link]

Developed predictive model achieving 71.7% F1 score to identify World Bank project success factors|implementing XGBoost

classification with hyperparameter optimization.

Engineered comprehensive data processing pipeline for 17,500+ hierarchical records|addressing 30%+ missing data and class

imbalance to deliver actionable insights for resource allocation.

Analysis on Various Commodities (LSTM|TensorFlow|Web Scraping|BeautifulSoup|Plotly)

[Link]

Engineered comprehensive commodities analysis system with custom web scraper|extracting 10 years of daily price data to identify strong cross-market correlations.

Implemented LSTM neural networks for time series forecasting across six major commodities|achieving low MSE scores (0.27-0.40) for accurate price movement predictions.

Car Price Predictor (Random Forest|EDA|Flask|Bootstrap|Pandas) [Link]

Developed machine learning application predicting used car prices with Random Forest

Regression|comparing multiple algorithms to identify optimal prediction accuracy.

Implemented comprehensive data pipeline with feature engineering and EDA|creating an intuitive Flask-based web interface for real-time price estimation.

Telco Customer Churn Project [Link]

Designed an end-to-end MLOps pipeline with modular architecture|implementing automated CI/CD workflows that reduced development time by 30%.

Engineered scalable ETL processes and predictive analytics system using MLflow experiment tracking|achieving 15% improvement in F1-score for churn prediction.

Developed and deployed containerized prediction API on AWS EC2/ECR through GitHub Actions|enabling robust model serving with automated testing and versioning.

Bollywood Movie Recommender System (Python|BeautifulSoup|Cosine Similarity|TF-IDF|Streamlit)

[Link]

Engineered comprehensive ETL pipeline extracting data from 3,100+ Bollywood movies using Python and BeautifulSoup|implementing statistical analysis workflows for recommendation algorithms. Developed automated data processing system with 11 normalized features|creating structured datasets that enabled content-based filtering and similarity analysis for personalized movie recommendations.

Bioacoustic Data Analytics (TensorFlow|STFT|CNN|Spectrogram Augment|Data Pipeline) [Link]

Engineered end-to-end audio processing pipeline transforming raw forest recordings into spectrogram datasets using TensorFlow|achieving 100% precision and recall for Capuchin bird call detection. Implemented signal processing algorithms with Short-Time Fourier Transform for feature extraction|optimizing CNN architecture to distinguish target patterns from environmental noise.

Sports Predictive Analytics (Scikit-learn|GridSearchCV|Feature Engineering|Flask|RMSE Metric) [Link]

Developed regression models predicting IPL cricket scores with RMSE of 20 runs|implementing feature engineering techniques and hyperparameter optimization through GridSearchCV to analyze complex sports metrics.

Engineered end-to-end ML solution from data preprocessing to deployment|creating Flask web application with interactive interface for real-time cricket score predictions based on match conditions.

Automated License Recognition (TensorFlow OD API|MobileNet-SSD|EasyOCR|OpenCV|ROI Filtering) [Link]

Implemented real-time computer vision system using TensorFlow Object Detection API and transfer learning with SSD MobileNet v2|creating an end-to-end pipeline for license plate detection. Engineered post-processing algorithms and OCR integration using EasyOCR|optimizing detection accuracy through ROI filtering and confidence thresholds for automated data extraction.

Anime Data Synthesis (PyTorch|DC-GAN|Custom Loss|Data Augment|Hyperparam Tuning) [Link]

Implemented Deep Convolutional Generative Adversarial Network (DC-GAN) using PyTorch|engineering custom data collection pipelines and preprocessing workflows for large-scale anime image datasets.

Developed specialized training architecture with modified loss functions|generating high-quality synthetic anime character images while maintaining style consistency and feature diversity.

Banking Investment Portfolio Dynamics (Tableau|Statistical Tests|SQL|Time-Series LODs|Regulatory Analysis) [Link]

Analyzed regulatory impact on banking portfolios using statistical analysis and Tableau|revealing 4-fold HTM ratio increases in large banks following capital requirement changes.

Created financial data visualizations tracking investment trend divergence between banking segments|providing actionable insights for regulatory compliance strategies.

Medical Appointment and Patient Record System (SQL|Data Modeling|Power BI|ETL|Validation Rules)
[Link]

Engineered SQL queries and database reporting system capturing patient-doctor relationships across 6 interconnected healthcare entities with complex data validation rules.

Developed analytics dashboards reducing report generation time by 40%|enabling data-driven resource allocation decisions for healthcare administrators.

ACHIEVEMENTS

Kaggle Notebook and Dataset Expert. || Teaching Assistant for Intro to Machine Learning (CSCI-739)

Open Source Contributor at OpenVino (Intel)|Ivy || 3x Hackathon Winner

RESEARCH PAPERS

Tubules Detection on Breast Carcinoma Whole-Slide Images – C-MIMI (Johns Hopkins)|Sep 2022.

Web-Based Mitosis Detection on Breast Cancer Images via Faster RCNN & YOLOv5 – IJACSA|Dec 2022.

RESEARCH PAPERS

Tubules Detection on Breast Carcinoma Whole Slide Images Using Artificial Intelligence - Presented at C-MIMI Conference|Johns Hopkins University

Web-Based Mitosis Detection on Breast Cancer Whole Slide Images Using Faster R-CNN and YOLOv5 - International Journal of Advanced Computer Science Applications

Here is a concise yet detailed version of your projects, synthesizing the key information from your summaries and detailed breakdowns.

FedDisco Mobile Federated Learning Application

Project: FedDisco Mobile Federated Learning Application

Company: Academic Capstone Project

Role: Lead Developer/Researcher

Duration: 5 Weeks

Situation: Traditional machine learning for mobile apps required centralizing sensitive user data (images, text), creating major privacy risks and communication bottlenecks.

Task: Implement the FedDisco federated learning algorithm from scratch to enable distributed, privacy-preserving ML on mobile devices, delivering a working prototype in 5 weeks for academic review.

Action:

Tech Stack: Built a complete system using React Native (mobile client), Python FastAPI (server), and TensorFlow Lite (on-device training).

Implementation: Implemented a custom FedDisco algorithm with optimized parameters (K=5 clients, E=3 epochs, B=16 batch size), starting with a modular architecture for image processing.

Result:

Delivered a working prototype in 5 weeks, passing academic validation.

Achieved a 70% reduction in communication overhead and 60% memory optimization.

Demonstrated 40% faster convergence, achieving stability in 15 rounds versus the theoretical 25.

Budget Justification Data Transfer Automation

Project: Budget Justification Data Transfer Automation

Company: Rochester Institute of Technology (Office of VP of Research)

Role: Machine Learning Engineer Co-op

Duration: June 2025 - Present

Situation: RIT's Pre-Award team manually transferred budget data from 800+ annual Excel proposals into Word templates, causing 5-8 calculation errors monthly and severe bottlenecks.

Task: Design and build a comprehensive automation solution to eliminate manual data transfer, ensure 100% calculation accuracy, and provide a user-friendly interface for non-technical staff.

Action:

Tech Stack: Built a Streamlit web portal using Python.

Core Logic: Implemented robust Excel parsing with pandas/openpyxl and automated Word document generation using python-docx.

Collaboration: Worked closely with the Pre-Award Director and stakeholders across 5 colleges to handle diverse Excel formats.

Result:

Achieved a 60% reduction in manual processing time.

Eliminated calculation errors (down from 5-8 monthly to zero).

Saved an estimated \$50,000 annually in administrative costs and successfully automated all 800+ annual proposals.

Conversational AI System for Clinical Trial Data Quality Analysis

Project: Conversational AI System for Clinical Trial Data Quality Analysis

Company: Daiichi Sankyo

Role: R&D Data Governance Intern

Duration: Ongoing

Situation: Daiichi Sankyo wanted to replace manual Tableau-based data quality analysis with a conversational AI, but in a highly regulated environment with severe technical constraints: no cloud access, no GPU, and only 32GB RAM systems.

Task: Build a proof-of-concept (POC) conversational AI that could query complex, relational clinical trial data locally while maintaining regulatory compliance.

Action:

Developed three iterative versions, progressing from basic tabular models (TAPAS) to a sophisticated RAG architecture using Mistral 7B.

Deployed the model locally using Ollama to function on CPU-only hardware.

Created custom clinical datasets and a modular system with "Fast," "Balanced," and "Quality" configurations to manage performance trade-offs.

Result:

Successfully delivered a POC demonstrating conversational AI feasibility on CPU-only hardware, with 4-9 minute response times.

Established a regulatory-compliant AI development framework for local deployment.

Identified a clear production pathway using a SQL Translation Layer (with Phi-3-mini) that would reduce response times to seconds once cloud approval is granted.

Automated ICF Classification System for Clinical Trial Compliance

Project: Automated ICF Classification System for Clinical Trial Compliance

Company: Daiichi Sankyo

Role: R&D Data Governance Intern

Duration: 10 Months (June 2024 - April 2025)

Situation: Legal and regulatory teams spent 4-5 minutes manually reviewing each Informed Consent Form (ICF) for data sharing clauses, creating critical bottlenecks across hundreds of documents.

Task: Build a complete automated classification and document management system to dramatically reduce review time, with zero tolerance for misclassification errors.

Action:

Developed the system over 10 months through four iterative versions, evolving from rule-based logic to a sophisticated two-stage transformer architecture.

The final system used FAISS embeddings (all-MiniLM-L6-v2) for fast retrieval and a fine-tuned DistilBERT model for accurate classification.

Built a comprehensive document management platform (Flask + PDF.js) with filtering (study, country, version), version control, and batch processing.

Result:

Achieved a 0.93 F1 score and 0.96 accuracy, meeting regulatory-grade standards.

Reduced processing time by 87%, from 4-5 minutes manually to just 33 seconds.

Delivered a production-ready system that eliminated critical bottlenecks for multi-study, multi-country trial operations.

CTMS Database Migration System (Alteryx to Python)

Project: CTMS Database Migration System - Alteryx to Python Migration

Company: Daiichi Sankyo

Role: R&D Data Governance Intern

Duration: 2 Weeks

Situation: The R&D department's Alteryx-based system for migrating clinical trial (CTMS) data created licensing bottlenecks, had limited customization, and dependency issues.

Task: In just 2 weeks, completely migrate the entire data migration system from Alteryx to a custom Python solution, preserving all functionality while adding enhancements.

Action:

Reverse-engineered all existing Alteryx workflows.

Rewrote the entire system in Python, using SQLAlchemy for database connections and pandas for data processing.

Implemented parallel processing using `concurrent.futures.ThreadPoolExecutor`, a capability the original Alteryx system lacked.

Result:

Delivered a production-ready Python system in 2 weeks, eliminating Alteryx licensing dependencies.

Achieved zero data loss during migration and improved performance via parallel processing.

Provided the team with enhanced error handling and monitoring capabilities.

AI-Driven Content Automation & SEO Optimization System

Project: AI-Driven Content Automation & SEO Optimization System

Company: White Label Resell

Role: Machine Learning Engineer (Sole Backend/AI/Cloud Developer)

Duration: Ongoing

Situation: White Label Resell's clients needed large-scale, multi-language, SEO-optimized content, but manual creation was too slow and expensive to scale for e-commerce needs.

Task: As the sole backend/AI/cloud developer, single-handedly build a comprehensive AI-powered content automation system from scratch to generate over 50,000 articles monthly.

Action:

AI Models: Integrated GPT-3.5, BERT, and FLAN-T5 for text; Stable Diffusion for images; and Google Translate API for 50+ languages.

Infrastructure: Architected a complete AWS cloud system (API Gateway, ECS, Lambda, S3, SageMaker).

Core Feature: Developed a custom graph-based AI algorithm for automated internal SEO linking.

Result:

As a solo developer, built an enterprise-scale system generating 10,000+ articles monthly, securing \$50-100K in quarterly contracts.

Achieved a 60% increase in organic traffic for clients via the AI-powered internal linking.

Boosted international engagement by 40% with multi-language support.

Sky Scraper - AI-Powered Chrome Extension

Project: Sky Scraper - AI-Powered Chrome Extension for Scalable Content Automation

Company: SEOContent.ai

Role: AI Infrastructure Engineer

Duration: 6 Months

Situation: SEOContent.ai clients spent hours on manual keyword research and article structuring, preventing them from building large-scale SEO content campaigns.

Task: Build an AI-powered Chrome extension to automate the entire content workflow—from keyword research to generating clusters of up to 15 interconnected articles.

Action:

Designed a microservices architecture on AWS (ECS, Lambda).
Used fine-tuned BERT models for keyword extraction and topic clustering.
Integrated GPT models (GPT-3.5, GPT-NeoX) for content generation.
The system combined web scraping (via the extension) with parallel processing NLP workflows.

Result:

Slashed content creation time by 60-70% (from 4 hours down to 10 minutes + editing).
Successfully launched the beta, leading to contract renewals and measurable organic traffic improvements for clients using the automated content clusters.

Zero-Maintenance AI Tools Platform

Project: Zero-Maintenance AI Tools Platform with Dynamic Configuration System

Company: SEOContent.ai

Role: AI Infrastructure Engineer

Duration: 3 Weeks

Situation: SEOContent.ai wanted a platform of 50+ AI tools, but knew that requiring developers to maintain each tool would create massive operational bottlenecks.

Task: In 3 weeks, build a "zero-maintenance" platform where non-technical content experts could manage, update, and deploy tools themselves.

Action:

Architected an innovative hybrid system: used Google Sheets as a simple CMS for non-technical staff.

An Apps Script trigger in Google Sheets called an AWS Lambda function, which processed the sheet and saved a new JSON configuration file to Amazon S3.

The JavaScript frontend dynamically rendered the tools at runtime based on the JSON config file, which was served via CloudFront CDN.

Result:

Launched in 3 weeks, gaining 5,000 users in the first month.

Cut platform operational costs by 98.4% (\$50/month vs. \$2000+/month for no-code tools).

Tool updates now take under 90 seconds (done by content experts) instead of waiting days for a developer deployment.

AI-Powered Content Generation API

Project: AI-Powered Content Generation API for SEO Marketing

Company: SEOContent.ai

Role: AI Infrastructure Engineer

Duration: 3 Weeks

Situation: SEOContent.ai needed to commercialize its existing internal AI content generation system by building a scalable, market-ready API for external clients.

Task: In 3 weeks, architect a complete API system using AWS API Gateway to securely expose the existing AI models (GPT-3.5, BERT, Stable Diffusion) and handle thousands of concurrent requests.

Action:

Built a comprehensive REST API using AWS API Gateway with Lambda functions as the integration layer to the existing AI pipeline.

Implemented AWS Cognito for secure API key management and integrated Stripe for usage-based billing.

Optimized performance using asynchronous processing patterns and API Gateway caching.

Result:

Delivered a production-ready API in 3 weeks.

Reduced content generation time by 85% (from 2.5 minutes to 45 seconds) through optimization.

The system serves 10,000+ monthly requests with 99.9% uptime, enabling 90% successful client integrations.

Medical Image Data Engineering Pipeline

Project: Medical Image Data Engineering Pipeline for Pathology Workflows

Company: Digital Clinics Research and Services

Role: Data Scientist

Duration: 12 Months

Situation: Pathologists at Digital Clinics faced major bottlenecks, unable to efficiently analyze massive 10-20GB medical imaging files (TIFF/SVS) for cancer diagnosis.

Task: Engineer a complete, automated data processing pipeline to break down these massive files, create efficient annotation workflows, and integrate with the CADD4MBC clinical platform.

Action:

Built a robust Python pipeline using OpenCV and NumPy for automated file processing (tiling, scaling, and format conversion).

Integrated QuPath (an annotation tool) with custom Groovy scripts to create a seamless annotation workflow for junior pathologists.

Used MongoDB for metadata tracking and Flask/FastAPI for integration APIs.

Result:

The pipeline processed 1.5TB of data weekly with zero data loss.

Reduced manual processing time by 60% and enabled the processing of 100,000+ images monthly.

This pipeline became the foundation for all downstream AI model development.

Tubule Detection using Distributed Deep Learning

Project: Tubule Detection for Breast Cancer Grading

Company: Digital Clinics Research and Services

Role: Data Scientist

Duration: 3 Months (within 12-month timeline)

Situation: To automate the Nottingham Grading System for breast cancer, the clinic needed to automate tubule detection, a process that was slow and had high inter-observer variability among pathologists.

Task: Develop a deep learning segmentation model with medical-grade accuracy (>90%) and optimize its training efficiency.

Action:

Fine-tuned an R2U-Net (a Recurrent Residual U-Net) architecture, which is specialized for medical image segmentation.

Implemented Horovod distributed training to run the workload across 4 GPUs (NVIDIA DGX), using the annotated dataset from my data pipeline (Project 10).

Result:

Achieved a 92% F1-score, exceeding the clinical accuracy goal.

Reduced training time by 3.9x (from 27 hours down to 7 hours).

Presented these findings at the SIIM-CMIMI22 International Conference at Johns Hopkins Hospital.

Mitosis Detection using Multi-Architecture Deep Learning

Project: Mitosis Detection for Breast Cancer Grading

Company: Digital Clinics Research and Services

Role: Data Scientist

Duration: 4 Months (within 12-month timeline)

Situation: The second critical component for the Nottingham Grading System was automating mitosis (cell division) detection, which was also extremely time-consuming and prone to disagreement between experts.

Task: Implement and compare two different object detection architectures (two-stage vs. single-stage) to find the best model for this task, using the 56,258 annotated images from my pipeline.

Action:

Implemented a two-stage detector, Faster R-CNN, using the Detectron2 framework.

Implemented a single-stage detector, YOLOv5, and hypothesized it would offer a better speed-precision balance.

Conducted a systematic performance comparison using the massive annotated dataset.

Result:

YOLOv5 outperformed Faster R-CNN, achieving an 84% F1 score (a 7% improvement).

This comparative analysis was published in the International Journal of Advanced Computer Science and Applications (IJACSA), establishing a new benchmark.

Software Portable Radio Simulator

Project: Software Portable Radio Simulator

Company: Edgeforce Solutions

Role: Data Scientist Intern

Duration: 3 Months

Situation: The Indian Army needed a cost-effective software-based training simulator to replace expensive physical radio hardware.

Task: As the sole developer, build a complete real-time voice communication system from scratch to connect 3-4 laptops with sub-500ms latency, while managing frequent client requirement changes.

Action:

Built the entire system in Python. Used WebRTC for real-time communication protocols and PyAudio for audio processing.

Made a critical technical decision to switch the GUI from Streamlit to Tkinter to meet the client's mandatory offline deployment requirement.

Created a Software Requirement Specification (SRS) to freeze requirements and prevent scope creep.

Result:

Delivered a successful system achieving sub-200ms latency (an 85% improvement from the initial 1.5s).

Reduced hardware dependency by 80% and helped Edgeforce Solutions win additional defense contracts.

AI-Powered Wine Inventory Management System (POC)

Project: AI-Powered Wine Inventory Management System (POC)

Company: Edgeforce Solutions

Role: Data Scientist Intern

Duration: 3-4 Weeks

Situation: Edgeforce Solutions needed to prove to army clients that AI could solve their manual wine inventory problems, which were full of counting errors and inefficient tracking.

Task: In just 3-4 weeks, build a complete proof-of-concept (POC) from scratch to validate that computer vision could accurately detect wine bottles and justify funding for a full-scale project.

Action:

Model: Implemented YOLOv5 for real-time object detection.

Data: Rapidly created an annotated dataset using Roboflow.

Demo: Developed a Streamlit web dashboard with live video streaming (via WebRTC) to present the solution live to stakeholders.

Result:

Delivered the POC in 3 weeks, achieving 87% detection accuracy.

Validated a potential 30% time reduction in manual counting.

The demo successfully convinced army stakeholders, who approved funding for the full-scale implementation.

Saahil Jain Portfolio:

Saahil Jain

I am currently working on You.com, a new search engine.

I'm broadly excited by artificial intelligence (AI), deep learning, and natural language processing (NLP). At Stanford, I researched these topics under Professor Andrew Ng in the Stanford Machine Learning Group. Much of my research focused on developing better methods of learning from limited/noisy labels as well as building multimodal datasets and models in resource-constrained domains such as healthcare.

I've worn various hats, working across engineering, product, and research. I received my MS in Computer Science at Stanford University. Prior to Stanford, I worked as a product manager at Microsoft. I received my BS at Columbia University, where I studied Computer Science and Economics.

To get in touch, feel free to email me at saahil.jain@cs.stanford.edu.

Email / Google Scholar / LinkedIn / Twitter / Github

profile photo

Experiences

Engineer, You.com [2020-Present]

Researcher, Stanford Machine Learning Group (Andrew Ng's group)

Fellow, 2021 Accel Innovation Scholar

Engineer, Datavant

Product Manager, Microsoft

Researcher, Tatonetti Lab, Columbia Medical Center

Researcher, Wireless & Mobile Networking Lab, Columbia University

Research

RadGraph: Extracting Clinical Entities and Relations from Radiology Reports

Saahil Jain*, Ashwin Agrawal*, Adriel Saporta*, Steven QH Truong, Du Nguyen Duong, Tan Bui, Pierre Chambon, Yuhao Zhang, Matthew P. Lungren, Andrew Y. Ng, Curtis P. Langlotz, Pranav Rajpurkar
Proceedings of the Neural Information Processing Systems (NeurIPS) Track on Datasets and Benchmarks, 2021, (Oral)

paper / dataset

VisualCheXbert: Addressing the Discrepancy Between Radiology Report Labels and Image Labels

Saahil Jain*, Akshay Smit*, Steven QH Truong, Chanh DT Nguyen, Minh-Thanh Huynh, Mudit Jain, Victoria A. Young, Andrew Y. Ng, Matthew P. Lungren, Pranav Rajpurkar

Proceedings of the Conference on Health, Inference, and Learning (ACM-CHIL), 2021

paper / code

Effect of Radiology Report Labeler Quality on Deep Learning Models for Chest X-Ray Interpretation

Saahil Jain*, Akshay Smit*, Andrew Y. Ng, Pranav Rajpurkar

Neural Information Processing Systems (NeurIPS) Workshop on Data-Centric AI (DCAI), 2021

paper

On the Opportunities and Risks of Foundation Models: §3.1 Healthcare and Biomedicine

Michihiro Yasunaga, Jing Huang, Camilo Ruiz, Yuhui Zhang, Giray Ogut, Saahil Jain, William Wang, Yusuf Roohani, Hongyu Ren, Antoine Bosselut, Ehsan Adeli, Jure Leskovec, Russ Altman

Whitepaper, 2021

paper

Q-Pain: A Question Answering Dataset to Measure Social Bias in Pain Management

Cécile Logé*, Emily Ross*, David Yaw Amoah Dadey, Saahil Jain, Adriel Saporta, Andrew Y. Ng, Pranav Rajpurkar

Proceedings of the Neural Information Processing Systems (NeurIPS) Track on Datasets and Benchmarks, 2021, (Oral)

paper / dataset

CheXbert: Combining Automatic Labelers and Expert Annotations for Accurate Radiology Report Labeling Using BERT

Akshay Smit*, Saahil Jain*, Pranav Rajpurkar*, Anuj Pareek, Andrew Y. Ng, Matthew P. Lungren

Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020

paper / code

Power-Aware Neighbor Discovery for Energy Harvesting Things: Demo Abstract

Tingjun Chen, Gregory Chen, Saahil Jain, Robert Margolies, Guy Grebla, Dan Rubenstein, Gil Zussman

Proceedings of the 14th ACM Conference on Embedded Network Sensor Systems (SenSys), 2016

paper

Teaching

Teaching Assistant: Stanford CS 372 [2020, 2021], AI for Disease Diagnosis and Information
Recommendations with Edward Chang

Teaching Assistant: Stanford CS 229 [2020], Machine Learning with Andrew Ng, Chris Ré, and Tengyu
Ma

Website Template

Hi Faseeh,

As a reminder, here is more information about your interview:

Hiring Manager Behavioral Interview - 30 min

Saahil Jain

October 29, 2025

1:30pm EDT

[Skip to main content](#)

[Products](#)

[Solutions](#)

[Company](#)

[Pricing](#)

[Developer Docs](#)

[Sign in](#)

[Book a Demo](#)

[Book a Demo](#)

AI search infrastructure built for enterprise.

Accelerate AI with the leading web search API and vertical-specific search indexes to deliver value in weeks.

[Talk to an Expert](#)

[Talk to an Expert](#)

[Start Building](#)

[Start Building](#)

Two ways to build

Composable AI for every enterprise workflow.

Web Search API for LLMs

API for LLMs

Create AI apps and agents powered by the most accurate web data, delivering citation-rich insights with unmatched freshness.

[Get your free API key](#)

[Get your free API key](#)

[Learn More about Web Search API](#)

[Learn More](#)

Enterprise AI Solutions

Indexes for LLMs

Deploy vertical and domain-specific indexes to deliver high-impact, trusted apps and agents.

[Book a Demo](#)

[Book a Demo](#)

[Learn More about Enterprise AI Solutions](#)

[Learn More](#)

features

Foundational models are powerful — but are not built to power great AI app experiences.
You.com provides the missing link in Enterprise AI success, with our AI search infrastructure.

Complete control
Easily build apps, improve accuracy, and curate sources.

Unparalleled accuracy
Citation-backed and auditable outputs you can trust.

Enterprise-grade scalability
Securely power your apps and workflows with AI governance and control.

apis

Connect your LLMs with real-time, trusted data.
One endpoint for the freshest, most grounded web data— no more juggling multiple endpoints or worrying about coverage gaps.

Explore Web Search API
Explore APIs

Purpose-built for LLMs
Optimized to support complex reasoning, long-form generation, and grounded outputs at scale.

Deeper than Google
Created for real-world factual scenarios where accuracy & relevance are paramount.

Designed for low latency
Deliver agentic AI applications that work at human speeds

Explore our full
suite of AI-ready APIs.
Deliver real-time, citation-backed web results with our Web Search API — so your AI is always accurate, grounded, and trustworthy.

Explore Web Search API
Explore APIs

Live News API

Stay ahead with up-to-the-minute news from leading global sources, capturing deep, uniquely detailed insights from millions of trusted publishers.

[Get Started for Free](#)

[Get Started for Free](#)

Image Search API

Bring your apps to life with our Image Search API, instantly surfacing the most relevant visuals from millions of images across the web.

[Get Started](#)

[Get Started](#)

Data API

Integrate your private data into LLM-ready custom pipelines, unlocking the full potential of your proprietary content with secure, tailored integration.

[Get Started for Free](#)

[Get Started for Free](#)

VERTICAL INDEXES

Deliver high-impact apps and agents with vertical indexes

Higher relevance, richer responses for superior end-user AI experiences in industries like hospitality, legal, retail, and finance

[Book a Demo](#)

[Book a Demo](#)

Curated Domain-Specific Sources

High-quality targeting of vertical and topic-specific data sources from the public web, partner portals, and licensed specialty sources.

Deeper Extraction & Structured Enrichment

Flexible, sophisticated search for AI

Engineered to scale. Loved by leaders.

Whether deploying AI worldwide or managing complex workloads, You.com empowers enterprises to innovate smarter and faster.

API calls monthly
1B+

agents built
100K+

Fortune 500 penetration
57%

Turning AI into ROI.

Discover how innovators are transforming their businesses with you.com.

"With You.com, leading publisher Wort & Bild Verlag significantly fast-tracked literature reviews and captioning, cutting process time by more than 90%"

Wort & Bild Verlag

"Through our collaboration with You.com, we've gained practical insights and hands-on experience that are crucial for staying ahead in this rapidly evolving landscape... ensuring we remain at the forefront of innovation in our industry."

Paul Heard

Chief Information Officer at Mimecast

"You.com is the AI company that had built research and citation models into their product from the start."

Dennis Ballwieser

Managing Director at Wort & Bild Verlag

"DPA, Germany's largest news agency, leveraged You.com to simplify and scale their journalistic processes, helping staff prepare for interviews and streamline their workflows – all within a matter of weeks."

dpa

"German insurance provider HanseMerkur partnered with You.com to launch compliant, enterprise-wide AI access, leveraging our hands-on training to get teams on board quickly and effectively."

HanseMerkur

"We are working to ensure that generative AI supports and enriches verified and factual journalism. In You.com, we have found an excellent and trustworthy partner for this."

Peter Kropsch

CEO of dpa

"With You.com, leading publisher Wort & Bild Verlag significantly fast-tracked literature reviews and captioning, cutting process time by more than 90%"

Wort & Bild Verlag

"Through our collaboration with You.com, we've gained practical insights and hands-on experience that are crucial for staying ahead in this rapidly evolving landscape... ensuring we remain at the forefront of innovation in our industry."

Paul Heard

Chief Information Officer at Mimecast

"You.com is the AI company that had built research and citation models into their product from the start."

Dennis Ballwieser

Managing Director at Wort & Bild Verlag

"DPA, Germany's largest news agency, leveraged You.com to simplify and scale their journalistic processes, helping staff prepare for interviews and streamline their workflows – all within a matter of weeks."

dpa

"German insurance provider HanseMerkur partnered with You.com to launch compliant, enterprise-wide AI access, leveraging our hands-on training to get teams on board quickly and effectively."

HanseMerkur

"We are working to ensure that generative AI supports and enriches verified and factual journalism. In You.com, we have found an excellent and trustworthy partner for this."

Peter Kropsch

CEO of dpa

FAQs

Answering your most common questions.

We get it. AI tools can be confusing. These answers cover the basics — but if you want to dive deeper, just reach out.

What is You.com?

What makes You.com different from ChatGPT or other LLMs?

What makes You.com different from other AI solution stacks?

Can I connect my own data to You.com?

How does You.com handle data privacy and security?

Ready to unlock the full potential of AI?

Schedule a demo today and discover how You.com transforms AI into real-world ROI.

Schedule a demo today and discover how You.com transforms AI into real-world ROI.

Book a Demo

Making enterprise AI accurate, secure, and yours.

Products

Web Search API

Enterprise AI Solutions

Chat

Solutions

Media
Customers
Pricing
Company

About
Careers
Resources

Blog
Developer Docs
© 2025 You.com

All rights reserved

Privacy Policy
Terms & Conditions

Reach out if you want to solve novel, messy problems and drive real world impact with AI — we're aggressively hiring for FDE roles (all levels) on my team!

36,568 followers

1mo • Edited • 1 month ago • Edited • Visible to anyone on or off LinkedIn

We're happy to announce that we've raised \$100M in Series C funding at a \$1.5B valuation, led by Cox Enterprises! Thanks to existing investors Georgian, Salesforce Ventures, Norwest, and others for continuing the journey. We're also thrilled to welcome David Yang from Cox Enterprises to our Board.

In this next chapter, we're building the highways for the agentic era.

Soon there will be more AI agents using the web than humans, but today's search infrastructure wasn't designed for this.

That's why we are building the next generation of Composable Infrastructure for the AI Enterprise.

We're now delivering the fastest, most accurate search capabilities for agents and LLMs. Today, our APIs process over 1B queries monthly for incredible companies like DuckDuckGo, Windsurf, Harvey, and many others.

Come build with us. [🔗](#) Links in the comments.

It's a behavioural interview you have everything

I want you to consume evrything and help me prepare in the best way possible for this 30 minute interview with saahil jain I have tomorrow and what kind of questions will he ask as this is and I want you to think

so its' more of. a forward deployed engineer role so

Customer Obsession

Who was your most difficult customer? (1)

Situation:

While working at Edgeforce Solutions, I worked with a customer who required a ML application for their business. They had a manual deployment process that depended on a specific team member's availability, causing delays. I suggested implementing a deployment pipeline to automate updates, streamline testing, and reduce manual effort. However, the customer dismissed the idea, concerned it would delay initial progress, despite the clear long-term benefits.

Task:

My goal was to deliver the project efficiently while addressing the customer's frequent requests for changes. I needed to find a way to resolve the delays caused by their manual process without creating friction or eroding trust."

Actions:

- I remained patient and continued to work within their existing process while noting inefficiencies.
- As the delays mounted, I revisited the suggestion of a deployment pipeline, carefully explaining how it could improve turnaround times and reduce frustration.
- To address their concerns, I created a proof of concept to demonstrate the pipeline's value and ensured it could be configured with manual triggers to align with their team's workflow.
- After showing how the pipeline would simplify updates and maintain their control, the customer agreed to proceed. I implemented the solution with GitLab, allowing manual deployments triggered at their discretion."

Result:

The pipeline reduced deployment times from hours to minutes, significantly improving efficiency. This not only shortened the overall project timeline but also eliminated scheduling bottlenecks. The customer appreciated the tailored approach and flexibility, leading to a smoother development process. This experience taught me the value of persistence, clear communication, and adapting technical solutions to align with customer concerns.

"Tell me about a time when you didn't meet customer expectations. What happened, and how did you deal with the situation? (2)

Situation:

At RIT's Office of Vice President of Research I am part of a Pre-Award Team who assist professors, PHD Students in applying for Proposals at NSF, NIH, DO. Initially a Pre Award Team member would work with a Professor in applying for grant and they would make changes in an excel and a doc where doc was used to submit a proposal and each grant had their own Template, due to editing at multiple places sometimes it resulted in data integrity issues which cause in rejection of a grant, so I was assigned a Project to build an automation tool for automatic transfer of budget data from Excel into Word templates for submission to grants. The users expected the tool to handle all Excel formats they used. After the first release, a few users reported that the tool failed for some previous versions of budget sheets with merged cells and custom formulas. This meant the tool did not fully meet their expectations on day one.

Task:

My responsibility was to fix the feature gap quickly, without breaking existing workflows, and to regain the users' trust.

Action:

I first acknowledged the issue openly and explained why those edge cases were missed initially. I then sat with the users to understand how they actually built their files. From a feature perspective, I improved the Excel parsing logic, added validations, and built fallback handling for merged cells and uncommon formats. I also added clear error messages so users knew exactly what needed fixing if a file failed.

Result:

The updated version handled all real-world budget files successfully. Processing time dropped, errors went to zero, and users no longer needed manual fixes. The team appreciated the responsiveness and later asked me to automate additional proposal sections beyond the original scope.

Learning:

I learned that customer expectations are defined by real usage, not assumptions, and early feedback on core features is critical.

Ownership

Tell me about a time when you took on a task that was beyond your job responsibilities. (7)

Situation:

While working as an R&D Data Governance Intern at Daiichi Sankyo, my main responsibility was AI-related work. During this time, I noticed that my team member was using Alteryx (a data analytic software) for data snapshot to monitor for data quality issues, which meant licensing costs, delays, and limited flexibility for the team.

Task:

Although this was not part of my assigned role, I decided to take ownership of the problem because it was costing the company in the long term while not providing that much flexibility. I proposed migrating the entire system from Alteryx to Python to remove the dependency and improve performance.

Action:

On my own initiative, I reverse-engineered the existing Alteryx workflows and rebuilt them in Python using SQLAlchemy and Spark. I also added parallel processing, better error handling, and logging. I worked on this alongside my regular responsibilities and completed the migration within two weeks.

Result:

The team completely removed the Alteryx dependency, saved licensing costs, and achieved faster and more reliable migrations with zero data loss. The solution became the new standard workflow, and my manager appreciated that I stepped beyond my role to solve a broader team problem, and I was assigned an additional Data Migration project.

Learning:

This experience taught me that taking ownership beyond assigned tasks can create meaningful impact when you focus on real team and business pain points.

Describe a situation where you made an important business decision without consulting your manager. Tell me about a time when you made a decision on your own without the manager's prior approval (9)

(Alteryx to Python Migration)

Situation:

While working as an R&D Data Governance Intern at Daiichi Sankyo, I noticed that a teammate was using Alteryx to generate data snapshots for monitoring data quality issues. This approach caused recurring licensing costs, slow execution, and limited flexibility for changes.

Decision:

Without prior approval from my manager, I decided to build a Python-based replacement. I made this decision because the work was low risk, the cost impact was clear, and the team needed a more flexible solution quickly.

Action:

I independently reverse-engineered the Alteryx workflow and rebuilt it in Python using SQLAlchemy and Spark. I added parallel processing, structured logging, and better error handling. I completed a working version within two weeks while continuing my regular responsibilities.

Result:

The Python solution fully replaced Alteryx, eliminated licensing costs, and delivered faster and more reliable data snapshots with zero data loss. After reviewing the results, my manager approved the approach, adopted it as the new standard workflow, and assigned me an additional data migration project.

Learning:

This experience taught me that making independent decisions is appropriate when the business impact is clear, the risk is controlled, and the solution can be validated quickly.

Tell me about a time when you had to decide between long-term value or short-term results.(10)

Situation:

At Daiichi Sankyo, I was asked to build a machine learning model to classify Informed Consent Form PDFs for data sharing clauses. Initially, there was no labeled dataset, and the first version of the model was delivered quickly through a simple Streamlit interface. While this gave fast results, the model achieved only about **0.63 F1 score**, and legal reviewers said they did not trust the output because they could not understand why a paragraph was classified a certain way.

Decision and Trade-off:

At that point, I had a choice. I could focus on short-term results by only tuning the model to slightly improve accuracy, or I could invest time in building explainability, feedback, and data foundations that would deliver long-term value. I chose the long-term approach, even though it meant delaying quick accuracy gains.

Action:

I went beyond model tuning and built a full product around it. I developed a frontend using React and a backend using Flask. I added a feature that allowed users to click on a prediction and see the exact paragraph highlighted in the PDF, giving them clear context. I also added a feedback mechanism so legal reviewers could correct classifications. This feedback was then used to create a high-quality labeled dataset and retrain the model over time.

Result:

By investing in the long-term foundation, the model's F1 score improved from **0.63 to 0.94**, user trust increased, and document review time dropped by over **80 percent**. More importantly, the system became part of the regular legal review workflow instead of a one-off experiment.

Learning:

This experience taught me that choosing long-term value, even at the cost of short-term speed, leads to better adoption, stronger outcomes, and solutions that actually last.

Tell me about a time when you showed an initiative to work on a challenging project.

Describe a time when you took on work outside of your comfort area.
(12)

ICF Project answer from streamlit to reactJS

Phrases

Was this the answer u were expecting

Does this answer your question

Deliver Results

Could you tell me about a time when you were working on a project in a pair and over time, that person lessened their involvement in the project and you had to take on more responsibility?

Situation:

During my capstone project I was building a mobile app based on federated learning, I was working in a pair. As we approached a sprint deadline, my partner's involvement reduced significantly due to getting ill suddenly. At that stage, we had to *demonstrate inference* or explain model behavior.

Task:

To ensure the project was complete and defensible, I needed to take over additional responsibility and design a proper **model inference experience** that clearly showed predictions, confidence scores, and correctness.

Action:

I took full ownership of the inference page. I designed and implemented a clean UI that loaded a federated model checkpoint, displayed the test image, ran inference on demand, and showed predicted class, ground truth, correctness, and per-class confidence scores. I wired this end to end with the FastAPI backend so results were reproducible and easy to validate during evaluation.

Result:

The inference page became the centerpiece of our final demo. Reviewers could clearly see how the federated model behaved, not just that it trained. Despite my partner stepping back, the project was delivered on time with a polished, explainable interface that strengthened both the technical credibility and presentation quality.

Learning:

This experience reinforced that when team capacity changes, stepping up on user-facing validation and explainability can significantly elevate the perceived and actual quality of a system.

Give an example of a time when you had to handle a variety of assignments. What was the outcome?

Situation:

During my internship at Edgeforce Solutions, I was working on an AI-powered wine inventory management proof of concept for army stakeholders. My primary responsibility was building the computer vision model for bottle detection on an edge device. At the same time, we had an upcoming stakeholder presentation that required a working frontend demo. My teammate, who was focused on backend and model support, did not have experience building frontends and was unable to take on that task close to the deadline.

Task:

I needed to handle multiple responsibilities at once: continue improving the AI model, build a presentable frontend for real-time inference, and support my teammate so the overall project stayed on track.

Action:

I prioritized the most time-sensitive work first, which was creating a functional demo. Alongside model training and optimization, I took ownership of building a Streamlit-based frontend using WebRTC to show live video streaming with real-time object detection and counting. In parallel, I shared reusable code and walkthroughs with my teammate to help them understand the frontend setup, while still coordinating model integration and testing.

Result:

We delivered a complete end-to-end demo on time. The system achieved around 87 percent detection accuracy, demonstrated real-time counting on an edge device, **and impressed army stakeholders. The successful presentation helped secure approval and funding for the next phase of development.**

Learning:

This experience taught me how to prioritize under pressure, switch contexts efficiently, and take ownership of additional responsibilities to ensure team success.

Give an example of a time when you've left a project in a better position than you've found it.

Situation:

When I joined Digital Clinics as a Data Scientist, the medical image processing workflow was largely manual. Large 10–15 GB TIFF files were manually broken down, sent to different annotators, collected back, reorganized, and then uploaded to the server. This process was slow, error-prone, and created delays in cancer diagnosis workflows.

Task:

My responsibility was to support the model development process, but I saw that the bigger issue was the inefficient pipeline itself.

Action:

I designed and built an end-to-end automated data engineering pipeline.

First, I wrote scripts to automatically break large TIFF files into PNGs in a structured way.

Next, I integrated annotation workflows using QuPath and Groovy so annotators could work directly without manual file handling.

Finally, I built scripts that allowed annotated images to be pushed directly to the server along with annotator name and annotation type, removing manual coordination completely.

I also learned Groovy, QuPath, and Linux tooling quickly to ensure the solution fit existing clinical workflows.

Result:

This resulted in making the workflow more efficient and faster as initially senior pathologists had to wait until a developer would upload it to the server for them to view which was automated and now they could instantly view the data as soon as junior pathologists were done annotating. The automated workflow became the foundation for downstream AI models and future projects at Digital Clinics.

Learning:

This experience taught me that improving core infrastructure and workflows can create long-term value far beyond the initial scope of a project.

Earn Trust

Describe a time when you had to speak up in a difficult or uncomfortable environment. (31)

Situation:

While working at Digital Clinics we were focused too much on the model building and training solution for breast cancer diagnosis, the team was focused on processing large TIFF files and delivering annotated data for downstream AI models. As deadlines approached, I noticed that most of the effort was going into training, while testing, validation checks, metadata management and documentation were being overlooked. Given that this was medical data, any silent data corruption or undocumented workflow could have serious downstream consequences.

Task:

Although I was relatively new to the team, I felt responsible for raising concerns about the risks of insufficient testing and documentation. Speaking up was uncomfortable because everyone was under pressure to deliver quickly, and slowing down was not a popular suggestion.

Action:

I raised this concern during a team discussion and explained specific failure scenarios, such as incorrect data lineage, incorrect annotation mapping, or missing metadata, and how these could affect clinical decisions or model training. Instead of just pointing out the problem, I proposed a practical solution: adding lightweight validation checks at each pipeline stage and maintaining concise documentation explaining file flow, annotation formats, and recovery steps. I also volunteered to implement the initial tests and write the first version of the documentation to reduce friction.

Result:

The team agreed to adopt the changes. The documentation made onboarding easier, and the pipeline became significantly more reliable which helped in understanding how 2 different xray machines were affecting the model's performance. This reduced rework later and increased confidence among both engineers and clinical collaborators. Speaking up improved the quality of the system without delaying delivery.

Learning:

This experience taught me that raising quality concerns early, even when it feels uncomfortable, is essential, especially in high-impact domains like healthcare. How you frame the concern matters as much as the concern itself.

What would you do to gain the trust of your team? (32)

To gain the trust of my team, I focus on four consistent behaviors: **transparency, reliability, ownership, and respect for feedback.**

Clear and Honest Communication(transparency):

I build trust by being transparent about progress, risks, and limitations. For example, during the ICF Classification project at Daiichi Sankyo, I openly communicated that early model versions had limited accuracy and explainability. Instead of overselling results, I clearly explained what the system could and could not do, which helped legal and technical teammates trust both the process and my judgment.

Reliability and Follow-Through(reliability):

I make sure that when I commit to something, I deliver. In the CTMS Alteryx-to-Python migration, I took ownership of rebuilding the workflows independently and delivered a working system within two weeks. Consistently meeting commitments showed the team that they could depend on me, even under tight timelines.

Ownership Beyond My Role(ownership):

I earn trust by prioritizing team success over role boundaries. In the ICF project, although my responsibility was only the ML model, I took initiative to build the frontend, backend, to make adoption of this tool more reliable. This demonstrated that I care about outcomes, not just tasks.

Listening and Adapting(respect for feedback):

I actively seek feedback and adapt based on team needs. While working with legal reviewers and clinicians, I incorporated their feedback directly into the product by adding paragraph highlighting and feedback loops. This made stakeholders feel heard and increased confidence in the system.

Overall, I believe trust is built when people see that you are honest, dependable, willing to take responsibility, and genuinely invested in the team's success.

Dive Deep

Tell me about the most complicated problem you've had to deal with.

(33)

Situation:

At SEOContent.ai, we wanted to launch a platform with more than 50 SEO-focused AI tools. From prior experience, we knew that a traditional approach, where developers manually maintain every tool change, would quickly become a bottleneck. SEO Engineers and Content experts needed to update prompts, inputs, and configurations frequently, but relying on engineers for every small change would slow the business and significantly increase operational costs.

Task:

My responsibility was to solve this at a foundational level. I needed to design a system that allowed non-technical content experts to create, update, and deploy SEO tools independently, without sacrificing performance, reliability, or cost efficiency. This had to be done in just three weeks and had to scale well beyond the initial set of tools.

Action:

I stepped back from conventional solutions and evaluated multiple options, including custom admin dashboards, CMS platforms, and no-code tools(make.com, airflow, etc). Each of these either added high costs or long-term maintenance overhead. I decided to design a configuration-driven architecture instead. I built a hybrid system where Google Sheets acted as the control plane for tool definitions, since it was already familiar to content teams. These configurations were converted into JSON and reflected dynamically by the frontend at runtime, meaning tools could change instantly without any code deployment. On the backend, I used a serverless AWS architecture to process requests efficiently and keep costs minimal. I also added validation and safeguards so that non-technical changes could not break the system.

Result:

The platform launched successfully within three weeks. Tool updates dropped from developer-led changes to under 90 seconds with no engineering involvement. We reduced operational costs by over 98 percent compared to traditional no-code platforms, onboarded 5,000 users in the first month, and established a system that could scale to hundreds of tools without increasing maintenance burden.

Why this was the most complicated problem:

This problem was not just technically complex. It required balancing long-term maintainability, cost optimization, user experience for non-technical stakeholders, and speed of delivery under ambiguity. The solution fundamentally changed how the business operated, not just how software was built.

Tell me about something that you have learned in your role.(35)

Situation:While working as a Data Scientist at Digital Clinics Research and Services, I joined a medical imaging project where the data pipeline was largely manual and poorly documented. Critical steps like TIFF and SVS file processing, annotation mapping, and uploads relied on individual knowledge rather than defined workflows.

Action:

As I worked on model training, I redesigned parts of the pipeline by documenting the full data flow, refactoring manual steps into modular Python scripts, and adding basic validation checks to prevent data loss. I also collaborated with pathologists to standardize how data was annotated and uploaded, reducing dependency on developers for routine tasks.

Result and Learning:

This experience taught me that strong documentation, clear processes, and validation are just as important as model accuracy. Without them, even good models fail to scale or gain trust. Since then, I always prioritize building systems that are understandable, reliable, and easy for others to use.

Bias for Action

"Provide an example of when you took a calculated risk.(28)"

Situation:

While working as an R&D Data Governance Intern at Daiichi Sankyo, I noticed that a critical data quality monitoring workflow relied on Alteryx. This caused high licensing costs, slow execution, and limited flexibility, but replacing it was considered risky because the workflow touched production data.

Task:

Although this was outside my assigned responsibilities, I decided to take a calculated risk by proposing a full migration from Alteryx to Python. The risk was that a failed migration could disrupt data quality monitoring, but the long-term cost and scalability benefits were significant.

Action:

Before making the change, I carefully analyzed the existing workflows, validated data outputs, and built a parallel Python version using SQLAlchemy and Spark. I added logging, error handling, and ran side-by-side comparisons with Alteryx to ensure zero data loss. I completed the migration alongside my regular work within two weeks.

Result:

The migration succeeded with zero data loss, eliminated licensing costs, improved performance through parallel processing, and became the new standard workflow. My manager appreciated the ownership and impact, and I was trusted with an additional data migration project.

Learning:

This experience taught me that calculated risks are about reducing uncertainty through validation and safeguards, not avoiding risk entirely.

Describe a situation when you took the initiative to correct a problem or a mistake rather than waiting for someone else to do it. (29)

(Alteryx to Python Data Migration)

"Tell me about a time when you were misunderstood by a team member or supervisor.

Situation:

While working as the sole backend and AI engineer at White Label Resell, I was building an AI-powered content automation system that relied heavily on GPT-3.5 for generating multiple content types such as headlines, metadata, FAQs, and long-form articles. During development, we started seeing frequent failures where content generation would fail even after multiple retries because the external API was intermittently unavailable.

Task:

When I raised this issue, my manager initially believed the problem was with my implementation and assumed I was blaming the external model instead of fixing bugs in my code. This created a misunderstanding, as the concern was actually about system reliability and dependency risk, not implementation quality.

Action:

Instead of escalating emotionally, I gathered concrete evidence. I shared detailed logs showing retry attempts, API response codes, and failure timestamps that aligned with known external service outages. I then explained the broader risk: relying entirely on a single external model made the system fragile and unpredictable at scale. To resolve this, I proposed and independently implemented a solution where we built internal datasets for each content type, fine-tuned lightweight models, and hosted them on our own infrastructure as fallbacks. This way, the system could continue functioning even if external APIs were unavailable.

Result:

Once my manager saw the data and the working solution, the misunderstanding was resolved. System reliability improved significantly, API failure impact was minimized, and content generation became more consistent. My manager appreciated the proactive approach and praised the decision to design for resilience rather than short-term fixes.

Learning:

This experience taught me that misunderstandings often come from incomplete context. Clear evidence, calm communication, and proposing concrete solutions are the best ways to regain alignment and build trust, especially when working on complex systems with external dependencies.

Tell me about a time when you required some information from somebody else, but they weren't responsive. What did you do? (30)

Have Backbone; Disagree and Commit

~~Give me an example of something you believe in that nobody else does.
Tell me about an unpopular decision of yours.~~

Situation:

While working at Digital Clinics on an automated breast cancer grading system, my task was to build the mitosis detection component. The initial plan, supported by senior team members, was to use Faster R-CNN because it was considered safer and more established for medical imaging tasks. However, based on early experiments and the need to balance accuracy with inference speed, I believed YOLOv5 could perform better for this specific problem.

Task:

This was an unpopular position because YOLOv5 was viewed as less precise and more suitable for general object detection, not sensitive medical use cases. I had to decide whether to follow the consensus or challenge it with data, knowing that choosing the wrong direction could cost time and credibility.

Action:

I decided to pursue a parallel evaluation instead of arguing theoretically. I implemented both Faster R-CNN and YOLOv5, ensured identical preprocessing and evaluation metrics, and ran a large-scale comparison on over 56,000 annotated mitosis tiles. I worked closely with pathologists to validate results in clinically meaningful terms, focusing on false positives and consistency, not just raw accuracy.

Result:

The results showed that YOLOv5 achieved an 84 percent F1 score, outperforming Faster R-CNN by 7 percent while also being significantly faster. Once the data was clear, the team aligned with the decision. The model became the final choice, set a new internal benchmark, and the work was published in a peer-reviewed journal.

Learning:

This experience taught me that unpopular decisions should be driven by evidence, not opinion. When you back a contrarian view with rigorous validation and stakeholder involvement, it can lead to better outcomes and stronger team trust.

~~Tell me about a situation where you had a conflict with someone on your team. What was it about? What did you do? How did they react? What was the outcome?~~

Describe a time when you disagreed with the approach of a team member. What did you do?

Situation:

While working at RIT's Office of the Vice President of Research, I was part of a project to analyze and visualize Proposal Approval rates.. A team member suggested using tools like Power BI or Google Data

Studio for dashboards. I disagreed because those tools limited customization and made it harder to implement domain-specific visualizations needed for deeper analysis.

Task:

My goal was to respectfully challenge the approach while ensuring we chose a solution that was flexible, fast to iterate on, and aligned with the project's analytical requirements.

Action:

I proposed a data-driven alternative instead of just stating an opinion. I explained that using Streamlit with Matplotlib and Seaborn would allow us to build fully customized visualizations, add advanced filters, and iterate quickly without vendor constraints. To support this, I built a small proof of concept showing interactive charts, category-based filtering, and tailored visual controls that would not be easily achievable in Power BI. I also took ownership of the implementation to avoid slowing the team down.

Result:

After reviewing the proof of concept, the team agreed to proceed with the Streamlit-based dashboard. The final solution provided more flexibility, better analytical depth, and was easier to extend as requirements evolved. This experience reinforced the importance of handling disagreements constructively by backing ideas with evidence and initiative rather than opinion.

~~Often, we must make decisions as a group. Give me an example of a time you committed to a group decision even though you disagreed.~~

Tell me about a time when you had to work with a decision that you disagreed with.

Situation:

While working at RIT's Office of the Vice President of Research, our team needed to build dashboards to visualize Employee Data. I initially recommended using Streamlit with Matplotlib and Seaborn because it would allow more customization and deeper analytical control. However, the group decided to use Tableau since it was already approved, widely used by stakeholders, and faster to adopt as a team.

Task:

Even though I disagreed with the choice from a technical perspective, my responsibility was to fully commit to the group decision and help deliver a high-quality dashboard using Tableau.

Action:

I aligned with the decision and focused on making the Tableau solution as effective as possible. I learned the tool quickly, helped design clear visualizations within its constraints, and suggested ways to structure the data so we could still support meaningful filtering and comparisons. I also supported teammates by sharing best practices and troubleshooting issues to keep the project moving.

Result:

The dashboard was delivered on time and met stakeholder expectations. It was easy for non-technical

users to explore trends and insights, and the project was considered successful. This experience reinforced that strong teamwork sometimes means setting aside personal preferences, committing fully to a shared decision, and focusing on outcomes rather than tools.

Think Big

Tell me about your most significant professional achievement. (25)

Situation:

While working as the sole backend and AI engineer at White Label Resell, I built an AI-powered content automation system that relied heavily on GPT-3.5 for generating headlines, metadata, FAQs, and long-form content. As usage scaled, we started facing frequent failures due to external API downtime, even after multiple retries.

Bold Decision / Vision:

Instead of treating this as a temporary reliability issue, I made the bold decision to reduce our dependency on a single external model. My vision was to design a resilient system that could continue operating even when third-party APIs were unavailable.

Action:

I collected detailed logs showing repeated API failures and aligned them with known outages to clearly demonstrate the risk to the management and why we need to host our own models. I then built internal datasets for each content type, fine-tuned models such as BERT, FLAN-T5, and Vicuna 13B, and hosted them on our own infrastructure as fallback generators. The system dynamically switched to these models whenever external APIs failed.

Result:

System reliability improved significantly, content generation became consistent, and API outages no longer blocked production. What started as a disagreement turned into one of the most impactful architectural improvements in the product. This became my most significant professional achievement because it transformed the system from fragile to production-grade.

How were they hosted like talk more about your architecture

Tell me about a time when you had to make a bold and challenging decision. (26)

Situation:

While working as the sole backend and AI engineer at White Label Resell, I built an AI-powered content automation system that relied heavily on GPT-3.5 for generating headlines, metadata, FAQs, and long-form content. As usage scaled, we started facing frequent failures due to external API downtime, even after multiple retries.

Bold Decision / Vision:

Instead of treating this as a temporary reliability issue, I made the bold decision to reduce our dependency on a single external model. My vision was to design a resilient system that could continue operating even when third-party APIs were unavailable.

Action:

I collected detailed logs showing repeated API failures and aligned them with known outages to clearly demonstrate the risk. I then built internal datasets for each content type, fine-tuned models such as

BERT, FLAN-T5, and Vicuna 13B, and hosted them on our own infrastructure as fallback generators. The system dynamically switched to these models whenever external APIs failed.

Result:

System reliability improved significantly, content generation became consistent, and API outages no longer blocked production. What started as a disagreement turned into one of the most impactful architectural improvements in the product.

Tell me about a time when your vision led to a great impact. (27)

Situation:

At SEOContent.ai, customers wanted direct API access to our AI content generation system instead of using a web interface. This created a major opportunity but also a risk, since exposing complex AI pipelines externally requires strong security, scalability, and billing controls.

Bold Decision / Vision:

I decided to treat this not as a simple wrapper, but as a full commercial API product. My vision was to turn our internal AI system into a scalable, revenue-generating platform that external teams could integrate in minutes.

Action:

In just three weeks, I designed and built a production-ready API using AWS API Gateway and Lambda. I integrated enterprise authentication with AWS Cognito, usage-based billing with Stripe, asynchronous processing for performance, and full monitoring. The API exposed multiple models, including GPT-3.5, BERT, FLAN-T5, and image generation, through clean, well-documented endpoints.

Result:

The API reduced content generation time by about 30 percent, achieved 99.9 percent uptime, handled over 10,000 monthly requests, and enabled multiple client integrations. It secured significant contracts and became a core business offering. This was a bold decision under a tight timeline, and it had lasting business impact.

Invent and Simplify

"Describe a time when you found a simple solution to a complex problem.

(13)"

"Tell me about a time when you tried to simplify a process. What would you have done differently?

(adding alerts push notifications)

Situation:

While working as a Data Scientist at Digital Clinics Research and Services, I joined a project where the medical image processing pipeline was largely manual and poorly documented. Large TIFF and SVS files were being broken down, annotated, and uploaded through ad hoc steps that depended on individual knowledge rather than clear processes. This made the system difficult to understand, risky to modify, and hard for new team members or clinicians to work with confidently.

Task:

My initial responsibility was to support medical image analysis, but I quickly realized that without clear documentation, standardized workflows, and validation checks, the pipeline would not scale or remain reliable in a clinical setting. I needed to improve the system while continuing to deliver on processing and annotation requirements.

Action:

Improved Documentation:

As I redesigned the pipeline, I documented the entire data pipeline from raw TIFF ingestion to PNG format used for training.

Pipeline Refactoring:

I replaced manual, error-prone steps with modular Python scripts that handled file conversion, metadata tracking, and deployment consistently. This reduced duplication and made each stage easier to test and maintain.

Validation and Quality Checks:

I introduced lightweight validation checks to ensure no data loss during file splitting, annotation mapping, and uploads.

Collaboration and Knowledge Sharing:

I worked closely with junior and senior pathologists to ensure the workflows are being used appropriately and trained them on the new annotation and upload process, reducing dependency on developers for routine tasks.

Result:

The system processed over 100,000 images per month with zero data loss, and onboarding new team members became much easier. The improved documentation and structure allowed the pipeline to support multiple downstream AI projects without constant rework.

Tell me about a time when you innovated and exceeded the expectation (14)

"Tell me about a time when you had to look for new ideas.

Tell me about a time when you had to do something new. (15)"

Situation:

While working as an R&D Data Governance Intern at Daiichi Sankyo, I noticed that a critical data quality monitoring workflow relied on Alteryx. This caused high licensing costs, slow execution, and limited flexibility, but replacing it was considered risky because the workflow touched production data.

Task:

Although this was outside my assigned responsibilities, I decided to take a calculated risk by proposing a full migration from Alteryx to Python. The risk was that a failed migration could disrupt data quality monitoring, but the long-term cost and scalability benefits were significant.

Action:

Before making the change, I carefully analyzed the existing workflows, validated data outputs, and built a parallel Python version using SQLAlchemy and Spark. I added logging, error handling, and ran side-by-side comparisons with Alteryx to ensure zero data loss. I completed the migration alongside my regular work within two weeks.

Result:

The migration succeeded with zero data loss, eliminated licensing costs, improved performance through parallel processing, and became the new standard workflow. My manager appreciated the ownership and impact, and I was trusted with an additional data migration project.

Learning:

This experience taught me that calculated risks are about reducing uncertainty through validation and safeguards, not avoiding risk entirely.

Have you ever solved an existing problem in an unconventional way that made life easier?

Tell me about a time when you had to innovate or invent a solution.

(16)" Data Engineering project from alteryx to python

Hire and Develop the Best

Insist On Highest Standards

Tell me about a time when you motivated others to go above and beyond. (23)

"Situation:

During a hackathon at RIT, my team and I were developing a Chrome extension that used AI to summarize terms and conditions, highlighting key clauses. As the submission deadline approached, we tested the extension on Chrome for the first time, only to find that it didn't work as expected. The extension failed to load, and after some investigation, we identified issues with the manifest file and API permissions. This unexpected setback caused frustration and some team members started losing motivation, doubting if we could complete the project in time.

Task:

As the team lead, I needed to keep the team focused and energized while solving the technical issue. My goal was to troubleshoot the problem quickly, inspire confidence in our ability to succeed, and ensure we delivered a working product by the deadline." "I encouraged the team to take a short break to step back and reset, as frustration wasn't helping our progress. After the break, I divided the tasks—while one member focused on debugging the manifest file issues, others tested various configurations and reviewed Chrome's developer documentation. I took the lead in researching specific API permissions we might have missed, and together, we worked systematically to resolve the errors.

To boost morale, I reminded the team that setbacks were a normal part of innovation, and overcoming challenges like this was why we were at the hackathon. I also highlighted how far we'd come in creating a unique solution, keeping the energy positive and collaborative."

"Result:

After a few hours of focused effort, we resolved the manifest file issues and successfully deployed the extension on Chrome. We completed the project just in time, delivering a fully functional extension that impressed the judges. Not only did the extension work seamlessly, but we went on to win the award for ""Best Overall Hack,"" receiving Xbox consoles for each team member. The experience brought our team closer together and validated our hard work.

Reflection:

This situation taught me the importance of maintaining focus and morale during difficult moments. By fostering a collaborative environment and breaking the problem into manageable tasks, we overcame the challenge and delivered a winning solution. Since then, I've applied this approach to other projects, where setbacks became opportunities to demonstrate resilience and leadership."

Negative QUestions

"Tell me about a time when you were misunderstood by a team member or supervisor.

" "Situation:

During a project at TCS, I was leading the migration of an OTT platform from JavaScript to React. My manager asked me to ""prioritize optimization"" for the front-end. While I interpreted this as a need to streamline the architecture for scalability and maintainability, the manager misunderstood my approach and thought I was spending unnecessary time refining the UI for pixel-perfect design, delaying progress.

Task:

The manager believed I was mismanaging time by focusing on non-essential details, which created friction and the perception that I wasn't aligned with the project goals.

" "I scheduled a meeting with the manager to address the misunderstanding directly. I walked through my approach, explaining how optimization at the architectural level was critical for scalability and maintainability, particularly for features like live-streaming components and high-traffic pages. I demonstrated that my focus was on improving the codebase for future flexibility, which would allow faster development of new features and better performance, rather than just UI enhancements. To regain alignment, I adjusted my task reporting to include regular updates on both functional and architectural progress, ensuring the manager could see the tangible results of my efforts." " Once the manager understood my rationale, they appreciated the foresight in my approach. My work not only improved performance but also reduced the time required for future updates and bug fixes. The final product exceeded the client's expectations, and the misunderstanding turned into a valuable learning moment.

This experience reinforced the importance of clear communication and proactive updates when working with team members or supervisors. I've since made it a habit to articulate the reasoning behind my approach and ensure alignment through consistent feedback loops. Misunderstandings are opportunities to improve collaboration and build trust."

Tab 57

Summary

- CUTES framework to assess desired experience
- Give CPR: Context, Problem, Relevance
- Cover why, who, how, so-what with check-ins
- Close with metrics, north stars, and impact



Tab 59

Tab 60

Tab 61

Tab 62

Tab 63

Tab 64

Tab 65

Tab 66

Tab 67

Tab 68

Tab 69

Tab 70

Tab 71

Tab 72

Tab 73

Tab 74

Tab 75

